你缺失嘅嗰門手機課 V1.8

v1.8

ClassTechStar

目录

一、基礎篇（第 0-7 章） **2**

第零章：嗨，開始之前 **2**

一、呢本書可以幫你做乜嘢？（同唔可以做嘅） 2

二、我哋嘅「暗號」說明 2

三、安卓同 iOS，我哋都講講 2

四、手機容量：點解買咗 128G，可用卻唔到 110G？ 2

五、你嘅手機係一座「數碼資產庫」 3

第一章：你好，我嘅私人電腦 **5**

一、手機硬件「五臟六腑」：深度解析 5

二、作業系統：安卓 VS iOS VS 鴻蒙 11

三、手機介面全解析 12

四、手機產品體系與命名規則 13

五、基帶與天線：手機通信嘅幕後功臣 14

六、你嘅手機，到底喺度扮演幾多角色？ 15

附：安卓手機記憶體擴展解析 16

第一章附：鴻蒙作業系統專題 18

第二章：整理你嘅數碼房間——檔案管理 **22**

一、三層存儲體系：你嘅數碼倉庫 22

二、認識「檔案管理器」 22

三、檔案系統結構：深入瞭解 23

四、數據備份與恢復：畀你嘅數據上保險 23

五、檔案安全：保護你嘅私隱 24

六、空間唔夠？嚟場大掃除 24

七、常見檔案格式全解析 24

八、圖片與檔案系統管理：等資產井井有條 26

第三章：等手機接上「生命線」——網絡 **28**

〇、聯網唔迷茫——分清 Wi-Fi 與流量嘅「路」與「車」 28

一、流動網絡（4G/5G）與流量 29

二、Wi-Fi：連接、分享與安全 30

三、藍牙：短距離無線通信嘅萬能選手 31

四、紅外遙控：手機變身萬能遙控器 32

五、NFC：碰一碰嘅魔法 32

六、個人熱點：將手機變 Wi-Fi 路由器 33

第四章：安裝同管理 App——手機嘅「技能包」 **34**

一、官方商店：最安全嘅「應用超市」 34

二、純淨/防護模式：開啟手機嘅「防毒牆」 34

三、第三方來源與「電子寄生蟲」：風險極高！ 35

四、裝、更新、卸載 36

五、權限管理：你係 App 嘅「老闆」 36

六、軟件安裝包嘅構成要素 36

第五章：等手機更懂你——基礎設定 **39**

一、聲音與震動 39

二、顯示與亮度 39

三、桌布、主題與桌面 39

四、效率設定：等手機成為你嘅「外掛大腦」 40

五、輔助功能與無障礙：科技向善嘅力量 41

第六章：守護你嘅數碼生活——安全與私隱 **46**

一、鎖屏：第一道物理防線 46

二、「查找我嘅手機」：終極防盜工具 46

三、系統更新：打補丁，堵漏洞 47

四、識別與防範電信詐騙 47

五、抵禦「開盒」與社工攻擊：斬斷私隱洩露鏈條 48

六、私隱保護：你嘅數據係你嘅財產 49

七、遺失 / 被盜後嘅「急救」清單 50

八、SIM 卡與 eSIM 安全 50

第七章：前沿功能全解析 **52**

一、衛星通信：冇信號都可以求救 52

二、無網通：近距離通信新方式 53

三、車禍檢測：關鍵時刻救命 53
四、星閃技術：下一代無線連接 54

二、軟件篇（第 8-14 章） 57

第八章：通訊：短訊、電話與社交 57
一、電話 App 嘅進階功能 57
二、短訊同 iMessage/RCS 59
三、微信/QQ：國民級社交應用嘅管理 60
四、視像通話對比 61
五、電郵：喺手機上配置同使用 62

第九章：流動辦公與學習 64
一、文檔處理：喺掌中編輯與創作 64
二、筆記應用：由隨手記到知識體系 64
三、掃描與 OCR：將紙質世界搬入手機 65
四、網盤點樣揀：冇最好嘅，淨係有最適合你嘅 65
五、一個真實嘅流動辦公場景 66

第十章：流動支付與生活服務 67
一、到底點樣綁定銀行卡？ 67
二、付款碼係「邊個睇邊個收錢」 67
三、呢啲安全設定，花 5 分鐘設好可以慳好多心 68
四、公交地鐵：手機就係交通卡 68
五、生活繳費、外賣、打車 68
六、用手機搞掂醫保、社保、政務 69

第十一章：攝影與圖片管理 70
一、你係咪都犯過呢啲錯？ 70
二、幾個即刻提升出片率嘅小技巧 70
三、人像模式與專業模式 71
四、後期修圖：唔好過度，啱啱好就得 71
五、相片太多點算？整理策略 72
六、截圖與錄屏 72
第十一章附：專業影像技術知識 73

第十二章：影音娛樂與閱讀 81

一、聽音樂：揀邊個 App，點樣聽先舒服 81
二、播客（Podcast）：通勤路上嘅「耳朵伴侶」 81
三、睇影片：由播放器到串流媒體嘅正確打開方式 82
四、電子書閱讀：手機上都可以好好睇書 82
五、手機遊戲：玩得爽，都唔傷手機 83

第十三章：工具類 App 推薦 85
一、效率工具：管理你嘅時間同注意力 85
二、健康運動 87
三、旅行出行 88
四、自動化工具啟蒙 90
五、瀏覽器同搜尋引擎：你嘅資訊入口 90
六、實用系統工具：清理、防騷擾同安全防護 93
七、天氣預警、自然災害預警同地震預警 95
八、跨裝置協同：遠控電腦、快傳同帳號安全 97

第十四章：密碼管理器專題 101
一、點解你需要密碼管理器？ 101
二、密碼管理器嘅工作原理 101
三、加密原理簡介 101
四、主流密碼管理器推薦 103
五、密碼強度真實評估 103
六、Passkey：無密碼登入嘅未來 105
七、硬件安全密鑰 107
八、家庭/團隊密碼共享 108
九、密碼管理器使用最佳實踐 110

三、進階篇（第 15-20 章） 113

第十五章：電池科學與續航優化 113
一、電池係點回事？循環與健康度 113
二、充電玄學？快充、無線充與黃金法則 113
三、快充協議全解析：點解你嘅充電器充唔快？ 114
四、電池新技術：矽碳負極電池 117
五、呢個先係真正嘅省電技巧 117

第十六章：帳號體系與雲服務 119

一、你嘅手機帳號，到底管住幾多嘢？ 119
二、三個必須做嘅事，而家就做 119
三、iCloud：蘋果用戶嘅「空氣」 120
四、同步同備份，唔好搞混 121
五、安卓用戶嘅帳號矩陣 121
六、家庭共享：等家人都受益 121

第十七章：手機與其他設備嘅聯動 123

一、與電腦協同：手機同電腦唔再係兩個世界 123
二、投屏到電視：全家一齊睇 123
三、與智能手錶/手環：健康管家長喺手腕上 124
四、與智能家居：手機係屋企嘅「總掣」 124
五、藍牙耳機與音箱：無線嘅自由 125
六、與車載系統：揸車時嘅安全拍檔 126
七、聯動中嘅安全同私隱 126

第十八章：數據備份與遷移 128

一、先搞清楚：你手機入面邊啲嘢唔見咗就冇咗 128
二、整機備份：一次搞掂所有嘢 128
三、分類備份策略：重要嘅單獨另存一份 129
四、換機指南：由舊手機到新手機 130
五、備份嘅黃金法則：三二一原則 130

第十九章：故障排查與自救 132

一、手機變卡、發熱——最常被問嘅問題 132
二、應用閃退、無響應——App 唔聽話喇 132
三、充唔入電或充電慢——可能唔係電池嘅鍋 133
四、網絡連唔上、Wi-Fi 成日斷 133
五、螢幕問題：觸控唔靈、閃屏、留印子 134
六、無法開機、卡喺 Logo 介面——先唔好慌 134
七、聲音問題：喇叭唔響、通話聲音細 134
八、搵官方維修，避開嗰啲坑 135
第二十章：開發者模式與高級權限 136
第二十章附：Termux——Android 設備上嘅便攜式 Linux 終端環境 143

四、超越篇（第 21-26 章） 147

第二十一章：移動網絡與資訊安全深探 147

一、VPN：網絡入面嘅“隱形斗篷” 147
二、公共 Wi-Fi 嘅“甜蜜陷阱”——比你想像嘅危險 147
三、HTTPS 係點樣保護你嘅？——SSL/TLS 握手 148
四、DNS 安全：你嘅“網絡電話簿”安全嗎？ 149
五、“無痕瀏覽”真係能隱身嗎？ 149
六、瀏覽器隱私：唔好等瀏覽器將你賣咗 149
七、實用安全清單：保護自己從而家開始 150

第二十二章：AI 點樣喺手機中工作 152

一、AI 相機：你嘅“後期暗房”搬入咗手機 152
二、語音助手：你嘅“聲音祕書”點樣聽明你？ 152
三、端側 AI vs. 雲端 AI 152
四、AI 大模型與手機：你嘅隨身智能助手 152
五、幾個典型應用場景與實操 154
六、提示詞優化：等 AI 聽你指揮嘅“溝通公式” 155
七、AI 嘅侷限與挑戰：保持清醒嘅判斷力 158

第二十三章：編程思維與自動化啟蒙 160

一、自動化到底係個乜？ 160
二、iOS「快捷指令」——蘋果用戶嘅自動化神器 160
三、安卓自動化——Tasker 同佢嘅朋友哋 161
四、自動化思維嘅日常應用 162
五、踩坑指南：自動化嘅常見問題 163

第二十三章附：GitHub 入門指南——由代碼託管到協作平臺 165

一、GitHub 到底係乜？ 165
二、Git 同 GitHub 嘅區別（關鍵概念！） 165
三、GitHub 嘅核心概念與功能 166
四、註冊與上手：五分鐘創建你嘅第一個倉庫 167
五、基本工作流：由本地到雲端 168
六、團隊協作流程：Pull Request 係點樣工作嘅 168
七、GitHub 嘅典型應用場景 169
八、畀新手嘅幾點建議 169
九、總結：點解每個人都應該瞭解 GitHub 170

第二十四章：Root 權限與 BL 鎖深度解析 **172**

一、深入理解 Bootloader 172

二、解鎖 BL 鎖嘅詳細流程（以小米為例） 173

三、Root 嘅多種方案 174

四、刷機入門 174

五、三星 Knox：BL 解鎖嘅不可逆代價 174

六、常見問題解答（Q& A） 175

第二十五章：二手手機處理與數字遺產 **176**

一、舊手機嘅「安樂死」：安全出售或轉贈指南 176

二、數字遺產：你嘅社交靈魂點樣安放？ 177

三、環保嘅最後一程 181

第二十六章：當下手機技術前沿 **183**

一、AI 手機：係營銷噱頭，定係真變革？ 183

二、各廠商 AI 戰略對比：邊個喺度認真做，邊個喺度貼牌？ 183

三、衛星通信：由旗艦「噱頭」到中端標配 184

四、摺疊屏生態成熟度：能當主力機嗎？ 184

五、6G 與 Wi-Fi 7：下一代連接嘅速度與邊界 185

六、矽碳負極電池：續航終於迎來材料級突破 185

七、本章速覽：一句話記住呢啲前沿 186

五、實戰篇（第 27-32 章） **188**

第二十七章：爸媽都能輕鬆用——適老化手機完全指南 **188**

零、先將心態擺正：呢個唔係「老人用唔嚟」，係「手機冇做好」 188

一、第一步：打開「簡易模式」——手機變清爽喇 188

二、如果唔開簡易模式，手動調都得——「大字 + 大圖標」五步走 189

三、防詐騙——呢章請一定帶住長輩一齊睇 190

四、遠程協助——你喺外地都能幫爸媽修手機 191

五、微信嘅「適老改造」——同長輩講呢幾句就夠 191

六、手機唔見咗點算？幫長輩做個「應急預案」 192

七、無線耳機同助聽器相容——被忽視嘅好功能 192

八、健康相關功能——手機唔淨係通訊工具 192

第二十八章：花對錢買到心儀機——手機選購終極指南 **194**

零、三步選機法：需求預算機型 194

一、核心參數點樣睇——唔使全識，睇識呢五項就夠 194
二、按預算分檔推薦（以 2026 年中市場為基準） 195
三、按場景選機——你嘅需求係乜？ 197
四、五個容易踩嘅坑——買之前對照睇一眼 197
五、幾時買最劃算？ 198
六、渠道點樣選——喺邊買靠譜？ 198

第二十九章：數字健康——同手機建立健康關係 200
一、先睇嚇你嘅「螢幕時間」——數字等人清醒 200
二、畀 App 設定「使用限額」——畀自己裝個限速器 200
三、「勿擾模式」嘅正確用法——唔係關掉通知，係關掉焦慮 201
四、護眼唔靠 App 靠習慣——20-20-20 法則 201
五、「低頭族」嘅頸椎自救指南 201
六、睡眠與手機——臥室入面便最該改嘅習慣 202
七、資訊焦慮症——學識「選擇性失明」 202

第三十章：投屏、演示與車載互聯 204
一、無線投屏三大陣營——唔好搞混 204
二、有線投屏——更穩定、零延遲 204
三、手機變電腦——桌面模式 205
四、車載互聯——揸車用手機嘅正確方式 205
五、會議室投屏實戰——唔再手忙腳亂 206

第三十一章：短影片拍攝與內容創作入門 207
一、拍之前先諗清楚：你要拍畀邊個睇？ 207
二、手機拍攝基礎設定——開拍前必做嘅 5 件事 207
三、運鏡五招——等畫面「鬱」起嚟 208
四、剪輯入門——剪映就夠 208
五、封面嘅重要性——70% 嘅人淨係睇封面決定睇唔睇 208
六、內容創作嘅三條心法 209

第三十二章：手機安全急救手冊 210
場景零：地震 / 極端天氣——手機點樣幫你保命 210
場景一：手機唔見咗/俾人偷咗——黃金 30 分鐘 211
場景二：俾人呃咗——72 小時黃金處理期 211
場景三：手機中咗病毒/木馬——點算好？ 212
場景四：帳號被盜——快速奪返控制權 212

場景五：收到勒索訊息——唔使驚，唔好畀錢 213
場景六：AI 換臉/AI 擬聲詐騙——眼見不再為實 213
場景七：SIM swap 攻擊（SIM 劫持）——你嘅號碼俾人「偷」走咗 214

六、展望篇與車聯篇（第 33-34 章） 218

第三十三章：未來展望——手機嘅下一程 218
一、AR/VR 與空間計算：手機之外嘅「第二桌面」 218
二、腦機介面：思想成為交互媒介 218
三、手機嘅「隱身」與新身份基石 219
四、倫理思考：我哋，仲係手機嘅主人嗎？ 219
五、保持連接，亦保持獨立——你嘅數字生活哲學 219

第三十四章：車機互聯深度指南——手機與汽車嘅融合 221
一、車機互聯嘅三大路線 221
二、CarPlay / Android Auto / HiCar 對比 221
三、鴻蒙座艙：手機與車機嘅無縫流轉 222
四、造車新勢力嘅車機系統 222
五、手機當車機——用得嗎？ 223
六、實用技巧 223

兒童與手機專題（第 35 章） 226

第三十五章：兒童與手機——數字時代嘅教育指南 226
一、幾歲畀仔手機？ 226
二、家長控制系統全攻略 227
三、青少年模式 228
四、網絡素養教育 228
五、健康使用習慣 229
六、數字遺產與兒童 229

第三十六章：手機同智能家居——令屋企變聰明 231
一、智能家居協議：揀邊個好重要 231
二、三大生態對比 232
三、手機當家居中樞 232
四、入門套件推薦 233

五、安全注意事項 233

七、AI 編程實戰篇（第 37-47 章） 236

第三十七章：範式革命——由「寫代碼」到「指揮 AI 寫代碼」 236

一、Vibe Coding 嘅誕生：一句話引發嘅運動 236
二、2026 年現狀：Agentic Engineering 時代 236
三、核心認知轉變：由 Coder 到 Conductor 237
四、點解方法論比工具更重要 238

第三十八章：AI 編程五大思維原則 240

一、原則一：Think Before Coding（先思考，再編碼） 240
二、原則二：Simplicity First（簡單至上） 241
三、原則三：Surgical Changes（精準修改） 242
四、原則四：Goal-Driven Execution（目標驅動執行） 242
五、原則五：Iterate and Verify（迭代驗證） 243
六、Harness Engineering：Agent = Model + Harness 244

第三十九章：上下文工程——令 AI 理解你嘅意圖 246

一、項目記憶檔案：AI 嘅「長期記憶」 246
二、規範驅動開發（SDD） 247
三、上下文視窗管理 248
四、上下文工程嘅實踐要點 248

第四十章：需求工程——將想法變成 AI 可執行嘅指令 250

一、由模糊想法到結構化需求 250
二、需求拆解嘅分層藝術 251
三、PRD 寫作模板同實踐 251
四、實戰：由想法到方案嘅完整推演 252

第四十一章：2026 年 AI 編程工具全景 254

一、三大流派同選擇框架 254
二、IDE 內置 AI 流派 255
三、終端 Agent 流派 255
四、雲端 Builder 流派 256
五、工具會變，方法論唔變 256

第四十二章：大模型全景同 API 實戰 258

一、國際主流大模型：三強格局 258
二、國產主流大模型：百花齊放 260
三、模型選擇決策框架 263
四、API 調用入門：由零開始接入大模型 263
五、Token Plan 購買指南 266
六、實戰：喺項目入面接入大模型 API 269

第四十三章：對話工程——同 AI 高效協作嘅對話技術 272
一、Prompt 分層策略 272
二、除錯對話：點樣描述錯誤等 AI 快速定位 273
三、迭代策略：小步快跑 vs 大步重構 273
四、多輪對話嘅上下文管理 274

第四十四章：從 0 到 1 實戰——完整項目全流程 276
一、項目一：極簡個人工具站（入門級） 276
二、項目二：全端記賬系統（進階級） 277
三、項目三：AI 增強應用（高級） 278
四、通用實戰流程清單 279

第四十五章：質素保障——令 AI 寫嘅代碼可靠 281
一、測試驅動：叫 AI 寫測試 281
二、代碼審查技巧 282
三、版本控制：Git 喺 AI 編程入面嘅角色 282
四、常見陷阱同避坑指南 283

第四十六章：進階之路——由用戶到架構師 285
一、MCP 協議同工具擴展 285
二、自訂工作流同 Skills 285
三、多 Agent 協作模式 286
四、持續學習路徑 286

第四十七章：速查手冊同資源 288
一、思維模型速查卡 288
二、工具選擇決策樹 288
三、常見問題 FAQ 289
四、推薦學習資源 289
五、結語：工具喺手，方法論喺心 290

參考文獻 291

八、安卓開發實戰篇（第 48-58 章） 293

第四十八章：認識安卓——由操作系統到開發生態 293

一、安卓嘅身世：由「小機械人」到「全球最大系統」 293

二、用乜嘢語言寫：Java 定 Kotlin？ 294

三、一個 App 係點樣誕生嘅？ 295

四、安卓技術棧全景：你嘅「武器清單」 295

五、安卓 vs iOS：開發者第一堂「揀邊邊」課 296

第四十九章：搭建開發環境——Android Studio 同第一個項目 298

一、安裝 Android Studio 298

二、開你嘅第一個項目 298

三、行起上嚟：模擬器同真機 299

四、工程結構拆解：每個檔案係做乜嘢嘅 300

五、小試牛刀：將「Hello」換成你個名 300

六、常見報錯急救：新手頭兩個禮拜嘅「急症室」 301

第五十章：Kotlin 快速上手——安卓開發嘅第一語言 302

一、變數同類型：畀資料改個名 302

二、函數：將邏輯打包成「掣」 303

三、條件同循環：畀程式識「思考」 303

四、空安全：Kotlin 嘅「保命符」 304

五、類同對象：整「零件」嘅模具 304

六、lambda 同高階函數：Kotlin 嘅「語法糖」 305

七、寫畀零基礎嘅文法速查卡 305

第五十一章：介面開發——Jetpack Compose 入門 307

一、Compose 嘅思想：介面就係函數 307

二、基礎組件：文字、按鈕、輸入框、圖片 308

三、佈局三件套：Column、Row、Box 308

四、狀態：畀介面「活」起上嚟 309

五、主題同深色模式：一次配置，全局生效 310

六、預覽、動畫同實時編輯：Compose 嘅「外掛」 310

第五十二章：四大組件同 Activity 生命週期 312

一、四大組件：安卓嘅「四樑八柱」 312
二、Activity 生命週期：安卓開發嘅第一張「藏寶圖」 312
三、Intent：App 內外通行嘅「車飛」 313
四、多頁面同導航 314
五、順便認識：Fragment 係乜嘢（同埋點解本書冇教） 314

第五十三章：資料儲存同網絡請求 316
一、儲存方案點樣揀：先睇呢張表 316
二、DataStore：存啲小配置 316
三、Room：畀資料安個「真正嘅屋企」 317
四、Retrofit：同伺服器講嘢 318
五、JSON 解析：畀網絡資料「翻譯」 318
六、協程：安卓並發嘅「普通話」 319
八、協程：安卓並發嘅「普通話」 319

第五十四章：項目一——我嘅記賬本（本地版） 321
一、需求思考：先問五條問題 321
二、資料層：畀「賬」有個屋企 321
三、MVVM：介面同邏輯「分家」 322
四、介面：列表頁 + 錄入頁 323
五、編輯同刪除：畀資料「入得到又出得到」 324
六、驗收：三層檢查 + 兩個進階挑戰 325
八、驗收：三層檢查 + 兩個進階挑戰 325

第五十五章：項目二——天氣查詢 App（網絡版） 326
一、申請免費天氣 API 326
二、網絡層：Retrofit + JSON 解析 326
三、三種介面狀態：載入中 / 成功 / 失敗 327
四、實戰介面程式碼：將天氣畫出嚟 328
五、介面同互動細節 328
七、介面同互動細節 328

第五十六章：打磨同發布——測試、除錯同上架 Google Play 330
一、除錯三板斧：Logcat、斷點、UI 檢查器 330
二、測試：畀代碼買保險 330
三、簽名打包：畀 App 蓋上「印章」 331
四、上架 Google Play 全流程 332

五、商店素材點樣做：第一印象就係下載量 332
五、性能優化同崩潰上報小清單 333

第五十七章：進階之路——權限、通知同現代架構 334
一、運行時權限：先敲門，再入門 334
二、通知：喺啱嘅時間講有用嘅嘢 335
三、架構進階：依賴注入同模組化 335
四、2026 年嘅新玩法：AI 輔助安卓開發 336
五、Compose 性能優化同除錯工具 336

第五十八章：速查手冊同資源 338
一、常用代碼速查卡 338
二、常見問題 FAQ 338
三、推薦學習資源 339
四、Gradle 常用命令速查 339
五、結語：由第一個 App 到第 N 個 App 340
七、結語：由第一個 App 到第 N 個 App 340
參考文獻 340

九、iOS 開發實戰篇（第 59-69 章） 343

第五十九章：認識 iOS——蘋果生態同開發生態 343
一、iOS 嘅身世：一部手機改變一個行業 343
二、圍牆花園：蘋果生態嘅「規矩」同「紅利」 343
三、用乜嘢語言寫：Objective-C 定係 Swift？ 344
四、一個 App 係點樣誕生嘅？ 344
五、iOS 技術棧全景：你嘅「武器清單」 345

第六十章：搭建開發環境——Xcode 同第一個項目 347
一、準備一部 Mac 347
二、安裝 Xcode 347
三、建立你嘅第一個項目 348
四、工程結構拆解：每個檔案係做乜嘢嘅 348
五、真機除錯：將 App 裝入你嘅 iPhone 349
五、Xcode 效率技巧：新手嘅五個「快捷鍵外掛」 349
六、真機除錯：將 App 裝入你嘅 iPhone 349
七、小試牛刀 + 常見報錯急救 350

九、小試牛刀 + 常見報錯急救 350

第六十一章：Swift 快速上手——蘋果開發嘅第一語言 351

一、變數同類型：畀數據改個名 351
二、可選值（Optional）：Swift 嘅“防崩護身符” 352
三、函數同閉包：將邏輯打包成“按鈕” 352
四、結構體同協議：Swift 嘅“樂高”體系 353
五、寫畀零基礎嘅語法速查卡 354

第六十二章：界面開發——SwiftUI 入門 355

一、SwiftUI 嘅思想：聲明式，唔係命令式 355
二、基礎視圖：文字、按鈕、輸入框、圖片 356
三、佈局三件套：VStack、HStack、ZStack 356
四、狀態同綁定：令界面“活”起嚟 357
五、預覽、主題同動畫：SwiftUI 嘅“外掛” 357
六、常用修飾符速查：一頁在手，界面唔使愁 358

第六十三章：視圖、狀態同導航 360

一、狀態管理嘅“數據流全景” 360
二、NavigationStack：頁面跳轉嘅“棧” 361
三、彈窗同底部標籤：sheet 同 TabView 361
四、List：iOS 嘅“國民列表” 362
五、程式生命週期：前台同後台 363

第六十四章：數據存儲同網絡請求 365

一、存儲方案點樣揀：先睇呢張表 365
二、UserDefaults：存啲小配置 365
三、SwiftData：畀數據安個“真正嘅屋企” 366
四、Codable：畀 JSON“翻譯” 367
五、URLSession + async/await：同佢服器講嘢 367
六、網絡錯誤都要“分級”：唔好將所有報錯都掟畀用戶 368

第六十五章：項目一——我嘅記賬本（本地版） 370

一、需求思考：先問五個問題 370
二、數據層：畀“賬”有個屋企 370
三、MVVM：界面同邏輯“分家” 371
四、界面：列表頁 + 錄入彈窗 372

五、編輯、刪除同匯總 373
六、驗收：三層檢查 + 兩個進階挑戰 373

第六十六章：項目二——天氣查詢 App（網絡版） 375
一、申請免費天氣 API 375
二、網絡層：URLSession + Codable 375
三、三種界面狀態：加載中 / 成功 / 失敗 376
四、實戰界面代碼：將天氣畫出嚟 377
五、交互細節同收藏城市 377
六、網絡請求點樣調試：打印、抓包同模擬慢網 378

第六十七章：打磨同發佈——測試、調試同上架 App Store 379
一、調試三板斧：print、斷點、預覽 379
二、測試：畀代碼上保險 379
三、簽名同打包：畀 App 蓋上“印章” 380
四、上架 App Store 全流程 381
五、商店素材點樣做：第一印象就係下載量 381
六、崩潰上報同性能優化小清單 381

第六十八章：進階之路——權限、通知同現代架構 383
一、權限：先敲門，再入門 383
二、通知：喺啱嘅時間講有用嘅嘢 384
三、架構進階：MVVM 深入同可測試性 384
四、2026 年嘅新玩法：AI 輔助 iOS 開發 385
五、Swift 並發要點：@MainActor 同 Task 385

第六十九章：速查手冊同資源 387
一、常用代碼速查卡 387
二、常見問題 FAQ 387
三、Xcode 高頻快捷鍵速查 388
四、推薦學習資源 389
六、推薦學習資源 389
五、結語：由第一個 App 到第 N 個 App 389
參考文獻 389

附錄 A：新手機開箱「保姆級」流程 392
開箱驗機「三部曲」： 392

激活與初始設定「重點提醒」： 392
裝 App「安全第一」： 392

附錄 B：動手練習小參考 **393**

附錄 C：極簡術語「黑話」對照表 **394**

附錄 D：靠譜資源去邊度搵？ **395**
官方支持係首選： 395
實用網站同博主： 395
安全自查一句話清單： 395

附錄 E：多謝與說明 **396**
6 版更新要點 396
7 版更新要點 396
8 版更新要點 398
內容增補：實用軟件與騷擾攔截 398
內容增補：天氣預警 / 自然災害預警 / 地震預警 398
內容增補：跨設備協同與效率工具 399

附錄 F：折疊螢幕選購與使用專題 **400**
一、折疊螢幕而家發展到乜嘢程度？ 400
二、乜嘢人適合買折疊螢幕？ 400
三、選購要點：睇呢 6 個指標 401
四、折疊螢幕獨有嘅使用技巧 402
五、保養要點：等折疊螢幕活得耐啲 402
六、折疊螢幕嘅「坑」同心理預期 403
七、主流品牌點樣揀（2026 年） 404

結語：保持好奇，保持清醒 **405**

附錄 G：全書速查索引 **406**

附錄 H：版本路線圖 **410**

一、基礎篇（第 0-7 章）

第零章：嗨，開始之前

喺正式探索前，我哋先傾啲基本規則，確保後面講嘅你都識。唔好急，磨刀不誤砍柴工嘛。

一、呢本書可以幫你做乜嘢？（同唔可以做嘅）

佢可以幫你係統性咁搞懂手機點樣用，明白功能背後嘅道理，而且無論你係安卓黨定蘋果派，大多數思路係通用嘅。佢會帶住你動手練習，等你由「識用」升級到「懂用」。

佢唔係某一款特定手機嘅說明書，亦唔係追新品嘅科技新聞，更唔係教你沉迷手機嘅指南。講真，如果你期待嘅係「點樣用手機刷影片更爽」，咁可能要失望啦——我哋更關心嘅係點樣等手機真正為你服務，而唔係反過嚟。

二、我哋嘅「暗號」說明

為咗講得更清楚，書裏會用一啲小標記，呢啲標記就似我哋之間嘅默契暗號：

- **[按鈕名]**：代表螢幕上具體嘅按鈕或 App 名，比如 [設定]、[確定]。
- **>**：代表操作順序，比如「撳 [設定] > [聲音]」，就係先撳設定，再撳聲音。
- **加粗**：非常重要！可能係關鍵概念或安全警告，請務必留意。
- *斜體*：一啲補充嘅小想法或比喻，幫你更好地理解。

三、安卓同 iOS，我哋都講講

你估點樣，有啲嘢叫法唔同，但做嘅事一樣。比如：應用（安卓）同 App（iOS），設定（都叫設定），檔案管理（安卓）同檔案（iOS）。知道係一回事就得，唔好俾名詞繞暈咗。

另外，而家安卓陣營裏仲有華為嘅鴻蒙系統，佢有自己嘅特色，後面我哋會專門講到。總之，唔理你用乜嘢手機，呢本書都幫到你。

四、手機容量：點解買咗 128G，可用卻唔到 110G？

容量單位大家知道：1TB = 1024GB，1GB = 1024MB，1MB = 1024KB……但呢度有個坑：硬碟廠商通常按 1000 進制算，而系統按 1024 進制算，所以你買到嘅「128G」硬碟，實際

容量會「縮水」。

但你手機容量縮水嘅原因主要有三個：

1. **系統自己霸咗地方**：作業系統就似房子嘅承重牆，必須存在。而家嘅系統動輒佔用 10-20GB，呢個好正常。
2. **廠商預裝嘅應用**：新手機裏自帶嘅嗰啲 App 都佔空間，有啲可以卸載，有啲唔可以。
3. **分區同格式化損耗**：技術原因，就似書包嘅襯布都佔體積。

買手機嘅小建議

而家 128GB 嘅手機基本唔夠用啦，256GB 起步比較穩陣。如果你鍾意影相、玩遊戲，建議直接上 512GB 甚至更大。記住，存儲空間呢啲嘢，寧可富餘，唔可捉襟見肘。

嚟動手：睇下你嘅手機「身份證」

打開 [設定]，搵到關於手機（或類似選項），記低：手機型號、系統版本、總存儲空間同剩餘空間。呢個係瞭解佢嘅第一步。順便睇下電池健康度，心裏有個數。

五、你嘅手機係一座「數碼資產庫」

喺開始具體知識之前，我哋先建立一個核心認知：**手機早已唔只係通信工具，佢係我哋最重要嘅數碼資產庫**。聯繫人、聊天記錄、家庭相、工作文檔、銀行 App、付款碼、健康數據、電子證件……呢啲數據加埋一齊，價值往往超過手機本身。

點解呢個好重要？

如果手機唔見咗或壞咗，你損失嘅唔只係幾千蚊硬件，**更可能係多年積累嘅不可再生數據**——小朋友嘅成長相、過世親人嘅語音、冇備份嘅重要合同。科學咁整理同防護習慣，可以從三個維度保護你：

- **提升運行效率**：規範嘅檔案分類等系統查找更快，手機長期唔卡頓。
- **延長使用壽命**：合理嘅存儲管理減少晶片讀寫負擔，電池同存儲都更耐用。
- **保障數據安全**：分層管理 + 定期備份 + 權限收斂，等「數據災難」遠離你。

我哋嘅核心理念：「預防為主，防治結合」

就似養生唔可以等到得病先睇醫生，**手機管理都要由「日常習慣」抓起**。本書反覆強調嘅唔係某個高級技巧，而係呢啲樸素嘅原則：

- **預防為主**：裝 App 淨係行官方商店、發相之前先脫敏、密碼唔重複——呢啲「小動作」成本極低，但可以擋住絕大多數風險。
- **定期體檢**：每月一次整理下載檔案夾、每季度一次權限審計、每半年一次全量備份——固定節奏比臨時救火有效一百倍。

- **留有退路：**開啟雲同步、記好帳號密碼、寫一份「數碼遺產清單」——確保意外發生時你同屋企人都可以從容應對。

第一章：你好，我嘅私人電腦

你嘅手機，其實係你口袋裏最私人嘅超級電腦。佢比當年將人類送上月球嘅電腦強大無數倍，卻安靜咁躺喺你嘅掌心。今日，我哋就嚟好好認識呢位「沉默嘅巨人」。

一、手機硬件「五臟六腑」：深度解析

手機雖小，五臟俱全。等我哋好似解剖課咁樣，逐個認識呢啲精密嘅組件。

SoC（系統級晶片）：手機嘅「大腦 + 心臟」

SoC 全稱 System on Chip，翻譯過嚟就係「將整個系統塞入一塊晶片」。佢唔只係 CPU（中央處理器），而係將 CPU、GPU（圖形處理器）、NPU（神經網絡處理器）、ISP（圖像信號處理器）、基帶（負責通信）等等全部集成埋一齊。

呢個就似將一個城市嘅政府大樓、發電廠、通信中心、警察局全部建喺一棟樓裏，效率極高。

CPU：通用計算嘅核心

CPU 係 SoC 中最核心嘅組件，負責處理絕大多數計算任務。手機 CPU 採用 ARM 架構，同電腦嘅 x86 架構唔同，ARM 架構以精簡指令集（RISC）為特色，功耗更低，更適合流動設備。

CPU 大小核設計：現代手機 CPU 普遍採用「大小核」架構（ARM 嘅 big.LITTLE 技術），將核心分為唔同性能等級：

- **超大核/大核**：性能最強，負責重度任務。如遊戲加載、應用啟動、複雜計算。頻率最高（3GHz+），但功耗都最大。
- **中核**：性能同功耗平衡，處理日常中等負載任務。刷微博、睇影片、多任務切換主要靠佢。
- **小核/能效核**：功耗最低，處理輕量級後台任務。待機、聽音樂、接收消息等，幾乎唔費電。

三個等級嘅核心協同工作：系統調度器根據任務負載自動分配——輕量任務交畀小核慳電，重活交畀大核猛做，做完再切返小核。呢個就係點解手機既識流暢打遊戲，又識待機成日。

各品牌架構特點：

- **高通驍龍 8 Elite：**採用 Oryon 自研 CPU 架構（源自 Nuvia 收購），2 個超級核（4.32GHz）+ 6 個性能核（3.53GHz），全部基於台積電第二代 3nm 製程。徹底告別 ARM 公版架構，單核同多核性能均實現跨越式提升。GPU 為 Adreno 830，採用切片架構，圖形性能提升 40%，Hexagon NPU 算力約 45 TOPS，AI 推理性能提升 45%。（註：三星 Galaxy 定製版頻率更高，超級核可達 4.47GHz。）
- **聯發科天璣 9400：**1 個超大核（Cortex-X925，3.62GHz）+ 3 個性能核（Cortex-X4，3.3GHz）+ 4 個能效核（Cortex-A720，2.4GHz），全大核架構延續，台積電第二代 3nm 製程。GPU 為 Immortalis-G925（12 核），峯值性能提升 41%，光線追蹤性能提升 40%。AI 處理器為第八代 NPU 890，支援端側大模型運行、端側 LoRA 訓練同端側影片生成。
- **蘋果 A18 Pro：**2 個性能核（約 4.04GHz）+ 4 個能效核（約 2.4GHz），台積電第二代 3nm 製程。6 核 GPU 支援硬件級光線追蹤，16 核 NPU（約 35 TOPS）為端側 AI 同 Apple Intelligence 提供強大算力。蘋果自研架構喺單核性能同能效比上依然領先，統一記憶體架構為 AI 推理提供高帶寬優勢。
- **華為麒麟 9020：**搭載於 Mate 70 系列，採用自研泰山 CPU 架構，NPU 同 ISP 能力持續增強，支援 5.5G 同衛星通信，代表國產晶片嘅最新突破。

GPU：圖形渲染引擎

GPU（Graphics Processing Unit）專門負責圖形同並行計算任務。佢嘅核心數量遠多過 CPU（動輒數百上千個核心），但每個核心嘅功能相對簡單，適合同時處理大量相似嘅計算。

圖形渲染流程：

1. **頂點處理：**計算 3D 模型中每個頂點嘅位置坐標。
2. **光柵化：**將 3D 圖形轉換為 2D 螢幕上嘅像素點。
3. **像素著色：**為每個像素計算顏色、紋理、光照效果。
4. **輸出合成：**將所有圖層合成最終畫面，送到螢幕顯示。

GPU 對日常使用嘅影響：唔只係遊戲！界面滑動、動畫效果、影片播放、相濾鏡、AR 應用都依賴 GPU。GPU 性能弱嘅手機，滑動界面可能掉幀，遊戲更係直接卡頓。

各品牌 GPU：高通使用自研 Adreno GPU；聯發科使用 ARM Mali GPU；蘋果使用自研 GPU，性能同能效都係行業標杆。

NPU：AI 加速引擎

NPU（Neural Processing Unit）係專門為神經網絡運算設計嘅處理器。相比 CPU 同 GPU，NPU 喺處理 AI 任務時效率更高、功耗更低。

神經網絡運算方式：NPU 嘅核心運算係矩陣乘法同卷積運算——呢個正係神經網絡中最基礎、最頻繁嘅計算。NPU 通過硬件級優化，可以並行處理大量矩陣運算，速度比 CPU 快幾十倍，功耗卻低得多。

AI 處理流程：

1. **數據輸入：**攝像頭圖像、麥克風音頻、傳感器數據等。
2. **預處理：**調整數據格式同尺寸，適配神經網絡輸入要求。
3. **推理計算：**NPU 加載訓練好嘅模型，對輸入數據進行前向計算，輸出結果。
4. **後處理：**將 AI 輸出轉換為可用嘅結果，如人臉坐標、文字內容、場景標籤。

實際應用場景：

- **計算攝影：**夜景合成、人像虛化、HDR 處理、AI 美顏。
- **語音助手：**語音識別、自然語言理解、語音合成。
- **實時翻譯：**AI 字幕、對話翻譯。
- **智能識別：**文字識別（OCR）、物體識別、場景識別。
- **端側大模型：**最新旗艦晶片已支援喺手機端運行小型 AI 大模型（7B 參數級別），實現離線 AI 對話、文檔摘要、智能寫作等功能。驍龍 8 Elite 嘅 Hexagon NPU 算力約 45 TOPS，天璣 9400 嘅 NPU 890 同樣支援端側大模型（LLM 推理速度提升 80%），蘋果 A18 Pro 嘅 16 核 NPU 提供約 35 TOPS 算力。
- **AI 圖像處理：**AI 消除（一鍵去除相中嘅路人）、AI 擴圖（擴展圖片邊界）、AI 風格轉換、文字生成圖片等。

ISP：影像信號處理器

ISP 處理攝像頭傳嚟嘅原始數據，將光線信號變成好睇嘅相。

性能點樣睇？睇型號！蘋果 A18 Pro、高通驍龍 8 Elite、聯發科天璣 9400、華為麒麟 9020 都係旗艦級別。中端睇驍龍 8s Gen3/7+ Gen3、天璣 8400。唔好淨係睇核心數，架構同製程（納米數）更重要。3nm 製程係目前最先進嘅，能效比遠優於 5nm 同 4nm。

記憶體（RAM）：臨時工作台

記憶體就似你嘅辦公桌，越大，可以同時攤開嘅檔案就越多。12GB 係而家嘅主流，16GB 適合重度用戶，24GB 以上就係遊戲玩家同極客嘅選擇。

記憶體類型都好重要：LPDDR5 比 LPDDR4X 更快更慳電。蘋果雖然記憶體唔大，但系統優化好，所以睇起嚟「夠用」。

記憶體唔夠會點樣？App 會被系統殺咗後台，切換時需要重新加載。呢個就係點解有時切個微信返嚟，遊戲就冇咗。

存儲（ROM/閃存）：長期倉庫

存儲就似你嘅書櫃，裝所有相、影片、App 嘅地方。而家主流係 UFS 3.1 或 UFS 4.0，速度比老式嘅 eMMC 快好幾倍。

點解存儲速度重要？佢決定咗 App 打開速度、遊戲加載速度、相保存速度。用耐咗之後，慢速存儲會等手機越嚟越卡。

螢幕：你嘅眼睛

螢幕係你同手機交互嘅主要介面，好壞直接影響體驗。

- **材質**：OLED 色彩艷麗、對比度高、更慳電（顯示黑色時像素唔發光），但低亮度下可能有頻閃傷眼；LCD 更護眼、成本低，但對比度同色彩唔如 OLED。
- **分辨率**：1080P 係主流，2K 更清晰但更耗電。蘋果嘅「視網膜屏」概念就係：喺一定距離下，人眼分辨唔出像素點。
- **刷新率**：60Hz 係標準，90Hz/120Hz 更流暢，滑動時絲般順滑。高刷新率會耗電，所以好多手機支援自適應刷新率（LTPO），根據內容動態調整。
- **觸控採樣率**：影響觸控響應速度，遊戲玩家要關注，越高越跟手。
- **調光方式**：DC 調光比 PWM 調光更護眼，低亮度下唔容易眼攰。而家好多手機支援高頻 PWM 調光，都係為咗護眼。

螢幕 HDR 技術

HDR（High Dynamic Range）等螢幕同時展現更亮嘅亮部同更暗嘅暗部，畫面層次感大幅提升。

- **實現原理**：普通螢幕嘅亮度範圍有限（SDR），最亮同最暗之間嘅差距唔夠大。HDR 螢幕嘅峯值亮度更高（通常 1000 尼特以上），對比度更大，色域更廣，能顯示更多細節。
- **主流標準**：HDR10（最基礎，免費開源）、HDR10+（三星推動，動態元數據）、Dolby Vision（杜比視界，畫質最佳但需授權）。手機支援邊個標準取決於螢幕硬件同系統授權。
- **實際效果**：睇 HDR 影片時，陽光更刺眼、陰影更深邃、色彩更豐富。前提係影片本身係 HDR 製作嘅，螢幕都支援 HDR 顯示。

影像 HDR 技術

影相時嘅 HDR 同螢幕 HDR 係兩回事。影像 HDR 係通過算法解決「亮處過曝、暗處死黑」嘅問題。

- **算法原理**：手機喺撳低快門時，實際拍攝咗多張唔同曝光嘅相（欠曝保留亮部細節、過曝保留暗部細節、正常曝光保留中間調），然後將佢哋合成為一張高動態範圍嘅相。

- **處理流程**：多幀捕獲對齊補償（防手震）曝光融合/色調映射色彩校正輸出最終相。
- **效果優化**：而家手機已經做到「實時 HDR」——取景器裏睇到嘅畫面就係 HDR 處理後嘅效果，所見即所得。蘋果嘅 Smart HDR 同安卓嘅自動 HDR 都非常成熟。

電池同充電：能量之源

電池係手機最「短命」嘅部件，用個兩三年就會明顯老化。而家主流容量係 5500-7500mAh。

- **充電功率**：安卓陣營已經捲到咗 100W 甚至 200W，十幾分鐘充滿；蘋果仲喺 20-30W 徘徊。快充傷電池嗎？用原裝配件問題唔大，手機有温控保護。
- **無線充電**：方便但效率低、發熱大。適合辦公室隨手充，唔適合急住出門時用。
- **反向充電**：手機變成充電寶，畀耳機、手錶充電，應急用。

攝像頭：光同影嘅魔法

而家手機攝像頭動輒三四個，各有分工：

- **主攝**：最常用嘅，傳感器最大，畫質最好。
- **超廣角**：影風景、建築，視野更闊。
- **長焦**：影遠處，有光學變焦同數碼變焦之分。
- **微距**：影極近距離嘅小嘢。

像素數唔係全部！傳感器尺寸（底大一級壓死人）、光圈大小（f/1.8 比 f/2.4 大）、防震（OIS 光學防震好重要）都影響畫質。計算攝影（AI 算法）而家越嚟越重要，可以將多張相合成一張好相。

潛望式長焦鏡頭

傳統長焦鏡頭需要較長嘅光學路徑，會等手機變得好厚。潛望式長焦通過一個 90 度嘅稜鏡將光線折射，等光路喺手機內部橫向展開，從而喺保持手機輕薄嘅同時實現 5 倍甚至 10 倍光學變焦。

- **結構原理**：光線由鏡頭進入稜鏡折射 90 度沿手機寬度方向穿過鏡片組到達傳感器。相當於將一個豎住嘅望遠鏡橫住放。
- **光學特性**：光學變焦倍數高（5x-10x），畫質遠超數碼變焦。但傳感器尺寸通常比主攝細，暗光表現一般。
- **成像優勢**：影月亮、影遠處建築、影舞台特寫，係普通手機無法企及嘅。

OIS 光學防震 vs EIS 電子防震

防震係影響影相同錄像畫質嘅關鍵技術，兩種方式原理完全唔同：

- **OIS（光學防震）**：通過物理移動鏡頭或傳感器嚟抵消手震。鏡頭內部有微型陀螺儀檢測震動方向同幅度，然後驅動馬達反向移動鏡片組。優點係唔損失畫質，缺點係成本高、佔用空間。
- **EIS（電子防震）**：通過軟件算法裁切同偏移畫面嚟抵消震動。手機錄製稍大嘅畫面範圍，然後根據陀螺儀數據裁切出穩定嘅區域。優點係唔需要額外硬件，缺點係會裁切畫面（視野變細），暗光下可能模糊。
- **實際選擇：**影相優先睇 OIS，錄像兩者結合效果最好。而家好多旗艦手機支援「Sensor-Shift」傳感器位移防震，比傳統鏡頭防震效果更好。

光學變焦 vs 數碼變焦

- **光學變焦**：通過物理移動鏡片組改變焦距，畫質唔損失。手機上嘅光學變焦通常淨係有固定倍數（如 2x、3x、5x、10x），唔可以好似相機咁樣連續變焦。
- **數碼變焦**：放大畫面中心區域，本質係裁切 + 插值放大。放大越多，畫質越差。100 倍數碼變焦睇住好型，但實際畫質已經好糊咗。
- **混合變焦**：結合光學同數碼，喺光學變焦倍數之間用多幀合成算法彌補畫質損失。效果比純數碼好，但仍不如光學。

音頻：揚聲器同獨顯晶片

單揚聲器 vs 雙揚聲器：

- **單揚聲器**：淨係底部一個發聲單元，聲音偏向一側，冇立體感。中低端手機常見。
- **雙揚聲器（立體聲）**：頂部同底部各一個揚聲器，形成立體聲場。睇影片、聽音樂時左右聲道分明，沉浸感強好多。聽筒兼做頂部揚聲器係常見方案。
- **音質差異**：雙揚聲器唔只係「聲音更大」，關鍵係空間感同層次感。用單揚聲器睇大片，總覺得少咗啲乜。

遊戲手機獨顯晶片：

部分遊戲手機同性能手機配備咗獨立嘅顯示處理晶片，佢嘅作用係：

- **插幀**：將 30fps 嘅遊戲畫面通過算法插值到 60fps 或 120fps，畫面更流暢。呢個過程喺獨顯晶片上完成，唔佔用 SoC 嘅 GPU 資源。
- **超分**：將低分辨率畫面通過 AI 算法提升到高分辨率，畫質更清晰。
- **降功耗**：因為插幀同超分由獨顯晶片負責，SoC 嘅負載降低，發熱同功耗都減少，遊戲更穩定。

GPS 定位：單頻同雙頻

手機定位主要依賴 GPS 等衛星導航系統。

- **單頻 GPS**：淨係接收 L1 頻段信號，定位精度約 3-5 米。喺開闊地帶夠用，但喺高樓密集嘅城市峽谷中，信號反射會導致定位偏移較大。
- **雙頻 GPS**：同時接收 L1 同 L5 兩個頻段信號。L5 頻段波長更長、抗幹擾能力更強，雙頻聯合解算可以消除大部分電離層誤差，定位精度可達 1-3 米。喺城市中導航、打車定位時優勢明顯。
- **多星座支援**：現代手機唔只支援 GPS，仲支援北斗（中國）、GLONASS（俄羅斯）、Galileo（歐洲）等多個衛星系統。同時連接嘅衛星越多，定位越快越準。

傳感器：手機嘅小觸角

手機裏收埋十幾個傳感器，默默工作：

- **加速度計 + 陀螺儀**：感知手機姿態，玩遊戲轉向、自動旋轉螢幕。
- **光線傳感器**：自動調節螢幕亮度。
- **距離傳感器**：打電話時手機貼近耳朵自動熄屏。
- **指紋傳感器**：而家主流係屏下光學或超聲波指紋。
- **面部識別**：蘋果嘅 Face ID 係 3D 結構光，安全級別高；好多安卓係 2D 相識別，安全性一般。
- **NFC**：近場通信，刷公交卡、門禁卡、銀行卡。
- **氣壓計**：測海拔高度，爬樓梯都識檢測到。
- **霍爾傳感器**：檢測磁吸殼嘅開合。

二、作業系統：安卓 VS iOS VS 鴻蒙

硬件係身體，作業系統係靈魂。三大系統各有特色：

- **安卓**：似開源社區，手機廠商可以深度定製，選擇多。你可以換主題、裝第三方應用商店、折騰各種玩法。但都因為開放，碎片化嚴重，唔同品牌體驗差異大。
- **iOS**：似精心打理嘅花園，蘋果軟硬件一手抓，體驗統一流暢。應用質量高，私隱保護好，但自由度低，好多事要按蘋果嘅規矩嚟。
- **鴻蒙**：華為嘅新物種，主打分佈式能力，可以等唔同設備無縫協同。後面我哋會專門講。

三者生態都極其豐富，揀你鍾意嘅生活方式就好。冇絕對嘅好壞，淨係有適唔適合。

三、手機介面全解析

手機介面係連接外部世界嘅橋樑，唔同介面差異好大：

Micro USB

- **物理特性**：扁平梯形介面，分正反面（插反咗插唔入去）。已經喺手機上基本淘汰，但部分低端設備同老設備仲喺度用。
- **傳輸限制**：淨係支援 USB 2.0 標準，最高傳輸速率 480Mbps，充電功率通常唔超過 10W。
- **缺點**：正反面唔通用、插拔壽命短、傳輸速度慢、充電功率低。

Lightning（蘋果閃電介面，已淘汰）

- **物理特性**：蘋果自研嘅 8 針介面，正反都可以插。iPhone 14 及更早機型使用，現已全面淘汰。
- **傳輸限制**：Lightning 介面物理上淨係支援 USB 2.0 標準，最高傳輸速率 480Mbps。即使用 USB-C 轉 Lightning 線，傳輸速度都受限於 Lightning 介面本身。
- **淘汰原因**：歐盟通過法規要求 2024 年起所有手機統一使用 USB-C 介面，蘋果由 iPhone 15 系列開始全面改用 Type-C。Lightning 介面正式退出歷史舞台。
- **注意**：蘋果 MFi 認證嘅數據線內含認證晶片，非認證線可能無法正常充電或傳輸數據。如果你仲喺度用舊款 iPhone，建議備好 Lightning 數據線。

USB Type-C

- **物理特性**：24 針腳嘅橢圓形介面，正反都可以插。已成為手機同多數電子設備嘅標準介面。歐盟已立法要求 2024 年起所有手機、平板、耳機等便攜設備統一使用 USB-C 介面，全球其他地區都喺度跟進。
- **傳輸協議**：Type-C 淨係介面形態，實際傳輸能力取決於內部協議。支援 USB 2.0（480Mbps）、USB 3.0/3.1/3.2（5-20Gbps）甚至 USB4（40Gbps）。
- **重要區分**：並非所有 Type-C 介面都支援 USB 3.0！好多手機雖然用 Type-C 介面，但內部淨係連接咗 USB 2.0 嘅引腳，傳輸速度同老介面一樣慢。iPhone 16/17 標準版淨係支援 USB 2.0 速度，Pro 版先支援 USB 3.0（需使用 USB-C to USB-C 線）。
- **充電能力**：支援 USB PD 等快充協議，最高可支援 240W 充電。
- **影片輸出**：支援 DisplayPort Alt Mode，可以通過轉接線輸出影片到顯示器。

5mm 音頻介面

- **物理特性**：圓形 3 段或 4 段插孔，3 段傳輸左右聲道音頻，4 段多一個麥克風通道。
- **特點**：模擬信號傳輸，即插即用，唔需要驅動或轉換。音質穩定，延遲為零。
- **現狀**：大部分旗艦手機已取消 3.5mm 介面，需要 Type-C 轉接線或使用藍牙耳機。部分中端機同遊戲手機仲保留。

四、手機產品體系與命名規則

面對各品牌眼花繚亂嘅產品線，搞清楚命名規則可以幫你快速定位適合自己嘅機型。

主流品牌產品系列：

- **蘋果**：iPhone（標準版）/ iPhone Plus（大螢幕版）/ iPhone Pro（專業版）/ iPhone Pro Max（大螢幕專業版）/ iPhone SE（平價版）/ iPhone 16e（平價 AI 版，搭載 A18 晶片同蘋果自研 C1 基帶）。2026 年 iPhone 17 系列全面搭載 A19 Pro 晶片，AI 能力進一步提升。
- **華為**：Mate 系列（商務旗艦）/ P/Pura 系列（影像旗艦）/ Mate X 系列（摺疊旗艦）/ nova 系列（年輕中端）/ 暢享系列（入門）
- **小米**：小米數字系列（旗艦）/ 小米 MIX（探索旗艦/摺疊）/ Redmi K 系列（性能旗艦）/ Redmi Note 系列（中端）/ Redmi 數字系列（入門）
- **OPPO**：Find X 系列（旗艦）/ Reno 系列（中端）/ A 系列（入門）/ 一加數字系列（性能旗艦）
- **vivo**：X 系列（影像旗艦）/ iQOO 數字系列（性能旗艦）/ S 系列（自拍中端）/ Y 系列（入門）
- **三星**：Galaxy S 系列（旗艦）/ Galaxy Z Fold/Flip（摺疊旗艦）/ Galaxy A 系列（中端）

常見後綴含義：

- **Pro**：Professional 嘅縮寫，專業版。通常喺標準版基礎上升級鏡頭、螢幕、處理器等核心配置。
- **Max**：最大版。螢幕最大、電池最大，但配置唔一定比 Pro 高。
- **Ultra**：極致版。通常代表該系列嘅最高配置，堆料最猛，價格都最高。如三星 S24 Ultra、小米 14 Ultra。
- **SE**：Special Edition 特別版，實際多為平價版。配置縮水，價格降低。如 iPhone SE。
- **Plus**：加大版。螢幕比標準版大，電池都大，但配置基本一致。

- **Lite/青春版**：輕量版。配置精簡，價格更低。
- **Note**：原本指大螢幕手機，而家各品牌用法不一。Redmi Note 係中端系列。
- **Neo**：新銳版。通常指性能取向嘅中端機型。

五、基帶與天線：手機通信嘅幕後功臣

手機可以打電話、上網，靠嘅係基帶晶片同天線呢對拍檔。

基帶晶片：

基帶係手機中負責無線通信嘅處理器，佢將數碼信號調製成無線電波發出去，都將接收到嘅無線電波解調成數碼信號。基帶性能直接影響通話質量、上網速度同信號穩定性。高通係最大嘅手機基帶供應商，聯發科、三星都有自研基帶。蘋果由 iPhone 16e 開始搭載自研嘅 C1 基帶晶片，標誌住蘋果正式進入自研基帶時代，C1 基帶採用 4nm 製程，功耗表現優異；iPhone 16/16 Plus/16 Pro/16 Pro Max 仲用緊高通基帶（驍龍 X75 5G），預計後續機型會逐步過渡到自研基帶。更早嘅 iPhone XS/11 系列用 Intel 基帶嗰陣信號表現唔好，係唔少用戶嘅痛點。

天線設計：

手機天線收埋喺機身內部，通常圍繞邊框佈置。天線設計直接影響信號接收性能：

- **MIMO 技術**：多入多出，使用多個天線同時收發數據，提升速度同穩定性。4×4 MIMO 比 2×2 MIMO 速度更快。
- **天線數量**：旗艦機通常有十幾個天線（蜂窩、Wi-Fi、藍牙、NFC、GPS 各需要獨立天線）。
- **金屬機身嘅影響**：金屬會屏蔽信號，所以金屬機身手機需要設計天線帶（邊框上嘅塑膠條）。玻璃同塑膠機身對信號影響較細。

網絡技術演進：

- **4G LTE**：理論峯值速率下行 1Gbps/上行 500Mbps（Cat.18），實際體驗通常 50-150Mbps。覆蓋最廣，夠用。
- **5G NR**：理論峯值速率下行 10Gbps+。Sub-6GHz 頻段覆蓋較好，mmWave 毫米波速度極快但覆蓋極細。
- **5G-A（5G Advanced）**：5G 嘅增強版，又叫 5.5G。下行速率可達 10Gbps，上行 1Gbps，連接數密度同定位精度大幅提升。2024 年起中國移動、電信、聯通開始喺重點城市部署 5G-A 網絡，華為 Mate 70/80 系列、小米 15/16 系列等旗艦手機已支援 5G-A。5G-A 仲帶嚟咗「通感一體」能力，基站可以同時通信同感知環境，為低空經濟（無人機配送）同車聯網提供基礎設施。

網絡頻段分類：

手機支援嘅網絡頻段決定咗佢可以喺邊啲國家、邊啲電訊商嘅網絡上使用：

- **FDD-LTE Band（B 系列）**：如 B1、B3、B5 等，4G 頻段。唔同國家使用唔同頻段，中國主要用 B1/B3/B5/B8/B34/B38/B39/B40/B41。
- **TDD-LTE Band**：4G 嘅另一制式，中國大量使用 B38/B39/B40/B41。
- **5G NR Band（N 系列）**：如 n1、n3、n28、n41、n78、n79 等。n78（3.5GHz）係中國 5G 主力頻段，n41（2.5GHz）移動使用較多。
- **購買建議**：如果需要出國使用，購買前確認手機係咪支援目標國家嘅頻段。國行手機通常對中國頻段支援最全，但可能缺少部分海外頻段。

六、你嘅手機，到底喺度扮演幾多角色？

佢早已唔只係電話：

1. **通訊工具**：電話、短訊、微信、郵件……
2. **智能相機**：影相、錄影、掃描文檔……
3. **銀包**：支付、公交卡、銀行卡、會員卡……
4. **資訊門戶**：新聞、天氣、股票、知識……
5. **遊戲機**：由消消樂到 3A 大作……
6. **流動辦公室**：文檔、表格、會議、協作……
7. **健康助手**：計步、心率、睡眠、運動……
8. **智能家居遙控器**：控制燈、冷氣、電視、掃地機械人……
9. **數碼身份證**：電子身份證、電子駕照、健康碼……
10. **娛樂中心**：音樂、影片、電子書、播客……

諗下，一台細細嘅設備，承載咗幾多生活！所以，學識用好佢，就係學識更好咁生活。

實操一下：瞭解硬件配置

喺 [設定] 入面，睇下可唔可以搵到處理器型號、記憶體（RAM）同機身存儲（ROM）嘅具體大小。對你手機嘅能力有個數。安卓用戶可以下載「安兔兔」或「DevCheck」查看詳細硬件資訊。

附：安卓手機記憶體擴展解析

買手機嗰陣，你成日見到「*12GB+8GB* 記憶體」呢啲宣傳——好似憑空多咗 *8GB* 記憶體。但佢真係可以令手機變快嗎？會唔會反而傷手機？呢一節我哋就將「記憶體擴展」呢項技術徹底講清楚。

一、本質定義

安卓手機嘅「記憶體擴展」（又稱記憶體融合、RAM Plus 等）**並非真正物理增加記憶體晶片**，而係一項系統級軟件技術：通過將手機部分內部存儲空間（ROM/UFS 閃存）動態劃轉，模擬為虛擬記憶體（RAM）使用。

打個比方：物理 RAM 係「高速工作台」，擴展記憶體就係「臨時儲物間」。當工作台唔夠用嗰陣，系統會將暫時唔用嘅數據搬到儲物間，需要嗰陣再搬返嚟。儲物間容量大，但來回搬運肯定比直接放喺工作台上慢。

二、技術原理

主流廠商嘅實現方式主要有兩種：

zRAM 壓縮

喺 RAM 內部劃分一塊壓縮數據空間，將唔活躍嘅應用代碼進行壓縮存儲，壓縮率約 2:1。數據仲喺 RAM 度，速度快、唔損耗閃存，但需 CPU 參與壓縮解壓。

swap 分區

直接喺 UFS 閃存中劃分固定空間作為虛擬記憶體，將後台進程數據寫入存儲。依賴存儲帶寬，讀寫速度遠低於物理記憶體。

廠商宣傳嘅「12GB+8GB 記憶體」通常係 12GB 物理 RAM+8GB 閃存虛擬記憶體嘅組合。值得注意嘅係，zRAM 預設並唔佔用機身存儲，淨係當 zRAM 被數據完全填滿嗰陣，系統先會喺存儲上創建交換檔案，且呢種情況極少發生。

三、速度差距係關鍵瓶頸

呢項技術嘅根本侷限在於速度差異。目前主流嘅 UFS 4.0 閃存順序讀速約 4200MB/s，僅為 LPDDR5X 物理記憶體嘅 1/10-1/15。因此喺輕負荷嘅 zRAM 壓縮階段速度接近真實 RAM，但一旦進入高負載場景，速度會驟降到幾十分之一。

四、適用場景與收益邊界

記憶體擴展並非萬能，效果因機型同使用場景而異：

- **有用嘅情況**：6GB 物理記憶體 +UFS 3.1 以上、剩餘空間 ≥30GB 嘅機型，開啟 3GB 擴展後後臺保活數可多 3-4 個，重載率降低 68%，適合外賣小哥、網約車司機等重度多任務用戶。
- **日常場景**：對微信多開、網頁標籤保活、即時通訊消息響應等改善顯著，後台應用冷啟動耗時明顯縮短。
- **有限場景**：對大型遊戲載入、4K 影片剪輯等高帶寬場景提升有限。
- **有害嘅情況**：8GB 記憶體 +UFS 2.2 且剩餘空間不足嘅機型，開啟後遊戲幀率波動反而擴大，刷短影片多耗電，切換卡頓。
- **無效嘅情況**：16GB 以上大記憶體機型收益幾乎為零，最多多保活 1 個 APP，卻會令 CPU 負載增加 12%、發熱上升 2-3℃。對 RAM 大過 12GB 嘅機型，建議直接關閉以避免無意義嘅壓縮計算。

五、各廠商命名與開啟方式

唔同品牌叫法各異：小米叫「記憶體擴展」、華為稱「記憶體融合」、三星叫「RAM Plus」、vivo 命名為「Extended RAM」、一加稱作「Memory Extension」。呢個功能屬系統原生能力，同安卓底層版本無直接綁定關係。

一般開啟路徑為：進入【設定】【更多設定】搵到【記憶體】或【性能】子項開啟「記憶體擴展」並選擇容量檔位（通常 2GB-7GB），重啟後生效。開啟後系統會顯示如「8GB+4GB」嘅記憶體組合。

注意事項

- 記憶體擴展會佔用部分機身存儲空間，請確保剩餘存儲唔少過 20-30GB，否則可能影響系統流暢度。
- 閃存作為虛擬記憶體會有讀寫損耗，雖然現代 UFS 閃存壽命較長，但長期高頻 swap 仍可能加速存儲老化。
- 如果手機本身物理記憶體較細（如 4GB 及以下），開啟記憶體擴展淨係可以緩解燃眉之急，無法從根本上解決卡頓問題。

小結

記憶體擴展係一項成熟嘅系統級優化技術，可以喺硬件不變嘅前提下提升中低端機型嘅多任務能力同後台保活率，但佢受限於閃存同物理記憶體嘅巨大速度鴻溝，並非「萬能神藥」。係咪開啟、開幾大，需結合自身機型嘅物理記憶體大小、閃存規格、剩餘存儲空間同使用習慣嚟判斷。

嚟動手：睇下你嘅手機有冇記憶體擴展

1. 打開【設定】，搜尋「記憶體擴展」、「記憶體融合」、「RAM Plus」或「Extended RAM」。
2. 搵到後查看目前係咪開啟，以及擴展容量設定係幾多。

3. 對照本節內容判斷：你嘅機型物理記憶體幾大？閃存規格係乜嘢？剩餘存儲空間係咪充足？

4. 嘗試關閉或調低擴展容量，觀察一週後台保活同發熱變化，搵到最適合自己嘅設定。

第一章附：鴻蒙作業系統專題

鴻蒙（*HarmonyOS*）係華為自主研發嘅作業系統，由 *2019* 年發佈至今，已經走過咗幾個大版本。佢唔只係「又一個安卓替代品」，而係有住獨特設計理念嘅新物種。等我哋一齊深入瞭解呢個正在崛起嘅作業系統。

一、鴻蒙係乜嘢？點解佢與眾不同？

鴻蒙嘅核心理念係「萬物互聯」。唔同於傳統作業系統主要服務於單一設備，鴻蒙由設計之初就考慮咗多設備協同。

分佈式架構係鴻蒙嘅靈魂。簡單講，佢可以將多個設備虛擬成一個「超級終端」。你嘅手機、平板、電腦、手錶、電視，唔再係孤立嘅個體，而係一個整體嘅「器官」，可以互相調用對方嘅能力。

舉個例：你喺度用手機睇影片，想喺大螢幕上睇，傳統方式係投屏；而喺鴻蒙上，你可以直接將影片「流轉」到電視上繼續睇，手機變成遙控器。呢個唔係簡單嘅投屏，而係應用本身喺度跨設備運行。

二、鴻蒙 4.3：基於 AOSP 嘅成熟版本

鴻蒙 4.3 係目前華為手機上廣泛部署嘅版本。需要說明嘅係，鴻蒙 4.3 兼容安卓開源項目（AOSP）嘅代碼框架，因此佢可以完整兼容安卓應用——你喺華為應用市場下載嘅大部分 App，本質上仲係安卓應用，通過鴻蒙嘅兼容層運行。但鴻蒙 4.3 並非簡單套殼安卓，佢喺 AOSP 基礎上加入咗大量華為自研嘅分佈式能力、方舟引擎等技術。

核心特性：

- **安卓應用兼容**：由於基於 AOSP 開發，鴻蒙 4.3 可以正常運行幾乎所有安卓應用，用戶幾乎感受唔到差異。
- **方舟引擎**：華為自研嘅編譯器同運行時，等應用運行更流暢、更慳電。相比早期版本，流暢度提升明顯。
- **超級終端**：喺控制中心，可以直觀咁睇到周圍嘅其他鴻蒙設備，拖拽即可連接。手機調用電腦嘅鍵盤滑鼠、平板調用手機嘅鏡頭，都好方便。
- **服務卡片**：類似 iOS 嘅小組件，但更靈活。可以上滑 App 圖標展開卡片，顯示關鍵資訊，唔使打開 App。

- **純淨模式**：淨係允許由官方應用市場安裝應用，減少惡意軟件風險。
- **私隱中心**：集中管理應用權限，查看應用行為記錄。

用戶介面特點：

- **雜誌鎖屏**：鎖屏介面顯示精美圖片同文章，劃走即進入桌面。
- **萬能卡片**：桌面圖標可以展開成卡片，顯示更多資訊。
- **超級中轉站**：一個臨時存儲區，可以將文字、圖片、檔案暫存，然後拖到任意 App 中使用。跨應用分享變得無比絲滑。
- **智慧搜尋**：全局搜尋，可以搜應用、檔案、設定、網頁。

三、鴻蒙 NEXT（鴻蒙 6）：華為自主作業系統嘅真正起點

鴻蒙 NEXT（HarmonyOS NEXT 5.0）同鴻蒙 4.3 有住根本性嘅區別——佢採用華為自研嘅鴻蒙微內核，唔再依賴 Linux 內核同 AOSP 代碼，都**唔再直接支援安卓應用**。呢個係鴻蒙由「兼容安卓嘅過渡形態」正式走向「完全獨立嘅作業系統」嘅關鍵一步。

重大變革：

- **完全自主內核**：鴻蒙 NEXT 使用華為自研嘅內核，唔再包含任何 AOSP 代碼。系統更精簡、更安全、更高效。
- **唔再支援安卓應用**：呢個係最大嘅變化。鴻蒙 NEXT 淨係可以運行原生鴻蒙應用，無法安裝同運行 APK 檔案。升級前請確認你常用嘅 App 已經有鴻蒙原生版本。
- **原生應用生態**：華為正在全力推動開發者將應用遷移到鴻蒙原生平臺。目前主流 App 如微信、支付寶、淘寶等已推出鴻蒙原生版本，但部分小眾應用可能尚未適配。
- **星盾安全架構**：全新嘅安全體系，由硬件到軟件嘅全鏈路保護。支援更細粒度嘅權限控制，比如「模糊定位」——應用淨係可以知道你大概喺邊，唔知道精確位置。
- **AI 深度融合**：系統級 AI 能力，唔只係某個 App 有 AI。比如 AI 字幕可以實時翻譯影片，AI 摳圖可以智能識別主體。
- **跨設備無縫流轉**：更進一步嘅分佈式能力。應用狀態可以喺設備間無縫切換，你喺手機上編輯嘅文檔，行到電腦前自動喺電腦上打開。
- **元服務**：無需安裝即可使用嘅輕量級服務。比如掃二維碼直接打開某個功能，唔使先下載 App。

升級鴻蒙 NEXT 前必讀

升級到鴻蒙 NEXT 後，你嘅手機將無法運行安卓應用。請務必喺升級前確認：(1) 你常用嘅 App 已有鴻蒙原生版本；(2) 你嘅工作或生活中冇必須依賴安卓應用嘅場景。升級後無

法回退到鴻蒙 4.3。

操作邏輯變化：

- **手勢導航**：左右邊緣滑動返回，底部上滑返桌面，上滑停頓入多任務。
- **智慧多窗**：支援分屏、懸浮窗、平行視界等多種多任務形態。
- **暢連通話**：華為嘅視像通話服務，支援螢幕共享、美顏、背景虛化。

四、鴻蒙生態：設備與應用

鴻蒙唔只係手機系統，而係一個覆蓋多種設備嘅生態系統。

支援嘅設備類型：

- **手機與平板**：核心設備，功能最完整。
- **智能手錶與手環**：獨立運行或同手機協同。
- **智慧屏（電視）**：家庭娛樂中心。
- **智能音箱**：語音互動入口。
- **汽車**：鴻蒙座艙，智能駕駛艙體驗。
- **智能家居**：雪櫃、冷氣、洗衣機等。

應用生態現狀：

鴻蒙應用市場已有數百萬應用，主流 App 基本都有。華為都喺度積極推動開發者開發原生鴻蒙應用。截至 2026 年中，微信、支付寶、淘寶、抖音、美團、京東等頭部應用均已推出鴻蒙原生版本，日常使用基本無障礙。部分小眾應用同行業應用可能仲喺度適配中，升級前建議確認你常用嘅 App 係咪已支援。

五、鴻蒙實用技巧

- **超級終端**：由螢幕右上角下滑打開控制中心，點擊「超級終端」，可以睇到周圍嘅其他設備，拖拽連接。
- **超級中轉站**：長按文字/圖片/檔案，選擇「複製到中轉站」，然後喺其他 App 中長按貼上，或拖拽出嚟。
- **服務卡片**：上滑 App 圖標展開卡片，長按卡片可以調整大小或添加到桌面。
- **智慧多窗**：由螢幕左右邊緣向內滑動並停頓，打開側邊欄，點擊應用以懸浮窗打開。
- **私隱保護**：設定 > 私隱，可以查看應用權限使用記錄，設定模糊定位等。

注意事項

鴻蒙系統喺度不斷更新迭代，唔同版本可能有啲差異。本文描述基於鴻蒙 4.3 同 6 嘅主要特性，具體功能請以你嘅手機實際版本為準。建議保持系統更新，以獲得最新功能同安全補丁。

而家就試：體驗鴻蒙特色功能

1. 打開控制中心，搵到「超級終端」，睇下週圍有冇其他鴻蒙設備。
2. 試下「超級中轉站」：複製一段文字到中轉站，然後喺備忘錄中拖出嚟。
3. 上滑幾個常用 App 圖標，睇下佢哋嘅服務卡片顯示乜嘢內容。
4. 去設定 > 私隱，查看最近邊啲應用使用咗敏感權限。

第二章：整理你嘅數碼房間——檔案管理

手機用耐咗會亂，好習慣由管理檔案開始。就好似你嘅房間需要定期整理一樣，你嘅數碼空間都需要打理。一個井井有條嘅手機，唔只搵嘢方便，運行都更流暢。

一、三層存儲體系：你嘅數碼倉庫

1. **機身存儲**：核心區，速度快，系統同 App 都喺度。呢個係你手機最寶貴嘅空間，要精打細算。
2. **SD 卡（如果支援）**：擴展衣櫃，速度慢啲，適合放電影、相片等大檔案。注意：而家好多旗艦機唔支援 SD 卡喇。
3. **雲存儲**：保險櫃 + 同步網盤。防遺失，都可以喺多個設備間同步。iCloud、百度網盤、OneDrive 都係常用選擇。

二、認識「檔案管理器」

安卓一般有個 [檔案] 或 [檔案管理] App，iOS 係自帶嘅 [檔案] App。佢哋係你查看手機所有檔案嘅「資源管理器」。

常見資料夾含義：

- **DCIM**：相機拍攝嘅相片同影片。
- **Pictures**：其他圖片，如截圖、儲存嘅圖片。
- **Download**：瀏覽器下載嘅檔案。
- **Music**：音樂檔案。
- **Movies**：影片檔案。
- **Documents**：文檔檔案。
- **Android**：安卓系統資料夾，入面係各 App 嘅數據，一般唔好手動刪除。

三、檔案系統結構：深入瞭解

安卓基於 Linux，檔案系統有一定嘅層級結構。雖然普通用戶唔需要深入瞭解，但知道一啲基本概念有幫助。

存儲分區：

- **/system**：系統分區，存放作業系統檔案。普通用戶無權訪問，需要 root 權限。
- **/data**：數據分區，存放用戶數據同應用數據。呢個係你日常使用嘅空間。
- **/sdcard 或 /storage/emulated/0**：用戶可訪問嘅存儲空間，你嘅相片、下載檔案都喺度。

App 數據存儲：

- **內部存儲**：App 嘅私有空間，其他 App 無法訪問。卸載 App 嗰陣會被刪除。
- **外部存儲**：App 喺用戶可訪問區域存儲嘅檔案，如相片、下載內容。卸載 App 嗰陣唔一定會被刪除。

四、數據備份與恢復：畀你嘅數據上保險

數據無價，備份係必須養成嘅習慣。

備份策略：

1. **雲備份**：自動、方便，但需要網絡同雲空間。iCloud、華為雲、小米雲等都支援。
2. **本地備份**：用數據線連接電腦，用官方工具備份。速度快、空間大，但需要手動操作。
3. **雙重備份**：雲備份 + 本地備份，最安全。

備份乜嘢？

- **相片同影片**：最佔空間，都最珍貴。
- **通訊錄**：存在雲帳號入面最方便。
- **微信聊天記錄**：用微信自帶嘅備份功能。
- **重要文檔**：存在雲盤或電腦上。

五、檔案安全：保護你嘅私隱

- **敏感檔案加密**：好多手機有「安全資料夾」或「私密空間」功能，可以將敏感檔案放入去，需要密碼或指紋先可以訪問。

- **刪除唔等於銷毀**：普通刪除淨係標記空間可覆蓋，數據仲喺度。要徹底刪除，需要用專門嘅安全擦除工具，或者多次覆蓋寫入。

- **雲端安全**：選擇信譽好嘅雲服務商，開啟雙重認證，定期檢查登錄設備。

六、空間唔夠？嚟場大掃除

- **清理快取**：App 嘅臨時記憶，清咗可以騰出地方。喺設定 > 存儲入面可以清理。

- **查找大檔案**：用檔案管理器按大小排序，揪出佔空間嘅「元兇」。

- **刪除殘留**：卸載 App 後，有時會留低垃圾檔案，手動清掉。

- **清理相簿**：刪除重複相片、模糊相片、唔需要嘅截圖。

- **清理微信**：微信係空間大戶，定期清理聊天記錄同快取好有必要。

七、常見檔案格式全解析

手機入面各種檔案後綴名令人頭大，搞懂佢哋可以幫你更好咁管理檔案：

格式	類型	特性	推薦打開方式
JPG/JPEG	圖片	有損壓縮，體積細，兼容性最好。手機影相預設格式。	相簿/任意圖片查看器
PNG	圖片	無損壓縮，支援透明背景。截圖、圖標常用。體積比 JPG 大。	相簿/任意圖片查看器
MP3	音頻	有損壓縮，體積細，兼容性極好。最通用嘅音頻格式。	任意音樂播放器
FLAC	音頻	無損壓縮，音質與 CD 一致，體積比 MP3 大 3-5 倍。發燒友首選。	音樂播放器/專業播放 App
WAV	音頻	無壓縮原始音頻，音質最好但體積巨大。專業錄音常用。	音樂播放器
M4A	音頻	AAC 編碼，比 MP3 同碼率音質更好。蘋果生態常用。	Apple Music/音樂播放器
MP4	影片	最通用嘅影片封裝格式，兼容性極好。手機錄製嘅影片預設格式。	相簿/任意影片播放器
MKV	影片	開源封裝格式，支援多音軌、多字幕。電影資源常用。	VLC/nPlayer
DOC/DOCX	文檔	Word 文檔。DOCX 係壓縮格式，體積更細。DOC 係舊版格式。	WPS/Office
PPT/PPTX	演示	PowerPoint 演示文稿。PPTX 係壓縮格式。	WPS/Office
XLS/XLSX	表格	Excel 電子表格。XLSX 係壓縮格式。	WPS/Office
PDF	文檔	跨平臺格式，排版固定唔可編輯。合同、論文、電子書常用。	Adobe Reader/WPS
ZIP	壓縮	最通用嘅壓縮格式，跨平臺兼容好。	系統自帶/解壓 App
RAR	壓縮	壓縮率比 ZIP 高，但需要專用軟件解壓。	解壓 App
ISO	鏡像	光碟鏡像檔案，完整複製光碟內容。系統安裝包常用。	解壓 App/虛擬光驅

練一練：整理「下載」資料夾

打開檔案管理器 > 進入「下載」資料夾。睇下入面都有啲乜嘢？將冇用嘅刪掉，有用嘅分門別類（比如新建「工作資料」、「表情包」資料夾放入去）。順便睇下存儲空間，瞭解邊啲類型檔案佔用最多。

八、圖片與檔案系統管理：等資產井井有條

上一節我哋學識咗「認識資料夾」，但淨係認識都唔夠。本節要講嘅係**點樣主動建立一套可長期執行嘅管理規範**，等你嘅檔案隨時搵得到、唔遺失、唔被偷窺。

檔案分類與命名規範

混亂嘅根源係冇規則。建立規則其實好簡單，關鍵在於**堅持**。

- **「時間 + 事件」命名法**：資料夾名入面帶時間戳同事件描述，方便日後回憶同檢索。例如 202606_ 旅行票據、2025 年終總結、2024 孩子生日。咁樣既能睇到時間線，一眼就知道內容。
- **三級分類體系**：
  - **一級**：按檔案類型（文檔 / 圖片 / 影片 / 音頻）
  - **二級**：按使用場景（工作 / 生活 / 學習）
  - **三級**：按具體事件或項目（如「2026Q2_ 項目 A」）
- **月度清理計劃**：每月固定一個日期（建議出糧嗰日或月初 1 號），對「下載」目錄進行集中整理：**歸檔有用嘅、刪除冇用嘅、轉存珍貴嘅**。一年 12 次小整理，比起攢一年大掃除輕鬆得多。

本月任務清單

揀一個空閒嘅週末，按以下順序操作：

1. 喺檔案管理器中創建 `2026Q2_ 項目 A` 形式嘅本月工作資料夾；
2. 將所有「下載」目錄入面本週嘅檔案過一遍，留低有用嘅、歸檔、刪除冇用嘅；
3. 畀今年嘅家庭相片建立 2025 家庭、2026 家庭呢種年度資料夾。

冗餘檔案處理策略

手機用耐咗會「長胖」，多數係冗餘檔案喺度作怪。

- **72 小時自動清理**：微信、QQ 等社交 App 接收嘅臨時檔案（表情包、聊天圖片、短影片），大多睇一次就冇用。喺 [微信] > [設定] > [通用] > [存儲空間] 入面開啟「自動清理」，將過期快取設成 72 小時。手機瞬間清爽唔少，仲唔丟重要記錄。

- **存儲空間定向清理**：安卓 [設定] > [存儲] > [清理加速]，iOS [設定] > [通用] > [iPhone 存儲空間]，系統會按「快取 / 臨時檔案 / 可卸載 App / 大檔案」分類列出。重點清理：(1) 快取檔案；(2) 已過期安裝包（升級 App 後留低嘅舊版安裝包）；(3) 重複檔案（同名同尺寸嘅相片）。
- **自動分類規則**：多數品牌相簿 App 支援智能分類：人物、風景、文檔、截圖。開啟後，新拍嘅合影會自動歸入「人物 > 張三」，掃描嘅合同會自動歸入「文檔」。**配合每月整理，可極大降低「搵唔到嘢」嘅概率**。

圖片安全處理規範

圖片係手機入面最「敏感」嘅數據類型——一張隨手拍就可以暴露你嘅位置、設備、時間。我哋建議**建立「雙重保護」工作流**：

發送證件相/敏感截圖嘅標準流程

1. **可見資訊打碼**：用系統自帶或第三方工具（如美圖秀秀、Snapseed）**塗抹身份證號中間幾位、車牌號、門牌、姓名、金額**等關鍵字段；
2. **元數據剝離**：發送前喺相簿中點「分享」或「導出」，系統會詢問係咪保留位置資訊，選擇**「唔保留」**。咁樣拍攝設備型號、GPS 坐標、拍攝時間就不會跟住圖片一齊發出去；
3. **臨時水印**（可選）：喺敏感截圖上加「僅供 XX 辦理使用」嘅文字水印，避免被濫用。

元數據（EXIF）清理嘅常用方法（基礎操作，深度防護見第六章「相片脫敏處理」）：

- **系統自帶**：iOS [設定] > [私隱與安全性] > [相機]，可關閉「精確位置」；分享嗰陣選「僅當前相片」會**自動剝離大部分 EXIF**。
- **安卓**：多數品牌相簿 App 喺「分享」嗰陣會提示「係咪移除位置資訊」，勾選即可。
- **專業工具**：ExifTool（電腦端）、Metapho（iOS）、Photo Metadata Remover（安卓）可以徹底清除所有元數據。

對於需要**長期保存**嘅敏感圖片（身份證、銀行卡、戶口本掃描件、護照），建議存入**加密相簿**：

- iPhone：[相片] > 選中相片 > 隱藏（僅 Face ID 驗證後可查看）；
- 華為/小米：[相簿] > 長按照片 > 「移入保險櫃」；
- 第三方加密 App：**360 安全相簿**、**騰訊相冊管家（私隱相簿）**、**KeepSafe** 等，單獨密碼保護，隱蔽性更好。

第三章：等手機接上「生命線」——網絡

冇網，智能手機就「智障」咗一半。網絡係現代手機嘅生命線，理解佢、用好佢、保護佢，係每個手機用戶嘅必修課。

〇、聯網唔迷茫——分清 Wi-Fi 與流量嘅「路」與「車」

好多人嘅手機螢幕上方同時亮住 Wi-Fi 同信號圖標，但對二者嘅本質區別卻一頭霧水。最經典嘅誤區就係：**連住 Wi-Fi 卻刷唔出頁面**——因為路由器斷網喇，而手機並冇自動切返流量。要徹底弄懂呢個問題，我哋需要建立一個「路與車」嘅認知模型。

核心比喻：路與車

將互聯網諗像成一座城市。「**路**」係你手機到互聯網之間嘅通道，「**車**」係跑喺路上嘅數據。Wi-Fi 同流量係兩條唔同嘅路，但數據（車）本身係一樣嘅。

本質區別：本地通道 vs 遠程高速

Wi-Fi 本質上係一種**局域網技術**，佢就好似你屋企樓下無線鋪設嘅路，負責將數據由路由器傳到手機上。但如果寬頻欠費或光纜斷咗，路雖然仲喺度，外面嘅「高速公路」卻斷咗，自然去邊度都去唔到。而網絡流量（流動數據）就係通過電訊商嘅 4G/5G 基站傳輸，相當於你直接開上咗電訊商嘅高速專線，只要有信號就可以直連互聯網。

費用與場景：固定月租 vs 按量計費

Wi-Fi 通常係一次性投入（寬頻月費），只要電費唔斷，全家人無限使用，非常適合下載電影、打遊戲等大流量場景。而流量就按使用量計費，超出套餐可能會降速或額外扣費，更適合喺地鐵、出差等流動場景下處理輕量級任務。

嚟試下：檢查你嘅流量套餐

打開手機嘅 [設定] > [流動網絡] > [流量管理]，睇下你本月仲剩幾多流量、邊啲 App 最耗流量。心入面有個數，出門先唔慌。

安全考量：公共 Wi-Fi 係隱秘嘅「釣魚場」

喺安全性上，流動數據通常比公共 Wi-Fi 更安全。流動數據由電訊商加密，而公共 Wi-Fi 確實容易被黑客搭建為「偽熱點」截獲未加密嘅流量。但呢度要分清楚：喺 HTTPS 全面普及嘅今日，攻擊者能截獲嘅係**你訪問咗邊啲網站**（域名），以及**未加密嘅 HTTP 內容**；而微信消息、銀行 App 嘅登錄密碼等經過 HTTPS 加密嘅數據，攻擊者**無法直接讀取**——除非實施更複雜嘅 SSL 剝離攻擊且用戶忽略咗證書警告。不過即便如此，喺外面進行支付操作嗰陣，直接用手機流量仍然係最穩妥嘅選擇。

安全提醒

喺機場、商場、咖啡廳等公共場所，唔好連接名可疑嘅免費 Wi-Fi（如「Free_WiFi_2024」）。即使連咗，都絕唔好喺上面登錄銀行 App 或輸入密碼。

手機端判斷：睇懂狀態欄嘅「暗語」

點樣知道手機此刻喺度用乜嘢聯網？以華為手機為例，狀態欄中電訊商信號旁或 WLAN 信號旁出現**雙向箭頭圖標**，即代表正在使用對應方式上網；若兩者同時亮起，就說明網絡加速生效，手機正同時利用兩種通道。

iPhone 用戶就更直觀：Wi-Fi 圖標亮住就係喺度用 Wi-Fi，唔亮嗰陣睇右上角嘅信號格就係喺度用流量。部分安卓手機仲會喺狀態欄顯示「5G」或「4G」字樣，直接話畀你知目前流動網絡類型。

一、流動網絡（4G/5G）與流量

流動網絡係你隨時隨地聯網嘅基礎。由 2G 到 5G，速度提升咗上千倍，但原理其實差唔多。

網絡制式：

- **4G LTE**：目前覆蓋最廣，速度夠用（理論上最高 100Mbps+），耗電相對低。
- **5G**：速度快（理論上最高 10Gbps），但覆蓋仲喺度完善，耗電高。適合下載大檔案、睇高清影片。
- **5G SA vs NSA**：SA 係獨立組網，NSA 係依託 4G 基站。SA 體驗更好，但需要電訊商同手機都支援。

流量管理：

- **流量係乜嘢**：就係手機上網產生嘅數據量。睇影片、刷圖最費流量，文字同語音相對慳。
- **慳流小貼士**：喺設定入面開啟「慳流量模式」，Wi-Fi 下再更新 App 或睇高清影片。
- **信號格 $\neq$ 網速**：信號強淨係代表連接好，網速仲要睇基站忙唔忙、你離基站幾遠。

重要警告：出國前關閉【數據漫遊】!

喺國外使用流動數據可能產生天價賬單！出國前務必喺設定入面關閉數據漫遊，或者提前購買當地流量卡/國際漫遊套餐。

二、Wi-Fi：連接、分享與安全

Wi-Fi 係家庭同辦公室嘅聯網主力，速度快、唔費流量。但 Wi-Fi 其實都喺度不斷進化，唔同標準之間差異好大。

連接 Wi-Fi：

去 [設定] > [WLAN] 選擇網絡輸密碼。而家好多手機支援掃碼連接，更方便。

分享密碼：

安卓同 iOS 都可以等已連接嘅手機快速分享密碼畀朋友。安卓生成二維碼，iOS 可以靠近自動分享。

Wi-Fi 標準演進：

Wi-Fi 標準由 IEEE 制定，每一代都有顯著提升：

標準	協議	頻段	最高速率	特點
Wi-Fi 4	802.11n	2.4/5GHz	600Mbps	老舊標準，覆蓋還行
Wi-Fi 5	802.11ac	5GHz	3.5Gbps	目前最普及，速度夠用
Wi-Fi 6	802.11ax	2.4/5GHz	9.6Gbps	多設備性能強，延遲低
Wi-Fi 6E	802.11ax	2.4/5/6GHz	9.6Gbps	增加 6GHz 頻段，唔擁堵
Wi-Fi 7	802.11be	2.4/5/6GHz	46Gbps	極速傳輸，多鏈路聚合

- **Wi-Fi 6 嘅關鍵技術：**OFDMA（正交頻分多址）等多個設備同時傳輸，MU-MIMO（多用戶多入多出）提升多設備並發效率，TWT（目標喚醒時間）等設備更慳電。
- **Wi-Fi 6E：**喺 Wi-Fi 6 基礎上增加咗 6GHz 頻段，提供更多信道，減少幹擾。需要手機同路由器都支援。
- **Wi-Fi 7：**引入 MLO（多鏈路操作），可同時使用多個頻段傳輸數據，大幅降低延遲。320MHz 超寬信道等速率飛躍。驍龍 8 Elite Gen5、天璣 9400 等旗艦晶片平臺均已支援 Wi-Fi 7，小米 17、華為 Mate 80、三星 S26 等機型可使用 Wi-Fi 7 網絡（需配合 Wi-Fi 7 路由器）。

- **2.4GHz vs 5GHz**：2.4GHz 穿牆好但速度慢、幹擾多；5GHz 速度快但穿牆弱。Wi-Fi 6E/7 嘅 6GHz 頻段速度最快，但穿牆最弱，適合同一房間內使用。

公共 Wi-Fi 風險：

公共 Wi-Fi 可能有黑客竊聽數據或設釣魚熱點。守則：

- 避免登錄銀行賬戶、輸入密碼。
- 最好用 VPN 加密連接。
- 唔好自動連接陌生 Wi-Fi。
- 用 HTTPS 網站（地址欄有鎖圖標）。

三、藍牙：短距離無線通信嘅萬能選手

藍牙可能係你手機上使用頻率最高嘅無線技術之一——無線耳機、智能手錶、車載系統、鍵盤滑鼠，都靠佢連接。

技術原理：

藍牙工作喺 2.4GHz ISM 頻段，使用跳頻擴頻技術（FHSS）將數據分散喺 79 個頻道上快速切換傳輸，有效避免幹擾。藍牙嘅通信距離分為三個等級：Class 1 約 100 米、Class 2 約 10 米（手機常用）、Class 3 約 1 米。

版本演進：

- **藍牙 4.0**：引入低功耗藍牙（BLE），等手環、心率帶呢類細電池設備都可以無線連接，續航由天變成月。
- **藍牙 4.2**：提升傳輸速度同私隱保護，支援 IPv6 直連。
- **藍牙 5.0**：傳輸速度翻倍（2Mbps），傳輸距離 4 倍（理論 300 米），廣播能力 8 倍。支援雙音頻（同時連兩副耳機）。
- **藍牙 5.1**：引入方向尋址功能（AoA/AoD），定位精度可達釐米級。
- **藍牙 5.2**：LE Audio 新標準，支援 LC3 編碼（音質更好、功耗更低）、多流音頻、廣播音頻。
- **藍牙 5.3/5.4**：進一步優化功耗同連接穩定性，增強週期性廣播。

LE Audio 普及情況：2025-2026 年，LE Audio 已進入快速普及階段。主流品牌嘅新款真無線耳機（如華為 FreeBuds Pro 5、小米 Buds 6 Pro、三星 Galaxy Buds3 Pro 等）均已支援 LE Audio 同 LC3 編碼，提供更低延遲同更好音質。購買新耳機嗰陣，建議優先選擇支援 LE Audio 嘅產品。

應用場景：

- **音頻傳輸**：無線耳機、車載音響。藍牙 5.2 嘅 LE Audio 等音質同延遲都有質嘅飛躍。
- **數據傳輸**：近距離傳檔案、相片。雖然速度唔及 Wi-Fi Direct，但勝在簡單。
- **設備配對**：智能手錶、手環、鍵盤滑鼠、遊戲手柄。
- **位置服務**：藍牙信標（Beacon）用於室內導航、物品追蹤（如蘋果 AirTag）。
- **智能家居**：藍牙 Mesh 網絡控制燈泡、門鎖等。

四、紅外遙控：手機變身萬能遙控器

紅外遙控係最古老嘅無線控制技術之一，但喺手機上依然實用。部分安卓手機（如小米、華為部分機型）內置咗紅外發射器，可以遙控電視、冷氣、機頂盒等家電。

工作原理：

紅外遙控通過紅外發光二極管發出特定頻率（通常 38kHz）嘅紅外光脈衝信號，家電嘅紅外接收器解碼呢啲脈衝執行對應操作。唔同品牌使用唔同嘅編碼協議，常見嘅有 NEC、RC5、RC6、SONY 等。手機入面嘅紅外 App 內置咗各品牌嘅編碼庫，選擇品牌同型號就可以匹配。

侷限性：

- **方向性**：紅外係直線傳播，需要對住設備發射，唔可以隔牆。
- **距離短**：有效距離一般 5-8 米。
- **蘋果唔支援**：iPhone 冇紅外發射器，無法使用呢個功能。

五、NFC：碰一碰嘅魔法

NFC（Near Field Communication，近場通信）係一種極短距離（10 釐米以內）嘅高頻無線通信技術，工作頻率 13.56MHz，傳輸速率最高 424Kbps。

通信原理與模式：

NFC 基於 RFID 技術演變而嚟，支援三種工作模式：

- **讀寫模式**：手機讀取 NFC 標籤中嘅資訊，如讀取 NFC 貼紙觸發動作。
- **點對點模式**：兩台 NFC 設備互相傳輸數據，如 Android Beam（已停用）。
- **卡模擬模式**：手機模擬一張智能卡，呢個係最常用嘅模式——公交卡、門禁卡、銀行卡都係呢個模式。

安全機制：

- **超短距離**：10 釐米內嘅通信距離本身就係物理安全保障，遠程竊聽幾乎唔可能。
- **安全晶片**：手機內置獨立嘅安全晶片（SE）或使用 HCE（主機卡模擬），支付密鑰存儲喺安全區域，即使手機被 root 都無法提取。
- **指紋/面容驗證**：支付嗰陣需要生物識別確認，即使手機遺失都無法盜刷。
- **令牌化**：Apple Pay、華為 Pay 等唔傳輸真實卡號，而係使用一次性令牌，即使被截獲都無法重複使用。

典型應用：

- **公交卡/門禁卡**：將實體卡資訊寫入手機，刷手機就得。
- **流動支付**：Apple Pay、華為 Pay 等，將銀行卡綁定到手機，碰一碰支付。
- **設備配對**：NFC 碰一碰快速配對藍牙耳機、音箱。
- **身份驗證**：NFC 身份證讀取、電子駕照驗證。

六、個人熱點：將手機變 Wi-Fi 路由器

自己流量多嗰陣，可以開熱點畀平板、電腦用。

- **開啟方法**：設定 > 個人熱點/網絡共享，開啟佢。
- **注意事項**：好耗電同流量，記得設個複雜密碼，並限制連接設備數。
- **USB 共享**：用數據線連電腦，比無線熱點更穩定、更快。

動手試下：設定並管理熱點

1. 打開 [設定]，搵到「個人熱點」或「網絡共享」，開啟佢，設好名稱同密碼。
2. 用另一台設備（比如電腦）搜尋並連接呢個 Wi-Fi。
3. 喺手機端睇下，可唔可以管理已連接嘅設備，或踢走唔識嘅設備。

第四章：安裝同管理 App——手機嘅「技能包」

App 等手機無所不能，但都要管好佢哋。就好似你會畀新員工安排工位、分配權限一樣，安裝 *App* 都需要一套規範流程。

一、官方商店：最安全嘅「應用超市」

- **安卓**：Google Play（國外）或各品牌自帶商店（如華為應用市場、小米應用商店）。
- **iOS**：淨係得 App Store。
- **鴻蒙**：華為應用市場。

核心安全意義：經過審核、更新有保障、支付安全。行動指南：**優先淨係用官方商店下載！**

二、純淨/防護模式：開啟手機嘅「防毒牆」

主流國產品牌（華為、小米、OPPO、vivo、榮耀）都內置咗**純淨模式**或**防護模式**，呢個係系統級別嘅安全開關。**強烈建議所有用戶都開啟**。

佢到底係乜嘢？

純淨模式（又叫「防護模式」）嘅**核心功能係阻斷第三方 App 嘅安裝**：當佢開啟嗰陣，系統會**淨係允許由官方應用商店安裝 App**，由瀏覽器、二維碼、聊天軟件等渠道下載嘅安裝包會被直接攔截，由源頭杜絕惡意軟件。

適用人羣

- **老年用戶**：分唔清「官方」同「假冒」廣告，純淨模式係最穩妥嘅護身符。
- **對應用權限唔敏感嘅用戶**：唔需要安裝小眾 App、測試版、外服遊戲。
- **希望提升設備安全性嘅所有用戶**：哪怕係發燒友，開住都好過唔開——真係有臨時需要，再單獨關一下即可。

開啟路徑（以華為/小米為例）

- **華為/鴻蒙**：[設定] > [系統同更新] >**[純淨模式]**，選擇「開啟」。
- **小米**：[設定] > [私隱保護] >**[保護私隱]** 或 [設定] > [應用設定] > [應用安裝]，關閉「安裝未知應用」。
- **OPPO/vivo**：[設定] > [安全] >**[安裝外部來源應用]**，逐個關閉授權。

應用商店嘅安全檢測機制

應用商店唔係「擺設」，佢背後有完整嘅安全流水線：

- **病毒掃描**：每個 App 上架前會經過多家殺毒引擎聯合掃描，識別已知木馬同病毒特徵。
- **權限審核**：商店會審查 App 申請嘅權限係咪「合理」——一個手電筒 App 要通訊錄權限？直接駁回或下架。
- **開發者認證**：上架需要企業資質、個人身份證或信用卡認證。出問題可以搵到人，唔係「黑App」。
- **持續監控**：已上架嘅 App 都會被持續複掃，新發現嘅惡意行為會被強制下架。

三、第三方來源與「電子寄生蟲」：風險極高！

由網頁或不明連結下載 App，可能夾帶病毒、竊取私隱、盜刷支付。呢度我哋特別提一個新概念：**「電子寄生蟲」**。

乜嘢係「電子寄生蟲」?

指嘅係嗰啲**被植入木馬嘅「破解版」軟件**——你以為係「慳咗錢」，實際係「中咗招」。常見形式：

- 「破解版」、「免費版」、「去廣告版」付費軟件；
- 「刷讚」、「搶紅包」、「遊戲外掛」等灰色工具；
- 羣聊入面「小廣告」推薦嘅所謂「內部版」App。

呢啲程式表面能用，背後可能喺度**偷通訊錄、讀短訊、監聽麥克風、後台挖礦**。純淨模式係防範佢哋嘅**第一道防線**——直接唔等你裝。

第二道防線建議：

- **定期全盤病毒掃描**：使用手機自帶安全中心（如華為手機管家、小米安全中心）每月掃一次。
- **謹慎授予敏感權限**：通訊錄、短訊、位置、相機、麥克風——呢五項「高危權限」必須嚴格審批。

- **關注耗電同流量異常**：手機突然發燙、電量跌得快、流量半夜跑——八成係後台收埋咗「寄生蟲」。

如果萬不得已要裝第三方 App：

1. 確保來源絕對可信。
2. 安裝前仔細睇權限要求。
3. 事後用安全 App 掃描。

終極建議：喺系統設定入面，關掉「安裝未知應用」嘅權限。

四、裝、更新、卸載

- **安裝**：商店入面點「取得」或「安裝」，裝完首次打開會問你要權限。
- **更新**：**必須更！**主要係修復安全漏洞。建議開啟自動更新。
- **卸載**：長按圖標拖到卸載區。諗住清得更徹底，去 [設定] > [應用管理] 入面搵到並卸載。

五、權限管理：你係 App 嘅「老闆」

App 要權限（比如通訊錄、位置）有時合理（地圖要位置），有時唔合理（手電筒要通訊錄）。

- **權限選項**：「始終允許」、「僅喺使用時允許」、「禁止」、「每次詢問」。
- **管理原則**：
  1. 最小化：非必要唔畀。
  2. 事後審計：定期去權限設定入面睇下邊個喺度「越權」。
  3. 情境化：好似打車軟件，淨係用嗰陣畀位置權限。

重點盯防嘅權限

- 通訊錄/短訊：可能被用嚟發垃圾短訊。
- 實時位置：可能被追蹤行蹤。
- 相機/麥克風：可能被偷拍偷錄。
- 檔案訪問：可能讀取你嘅私隱檔案。

六、軟件安裝包嘅構成要素

你有冇諗過，由應用商店下載嗰個安裝包，入面到底裝咗乜嘢？瞭解安裝包嘅構成，可以幫你更好咁理解 App 係點樣工作嘅，都可以喺遇到問題嗰陣知道該由邊度排查。

安卓 APK 檔案結構

APK（Android Package Kit）本質係一個 ZIP 壓縮包，解壓後可以睇到以下核心目錄同檔案：

- **AndroidManifest.xml：**應用嘅「身份證」同「說明書」。記錄咗應用包名、版本號、所需權限、支援嘅系統版本、入口 Activity 等核心資訊。系統安裝應用嗰陣首先讀取呢個檔案。
- **classes.dex：**應用嘅核心代碼。Java/Kotlin 源代碼編譯後變成 Dalvik 字節碼，存喺呢個檔案入面。一個應用可能有多個 dex 檔案（如 classes2.dex、classes3.dex）。
- **resources.arsc：**編譯後嘅資源索引表。記錄咗所有資源嘅 ID 同對應關係。
- **res/目錄：**資源資料夾。包含圖片、佈局檔案、字串、顏色定義等。唔同分辨率嘅圖片放喺唔同嘅 drawable 資料夾入面（如 drawable-hdpi、drawable-xhdpi）。
- **assets/目錄：**原始資源資料夾。存放唔經過編譯嘅資源，如網頁檔案、字體、音頻等。App 運行嗰陣可以通過 AssetManager 讀取。
- **lib/目錄：**本地庫資料夾。存放編譯好嘅.so 檔案（C/C++ 代碼編譯產物），按 CPU 架構分類（如 arm64-v8a、armeabi-v7a）。遊戲同需要高性能計算嘅 App 通常有呢個目錄。
- **META-INF/目錄：**簽名資訊資料夾。包含 MANIFEST.MF、CERT.SF、CERT.RSA 三個檔案，用於驗證 APK 嘅完整性同簽名。

iOS IPA 檔案結構

IPA（iOS App Store Package）同樣係壓縮包，結構略有唔同：

- **Payload/目錄：**核心內容所在。入面有一個.app 資料夾，包含應用嘅所有內容。
- **Info.plist：**應用嘅配置檔案，類似安卓嘅 AndroidManifest.xml。記錄咗應用 ID、版本、權限需求、支援嘅設備等。
- **可執行檔案：**與.app 資料夾同名嘅主程式，係編譯後嘅 Mach-O 格式二進制檔案。
- **Frameworks/目錄：**應用依賴嘅動態庫同框架。
- **資源檔案：**圖片、音頻、影片等資源，通常存放在.app 資料夾內。
- **_CodeSignature/目錄：**代碼簽名資訊，用於驗證應用嘅完整性同來源。

簽名驗證機制

點解由官方商店下載嘅 App 更安全？簽名驗證係關鍵。

- **安卓簽名**：開發者用私鑰對 APK 進行簽名，系統安裝嗰陣用公鑰驗證。如果 APK 被篡改（比如植入病毒），簽名就會失效，系統會拒絕安裝。Google Play 仲要求 App Signing，由 Google 保管簽名密鑰，進一步保護應用安全。
- **iOS 簽名**：蘋果嘅審核更嚴格。每個 IPA 必須有蘋果頒發嘅證書簽名，否則無法安裝。呢個就係點解 iOS 淨係可以由 App Store 或企業證書安裝應用。企業證書簽名嘅應用有效期有限，且蘋果可以隨時吊銷。
- **簽名嘅作用**：
  1. 驗證來源：確認應用確實來自聲稱嘅開發者。
  2. 驗證完整性：確保應用喺傳輸過程中未被篡改。
  3. 防止冒充：惡意應用無法偽裝成其他應用。

安裝包大小與優化

你可能會發現，應用商店顯示嘅 App 大小同安裝後佔用嘅空間唔一樣。呢個係因為：

- **下載大小**：壓縮後嘅 APK/IPA 大小。
- **安裝大小**：解壓後嘅大小，通常比下載大小大好多。
- **運行時佔用**：App 運行嗰陣仲會產生快取、數據檔案，佔用更多空間。

現代 App 嘅優化手段：

- **App Bundle（安卓）**：Google Play 嘅動態分發技術。用戶下載嗰陣淨係獲取適合自己設備嘅資源（如淨係下載對應螢幕分辨率嘅圖片），大大減少下載大小。
- **App Thinning（iOS）**：類似嘅技術，蘋果會根據設備自動裁剪唔需要嘅資源。
- **按需下載**：遊戲等大型應用可以先下載核心包，後續再下載資源包。

小知識：點解有啲 App 安裝包特別大？

遊戲類 App 通常最大，因為包含大量高清貼圖、3D 模型、音頻檔案。社交類 App 都比較大，因為要支援各種功能模組。工具類 App 通常最小。如果你發現某個工具類 App 特別大，要警惕係咪捆綁咗唔必要嘅功能。

實操環節：審查 3 個 App 嘅權限

揀 3 個常用 App（如社交、購物、工具類），進入 [設定] 嘅應用權限管理頁，逐項審查，將唔必要嘅敏感權限關掉或改成「僅喺使用時允許」。

第五章：等手機更懂你——基礎設定

花幾分鐘設定，體驗提升巨大。呢啲設定就好似畀手機「調音」，等佢更符合你嘅使用習慣。

一、聲音與震動

設個鍾意嘅鈴聲同通知聲。靈活使用靜音模式（開會）、勿擾模式（瞓覺），世界瞬間清靜。

- **鈴聲**：揀一個你聽到嘅、唔尷尬嘅。
- **震動模式**：靜音嗰陣靠震動提醒。
- **勿擾模式**：可以設定例外，比如允許通訊錄來電。
- **音量分離**：媒體音量、鈴聲音量、鬧鐘音量可以分開調。

二、顯示與亮度

- **自動亮度**：等手機根據環境光自動調節，慳心。
- **護眼/夜覽模式**：過濾藍光，夜晚睇手機舒服啲。
- **深色模式**：慳電（對 OLED 螢幕特別有效），且夜間唔刺眼。好多 App 都支援深色模式。
- **螢幕刷新率**：部分安卓機可調，越高（如 120Hz）滑動越絲滑。
- **自動鎖屏**：設定 15 秒或 30 秒，安全又慳電。
- **字體大小**：覺得字細？調大啲。

三、桌布、主題與桌面

換張好睇嘅桌布（靜態或動態），心情都變咗。安卓品牌手機常有主題商店，能整套換圖標同風格。

桌面小組件：天氣、日程、音樂控制放桌面，一步直達。iOS 同安卓都支援，長按桌面空白處添加。

四、效率設定：等手機成為你嘅「外掛大腦」

呢部分係「一旦設定就返唔到轉頭」嘅功能，每個都可以等你嘅日常效率顯著提升。

專注 / 免打擾模式：奪回被通知奪走嘅注意力

- **核心思想**：唔係「少睇手機」，而係「喺該專注嗰陣專注，該聯絡嗰陣再聯絡」。
- **使用建議**：
  - 工作模式：僅允許「星標」聯絡人同必要 App 通過；
  - 睡眠模式：自動開啟，僅允許鬧鐘；
  - 健身 / 閱讀 / 會議：根據目前活動定製。
- **進階玩法**：用自動化 + 位置 / 時間觸發，等模式自動切換。例如：iOS 快捷指令可設定「到達公司即自動進入工作模式」。

通知管理：等紅點唔再「奪命連環 call」

- **關閉營銷通知**：90% 嘅推送其實你並唔需要。
- **按重要性排序**：將人際通訊類（電話、短訊、即時通訊）置頂；新聞、購物、社交平臺類可設為「靜默推送」。
- **分組通知**：將同類通知摺疊埋一齊，避免一次解鎖被幾十條消息淹沒。

快捷手勢：等操作快人一步

- **截屏**：電源 + 音量下（iOS 全面屏）/電源 + 音量下（安卓通用）。
- **截長圖 / 滾動截屏**：普通截圖後點擊「長截圖」選項。
- **一鍵掃碼**：控制中心 / 快捷設定入面添加「掃一掃」圖標。
- **輕點背面（iPhone）**：喺 [設定] > [輔助功能] > [觸控] > [輕點背面] 中綁定「截屏」或「鎖屏」等動作。
- **語音喚醒**：Siri / 小愛同學 / 小藝 / 小布，嗌一聲就可以查天氣、設鬧鐘。

輸入法：最被低估嘅效率神器

- **揀啱輸入法**：
  - 搜狗 / 百度：詞庫大，流行語更新快；
  - 訊飛：語音輸入識別率最高，開會記筆記神器；
  - Gboard（谷歌）：簡潔、跨平臺同步強、支援滑行輸入；
  - iOS 自帶：私隱最好，同系統深度集成。
- **必備開關**：智能糾錯、聯想詞、雲詞庫同步、上滑輸入數字符號。

- **私隱提示**：慎用「整句聯想」或「個性化推薦」功能，可能上傳輸入內容。

桌面與多任務：等工作流順暢起嚟

- **App 分屏 / 畫中畫**：睇影片嗰陣回消息，上網嗰陣記筆記。
- **小窗 / 懸浮窗**：微信 / 抖音支援「小窗模式」，回覆完即收起。
- **常用 App 置頂**：Dock 欄（螢幕底部）淨係放最高頻嘅 4 個，減少查找時間。

「上工第一天」檢查清單

攞到新手機或大版本更新後，建議花 10 分鐘過一遍呢啲設定：
(1) 鎖屏密碼 + 生物識別 (2) 通知分組 + 關閉營銷推送 (3) 字體大小 / 刷新率 / 深色模式 (4) 預設輸入法 / 瀏覽器 / 郵件 (5) 緊急聯絡人 / 醫療卡 (6) 備份設定

五、輔助功能與無障礙：科技向善嘅力量

唔好被「輔助功能」呢個名嚇親，呢啲功能人人可以用。佢哋最初係為殘障人士設計嘅，但對普通人都好有用——臨時手受傷抱娃、地鐵上耳機壞咗聽唔清、戴老花鏡睇細字……都可以救場。講到底，無障礙設計體現咗**科技向善**嘅理念：等每個人都可以平等咁享受數碼生活。

視障用戶嘅「眼睛」——螢幕閱讀器全攻略

螢幕閱讀器係視障用戶使用手機嘅核心工具。佢會將螢幕上**所有可視內容朗讀出嚟**，用戶通過手勢同語音操作手機。下面分別介紹 iOS 「旁白」同安卓 TalkBack 嘅使用方法。

iOS 旁白（VoiceOver）

開啟路徑：[設定] > [輔助功能] > [旁白]，打開開關。都可三擊側邊鍵（或主螢幕鍵）快速開關。
基本手勢：

1. 單指滑動：逐元素瀏覽螢幕內容；
2. 雙指輕掃：上下/左右整頁滾動；
3. 單指雙擊：激活目前焦點元素（相當於「點擊」）；
4. 三指輕掃：切換頁面；
5. 單指三擊：放大或選擇（與普通三擊唔同）。

個性化設定：[旁白] > [語音]，可調節**朗讀語速**（預設約 50%，可調到 70-80% 聽感更流暢）、**音調**、切換唔同語音包。iOS 自帶「增強語音」同「語音控制器」，前者能減少呼吸聲同停頓，聽感更自然。

安卓 TalkBack

開啟路徑：[設定] > [輔助功能] > [TalkBack]，打開開關。或長按音量上下鍵 3 秒快速啟用（部分品牌支援）。
基本手勢：

1. 單指左右滑動：切換焦點元素；
2. 單指雙擊：激活元素；
3. 雙指上下/左右滑：滾動；
4. 三指單擊：進入閱讀模式（連續朗讀）；
5. 四指單擊：暫停/繼續朗讀。

個性化設定：[TalkBack] > [文字轉語音設定]，可調整**語速**、**音調**、切換**語音引擎**（建議安裝 Google 文字轉語音引擎，朗讀更自然）。

等細字變大、等模糊變清晰——視覺調整

放手勢

- **iOS**：[設定] > [輔助功能] > [縮放]，開啟後**三指輕點螢幕**即可放大。預設全屏縮放，可選窗口縮放（局部放大窗）。**雙指開合可調節倍數（1.5x 至 15x）**，三指雙擊鎖定目前縮放比例。

- **安卓**：[設定] > [輔助功能] > [放大]，三指點擊放大，再三指點擊退出；或直接**三指點擊螢幕三下**啟用臨時放大（最多 8 倍）。

高對比度文字 / 粗體文字

- **iOS**：[設定] > [輔助功能] > [顯示與文字大小]，開啟「粗體文字」、「增強對比度」、「降低透明度」等開關。「智能反轉」可喺淺色背景下等文字保持黑色，避免白底白字。

- **安卓**：[設定] > [顯示] > [字體大小]，可調節「細、普通、大、超大」四檔。華為/小米額外提供「粗體」同「高對比度文本」開關。

色彩濾鏡（色盲友好）

- **iOS**：[設定] > [輔助功能] > [顯示與文字大小] > [色彩濾鏡]，提供**紅/綠/藍/黃四種色盲模式**，以及「灰度」同「反轉顏色」選項。建議先做一次色覺測試（系統自帶）再選對應濾鏡。

- **安卓**：[設定] > [輔助功能] > [顏色校正]，含**三種色盲模式** + 「反轉顏色」開關。

AI 圖像描述：等 AI 當你嘅「眼睛」

當螢幕閱讀器遇到圖片嗰陣，傳統做法係跳過或淨係讀出「圖片」兩個字。但而家 AI 可以**用一句話描述圖片內容**，對識別驗證碼、讀圖、識別菜單好有幫助。

各系統 AI 圖像描述支援情況

- **iOS 15+**：喺 [設定] > [輔助功能] > [旁白] > [語音描述] 中開啟「圖像描述」。系統會調用**本地 AI 模型**自動生成圖片文字描述（需 iPhone 12 及以上）。
- **新版 TalkBack 應用**：Google TalkBack 喺較新版本中集成**圖片問答**功能，可以問「呢張圖入面有乜嘢」，AI 實時回答（唔嚴格依賴安卓系統版本，更新 TalkBack 應用即可體驗）。
- **第三方 App**：「Seeing AI」（iOS）、「Lookout」（安卓）、「小藝幫幫」（華為）都係優秀嘅 AI 視覺助手，可識別物品、讀檔案、識別人臉表情。

使用場景

試下對住一張街拍相片問「呢張圖入面有幾多人？佢哋喺度做乜嘢?」，或者打開外賣 App，等 AI 讀出菜單圖入面嘅菜品名——**呢個係真正嘅「科技等生活更平等」**。

聽障用戶嘅「耳朵」——實時字幕全攻略

實時字幕（Live Captions）係**系統級嘅語音轉文字**功能，聽障用戶喺嘈雜環境、圖書館、外語場景下都特別有用。

實時字幕開啟與優化

iOS：[設定] > [輔助功能] > [實時字幕]，打開開關。系統會檢測設備所有音頻（通話、影片、播客、FaceTime、語音備忘錄），自動顯示字幕喺螢幕上方。
安卓 12+：按**音量鍵**點擊彈出框中嘅「實時字幕」圖標；或喺 [設定] > [輔助功能] > [實時字幕] 中開啟。
準確率優化技巧：

1. 盡量喺**安靜環境**下使用，遠離冷氣、機器轟鳴；
2. 說話者**咬字清晰、語速適中**嗰陣識別率最高；
3. 多人對話會降低準確率，可使用「淨係顯示一人」模式；
4. 支援中文普通話、粵語、英語等主流語種，**小語種識別率較低**；
5. 說話中途有背景音樂或人聲幹擾，識別會斷斷續續。

通話場景雙通道：語音轉文字 + 文字轉語音

當用戶**既聽唔到都講唔到嘢**嗰陣（如突發失聲、喉部手術恢復期、嚴重口吃），手機仲可以當「傳話員」。

通話實時轉譯（TTY / RTT）

iOS：[設定] > [輔助功能] > [RTT/TTY]，開啟後通話嗰陣螢幕上會顯示鍵盤，對方嘅語音會被實時打成文字，**你打嘅字都會被念出嚟**畀對方聽。
安卓：[設定] > [輔助功能] > [實時轉錄] + [聲音通知]。「實時轉錄」將對方說話實時顯示為文字；「聲音通知」喺不便語音嗰陣將你嘅輸入轉為語音播放。
兼容性：

- **RTT（Real-Time Text）**：雙方都需支援，文字邊打邊顯示，無延遲，主流電訊商已支援；
- **TTY（電傳打字）**：需專用 TTY 設備，已逐步被 RTT 取代；
- 微信/QQ 嘅**語音轉文字功能**都係替代方案：長按語音消息「轉文字」。

面向所有人嘅「細但實用」功能

- **輕點背面**（iOS 14+）：[設定] > [輔助功能] > [觸控] > [輕點背面]，可綁定**雙擊/三擊**觸發截圖、鎖屏、控制中心等動作。雙手攞快遞嗰陣特別好用。
- **輔助觸控 / 懸浮球**：iOS [輔助功能] > [觸控] > [輔助觸控]，安卓 [輔助功能] > [懸浮導航]。虛擬按鈕替代手勢或物理按鍵，按鍵唔靈或單手操作嗰陣救場。
- **切換控制**：通過外部開關、單次點擊、頭部動作控制手機，**為肢體障礙用戶設計**。iOS [輔助功能] > [切換控制]，安卓 [輔助功能] > [開關控制]。
- **聲音識別**（iOS）：[輔助功能] > [聲音識別]，開啟後手機會「聽」門鈴、嬰兒喊聲、火警警報、警笛等聲音，**識別到就震動 + 螢幕通知**，對聽障用戶同睡眠期間好有價值。
- **LED 閃爍提醒**：iOS [輔助功能] > [音頻/視覺] > [LED 閃爍以示提醒]，安卓 [設定] > [通知] > [閃光燈提醒]。來電同通知嗰陣閃光燈閃爍，**嘈雜環境無聲都可以感知**。
- **單聲道音頻**：[設定] > [輔助功能] > [音頻] > [單聲道音頻]，將左右聲道合併。單耳聽力受損嘅用戶唔會丟失另一半聲音資訊。

蘋果嘅無障礙設計哲學

蘋果喺無障礙領域投入巨大，每年都會更新同新增輔助功能。蘋果官網有專門嘅無障礙頁面，詳細介紹各項功能嘅使用方法。iOS 嘅無障礙功能通常比安卓更完善、體驗更流暢，呢個都係好多殘障用戶選擇蘋果嘅重要原因。

花 30 秒體驗一下

搵一段空閒時間，做以下體驗：

1. 開啟**旁白 / TalkBack**，試下唔靠眼用三指滑動瀏覽微信；
2. 打開**實時字幕**，對住手機說話睇識別準確率；
3. 用**三指點擊**放大一段文字到 200%，體驗視障用戶嘅日常；

4. 關掉之前所有功能，諗下「如果我永遠需要呢啲功能，我會希望佢點樣設計」。

第六章：守護你嘅數碼生活——安全與私隱

手機安全 = 私隱安全 + 財產安全。喺呢個數據即資產嘅時代，保護手機就係保護你自己。

一、鎖屏：第一道物理防線

鎖屏係你手機嘅第一道防線，都係最容易被忽視嘅。

安全等級排序：

1. **密碼/PIN 碼**：安全度最高，係最終保障（唔記指紋/臉嗰陣靠佢）。
2. **指紋識別**：安全與便利嘅完美平衡。
3. **人臉識別**：最方便，但安全性取決於技術（3D 結構光安全，2D 相片易呃過）。
4. **圖案解鎖**：方便但容易留低痕跡被猜到。

設定強鎖屏：

- 務必設定一個複雜密碼（唔好用生日、123456）。
- 開啟生物識別。
- 鎖屏時間設成「立即」或最短。

二、「查找我嘅手機」：終極防盜工具

必須開啟！佢可以：

- **定位**：手機丟喺邊，喺地圖上搵。
- **播放聲音**：等手機響鈴，適合喺屋企搵。
- **遺失模式**：遠程鎖死手機並顯示聯絡號碼。

- **遠程擦除**：最後一招，清除所有數據（但擦除後無法再定位）。

去 [設定] 入面，蘋果搵「查找我嘅 iPhone」，安卓搵「查找我嘅設備」或品牌類似服務，確保佢係綠色開啟狀態。

三、系統更新：打補丁，堵漏洞

系統更新首要目的係修復安全漏洞，就好似畀屋企補牆洞。所以，見到系統更新提示，盡量第一時間升級。

當然，大版本更新（比如 iOS 17 到 iOS 18）可能會有兼容性問題，可以觀望幾日再升。

四、識別與防範電信詐騙

喺 AI 技術迅猛發展嘅今日，傳統嘅「眼見為實」認知模式已完全失效。詐騙分子正利用深度偽造（Deepfake）等 AI 技術，實現咗由「廣撒網」式詐騙到「精準圍獵」式犯罪嘅全面升級。

AI 時代最常見嘅幾種詐騙套路

(1) AI 換臉擬聲詐騙

不法分子通過非法蒐集目標對象喺社交媒體發佈嘅高清相片或僅需幾秒鐘嘅語音樣本，利用 AI 深度偽造技術克隆出足以亂真嘅「親友影片」或「語音」。常見劇本包括：**突發車禍、緊急借款、遭遇綁架**等。呢類詐騙利用親情、友情等情感紐帶，**迷惑性極高**。

(2) 冒充權威 / 客服詐騙

AI 技術能夠精準模擬官方客服、公檢法機關人員嘅聲音同語調，甚至可以合成高度逼真嘅虛假轉賬截圖、電子公文、通緝令等檔案，通過**製造恐慌心理或權威壓力**，誘導受害者轉賬或開啟螢幕共享功能，從而竊取賬戶資訊。

(3) AI 投資理財詐騙

通過 AI 算法偽造「穩賺唔賠」嘅虛假收益截圖、虛擬量化選股軟件介面或「大師級」投資分析報告，以高額回報為誘餌，售賣高價投資服務或誘導受害者將資金轉入佢控制嘅交易賬戶，**最終捲款走人**。

破局之法——「照妖鏡」防護思路

AI 可以克隆聲音同面容，但都唔係無懈可擊，有幾個破綻可以利用：

- **動作驗證法**：進行視像通話嗰陣，主動要求對方完成預設嘅連貫動作指令，例如「用手喺臉前快速揮過」、「側臉展示耳朵輪廓」或「連續快速眨眼 5 次」。目前嘅 AI 換臉技術喺處理複雜動態遮擋、快速運動場景同光影變化嗰陣，**極容易出現畫面卡頓、面部扭曲或邊緣穿模**等破綻。

- **暗號核驗法**：建立並使用預設嘅私密核驗問題，例如「我哋上次一齊聚餐嗰陣你點嘅乜嘢菜」、「我送你嘅第一份禮物係乜嘢」等只有雙方知曉嘅細節。若對方出現明顯遲疑或答非所問，應**立即終止通話並啟動安全預案**。
- **二次核實法**：嚴格遵守公安部網安局發佈嘅「**四不一核實**」原則——唔輕信陌生資訊、唔隨意轉賬匯款、唔洩露個人敏感資訊、唔傳播可疑內容。涉及任何錢財往來嗰陣，務必通過**已知嘅原號碼回撥或當面確認**。進行轉賬操作嗰陣，建議選擇「24 小時到賬」模式，為潛在詐騙識別同資金挽回預留充足時間窗口。

傳統常見套路（仍要警惕）：

- **中獎/刷單返利**：利誘，先畀你啲甜頭再呃大錢。
- **網戀詐騙**：殺豬盤，培養感情後呃錢。
- **冒充熟人**：盜號後借錢（AI 時代更難分辨）。

手機自帶防護：

- 騷擾電話攔截。
- 驗證碼防竊取。
- 短訊連結安全檢測。

防騙口訣：

唔輕信、唔透露、唔點擊、唔轉賬、有疑問先核實！遇到「急事」先冷靜 5 分鐘，幾乎所有詐騙都會喺呢一刻露出馬腳。

五、抵禦「開盒」與社工攻擊：斬斷私隱洩露鏈條

「開盒」（社會工程學攻擊）嘅核心原理在於**系統性蒐集、分析並拼湊個人數碼碎片，構建完整嘅個人畫像**。防範嘅關鍵在於為個人私隱劃定明確紅線，建立多層次防護體系。

相片脫敏處理

基礎 EXIF 清理方法見第二章「圖片與檔案系統管理」。喺社交媒體分享相片前，仲需進一步**徹底刪除相片嘅元數據（EXIF 資訊）**，包括拍攝時間、設備型號、GPS 位置坐標等敏感數據。對相片中包含嘅**門牌號碼、車牌資訊、車票條形碼、身份證號**等關鍵資訊進行嚴格打碼或模糊處理，避免無意中洩露個人行蹤軌跡。

- iOS：[設定] > [私隱與安全性] > [相機] 中可關閉「精確位置」；分享嗰陣選「僅當前相片」可剝離元數據。
- 安卓：部分相簿 App（如 Google 相冊）分享嗰陣會提示係咪保留位置資訊。

- 通用工具：ExifTool、Metapho 等可查看/編輯 EXIF。

應用權限收斂

基礎權限管理方法詳見第四章「權限管理」。喺「防開盒」場景下，仲需進一步定期審查並關閉電商類、工具類 App 嘅**麥克風、通訊錄、位置資訊**等非必要權限，防止 App 喺後台「偷聽」日常對話或非法收集個人關係網絡；對長期閒置但仍佔用高權限嘅應用程式應及時卸載，由源頭減少私隱洩露風險點。

- 見到「通訊錄權限」申請嗰陣，先問一句：呢 App 真係有必要知道我所有聯絡人嗎？
- 三個月冇用過嘅 App，果斷卸載。權限係越用越緊嘅，唔係越用越鬆。

智能設備安全設防

智能攝像頭、智能門鎖、智能音箱等 IoT 設備喺初次使用嗰陣必須**立即修改預設登錄密碼**，設定包含大小寫字母、數字同特殊符號嘅高強度獨立密碼；唔常用嘅遠程訪問功能應保持關閉狀態，定期檢查設備固件更新並及時修補安全漏洞，**防止黑客入侵並窺探家庭私隱空間**。

> **真實案例警示**
>
> 2022 年某地警方通報，黑客通過破解某品牌家用攝像頭嘅預設密碼（好多用戶從未修改），竊取咗上千戶家庭嘅睡房畫面並喺網上兜售。**唔改預設密碼 = 將屋企大門敞開**。

六、私隱保護：你嘅數據係你嘅財產

手機入面嘅相片、聊天記錄、位置軌跡，本質上都係你嘅「數碼資產」。一旦洩露，輕則廣告騷擾，重則被人肉、詐騙甚至敲詐。

App 權限嘅「最小化原則」：

- **位置**：打車/地圖用「始終允許」；其他 App 用「僅使用時允許」。
- **通訊錄**：除非必要（如微信加好友）先畀，否則一律禁止。
- **相簿 / 全部相片**：iOS 14+ 起可授權「選定嘅相片」，安卓 13+ 同樣支援精細授權。
- **麥克風 / 攝像頭**：平時關掉，需要嗰陣再開啟。懷疑被偷錄嗰陣，滑出控制中心睇下有冇「小綠點/小橙點」。
- **後台跟蹤**：iOS 14.5+ 起強制要求 App 使用 IDFA 前必須詢問；安卓可關閉「廣告 ID」。

私隱管理實戰：

- **私隱報告（iOS）**：[設定] > [私隱與安全性] > [App 私隱報告]，查看邊啲 App 喺度偷偷訪問你嘅位置、相機、麥克風。

- **權限使用記錄（安卓）**：[設定] > [私隱] > [權限使用記錄]，睇下邊個喺乜嘢時候調用咗敏感權限。
- **限制廣告跟蹤**：喺系統設定入面關閉「廣告 ID」或「限制廣告追蹤」，能減少「被精準廣告騷擾」。

七、遺失 / 被盜後嘅「急救」清單

手機丟咗唔好慌，按以下順序處理：

1. **立即凍結 SIM 卡**：聯絡電訊商掛失，避免被用於接收驗證碼。
2. **遠程鎖定**：通過雲服務（iCloud / 品牌雲）開啟「遺失模式」。
3. **修改重要密碼**：登錄過手機嘅所有重要帳號（支付、銀行、郵箱）立即改密碼。
4. **報案**：去派出所報案，記錄 IMEI 號碼。手機搵返嗰陣都便於證明身份。
5. **通知親友**：防止騙子以你名義借錢。
6. **（最後）遠程擦除**：確認搵返無望嗰陣再操作，擦除後定位功能失效。

八、SIM 卡與 eSIM 安全

- **設定 SIM 卡 PIN**：開啟後，每次重啟手機或換卡都需要輸入 PIN 碼，防止「拔卡換手機」繞過鎖屏。
- **eSIM 優勢**：冇實體卡可被拔出，安全性更高；同時支援多號切換，旅行嗰陣非常方便。
- **二次換卡風險**：警惕「電訊商二次換卡」騙局——騙子冒用你嘅身份補辦 SIM 卡，從而接管你嘅手機號。設定「補卡需機主本人持身份證到營業廳辦理」係最有效嘅防線。

千萬唔好做嘅事

- 用生日 / 123456 / qwerty 等弱密碼；
- 將驗證碼發畀任何人（哪怕係「客服」）；
- 點擊短訊入面嘅不明連結，特別係 00 開頭嘅境外號碼；
- 喺公共 Wi-Fi 下登錄網銀或支付；
- 畀陌生人開啟「螢幕共享」或「遠程控制」。

試下啟用並模擬「遠程鎖定」

1. 確保你嘅「查找我嘅手機」功能已開啟。
2. 喺電腦瀏覽器或另一部手機上，登錄對應嘅雲服務帳號（iCloud.com 或品牌雲服務網站）。

3. 搵到「查找設備」功能，嘗試對你自己嘅手機發起一次「播放聲音」或模擬「鎖定」指令，熟悉整個操作流程。

第七章：前沿功能全解析

科技日新月異，手機功能都喺度不斷進化。衛星通信、無網通、車禍檢測、星閃技術……呢啲聽落好科幻嘅功能，其實已經走入我哋嘅生活。等我哋一齊嚟瞭解呢啲前沿技術。

一、衛星通信：冇信號都可以求救

衛星通信係近年嚟手機領域嘅一大突破。當你喺荒郊野外、深山老林，手機完全冇信號嗰陣，衛星通信可能就係救命稻草。

工作原理：

手機直接與軌道上嘅通信衛星建立連接，發送求救資訊。目前主要支援發送短訊同位置資訊，唔可以打電話。

支援機型：

- **蘋果**：iPhone 14 系列及以後，支援通過衛星發送緊急 SOS。iPhone 17 系列進一步支援衛星發送 iMessage 同短訊，且擴展咗更多地區嘅覆蓋。
- **華為**：Mate 50 系列開始支援北斗衛星消息（僅文字），Mate 60 Pro 系列進一步支援天通衛星通話（可打電話），Mate 70 系列衛星通信能力進一步增強。
- **小米/OPPO/vivo**：2024 年起旗艦機型陸續支援北斗衛星消息功能。
- **發展趨勢**：衛星通信正由高端旗艦專屬功能向下普及，未來中端機型都有望支援。中國北斗衛星系統喺國產手機上嘅支援最為廣泛。

使用場景：

- 戶外探險、登山遇險。
- 海上遇險。
- 自然災害導致地面通信中斷。

使用方法：

1. 確保喺開闊地帶，可以睇到天空。
2. 打開手機嘅衛星通信功能（通常喺緊急呼叫介面）。
3. 按照螢幕指引，對準衛星方向。
4. 發送預設嘅求救資訊同位置。

> **小知識**
>
> 衛星通信需要手機有特殊嘅硬件天線，唔係所有手機都支援。另外，衛星通信通常需要付費訂閱服務，但緊急求救功能一般係免費嘅。

二、無網通：近距離通信新方式

「無網通」係指喺冇流動網絡同 Wi-Fi 嘅情況下，兩台手機之間直接通信嘅技術。

技術原理：

利用藍牙或專用嘅無線電頻率，喺短距離內（通常幾百米到幾公里）建立點對點連接。

應用場景：

- **戶外活動：**徒步、露營嗰陣隊友間通信。
- **災難救援：**地震、洪水後通信中斷嗰陣。
- **大型活動：**演唱會、體育賽事人多信號差嗰陣。

支援平臺：

- **蘋果：**iOS 17 嘅「NameDrop」同「離線搵朋友」功能。
- **華為：**暢連 App 支援無網通話。
- **小米：**小米妙享中心嘅部分功能。

三、車禍檢測：關鍵時刻救命

車禍檢測係利用手機傳感器檢測嚴重車禍，並自動撥打緊急電話嘅功能。

工作原理：

手機通過加速度計、陀螺儀、氣壓計、麥克風等多個傳感器，綜合判斷係咪發生咗嚴重碰撞。檢測到車禍後，手機會：

1. 彈出倒計時介面，詢問你係咪冇事。
2. 如果倒計時結束你冇回應，自動撥打緊急電話。
3. 向緊急聯絡人發送你嘅位置。

支援機型：

- **蘋果**：iPhone 14 系列及以後。
- **安卓**：部分高端機型陸續支援。

注意事項：

- 功能需要開啟先可以使用。
- 可能會誤觸發（比如過山車、劇烈運動），及時取消即可。
- 依賴傳感器同算法，唔係 100% 可靠，但多一層保障。

四、星閃技術：下一代無線連接

星閃（SparkLink）係華為牽頭研發嘅新一代無線通信技術，目標係喺某些場景下替代藍牙同 Wi-Fi。佢喺關鍵指標上對標甚至超越現有方案：空口速率可達 900Mbps（遠超藍牙嘅 50Mbps），延遲低至 20 μ s（藍牙約 15ms），單網絡支援 4096 個設備同時連接（藍牙通常 8 個以內）。

技術特點：

- **超低延遲**：端到端延遲僅 20 微秒（0.02ms），比藍牙低兩個數量級，遊戲同音樂同步幾乎無感知。
- **高速率**：單鏈路峯值速率 900Mbps，係藍牙音頻嘅數十倍，可傳輸無損音頻甚至高清影片。
- **低功耗**：待機功耗與藍牙相當，但傳輸效率更高，單位數據嘅能耗更低。
- **廣覆蓋 + 高並發**：連接距離比藍牙遠 50% 以上，單網絡支援上千設備同時在線。

應用場景：

- **無線耳機：**更低延遲，玩遊戲音畫同步。
- **智能手錶：**更穩定嘅連接。
- **智能家居：**更多設備同時連接。
- **車載互聯：**更流暢嘅手機-車機連接。

支援設備：

目前主要係華為生態設備，如華為 Mate 60 系列、部分華為耳機同手錶。未來會有更多品牌支援。

技術展望

星閃技術代表咗無線通信嘅未來方向。隨住更多廠商加入，我哋可能會見到更多支援星閃嘅設備，享受更好嘅無線體驗。

去檢查一下：你嘅手機係咪支援呢啲功能

1. 去設定入面搜尋「衛星」，睇下有冇衛星通信相關選項。
2. 搜尋「車禍檢測」或「緊急 SOS」，檢查係咪開啟。
3. 如果你嘅手機係華為，睇下係咪支援星閃設備連接。
4. 瞭解呢啲功能嘅使用方法，關鍵時刻可能派上用場。

二、軟件篇（第 8-14 章）

第八章：通訊：短訊、電話與社交

通訊係手機最原始、最核心嘅使命。而家，佢已經由簡單嘅通話同短訊，演變成一個包含即時通訊、社交網絡同電郵在內嘅複雜生態系統。

一、電話 App 嘅進階功能

現代手機嘅電話應用，早已經超越咗一個簡單嘅撥號盤，佢整合咗錄音、騷擾攔截、智能識別等強大工具。

通話錄音（留意法律合規性）：

- 作用：記錄重要通話內容，用嚟做工作紀要、訪問、客戶承諾或者爭議留存。
- 法律提示：喺中國境內，通話錄音作為證據使用嘅前提係取得對方同意，或者用嚟維護自身合法權益。
- 單方錄音 vs 雙方同意：法律層面，「未經對方同意」取得嘅錄音並非當然無效。最高人民法院相關司法解釋中，只要錄音取得方式未有嚴重侵害他人合法權益、未有違反法律禁止性規定、未有違背公序良俗，仍然可以作為證據提交。但部分省份法院喺民事審判中對「偷偷錄音」嘅採信標準更嚴格；刑事案件就通常要求更嚴格嘅合法性審查。
- 各地差異：北京、上海、廣東等地法院對單方錄音作為民事證據較為寬容；而涉及商業秘密、個人私隱嘅錄音，無論喺邊個省份都要特別謹慎，可能因為侵害私隱權而被排除。
- 點樣操作：Android 通常喺通話介面有明確嘅「錄音」按鈕；iOS 系統唔提供原生通話錄音功能，要借助第三方 App 或者使用有錄音功能嘅藍牙耳機。

錄音使用紅線

- 唔可以將錄音用嚟勒索、傳播或者公開侮辱他人。
- 涉及第三方私隱（例如偶然錄到其他人對話）嘅內容唔可以隨便擴散。
- 重要場景建議喺通話開始時口頭告知「今次通話將會被錄音」，咁樣又合規又更具證據效力。

騷擾電話治理深入內容：

- 原理：電話 App 透過網絡查詢龐大嘅號碼庫，喺來電時顯示「騷擾電話」、「詐騙電話」或者公司名稱。
- 點樣開啟：Android 入電話 App > 設定 > 騷擾攔截；iOS 入設定 > 電話 > 來電阻止與身份識別。各品牌詳細開關路徑同推薦開啟項目，見第十三章「實用系統工具」入面嘅品牌對照表。
- 主流標記平臺：國內號碼標記主要由三家數據服務商提供——騰訊手機管家、360 手機衛士、電話邦（Tapir）。電信商（移動/聯通/電信）嘅「機主姓名」顯示、「騷擾攔截」功能亦都經常調用呢啲數據。手機廠商（華為、小米、OPPO、vivo）嘅標記庫多數同上述服務商合作。
- 被誤標記點算好：如果你正常工作號碼被錯誤標記做「外賣」「快遞」「騷擾電話」，可以透過以下渠道申訴：電話邦：訪問號碼申訴平臺（haoma.tk.cn）提交身份證明。
- 360：透過 360 手機衛士 App 入面「號碼申訴」功能提交。
- 騰訊：透過「騰訊手機管家」小程式「號碼申訴」頁面提交。
- 三大電信商：中國移動用戶可以發短訊「KTFSR」去 10086 開通防騷擾功能；電信用戶關注「天翼防騷擾」公眾號；聯通用戶喺「聯通手機管家」入面設定。

防騷擾進階：開啟電信商嘅「高頻呼叫攔截」（例如移動嘅高頻呼叫限制），可以自動攔截 AI 語音外呼。

手機自帶攔截常見能力：來電識別 / 智能攔截：按號碼庫自動標記並攔截騷擾、詐騙、廣告推銷。

黑名單 / 白名單：手動拉黑已知騷擾號；將通訊錄、快遞外賣號加入白名單，避免誤殺。

短訊關鍵詞過濾：包含「中獎」「貸款」「點擊連結」等關鍵詞嘅短訊會自動歸入攔截箱。

iOS 靜音未知來電：唔喺通訊錄入面嘅來電只會靜音轉去語音信箱，唔會打斷你；適合騷擾電話多嘅用戶，但要留意漏接陌生重要電話。

勿擾模式聯動：夜晚或者開會時只允許通訊錄來電，配合定時開關，睡眠質素即刻改善。

實用軟件點樣配合：優先用系統自帶攔截，再疊加國家反詐中心同電信商防騷擾；清理類可以揀清濁等輕量工具，但同一類安全/清理軟件裝一套就可以，詳見第十三章。

VoLTE/VoNR 係乜嘢同點樣開啟：

- VoLTE（Voice over LTE）：基於 4G 網絡嘅語音通話技術，通話時唔再回落去 2G/3G，可以一邊打電話一邊上網，音質比傳統通話更清晰，接通更快。
- VoNR（Voice over NR）：基於 5G 嘅語音通話，係 VoLTE 嘅下一代演進。延遲更低、音質更好，仲可以支援高清視像通話。要 5G 手機 + 5G 套餐 + 當地 5G 覆蓋三者具備先至用得到。

- 開啟方法：iOS 喺「設定 > 流動網絡 > 流動數據選項 > 語音與數據」入面開啟「VoLTE」；Android 喺「設定 > 移動網絡 > 移動數據 > 進階」搵到「VoLTE 高清通話」開關。電信商通常預設開啟，如果發現通話時網絡斷開、唔能夠上網，多數係 VoLTE 未啟用。

小知識

如果你嘅手機喺通話時上網速度突然變慢或者斷網，多數係 VoLTE/VoNR 未開啟，回落咗去 2G/3G 網絡。開啟之後就可以做到「一邊打電話一邊刷片」。

二、短訊同 iMessage/RCS

傳統短訊並冇被淘汰，佢仍然係接收驗證碼、銀行通知同官方信息嘅可靠渠道。

- iMessage：蘋果裝置之間透過互聯網發送嘅訊息，顯示做藍色氣泡。支援高質素圖片影片、已讀回執等，完全免費而且端到端加密。
- RCS：由全球電信商同 Google 推動嘅下一代通訊協議。功能類似 iMessage 或者微信，但係基於電話號碼。
- iMessage 同 RCS 嘅互操作性：由 iOS 18 開始，蘋果開始支援 RCS 標準，即係話 iPhone 同 Android 手機之間可以透過 RCS 發送高清圖片、已讀回執同打字狀態提示，告別「綠色氣泡」嘅體驗斷層。但前提係雙方電信商都開通 RCS，而且手機系統版本足夠新。

RCS 喺國內嘅部署情況：

- 中國移動：較早部署 RCS，部分省份已經支援「5G 消息」業務，可以發送富媒體消息、互動卡片。要手機終端支援而且喺營業廳開通。
- 中國電信：透過「電信 5G 消息」提供服務，部分套餐預設支援，舊用戶要發指定短訊指令開通。
- 中國聯通：喺主要城市部署 5G 消息，但終端覆蓋參差，好多 Android 機型仍然要手動喺短訊 App 入面開啟「增強信息」或者「5G 消息」開關。
- 實際體驗：國內 RCS 推廣速度慢過預期，主要受微信生態強勢、終端適配唔統一等因素影響。日常通訊仍然以微信為主，RCS 更多承擔「官方推送」角色（例如銀行通知、政務消息）。

驗證碼安全紅線

- 唔好轉發驗證碼：任何叫你「讀出驗證碼」或者「轉發短訊」嘅人都可能係詐騙分子，包括自稱「客服」「快遞員」「公檢法」嘅人。驗證碼 = 你個錢袋。
- 防釣魚短訊：警惕「ETC 過期」「醫保卡凍結」「快遞丟失理賠」等短訊入面嘅連結，正規機構唔會透過短訊連結索取銀行卡號、密碼。
- 號碼識別：銀行官方短訊通常來自 95 開頭短號或者 1069+ 企業簽名，凡係由普通 11

位手機號發嚟嘅「銀行通知」基本上都係詐騙。

- 雙因子備份：綁定重要帳號嘅手機號唔好隨便更換；如果一定要換號，務必先解除綁定所有關鍵帳號，否則新機主可能透過短訊驗證碼重設你嘅密碼。

三、微信/QQ：國民級社交應用嘅管理

微信同 QQ 已經超越通訊工具，成為集社交、支付、資訊獲取於一身嘅「超級 App」。

聊天記錄備份同遷移：

1. 喺電腦上登入 PC 版或者 Mac 版微信。
2. 點擊左下角選單 > [遷移與備份] > [備份與還原]。
3. 揀「備份聊天記錄至電腦」，喺手機上揀需要備份嘅聊天。
4. 遷移到新手機：喺原手機微信 [我] > [設定] > [一般] > [聊天記錄遷移與備份] > [遷移] > [遷移到手機/平板]，掃碼就可以完成。

儲存空間清理深入方法：

- 常規入口：微信 [我] > [設定] > [一般] > [儲存空間]，可以清理緩存同按時間、檔案類型精細化清理聊天記錄。
- 聊天內清理：入某個聊天 > 右上角「…」> [聊天記錄] > [管理儲存空間]，可以單獨清理某聊天入面嘅圖片、影片、檔案，保留文字。
- 原圖/原片清理：群組入面接收嘅高清原圖原片最佔空間，可以喺 [管理儲存空間] 入面剔選「原圖」批量刪除。
- 關閉自動下載：[設定] > [一般] > [相片、影片、檔案和通話]，關閉「自動下載」，避免喺 WiFi 下默默緩存大量群組檔案。
- 小程式緩存：喺 [發現] > [小程式] 入面長按唔常用嘅小程式刪除，可以釋放佢嘅本地緩存。
- 聊天記錄壓縮備份：電腦版微信備份時剔選「壓縮」，可以將幾 GB 嘅聊天記錄壓縮到原體積嘅 30%-50%。

群組管理技巧：

- 群組公告：群主/管理員喺群聊右上角「…」> [群組公告] 入面發佈，全員強提醒，適合重要通知。新成員入群時自動彈出。
- 群組待辦：群主可以將置頂訊息設做「群組待辦」，所有成員頭頂會出現紅色提示，完成點擊後自動消失。適合活動報名、任務收集。

- 群組接龍：喺輸入框輸入「# 接龍」可以發起接龍，適合點餐、統計報名。接龍結果自動匯總到群聊頂部，避免洗版。
- 群組直播：群聊右上角「+」> [群組直播]，可以發起只供本群觀看嘅直播，適合企業內訓、家族聚會、產品內測。
- 摺疊該群聊：喺群組列表向左掃群名 > [摺疊]，可以將唔重要嘅群收納到「摺疊嘅群聊」入面，只會喺新 @ 你時提醒。
- 只限群主/管理員 @ 所有人：群成員 500 人以上時，普通成員唔能夠 @ 所有人，避免被打擾。

朋友圈權限精細化管理：

- 分組可見性：發佈時點擊「邊個可以睇」，揀「部分可見」或者「唔畀邊個睇」，基於預先設定嘅標籤分組（例如「同事」「家人」「客戶」）精準推送。
- 允許朋友查看範圍：[我] > [設定] > [朋友權限] > [朋友圈] > [允許朋友查看朋友圈嘅範圍]，可以揀「最近三日/一個月/半年/全部」，建議設做「半年」以平衡私隱同社交。
- 封鎖某條朋友圈：長按朋友圈某條動態 > [唔睇佢嘅朋友圈] 或者 [唔再關注]，對方唔會知道。
- 朋友圈廣告管理：[我] > [設定] > [關於微信] > [個人資訊與權限] > [個性化廣告管理]，可以關閉基於行為嘅個性化推薦。
- 陌生人十條：[朋友權限] > [朋友圈] > [允許陌生人查看十條朋友圈]，關閉可以避免陌生人睇你嘅過往動態。

四、視像通話對比

唔同視像通話工具各有側重，根據場景揀合適嘅工具可以令溝通更順暢。

- 微信視像：國民級覆蓋，老少咸宜。適合親友日常聯繫、快速溝通。最多支援 20 人視像（要小程式「群組視像」），缺點係網絡差時畫質下降明顯。
- FaceTime：蘋果生態專屬，畫質優秀、端到端加密。iOS 15 起支援「連結」邀請 Android/Web 用戶加入，但主持人必須係蘋果裝置。適合家庭跨裝置溝通。
- Zoom：國際會議首選，支援最多 100 人免費 40 分鐘會議。螢幕共享、虛擬背景、分組討論室等功能完善，適合跨國/遙距協作。
- 騰訊會議：國內辦公首選，同微信深度整合，掃碼即入。支援 300 人會議（付費版），雲端錄製、即時字幕、會議紀要功能強大，適合企業內訓同大型會議。
- 使用場景建議：1 對 1 日常聯繫揀微信；蘋果用戶之間揀 FaceTime；正式商務會議揀 Zoom 或者騰訊會議（國內揀騰訊，跨境揀 Zoom）。

網絡帶寬要求同流量消耗參考：

- 微信視像通話：大約需要 500Kbps-1Mbps 帶寬，標準畫質每分鐘大約消耗 3-5MB 流量，高清大約 8-12MB。
- FaceTime：標清大約 1Mbps，720p 大約 2Mbps，1080p 要 4Mbps 以上。每分鐘流量大約 5-15MB。
- Zoom/騰訊會議：720p 大約 1.5-3Mbps，1080p 要 3-5Mbps。每小時流量大約 300-800MB，多人會議主持人流量更大。
- 建議：長時間視像通話盡量用 WiFi；流動網絡下可以主動降低畫質（喺 App 設定入面關閉「高清視像」）以節省流量。

視像通話畫質優化技巧：

- 光線：背對窗、面向光源（自然光最佳），避免逆光導致塊面變黑。
- 角度：手機鏡頭略高過眼睛，避免雙下巴；橫屏通話時雙手揸機更穩定。
- 音訊：用耳機可以避免回聲同嘯叫；安靜環境下唔好開「揚聲器」模式。
- 網絡：優先 WiFi；4G/5G 視像時盡量靠近窗戶避免訊號衰減；多人會議可以關閉其他視像畫面只保留發言人。
- 虛化背景：騰訊會議、Zoom 都支援 AI 虛化，可以遮蔽雜亂背景。留意著純色衣物，避免同背景色相近導致「綠幕穿幫」。

小貼士

網絡卡頓時，先關閉視像保留語音，比直接掛斷更有禮貌。重要會議前可以用「騰訊會議」自帶嘅「網絡檢測」功能提前測試帶寬。

五、電郵：喺手機上配置同使用

- 加入電郵帳戶：iOS 喺設定 > 電郵 > 帳戶 > 加入帳戶；Android 通常有「電郵」App。
- 使用技巧：為唔同電郵帳戶設定唔同嘅通知方式；善用掃手勢快速歸檔、刪除或者標記電郵。

IMAP vs POP3 vs Exchange 協議區別：

- POP3：由伺服器下載電郵後預設刪除原件，只係本地保留。適合只用一部裝置、需要離線閱讀嘅場景。缺點係換裝置之後睇唔到歷史電郵，已經唔推薦。
- IMAP：電郵始終保存喺伺服器，本地只係做緩存。多裝置（手機/電腦/平板）同步狀態一致——一邊已讀，全部已讀。係目前主流推薦協議。

- Exchange（含 ActiveSync）：微軟協議，除咗電郵之外仲同步日曆、聯絡人、任務。企業電郵首選。Outlook、網易企業電郵、騰訊企業電郵都支援。
- 揀選建議：個人電郵用 IMAP；公司電郵優先 Exchange，其次 IMAP；盡量唔好用 POP3，除非有特殊歸檔需求。

企業電郵配置方法：

1. iOS：[設定] > [電郵] > [帳戶] > [加入帳戶] > 揀「Microsoft Exchange」或者「其他」。
2. Android：喺自帶嘅「電郵」App 或者 Outlook App 入面揀「加入帳戶」，輸入完整電郵地址同密碼。
3. 伺服器填寫（以 Exchange 為例）：伺服器地址通常係「mail. 公司域名.com」或者「outlook.office365.com」，域名填公司簡稱。
4. 如果提示「無法驗證伺服器」，可以手動輸入：傳入伺服器（IMAP）埠 993，加密 SSL；傳出伺服器（SMTP）埠 465 或者 587，加密 TLS。
5. 登入之後建議喺 [帳戶設定] 入面開啟「推送」（Push）以即時接收電郵，並關閉「電郵預覽」以保護商業機密。

小提示

企業電郵通常要管理員開通「用戶端授權密碼」或者「IMAP/SMTP 授權碼」，唔可以直接用網頁登入密碼。登入失敗時先去網頁版電郵設定入面產生授權碼再填入手機。

實操：備份一次微信聊天記錄到電腦

1. 喺電腦上啟動備份：打開電腦版微信 > 左下角選單 > [遷移與備份] > [備份與還原]。
2. 揀備份到電腦：點擊「備份聊天記錄至電腦」。
3. 喺手機上揀備份內容，等待備份完成。

第九章：流動辦公與學習

我以前都覺得手機上辦公就係「求其一下」，直到有一次出差唔記帶電腦，硬係用手機完成咗標書修改同合同簽署。講真，手機辦公嘅瓶頸往往唔喺手機本身，而係你唔熟悉佢嘅能力邊界。

一、文檔處理：喺掌中編輯與創作

喺手機螢幕上編輯文檔，同電腦上感覺確實唔一樣。最大嘅問題係螢幕細、手指粗、排版容易跑偏。但有幾個技巧可以等體驗好唔少：

- **WPS Office**：本土化做得最好，完全兼容微軟格式（docx/xlsx/pptx），內置嘅 PDF 工具集好豐富。我特別鍾意佢嘅「語音備註」功能——睇合同嗰陣唔方便打字，直接對住手機講幾句意見，佢就自動轉成文字批註貼喺文檔入面。另外 WPS 內置咗海量免費模板，做簡歷、做匯報 PPT 嗰陣可以慳好多時間。
- **Microsoft 365**：如果你公司用正版 Office，呢個就係最佳選擇。同桌面版嘅格式兼容性幾乎完美，唔會出現 WPS 打開某啲複雜排版會輕微移位嘅情況。深度集成 OneDrive，你喺電腦上改到一半嘅文檔，手機上打開就係最新版本。
- **石墨文檔/騰訊文檔**：呢兩個係「在線協同」嘅代表。適合多人同時編輯一份文檔，每個人改嘅地方實時顯示。我用石墨文檔同朋友一齊做旅行攻略，兩個人同時往表格入面填資訊，效率比一個人弄高得多。

實用工作流：我而家嘅習慣係——重要檔案存 OneDrive，日常編輯用 WPS，團隊協作開石墨文檔。檔案自從上咗雲之後，再也冇出現過「唔記帶 USB 手指」嘅尷尬。

二、筆記應用：由隨手記到知識體系

記筆記呢件事，好多人一上嚟就諗住搞一套好複雜嘅系統，結果堅持唔到三日。我嘅建議係先輕後重——先養成「記」嘅習慣，再考慮「管理」嘅問題。

- **隨手記——系統自帶就夠**：iOS 備忘錄啟動極快，而且支援掃描文檔、手寫、錄音。安卓各品牌嘅便籤都唔差，鎖屏狀態下就可以快速啟動。呢啲工具適合記臨時想法、購物清單、會議要點。關鍵係啟動快，諗到就可以記，唔使等 App 加載。

- **稍後讀——收集資訊**：微信嘅「浮窗」功能其實係個隱藏神器。見到好文章來不及細看，點右上角「浮窗」，佢就會縮成一個小圓點貼喺螢幕邊緣，隨時可以點開繼續睇。比轉發到「檔案傳輸助手」方便得多。另外，Cubox 或 Pocket 呢類稍後讀工具，可以跨平臺收集你喺任何地方見到嘅好內容。

- **搭建體系——Notion 同飛書**：當你覺得零散嘅筆記開始唔好搵嗰陣，就可以上 Notion 喇。呢玩意真係好靈活——你可以建一個「讀書筆記數據庫」，自動按書名、作者、評分嚟組織；都可以搭一個「項目看板」，將工作任務嘅進度可視化。飛書嘅「多維表格」都類似，而且國內訪問速度更快。我用 Notion 管理我所有嘅寫作素材、學習筆記同項目進度，已經用咗三年幾。

一個實用嘅筆記策略：我自己嘅習慣係「三件套」——臨時想法用系統便籤（秒開），待讀文章用微信浮窗（順手），需要長期積累嘅知識放 Notion（體系化）。唔好一上嚟就諗住將所有嘢塞入一個工具入面，咁樣反而容易放棄。

三、掃描與 OCR：將紙質世界搬入手機

手機掃描呢件事，用好咗真係可以慳返好多跑印刷舖嘅麻煩。我上次辦簽證需要一堆材料嘅電子版，全部都係用手機搞掂嘅。

- **iPhone 用戶**：iOS 備忘錄自帶嘅掃描功能已經好好。打開備忘錄點相機圖標選「掃描文稿」，對準檔案佢會自動識別邊框並矯正透視。拍完之後可以調顏色——選「黑白」模式出嚟嘅效果最接近掃描儀，「灰度」適合有圖片嘅資料。

- **安卓用戶**：好多品牌嘅相機自帶「文檔掃描」模式。如果搵唔到，去應用商店下載 Microsoft Lens（免費，而且同 OneDrive 打通）。

- **OCR 文字識別**：拍完嘅掃描件，如果需要將上面嘅文字提取出嚟編輯，就需要 OCR 喇。微信自帶嘅「提取文字」功能最簡單——長按圖片就可以用，識別準確率都唔錯。如果大量處理，可以用白描 App 或者全能掃描王。對於中文識別，百度嘅 OCR 引擎效果比較好；WPS 嘅圖片轉文字功能都很準。

- **掃描小技巧**：放喺光線充足嘅地方拍，最好有枱燈由側面打光，避免喺文檔上留低你嘅手影。如果檔案折角咗，先用手壓平再拍。多頁檔案可以連續掃描後合併成一個 PDF，唔使一頁頁單獨存。

四、網盤點樣揀：冇最好嘅，淨係有最適合你嘅

揀網盤呢件事我琢磨咗好耐，最後發現其實係睇你嘅「主力生態」係乜嘢。

- **蘋果全家桶 iCloud**：無縫同步到你感覺唔到佢嘅存在。你喺手機上影嘅相，幾分鐘後電腦上就自動有。而且蘋果嘅私隱保護做得最嚴格。但缺點都好明顯：跨平臺分享畀安卓或 Windows 用戶比較麻煩，免費淨係得 5GB，擴容要使錢。

- **Windows 主力 OneDrive**：如果你用嘅係 Windows 電腦 +Office 365，OneDrive 就係天生一對。Word 入面儲存文檔，自動就存到雲上。1TB 空間配合 Office 訂閱好劃算。手機端 App 嘅自動備份相片功能都好好用。
- **國內分享百度網盤**：講真，百度網盤嘅下載限速一直畀人彈，但佢嘅分享生態確實無人能敵。中文互聯網上大量嘅學習資料、影視資源都係用百度網盤分享嘅。如果淨係偶爾下載別人分享嘅檔案，唔充會員都可以湊合用。如果你係重度嘅資源收集者，會員該開就開啦，慳返嘅時間值錢得多。
- **阿里雲盤/夸克網盤**：呢兩年係後起之秀。阿里雲盤上傳下載唔限速（目前係），夸克網盤同夸克瀏覽器深度綁定，搜尋資料嗰陣可以直接儲存到網盤，好方便。

我嘅組合方案：iCloud 管個人相片同日常檔案（私隱優先），百度網盤存資源同接收分享（生態優先），OneDrive 管工作文檔（Office 集成優先）。三個網盤各司其職，互不幹擾。

五、一個真實嘅流動辦公場景

講個我自己嘅真實經歷啦。有一次出差到外地，客戶臨時要改合同條款。我嘅工作流程係咁樣嘅：

1. 喺酒店收到客戶微信發嚟嘅修改意見（文字消息 + 一張手寫批註嘅相片）。
2. 打開 OneDrive 搵到合同原件，用 WPS 打開編輯，參照客戶嘅微信消息逐條修改條款。
3. 改完後用 WPS 嘅「輸出為 PDF」功能導出，發到客戶微信羣入面確認。
4. 客戶提咗第二輪意見，我用 WPS 嘅畫筆工具直接喺 PDF 上圈出修改位置，配合語音批註講清楚改動思路。
5. 確認無誤後，用 WPS 嘅「電子簽名」功能在線簽署，發畀對方同樣簽署。
6. 最後將簽好嘅合同 PDF 存檔到百度網盤同 iCloud 各一份，同時喺 Notion 嘅工作日誌入面記咗一筆。

整個過程由開始到結束大概四十分鐘，全程冇開電腦。講真，三年前我根本唔信手機可以做到呢啲嘢，但而家確實可以。關鍵係提前熟悉呢啲工具，唔好等到需要嗰陣先手忙腳亂咁下載註冊。

嚟試下：搭建你嘅流動辦公工作流

1. 選定你嘅主力文檔 App（WPS 或 Microsoft 365），登錄並關聯雲盤。
2. 打開備忘錄或便籤，用掃描功能拍一份檔案，導出為 PDF 發畀自己。
3. 揀一個網盤 App，設為手機相片自動備份（選「僅 WiFi 下備份」慳流量）。
4. 喺 Notion 或飛書入面新建一頁，試下搭建一個簡單嘅「工作日誌」模板。

第十章：流動支付與生活服務

講真，我爸媽嗰一輩人對「手機畀錢」呢件事，一開始係抗拒嘅。「錢喺手機入面，被人偷咗點算？」呢個擔心好正常，但現實係：你褸袋入面揣住嘅現金，遺失嘅概率比手機支付被盜刷嘅概率高得多。手機支付唔係將錢存入手機，佢淨係一把鑰匙，一把打開你銀行賬戶嘅鑰匙。

一、到底點樣綁定銀行卡？

唔管你係用支付寶定微信，第一次綁卡都要行呢幾步：

1. 打開 App，搵到「我嘅」或「錢包」。點「添加銀行卡」。
2. 輸入卡號。而家大部分 App 都支援直接影相識別卡號，唔使一個個數字敲。
3. 銀行會發一條驗證碼短訊到你預留嘅手機號上。填入去，設定一個 6 位數字嘅支付密碼。
4. 呢個支付密碼同你嘅銀行卡密碼、鎖屏密碼，最好係三個唔同嘅密碼，唔好圖省事設成一樣。

有個小竅門：如果擔心安全問題，可以去銀行 App 入面開一張 II 類或 III 類電子賬戶，專門用嚟綁定手機支付。呢張卡入面淨係放平時零花嘅錢，就算出咗事，損失都有限。

二、付款碼係「邊個睇邊個收錢」

好多人搞唔清楚付款碼同收款碼嘅區別，呢個區別好重要。

- **付款碼**：你展示畀收銀員掃嗰個條形碼 + 二維碼。呢個碼相當於你嘅「電子銀包」，邊個掃咗邊個就可以由你賬上掃走錢。所以**千萬唔好截圖發畀任何人**。呢個碼每分鐘自動刷新一次，就係為咗防止被盜用。
- **收款碼**：別人掃咗呢個碼畀你轉賬。呢個碼可以截圖保存，可以發畀別人，安全。

認準呢個區別，可以避開好多詐騙陷阱。凡係有人等你「將付款碼截圖發過嚟」嘅，百分之百係騙子。

三、呢啲安全設定，花 5 分鐘設好可以慳好多心

- **指紋/面容支付**：喺支付寶同微信嘅支付設定入面，將生物支付打開。咁樣每次付款要按指紋或掃臉，就算手機丟咗，執到嘅人都冇法使你嘅錢。

- **安全鎖**：支付寶同微信都有「安全鎖」功能。打開之後，進入 App 嘅「錢包」頁面需要額外驗證一次身份。路徑：支付寶我嘅設定帳號與安全安全中心安全鎖。

- **免密支付檢查**：好多人簽約咗各種免密支付，自己都唔記得。去支付寶同微信嘅「支付設定」入面，搵到「免密支付/自動扣款」，睇下簽約咗邊啲服務，將唔需要嘅關掉。

- **登錄設備管理**：定期睇下邊啲設備登錄過你嘅帳號。如果發現唔識嘅設備，即刻踢走。路徑：支付寶設定帳號與安全登錄設備管理。

四、公交地鐵：手機就係交通卡

而家出門坐公交地鐵，已經好少人用實體嘅交通卡喇。手機刷閘機，有兩條路可以行：

- **手機錢包入面嘅虛擬交通卡**：喺蘋果錢包、華為錢包、小米錢包入面直接開通一張虛擬交通卡。呢個方案嘅好處係：刷嗰陣唔使解鎖手機，直接靠上去就得，響應速度最快，而且冇網都可以刷。部分城市嘅虛擬交通卡仲有同實體卡一樣嘅換乘優惠。

- **支付寶/微信嘅乘車碼**：喺 App 入面搜尋「乘車碼」，開通你所在城市嘅公交或地鐵乘車碼。呢個方案嘅好處係：唔使充值，直接由餘額或綁定嘅銀行卡入面扣。但需要聯網，而且刷之前要打開 App 搵到碼。

我個人更推薦用手機錢包嘅虛擬交通卡。雖然第一次要充個 10 蚊 20 蚊入去，但用起嚟真係方便太多，尤其係趕地鐵嗰陣，唔使停下嚟解鎖手機搵 App。

五、生活繳費、外賣、打車

- **水電氣繳費**：喺支付寶或微信嘅「生活繳費」入面，將你屋企嘅水、電、燃氣戶號綁定好。以後每個月收到賬單，直接點入去就可以交。戶號一般喺你屋企嘅紙質賬單上可以搵到，或者打客服電話問。

- **外賣比價**：同一間舖喺美團同餓了麼上嘅價格同優惠經常唔一樣。落單前兩邊都睇一眼，有時候可以慳幾蚊。另外，美團嗰個「拼好飯」功能，同朋友一齊點，價格可以平唔少。

- **打車安全**：喺滴滴或者高德打車入面，記得將「緊急聯絡人」同「行程分享」設定好。打車嗰陣，App 會自動將你嘅行程分享畀你設定嘅聯絡人。另外，深夜打車優先選「的士」而唔係「快車」，的士有行業監管，安全系數更高啲。

六、用手機搞掂醫保、社保、政務

呢幾年嘅變化好大，以前要跑大廳排隊嘅好多事，而家手機上幾分鐘就辦完。

- **醫保電子憑證**：喺支付寶或微信入面搜「醫保電子憑證」，激活之後，去醫院睇病、藥房買藥直接刷手機就得，唔使帶實體醫保卡。第一次需要人臉識別驗證，之後每次用嗰陣會生成一個動態碼。
- **社保查詢**：下載「掌上 12333」App 或者用支付寶入面嘅「市民中心」，可以查社保繳納記錄、養老金賬戶餘額、失業金申領等。
- **個稅**：每年 3 月到 6 月嘅個稅匯算清繳，喺「個人所得稅」App 上就可以搞掂。記得睇下有冇可以申報嘅專項附加扣除（子女教育、房貸利息、贍養老人等），呢啲都可以幫你少交稅或者退稅。

記住呢幾條，防騙保平安

- 公檢法、銀行、客服，唔會等你轉賬到「安全賬戶」。
- 任何人等你將付款碼截圖發過去，都係騙子。
- 陌生連結唔好點，尤其係「ETC 過期」「社保卡停用」呢類短訊。
- 網購退款會原路返回，唔需要你提供銀行卡號。

而家就做：畀你嘅支付 App 做一次安全體檢

1. 打開支付寶同微信，確認已開啟指紋/面容支付。
2. 搵到「安全鎖」功能並打開。
3. 檢查「免密支付/自動扣款」列表，關掉唔再需要嘅服務。
4. 查看「登錄設備」列表，移除唔識嘅設備。
5. 激活你嘅醫保電子憑證，下次去醫院就唔使帶實體卡喇。

第十一章：攝影與圖片管理

講句真話，多數人用手機影相，影出嚟嘅效果遠遠唔及手機本身嘅潛力。唔係你技術唔得，而係好多小設定同小習慣你冇注意到。呢一章唔講虛嘅，就講嗰啲你可以即刻用得上嘅嘢。

一、你係咪都犯過呢啲錯？

先講最常見嘅幾個問題，你睇下中咗幾條：

- **鏡頭係污糟嘅**：呢個真係最普遍嘅問題。攞手機嗰陣指紋按喺鏡頭上，影出嚟霧濛濛嘅，仲以為手機壞咗。養成習慣，影相前用衫角抹一下鏡頭。
- **從來唔調曝光**：預設模式下，點一下螢幕係對焦，但如果你喺點完之後上下滑動，係可以調節亮度嘅。好多人影嘅相要麼太暗要麼太亮，就係唔知呢個操作。
- **閃光燈永遠開住**：手機 LED 閃光燈影出嚟嘅效果真係好差，光線又硬又冷，將人影到好嚇人。除非真係伸手不見五指，否則關掉佢。暗光環境下，用夜景模式遠比閃光燈效果好。
- **放大靠兩根手指拉伸**：喺螢幕上雙指縮放係數碼變焦，本質就係裁切畫面然後放大，畫質會明顯下降。如果想影遠處嘅，直接用取景框上嘅變焦按鈕（0.5x/1x/2x/3x）。其中 0.5x（超廣角）同 1x（主攝）係原生光學鏡頭，唔損失畫質；2x/3x 則多為「光學品質變焦」——通過高像素傳感器裁切實現，畫質接近光學變焦但並非獨立鏡頭。超過 3x 嘅變焦通常就係混合變焦或數碼變焦喇，畫質會有明顯下降。

二、幾個即刻提升出片率嘅小技巧

呢啲唔需要乜嘢理論知識，用一次就可以睇到效果：

- **三分法構圖 ＋ 網格線**：喺相機設定入面打開「網格線」（又叫參考線）。影相嗰陣將你想突出嘅嘢放喺四條線嘅交叉點上，畫面即刻就會「對」好多。呢個係攝影入面最簡單都最有效嘅一個規則，冇乜玄學，就係符合人眼嘅視覺習慣。
- **搵一搵「引導線」**：影風景或者街景嗰陣，留意一下畫面入面有冇自然嘅線條——馬路、欄杆、牆壁、一行樹——等呢啲線條由畫面邊緣延伸到主體。呢個技巧用好咗，路人睇你影嘅相會以為你學過攝影。

- **唔好老係企住影**：大部分人影相永遠係自己眼睛嘅高度，所以影出嚟嘅視角千篇一律。試下蹲低由低角度影，或者搵個高處往下影，同一個地方出嚟嘅感覺完全唔一樣。影小朋友同寵物尤其需要蹲低，平視先可以影出自然嘅感覺。

- **光線係第一要素**：中午大太陽下影人像，塊面嘅陰影又硬又唔自然。最好嘅光線係清晨同傍晚嘅「黃金時段」，陽光係斜嘅、顏色偏暖，影乜嘢都好睇。陰天嘅光線雖然唔出彩，但勝在均勻柔和，影人像反而比大太陽下容易出片。

- **夜景模式使用要領**：打開夜景模式之後，手機會自動連拍多張然後合成。呢個時候最關鍵嘅係——手絕對唔可以抖。可以靠牆就靠牆，可以放枱面就放枱面，實在唔得就屏住呼吸。夜景模式需要你穩住 2 到 4 秒，抖咗畫面就糊咗。

三、人像模式與專業模式

- **人像模式**：佢可以影出背景模糊、主體清晰嘅效果（淺景深）。手機因為鏡頭物理尺寸細，真實虛化能力有限，所以靠嘅係 AI 算法嚟實現——簡單講就係 AI 將你嘅人摳出嚟，然後模糊背景。用嗰陣保持 1-2 米距離效果最好，離太遠或太近算法都容易出錯。仲有，影人像嗰陣，注意背景入面唔好有太複雜嘅線條，否則 AI 扣圖會喺頭髮邊緣出現「翻車」。

- **專業模式**：如果你想要完全掌控畫面效果，可以試下專業模式。幾個關鍵參數要知道：ISO 越低畫質越乾淨，但畫面會暗；快門越慢入光越多，但手抖會糊；白平衡管色彩嘅冷暖，調唔好相會偏藍或偏黃。一般人唔需要常年用專業模式，但瞭解呢幾個參數至少可以等你知道相機喺度做乜嘢。

四、後期修圖：唔好過度，啱啱好就得

好多人對修圖有誤解，覺得修咗就唔真實。其實相機影出嚟嘅相本來就同肉眼睇到嘅唔一樣，適當調整係為咗還原你嗰陣睇到嘅畫面，而唔係造假。

- **Snapseed**：谷歌出嘅免費修圖工具，功能專業但冇廣告。我日常修圖嘅流程係：先用「調整圖片」拉一拉亮度同對比度用「突出細節」銳化一下用「白平衡」糾正偏色最後「裁剪」成想要嘅構圖。Snapseed 嘅「局部」功能好好用，可以淨係調畫面中某一個區域，唔使全圖一齊動。

- **醒圖/美圖秀秀**：呢兩個適合修人像。但我覺得有個原則：美顏濾鏡拉到 30% 以內就好，超過呢個數就假喇，皮膚好似塑膠嗷。自然比完美好睇。

- **Lightroom 手機版**：如果你對色彩有追求，呢個係最專業嘅。佢有曲線調整、HSL 分色調節呢啲高級功能。不過對多數人嚟講有啲殺雞用牛刀。

- **修圖工作流建議**：我自己嘅習慣係先影好原片，返到屋企喺電腦螢幕或 iPad 上慢慢修，手機螢幕太細，色彩都唔太準。如果必須喺手機上修，記得將螢幕亮度調到中等偏上，唔好喺慳電模式下修圖。

五、相片太多點算？整理策略

手機相簿入面堆咗幾千張相，諗住搵一張特定嘅翻半天——呢個體驗大家都有。幾個實用嘅整理方法：

- **善用搜尋**：而家 iPhone 同大部分安卓手機嘅相簿都支援智能搜尋。你可以搜「貓」「海邊」「身份證」「花」呢啲關鍵詞，系統會用 AI 識別畫面內容幫你搵出嚟。仲可以搜日期同地點，比如「舊年五月」「北京」。呢個功能知嘅人唔多，但真係好好用。

- **定期清理垃圾**：截圖、重複嘅相、影糊咗嘅廢片，呢啲先係佔空間嘅元兇。iOS 相簿有「重複項目」自動合併功能；安卓可以去檔案管理入面搜「screenshots」資料夾，批量刪唔需要嘅截圖。我每個月花五分鐘清理，就可以騰出好幾個 G。

- **創建相簿，唔好全靠預設排序**：系統預設按時間排序冇錯，但對於「小朋友嘅成長相」「旅行相片」「重要證件」呢種跨時間段嘅內容，你應該手動建相簿歸類。蘋果嘅「共享相簿」仲可以同家人一齊往入面加相，一個人建，全家都可以睇。

- **備份係底線**：相片係最不可替代嘅數據——檔案丟咗可以重建，聊天記錄冇咗也就冇咗，但小朋友嘅相、老人家嘅影片丟咗就真係冇咗。一定要開雲備份。iCloud 相片最慳心，安卓呢邊各品牌都有自己嘅雲服務，或者用百度網盤/阿里雲盤自動備份。原則係：至少有一份雲備份，唔好將雞蛋放喺同一個籃入面。

六、截圖與錄屏

- **截圖**：同時按電源鍵 + 音量加鍵，所有手機通用嘅方法。好多安卓手機仲支援三指下滑截圖、指關節雙擊截圖。長截圖（滾動截圖）喺安卓上好成熟——截完一張後點「滾動」按鈕，手機會自動往下翻頁並拼接成一張長圖，適合截聊天記錄同網頁。

- **錄屏**：由控制中心添加「螢幕錄製」，點一下開始錄，再點停止。錄之前長按錄製按鈕可以選擇係咪錄製麥克風聲音——你要做教程解說就開麥，淨係錄操作畫面就關掉。錄完嘅影片會自動存到相簿入面。

練一練：用手機完成一次完整嘅影相 + 修圖 + 整理

1. 打開相機設定入面嘅「網格線」，搵一個有自然光嘅地方影三張相（注意抹鏡頭、調曝光）。
2. 揀最好嘅一張，用 Snapseed 做基礎調整（亮度、對比度、裁剪）。
3. 畀呢張相建一個專屬相簿，命名為「我嘅作品」。
4. 檢查一下雲備份有冇開啟，確保相片已經喺雲端存咗一份。

第十一章附：專業影像技術知識

想要真正理解手機攝影同影片嘅奧秘，你需要了解一啲專業嘅影像技術知識。呢啲知識唔淨係可以幫你更好咁使用手機相機，仲可以等你喺選擇設備嗰陣更有判斷力。

一、分辨率、幀率與碼率：影片質量嘅三要素

分辨率（Resolution）：

分辨率決定咗畫面嘅清晰度，用像素數表示。

- **720P (HD)**：1280×720 像素，入門級清晰度。
- **1080P (Full HD)**：1920×1080 像素，目前最主流嘅分辨率。
- **2K (QHD)**：2560×1440 像素，比 1080P 更清晰。
- **4K (UHD)**：3840×2160 像素，超高清，細節豐富。
- **8K**：7680×4320 像素，未來標準，目前設備支援有限。

分辨率越高，畫面越清晰，但檔案都越大，對存儲同處理能力要求越高。

幀率（Frame Rate）：

幀率決定咗畫面嘅流暢度，用每秒幀數（fps）表示。

- **24fps**：電影標準幀率，有獨特嘅「電影感」。
- **30fps**：電視同網絡影片常用，流暢自然。
- **60fps**：高幀率，適合運動場景同遊戲錄製，非常流暢。
- **120fps/240fps**：超高幀率，可用於慢動作影片。

幀率越高，畫面越流暢，慢動作效果越好，但檔案都越大。

碼率（Bitrate）：

碼率決定咗影片嘅數據密度，用每秒數據量（Mbps）表示。

- 碼率越高，畫質越好，細節保留越多，但檔案都越大。
- 碼率過低會導致畫面出現馬賽克、色塊等壓縮痕跡。
- 專業影片拍攝通常需要高碼率以保留更多後期處理空間。

二、影片編碼格式：H.264、H.265 與 ProRes

H.264 (AVC)：

- **特點：**最廣泛使用嘅編碼格式，兼容性極好。
- **優點：**幾乎所有設備都可以播放，壓縮效率適中。
- **缺點：**相比新格式，壓縮效率較低，檔案較大。
- **適用場景：**網絡影片分享、普通用戶日常拍攝。

H.265 (HEVC)：

- **特點：**H.264 嘅升級版，壓縮效率提升約 50%。
- **優點：**相同畫質下檔案更細，支援更高分辨率同幀率。
- **缺點：**編碼同解碼需要更強嘅處理器，部分老設備唔兼容。
- **適用場景：**4K 影片拍攝、存儲空間有限嘅場景。

Apple ProRes：

- **特點：**蘋果開發嘅專業影片編碼，幾乎無損壓縮。
- **優點：**畫質極高，後期處理空間大，色彩準確。
- **缺點：**檔案極大，需要大量存儲空間，處理需要專業設備。
- **適用場景：**專業影片製作、電影級拍攝。

ProRes RAW：

- **特點：**結合咗 ProRes 同 RAW 格式嘅優勢。
- **RAW 係乜嘢：**未經處理嘅原始傳感器數據，保留最大資訊量。
- **優點：**後期調整空間極大，可以改變曝光、白平衡等參數而唔損失畫質。
- **缺點：**檔案極其巨大，需要專業後期軟件處理。
- **適用場景：**高端專業製作。iPhone 15 Pro 系列及以後支援拍攝 Apple ProRes 影片（最高 4K 60fps），iPhone 17 Pro 進一步支援 ProRes RAW 直接機內錄製。

三、HDR：高動態範圍技術

HDR（High Dynamic Range）技術可以同時保留畫面中最亮同最暗部分嘅細節。

- **HDR10**：最基礎嘅 HDR 標準，兼容性好。
- **HDR10+**：三星推動嘅標準，支援動態元數據。
- **Dolby Vision**：杜比開發嘅頂級 HDR 標準，畫質最佳，但需要授權。
- **HLG**：廣播級 HDR 標準，兼容普通顯示器。

拍攝 HDR 影片嗰陣，需要支援 HDR 嘅顯示器先可以睇到完整效果。

四、色彩空間與色深

色彩空間：

- **sRGB**：標準色彩空間，兼容性最好。
- **Display P3**：蘋果推廣嘅廣色域標準，色彩更豐富。
- **Rec.2020**：HDR 影片嘅標準色彩空間，色域最廣。

色深：

- **8-bit**：1670 萬色，普通影片標準。
- **10-bit**：10.7 億色，專業影片標準，色彩過渡更平滑。
- **12-bit**：680 億色，高端專業標準。

實用建議

- 日常拍攝：1080P/4K + 30fps + H.265 編碼，平衡畫質同檔案大小。
- 運動拍攝：1080P/4K + 60fps，保證流暢度。
- 專業創作：4K + ProRes（如果支援），保留最大後期空間。
- 存儲建議：高碼率影片好佔空間，建議定期備份到電腦或雲盤。

去探索一下：你嘅手機影片設定

1. 打開相機 App，切換到影片模式。
2. 查看分辨率同幀率選項，嘗試唔同嘅組合。
3. 如果支援，查看有冇 HDR、ProRes 等高級選項。
4. 拍攝幾段唔同設定嘅影片，對比畫質同檔案大小。

五、RAW 格式與 HEIF/JPEG 對比

乜嘢係 RAW 格式：

RAW 格式係相機傳感器記錄嘅「原始數據」——未經任何壓縮同處理嘅純原始資訊。佢就好似生鮮食材，需要後期「烹飪」先可以成品。手機因為存儲同算力有限，預設影出嚟嘅都已經係「烹飪好」嘅 JPEG 或 HEIF，RAW 需要專門開啟。

RAW vs HEIF vs JPEG 對比：

- **JPEG**：最老嘅格式，兼容性最好（任何設備都可以打開）。8-bit 色深，有損壓縮，檔案最小（一張相 2-5MB）。後期空間有限，強行提亮暗部容易出現色塊。
- **HEIF/HEIC**：蘋果主推嘅新格式，同樣畫質下檔案淨係得 JPEG 嘅一半（一張相 1-3MB）。支援 10-bit 色深同透明通道。缺點係部分老設備同 Windows 需要安裝插件先可以打開。
- **RAW**：未經壓縮嘅原始數據，檔案最大（一張相 20-100MB）。色深通常 12-16bit，後期空間最大。缺點係必須用專業軟件（Lightroom、Snapseed 等）先可以打開同處理，無法直接分享。

點解專業攝影師要影 RAW：

- **白平衡可後期無損調整**：JPEG 影錯咗白平衡（比如白熾燈下偏黃），後期糾正會損失畫質；RAW 可以重新設定色温，完全無損。
- **曝光容差大**：JPEG 欠曝 2 檔嘅暗部基本係死黑一片，提亮後全係噪點；RAW 可以由欠曝 3-4 檔嘅暗部「救」返可用細節。過曝同理。
- **暗部同高光細節更豐富**：RAW 嘅動態範圍遠大過 JPEG，亮部不過曝、暗部不死黑嘅餘地大得多。
- **銳化、降噪、鏡頭矯正都喺後期可控**：相機嘅機內處理係「一錘子買賣」，RAW 可以反覆調整，每次都由原始數據出發。

手機點樣開啟 RAW：

- **iPhone ProRAW**：僅 iPhone 12 Pro 及以上 Pro 機型支援。打開「設定相機格式」中開啟「Apple ProRAW」。然後喺相機介面頂部會出現一個「RAW」按鈕，點亮即可。檔案約 25MB/張，建議有需要嗰陣再開。
- **安卓 Motion Raw / Ultra RAW**：各品牌叫法唔同。小米/OPPO/vivo 旗艦機型喺相機「專業模式」或「更多」入面可以搵到 RAW 開關；華為喺「專業模式」中開啟 RAW；三星 Galaxy S 系列喺「專業」模式入面有 RAW 選項。檔案大小通常喺 20-50MB。
- **通用建議**：日常隨手影冇必要開 RAW，檔案太大佔空間；淨係喺重要場景（風光、人像、需要後期）嗰陣先開啟。

六、計算攝影原理

現代手機攝影已經離不開「計算攝影」——亦即係靠算法而唔淨係鏡頭同傳感器嚟生成相片。理解呢幾個核心原理，可以等你更明白手機喺撳快門嗰一刻做咗乜嘢。

多幀合成：

手機撳一次快門，其實影咗多張相。算法將多幀對齊、加權融合，得到一張高質量成片。

- **夜景模式**：1-3 秒內連拍多張欠曝相，對齊後疊加——噪點互相抵消、亮度累加。結果就係暗光都可以影出乾淨嘅畫面。
- **HDR（高動態範圍）**：分別拍攝欠曝、正常、過曝三張（或多張），合成一張亮部同暗部都有細節嘅相。逆光場景尤其有效。
- **Deep Fusion（蘋果）/Photonic Engine（光子引擎）**：針對中低光場景，逐像素分析多幀並融合，重點提升紋理同細節，冷衫、毛髮呢類細節會明顯更清晰。

語義分割：

AI 唔淨係將成張圖一刀切處理，而係先「睇懂」畫面入面有乜嘢——天空、人臉、草地、建築、衫——然後對每個區域分別應用唔同嘅處理策略。

- **天空**：單獨增強藍色、壓低亮度，避免過曝。
- **人臉**：識別膚色區域，單獨提亮、平滑膚色，唔動五官同頭髮細節。
- **草地/樹葉**：增強綠色飽和度，提升細節銳化。
- **背景虛化**：人像模式下，AI 識別主體輪廓，背景模糊、主體清晰。

深度學習嘅應用：

- **AI 場景識別**：相機自動識別「美食」「寵物」「夜景」「文檔」等場景，並對應優化參數。比如美食模式會增強暖色調同飽和度，等食物睇落更誘人。
- **AI 人像虛化**：手機鏡頭物理上無法做到單反嗰種淺景深，靠深度學習模型預測主體邊緣，再對背景做模糊。最新算法已經可以正確處理頭髮絲、玻璃邊緣呢啲細節。
- **AI 消除/AI 擴圖**：選中畫面中嘅路人或雜物，AI 基於上下文「補全」被遮擋嘅部分。呢個係生成式 AI 喺影像領域嘅應用。

計算攝影 vs 光學攝影：

- **計算攝影嘅優勢**：細體積都可以得到好畫質，暗光、逆光等極端場景都可以出片，普通用戶唔需要懂攝影知識就可以影出「好睇」嘅相。

- **計算攝影嘅劣勢**：畫面有時「過度處理」——銳化過度、色彩失真、天空顏色唔自然；多幀合成喺主體移動嗰陣會留低重影；細節係「算出嚟」嘅而唔係真實記錄嘅。
- **光學攝影嘅優勢**：大底傳感器 + 大光圈鏡頭能真實捕捉光線，畫質自然，後期空間大；唔會因為算法「腦補」產生偽影。
- **趨勢**：現代旗艦手機係「光學 + 計算」結合——用大底傳感器獲取真實資訊，用算法優化呈現。兩者並不對立。

七、鏡頭類型與焦段

現代旗艦手機通常配三到四顆鏡頭，每顆對應唔同焦段。理解焦段，可以等你喺唔同場景下揀啱鏡頭。

常見鏡頭與等效焦距：

- **超廣角（13-24mm 等效）**：視野最廣，適合風光、建築、室內空間。能影到眼前所見嘅大場面，但邊緣會有畸變（直線變彎），影人像嗰陣人臉會被拉伸。適合「大場景」，唔適合「特寫」。
- **主攝（24-28mm 等效）**：日常最常用嘅鏡頭，畫質最好、光圈最大。兼顧視野同畫質，影人影景都可以用。多數手機嘅「1x」就係主攝。
- **長焦（70-200mm 等效）**：用於影遠景、人像特寫。長焦能壓縮空間感（遠處嘅山同近處嘅人睇落更近），影人像能獲得更自然嘅五官比例同背景虛化。
- **潛望式長焦（200mm+ 等效）**：通過反射式光路設計，喺手機厚度內實現更長焦距。能喺好遠距離影清楚細節，適合演唱會、體育賽事、野生動物。旗艦機型先有。

傳感器尺寸與像素大小嘅權衡：

傳感器係「底」，決定能接收幾多光線。行業入面講「底大一級壓死人」——同樣像素數下，傳感器越大，單個像素都越大，入光量越多，畫質越好。

- **大底感應器**：進光量大，夜景畫質好，動態範圍寬。缺點係手機要做厚（鏡頭模組凸起），成本高。
- **高像素小底**：白天光線充足時，高像素可以影到更清晰嘅相，方便後期裁切。但係暗光下每個像素進光不足，噪點多。
- **主流旗艦選擇**：主攝多數係 1/1.3 英寸到 1 英寸（部分機型已經用緊 1 英寸大底），像素數 5000 萬左右——呢個係畫質同體積嘅平衡點。

像素融合技術（Pixel Binning）：

為咗兼顧白天高像素同夜晚大像素，手機廠商用「像素融合」——將多個小像素合併成一個大像素輸出。

- **四合一像素（Quad Bayer）**：5000 萬像素感應器，輸出 1250 萬像素相。每個輸出像素由 4 個相鄰像素合成，進光量提升 4 倍，畫質更乾淨。

- **九合一像素（Nonacell）**：用喺 1 億像素以上嘅感應器，輸出約 1100-1200 萬像素。同樣原理，進光量更大。

- **實際效果**：預設影相模式下，你攞到嘅就係融合後嘅低像素相——畫質好、檔案細。需要極致細節時（比如影文件、遠景），可以手動切到「高像素模式」輸出全解像度。

八、特殊拍攝模式

除咗日常影相，手機入面仲有幾個特殊模式，用啱可以影到平時影唔到嘅畫面。

全景模式：

- 拍攝時按提示箭頭緩慢平穩咁橫向移動手機，唔好快——快咗會出現拼接錯位。

- 保持手機垂直方向穩定，淨係橫向平移。如果上下晃動，地平線會變波浪。

- 有移動嘅人或車時慎用——拼接時會出現「半個身體」或「鬼影」。

- 全景模式都可以打直影——直影橫拼接，適合影高聳嘅建築或瀑布。

慢動作與延時攝影：

- **慢動作**：用 120fps 或 240fps 高幀率拍攝，回放時變成慢鏡頭。適合影水滴、運動瞬間、寵物跳躍。光線要充足，慢動作模式下進光時間短，暗光畫質差。

- **延時攝影**：間隔拍攝多張相合成快進影片。適合影車流、雲彩移動、日落、花朵開放。拍攝時長幾分鐘到幾個鐘不等，務必用三腳架固定手機，手持必糊。

- **小技巧**：延時攝影影雲彩時，將手機架好，影 15-30 分鐘，最後得到 10 秒左右嘅延時影片，效果震撼。

星空/天文模式：

- 需要遠離城市光污染，搵完全冇路燈幹擾嘅郊外或山區。城市入面影唔出銀河。

- 必須用三腳架，手持必糊。手機要穩穩架住幾秒到幾十秒。

- 天氣要晴朗無雲，月亮唔好太大（滿月會蓋住星光）。農曆月初或月末嘅暗夜最適合。

- iPhone 嘅「夜間模式」會自動識別星空並延長曝光時間（最長 30 秒）；安卓旗艦嘅「星空模式」或「天文模式」效果類似。要耐心等待，撳快門後唔好鬱手機。

微距模式：

- 用於拍攝花朵、昆蟲、珠寶、文字特寫等小物體。可以貼近到 1-2 釐米距離對焦。
- 部分手機有專門嘅微距鏡頭（多數係 200 萬像素或 500 萬像素嘅小底鏡頭），畫質一般；旗艦機用超廣角鏡頭兼顧微距，畫質更好。
- 貼近拍攝時鏡頭離主體好近，會遮擋光線，必要時用枱燈或手電筒補光。
- 對焦距離極近，景深好淺，主體前後移動幾毫米就可能脫焦。拍攝時手要穩，最好等主體保持唔鬱。

靜態攝影實用建議

- 日常拍攝：HEIF/JPEG 預設即可，檔案細、畫質夠用。
- 重要場景（風光、人像、需要後期）：開啟 RAW，保留最大後期空間。
- 選啱鏡頭：風光用超廣角，人像用長焦，日常用主攝。
- 夜景同星空：務必固定手機，開夜景模式或天文模式。
- 大底手機（如 1 英寸感應器機型）畫質上限更高，但操作習慣仍係關鍵。

動手試試：摸清你嘅相機能力

1. 查看你嘅手機係咪支援 RAW 拍攝（設定相機格式），試下開啟並拍攝一張，對比同一場景嘅 JPEG 同 RAW 檔案大小。
2. 切換到超廣角、主攝、長焦三個焦段，分別影同一處遠景，體會視角差異同畫質差異。
3. 搵一個有夜景模式或星空模式嘅場景，固定手機後拍攝一張夜景相。
4. 嘗試用全景模式橫影一張大場景，注意保持手機平穩移動。

第十二章：影音娛樂與閱讀

手機而家基本上已經取代咗 *MP3*、電視遙控器、*Kindle* 同掌上遊戲機。但係講真，好多人淨係用咗呢啲 *App* 最基本嘅功能，仲有好多好用嘅地方未發現。呢一章我哋就嚟傾下點樣等手機娛樂更順手、更舒服。

一、聽音樂：揀邊個 App，點樣聽先舒服

音樂 App 而家功能都差唔多，但各有各嘅側重點，揀啱可以省唔少心：

- **網易雲音樂**：佢嘅強項係歌單社區同評論區。你想聽「適合落雨天聽嘅歌」或者「學到劫時放鬆用嘅純音樂」，搜一下就有別人整理好嘅歌單。評論區有時候比歌仲精彩。缺點係部分歌曲版權不全，有啲歌聽到聽到就灰咗。
- **QQ 音樂**：版權最全，周杰倫、五月天呢啲熱門歌手嘅基本上淨係 QQ 音樂有。音質選項都好豐富，會員可以開無損音質。如果你對「想聽嘅歌都搜得到」呢件事好在意，QQ 音樂係最穩妥嘅選擇。
- **Apple Music**：蘋果用戶嘅首選。介面乾淨、冇廣告、音質好，而且支援無損同杜比全景聲。但佢嘅推薦算法不如國內 App 識中文用戶嘅喜好，歌單文化都冇網易雲咁發達。適合鍾意簡潔體驗、唔太在乎社交功能嘅人。
- **Spotify**：全球用戶最多，推薦算法公認最強。「每週發現」同「日推」準確率高得嚇人，用得越耐越識你。缺點係免費版廣告多，而且某些中文歌版權覆蓋不全。需要特殊網絡先用得到。

幾個聽歌小技巧：如果你用嘅係藍牙耳機，去音樂 App 設定入面睇下有冇「音效調節」或「均衡器」，稍微調一下低音同高音，效果提升明顯。另外，串流媒體聽歌好食流量，記得喺設定入面將「僅 WiFi 下播放無損」打開，或者提前下載好鍾意嘅歌單離線聽。

二、播客（Podcast）：通勤路上嘅「耳朵伴侶」

播客呢樣嘢，未聽過嘅人覺得同自己無關，一旦開始聽就返唔到轉頭。本質就係「電台節目嘅 Netflix 版」——有人傾科技，有人講歷史，有人讀故事，你想聽乜嘢類型嘅都有。

- **小宇宙 App：**國內做得最好嘅播客 App，介面清爽，冇亂七八糟嘅功能。佢嘅「每日推薦」同「編輯精選」幫你發現新節目，評論區都好活躍。我個人聽得最多嘅係《忽左忽右》（文化歷史類）同《不合時宜》（社會議題類），每期都好似喺旁聽幾個聰明人傾偈。

- **蘋果播客：**iPhone 自帶，唔需要額外下載 App。同 CarPlay 嘅整合最好，揸車嗰陣好方便。

- **收聽技巧：**大部分播客 App 都支援 1.2 倍速或 1.5 倍速播放，習慣咗之後可以省唔少時間。我一般 1.2 倍聽中文，1.5 倍聽英文。另外一定要開「自動下載新節目」同「播完自動刪除」，咁樣你嘅手機唔會堆滿聽過嘅音頻檔案。

三、睇影片：由播放器到串流媒體嘅正確打開方式

好多人唔知，手機上可以播嘅格式遠比串流媒體 App 入面提供嘅多。一個好嘅本地播放器可以解決好多問題。

- **VLC 播放器：**開源免費，幾乎乜嘢格式都播到——MKV、AVI、FLAC、甚至係 ISO 光碟鏡像。如果你下載咗一個影片喺電腦上播到但手機上播唔到，八成係格式問題，裝個 VLC 就搞掂。佢仲支援外掛字幕，將.srt 字幕檔案同影片放喺同一個資料夾，VLC 會自動載入。

- **nPlayer：**如果你想要更精緻嘅體驗，呢個付費 App 值得買。介面靚，手勢控制好順手——左邊上下滑調亮度，右邊上下滑調音量，左右滑快進快退。支援 DTS 同杜比音效，睇大片嗰陣臨場感強好多。

- **Bilibili（B 站）**已經唔淨係二次元網站，而家上面嘅學習資源多到離譜。考研、編程、攝影、烹飪，乜嘢教程都有。彈幕文化係 B 站嘅靈魂，睇影片嗰陣開住彈幕，感覺好似有一羣人喺陪你一齊睇。不過睇學習影片嗰陣建議關彈幕，唔係容易分心。

- **投屏到電視：**手機螢幕再大都唔如電視睇得舒服。同一個 WiFi 下，打開影片 App 右上角嘅「TV」投屏掣，就可以將畫面投射到電視上。大部分智能電視同投影儀都支援，睇劇睇戲體驗好太多。iPhone 用戶仲可以用 AirPlay 直接鏡像整個螢幕。

四、電子書閱讀：手機上都可以好好睇書

有人話手機上讀書傷眼、容易分心，呢啲確實有道理。但手機閱讀嘅優勢都好明顯——隨時隨地，碎片時間利用最大化。我喺地鐵上、排隊嗰陣、瞓前，用手機讀咗唔少書。

- **微信讀書：**目前國內體驗最好嘅閱讀 App。書庫好全，出版書同網文都有。佢用「閱讀時長換書幣」嘅模式，睇得多就可以免費換書，對學生黨好友好。佢嘅「劃線」功能可以睇別人喺邊啲段落做咗標記，有時候睇到別人標註嘅好句子，確實有共鳴。不過「閱讀排行榜」呢樣嘢有啲卷，容易變成「為咗刷時長而讀書」，建議關掉。

- **Kindle App**：如果你有 Kindle 設備，手機端 Kindle App 可以同步閱讀進度。Kindle 嘅正版電子書排版質量比盜版 PDF 好太多，但 Kindle 中國區書城已經關咗，需要切換到海外區。
- **Apple Books（圖書）**：iPhone 自帶，適合讀 EPUB 同 PDF。介面極簡，翻頁動畫好舒服。如果你由網上下載咗電子書檔案，直接「分享到圖書」就可以匯入。
- **閱讀設定建議**：唔好用邊個 App，一定要將背景設成暖黃色調（護眼模式），字體調大啲，行距拉開。夜晚讀書打開「深色模式」——黑底白字，眼舒服好多。千萬唔好白底黑字細字體喺暗光下讀太耐，眼疲勞係累積嘅。

五、手機遊戲：玩得爽，都唔傷手機

我對手機遊戲嘅態度係：玩可以，但唔好等佢反過嚟控制你。合理使用遊戲工具，玩得更舒服，都唔耽誤正事。

- **遊戲模式/遊戲空間**：而家主流手機都有呢個功能。打開之後手機會自動清後台、遮蔽通知、防止誤觸——你正在打團戰，突然一個電話彈出嚟，呢種嘢遊戲模式可以幫你擋掉。部分手機仲支援性能模式/E-sports 模式，強制處理器跑滿，但代價係發熱同耗電都快啲。
- **畫質同幀率點樣調**：唔好無腦拉滿。先保證幀率（流暢度），再追求畫質。60 幀比 30 幀舒服太多，但 120 幀對手機壓力好大，中端機唔使強求。如果你玩嘅係競技類遊戲（比如王者、吃雞），幀率優先級遠高於畫質——流暢嘅操作手感比好睇嘅草皮重要一百倍。
- **遊戲時手機發燙點算**：呢個係正常嘅，但長期高温會損傷電池壽命。幾個降温辦法：除低手機殼散熱；將螢幕亮度降到 70% 左右（亮度係高發熱源）；唔好一邊充電一邊打機（發熱疊加，電池死得快）。如果手機燙到手握唔住嘅程度，趕緊退出嚟等佢歇一陣。
- **防沉迷小工具**：iOS 嘅「螢幕使用時間」同安卓嘅「數碼健康」都可以畀遊戲 App 設每日時間限額。我畀自己設嘅係工作日最多一個鐘，週末兩個鐘。到時間咗 App 圖示會變灰，撳唔入去。呢個限制係真係有用，唔係裝樣嘅。
- **雲遊戲**：如果你手機性能唔夠但係又想玩大作，可以試下雲遊戲平臺（比如騰訊 START 雲遊戲、網易雲遊戲）。遊戲喺遠端伺服器上跑，畫面以影片流嘅方式傳到你嘅手機上。對網絡要求高但對你手機性能幾乎冇要求。缺點係延遲感不可避免，適合玩策略類、RPG 類，唔適合 FPS 射擊類。

嚟動手：打造你嘅手機娛樂系統

1. 喺音樂 App 入面下載你最鍾意嘅 3 張專輯或歌單，確保離線可聽。
2. 去小宇宙或蘋果播客訂閱 3 個感興趣嘅頻道，下載一期試聽。
3. 安裝 VLC 播放器，嘗試用佢播放一段電腦拷貝過嚟嘅影片檔案。
4. 喺微信讀書入面搵一本想讀嘅書，設定暖色背景同護眼模式，將字體調到適合自己嘅大細。

5. 如果你打機，打開遊戲模式睇下有邊啲設定可以優化（免打擾、幀率優先等）。

第十三章：工具類 App 推薦

本章精選效率、健康、出行類工具，並補充實用系統同跨裝置能力：清濁等清理同防騷擾、手機自帶攔截開關、天氣同地震預警；向日葵/*ToDesk* 遠控電腦、*Trae* 等 *AI* 辦公、人工桌面、*LocalSend* 等區域網絡快傳、跨裝置剪貼簿、*Aegis/Microsoft Authenticator* 動態口令、滴答清單日程同紙質掃描。

一、效率工具：管理你嘅時間同注意力

效率工具係數碼生活嘅「骨架」，幫你將零散嘅想法、任務、資料整理得井井有條。下面按場景分類推薦。

待辦清單（任務管理）

- 滴答清單（TickTick）　適用人群：需要將工作、生活、學習任務同日程統一記錄嘅人；本節重點推薦之一。核心賣點：跨手機/電腦/平板/Web 同步；清單 + 日曆 + 番茄鐘三合一；自然語言輸入「聽日下晝 3 點開會」「每個禮拜一交週報」自動識別時間同重複規則；支援子任務、標籤、優先級、附件同協作清單。適合「日程安排 + 任務打卡 + 每日回顧」一體化管理。
- 滴答清單點樣用得更順：收集箱：先全部丟入「今日/收集箱」，夜晚再歸類去工作、家庭、學習清單。
- 日曆視圖：會議、截止日拖到日曆上睇衝突，比只看待辦列表更直觀。
- 重複任務：食藥、交費、健身設成規則任務，唔使每次手填。
- 番茄鐘：喺任務上直接開專注，統計今個禮拜「真正做咗幾多」。
- 同手機功能聯動：語音助手話「提醒我…」可以入系統提醒；重要截止日同時開系統日曆雙保險。

Todoist　適用人群：追求極簡、跨平臺用戶。核心賣點：介面克制清爽，項目層級清晰（任務子任務子子任務），karma 積分激勵習慣養成。

Microsoft To Do　適用人群：辦公一族、Outlook 用戶。核心賣點：完全免費、無廣告，「我的一天」功能每日清空、減輕心理負擔，同微軟生態深度整合。

系統便箋/提醒事項　適用人群：唔想裝第三方 App 嘅輕度用戶。核心賣點：原生體驗、零延遲，Siri/小愛語音直接加入，同系統 widget 聯動。

番茄鐘（專注力訓練）

- Forest　適用人群：管唔住手、容易撳手機嘅人。核心賣點：專注時種樹，提前提開 App 樹會枯死，用「虛擬生命」建立正向反饋，可以同好友合種。
- 潮汐　適用人群：需要白噪音輔助專注/入睡嘅人。核心賣點：自然音效（雨聲、海浪、森林）+ 番茄鐘 + 冥想三合一，介面極簡治癒。
- 原則：經典 25 分鐘專注 + 5 分鐘休息，可以按自身狀態調整做 50/10 或者 90/20。

日曆（時間可視化）

- 系統日曆　適用人群：所有用戶。核心賣點：原生推送最穩、耗電極低，可以用 Siri/小愛語音加入。
- Google Calendar　適用人群：多裝置用戶、需要同他人共享日程嘅人。核心賣點：「目標」功能自動幫你安排健身/學習時間，同 Gmail 智能識別電郵入面嘅事件。

筆記類（知識沉澱）

- Notion　適用人群：需要將筆記、任務、數據庫、wiki 統一管理嘅人；學生、自由工作者、團隊協作。核心賣點：全能知識庫，Block 模組化編輯，可以搭建個人主頁、閱讀清單、項目看板，模板生態豐富。
- Obsidian　適用人群：深度知識管理者、研究者、寫作人。核心賣點：本地 Markdown 儲存，雙向連結 + 關係圖譜，令你嘅筆記形成「第二腦袋」，數據完全歸你所有。
- flomo　適用人群：想記錄碎片靈感但唔想寫長文嘅人。核心賣點：卡片式筆記，類似發朋友圈一樣記錄想法，支援 # 標籤分類，按時間線回溯。
- 蘋果備忘錄　適用人群：iPhone/Mac 用戶、輕度記錄需求。核心賣點：啟動快、支援掃描文件/手繪/清單/iCloud 同步，免費而且同系統深度整合。
- 印象筆記　適用人群：需要大量網頁剪藏、資料歸檔嘅資深用戶。核心賣點：網頁剪藏插件強大，OCR 識別圖片文字，老牌穩定，但進階版要付費。

PDF 工具

- PDF Expert　適用人群：iPad/iPhone 用戶、需要批註合約嘅商務人士、學生。核心賣點：iOS 上公認最強嘅 PDF 編輯器，標註流暢、支援簽名、合併拆分、填寫表格。
- WPS　適用人群：需要同時處理 Word/Excel/PPT/PDF 嘅辦公族。核心賣點：一站式辦公套件，國內網盤/微信檔案兼容好，免費版功能已經足夠日常。

- Adobe Acrobat　適用人群：對 PDF 格式規範要求高嘅人（法律、設計、出版）。核心賣點：官方標準、轉換 PDF/A 格式無誤，跨裝置同步同電子簽名功能專業。

掃描類（紙質文件數碼化）

紙質合約、發票、功課、白板筆記、身份證複印件……手機鏡頭已經可以替代大部分掃描儀。核心流程：影清楚自動裁邊同矯正匯出 PDF/圖片 OCR 可選。

- 掃描全能王（CamScanner）　適用人群：經常需要掃描合約、票據、證件嘅商務人士同學生。核心賣點：邊緣自動識別、透視矯正、OCR 文字識別，可以生成多頁 PDF 並雲同步。
- 微軟 Lens　適用人群：學生、需要將白板/書頁掃描成 Word 嘅人。核心賣點：免費無廣告，可以一鍵匯出到 OneNote/Word，識別文字準確率高。
- 蘋果備忘錄掃描　適用人群：iPhone 用戶、輕度掃描需求。核心賣點：唔使裝 App，備忘錄內點擊相機圖示就可以掃描，自動去陰影、簽名標註，免費好用。
- 白描 / 掃描全能王網頁版 / 品牌相機「文件掃描」：華為、小米、OPPO、vivo 系統相機大多內置「文件/掃描」模式；Android 亦可以用 Google 相機或者系統掃一掃。適合應急影一頁合約。
- 影相小技巧：光線均勻、手機同紙面平行、對準四個角；多頁文件按順序影完再合併 PDF；敏感證件留意水印同儲存位置，用完可以從「最近刪除」徹底清理。

二、健康運動

健康類 App 係手機最被低估嘅「私人助理」——佢哋可以持續追蹤你嘅睡眠、心率、運動量，比偶爾去一次醫院更能反映長期狀態。

運動健身

- Keep　適用人群：居家訓練者、零基礎新手、想減脂塑形嘅人。核心賣點：海量免費課程，由 HIIT 到瑜伽全覆蓋，AI 自動生成訓練計劃，社區氛圍濃厚。
- Nike Training Club（NTC）　適用人群：追求專業教練指導、希望系統訓練嘅人。核心賣點：Nike 官方出品，課程全部免費，動作示範由專業教練錄製，按肌羣/器械分類清晰。
- 蘋果健身（Fitness）　適用人群：Apple Watch 用戶、想用閉環激勵自己郁動嘅人。核心賣點：每日三個活動圓環（移動/鍛煉/站立），可視化目標，同手錶心率數據無縫打通。

睡眠追蹤

- AutoSleep　適用人群：Apple Watch 用戶、想知道自己每晚深睡/淺睡比例嘅人。核心賣點：戴錶入睡自動追蹤，唔使手動啟停，畀出睡眠質素評分同就寢建議。

- Sleep Cycle　適用人群：唔戴手錶瞓覺、想用手機監測嘅人。核心賣點：透過麥克風分析翻身/鼻鼾聲，喺淺睡階段智能喚醒，令你起床唔痛苦。

- 小米運動健康　適用人群：小米手環/手錶用戶。核心賣點：免費而且同小米手環深度聯動，心率/血氧/睡眠一體追蹤，性價比高。

冥想放鬆

- Calm　適用人群：壓力大嘅上班族、入睡困難嘅人。核心賣點：「每日平靜」冥想引導 + 睡眠故事（由名人朗讀），介面唯美，英文內容豐富。

- 潮汐　適用人群：需要白噪音專注/入睡嘅中文用戶。核心賣點：國產精品，自然音效 + 冥想課程 + 番茄鐘，中文引導更親切。

- 小睡眠　適用人群：需要助眠音效、記錄夢話/鼻鼾聲嘅人。核心賣點：海量白噪音可以疊加組合（雨聲 + 風扇 + 篝火），瞓前定時關閉，夢話錄音有趣又實用。

女性健康

- Flo　適用人群：需要記錄經期、備孕或者關注女性健康嘅用戶。核心賣點：AI 預測下次經期/排卵日準確度高，社區可以匿名發問，介面設計精緻。

- 美柚　適用人群：國內女性用戶、希望睇中文社區同科普嘅人。核心賣點：本地化內容豐富，按經期/備孕/孕期/育兒分階段提供對應知識。

用藥提醒

- 用藥助手　適用人群：需要長期服藥嘅慢性病患者、需要照顧老人家用藥嘅子女。核心賣點：掃碼識別藥品、自動建立用藥清單，按時彈窗提醒，記錄服藥情況可以分享畀家屬。

數據中樞

- 蘋果健康/Google Fit　適用人群：所有智能手機用戶。核心賣點：匯聚來自手機、手環及第三方健康 App 嘅所有數據（步數、心率、睡眠、經期等），係統一嘅「健康檔案中心」。

三、旅行出行

出門在外，幾款趁手嘅 App 可以令你少踩無數坑。下面按出行全流程推薦。

航班高鐵

- 航旅縱橫　適用人群：經常坐飛機嘅商務人士。核心賣點：官方民航數據，航班動態/登機口變更/延誤預警最權威，可以查機型同座位圖。

- 12306　適用人群：所有國內火車出行者。核心賣點：官方唯一可信購票渠道，支援候補購票（有人退票自動搶），學生票/團體票辦理方便。

- 飛常準　適用人群：對航班準點率敏感、需要接送機嘅人。核心賣點：航班延誤預測準確，可以分享行程畀家人即時查看你嘅位置同到達時間。

地圖導航

- 高德地圖　適用人群：國內城市用戶、自駕/搭的士一族。核心賣點：即時路況準確、避開堵塞能力強，搭的士/搭的士合乘一站搞掂，公共交通轉乘方案豐富。

- 百度地圖　適用人群：對地點信息、室內導航有需求嘅用戶。核心賣點：全景地圖/AR 實景導航，景區/商場室內地圖細緻，語音包豐富。

- Google Maps　適用人群：出國旅行者。核心賣點：全球覆蓋最全、本地評價真實，街景同離線地圖係異國他鄉嘅救星。

翻譯

- Google 翻譯　適用人群：出國旅行者、跨語言學習者。核心賣點：「對話」即時雙向翻譯、「相機」即時取詞翻譯係旅行神器，支援離線語言包下載。

匯率換算

- 極簡匯率　適用人群：出國旅行、海淘、跨境購物嘅人。核心賣點：介面極致清爽，支援 160+ 貨幣，匯率即時更新，可以同時對比多種貨幣，長按數字快速換算。

天氣

- 系統天氣 / 中國天氣網　適用人群：所有人，查預報同官方氣象預警嘅首選。核心賣點：廠商天氣 App 同「中國天氣」等官方渠道對接氣象台預警，暴雨、颱風、高溫等預警會以橫幅/通知推送，權威性高。

- 彩雲天氣　適用人群：需要精準到分鐘級降雨預報嘅人。核心賣點：「未來兩小時會唔會落雨」準確度業內領先，降雨動態地圖直觀可見，落雨前主動推送提醒。

- Carrot Weather　適用人群：追求趣味性同豐富 widget 嘅天氣發燒友。核心賣點：毒舌 AI 語音播報天氣、超多自定義 widget 桌面組件、雷達圖專業，付費版功能更全。

- 預警要點：天氣類 App 要允許通知先收得到預警；預警分藍、黃、橙、紅四級，紅色表示災害性天氣風險最高，應該按提示調整出行。更多開啟方式見本章第七節「天氣預警 / 自然災害預警 / 地震預警」。

旅行攻略

- 馬蜂窩　適用人群：鍾意做詳細攻略、自駕遊/深度遊嘅人。核心賣點：用戶原創遊記內容豐富，目的地玩法/路線/避坑指南詳盡，可以下載離線攻略。

- 小紅書　適用人群：想睇真實體驗、搵小眾打卡點/美食嘅年輕旅行者。核心賣點：圖文短片形式直觀，「XX 城市 3 日攻略」等內容更新快、時效性強，搜索體驗好。

四、自動化工具啟蒙

自動化係「用一次配置換長期省事」嘅槓桿，可以將你每日重複嘅幾十次操作壓縮成零。本節只做概念啟蒙，具體玩法同詳細教程請見第二十三章：編程思維與自動化啟蒙。

- iOS「快捷指令」：蘋果官方自動化工具，唔使編程，可以一鍵完成「返到屋企開 WiFi + 亮度調低 + 播報天氣」等組合動作。

- Android 自動化：系統級場景聯動（例如小米「自動任務」、華為「情景智能」）上手最簡單；Tasker 功能強大但學習曲線陡峭，適合極客。

- 核心思路：先識別「每日都要重複做嘅操作」，再想「可唔可以用一個觸發條件 + 多個動作替代」。詳細示例同進階玩法見第 23 章。

五、瀏覽器同搜尋引擎：你嘅資訊入口

瀏覽器係你通往互聯網嘅窗口，搜尋引擎係你喺資訊海洋中導航嘅羅盤。用好佢哋，可以令你少行好多冤枉路。

瀏覽器嘅工作原理：

當你喺瀏覽器入面輸入網址之後，瀏覽器會：(1) 向 DNS 伺服器查詢域名對應嘅 IP 地址；(2) 向該 IP 地址嘅伺服器發送 HTTP/HTTPS 請求；(3) 伺服器返回網頁代碼（HTML、CSS、JavaScript）；(4) 瀏覽器解析代碼並渲染成你睇到嘅頁面。

主流瀏覽器推薦：

- 微軟 Edge：基於 Chromium 核心，速度快、兼容好。內置 Copilot AI 助手，可以總結網頁、翻譯、對話。支援跨裝置同步，個人強烈推薦作為主力瀏覽器。

- Safari：蘋果裝置預設瀏覽器，同系統深度整合，慳電高效。私隱保護好，但擴充插件較少。

- Chrome：全球使用率最高，擴充豐富。但記憶體佔用大，手機端相對耗電。

搜尋引擎選擇：

- Bing：微軟旗下搜尋引擎，同 Edge 深度整合。AI 摘要功能可以直接畀出答案而唔只係連結，搜尋效率好高。圖片搜尋體驗亦唔錯。
- Yandex：俄羅斯最大嘅搜尋引擎，圖片搜尋同反向圖片搜尋能力好強。搵相似圖片、識別物體時非常好用。
- 百度：國內資訊搜尋嘅主力，但廣告較多，需要一定嘅辨別能力。
- Google：全球最精準嘅搜尋引擎，但國內要特殊網絡先用得到。

高效資訊檢索指南：

- 善用 AI 搜尋：Bing 嘅 AI 摘要、Edge 嘅 Copilot 可以直接回答問題，幫你由海量結果中提煉關鍵資訊，比逐條點擊連結高效得多。
- 識別虛假資訊同廣告：搜尋結果中標注「廣告」或者「贊助」嘅條目係付費推廣，唔係自然搜尋結果。留意甄別資訊來源嘅權威性，優先揀官方網站、知名媒體嘅內容。
- 正確下載應用：喺搜尋引擎入面輸入「應用名 + 官網」，例如「微信官網」，優先點擊帶有官方域名標識嘅結果。唔好由第三方下載站安裝軟件，佢哋經常捆綁流氓軟件。
- 官網鑑別技巧：下載 Windows 11 應該訪問 microsoft.com，下載 Chrome 應該訪問 google.com。睇域名係咪同品牌一致，係最基本嘅鑑別方法。

網站域名知識：

域名係網站嘅「門牌號」，讀懂佢可以幫你判斷網站嘅性質同來源：

- 常見後綴含義：.com —商業機構，最常見
- .cn —中國國家域名
- .org —非牟利組織
- .net —網絡服務機構
- .edu —教育機構
- .gov —政府機構
- .xyz/.tech/.info —新興通用域名，門檻低，要謹慎

國家/地區域名：.jp —日本

.kr —韓國

.uk —英國

.de —德國

.tw —台灣

.hk —香港

安全提示：.gov 同.edu 後綴嘅網站通常更可信，因為申請門檻高。.xyz、.top 等廉價域名容易被釣魚網站利用，訪問時要多加提防。

廣告攔截：

- AdGuard　適用人群：被各類彈窗、貼片廣告、應用內廣告困擾嘅所有用戶。核心賣點：系統級廣告攔截（唔只限瀏覽器），可以過濾 App 內橫幅廣告同開屏追蹤，慳流量、加速頁面載入，付費版支援全裝置。

私隱瀏覽器：

- Brave　適用人群：注重私隱、唔希望被廣告追蹤嘅日常瀏覽者。核心賣點：基於 Chromium 核心，開箱即用屏蔽廣告同追蹤器，自帶 Tor 隱身模式，速度比 Chrome 快、記憶體佔用更低。

- DuckDuckGo　適用人群：希望「搜一個詞唔會被全網追住推廣告」嘅用戶。核心賣點：自帶同名搜尋引擎（唔記錄搜尋歷史），一鍵清除所有瀏覽痕跡（「燃燒按鈕」），跨平臺體驗統一。

瀏覽器對比一覽：

瀏覽器	核心賣點	適用人群
微軟 Edge	Chromium 核心 + 內置 Copilot AI，可以總結網頁/翻譯/對話	追求效率嘅辦公族、AI 重度用戶（推薦主力）
Safari	同蘋果生態深度整合，慳電高效，私隱保護強	iPhone/Mac 用戶，重視續航同系統聯動
Chrome	全球使用率最高，擴充插件生態最豐富	需要大量擴充插件嘅進階用戶、跨裝置同步一族
Brave	開箱即用屏蔽廣告同追蹤器，速度更快	注重私隱、唔裝插件都想屏蔽廣告嘅用戶
DuckDuckGo	同名搜尋引擎唔記錄歷史，一鍵清除痕跡	唔希望被個性化追蹤嘅私隱敏感用戶

試下建立一個「瞓前放鬆」嘅自動化流程

1. 建立觸發條件：特定時間（例如 22:30）。
2. 加入執行動作：調低音量 > 開啟勿擾模式 > 播放放鬆歌單 > 調低亮度。
3. 儲存並測試。

六、實用系統工具：清理、防騷擾同安全防護

除咗效率同健康類 App，真正影響日常體驗嘅往往係「少被打擾、儲存夠用、騙子入唔到嚟」呢三件事。本節先盤點常用實用軟件，再單獨展開手機自帶嘅騷擾攔截功能——好多人買咗手機多年，但從來未開過呢啲開關。

系統清理同儲存優化

- 清濁　適用人群：Android 用戶，希望輕量清理緩存、殘留檔案、空資料夾，又唔想被「手機管家」全家桶打擾嘅人。核心賣點：體積細、無廣告干擾，支援深度掃描應用殘留、大檔案定位、儲存佔用分析；比商業「一鍵加速」更克制，適合動手能力強嘅用戶做定期維護。留意：清理前先確認路徑，微信、相冊、文件目錄唔好盲目全選；系統級緩存盡量優先用品牌自帶儲存清理，第三方工具只作補充。
- 系統自帶儲存清理　適用人群：所有人，尤其是長輩同怕折騰嘅用戶。核心賣點：Android [設定] > [儲存] > [清理加速]；iOS [設定] > [一般] > [iPhone 儲存空間]。系統更了解自家 App 緩存結構，誤刪風險最低，應作為第一選擇。
- 微信/QQ 內置儲存管理　適用人群：聊天大戶。核心賣點：微信 [我] > [設定] > [一般] > [儲存空間]，可以按聊天、檔案類型精細清理；比外部「清微信」類工具更安全。
- 原則：可以用系統/App 自帶嘅，就唔用第三方；可以按路徑/類型剔選嘅，就唔好「一鍵全清」。清理嘅目標係釋放空間，唔係將重要數據一齊掃走。

防騷擾同安全類實用軟件

- 國家反詐中心　適用人群：所有人，尤其是長輩、財務相關崗位、經常網購/接陌生電話嘅人。核心賣點：官方出品，可以開啟來電預警、短訊風險提示、可疑網址檢測；同公安反詐數據庫聯動，係「三道防線」入面最有官方背書嘅一環。
- 品牌安全中心 / 手機管家　適用人群：對應品牌用戶。核心賣點：華為手機管家、小米安全中心、OPPO 手機管家、vivo 手機管家等，整合騷擾攔截、病毒查殺、權限管理、支付保護，通常免費而且同系統底層協同更好，唔建議再裝第二套商業安全軟件造成權限衝突。
- 電信商防騷擾服務　適用人群：被高頻外呼、AI 語音電話轟炸嘅用戶。核心賣點：移動發送 KTFSR 去 10086 開通；電信關注「天翼防騷擾」；聯通喺「聯通手機管家」入面設定。攔截發生喺電信商側，手機唔響鈴都可以擋住一批騷擾號。
- AdGuard　適用人群：被各類彈窗、貼片廣告、應用內廣告困擾嘅所有用戶。核心賣點：系統級廣告攔截（唔只限瀏覽器），可以過濾 App 內橫幅廣告同開屏追蹤，慳流量、加速頁面載入，付費版支援全裝置。
- 慎裝「清理加速 / 騷擾攔截」全家桶：部分第三方管家常駐後台、反覆請求通訊錄同短訊權限，攔截能力未必比系統強，反而增加耗電同私隱風險。優先打開系統自帶功能，再考慮官方反詐 App。

實用軟件選型原則

- 系統優先：攔截、清理、權限管理，先打開手機自帶開關。
- 官方補充：國家反詐中心、電信商防騷擾屬於官方通道，可以疊加。
- 第三方克制：同一類功能裝一個就夠，避免權限打交叉同後台搶資源。
- 少即是多：工具裝得越多，通知同權限面越大，被「工具本身」打擾嘅概率越高。

手機自帶騷擾攔截功能：各品牌點樣開

下面按品牌列出入口。名稱可能隨系統版本略有差異，可以喺設定入面直接搜尋「騷擾」「攔截」「未知來電」。同第八章「騷擾電話治理」互為補充：呢度側重開關位置，嗰度側重號碼標記同誤標申訴。

平臺	入口路徑	推薦開啟項目
iPhone / iOS	[設定] > [電話] / [訊息]	靜音未知來電；已阻止嘅聯絡人；訊息入面「過濾未知發件人」
小米 / Redmi	電話 App > 設定 > 騷擾攔截；或者安全中心	智能攔截、陌生號響一聲掛斷、短訊關鍵詞過濾
華為 / 榮耀	電話 > 騷擾攔截；手機管家 > 騷擾攔截	來電識別、黑名單、短訊智能攔截
OPPO / realme / 一加	電話 App > 騷擾攔截；或者本機 > 騷擾攔截	來電攔截、詐騙電話識別、短訊攔截規則
vivo / iQOO	電話 > 騷擾電話攔截；i 管家	騷擾攔截、詐騙電話預警、訊息過濾
三星	電話 > 設定 > 阻止號碼 / 來電顯示同垃圾電話	阻止未知號碼、垃圾電話識別（地區支援度唔同）

三層防護疊加建議

日常推薦「系統攔截 + 電信商防騷擾 + 國家反詐中心」三層疊加：系統管標記同黑名單，電信商喺網側擋高頻外呼，反詐中心補官方預警。三層唔使都開到最嚴，避免將快遞、外賣、醫生電話都誤殺——可以對通訊錄同最近聯絡人放行。

動手練習：今日就將騷擾攔截打開

1. 打開電話 App 設定，搵到「騷擾攔截 / 靜音未知來電」，開啟智能攔截。
2. 幫爸媽手機都檢查一次：同樣路徑 + 開通電信商防騷擾（移動可以發 KTFSR 去 10086）。
3. 應用商店安裝「國家反詐中心」，註冊並打開來電預警。
4. 試打一個已被標記嘅測試號（或者睇攔截記錄），確認功能生效。

七、天氣預警、自然災害預警同地震預警

騷擾攔截擋住嘅係「電話詐騙」，預警功能擋住嘅係「天災同極端天氣」。好多人只裝咗天氣 App 睇溫度，但冇開通知、冇開地震預警——暴雨紅色預警、颱風、地震來臨前嗰幾秒到幾十秒，手機本來可以幫你爭取到寶貴嘅避險時間。

天氣預警：暴雨、颱風、高溫點樣收

- 官方渠道優先：中國氣象局 / 中央氣象台 / 各省氣象局發佈嘅預警最權威。可以關注「中國天氣」微信公眾號、微博，或者下載「中國天氣」等官方類 App。
- 系統天氣 App：華為天氣、小米天氣、OPPO 天氣、vivo 天氣、蘋果「天氣」等，多數會推送當地氣象台預警；入天氣 App 設定，確認「氣象預警 / 災害性天氣」通知已開啟。
- 彩雲天氣等分鐘級降雨：適合通勤、接送小朋友前判斷「呢半個鐘使唔使帶遮」；預警能力仍然以官方氣象台為準。
- 預警顏色含義（通用）：藍色：注意防範，一般照常出行、關注更新。
- 黃色：加強防範，減少不必要戶外活動。
- 橙色：嚴重，盡量留喺室內，遠離危險區域。
- 紅色：特別嚴重，聽從停課、停工、避險指引，唔好硬闖積水、山洪、大風路段。

常見預警類型：暴雨、暴雪、颱風、高溫、寒潮、大風、雷電、冰雹、道路結冰、大霧、乾旱、地質災害風險。

自然災害預警：唔只係「落雨」

- 發佈主體：氣象台負責氣象類；地質災害、山洪等通常由自然資源、水利、應急管理部門聯合發佈；地方政府亦會透過應急廣播、短訊、12379 等渠道發佈。
- 點樣多渠道接收：手機系統通知 + 天氣 App 預警。
- 微信關注當地「應急管理局」「氣象局」公眾號。
- 12379 國家預警信息發佈相關渠道（短訊/服務，視地區開通情況）。
- 電視、應急廣播、村/社區大喇叭——山區、汛期唔好只睇手機。

戶外同出行提醒：颱風天唔好貼近廣告牌、唔好行樹下；暴雨唔好入地下車庫、隧道、下穿橋；山區旅遊遇「地質災害氣象風險」預警應即刻離開陡坡、溝谷。

同急救手冊聯動：災後失聯、手機冇電、需要定位求救時，見第三十二章地震/災害場景，以及第七章衛星通訊同無網通。

地震預警：同「地震預報」唔係一回事

地震預警（EEW）係喺地震波（尤其是破壞力更強嘅橫波）到達目標區之前，利用電磁波比地震波快、以及台站更快捕捉到縱波嘅原理，向可能受影響區域提前幾秒到幾十秒推送警報。佢唔能夠預測地震係咪會發生，但可以搶出「就近避險、切斷火源、衝出電梯」嘅窗口。距離震中越近，預警時間越短，甚至可能嚟唔切。

- 官方同行業方案：中國大陸常見接入方包括中國地震預警網相關能力，以及成都高新減災研究所等機構推動嘅預警服務；部分廠商手機會預裝或者可以聯網接收預警。
- 第三方 App：應用商店搜尋「地震預警」等官方/機構相關應用（安裝前睇清楚開發者，避免山寨「假預警」App 要你開一堆無關權限）。
- 收到警報後點樣做：室內就近躲避（枱下、承重牆角、堅固傢俬旁），護住頭頸；遠離窗戶、外牆、高大家俬；唔好盲目衝樓梯/跳樓；電梯入面即刻喺最近樓層離開；室外去開闊地，遠離電線、廣告牌、危房。震後檢查燃氣同電路，留意餘震。
- 勿擾勿關：地震預警應允許「突破靜音/勿擾」響鈴；瞓前將手機放伸手可及嘅地方，鈴聲音量唔好調到 0。

手機自帶地震預警：各品牌點樣開

下列入口隨系統版本可能略有差異，可以喺設定中直接搜尋「地震」「預警」「安全預警」。iOS 系統本身對國內地震預警支援有限，通常要安裝可靠嘅第三方預警 App，並保持定位同通知權限。

平臺	常見入口	說明
小米 / Redmi	手機管家 / 安全中心 > 地震預警；或者設定搜尋「地震」	部分版本系統內置預警服務，要聯網並開啟定位
華為 / 榮耀	設定 > 安全 > 應急預警通知 / 地震預警	名稱可能是「應急預警通知」，內含地震等
OPPO / realme / 一加	設定 > 安全（SOS 緊急聯絡）> 地震預警	部分機型同地區服務覆蓋有關
vivo / iQOO	天氣 App 或者設定 > 安全/更多 > 地震預警	以實際系統版本選單為準
iPhone / iOS	系統設定中通常冇完整內置開關；安裝可靠地震預警 App 並允許通知	App 要後台運行同定位權限；系統勿擾唔好屏蔽佢嘅警報
三星等	設定 > 安全與緊急情況 / 無線緊急警報	「緊急警報」覆蓋範圍因地區同電信商而異

關於地震預警嘅常見誤區

- 預警 ≠ 預報：佢唔能夠話你知「聽日會唔會地震」，只會喺已發生地震、波尚未到達時報警。

- 唔係邊度都可以提前好多秒：震中附近可能只有幾秒甚至嚟唔切，預警主要惠及周邊區域。
- 關通知就等於冇裝：再好嘅預警，靜音、殺後台、唔畀定位，都可能收唔到。
- 山寨 App：警惕要通訊錄、短訊權限嘅「地震預報」軟件，優先系統內置或者正規機構方案。

預警能力點樣搭配

日常建議：「系統天氣預警通知 + 官方氣象/應急渠道 + 地震預警（系統或者正規 App）」。同防騷擾一樣，唔使裝一堆同類軟件；關鍵係通知權限打開、定位允許、勿擾可以突破。幫長輩手機做完騷擾攔截之後，順手將天氣預警同地震預警都檢查一次。

動手練習：10 分鐘檢查預警設定

1. 打開系統天氣 App，入設定，開啟「氣象預警 / 災害性天氣」通知。
2. 喺系統設定入面搜尋「地震」或者「預警」，搵到並開啟地震預警/應急預警（Android）；iPhone 就安裝正規地震預警 App 並允許通知。
3. 關注當地氣象局或者應急局嘅公眾號/服務號，允許佢推送。
4. 確認預警聲音唔會被勿擾完全靜音；幫爸媽手機同樣檢查一次。

八、跨裝置協同：遠控電腦、快傳同帳號安全

手機用得順，多數係因為佢同電腦、平板「組咗隊」。本節補充幾類高頻實用工具：手機遠控電腦、AI 辦公同遙距協作、動態桌布、區域網絡快傳、跨裝置剪貼簿、動態口令 2FA，以及紙質掃描同日程任務嘅銜接。更系統嘅多裝置聯動見第十七章。

手機遠控電腦：向日葵 / ToDesk

- 向日葵遠程控制　適用人群：需要喺出街時操作屋企/公司 Windows 或者 Mac 嘅用戶；亦經常用嚟幫長輩遙距修手機（見第二十七章）。核心賣點：國產生態成熟，手機 App 控制電腦桌面、傳檔案、開關機（部分要硬件）；免費版一般夠個人輕度使用。
- ToDesk　適用人群：追求連接穩定、低延遲遙距辦公/維運嘅用戶。核心賣點：裝置碼 + 密碼連接，手機控電腦、電腦控手機（視版本）、檔案傳輸同多顯示器切換較順手，國內節點體驗通常較好。
- 基本用法：電腦端安裝用戶端並登入/保持在線手機端裝 App 用裝置識別碼 + 連接密碼（或者同一帳號）建立會話手機觸控板/手勢操作桌面。
- 安全紅線（務必記住）：陌生人自稱「客服/公檢法/電信商」叫你裝向日葵、ToDesk 並遙距你手機或者電腦，一律拒絕——呢個係典型詐騙手法（詳見第二十七章）。

- 自己用時：開啟二次驗證、修改預設密碼、唔用時退出帳號或者關閉無人值守；公司電腦遵守單位安全規定。

- 公共場合遙距辦公留意周圍有冇人偷睇螢幕同口令。

AI 辦公軟件：Trae / WorkBuddy / Dumate

- Trae　適用人群：編程同文件協作用戶，中文環境友善。核心賣點：字節跳動推出嘅 AI IDE/編程助手，SOLO 模式可以一句話生成項目；日常寫代碼、改文件效率高。如果電腦端運行，可以喺移動端配合遠控或者官方配套能力查看/跟進任務（以產品最新形態為準）。詳見第四十一章 AI 編程工具全景。

- WorkBuddy　適用人群：希望 AI 幫手整理會議紀要、電郵、匯報、辦公文件嘅職場用戶。核心賣點：面向辦公場景嘅 AI 助手，適合「起草—潤色—總結」呢類高頻文案；部分工作流可以喺手機端發起，重活放喺電腦端完成。

- Dumate　適用人群：需要多人協作、文件同知識管理一體化嘅團隊/個人。核心賣點：AI 協作向辦公工具，強調資料匯聚同協同編輯；手機端可以查看同輕量編輯，複雜排版建議喺電腦大螢幕處理。

- 「手機端遠程控制電腦」點樣理解：好多 AI 辦公能力跑喺電腦/網頁端；喺手機上透過向日葵/ToDesk 連返電腦，或者使用產品自身嘅手機 App/網頁，就可以喺通勤時「遙控」電腦上嘅 AI 任務。原則：重計算、長文件放電腦；睇進度、改幾句放手機。

- 私隱提醒：上載合約、身份證、客戶數據前睇清楚產品私隱條款同企業合規要求；公司資料優先用單位指定工具。

人工桌面：米哈遊動態桌布（手機 + PC）

- 人工桌面　適用人群：鍾意米哈遊角色、想要免費動態桌布嘅用戶。核心賣點：米哈遊推出嘅免費動態桌布應用，支援手機端同 PC 端；角色 Live2D/動態演出可以做桌面，部分機型/系統可以設做鎖屏或者桌面桌布。

- 慳電同性能：動態桌布比靜態圖更耗電、佔 GPU；低電量或者舊機型可以只喺充電時使用，或者降低幀率（如果 App 提供選項）。

- 安裝來源：優先官網/正規應用商店，避免第三方「破解版」捆綁木馬；PC 版留意安裝路徑同開機自啟係咪需要。

區域網絡檔案快傳：LocalSend / 飛鴿傳書 / 隔空投送

- LocalSend　適用人群：Android/Windows/iOS/Mac/Linux 混用，想要跨平臺、唔傳公有雲快傳嘅人。核心賣點：開源、區域網絡直連，同一 Wi-Fi 下互傳相片影片文件，唔使數據線、唔經過廠商伺服器，速度取決於路由器。

- 飛鴿傳書　適用人群：辦公室 Windows 電腦之間、舊區域網絡辦公場景。核心賣點：老牌區域網絡傳輸/聊天工具，適合內網傳大檔案；留意由正規渠道下載，企業內網遵守 IT 策略。
- 隔空投送（AirDrop）　適用人群：iPhone、iPad、Mac 用戶。核心賣點：蘋果生態一鍵分享，藍牙發現 + Wi-Fi 直連傳輸，免流量；可以設「僅聯絡人」降低陌生人騷擾。
- 其他補充：小米互傳、華為分享、vivo 互傳等品牌方案喺同一品牌裝置之間好快；微信「檔案傳輸助手」方便但受檔案大小同登入裝置限制，大檔案更推薦 LocalSend/隔空投送。
- 使用提示：雙方連同一 Wi-Fi；公共 Wi-Fi 傳敏感檔案有風險，優先私人熱點；傳輸完成後喺接收方確認檔案係咪完整。

跨裝置剪貼簿同步：KDE Connect / 手機電腦互傳

- 蘋果「通用剪貼簿」　適用人群：iPhone + Mac 用戶。核心賣點：登入同一 Apple ID、開接力之後，手機複製 Mac 直接貼上（反之亦然），文字同部分檔案都可以。
- KDE Connect　適用人群：Android + Windows/Linux/Mac，想要開源剪貼簿同步、通知鏡像、遙距輸入嘅用戶。核心賣點：區域網絡配對後同步剪貼簿、推送手機通知到電腦、用手機做幻燈片翻頁器/觸控板；數據唔經過第三方帳號。
- 品牌「手機電腦互傳」：華為多屏協同/超級終端、小米妙享、vivo 互傳、OPPO 互連、榮耀互聯等，通常自帶跨屏剪貼簿同檔案拖拽，同品牌體驗最好。
- Windows + Android 官方通道：「手機連接 / Link to Windows」（Your Phone 演進）可以喺電腦上睇手機通知、相片，並喺支援嘅場景複製貼上；三星等機型整合更深。
- 安全提醒：剪貼簿可能殘留密碼、驗證碼；喺共用電腦上慎用全局同步，用完喺密碼框避免二次貼上洩露；部分輸入法亦會讀剪貼簿，留意權限。

動態口令 2FA：Aegis / Microsoft Authenticator

- 點解要用認證器而唔係短訊：短訊驗證碼可能遭遇 SIM 補卡劫持；TOTP 動態口令喺本地按時間算法生成，唔使依賴手機號訊號，更抗釣魚同攔截（仍然要防木馬同截屏）。
- Aegis Authenticator　適用人群：Android 用戶，偏好開源、本地可控。核心賣點：開源免費，支援加密備份同導入導出，介面乾淨，可以用指紋/人臉解鎖令牌列表。
- Microsoft Authenticator　適用人群：使用微軟帳號、Office 365/企業帳號嘅用戶；跨 iOS/Android。核心賣點：推送批准登入、密碼監控同帳號備份（按產品版本），同 Windows/企業身份體系結合緊密。
- Google Authenticator / 2FAS 等：生態入面亦常見；關鍵係有加密備份，換機唔會失令牌。
- 換機必做：舊機遷移或者用服務入面嘅「備用碼/復原密鑰」；備用碼打印喺紙上存兩處，唔好只存喺同一部手機。

- 同密碼管理器：部分密碼庫亦可以存 TOTP，方便但風險更集中；高價值帳號建議「密碼管理器 + 獨立認證器」分工。詳見第十四章。

推薦組合：一日入面點樣串起嚟

實用工作流示例

- 早上：滴答清單睇今日日程天氣/預警掃一眼電腦開 Trae/文件，手機放埋一邊。
- 日頭：紙質材料用掃描類 App 轉 PDF，經 LocalSend/隔空投送去到電腦；長文喺電腦用 AI 辦公工具起草。
- 出街：用 ToDesk/向日葵連返電腦繼續任務；重要帳號登入行 Aegis/Microsoft Authenticator 動態口令。
- 夜晚：滴答清單剔選完成項目；需要時人工桌面換個桌布放鬆一下。

遠控同帳號安全口訣

- 人哋叫你裝遠控軟件掛線，自己搵官方客服。
- 驗證碼、動態口令唔好話畀任何人知，包括「快遞」「客服」。
- 公司電腦遠控同安裝 AI 軟件之前，先問 IT 可唔可以用。
- 公共網絡傳檔案、遙距登入時格外警惕。

動手練習：將「跨裝置」鏈路跑通一次

1. 手機同電腦連同一 Wi-Fi，用 LocalSend（或者隔空投送）互傳一張相。
2. 喺滴答清單建立一條「今晚 8 點：整理掃描件」任務，確認手機同電腦都睇得到。
3. 為電郵或者微信開啟 2FA（如果支援），用 Aegis 或者 Microsoft Authenticator 綁定，並保存備用碼。
4. 喺自己電腦上安裝向日葵或者 ToDesk，用自己手機成功連一次，再退出或者加固密碼，熟悉流程但唔留安全後門。

第十四章：密碼管理器專題

喺數碼時代，密碼就係你嘅門匙。但匙太多、太複雜，點算？密碼管理器就係你嘅「萬能匙管家」。

一、點解你需要密碼管理器？

現代人平均有幾十甚至上百個網絡帳號。如果每個帳號都用相同嘅密碼，一旦一個網站被駭，所有帳號都危險。如果每個帳號都用唔同嘅複雜密碼，根本記唔住。

密碼管理器解決咗呢個困境：你淨係需要記住一個主密碼，佢幫你記住所有其他密碼。

二、密碼管理器嘅工作原理

1. **加密存儲**：所有密碼都經過高強度加密存儲，淨係你可以解密。
2. **主密碼**：你設定一個主密碼，用於解鎖密碼庫。主密碼本身唔存儲喺伺服器上。
3. **自動填充**：喺網站或 App 登入介面，密碼管理器自動填充帳號密碼。
4. **密碼生成**：可以生成隨機、複雜嘅密碼，提高安全性。

三、加密原理簡介

瞭解密碼管理器嘅加密原理，可以幫你判斷一個服務商係咪值得信任。下面用通俗嘅語言解釋幾個核心概念。

AES-256 加密

AES（高級加密標準）係美國國家標準與技術研究院（NIST）認可嘅對稱加密算法，被銀行、軍方、政府廣泛使用。「256」指密鑰長度為 256 位——理論上需要超過宇宙年齡嘅時間先可以暴力破解。主流密碼管理器（1Password、Bitwarden、KeePass 等）都用 AES-256 加密你嘅密碼庫。

零知識架構（Zero-Knowledge Architecture）

「零知識」唔係話服務商乜都唔存，而係話**加密同解密都喺你嘅設備本地完成，伺服器淨係保存已加密嘅密文**。具體流程：

1. 你喺用戶端輸入主密碼。
2. 用戶端用主密碼派生出加密密鑰（呢一步喺你手機/電腦上完成）。
3. 淨係加密後嘅密文先上傳到伺服器。
4. 下次登入時，伺服器將密文下發，用戶端再次用主密碼解密。

關鍵點：**主密碼本身從不離開你嘅設備**，伺服器上都冇。所以即使服務商被駭，黑客攞到嘅都淨係一堆無法解密嘅密文——除非佢哋可以猜中你嘅主密碼。

點解「零知識」意味住服務商都睇唔到你嘅密碼？

因為解密密鑰由你嘅主密碼派生，而主密碼淨係存在你腦中、淨係喺你本地設備上使用。服務商嘅伺服器上既冇主密碼，都冇解密密鑰，淨係得密文。呢個就係點解 Bitwarden、1Password 等正規服務商都明確聲明「我哋都無法睇到你嘅密碼」——唔係佢哋唔想睇，係技術上做不到。

密鑰派生函數：PBKDF2 與 Argon2

主密碼唔可以直接用作加密密鑰——佢需要經過「密鑰派生函數」處理。呢個函數嘅作用係**故意等計算變慢**，等黑客暴力破解嘅成本高到不可行。

- **PBKDF2**：老牌標準，被 iCloud 鑰匙串、Bitwarden、LastPass 等廣泛使用。通過反覆迭代（通常 10 萬次以上）雜湊嚟拖慢破解。
- **Argon2**：2015 年密碼雜湊競賽冠軍，比 PBKDF2 更現代。除咗「算力消耗」仲引入咗「記憶體消耗」——等 GPU/ASIC 等專用硬件都難以加速破解。Bitwarden、KeePassXC 等已支援。

迭代次數越高越安全，但解鎖速度都會變慢。正規服務商都會公開佢使用嘅派生算法同迭代次數，方便安全社區審計。

案例警示：LastPass 2022 年數據洩露

2022 年 8 月，老牌密碼管理器 LastPass 公告佢嘅開發環境被入侵；同年 11 月進一步披露，黑客攞走咗用戶嘅密碼庫備份（加密後嘅密文）。事件嘅關鍵教訓：

- **零知識架構守住咗底線**：由於密碼庫本身係加密嘅，黑客攞到密文但解唔開——前提係用戶嘅主密碼足夠強。

- **弱主密碼用戶最危險**：黑客可以離線暴力破解弱主密碼，因此使用弱主密碼或重複主密碼嘅用戶面臨實質風險。
- **元數據都洩露**：雖然密碼內容加密，但站點 URL、備註等元數據可能未加密，會暴露用戶使用咗邊啲服務。

由 LastPass 事件學到嘅

第一，主密碼必須長而且獨特（建議 16 位以上，包含大小寫、數字、符號）；第二，優先選擇支援 Argon2id、迭代次數高嘅服務商；第三，對高價值帳號（銀行、郵箱）務必開啟獨立嘅二次驗證，唔好淨係依賴密碼管理器。

四、主流密碼管理器推薦

系統自帶：

- **iCloud 鑰匙串**：蘋果生態用戶首選。同 Safari、App 無縫整合，免費，安全。缺點係跨平臺使用不便。
- **Google 密碼管理器**：安卓用戶方便。同 Chrome、安卓系統整合，免費。
- **華為密碼保險箱**：鴻蒙系統內置，適合華為用戶。

第三方專業工具：

- **Bitwarden**：開源免費，功能強大，支援所有平臺。強烈推薦畀注重私隱嘅用戶。
- **1Password**：介面美觀，功能豐富，適閤家庭同團隊。付費。
- **LastPass**：老牌密碼管理器，免費版功能受限。
- **KeePass**：完全本地存儲，開源免費。適合極客用戶。

五、密碼強度真實評估

密碼管理器普遍會顯示「密碼強度」指示條（紅/黃/綠），但呢個評估往往淨係睇長度同字符種類，並不全面。真正判斷密碼係咪安全，需要考慮「熵」同「係咪已洩露」。

熵（Entropy）：密碼強度嘅科學度量

「熵」衡量密碼嘅不確定性，單位係比特。熵越高，暴力破解所需嘗試次數越多。

- **"123456"**：熵極低，瞬間破解。
- **"P@ssw0rd"**：睇落複雜，但出現喺所有字典攻擊列表入面，熵依然低。

- **"correct horse battery staple"**：四個隨機單詞，熵約 44 比特，破解難度遠超「P@ssw0rd」。
- **16 位隨機字符（含大小寫 + 數字 + 符號）**：熵約 100 比特，目前算力無法暴力破解。

結論：**「長度 + 隨機性」比「複雜符號」更重要**。呢個都係密碼管理器生成嘅隨機長密碼真正安全嘅原因。

HaveIBeenPwned：查你嘅密碼係咪已洩露

安全研究員 Troy Hunt 維護嘅 haveibeenpwned.com 收錄咗歷次大型數據洩露中流出嘅帳號同密碼。你可以：

1. 輸入郵箱，查你嘅帳號係咪喺某次洩露中被曝光。
2. 用 Pwned Passwords 功能查詢某個密碼係咪喺已知洩露庫中（密碼管理器同瀏覽器都已內置此查詢）。

如果一個密碼出現喺洩露庫中，**無論佢睇落幾複雜都必須立即更換**——因為字典攻擊會優先嘗試呢啲已知密碼。

k-Anonymity 查詢原理

擔心喺 HaveIBeenPwned 輸入密碼會再次洩露？唔使擔心。現代查詢採用「k-匿名」協議：瀏覽器淨係將密碼雜湊嘅前 5 位發畀伺服器，伺服器返所有以呢 5 位開頭嘅雜湊列表，本地再比對。即係話，伺服器永遠睇唔到你嘅完整密碼雜湊。1Password、Bitwarden 嘅「已洩露密碼檢測」都係咁樣工作嘅。

密碼管理器內置嘅監控功能

- **1Password Watchtower**：實時監控弱密碼、重複密碼、已洩露密碼、過期密碼、唔使用 2FA 嘅網站，並以紅/橙/黃分級展示。
- **Bitwarden Vault Health**：免費版即可查看弱密碼、重複密碼；付費版可檢測已洩露密碼（基於 HaveIBeenPwned）。
- **iCloud 鑰匙串「安全建議」**：iOS 設定密碼安全建議，會提示你邊啲密碼出現喺已知洩露中、邊啲重複使用。
- **Google 密碼管理器「密碼檢查工具」**：Chrome 內置，可以一鍵檢查已洩露、弱、重複密碼。

而家就查：你嘅密碼係咪安全

1. 打開 haveibeenpwned.com，輸入你嘅常用郵箱，睇下出現喺邊啲洩露事件中。
2. iPhone 用戶：設定密碼安全建議，逐條處理紅色警告。

3. Chrome 用戶：設定密碼檢查密碼，按提示修改。
4. 1Password/Bitwarden 用戶：打開 Watchtower/Health Reports，優先處理「已洩露」同「重複」項。

六、Passkey：無密碼登入嘅未來

2024-2026 年，密碼技術正在經歷一場革命——**Passkey（通行密鑰）**有望徹底淘汰傳統密碼。Apple、Google、Microsoft 三大平臺聯合推動，FIDO 聯盟同萬維網聯盟（W3C）背書，呢個係數十年嚟登入方式最大嘅一次變革。

乜嘢係 Passkey？

Passkey 基於 FIDO2 / WebAuthn 標準，用**設備上嘅密鑰對**替代密碼。原理可以簡化為：

- 你嘅手機生成一對密鑰：**公鑰**（註冊畀網站）+ **私鑰**（永遠存在你設備嘅安全晶片中）。
- 登入時，網站發一個「挑戰」，你嘅手機用私鑰簽名，再由網站用公鑰驗證。
- 簽名前需要你通過生物識別（Face ID / 指紋）或設備 PIN 確認身份。
- 整個過程中，**冇任何密碼喺網絡中傳輸**。

Passkey vs 傳統密碼

維度	傳統密碼	Passkey
抗釣魚	差，釣魚網站可竊取密碼	天然抗釣魚，私鑰淨係對真實域名簽名
伺服器洩露影響	密碼雜湊可能被破解	伺服器淨係存公鑰，洩露無影響
便捷性	需記憶/輸入/管理	Face ID/指紋一秒登入
重放攻擊	可能被中間人截獲重放	每次簽名一次性，無法重放

各平臺支援情況

- **Apple iCloud 鑰匙串 Passkey**：iOS 16+ / macOS Ventura+ 原生支援，Passkey 可以喺所有登入同一 Apple ID 嘅設備間同步，新設備登入網站時唔使重新創建。
- **Google 密碼管理器 Passkey**：安卓 + Chrome 原生支援，Passkey 存喺 Google 賬戶中，跨安卓設備同步；2023 年起 Google 個人帳號已支援用 Passkey 登入。
- **Microsoft**：Windows 11 / Edge 已支援 Passkey；微軟帳號、Outlook、Office 等服務陸續開放 Passkey 登入，並可以喺 Microsoft Authenticator 中管理。

- **1Password / Bitwarden**：已支援 Passkey 嘅創建、存儲、跨平臺同步，等你唔受限於單一生態。

跨設備同步係關鍵突破

早期 Passkey 嘅最大痛點係「綁定單台設備」——換手機就要重新創建。2023 年起 Apple、Google、Microsoft 實現咗 Passkey 嘅雲端同步：iCloud 鑰匙串同步跨 Apple 設備，Google 密碼管理器同步跨安卓設備。1Password 等第三方就打通咗跨生態同步。呢個意味住 Passkey 已具備替代密碼嘅實用性。

點樣喺手機上創建同使用 Passkey

iOS（iPhone）路徑：

1. 喺支援 Passkey 嘅網站（如 Google、微軟、亞馬遜、eBay、PayPal）帳號安全頁選擇「添加通行密鑰 / Add passkey」。
2. 系統彈出 Face ID / Touch ID 驗證視窗，通過後 Passkey 自動生成並儲存到 iCloud 鑰匙串。
3. 下次登入該網站時，輸入用戶名後選擇「使用 iPhone 登入」，Face ID 一嘟即可。
4. 管理入口：設定密碼搵到對應網站，可見 Passkey 條目。

安卓路徑：

1. 喺支援 Passkey 嘅網站帳號安全頁選擇「添加通行密鑰」。
2. Google 密碼管理器彈窗選擇你嘅 Google 帳號螢幕鎖驗證（指紋/圖案/PIN）。
3. Passkey 自動儲存到 Google 密碼管理器，跨同帳號安卓設備同步。
4. 下次登入時，選擇「使用 Passkey」，螢幕鎖一解即可。
5. 管理入口：安卓設定密碼、密鑰同自動填充密碼管理器 google.com。

Passkey 跨設備登入

如果你喺 Windows 電腦上登入一個淨係得 iPhone 上有 Passkey 嘅網站，可以掃碼授權：

1. 網站登入頁選擇「使用 Passkey」。
2. 選擇「使用其他設備」。
3. 電腦顯示二維碼，用 iPhone 掃碼。
4. iPhone 彈出 Face ID 驗證，確認後網站喺電腦上完成登入。

呢個過程通過藍牙 + 互聯網雙重加密通信，Passkey 私鑰唔會離開你嘅 iPhone。

而家就試：體驗無密碼登入

1. 登入 Google 帳號安全通行密鑰添加通行密鑰，體驗 Face ID/指紋登入。
2. 登入微軟帳號安全進階安全選項添加通行密鑰。
3. 喺淘寶、京東、eBay、PayPal 等已支援 Passkey 嘅服務上嘗鮮。
4. 保留傳統密碼作為後備，逐步遷移高價值帳號到 Passkey。

過渡期建議

Passkey 並未完全取代密碼——好多網站仍處於「Passkey + 密碼」並存階段。建議：高價值帳號優先開啟 Passkey，但**唔好立即刪除傳統密碼**，留作備用；同時繼續使用密碼管理器，因為唔係所有網站都支援 Passkey。將 Passkey 睇作「密碼管理器嘅強力補充」而唔係「立刻替換」。

七、硬件安全密鑰

如果你對安全嘅要求高過普通用戶——比如持有大量加密貨幣、係高管/記者/活動人士、或保護企業核心帳號——硬件安全密鑰係比短訊驗證碼更可靠嘅選擇。

乜嘢係硬件安全密鑰？

硬件密鑰係一個物理設備（形似 U 盤），內部安全晶片存儲你嘅私鑰，需要物理接觸或 NFC 先可以完成驗證。代表產品：

- **YubiKey**（Yubico 公司）：最主流，支援 FIDO2/WebAuthn、U2F、OTP、PIV、OpenPGP，USB-C / USB-A / NFC 等多種介面。5 系列 50-70 美元。
- **Google Titan**：Google 自家出品，USB-C + NFC / 藍牙，約 30-40 美元。
- **OnlyKey**：開源替代品，支援多種協議。
- **國產替代**：如飛天誠信 ePasskey，價格更親民。

點解硬件密鑰比短訊驗證碼安全？

- **抗釣魚**：同 Passkey 一樣，硬件密鑰淨係對真實域名響應，釣魚網站無法騙取驗證碼。
- **抗 SIM 劫持**：短訊驗證碼可被 SIM-swap 攻擊截獲，硬件密鑰唔使手機號。
- **抗中間人**：FIDO2 協議每次簽名都包含挑戰值，無法重放。
- **物理隔離**：私鑰永遠喺硬件晶片內，惡意軟件即使攻陷你嘅電腦都讀唔到。

適用場景

- 加密貨幣交易所帳號、錢包恢復。
- 企業 SSO、遠端辦公 VPN、SSH 登入伺服器。
- 郵箱、GitHub、社交媒體等「主帳號」——一旦被攻破可重置所有其他帳號。
- 記者、律師、高管等高敏感人羣。

點樣配置硬件密鑰作為二次驗證

以 Google 帳號 + YubiKey 為例：

1. 購買 YubiKey（建議一次買兩把，一把日常用、一把備用，存放喺唔同地點）。
2. 登入 myaccount.google.com 安全兩步驗證。
3. 選擇「添加安全密鑰」將 YubiKey 插入 USB-C 口或貼近手機 NFC 感應區。
4. 按提示觸摸 YubiKey 金屬觸點完成註冊。
5. 建議註冊第二把備用密鑰。
6. 喺「兩步驗證」設定入面，將硬件密鑰設為首選驗證方式，短訊驗證碼作為備用。

主流支援嘅網站：Google、GitHub、Microsoft、Apple ID、Facebook、Twitter、Dropbox、GitLab、AWS、Coinbase、Binance 等。

雙密鑰策略

硬件密鑰最大嘅風險係「遺失」。建議：(1) 同時註冊兩把唔同嘅密鑰；(2) 一把隨身攜帶（鑰匙扣），一把存放喺屋企保險櫃或銀行保管箱；(3) 部分網站允許將「備用恢復碼」列印出嚟離線保存。咁樣即使主密鑰遺失，都唔會被永久鎖喺帳號外。

購買渠道要謹慎

務必由官方店鋪或授權代理購買，唔好買二手或來源不明嘅密鑰——理論上預置過嘅硬件可能已被植入後門。YubiKey 嘅韌體喺出廠後無法更新，但 Yubico 會持續推送安全公告，關注佢官網即可。

八、家庭/團隊密碼共享

家庭場景下，Netflix、家庭 Wi-Fi、路由器後台、智能門鎖密碼需要全家共用；團隊場景下，伺服器、數據庫、社交媒體帳號需要多人協作。**用微信發密碼、寫喺共享文檔入面都係高危做法**——一旦聊天記錄或文檔權限洩露，密碼就洩露咗。

1Password 嘅「共享保險庫」（Vault）

1Password 係家庭同團隊共享嘅標杆產品。佢嘅核心機制係**多個保險庫**：

- **個人保險庫**：淨係你可以睇，如銀行、郵箱等私人帳號。
- **共享保險庫**：家人或團隊成員都可以睇，如 Netflix、家庭 Wi-Fi、共享信用卡。
- **權限分級**：團隊版可設定某些成員「淨係查看」或「淨係查看但不編輯」。

家人或團隊成員唔使共享主密碼，各自用自己嘅帳號登入，但可以睇到被授權嘅共享保險庫內容。密碼修改後所有成員實時同步。

Bitwarden 嘅「組織」功能

Bitwarden 嘅免費版支援個人密碼共享（通過「發送 Send」功能限時分享單條密碼）；付費版可創建「組織」，將若干條目共享畀家庭成員或團隊同事，權限可細分。Bitwarden 仲提供自託管選項，適合對數據主權要求高嘅企業。

家庭場景：共享 Netflix、Wi-Fi 等密碼嘅安全方式

- **Netflix / 影片網站**：用 1Password 家庭版嘅「共享保險庫」存登入密碼，家庭成員各自登入即可，唔使喺羣入面發。
- **家庭 Wi-Fi 密碼**：iOS 設備間可通過 AirDrop 直接共享 Wi-Fi 密碼畀來訪嘅客人，唔使讀出嚟；安卓 12+ 都有「附近分享 Wi-Fi」功能。
- **智能門鎖、智能家電**：盡量用廠商 App 嘅「家庭成員」功能為每人單獨創建帳號，而唔係共享一個管理員帳號。
- **家用路由器後台**：管理員密碼存到 1Password 共享保險庫，訪客 Wi-Fi 單獨開一個 SSID。

1Password 家庭版嘅性價比

1Password Families 約 4.99 美元/月，支援最多 5 位家庭成員，每人都有獨立嘅私人保險庫 + 4 個共享保險庫。計落人均唔到 1 美元/月，係目前家庭共享體驗最好嘅方案。Bitwarden 嘅家庭版 10 美元/年支援 6 人，性價比更高，但介面體驗略遜。

團隊場景：伺服器/數據庫密碼共享

- **1Password Teams / Business**：可按部門、項目組創建多個保險庫，細粒度權限控制，所有訪問行為可審計。
- **Bitwarden Teams / Enterprise**：開源方案，可自託管，企業版支援 SSO 整合、目錄同步。
- **職責分離原則**：管理員密碼唔應該淨係得一人掌握——設定 2-3 名「緊急聯絡人」，確保關鍵人員離職/休假時業務不中斷。

- **離職流程**：員工離職時立即撤銷其密碼管理器訪問權限，必要時旋轉所有佢接觸過嘅高敏感密碼。

緊急訪問（Emergency Access）：萬一你出事

想像一個場景：你獨自管理家庭所有重要帳號，但意外住院或不幸離世，家人無法登入銀行、郵箱、保險帳號。「**緊急訪問**」**功能**正係為此設計——授權一位信任嘅家人/朋友，喺你無法響應時獲取你嘅密碼庫。

Bitwarden 緊急訪問設定：

1. 登入 Bitwarden Web Vault 設定緊急訪問。
2. 添加信任聯絡人嘅郵箱，邀請對方接受。
3. 設定「等待期」（建議 7-14 天）：由對方發起請求到你確認嘅時間窗口。
4. 如果你喺等待期內唔響應（拒絕或唔操作），到期後對方自動獲得訪問權。
5. 你隨時可以手動拒絕請求或撤銷緊急訪問權限。

1Password 緊急訪問：

1. 1Password 唔直接叫「緊急訪問」，但通過「家庭/團隊組織」功能可實現——將家人加入家庭版後，將部分條目放入「共享保險庫」。
2. 對個人版用戶，建議將主密碼告知一位高度信任嘅家人，或寫入密封信封存放於保險櫃。
3. 1Password 仲提供「恢復碼」，開啟雙重認證時務必列印保存。

而家就設：為家人留一把「備用匙」

1. 選定一位信任嘅家人作為「緊急聯絡人」。
2. 喺密碼管理器中設定緊急訪問（Bitwarden）或加入家庭共享保險庫（1Password）。
3. 將主密碼 + 恢復碼寫喺紙上，封入信封，存放於屋企保險櫃或告知可信家人。
4. 每半年回顧一次，確認緊急聯絡人係咪仍然合適。

九、密碼管理器使用最佳實踐

- **設定強主密碼**：主密碼係最後一道防線，務必設定一個長而且複雜嘅密碼，並牢記。
- **啟用雙重認證**：為密碼管理器帳號開啟雙重認證，增加一層保護。
- **定期檢查弱密碼**：大多數密碼管理器有「安全審計」功能，幫你搵出弱密碼同重複密碼。
- **唔好忘記主密碼**：主密碼無法搵返，忘記就意味住遺失所有密碼。建議寫喺紙上，放喺安全嘅地方。
- **緊急訪問**：設定緊急聯絡人，萬一你無法訪問，信任嘅人可以獲取你嘅密碼。

重要提醒

密碼管理器雖然方便，但都有風險：如果主密碼洩露，所有密碼都暴露咗。所以主密碼絕對唔可以同其他密碼相同，都唔可以寫喺容易被搵到嘅地方。另外，建議選擇有良好安全記錄嘅服務商。

而家就試：開始使用密碼管理器

1. 選擇一個密碼管理器（推薦 Bitwarden 或系統自帶）。
2. 註冊帳號，設定一個強主密碼。
3. 匯入現有密碼（如果有嘅話）或開始添加新密碼。
4. 安裝瀏覽器插件或手機 App，體驗自動填充功能。
5. 檢查安全審計報告，搵出需要改進嘅密碼。

三、進階篇（第 15-20 章）

第十五章：電池科學與續航優化

電池就係你手機嘅「能量心臟」。呢章帶你搞掂電池嘅小脾氣，學識科學嘅充電同續航技巧，徹底告別電量焦慮。

一、電池係點回事？循環與健康度

手機電池（鋰離子或鋰聚合物）好似一塊能量海綿：充電就係吸水（存電），放電就係揸水（用電）。有幾個特點值得記住：

- **隨用隨充**：同舊電池唔同，佢冇記憶效應，唔使一定要用完再充。
- **怕熱**：高温係電池頭號殺手。
- **會老化**：用得耐，最大容量就會下降，呢個係正常現象。

核心指標：循環次數與健康度

- 一次循環唔係充一次電。比如今日用 60% 充滿，聽日用 40% 充滿，總消耗 100% 電量先算一次循環。
- 電池健康度就係當前最大容量佔全新時嘅百分比。低於 80% 時，你會明顯感覺續航變短。
- **點樣睇？**iPhone：設定 > 電池 > 電池健康與充電；安卓：一般去設定入面搜電池健康。

二、充電玄學？快充、無線充與黃金法則

- **快充傷電池？**用原裝或認證嘅快充配件，傷害好細。手機同充電頭有温控，過熱會自動降功率。
- **無線充電**：方便但效率較低，發熱更明顯。適合成晚充電。
- **黃金養生法則**：盡量等電量保持喺 20%-80% 之間，唔好成日等佢餓住（0%）或撐住（100%）。

iPhone「80% 充電限制」功能——將黃金法則變成系統設定

iPhone 15/16 系列（iOS 17 以上）已經將「20%-80% 法則」做成咗系統功能。開啟後，手機充到 80% 會自動停止充電，從而大幅延緩電池老化。設定路徑：

設定 > 電池 > 電池健康與充電 > 充電優化 > 80% 限制
三個選項嘅含義：(1) **不限**——充到 100%；(2) **優化電池充電**——學習你嘅作息，淨係喺凌晨最後一段延遲到 100%；(3)**80% 限制**——直接停喺 80%，最護電池。長期喺屋企/辦公室有充電條件嘅人，強烈建議選 80% 限制；經常出差、需要滿電出門嘅人，可以選優化電池充電。

GaN 充電器——出差黨嘅好拍檔

你可能注意到而家嘅快充頭越做越細，背後功臣係 **GaN（氮化鎵）**呢種新型半導體材料。相比傳統矽基充電器，GaN 充電器有幾個明顯優勢：

- **體積細**：同等功率下，GaN 充電器可以做到傳統矽基嘅 1/2 甚至 1/3 大細，65W 功率嘅 GaN 頭比蘋果舊嘅 5W 頭仲細。
- **效率高、發熱低**：氮化鎵嘅開關損耗遠低於矽，能量轉換效率可達 95% 以上，充電時唔容易燙手。
- **適合出差**：一個 65W 或 100W 嘅 GaN 多口充電器，可以同時畀手機、平板、筆記本充電，比帶三塊「磚頭」輕便得多。
- **選購建議**：認準支援 PD 協議嘅 GaN 充電器（65W 起步可充筆電），知名品牌如倍思、安克、努比亞、小米等均可。注意核對數據線係咪匹配（高功率需用 E-Marker 晶片嘅 Type-C 線）。

三、快充協議全解析：點解你嘅充電器充唔快？

你可能遇過呢種情況：明明手機支援快充，換咗充電器卻變得好慢。呢個係因為快充唔係簡單嘅「功率大就得」，而係需要充電頭、數據線、手機三方「講同一種語言」——呢個就係快充協議。

快充嘅基本原理

充電功率 = 電壓 × 電流（$P = U \times I$）。要提升功率，要麼提高電壓，要麼提高電流，或者兩者都提高。唔同協議選擇咗唔同嘅技術路線。

快充嘅工作流程：

1. **握手階段**：手機同充電頭通過數據線通信，交換各自支援嘅協議同功率資訊。
2. **協商階段**：雙方確定一個共同支援嘅最高功率檔位。
3. **充電階段**：充電頭按協商好嘅電壓電流輸出，手機端進行電源管理。
4. **調整階段**：隨住電量增加，手機會逐步降低充電功率，保護電池。

主流快充協議詳解

USB PD（Power Delivery）——通用協議之王

- **特點**：USB-IF 組織制定嘅官方標準，覆蓋範圍最廣。PD 3.0 支援最高 100W（20V/5A），PD 3.1 擴展到 240W（48V/5A）。
- **電壓檔位**：5V、9V、15V、20V，PD 3.1 仲支援 28V、36V、48V。
- **兼容性**：iPhone 全系、谷歌 Pixel、部分安卓旗艦、筆記本電腦、Switch 遊戲機都支援 PD 協議。
- **優勢**：通用性強，一個充電頭可以畀多種設備充電；支援 PPS（可編程電源），可以精細調節電壓電流。
- **劣勢**：需要 Type-C 介面同專用晶片，成本較高。

高通 QC（Quick Charge）——安卓陣營嘅元老

- **QC 3.0**：支援 3.6V-20V 以 0.2V 為步進嘅可調電壓，最高 18W。
- **QC 4.0/4+**：兼容 USB PD 協議，支援最高 27W，加入雙路充電技術。
- **QC 5.0**：支援最高 100W+，0-50% 僅需 5 分鐘，大幅改善發熱。
- **兼容性**：使用高通驍龍處理器嘅手機大多支援，但各品牌通常會優先使用自己嘅私有協議。

OPPO VOOC/一加 WARP/真我 Dart——低壓大電流嘅代表

- **技術路線**：低壓大電流（5V/4A 起步），發熱更細，效率更高。
- **VOOC 1.0**：20W（5V/4A），2014 年發布，開創快充時代。
- **SuperVOOC**：50W-65W，採用雙電芯串聯設計。
- **SuperVOOC 2.0**：65W，30 分鐘充滿 4000mAh 電池。
- **150W/240W 超級閃充**：目前最快嘅量產快充技術。
- **特點**：需要專用充電頭同數據線（有額外觸點），唔兼容其他協議。

華為 SuperCharge——高壓快充嘅演進

- **FCP（Fast Charge Protocol）**：9V/2A，18W，兼容 QC 2.0。
- **SCP（Super Charge Protocol）**：支援更高功率，由 22.5W 到 66W 再到 100W。
- **技術特點**：採用電荷泵技術，將高壓轉換為低壓大電流輸入電池，效率高達 98%。

- **兼容性**：華為/榮耀手機專用，部分充電器支援 PD 協議。

小米澎湃快充——自研協議嘅崛起

- **澎湃 P1 晶片**：小米自研嘅快充管理晶片，支援 120W 有線快充。
- **澎湃 G1 晶片**：電池管理晶片，同 P1 配合實現更高效嘅快充。
- **技術特點**：採用雙電芯串聯 + 電荷泵方案，支援多種協議兼容。

蘋果快充——保守但穩定

- **協議**：淨係支援 USB PD 協議。
- **功率**：iPhone 8 及以後支援快充，最高約 27W（iPhone 15 Pro Max 可達 27W+）。
- **特點**：功率相對保守，但充電曲線優化好，發熱控制出色。
- **配件要求**：需要 USB-C to Lightning 線（或 USB-C to USB-C 線）同 PD 充電頭。

快充協議對比表

協議	最高功率	技術路線	兼容性
USB PD 3.1	240W	高壓/多檔位	最廣泛
QC 5.0	100W+	高壓可調	高通機型
VOOC/Dart	240W	低壓大電流	OPPO 系專用
SuperCharge	100W	電荷泵	華為/榮耀
蘋果快充	~27W	PD 協議	iPhone/iPad

無線充電協議

無線充電都有自己嘅協議標準：

- **Qi 標準**：WPC（無線充電聯盟）制定，最通用嘅標準。基礎 Qi 支援 5W-15W。
- **Qi2 標準**：2023 年發布，引入磁吸定位（類似蘋果 MagSafe），支援 15W+。
- **私有協議**：各品牌有自己嘅高速無線充電，如小米 50W 無線充、華為 50W 無線充等，需要專用充電器。

快充選購指南

- **睇協議匹配**：購買充電器時，確認佢支援你手機嘅快充協議。第三方充電器通常會標註支援嘅協議。
- **睇數據線**：大功率快充需要專用數據線。普通數據線可能只支援 60W 以下。
- **睇認證**：蘋果用戶建議選擇 MFi 認證嘅數據線；安卓用戶選擇品牌原裝或知名第三方。
- **睇發熱**：快充時手機發熱係正常嘅，但如果燙手就唔正常喇，建議暫停充電或更換充電器。

快充安全提示

- 避免邊充邊玩大型遊戲，會加劇發熱。
- 唔好使用劣質充電器同數據線，可能導致安全隱患。
- 充電時除掉厚重嘅手機殼，幫助散熱。
- 如果充電器或手機異常發熱、有異味，立即停止使用。

四、電池新技術：矽碳負極電池

如果你留意 2024-2026 年嘅旗艦手機發布會，「矽碳負極」呢個詞幾乎成咗高端電池嘅代名詞。呢個係鋰電池負極材料嘅一次重要升級，原理其實唔複雜：

- **傳統石墨負極**：手機電池一直用嘅係石墨做負極，技術成熟、穩定，但能量密度已接近理論上限（約 372mAh/g）。
- **摻入矽材料**：矽嘅理論比容量係石墨嘅 10 倍左右（約 4200mAh/g）。將少量矽摻入石墨入面做成「矽碳複合負極」，就可以顯著提升整塊電池嘅能量密度。
- **能量密度提升**：同等體積下，矽碳負極電池嘅容量可以提升約 **15%-20%**。呢個正係 2024-2026 年旗艦機可以喺 7-9mm 機身入面塞進 5500-6000mAh 大電池嘅關鍵原因。

邊啲機型用上咗？

- **榮耀**：Magic 系列率先商用矽碳負極，Magic6 Pro、Magic7 系列、Magic V3 摺疊屏均採用。
- **小米**：小米 14 Ultra、小米 15 系列搭載新一代矽碳負極電池。
- **vivo**：X100 Ultra、X200 Pro 系列使用高密度矽碳負極方案。
- **華為/OPPO 等**：2025-2026 年旗艦普遍跟進，矽碳負極正在由「黑科技」變成高端機嘅標配。

對用戶嘅實際影響：矽碳負極帶嚟嘅最大好處係「同體積電池容量更大」——手機可以做得更薄，同時續航唔縮水；或者機身不變厚，但電池由 4500mAh 漲到 5500mAh。換句話講，你唔使為續航付「厚重」嘅代價喇。需要注意嘅係，矽碳負極電池喺低温同滿充管理上仍需配合快充晶片做優化，呢個都係廠商建議長期保持 80% 充電嘅原因之一。

五、呢個先係真正嘅省電技巧

- **螢幕（耗電王）**：調低亮度、縮短自動鎖屏時間。OLED 屏手機開深色模式可以省唔少電。
- **訊號差**：喺電梯、地下室，手機會拼命搜訊號，耗電極快。
- **5G**：如果訊號唔好，手動切返 4G，續航會好好多。

- **定位服務**：非導航類 App，定位權限設為「僅使用時允許」。
- **限制後台活動**：iPhone 喺設定 > 通用 > 後台 App 刷新；安卓喺應用電池設定入面。

嚟檢查一下：畀你嘅電池做個體檢

1. 查看健康度：搵到並記低你嘅電池最大容量。
2. 分析耗電詳情：睇電池設定入面嘅圖表同 App 列表，搵出耗電元兇。
3. 制定對策：限制後台耗電異常嘅 App。

第十六章：帳號體系與雲服務

我有個朋友，舊年手機被偷咗。佢第一時間諗嘅唔係「手機幾多錢」，而係「我入面嘅相仲可唔可以搵返」。好可惜，佢冇開雲備份。手機係身外之物，但帳號入面嘅數據，係真係唔見咗就再也返唔到嚟嘅嘢。呢章我哋就嚟傾下，點樣畀你嘅數碼生活上個真正嘅保險。

一、你嘅手機帳號，到底管住幾多嘢？

好多人覺得帳號就係用嚟下載 App 嘅。其實，一個手機帳號背後牽住嘅嘢，比你想像嘅要多得多：

- **通訊錄**：你存咗幾年甚至十幾年嘅電話號碼。
- **相同影片**：由細路出生到畢業典禮，由第一次約會到結婚旅行。
- **備忘錄**：你隨手記嘅靈感、日記、帳號密碼（對，好多人將密碼記喺備忘錄入面）。
- **App 購買記錄**：花咗幾百蚊買嘅付費 App 同內購。
- **查找手機**：手機唔見咗之後，呢個係唯一可以幫你定位、鎖機、遠端抹數據嘅工具。

如果你用嘅係蘋果手機，以上所有嘢都綁喺你嘅 Apple ID 上。如果你用嘅係安卓，那就綁喺你嘅華為/小米/OPPO/vivo 帳號上，仲有你嘅谷歌帳號。呢啲帳號就係你數碼生活嘅「戶口本」，唔見咗戶口本，好多嘢就講唔清喇。

二、三個必須做嘅事，而家就做

1. **開啟雙重認證**：唔管你用嘅係蘋果定安卓，第一件事就係去帳號設定入面將「雙重認證」或「兩步驗證」打開。呢樣嘢好似畀你嘅門加咗一把反鎖——淨係有密碼唔夠，仲要你嘅手機收個驗證碼。就算有人喺網上猜中咗你嘅密碼，冇你嘅手機，佢都入唔到。
2. **綁定一個可靠嘅恢復方式**：雙重認證好安全，但都有個風險——你自己嘅手機唔見咗點算？所以系統會提示你設定「恢復密鑰」或「備用郵箱/手機號」。呢個係你喺緊急情況下攞返帳號嘅唯一途徑，最好記喺紙上，唔好存喺手機入面。
3. **確認「查找手機」已開啟**：呢個功能喺手機唔見咗之後可以幫你定位、鎖定、播放聲音、甚至遠端擦除所有數據。係最後一道防線。好多人買手機嗰陣跳過咗呢一步，後面想用嘅時候發現冇開，就遲喇。

三、iCloud：蘋果用戶嘅「空氣」

用蘋果手機嘅人，遲早要面對一個靈魂拷問：iCloud 空間滿咗點算？

蘋果免費畀你 5GB，講真，呢 5GB 連備份一次都唔夠。所以大部分人最後都會收到嗰個煩人嘅彈窗：「iCloud 儲存空間已滿」。呢個時候你有三個選擇：

- **清理**：去設定你嘅名 iCloud 管理儲存空間，睇下係邊個佔咗你嘅空間。一般嚟講，相同備份係大頭。可以先刪掉唔需要嘅舊備份。
- **升級**：50GB 每個月 6 蚊，200GB 每個月 21 蚊。個人覺得 50GB 對大多數人嚟講夠，主要係用嚟備份同存相嘅縮圖。
- **曲線救國**：如果你唔想花錢，可以將相定期匯出到電腦上，然後關掉 iCloud 相同步。但係咁樣嘅話，你就失去咗相自動備份嘅便利性。

一個重要嘅提醒：iCloud 相圖庫唔係「網盤」——佢係「同步」。你喺手機上刪咗一張相，iCloud 上都會刪。如果你想要嘅係「備份到雲端然後清空手機」，咁 iCloud 相唔係你要嘅嘢，你需要嘅係 iCloud 雲備份，或者第三方嘅網盤。

iCloud+：付費用戶先可以享受嘅私隱三件套

只要升級到任意付費 iCloud 檔位（50GB/200GB/2TB 等），就自動成為 **iCloud+** 用戶，解鎖三個私隱保護功能：

- **Private Relay（私密中繼）**：蘋果版嘅「VPN」。Safari 瀏覽嘅流量經蘋果伺服器中轉後再發往網站，冇人（包括蘋果）可以同時睇到你係邊個、你訪問咗乜嘢。適合喺公共 Wi-Fi 下保護私隱。設定路徑：設定 > 你嘅名 > iCloud > 私密中繼。
- **Hide My Email（隱藏郵件）**：喺網站/App 註冊時生成一次性隨機郵箱地址，郵件會自動轉發到你嘅真實郵箱。唔想再收呢個網站嘅郵件，直接關掉呢個隨機地址就得，真實郵箱唔暴露。設定路徑：設定 > 你嘅名 > iCloud > 隱藏郵件。
- **自訂電郵域名**：可以用你自己嘅域名（比如 @ 你嘅姓.com）收發 iCloud 郵件，適合個人品牌或家庭郵箱管理。

提示：Private Relay 喺某些國家/地區唔可用，國內用戶可能需要其他方案補充。

iCloud 高級數據保護（ADP）——真正嘅端到端加密

2022 年底蘋果推出咗 **iCloud 高級數據保護（Advanced Data Protection，ADP）**，將端到端加密（E2EE）由原本嘅健康、密碼等少數數據，擴展到**大部分 iCloud 數據**——相、備忘錄、iCloud 雲備份、設備資訊等共 23 類數據。

- **乜嘢係 E2EE**：加密密鑰淨係存在你嘅設備上，連蘋果伺服器都冇。就算蘋果被黑客攻破，或者執法部門要求調取，蘋果都拎唔出可以解密嘅明文數據。
- **開啟路徑**：設定 > 你嘅名 > iCloud > 高級數據保護，按提示設定恢復聯絡人/恢復密鑰（**必做**，唔見咗密鑰你嘅數據都救唔返）。

- **注意事項**：(1) 開啟後務必妥善保管恢復密鑰，否則帳號遺失時無法搵返數據；(2) 共享相簿、iCloud 共享圖庫等少數功能唔參與 E2EE；(3) 中國大陸區 Apple ID 暫唔支援 ADP，需要切換到其他地區帳號。

一句建議：如果你嘅帳號入面有重要私隱數據，而且帳號地區支援，強烈建議開啟 ADP——呢個係普通人可以獲得嘅最強雲端數據保護。

四、同步同備份，唔好搞混

呢兩個詞聽落差唔多，但後果完全唔同：

- **同步**係雙向嘅：你喺手機上刪咗，雲端都刪咗；你喺雲端刪咗，手機上都跟住刪。iCloud 相、通訊錄、備忘錄呢啲都係同步。好處係多設備之間保持一致，壞處係「刪咗就真係冇喇」。
- **備份**係單向嘅：手機將當前狀態影一張「快照」存到雲端。之後你喺手機上刪咗嘢，雲端備份入面嗰份仲喺度。iCloud 雲備份、電腦本地備份，都係備份。好處係數據更安全，壞處係佔空間，而且唔可以實時更新。

五、安卓用戶嘅帳號矩陣

安卓呢邊嘅情況比蘋果複雜啲，因為你嘅數據可能分散喺好幾個帳號入面：

- **品牌帳號**（華為/小米/OPPO/vivo）：管住你嘅雲空間、查找手機、應用商店、錢包。每個品牌都有自己嘅雲服務，通常係基礎免費 + 擴容收費嘅模式。
- **谷歌帳號**：如果你用 Gmail、Google 相、Google 通訊錄，呢啲嘢都喺谷歌帳號入面。谷歌免費畀 15GB，比蘋果大方，但都係同 Gmail、Drive 共用嘅。
- **微信/QQ**：佢哋係獨立於手機系統嘅。你嘅聊天記錄同微信收藏，既唔喺品牌帳號入面都唔喺谷歌帳號入面，淨係存喺微信自己嘅伺服器上。

所以安卓用戶做備份嘅時候，要分別檢查：品牌雲備份、谷歌備份、微信聊天記錄備份，三者缺一不可。

六、家庭共享：等家人都受益

蘋果嘅「家人共享」可以等你同家人共享 App 購買、音樂、電影、iCloud 空間，仲可以共享位置。最多可以加 6 個人。設定方法係：設定你嘅名家人共享。華為同小米都有類似嘅功能，喺帳號設定入面可以搵到「家庭共享」或「家庭雲」。

一個好實用嘅場景：畀老豆老母嘅手機上設定好家人共享，咁你就可以隨時睇到佢哋嘅位置（前提係佢哋同意），萬一老人走失咗或者手機聯絡唔上，可以有個定位。

帳號被盜嘅後果比你想像嘅要嚴重

如果黑客攞到你嘅帳號密碼，佢可以：遠端鎖定你嘅手機並勒索贖金、查看你嘅相同通訊錄、用你嘅支付方式買嘢。更麻煩嘅係，如果你嘅帳號綁定咗太多嘢，搵返帳號本身就係一場噩夢。所以，雙重認證，而家就去開。

實操：畀你嘅帳號做一次全面體檢

1. 打開手機設定，進入帳號頁面，確認雙重認證已開啟。
2. 檢查「受信任嘅電話號碼」同「恢復密鑰」是否已設定，密鑰記喺紙上。
3. 進入 iCloud 或雲空間管理，睇下邊啲數據喺同步，關閉唔需要嘅。
4. 確認「查找我嘅手機」已開啟。
5. 如果你用嘅係安卓，檢查品牌雲備份、谷歌備份、微信備份三者係咪都覆蓋到。

第十七章：手機與其他設備嘅聯動

你手入面嘅手機，其實係一個超級遙控器兼數據中心。佢唔淨係可以打電話，仲可以將電腦、電視、手錶、音箱、汽車串埋一齊，等呢啲設備唔再係孤島。呢章我哋就嚟傾下點樣等佢哋「組隊打怪」。

一、與電腦協同：手機同電腦唔再係兩個世界

講真，我以前最煩嘅就係手機影咗一張相，諗住擺到電腦上執下，仲要搵數據線、等傳輸。後來發現，其實根本唔需要數據線，好嘅聯動比數據線快十倍。

- **蘋果生態（AirDrop + 通用剪貼板）**：如果你用 iPhone 配 Mac，隔空投送係你第一個應該學識嘅功能。喺手機上撳分享隔空投送揀你嘅 Mac，相瞬間就過咗，唔使流量唔使 Wi-Fi，用嘅係藍牙同 Wi-Fi 直連。仲有個容易被忽略嘅功能叫「通用剪貼板」——手機上複製一段文字，喺 Mac 上直接貼上就出嚟，簡直好似魔法咁。
- **安卓 +Windows（連接至 Windows）**：微軟有個叫「連接至 Windows」嘅 App，三星手機內置，其他安卓手機去應用商店下載就得。配對之後你可以喺電腦上直接睇手機通知、回 SMS、揭相，甚至接打電話。我自己用咗兩年，開會嗰陣唔使一路低頭睇手機，電腦螢幕上彈個通知就夠。
- **品牌多屏協同**：華為嘅「多屏協同」同小米嘅「小米妙享」更狠——直接將手機螢幕投到電腦上，你可以用滑鼠拖拽檔案由手機到電腦，都可以喺電腦上碌抖音、回微信。跨設備複製貼上、檔案拖拽都得，體驗非常絲滑。

二、投屏到電視：全家一齊睇

過年返屋企嗰陣，我堂妹想喺電視上放佢手機入面嘅畢業旅行影片。折騰咗半天，最後發現其實非常簡單：

- **iPhone 電視**：由右上角下拉控制中心，撳嗰個兩個方框疊埋一齊嘅圖示（螢幕鏡像），搵到你嘅電視名稱，一撳就連上。前提係電視同手機連嘅同一個 Wi-Fi，而且電視支援 AirPlay（大部分智能電視都支援）。
- **安卓電視**：各家叫法唔同，華為叫「無線投屏」，小米叫「投射」，OPPO/vivo 叫「多屏互動」。一般喺快捷開關面板入面可以搵到，搵唔到就去設定入面搜「投屏」。

- **影片 App 直接投**：如果你淨係想喺電視上睇愛奇藝、B 站、YouTube，其實更簡單——喺播放介面搵一個矩形帶 Wi-Fi 符號嘅細圖示，撳一下，揀電視，影片就過去喇。呢個時候手機可以做第二啲，影片喺電視上繼續放，互不影響。
- **投屏唔成功點算**：90% 嘅情況係冇連同一個 Wi-Fi。仲有 10% 係電視嘅投屏功能冇開，去電視設定入面搵「多屏互動」或「AirPlay」開關。

三、與智能手錶/手環：健康管家長喺手腕上

我畀我老母買咗個手環，佢一開始話「我又唔運動買呢個做乜」。結果用咗兩個禮拜，佢發現自己每日淨係瞓五個幾鐘，開始主動調整作息。呢個就係手錶/手環最實用嘅價值——將你睇唔到嘅健康數據變成睇到嘅數字。

- **通知過濾器**：手機喺袋入面或口袋入面震個不停嗰陣，抬起手腕睇一眼——係老細嘅微信定拼多多嘅促銷，一秒判斷，唔值得掏手機嘅消息就等佢去。
- **健康數據同步**：手錶 24 小時監測心率、血氧、睡眠、步數，呢啲數據會自動匯總到手機上嘅健康 App 入面。久而久之你可以睇到自己嘅趨勢——呢個月係咪行得太少，呢個禮拜睡眠質素有冇變好。
- **交通卡/門禁卡同步**：喺手機上綁好嘅公交卡同門禁卡，大部分品牌會自動同步到手錶上。搭地鐵嗰陣唔使掏手機，手腕一靠閘機就過咗，真係會有種「未來已來」嘅感覺。
- **搵手機神器**：手錶上一般都有「搵手機」功能，撳一下手機就會響。我自己用呢個功能嘅頻率遠超預期——手機隨手一放就忘記喺邊。

四、與智能家居：手機係屋企嘅「總掣」

智能家居聽落好玄，但講穿咗就三件事：用手機遠端控制、設定自動化規則、語音操控。講下我自己嘅幾個實際用法，你就知呢樣嘢幾實用：

- **冷氣**：夏天返屋企路上提前用手機將冷氣打開，入門就係涼爽嘅。花一百幾蚊買個冷氣伴插座就可以實現，唔使換冷氣。
- **燈**：瞓前躺喺牀，唔願起身熄燈，手機上撳一下或者嗌聲「嘿 Siri/小愛同學，熄燈」。
- **掃地機械人**：出門前手機上撳一下「開始清掃」，返到嚟地就乾淨喇。仲可以設每日定時掃。
- **鏡頭**：屋企有老人細路或者寵物嘅，手機上隨時可以睇一眼屋企嘅情況。

揀邊個生態？一句話總結：

- 用 iPhone 優先選支援 Apple HomeKit 嘅設備（貴啲但私隱保護好）。
- 設備多求性價比小米米家生態最全，燈、插座、掃地機乜都有。
- 用華為手機優先鴻蒙智聯，同華為設備體驗最好。

五、藍牙耳機與音箱：無線嘅自由

由有線耳機換成藍牙耳機，對我嚟講係手機體驗升級入面最明顯嘅一個。再都唔會由袋入面掏出一團亂麻，都唔使擔心線掛喺門把手上。

- **配對好簡單**：耳機第一次開機一般自動進入配對模式（閃燈），手機打開藍牙就可以搜到，撳一下就連上。之後每次攞出嚟自動連，乜都唔使做。
- **同品牌有加成**：AirPods 配 iPhone 會自動彈窗顯示電量，仲可以喺多個蘋果設備之間無縫切換。華為 FreeBuds 配華為手機、小米耳機配小米手機都有類似嘅彈窗同優化。
- **藍牙音箱**：家庭聚會、戶外露營嗰陣，一個藍牙音箱可以等氣氛翻倍。連上之後，手機就係你嘅 DJ 臺。
- **注意延遲**：打機嗰陣如果用藍牙耳機，可能會有輕微延遲（槍聲響咗半秒先聽到槍聲）。如果對延遲敏感，要麼用有線耳機，要麼買支援「低延遲模式」嘅藍牙耳機。

UWB 超寬頻技術——比藍牙更精準嘅「空間感」

除咗藍牙，新款手機入面仲藏住一顆 UWB（Ultra-Wideband，超寬頻）晶片。佢嘅能力係**精確測距同空間定位**——可以計出另一個設備離你幾遠、喺邊個方向，精度可達釐米級。藍牙淨係可以話畀你知「喺附近」，UWB 可以話畀你知「喺你左前方 1.2 米」。常見應用：

- **AirTag/查找精確**：iPhone 查找 AirTag 時顯示「7 米遠、向左行」嘅箭嘴導航，靠嘅就係 UWB。普通藍牙淨係可以話「喺唔喺範圍內」。
- **數碼車匙**：搭載 UWB 嘅 iPhone 同安卓旗艦，可以當車匙行近車輛自動解鎖、行遠自動鎖車，仲可以區分司機同乘客位置，防中繼破解。
- **室內導航**：商場、機場入面通過 UWB 信標實現「過道級」導航，比 GPS 準得多。
- **支援嘅機型**：iPhone 11 及以後機型均帶 UWB 晶片（U1/U2）；安卓陣營小米、三星、vivo、OPPO 部分旗艦也已搭載。

LE Audio 與 Auracast——藍牙音頻嘅新一代標準

用咗二十年嘅經典藍牙音頻（A2DP）正在被 **LE Audio**（低功耗音頻）取代，呢個係藍牙 5.2 以後嘅新一代音頻標準：

- **更省電、音質更好**：採用全新嘅 LC3 編碼，比老標準嘅 SBC 音質更好、功耗更低，耳機續航明顯延長。
- **雙路音頻**：左右耳塞獨立傳輸，單耳模式切換唔掉線。
- **Auracast 廣播音頻**：LE Audio 嘅招牌功能。一個手機/電視可以同時向**無數副**耳機廣播音頻——機場候機廳、博物館、健身室、課室入面都可以用一個發射源，多人用自己嘅耳機聽同一畫面。
- **現狀**：2024-2026 年新上市嘅旗艦耳機同手機普遍支援 LE Audio，舊設備需硬件升級

無法軟件解鎖。買耳機時可以留意係咪支援 LE Audio/Auracast 標識。

六、與車載系統：揸車時嘅安全拍檔

揸車睇手機係違法嘅，都係攞命嘅。但有時候確實需要導航、接電話、放音樂。呢個時候 CarPlay（蘋果）同 Android Auto（安卓）就派上用場喇。

- **點樣連**：如果你嘅車支援（近幾年大部分新車都支援），用數據線將手機插到車上嘅 USB 口，車載螢幕會自動切換成 CarPlay 或 Android Auto 介面。部分新車支援無線連接，更省事。
- **可以做乜**：導航（比車載導航好用得多）、打電話（軚盤接聽）、聽歌（手機入面嘅音樂 App 直接投到車上）、語音發微信（Siri 或小愛幫忙讀消息同回覆）。
- **舊車都可以用**：如果你嘅車比較舊唔支援，花幾百蚊買個外掛嘅 CarPlay 螢幕，黐喺儀表台上，體驗差唔多。

七、聯動中嘅安全同私隱

設備聯得越多，私隱洩露嘅風險面就越大。幾點需要注意：

- 公共場合投屏之前，先睇下手機通知欄有冇私密消息，唔好等全家睇到唔應該睇嘅內容。
- 將設備送畀別人或者賣掉之前，先解綁——去手機設定入面搵到嗰個設備，揀「移除」或「取消配對」。
- 智能家居鏡頭，買有品牌嘅、有加密嘅，唔好買幾十蚊嘅雜牌。聽過太多鏡頭被黑客直播嘅新聞喇。

嚟試下：將手機螢幕投到電視上，並設定一個智能場景

1. 確保手機同電視連同一個 Wi-Fi。
2. 喺電視上搵到「螢幕鏡像」或「AirPlay」功能並打開。
3. 喺手機上：iPhone 由控制中心撳「螢幕鏡像」；安卓下拉搵「無線投屏」。
4. 揀一張你最鍾意嘅相投到電視上，等家人睇下。
5. 如果有智能家居設備，試下喺手機 App 入面創建一個「離家模式」：一鍵熄燈、熄冷氣、啟動掃地機械人。

案例 1：電池續航優化實戰

- **場景**：用戶小米 14 續航差，半日就冇電。
- **步驟**：(1) 設定電池查看耗電排行 (2) 識別異常後台 App (3) 關閉 5G 切 4G (4) 限制

後台 (5) 開啟省電模式 (6) 充電時除殼散熱。

- **預期結果：**續航延長 30-50%，電池健康度數據可讀。

第十八章：數據備份與遷移

我一個鄰居，手機跌咗入水池撈出嚟已經開唔到機。佢最心疼嘅唔係手機本身，而係入面三年冇備份過嘅相——細路由出生到識行路嘅全部記錄。聽佢講完我心入面特別唔係滋味，因為呢件事完全係可以避免嘅。呢章我哋就認認真真咁畀數據上一道保險。

一、先搞清楚：你手機入面邊啲嘢唔見咗就冇咗

唔同數據嘅「生存狀態」唔一樣，你要分清楚：

- **存喺手機本地、唔見咗就真係冇咗嘅：**相同影片（冇開雲同步嘅話）、備忘錄、SMS、通話記錄、下載嘅檔案。
- **存喺 App 伺服器上、換手機重新登入就有：**微信好友列表、淘寶訂單、銀行 App 入面嘅交易記錄。呢啲嘢存儲喺對應公司嘅伺服器上，唔係存喺你手機入面嘅。
- **處於灰色地帶、需要特別操作嘅：**微信聊天記錄（雖然微信伺服器存咗，但你自己冇法由伺服器下載）、手機銀行綁定嘅設備憑證、兩步驗證 App 入面嘅密鑰（Google Authenticator 嗰種）。

所以你要重點保護嘅，係第一類同第三類。

二、整機備份：一次搞掂所有嘢

整機備份就好似畀手機影一張完整嘅「快照」——當前系統設定、桌面佈局、App 數據、相、通訊錄，全部打包。

方案一：電腦本地備份（最可靠，推薦有電腦嘅人）

- **iPhone：**將手機用數據線連到電腦上，打開 iTunes（Windows 舊版）或 Finder（Mac），喺設備頁面入面揀「立即備份」。注意要勾上「畀備份加密」，否則密碼同健康數據唔會被備份入去。
- **安卓：**各品牌有自己嘅電腦端軟件，比如華為手機助手、小米助手、三星 Smart Switch。連上電腦按提示操作就得。

- 電腦備份嘅好處係唔使花錢買雲空間，而且恢復速度比雲端恢復快得多。缺點係需要你記得定期去插線備份——大部分人做到唔到呢一點。

方案二：雲備份（最省心，推薦所有人）

- **iPhone**：設定你嘅名 iCloud iCloud 雲備份打開。手機插住電源、連 Wi-Fi、鎖屏嘅時候會自動備份。免費嘅 5GB 大概率唔夠，建議花 6 蚊升級到 50GB。
- **安卓**：各家品牌雲都有備份功能。華為手機去設定搜「雲備份」，小米去「小米雲服務」，OPPO/vivo 都類似。另外仲有一個谷歌自帶嘅備份（設定系統備份），呢個都打開佢。
- 雲備份最大嘅好處係「唔使你操心」，自動搞掂。最大嘅壞處係恢復嗰陣要下載好耐，而且免費空間一般唔夠用。

三、分類備份策略：重要嘅單獨另存一份

整機備份係兜底嘅，但有啲嘢太重要喇，值得你單獨另存一份：

相同影片（最珍貴嘅數碼資產）

- **雙保險方案**：開 iCloud 相/Google 相自動同步，同時每隔幾個月手動將相複製一份到電腦或流動硬碟上。咁樣雲端一份、本地一份，互為備份。
- **唔想花錢嘅方案**：Google 相免費畀 15GB，可以設為「壓縮畫質」嚟省空間。或者每隔一個月用數據線將相全部拷到電腦上。

通訊錄（容易忽略但極其重要）

- iPhone 通訊錄預設同 iCloud 同步，淨係開咗同步就唔使額外備份。但如果你想再安全啲，可以去 iCloud.com 匯出 vCard 檔案存電腦上。
- 安卓通訊錄通常同谷歌或品牌雲同步。去通訊錄 App 嘅設定入面匯出.vcf 檔案，就係一份完整嘅備份。

微信聊天記錄（單獨處理，重中之重）

呢個必須單獨攞出嚟講，因為整機備份入面微信聊天記錄好多時候唔完整，而且換機工具根本搬唔動佢。

- **方法一：遷移到新手機**。微信我設定通用聊天記錄遷移與備份遷移遷移到手機/平板。新舊手機連同一個 Wi-Fi，掃碼就可以傳。注意：遷移完成之前，舊手機唔好退出微信。
- **方法二：備份到電腦**。電腦上裝微信 PC 版，左下角三條槓遷移與備份備份與恢復備份聊天記錄到電腦。之後換手機可以再恢復到新手機上。

- **方法三：備份到雲端**。微信都支援備份到 iCloud 或手機雲空間，但注意：呢個備份淨係可以喺你當前手機上恢復，唔可以跨設備還原。換手機嘅話，仲係要用前兩種方法。

四、換機指南：由舊手機到新手機

換新手機係高興事，但如果數據冇搬過去就變成頭痛事喇。三種情況分別對待：

同品牌升級（最簡單）

- **iPhone 新 iPhone**：開機激活時將舊手機放喺旁邊，會自動彈出「快速開始」，用舊手機掃新手機上嘅圖案就得。所有數據、設定、App 佈局都會原封不動搬過去，包括 Wi-Fi 密碼都會自動帶過去。大概需要二三十分鐘。
- **安卓同品牌**：用品牌自帶嘅換機工具。華為用「手機克隆」，小米用「小米換機」，OPPO 用「手機搬家」，vivo 用「互傳」。步驟都差唔多：新舊手機裝同一個 App、掃碼連接、勾選要搬嘅嘢、開傳。

跨品牌換機（安卓換安卓，唔同品牌）

用第三方換機工具或者用舊品牌自帶嘅換機 App（大部分品牌嘅換機 App 都支援換到其他安卓手機）。可以搬走嘅有：通訊錄、相、SMS、通話記錄、部分 App。搬唔走嘅：App 內嘅登入狀態（你要重新登入）、微信聊天記錄（用微信自己嘅遷移功能）。

跨平臺換機（安卓換 iPhone，或反過來）

- **安卓 iPhone**：蘋果有個「轉移到 iOS」App，安卓手機上下載後，喺新 iPhone 激活時揀「由安卓設備轉移數據」。可以搬走通訊錄、相、SMS 等基礎數據，但 App 數據搬唔到。微信聊天記錄仲係要用微信自己嘅遷移功能。
- **iPhone 安卓**：各安卓品牌有對應嘅遷移工具，但講真，跨平臺遷移體驗不如同平臺好。最重要嘅事先單獨做咗：通訊錄匯出、相備份、微信聊天記錄遷移。剩下嘅慢慢嚟。

五、備份嘅黃金法則：三二一原則

呢個係數據安全領域嘅一個經典原則，簡單易記：

- **3**：重要數據至少保留三份（一份原件，兩份備份）。
- **2**：備份用兩種唔同嘅存儲介質（比如雲端一份，流動硬碟一份）。
- **1**：至少有一份備份放喺唔同嘅物理位置（防止火災、盜竊等同時毀掉所有數據）。

對普通人嚟講，做到「雲端一份、電腦一份」就已經算好好喇。唔使搞太複雜，關鍵係你真係去做咗。

微信聊天記錄特別注意

微信聊天記錄無法通過任何手機換機工具遷移！佢唔喺整機備份嘅覆蓋範圍內。必須用微信自帶嘅「聊天記錄遷移」功能，或備份到電腦版微信。好多人喺換完新手機之後先發現聊天記錄全冇咗，嗰個時候已經遲喇——舊手機可能已經恢復出廠設定咗。

練一練：做一次完整嘅備份演練

1. 檢查 iCloud 雲備份或品牌雲備份係咪開啟，確認上一次備份時間。
2. 手動備份相：用數據線連接電腦，將 DCIM 同 Pictures 資料夾複製到電腦。
3. 備份通訊錄：iPhone 用戶去 iCloud.com 匯出 vCard；安卓用戶喺聯絡人 App 設定入面匯出.vcf 檔案。
4. 備份微信聊天記錄：打開微信我設定通用聊天記錄遷移與備份備份到電腦。
5. 將匯出嘅通訊錄 vcf 同相資料夾複製一份到流動硬碟或 USB 手指，完成「三二一」入面嘅「一份離線喺別處」。

第十九章：故障排查與自救

手機同人一樣，偶爾都會「生病」。卡頓、閃退、黑屏、充唔入電……大部分毛病其實冇咁可怕，你完全可以喺送去維修之前自己先試下。呢章就係你嘅「手機急診手冊」——先用低成本嘅方法試，唔得再搵專業醫生。

一、手機變卡、發熱——最常被問嘅問題

講真，手機用咗一年左右開始變卡，太正常喇。原因唔一定係手機壞咗，更多時候係「積灰」——垃圾檔案、後台進程、存儲空間慢慢被填滿。

先做診斷，唔好急住修：

1. **查存儲空間**：去設定存儲睇剩低空間。如果唔夠 10% 喇，系統會變得非常卡，因為佢喺掙扎住騰地方。我嘅經驗係：保持至少 15%-20% 嘅剩低空間，手機先可以比較順暢。

2. **查耗電排行**：去電池設定入面睇過去 24 個鐘嘅耗電排行。如果某個 App 佔咗大頭但你根本冇點用，咁就係佢喺後台偷跑。直接卸載或者限制後台活動。

3. **聯想最近做咗乜**：係升級咗系統後變卡嘅？（大版本更新後前幾日變卡正常，系統喺後台重建索引）。仲係安裝咗某個新 App 之後變卡嘅？（卸載試下）。

4. **睇使用環境**：手機喺太陽底下暴曬、套住厚厚嘅殼邊充邊打機，發燙係必然嘅。高温仲會導致 CPU 自動降頻，你就感覺手機突然變卡喇。涼返再試。

治療方法（由簡單到複雜）：重啟手機（解決 60% 嘅問題）清存儲空間（刪掉冇用嘅 App 同影片）某個 App 有問題就清佢嘅快取或卸載重裝最後一招：備份數據後恢復出廠設定。

二、應用閃退、無響應——App 唔聽話喇

某個 App 撳開就閃退、或者用着用着突然卡死唔鬱。處理流程就好似醫生問診，一步步嚟：

1. **先關掉重開**：由後台劃走 App，再重新打開。大多數臨時 bug 呢一下就好喇。

2. **更新 App**：去應用商店睇有冇更新。有時候係因為 App 版本太舊，同系統唔兼容。

3. **檢查權限**：有啲 App 需要某個權限先可以正常工作，但你冇畀。比如相機 App 冇畀相機權限打開就閃退。去設定入面搵到嗰個 App，將該畀嘅權限都畀咗。

4. **清快取**：去設定應用管理搵到嗰個 App 存儲清除快取。注意係「清除快取」唔係「清除數據」！清除數據會將你 App 入面嘅登入狀態同設定全刪咗，相當於恢復出廠畀 App 嘅狀態。

5. **卸載重裝**：最後一招。卸載之前先確認呢個 App 嘅數據會唔會冇——微信嘅話先備份聊天記錄，遊戲嘅話先確認帳號有冇綁定手機號。

三、充唔入電或充電慢——可能唔係電池嘅鍋

充唔入電係特別容易等人焦慮嘅問題，但好多時候原因係出喺配件上，唔係手機壞咗。

- **先排查充電口**：用牙籤或者唔導電嘅細工具，輕輕咁將充電口入面嘅灰塵同棉絮掏一掏。呢個我自己遇過——口袋入面跌咗一條線頭啱啱堵喺充電口入面，插落去以為係接觸不良，掏出嚟就好喇。

- **換根線同充電頭試下**：線嘅損壞率比充電頭高得多，尤其係經常卷來捲去嘅線。換一條確定冇問題嘅線同充電頭交叉測試，好快可以確定係邊個配件壞咗。

- **無線充電嘅問題**：如果你用無線充電，檢查一下手機殼係咪太厚，或者殼後面有冇金屬片（磁吸支架之類）。仲有，將手機準確咁放到充電板嘅正中間——歪咗就可能充唔上。

- **充電慢**：確認充電頭功率夠唔夠（iPhone 想要快充至少需要 20W 嘅 PD 充電頭），線支唔支援快充（唔係所有嘅線都可以過 3A 以上電流），手機係咪太燙（太燙會自動降功率保護電池）。

四、網絡連唔上、Wi-Fi 成日斷

喺出面用流量好地地，返屋企連唔上 Wi-Fi；或者 Wi-Fi 連住但就係冇網。按呢個順序排查：

1. **開關飛行模式**：下拉控制中心，打開飛行模式等 5 秒再關掉。呢個等於等手機重新搜一次網，非常有效。

2. **Wi-Fi「忘記」再重連**：去 Wi-Fi 設定入面，撳嗰個網絡名旁邊嘅感歎號（安卓係齒輪），揀「忽略此網絡」或「忘記」，然後重新輸入密碼連接。

3. **重啟路由器**：好多人屋企嘅路由器幾個月冇關過，時間耐咗會唔太穩定。拔掉電源等 30 秒再插返去，大部分 Wi-Fi 問題就解決喇。

4. **檢查係咪手機嘅問題**：別嘅設備連同一個 Wi-Fi 正常嗎？如果都正常就你嘅手機唔得，咁手機可能需要重啟或者重置網絡設定（設定通用傳輸或還原還原網絡設定，唔會刪你嘅數據，但會將所有 Wi-Fi 密碼都清咗）。

五、螢幕問題：觸控唔靈、閃屏、留印子

- **觸控唔靈敏**：先撕掉鋼化膜睇下係咪膜嘅鍋——平鋼化膜用耐咗確實會影響觸控。螢幕上有水或手上有油都會導致觸控漂移。都唔係嘅話，重啟一下再試。
- **螢幕上有印子/殘影（燒屏）**：長時間顯示同一個畫面（比如一直亮屏碌抖音），會喺 OLED 螢幕上留低「殘影」。呢個唔係螢幕壞咗，淨係暫時嘅。關屏休息半個鐘一般可以恢復。
- **螢幕閃爍**：如果係低亮度下閃爍，可能係 PWM 調光嘅問題，多見於 OLED 螢幕。去設定入面搵「DC 調光」或「防頻閃」選項，打開後會好好多。唔係所有手機都有呢個功能。
- **螢幕有一塊區域完全黑咗/亮咗**：呢個比較嚴重，大概率係螢幕物理損壞，需要換屏。非官方維修換嘅螢幕，色彩同亮度通常不如原裝，都影響防水。

六、無法開機、卡喺 Logo 介面——先唔好慌

呢種情況最容易等人心跳加速，但冷靜，先試下呢幾個：

- **充電半個鐘再講**：好多時候淨係電徹底耗盡咗。插上原裝充電器，等至少 30 分鐘，再嘗試開機。有啲手機喺電耗盡後需要充到一定電量先會有反應。
- **強制重啟**：iPhone 全面屏機型撳一下音量加、一下音量減、長撳側邊電源鍵唔放，直到螢幕出現蘋果 Logo（可能要撳住 10-15 秒）。安卓通用方法：長撳電源鍵 10 秒以上，直到震動或亮屏。
- **卡喺啟動介面反覆重啟**：呢種情況叫「白蘋果」（iPhone）或「卡 Logo」（安卓）。如果強制重啟唔得，可能需要進入恢復模式用電腦修復。iPhone 連電腦打開 iTunes/Finder，進入恢復模式後揀「更新」（唔係「恢復」），會重裝系統但保留數據。安卓進 Fastboot 或 Recovery 模式，各品牌組合鍵唔同，建議搜尋你手機型號 +「進恢復模式」。
- **以上都唔得**：那就只能求助專業維修喇。數據如果備份過，就唔使太擔心。

七、聲音問題：喇叭唔響、通話聲音細

- **喇叭冇聲**：先確定唔係靜音模式（iPhone 側邊嘅靜音撥片）、勿擾模式、或者音量被撳到最低咗。聽落好似好基本，但呢種情況佔咗 40%。然後檢查藍牙係咪連住某個耳機或音箱。
- **通話對方聽唔到你講嘢**：檢查話筒（麥克風）孔有冇堵住——手機底部嘅細孔好容易俾灰塵堵住。用軟毛刷輕輕清理。唔好用針或牙籤硬拮，會拮壞防塵網。
- **聲音沙啞/破音**：如果係喇叭入水導致嘅——唔好用風筒熱風吹！用紙巾吸乾表面水分，然後放住自然晾乾。網上話用聲波排水 App 有一定效果（播放低頻聲音將水振出去），可以試下。

八、搵官方維修，避開嗰啲坑

如果上面嘅方法都試過都唔得，咁就要搵專業維修喇。但維修市場水好深，講下我自己踩過嘅坑同經驗：

- **搵真官方，唔好搵「睇落似官方」嘅：**去品牌官網（Apple.com.cn、Huawei.com.cn 等）搵「授權服務商」列表。街邊掛個蘋果 Logo 嘅非授權店，用嘅可能唔係原裝配件，修完你嘅防水就冇咗一剛。

- **送修前必須做嘅事：**備份所有數據（唔好指望修嘅時候數據唔會冇），退出所有重要帳號（Apple ID、微信、支付寶），關閉「查找我嘅手機」。有啲維修點會要求你提供鎖屏密碼，呢件事有私隱風險——你最好自己先退出帳號再送過去。

- **換螢幕嘅坑：**非官方螢幕分「原壓屏」（原裝玻璃換過）、「後壓屏」（非原裝玻璃）、「組裝屏」（全非原裝）。價格差好幾倍，但顯示效果同耐久度都差好多。如果手機仲喺保修期，老老實實走官方。

- **換電池嘅坑：**非原裝電池可能虛標容量，而且用咗半年就衰減得比原裝電池兩年嘅損耗仲大。蘋果換電池官方價格比非官方貴唔了多少，建議走官方。

- **數據恢復嘅坑：**手機入水/摔壞後，去維修如果對方話「數據恢復要加錢」，小心——有啲不良商家會故意誇大損壞程度嚟多收錢。正規嘅數據恢復係先報價再操作，而且恢復唔成功唔收費。

萬能第一招：重啟手機

唔管你遇到乜嘢問題——卡頓、閃退、連唔上網、聲音異常——先重啟一下手機。呢句話聽落似係敷衍，但以我自己嘅經驗，至少有六成嘅小毛病重啟之後就好喇。手機同電腦一樣，跑得耐記憶體入面會積累各種亂七八糟嘅嘢，重啟就係清空重來。所以，下次手機出問題，先唔好急住搜「點算」，重啟完咗再睇下。

而家就記：呢啲救命操作要爛熟於心

1. **強制重啟：**全面屏 iPhone 撳音量加音量減長撳電源鍵到出現 Logo。安卓長撳電源鍵 10-15 秒直到震動。
2. **清理充電口：**搵一個細工具（牙籤或手機取卡針），輕輕清理充電口嘅灰塵。
3. **Wi-Fi 重連：**去 Wi-Fi 設定入面「忘記」你當前嘅 Wi-Fi，重新輸入密碼連接。
4. **開/關飛行模式：**下拉控制中心，打開飛行模式，心入面數 5 秒，關掉，睇網絡係咪恢復。
5. 將呢幾個操作嘅步驟截圖保存，或者寫喺備忘錄入面。遇到問題嘅時候唔使慌慌張張去搜，直接揭備忘錄就得。

第二十章：開發者模式與高級權限

呢一章係畀嗰啲想要深入探索手機「隱藏功能」嘅用戶準備嘅。開發者模式、Root 權限、BL 鎖……呢啲聽落好極客嘅概念，其實離普通用戶並唔遙遠。

一、開發者模式：解鎖隱藏設定

開發者模式係手機系統內置嘅一個隱藏選單，入面有好多普通用戶用唔到但開發者需要嘅功能。

點樣開啟：

- **安卓**：進入設定 > 關於手機 > 連續撳「版本號」7 次，直到提示「您已處於開發者模式」。
- **鴻蒙**：設定 > 關於手機 > 連續撳「HarmonyOS 版本」7 次。
- **iOS**：普通用戶無法開啟完整嘅開發者模式，需要付費嘅 Apple Developer 帳號。

常用功能一覽：

- **USB 調試**：連接電腦進行高級操作（如刷機、數據恢復、ADB 命令）嘅必開開關，詳見本章「ADB 實用命令」小節。
- **顯示觸摸操作**：螢幕上顯示觸控點（一個細圓點跟隨手指），適合錄製操作教程或演示。

性能與流暢度相關：

- **動畫速度調節（視窗動畫縮放 / 過渡動畫縮放 / 動畫程式時長縮放）**：呢三個選項控制打開 App、切換頁面時嘅動畫時長。預設係 1x，調到 0.5x 會等手機「睇落」更快（實際上程式運行速度冇變，淨係動畫變快咗），調到關閉就直接無動畫跳轉。多數極客會設為 0.5x，既保留過渡感又顯快；完全關閉會等操作顯得生硬，而且可能掩蓋程式真實嘅卡頓，反而等你誤以為「好流暢」。
- **強制 GPU 渲染（Force GPU Rendering）**：強制所有 2D 繪圖通過 GPU 而唔係 CPU 處理。喺舊機型或某些 App 優化唔佳時可以提升滾動流暢度，但都會增加 GPU 負載同耗電。現代手機預設已大量使用 GPU 渲染，呢個開關對中高端機型影響有限，更多係歷史遺留選項，普通用戶保持預設即可。
- **顯示 GPU 呈現模式分析（Profile GPU Rendering）**：喺螢幕上以柱狀圖實時顯示每幀嘅渲染耗時，綠色橫線代表 16.6ms（60fps 嘅幀時間閾值）。柱超過綠線就說明掉幀喇。呢個係開發者排查 App 卡頓嘅利器，普通用戶都可以用佢嚟對比邊個 App 優化得差——比如某個 App 滾動時柱成日飆到紅線以上，咁基本就係佢嘅鍋。

後台與續航相關：

- **後台進程限制**：可以設定「標準限制」「最多 1 個進程」「最多 2 個進程」等。表面睇限制後臺進程可以省電，但實際上 Android 自帶嘅後臺管理已經做得唔錯，強行限制反而會導致 App 頻繁重新載入（冷啟動反而更耗電、體驗都變差）。呢個選項更適合調試用，唔建議普通用戶長期開啟，保持「標準限制」即可。

- **唔保留活動**：每離開一個介面就立即銷毀佢。極不推薦，會導致 App 頻繁白屏、重新載入，僅用於測試 App 嘅恢復邏輯。

注意事項

開發者模式入面嘅設定可能會影響系統穩定性，唔明嘅選項唔好亂改。開啟 USB 調試後，連接電腦嗰陣要小心，唔好連接唔信任嘅電腦。

二、OEM 解鎖：開發者選項入面嘅「刷機開關」

喺開發者選項入面有一個唔太起眼但好重要嘅開關——「OEM 解鎖」（部分品牌叫「OEM 解鎖允許」或「設備解鎖」）。呢個開關本身唔會解鎖 BL 鎖，但佢係一個「前置許可」：淨係打開佢，廠商先允許你通過 fastboot 命令或品牌工具去解鎖 BL。

- **開關嘅作用**：相當於向廠商聲明「我同意放棄保修、自負風險，請允許我解鎖 BL」。如果呢個開關係關閉嘅，任何解鎖 BL 嘅命令都會被直接拒絕。

- **同 BL 鎖嘅關係**：OEM 解鎖係「申請資格」，BL 解鎖先係「實際操作」。前者喺手機設定入面撳一下就得，後者需要去廠商官網申請解鎖碼、用電腦跑 fastboot 命令（如 `fastboot flashing unlock`），兩者缺一不可。

- **恢復出廠設定會重置嗎**：會。「OEM 解鎖」開關喺恢復出廠後會返到關閉狀態，但 BL 鎖一旦解鎖就係物理級別嘅狀態，唔會因為恢復出廠而重新上鎖——需要手動執行 `fastboot flashing lock` 先可以重新上鎖。

- **乜嘢時候應該打開**：除非你確定要刷機或 Root，否則保持關閉。佢同「USB 調試」係兩個獨立開關，刷機時通常需要同時打開。

小知識：點解廠商要設定呢個開關

早期安卓冇呢個開關，惡意軟件淨係攞到權限就可以偷偷刷入系統。Google 後來要求所有安卓廠商喺開發者選項中加入「OEM 解鎖」開關，作為一道「用戶知情同意」嘅確認。呢個都係點解即便你解咗 BL 鎖，如果 OEM 解鎖開關被關閉，重新刷機仍然會失敗。

三、BL 鎖（Bootloader 鎖）：刷機嘅第一道門

Bootloader 係手機啟動時載入作業系統嘅程式。BL 鎖就係廠商為咗安全而加嘅鎖，防止用戶刷入非官方系統。

BL 鎖嘅作用：

- **安全保護**：防止惡意軟件刷入系統。
- **防盜**：即使手機被刷機，冇原帳號密碼都無法使用。
- **保修限制**：解鎖 BL 通常意味住失去官方保修。

各品牌 BL 鎖政策：

- **小米**：可以解鎖，需要綁定帳號等待期（7-30 日）。
- **華為**：2018 年後停止提供解鎖碼，基本無法官方解鎖。
- **一加**：可以解鎖，相對寬鬆。
- **OPPO/vivo**：部分機型可以解鎖，需要申請。
- **三星**：可以解鎖，但會觸發 Knox 熔斷，失去 Samsung Pay 等功能。
- **蘋果**：iOS 設備冇傳統意義上嘅 BL 鎖概念，但有類似嘅安全機制。

四、ADB 實用命令：通過電腦控制手機

ADB 全稱 Android Debug Bridge（安卓調試橋），係 Google 官方提供嘅命令行工具，等電腦通過 USB 或 Wi-Fi 與安卓手機通信。佢本身唔需要 Root，淨係手機開咗「USB 調試」就可以用。可以將佢理解為「用電腦畀手機下指令嘅遙控器」——刷機、裝 App、傳檔案、查日誌，都靠佢。

點樣通過 USB 連接：

1. 手機端：進入開發者選項打開「USB 調試」。
2. 電腦端：由 Google 官方下載 Platform Tools（含 `adb` 同 `fastboot` 兩個可執行檔案），解壓到任意目錄。
3. 用數據線連接手機同電腦，手機上會彈窗「係咪允許 USB 調試」，勾選「始終允許」並確定。
4. 喺電腦命令行（Windows 嘅 PowerShell 或 cmd）入面執行 `adb devices`，如果輸出類似 `List of devices attached /xxxxxxxx device`，就說明連上咗。

驅動同授權彈窗唔好忽略

Windows 用戶如果 `adb devices` 顯示“unauthorized”或根本唔列出設備，多半係驅動冇裝好或者唔記得喺手機上撳「允許」。三星/小米等品牌可以裝各自嘅「USB 驅動」或「手機助手」嚟補齊驅動；Mac 同 Linux 一般免驅。如果手機彈窗被你手滑撳冇咗，拔插一下數據線就會重新彈出嚟。

常用 ADB 命令：

- `adb install app.apk`：將電腦上嘅 APK 裝到手機（`-r` 表示覆蓋安裝並保留數據；`-s` 裝到 SD 卡）。
- `adb uninstall com.xxx.xxx`：按包名卸載 App。
- `adb push` 電腦路徑 `/sdcard/`目標路徑：將電腦檔案推送到手機。
- `adb pull /sdcard/`檔案電腦路徑：將手機檔案拉到電腦（比發畀自己微信再下載快得多）。
- `adb shell`：進入手機嘅 Linux Shell，可以直接執行 `ls`、`pm`、`am` 等命令。
- `adb reboot bootloader`：重啟手機進入 Fastboot 模式（刷機嘅入口）。
- `adb reboot recovery`：重啟進入 Recovery 模式。
- `adb shell pm list packages`：列出手機上所有已安裝應用嘅包名（搵包名必用）。加 `-s` 淨係睇系統應用，加 `-3` 淨係睇第三方應用。
- `adb shell pm uninstall --user 0` 包名：為當前用戶卸載預裝 App，免 Root，詳見下方實戰。

免 Root 卸載預裝 App 實戰：

呢個係 ADB 最實用嘅功能之一：用 `pm uninstall --user 0` 命令可以為「當前用戶」卸載預裝 App，而唔需要 Root，都唔影響系統分區。被卸載嘅 App 喺系統入面「消失」，但實際仲喺系統分區入面，恢復出廠設定後會返嚟。

1. 先用 `adb shell pm list packages -s` 列出所有系統應用，搵到你想卸載嘅包名。比如小米瀏覽器係 `com.android.browser`，華為應用市場係 `com.vmall.client`。
2. 唔確定包名對應邊個 App？可以喺電腦上用 `adb shell pm list packages | findstr` 關鍵字搜尋，或者裝一個第三方 App "Package Viewer" 喺手機上直接查看每個 App 嘅包名。
3. 執行卸載：`adb shell pm uninstall --user 0` 包名，比如 `adb shell pm uninstall --user 0 com.android.browser`。返回 `Success` 就成功喇。
4. 想恢復點算？兩種方法：(1) 恢復出廠設定（最徹底但麻煩）；(2) 重新安裝：`adb shell cmd package install-existing` 包名。

千萬唔好亂刪系統核心包

"pm uninstall –user 0"雖然係免 Root 嘅安全方式，但亂刪關鍵系統應用（如 `com.android.systemui` 狀態欄、`com.android.phone` 電話、`com.android.settings` 設定）會導致系統崩潰、黑屏、無法開機等嚴重問題。刪之前先 Google 一下「呢個包名可唔可以刪」，寧可少刪唔好錯刪。建議一次淨係刪一兩個，確認冇問題再繼續。

小知識：–user 0 係乜嘢意思

安卓 4.2 之後支援多用戶，每個用戶有獨立嘅 ID（機主係 0，附加用戶由 10 開始）。`--user 0` 表示「淨係對當前機主卸載」，系統分區入面嘅 APK 檔案依然存在，淨係唔畀機主顯示。呢個就係點解佢唔需要 Root——冇鬱系統檔案，淨係改咗「用戶配置」。呢個都係佢同 Root 後卸載最大嘅區別：Root 卸載係真正由系統入面刪掉，所有用戶都唔再有；ADB 卸載淨係「對當前用戶隱藏」。

五、Root 權限：安卓嘅「超級管理員」

Root 權限相當於 Linux 中嘅超級用戶（對/目錄擁有完全控制權，可以讀寫任意檔案），係安卓系統嘅最高權限。佢同 Windows 嘅 Administrator 有本質區別：Windows 出於安全考慮對管理員做咗諸多限制（真正嘅最高權限其實係 TrustedInstaller），即便係 Administrator 都無法像 Linux root 咁樣直接執行 `sudo rm -rf /*`。獲得 Root 權限後，你可以修改系統檔案、卸載預裝應用、使用需要 Root 嘅高級 App。

Root 嘅好處：

- 卸載無法卸載嘅預裝應用。
- 使用高級功能 App（如鈦備份、綠色守護）。
- 修改系統介面同功能。
- 安裝第三方 Recovery 同自訂 ROM。

Root 嘅風險：

- **失去保修**：大多數廠商唔保修 Root 過嘅手機。
- **安全風險**：惡意 App 可能獲得 Root 權限，造成嚴重後果。
- **系統不穩定**：誤操作可能導致系統崩潰。
- **銀行 App 唔可以用**：好多銀行同支付 App 檢測到 Root 會拒絕運行。
- **OTA 更新失敗**：可能無法正常接收官方系統更新。

嚴重警告

Root 操作有一定風險，可能導致手機變磚（無法開機）。如果你係新手，建議先充分了解相關知識，並確保有完整嘅備份。操作前請仔細閱讀教程，確認你嘅機型同系統版本。

六、SafetyNet 與 Play Integrity 認證：點解 Root 後銀行 App 唔可以用

好多 Root 用戶都遇過呢樣嘅崩潰瞬間：Root 之後興致勃勃咁裝咗一堆模組，結果打開銀行 App——「您嘅設備不安全，無法使用」，Google Pay 都用唔到，連 Netflix 都拒絕高清播放。呢個背後係 Google 嘅兩套設備完整性認證機制：舊嘅 SafetyNet 同新嘅 Play Integrity API。

乜嘢係 SafetyNet / Play Integrity：

- **SafetyNet Attestation（舊版，2017 年起）**：Google 早期推出嘅設備完整性檢查 API，應用通過佢向 Google 伺服器查詢「呢臺設備係咪通過 CTS 兼容性測試、係咪處於未修改嘅官方狀態」。返回結果分為 Basic Attestation（基礎校驗）同 Strong Attestation（強校驗，依賴硬件密鑰）。Google 已宣佈 SafetyNet 將喺 2024 年後逐步停用。

- **Play Integrity API（新版，2022 年起）**：Google 推出嘅替代方案，比 SafetyNet 更嚴格也更全面。除咗設備完整性，仲包括「應用完整性」（App 係咪被篡改）同「帳號/授權完整性」（用戶帳號係咪被 Google 信任）。新 App 應遷移到 Play Integrity。

- **邊個用**：銀行、金融、支付類 App（如 Google Pay、各銀行嘅網銀、支付寶/微信支付嘅部分場景）、串流媒體（Netflix 高清版權保護）、部分遊戲反作弊（如騰訊/網易嘅部分遊戲）、企業辦公 App（MDM 設備合規性檢查）。

Play Integrity 嘅三級認證結果：

- **MEETS_DEVICE_INTEGRITY（設備完整性通過）**：設備運行正版 Android 系統，Bootloader 未解鎖，冇 Root，CTS 兼容性測試通過。可以正常使用銀行、支付類 App。

- **MEETS_BASIC_INTEGRITY（基礎完整性通過）**：設備基本可信，但可能存在輕微異常。某些 App 會降級使用（如 Netflix 仍可播放但限制清晰度），某些嚴格嘅銀行 App 會直接拒絕。

- **MEETS_STRONG_INTEGRITY（強完整性通過）**：最高級別，要求硬件密鑰背書（如 Google Pixel 系列嘅 Titan M 晶片），大多數第三方品牌機型即使未 Root 都達唔到呢一級。

- **無任何標籤（NO_INTEGRITY）**：設備已 Root、解鎖 BL、刷咗第三方 ROM 或被識別為模擬器，絕大多數支付/銀行 App 會直接拒絕運行。

點解 Root 後呢啲 App 會失效：

應用通過 Play Integrity API 向 Google 伺服器發起認證請求，Google 會檢查設備嘅多項指標——Bootloader 鎖定狀態、系統分區係咪被修改、係咪存在 su 二進制檔案、SELinux 係咪處於寬鬆模式等。任何一項異常都會導致返回嘅認證級別降低或為空，銀行 App 一睇「設備不安全」，自然就拒絕服務喇。

現代隱藏方案：Magisk Hide / Shamiko / Zygisk：

為咗喺享受 Root 便利嘅同時仲可以用銀行 App，社區開發咗一系列「隱藏 Root 狀態」嘅方案。核心思路都係：等檢測程式睇到嘅唔係一個被修改過嘅系統，而係一個「睇落正常」嘅設備。

- **Magisk**：當前最主流嘅 Systemless Root 方案——唔修改系統分區，而係通過掛載嘅方式實現 Root。咁樣設計嘅好處係 OTA 更新失敗嘅幾率細、卸載比較乾淨，係後續所有隱藏方案嘅基礎。但使用 Magisk 仍然需要解鎖 BL 鎖。
- **Magisk Hide（已停更）**：Magisk 早期內置嘅隱藏模組，可以對指定 App 隱藏 Root。但 2021 年起已被 Magisk 官方移除，原因係 Google 持續升級檢測手段，雙方陷入「貓鼠遊戲」。
- **Zygisk**：Magisk 24.0+ 引入嘅新機制，類似 Riru，通過注入 Zygote 進程實現模組載入，比直接修改系統更隱蔽。開啟 Zygisk 係使用現代隱藏方案嘅前提。
- **Shamiko**：當前最主流嘅隱藏模組，配合 Zygisk 工作，可以為指定 App（「反向白名單」模式）隱藏 Root、隱藏 Magisk 自身、偽造 Bootloader 鎖定狀態。Shamiko 通過 LSPosed 倉庫或 GitHub 分發。
- **Play Integrity Fix（osm0sis）**：專門針對 Google Play Integrity 檢測嘅 Magisk 模組，會修改設備指紋（fingerprint）等 Google 認為你嘅設備係某個未 Root 嘅官方機型。

貓鼠遊戲永遠唔會結束

Google 每隔一段時間就會更新檢測規則，App 廠商都會跟進升級 Play Integrity 校驗。今日仲可以用嘅隱藏方案，聽日可能就被識別。如果你完全依賴銀行/支付 App（比如日常轉賬、掃碼付款），最穩妥嘅選擇仍然係：唔好 Root、唔好解 BL 鎖。隱藏方案更適合「主力機唔 Root + 後備機玩 Root」嘅玩家配置。

七、畀普通用戶嘅建議

- **開發者模式**：可以開啟，體驗下動畫加速、GPU 呈現分析等功能，但唔好亂改其他唔理解嘅選項。
- **OEM 解鎖與 BL 鎖**：如果你唔打算刷機或 Root，兩個開關都保持預設（關閉）狀態最安全。
- **ADB**：免 Root 嘅強大工具，普通人都可以用——比如用 `pm uninstall --user 0` 卸載預裝 App。但刪之前一定要查清楚包名可唔可以刪。
- **Root**：普通用戶唔建議 Root，風險大過收益。如果確實需要某些功能，可以先了解有冇唔需要 Root 嘅替代方案（比如用 ADB 卸載預裝、用 Shizuku 替代部分 Root 功能）。

嚟動手：開啟開發者模式並調整動畫速度

1. 進入設定 > 關於手機。
2. 連續撳「版本號」7 次，直到提示已開啟開發者模式。
3. 返去設定，搵到「開發者選項」（通常喺設定底部或「系統」入面）。
4. 搵到「視窗動畫縮放」、「過渡動畫縮放」、「動畫程式時長縮放」，將佢哋都設為 0.5x 或關閉。

5. 感受一下手機係咪變快咗。

第二十章附：Termux——Android 設備上嘅便攜式 Linux 終端環境

如果你覺得開發者模式仲唔夠「硬核」，想要喺手機上體驗完整嘅 *Linux* 命令行，咁 *Termux* 正係為你準備嘅。佢無需 *root* 權限，就可以將一臺普通嘅安卓手機變成一臺便攜嘅 *Linux* 工作站，係進階用戶探索流動端開發與運維嘅利器。

一、乜嘢係 Termux

Termux 係一款專為 Android 設備設計嘅開源終端模擬器同 Linux 環境應用程式，佢無需 root 權限即可喺手機上提供完整嘅 Linux shell 體驗。作為一個輕量級應用（安裝包淨係 30MB 左右），Termux 通過 APT 包管理系統支援豐富嘅軟件生態，允許用戶喺流動設備上運行 Python、Node.js、Git、SSH 等工具，甚至搭建伺服器或進行滲透測試。

Termux 嘅核心優勢喺於輕量化同高度可擴展性，適合開發、學習及系統管理等多種場景。佢將標準嘅軟件包同精確嘅終端仿真結合喺一個開源解決方案中。

二、核心功能與特性

完整嘅 Linux 環境

Termux **淨係內置 Bash** 呢一種 Shell。如果想使用 Zsh 或 fish 等其他 Shell，需要通過包管理器手動安裝，例如：`pkg install zsh` 或 `pkg install fish`。同時 Termux 支援 Vim、Nano、Emacs 等編輯器，提供接近原生嘅命令行體驗。用戶可以通過“pkg install”安裝 2000+ 軟件包，包括 GCC、OpenSSH、SQLMap 等開發同安全工具。

強大嘅包管理

使用 APT 包管理器（類似 Debian 同 Ubuntu），可安裝數千種 Linux 軟件包，涵蓋開發工具、網絡工具等。支援 Python、Ruby、Node.js、Go、Rust、Swift 等多種編程語言環境。

安全與網絡功能

- **SSH 遠端連接：**支援 SSH 用戶端同伺服器功能，可遠端連接其他設備或等手機成為 SSH 主機。
- **加密工具：**提供 OpenSSL 等加密工具，支援數據加密同安全通信。
- **伺服器搭建：**可配置 Nginx 或 Apache 搭建靜態網頁伺服器。

系統集成與擴展

- 訪問手機存儲權限，實現本地檔案管理。

- 通過 Termux:API 插件調用傳感器、鏡頭等 Android 硬件功能。
- 支援藍牙鍵盤同外部顯示器連接，具有完整嘅滑鼠支援。

三、插件生態系統

Termux 嘅核心應用程式可以通過幾個可選嘅插件應用進行擴展，提供額外嘅功能同埋集成：

插件名稱	功能描述
Termux:API	向命令行程式公開 Android API 功能
Termux: Boot	允許喺設備啟動時運行腳本
Termux: Float	提供浮動終端視窗
Termux: Styling	為終端啟用顏色方案同埋字體自定義
Termux: Tasker	將 Termux 腳本同 Tasker 自動化應用集成
Termux: Widget	允許從主螢幕小工具運行腳本

重要提示

Termux 同埋佢嘅插件必須從同一來源安裝，因為佢哋共享相同嘅 sharedUserId。混合來自唔同來源（如 F-Droid 同 GitHub）嘅安裝會導致安裝錯誤。

四、安裝與配置指南

安裝步驟

1. **下載應用：**從官方渠道（F-Droid 或 GitHub）下載安裝 Termux，無需 root 權限。
2. **初始化系統：**首次啟動後等待”Installing bootstrap packages” 完成。
3. **獲取存儲權限：**執行 `termux-setup-storage` 獲取手機檔案系統權限。
4. **更新軟件源：**執行 `pkg update && pkg upgrade` 更新軟件包列表。

基礎優化

- **換鏡像源：**使用 `termux-change-repo` 命令通過圖形界面更換鏡像源（推薦清華大學鏡像源）。
- **快捷鍵設定：**音量減鍵模擬 Ctrl，音量加鍵模擬 Esc，提升輸入效率。
- **界面美化：**安裝 Zsh + Oh My Zsh，配置主題同埋插件。

五、典型應用場景

移動開發測試

喺手機上搭建完整嘅開發環境，編寫並運行 Python 腳本，使用 GCC 編譯 C 程式，通過 Git 管理代碼倉庫，結合 SSH 同遠程伺服器同步。

伺服器運維

通過 SSH 遠程管理伺服器，執行系統維護、日誌分析等任務，隨時隨地處理伺服器問題。

學習 Linux 操作

提供真實嘅 Linux 環境，係學習命令行同埋系統管理嘅理想平臺，可練習 Shell 腳本編寫、Linux 檔案權限管理（如 `chmod 755` 設定 rwxr-xr-x 權限位、`chown user: group` 修改檔案歸屬、`ls -l` 查看 owner/group/other 三組九位權限）等基礎命令。注意呢個係 Unix/Linux 經典嘅“三組九位”權限模型（owner / group / other × 讀 / 寫 / 執行），同 Windows NTFS 嘅 ACL 訪問控制列表係完全唔同嘅兩套體系。

網絡安全測試

安裝 Nmap、SQLMap、Hydra、Metasploit 等安全工具，進行網絡掃描同埋滲透測試。

六、版本與兼容性

Termux 嘅最新穩定版本係 v0.118.2，適用於 Android ≥ 7 嘅設備。需要注意嘅係，Termux 喺 Android 12 同埋更高版本上可能存在唔穩定性，建議關注官方更新日誌以獲取最新兼容性資訊。

總結

Termux 將強大嘅 Linux 終端體驗帶到咗 Android 設備上，等手機成為一臺便攜嘅開發同埋學習工具。無論係開發者、系統管理員定係技術愛好者，都可以利用 Termux 喺移動設備上享受完整嘅命令行體驗，實現“口袋入面嘅 Linux”呢個理念。

嚟動手：喺手機上跑通第一條 Linux 命令

1. 從 F-Droid 下載並安裝 Termux，首次啟動等待初始化完成。
2. 執行 `pkg update && pkg upgrade` 更新軟件包。
3. 安裝 Python：`pkg install python`。
4. 輸入 `python3 -c "print('Hello from Termux!')"`，感受喺手機上運行代碼嘅樂趣。

四、超越篇（第 21-26 章）

第二十一章：移動網絡與資訊安全深探

我哋每日用手機上網、支付、傾偈，感覺方便得好。但你有冇諗過，你嘅傾偈記錄、支付密碼、相片……呢啲資訊喺網上“裸跑”嘅時候多嗎？講真，大部分人根本冇諗過呢個問題，但一旦出事——比如帳號被盜、銀行卡被碌——嗰陣時再後悔就遲啦。呢一章我哋就好好傾下，點樣等你嘅數字生活唔咁“透明”。

一、VPN：網絡入面嘅“隱形斗篷”

VPN，全稱虛擬專用網絡（Virtual Private Network）。你可以將佢想像成喺你屋企同你要去嘅網站之間，建起咗一條專屬嘅、加密嘅地下隧道。你喺隧道入面行來行去，外面嘅人睇唔到你手入面揸住乜嘢、要去邊。

- **普通上網**：你嘅手機公共網絡目標網站。中間經過嘅路由器、交換機，隨便邊個環節都可能有人”偷睇”你發送嘅數據。尤其係喺咖啡廳、機場嗰種公共 Wi-Fi 上，講難聽啲，簡直係”裸跑”。
- **使用 VPN**：你嘅手機 VPN 加密隧道 VPN 伺服器目標網站。數據被加密封裝，”偷窺者”淨係睇到一堆亂碼。另外，目標網站睇到嘅係 VPN 伺服器嘅 IP 地址，而唔係你嘅真實 IP。

噉 VPN 到底能做乜？簡單講：第一，**保護你喺公共 Wi-Fi 上嘅隱私**；第二，**繞過地域限制**（比如睇某啲淨係喺特定國家上線嘅內容）；第三，**防止你嘅網絡營辦商（ISP）記錄你嘅瀏覽習慣**。不過我要提醒一句：**唔好用免費 VPN**！免費嘅背後往往意味住你嘅數據就係佢哋嘅“商品”。我有個朋友用咗某個免費 VPN，結果手機莫名其妙開始彈廣告，後來發現嗰個 VPN 將佢嘅瀏覽數據打包賣畀廣告商啦。所以揀 VPN，要麼揀付費嘅大品牌（如 NordVPN、ExpressVPN），要麼乾脆唔好用。

另外，VPN 唔係萬能嘅。佢淨係可以保護“傳輸中”嘅數據，如果目標網站本身唔安全、或者你嘅手機上已經中咗惡意軟件，VPN 都幫唔到你。

二、公共 Wi-Fi 嘅“甜蜜陷阱”——比你想像嘅危險

你可能覺得，唔就係連個免費 Wi-Fi 嘛，能有多大風險？我同你講個真事：安全研究人員曾經做過實驗，喺機場擺咗一個叫“Starbucks_Free”嘅 Wi-Fi 熱點，唔夠一個鐘就有上百人

連接。而實際上呢個熱點係研究人員自己搭嘅，所有經過呢個熱點嘅未加密流量——比如你訪問嘅 HTTP 網頁內容、未啟用 TLS 嘅 App 通信——佢哋確實能截獲同埋查看。

但要注意一個關鍵前提：今日你日常用嘅微信、網銀、支付寶，絕大多數都已經行咗 HTTPS 加密。講白啲，攻擊者即使截獲咗數據包，睇到嘅都淨係一堆亂碼，**無法直接讀出你嘅傾偈內容或支付密碼**。不過佢哋依然可以知道你訪問咗邊啲網站（域名），如果某啲老舊 App 或網頁仲喺度用 HTTP 明文傳輸，嗰內容就會完全暴露。呢種攻擊叫“中間人攻擊”（Man-in-the-Middle），原理係攻擊者喺你同真正嘅 Wi-Fi 路由器之間“插咗一腳”。更可怕嘅係，好多人嘅手機設定咗“自動連接已知 Wi-Fi”，攻擊者只要將自己嘅熱點名設成你之前連過嘅某個 Wi-Fi（比如“ChinaNet”或“CMCC”），你嘅手機就會自動連上去。

嗰點樣防？幾條實用建議：

- **認準 HTTPS**：地址欄開頭係 https://，多出嘅”s” 代表 Secure。訪問任何需要輸密碼嘅網站時，如果睇到嘅係 http://（冇 s），即刻停低唔好輸！而家主流瀏覽器都會喺非 HTTPS 網站前顯示” 唔安全” 嘅警告，睇到呢個標記就要警惕。
- **關掉” 自動連接”**：唔論係 iPhone 定係安卓，都喺 Wi-Fi 設定入面將” 自動加入” 或” 自動連接” 關掉。寧可每次手動點一下，都唔好等手機自己判斷。
- **用手機流量做敏感操作**：轉賬、登入網銀、輸入身份證號……呢啲操作老老實實用自己嘅 4G/5G 流量，唔好慳嗰啲流量錢。
- **忘記唔常用嘅 Wi-Fi**：出差返嚟、旅遊返嚟，將之前臨時連過嘅酒店 Wi-Fi、機場 Wi-Fi 都” 忘記” 掉。嗰下次手機唔會自動連到同名假熱點。

三、HTTPS 係點樣保護你嘅？——SSL/TLS 握手

啱啱提到咗 HTTPS，嗰佢到底係點樣工作嘅呢？每次你訪問一個 HTTPS 網站，瀏覽器同伺服器之間會進行一次“SSL/TLS 握手”，成個過程大概就係嗰：

1. 你嘅瀏覽器話：” 嘿，伺服器，我想同你安全通信，呢個係我嘅加密能力清單（支援嘅加密算法等）。”
2. 伺服器回覆：” 好嘅，我從你嘅清單入面揀咗呢套加密方案，呢個係我嘅數字證書，入面麵包含我嘅公鑰，由某某 CA 機構簽名認證。”
3. 瀏覽器驗證證書嘅真假：係唔係由可信嘅 CA 簽發？有冇過期？域名對唔對得上？
4. 驗證通過後，雙方用公鑰加密協商出一個” 會話密鑰”，之後嘅通信全用呢個密鑰加密。

呢個就似你去銀行辦事：你先遞上身份證（證書），櫃員核實你嘅身份，然後你哋兩個約定一個暗號，之後嘅對話淨係得你哋兩個聽得明。即使有人喺旁邊偷聽，都淨係聽到加密後嘅亂碼。

但注意：HTTPS 淨係保證“傳輸過程”係加密嘅，唔保證目標網站係好人。釣魚網站都可以用 HTTPS——嗰個小鎖圖示淨係說明你連接嘅係“某個網站”，唔代表嗰個網站就係你想去嘅嗰個。所以都係要自己核對網址係咪正確。

四、DNS 安全：你嘅“網絡電話簿”安全嗎？

DNS（域名系統）就似互聯網嘅電話簿：你輸入“baidu.com”，DNS 幫你查到佢對應邊個 IP 地址（比如 110.242.68.66），然後瀏覽器先可以連過去。問題係，傳統嘅 DNS 查詢係明文嘅、冇加密嘅！等於話，你嘅 ISP、Wi-Fi 提供者、或者任何中間節點，都能睇到你訪問過邊啲網站。

更惡心嘅係“DNS 劫持”：攻擊者喺 DNS 環節做手腳，將“baidu.com”指向一個假嘅釣魚網站。你明明輸入嘅係正確網址，卻被帶到咗一個一模一樣但實際係假嘅登入頁面，一輸入帳號密碼就玩完啦。

好喺而家有咗加密 DNS 方案：

- **DNS over HTTPS（DoH）**：將 DNS 查詢收埋喺 HTTPS 流量入面，混喺正常嘅網頁瀏覽流量中，根本分唔出邊啲係 DNS 查詢。
- **DNS over TLS（DoT）**：用 TLS 加密協議專門保護 DNS 查詢，埠獨立，網管能知道你用咗 DoT 但睇唔到內容。

我個人推薦用 Cloudflare 嘅 1.1.1.1 或者 Google 嘅 8.8.8.8 作為 DNS 伺服器，佢哋都支援加密 DNS。喺安卓嘅“私人 DNS”設定入面直接填“one.one.one.one”或者“dns.google”就得，超簡單。iPhone 稍微麻煩啲，需要裝個描述檔或者通過 App 設定，但都值得搞下。

五、“無痕瀏覽”真係能隱身嗎？

先講結論：**無痕模式係你屋企窗簾，唔係隱身斗篷。**

佢做咗乜：唔喺本地留低痕跡——關閉無痕視窗後，瀏覽器唔保存你嘅瀏覽歷史、Cookies、表單填寫記錄、搜索記錄。借人哋電腦查嘢、或者買禮物唔想被家人從瀏覽記錄發現，用無痕模式係合適嘅。

佢做唔到乜：

- 你訪問嘅網站**照樣能睇到你嚟咗**，你嘅 IP 地址、設備資訊、瀏覽行為，網站端一清二楚。
- 你嘅網絡營辦商**照樣能睇到你訪問咗邊啲網站**，淨係睇唔到具體內容（如果用咗 HTTPS）。
- 公司/學校嘅網絡管理員**一樣能監控**你嘅上網行為。
- 你下載嘅檔案、添加嘅書籤會**保留落嚟**，唔會隨住關閉視窗而消失。

所以記住：無痕模式係“本地擦除器”，唔係“網絡隱身衣”。如果你真係需要隱藏上網痕跡，就要用 VPN+Tor 瀏覽器呢種組合拳，不過對於日常使用嚟講，瞭解呢啲就夠用啦。

六、瀏覽器隱私：唔好等瀏覽器將你賣咗

你可能冇意識到，瀏覽器本身就係一個巨大嘅資訊收集器。Cookies、瀏覽歷史、快取、自動填充、位置權限……每樣都可能洩露你嘅隱私。幾個建議：

- **定期清理瀏覽數據**：每隔一段時間，去設定入面將 Cookies、快取、歷史記錄清一清。唔係為咗防止”被追蹤”——嗰個靠清 Cookies 已經唔太管用啦——而係為咗防止手機越來越卡。瀏覽器快取能佔好幾個 G 嘅空間。
- **關掉”請勿追蹤”嘅幻想**：瀏覽器入面嘅”發送’請勿追蹤’請求”選項，本質上係”請”網站唔好追蹤你。但網站聽唔聽完全係另一回事，大部分網站直接忽略。唔好指望佢。
- **善用隱私保護瀏覽器**：Firefox Focus、DuckDuckGo 瀏覽器、Brave 呢啲瀏覽器預設攔截追蹤器同埋廣告，比 Chrome 同 Safari 嘅預設設定要乾淨得多。
- **搜索引擎都重要**：用 DuckDuckGo 代替百度/Google 搜索，佢唔記錄你嘅搜索歷史，唔建立用戶畫像。講真搜索結果確實唔如 Google 精準，但日常查資料夠用啦。
- **檢查 App 權限**：去手機設定入面睇下你嘅瀏覽器被授予咗乜嘢權限。佢真係需要訪問你嘅位置、咪、通訊錄嗎？大概率唔需要。

七、實用安全清單：保護自己從而家開始

咁樣講咗咁多，最後畀你一個簡單粗暴嘅安全清單，花半個鐘搞掂，你嘅手機安全性會有質嘅飛躍：

1. **關掉 Wi-Fi 自動連接**：iPhone 喺設定-無線局域網入面，安卓喺 Wi-Fi 設定入面。
2. **換一個加密 DNS**：安卓直接設定私人 DNS 為”dns.google”，iPhone 用 App 或描述檔設定 1.1.1.1 嘅 DoH。
3. **開啟雙因素認證（2FA）**：微信、支付寶、Apple ID、Google 帳號，能開 2FA 嘅都開。短訊驗證碼已經係最低級別嘅 2FA 啦，能用 TOTP（如 Google Authenticator、Microsoft Authenticator）更好。
4. **查一遍 App 權限**：將每個 App 嘅權限過一遍，特別係位置、通訊錄、短訊、咪、相機呢五項。你會發現有啲電筒 App 居然要你嘅通訊錄——即刻卸載。
5. **定期清理瀏覽器數據**：設定個日曆提醒，每月一次，清快取、Cookies。
6. **唔好用公共充電樁嘅數據線**：USB 口既能充電都能傳數據，惡意充電樁（叫”Juice Jacking”）可以趁你充電時偷數據。出街帶自己嘅充電頭同埋線。

而家就做：畀你嘅手機做一次”安全體檢”

1. **DNS 升級**：安卓：設定 > 連接與共享 > 私人 DNS > 填入”one.one.one.one”或”dns.google”。iPhone：用 Safari 搜索”1.1.1.1 DNS profile”安裝描述檔，或下載 Cloudflare 嘅 1.1.1.1 App。
2. **Wi-Fi 審計**：打開 Wi-Fi 列表，將唔識嘅、唔常用嘅、名稱可疑嘅熱點都”忘記”掉。
3. **權限大掃除**：進入設定 > 隱私/權限管理，逐個 App 審查權限。標記出嗰啲權限要求唔合理（比如計算機要通訊錄）嘅 App，考慮卸載或用替代品。

4. **檢查帳號安全**：打開 Apple ID 或 Google 帳號嘅安全頁面，睇下有冇不明設備登入，確認 2FA 已開啟，檢查恢復聯絡人係咪可信。

第二十二章：AI 點樣喺手機中工作

歡迎嚟到手機最酷嘅“魔術現場”，主角係：人工智能。你每日喺度用嘅“一拍即得”嘅靚相、張口就嚟嘅語音助手、越用越明你嘅輸入法，背後都企住呢位隱形大腦。

一、AI 相機：你嘅“後期暗房”搬入咗手機

而家嘅 AI 計算攝影，更似係將一個專業修圖師 + 攝影師團隊直接塞入咗快門鍵入面。喺你撳快門前，AI 就已經開始工作啦：掃描場景，識別出“呢個係藍天”、“呢個係人臉”、“呢個係夜景”。

二、語音助手：你嘅“聲音祕書”點樣聽明你？

1. **聽清（語音識別）**：將聲波轉變成文字。

2. **聽明（自然語言理解）**：理解意圖，提取關鍵實體。

3. **執行與回答**：調用對應 App 或服務，再通過語音合成” 講” 畀你聽。

三、端側 AI vs. 雲端 AI

端側 AI 將計算留喺手機入面，速度快、隱私高、可離線。雲端 AI 需要聯網，但能處理更複雜嘅任務。

四、AI 大模型與手機：你嘅隨身智能助手

2025-2026 年，AI 大模型（如 DeepSeek R1、ChatGPT-5、文心一言 5.0、通義千問 3 等）嘅爆發深刻改變咗手機嘅使用方式。手機唔再淨係執行指令嘅工具，而係能理解、推理同埋創作嘅智能夥伴。端側 AI 晶片（NPU 算力突破 80 TOPS）等大模型推理首次喺手機本地跑通，無需聯網都能用 AI。

乜嘢係 AI 大模型？

AI 大模型（Large Language Model，LLM）係通過海量文本數據訓練出嚟嘅 AI 系統，能理解同埋生成自然語言。你可以將佢想像成一個讀過互聯網上幾乎所有文字嘅“超級學霸”，能回答問題、寫文章、翻譯、編程、分析數據。

手機上嘅 AI 大模型應用：

- **對話助手**：DeepSeek、ChatGPT、豆包、Kimi 等 App，可以隨時提問、翻譯、寫文案、總結文章。比傳統搜索引擎更智能——佢唔淨係畀你連結，而係直接畀你答案。
- **寫作與創作**：等 AI 幫你寫郵件、總結會議紀要、潤色文章、生成創意文案。
- **圖像生成與編輯**：AI 一鍵消除相片中嘅路人、擴展圖片邊界、風格轉換、甚至根據文字描述生成圖片。
- **實時翻譯**：通話實時翻譯、面對面對話翻譯、影相翻譯，出國旅行唔再有語言障礙。
- **學習輔導**：AI 可以解釋概念、出題練習、批改功課，係隨身家教。

端側大模型：手機自己跑 AI

最新嘅旗艦手機已經能喺本地運行小型 AI 大模型（通常 7B 參數左右），帶嚟嘅變化係：

- **唔需要網絡**：斷網都能用 AI 功能，飛機上、地鐵入面都得。
- **隱私更安全**：你嘅對話同埋數據唔會上傳到雲端，完全喺手機本地處理。
- **響應更快**：冇網絡延遲，AI 回覆幾乎係即時嘅。
- **當前限制**：端側模型嘅能力仲比唔上雲端大模型，複雜任務仍需聯網。但隨住晶片 AI 算力嘅提升，端側模型會越來越強。

系統級 AI 集成：

- **蘋果 Apple Intelligence**：iOS 18 引入嘅 AI 系統，集成寫作工具、圖像魔法棒、Siri 增強、通知摘要等功能。端側處理優先，複雜任務先上雲端。
- **華為盤古大模型**：集成到鴻蒙系統中，小藝助手獲得大模型能力，支援檔案摘要、智慧識屏、AI 修圖等。
- **小米澎湃 AI**：系統級 AI 能力，支援 AI 搜圖、AI 摘要、AI 實時字幕、端側大模型對話等。
- **三星 Galaxy AI**：通話實時翻譯、筆記 AI 摘要、圖片生成編輯、搜索圈選等功能。

五、幾個典型應用場景與實操

為咗等你更直觀咁感受手機 AI 嘅“超能力”，下面我哋用五個具體場景，展示 AI 係點樣喺日常生活中“出手相救”嘅：

場景 1：實時翻譯——出國旅行嘅隨身翻譯官

實際表現：喺巴黎點餐唔識法語？iPhone 嘅“翻譯”App 支援面對面雙人對話模式，雙方各睇自己語言嘅字幕，甚至可以同聲傳譯。安卓陣營嘅三星 Galaxy AI 通話實時翻譯能等你喺電話入面直接講中文，對方聽到嘅係英文，反之亦然。

背後原理：語音識別（ASR）機器翻譯（NMT）語音合成（TTS）三步流水線，端到端延遲已可控制喺 1 秒內。

場景 2：AI 消除——拯救被路人“毀掉”嘅遊客照

實際表現：喺景點影咗一張絕美嘅自拍，但背景入面擠滿咗其他遊客？用“魔法消除”圈選路人，手機會自動識別並將路人“塗掉”，再智能填充背景，效果成日令人驚艷。

背後原理：圖像分割（Inpainting）模型先識別並刪除選中區域，然後根據周圍像素同埋語義理解“腦補”出合理嘅填充內容。呢背後嘅 AI 模型訓練數據量高達數百萬張圖片。

場景 3：AI 修圖——一鍵變“攝影大師”

實際表現：影咗一張陰天嘅灰濛濛風景相？AI“魔法天空”可以一鍵將天空換成晴空、晚霞甚至星空。AI 擴圖可以等豎構圖變成橫構圖，手機“腦補”出畫面外嘅場景。

背後原理：基於生成式 AI（Generative AI）嘅擴散模型（Diffusion Model），理解圖像嘅語義內容後生成新像素。呢個係當前 AI 圖像處理嘅“最前沿”。

場景 4：檔案處理——辦公族嘅效率倍增器

實際表現：會議結束後，將會議錄音掟畀 AI，佢能自動整理出結構化嘅會議紀要、提取待辦事項、生成摘要。一份 10 頁嘅英文 PDF 報告，AI 可以喺 10 秒內畀你提煉出核心結論。

背後原理：長上下文大模型（Long Context LLM）可以“讀完”數萬字嘅檔案，進行多輪推理同埋資訊抽取。Apple Intelligence、華為小藝、WPS AI、騰訊元寶等都具備此能力。

場景 5：AI 搜圖與識物——你嘅“視覺搜索引擎”

實際表現：喺街度睇到一種唔識嘅花，相機一開，AI 即刻話你知“呢個係木槿花”。相簿入面翻搵舊年喺海邊影嘅相，搜索“大海”，AI 自動將所有含海嘅相搵出嚟，唔使手動翻。

背後原理：計算機視覺（CV）模型對圖片內容進行向量化（Embedding），再同文字查詢進行相似度匹配。呢種“多模態搜索”係手機相簿管理嘅一大飛躍。

六、提示詞優化：等 AI 聽你指揮嘅“溝通公式”

AI 嘅回答質量，好大程度上取決於你點樣“提問”。同一個問題，問法唔同，AI 畀嘅答案天差地別。下面我哋介紹一套**最實用、最容易上手嘅“角色-背景-任務-限制”溝通思路**，並配以大量實例。

點解需要框架？

好多人用 AI 似用百度嗷——敲幾個關鍵詞入去。但 AI 唔係搜索引擎，佢係一個“閱讀理解後回答”嘅語言模型。**畀佢更多上下文、講清你要乜、限制佢唔好跑偏**，佢就能畀你精準嘅結果。框架就係將你腦入面嘅“好答案長點樣”翻譯成 AI 能理解嘅指令。

角色（Role）：話畀 AI 聽佢係邊個

畀 AI 一個具體嘅“身份”，佢就會調用對應領域嘅專業知識同埋表達方式。**角色設定得越具體，輸出越專業**。

實用示例：

- ”你係一位**有 10 年經驗嘅小米手機維修工程師**。”（技術問題）
- ”你係一位**擅長將複雜概念講簡單嘅中學物理老師**。”（科普問題）
- ”你係一位**審美好、讀得多、有自己風格嘅圖書編輯**。”（寫作問題）
- ”你係一位**每日畀上班族做早餐嘅烹飪博主**。”（生活問題）
- ”你係一位**耐心、溫同、永遠唔嘲笑新手嘅中醫**。”（健康問題）

點解有效？角色設定咗 AI 嘅**知識範圍、表達風格**同埋**專業深度**。冇角色嘅 AI 係個“全才但乜都唔精”嘅通才；有咗角色，佢能“進入”嗰個身份，似嗰位專業人士嗷思考。

背景（Background）：話畀 AI 聽你嘅情況

將“你係邊個、你嘅現狀、你嘅約束條件”提前講清楚，AI 就能畀出**真正適合你**嘅答案，而唔係泛泛而談。

實用示例：

- ”我嘅手機係**小米 14，買咗半年，最近打機時發燙嚴重**。”（設備情況）
- ”我係一個 60 歲嘅退休阿姨，眼唔太好，唔太識用智能手機。”（用戶畫像）
- ”我正在**準備 2026 年 6 月嘅英語六級考試，基礎一般**。”（學習背景）
- ”我開咗一間**三線城市嘅小咖啡店，月流水 3 萬，請咗兩個員工**。”（商業背景）
- ”我啱啱生完 **BB 喺屋企坐月，睡眠嚴重不足**。”（生活階段）

點解有效？背景資訊等 AI 知道**“為邊個服務”**同埋**“喺乜嘢條件下”**。同一道題，畀學生同畀家長講法唔同；畀城入麪人同畀鄉鎮人建議都唔同。背景缺失，AI 淨係可以畀“最大公約數”嘅答案，往往唔痛唔癢。

任務（Task）：明確話畀 AI 聽做乜

呢一步要用**動詞開頭**（寫、列、對比、翻譯、解釋、總結……），將任務描述得**具體、可執行**。含糊嘅任務淨係會換嚟含糊嘅答案。

實用示例：

- ”請**列出 5 個我應該即刻關閉嘅權限**，並說明原因。”（動作明確）
- ”請**將下面呢段英文郵件翻譯成中文**，保持商務口吻。”（任務具體）
- ”請**對比呢四款 3000 元價位嘅手機**，畀出一張優缺點對比表。”（格式明確）
- ”請**用小學生能明嘅話解釋’乜嘢係 5G’**。”（風格明確）
- ”請**畀我嘅早餐店寫三條短影片文案**，分別針對上班族、情侶、帶仔女嘅家長。”（細分明確）

點解有效？任務部分係 AI 行動嘅“指令中樞”。**動詞越清晰、要求越具體，AI 越知道“交付乜嘢”**。模糊嘅“搞下”、“整個”、“睇下”係 AI 大忌——佢會揣測你嘅意思，結果成日跑偏。

限制（Constraints）：話畀 AI 聽唔好做乜

“限制”聽起嚟似“約束”，其實係**等 AI 收斂到最適合你嘅輸出**嘅利器。常見維度有：字數、格式、語氣、避免項、必含項。

實用示例：

- ”**唔超過 200 字，分點列出**。”（長度 + 格式）
- ”**語氣要親切自然，似朋友傾偈**，唔好用’首先、其次、再次’呢種官話。”（語氣）
- ”**唔好使用專業術語**，遇到專業名詞要順便解釋。”（用詞限制）
- ”**每段唔超過 3 行**，方便手機閱讀。”（排版）
- ”**必須包含：使用場景、優缺點、我嘅預算匹配度**。”（必含項）
- ”**唔好畀具體投資建議**，淨係解釋原理。”（邊界）

點解有效？冇限制，AI 容易“自由發揮”——要麼太學術、要麼太長、要麼跑題。明確嘅限制 = 明確嘅“交付標準”，雙方對齊預期，結果先至可控。

提示詞質量評估：好提示詞嘅 5 個特徵

寫好提示詞唔係一次性嘅事，**寫完先自查呢五條**，再發畀 AI：

1. **具體而非抽象**：唔係” 寫篇文章”，而係” 寫一篇 800 字、3 個小節、面向新手媽媽嘅手機選購指南”。
2. **包含背景而非孤立**：話畀 AI 聽” 我係邊個、點解問呢個”，佢先至能畀出” 為你定製” 嘅答案。
3. **有明確輸出格式**：係表格、列表、段落、定係 JSON？提前講。
4. **限制到位**：字數、風格、避項，三選一就有明顯提升。
5. **可被 AI 驗證**：問完之後，等 AI 複述一遍你嘅要求；講唔清楚就係冇講清楚。

常見問題與解法

症狀	解法
回答太泛、唔實用	加背景（你係邊個/乜嘢場景）
回答太學術/太官方	加語氣限制 + 拒絕” 官話” 清單
回答跑題/答非所問	將任務用動詞重新講一遍，並要求” 淨係回答呢個”
回答太長睇唔完	硬性限制字數（200 字內）
AI 一本正經胡說	加” 如不確定請標註’ 我不確定’” + 重要事實自查
第一版唔滿意	唔好重寫，喺原提示詞後追加” 再…一啲”

三個完整場景案例

場景一：生活助手（早餐搭配建議）

提示詞：
你係一位擅長營養搭配嘅早餐博主（**角色**）。我係一位上班族媽媽，每日朝早淨係得 15 分鐘做早餐，屋企有 6 歲嘅細路同埋 60 歲嘅奶奶（**背景**）。請**列出 7 日唔重樣嘅早餐搭配**（**任務**）。每份早餐要求：(1) 準備時間唔超過 15 分鐘 (2) 營養均衡（蛋白 + 碳水 + 蔬果）(3) 老少皆宜 (4) 食材喺普通超市能買到（**限制**）。
點解有效：角色定專業度，背景畀三類人共性，任務明確交付物，4 條限制確保可執行。

場景二：學習輔助（英語六級備考）

提示詞：
你係一位**經驗豐富嘅英語六級老師**（角色）。我係一名普通本科生，**四級 480 分，六級目標 500，仲有 3 個月**（背景）。請**為我制定一份可執行嘅 12 週備考計劃**（任務）。要求：(1) 每週明確聽說讀寫嘅時間分配 (2) 畀出每個月嘅重點突破方向 (3) 推薦 3 本合適嘅複習資料（中文說明理由）(4) 表格形式輸出（限制）。

點解有效：角色定專業背景，背景畀出分數同埋時間錨點，任務明確“分階段計劃”，限制要求“表格化”同埋“可執行”。

場景三：工作效率（會議紀要整理）

提示詞：
你係一位**資深行政助理**（角色）。我啱啱開完一個 1 小時嘅部門會議，**討論 Q2 營銷推廣方案**（背景）。**請將以下會議記錄整理成結構化紀要**（任務）。要求：(1) 分「決議事項 / 待辦事項 / 風險與異議」三部分 (2) 待辦事項標出負責人同截止時間（如未提到則寫「待確認」）(3) 總字數控制喺 600 字內 (4) 用 markdown 格式輸出（限制）。
點解有效：角色定行政視角，背景畀會議主題，任務明確結構化輸出，4 條限制等結果可以直接重用。

進階玩法：等 AI 成為你嘅「私人助理」

試嚇呢啲進階用法，體驗 AI 嘅真正實力：

- **多輪對話優化**：對 AI 嘅回答唔滿意時，可以講「呢個太專業喇，請用小學生聽得明嘅話再解釋一次」。
- **鏈式思考（CoT）**：等 AI「一步步思考」，回答質素通常更高：「請一步步分析呢道數學題」。
- **多 AI 協作**：用 A 寫初稿，B 潤色，C 翻譯——組合使用效果更佳。
- **系統提示詞**：喺 AI 設定中預設你嘅身份、偏好、忌諱等，等回答更貼合你嘅需求。

七、AI 嘅侷限與挑戰：保持清醒嘅判斷力

AI 雖然強大，但絕非「無所不能」。清醒地認識佢嘅侷限，先至可以更好咁使用佢：

- **「幻覺」問題**：AI 可能一本正經咁胡說八道（專業術語叫 Hallucination）。佢會編造唔存在嘅歷史事件、錯誤嘅引言、虛假嘅統計數據。重要資訊必須人工核實。
- **知識截止**：大多數 AI 嘅訓練數據有截止日期（如 2024 年初），佢唔知之後發生嘅事。
- **推理能力有限**：複雜嘅多步推理、數學證明、邏輯陷阱，AI 仍然容易出錯。
- **私隱風險**：喺雲端 AI 服務中，你嘅對話可能被用於模型訓練。敏感資訊應優先使用端側 AI。
- **偏見與價值觀**：AI 可能帶有訓練數據中嘅偏見，需要保持獨立思考。

AI 使用小貼士

- AI 生成嘅內容可能唔準確（「幻覺」問題），重要資訊務必核實。
- 唔好向 AI 輸入個人私隱資訊（身份證號、密碼、銀行資訊等）。

- 善用提示詞（Prompt）：越具體、越清晰嘅描述，AI 嘅回答質素越高。
- AI 係工具唔係權威，保持獨立思考同判斷力。
- 遇到關鍵決策（醫療、法律、財務），諮詢專業人士而唔係依賴 AI。

嚟挑戰：三道 AI 實戰題

1. **挑戰一：複雜指令：**對語音助手講：「明早 7 點提醒我帶遮，同埋將今日下晝 3 點嘅日曆改到 4 點。」睇吓佢能否一次完成多步操作。
2. **挑戰二：AI 修圖：**影一張有路人嘅遊客照，使用「魔法消除」或「AI 橡皮擦」功能，將路人清除乾淨。觀察 AI 對背景嘅「腦補」效果。
3. **挑戰三：提示詞優化：**先用簡單提示詞「寫一段旅遊攻略」問 AI，再用 CRISPE 框架寫一個詳細提示詞。比較兩次回答嘅質素差異。

案例 2：換機數據遷移完整流程

- **場景：**iPhone 13 小米 15 跨品牌換機。
- **步驟：**(1) 舊機 iCloud/微信自帶備份 (2) 退出 Apple ID + 關閉「查找」(3) 新機用「小米換機」App 掃碼 (4) 選擇遷移項 (5) 微信單獨用自帶「聊天記錄遷移」(6) 驗證關鍵數據（通訊錄/相冊/微信）(7) 舊機加密擦除。
- **預期結果：**數據零丟失，舊機可安全轉賣。

第二十三章：編程思維與自動化啟蒙

前兩章，我哋畀手機築起咗安全嘅「邊界牆」，亦見識咗佢喺牆內日益聰明嘅「*AI* 大腦」。而家，等我哋解鎖手機嘅終極潛力之一：等佢好似程序員咁「思考」，並自動完成你設定嘅連續任務。你可能會講：「我又唔識寫代碼，呢樣同我有乜嘢關係？」放心，本章講嘅根本唔需要你寫一行代碼——現代手機已經將「編程」嘅門檻降到拖拖拽拽就搞得掂嘅程度。

一、自動化到底係個乜？

用大白話講，自動化就係：**等手機喺某個條件滿足時，自動幫你做一連串事**。你唔需要每次都手動操作，亦唔使記住要做乜——手機會自己判斷時機，自己執行。

舉個例：每日朝早你醒咗，要手動閂掉勿擾模式、打開微信睇嚇消息、打開天氣 App 睇今日著乜、打開網易雲放首歌。如果將呢啲步驟打包成一個自動化流程，設定成「每日朝早 7 點自動執行」，咁你醒咗只需要攞起手機，一切都已經準備好喇。係咪諗嚇就好爽？

自動化本質上就係「如果……咁樣……」嘅邏輯鏈條，編程入面便嘅 if-else。學識呢個思維方式，哪怕你一世唔寫代碼，處理問題嘅條理性都會強好多。核心要素：

1. **觸發條件**：乜嘢時候開始？可以係「每日朝早 7 點」、「當我連接到屋企嘅 Wi-Fi」、「當我到達某個地點」、「當手機電量低於 20%」。

2. **執行動作**：具體做乜？可以係發一條微信、打開某個 App、調整音量、切換桌布、播放音樂、發送通知……幾乎所有你喺手機上手動做嘅事，自動化都識做。

3. **中間邏輯**（可選但好強大）：如果情況 A 就做 X，否則做 Y。比如「如果係工作日就播放新聞播客，週末就播放音樂」，呢個叫條件分支。

二、iOS「快捷指令」——蘋果用戶嘅自動化神器

蘋果嘅「快捷指令」（Shortcuts）App 係自帶嘅應用，唔使下載。佢嘅核心概念就係將各種操作好似砌積木咁拼埋一齊。先講幾個關鍵概念：

- **「快捷指令」**：一個完整嘅自動化流程，可以手動撳運行，亦可以透過 Siri 語音喚醒。比如你講「嘿 Siri，返屋企模式」，佢就自動執行你設定好嘅一連串操作。

- **「自動化」**：由觸發條件自動運行嘅指令，唔需要你手動觸發。比如「當手機連接到車載藍牙時，自動打開地圖 App」。自動化喺「自動化」標籤頁下設定，同「快捷指令」係分開嘅。
- **「動作」**：最細嘅執行單元。打開 App 係一個動作，發送消息係一個動作，獲取天氣係一個動作，調整亮度係一個動作。你可以將成百上千個動作串埋一齊。

畀你睇幾個實實喺喺好用嘅例子，你跟住做一遍就明喇：

實例一：一鍵「早安流程」

每日朝早點一下（或對 Siri 講「早晨」），手機會自動：獲取今日嘅天氣朗讀天氣同氣溫顯示今日嘅日曆安排打開你常用嘅音樂 App 開始播放。操作步驟：新建快捷指令添加動作「獲取當前天氣」添加「朗讀文本」添加「獲取日曆事件」添加「顯示提醒」添加「打開 App」選網易雲音樂。就咁簡單，全程拖拽。

實例二：自動「省電模式」

喺「自動化」標籤頁，新建自動化選「電池電量」設「低於 30%」添加動作「設定低電量模式為開啟」、「降低螢幕亮度到 30%」、「關閉藍牙」。從此手機電量低於 30% 就自動進入省電狀態，唔使你操心。

實例三：掃碼自動記賬

每次你用微信/支付寶掃碼支付後，手機會讀取支付通知，自動將金額同商家記錄到一個備忘錄或 Numbers 表格入面便。呢個需要用到快捷指令嘅「自動化」入面便嘅「收到短訊/通知」觸發條件（iOS 17+ 支援讀取錢包交易），配合「追加到 Numbers」動作。對於想記賬又懶得手動記嘅人嚟講，簡直係救星。

仲有一個好多人唔知嘅巧用：你可以將快捷指令添加到主螢幕，變成一個 App 圖標。比如做一個「打嘅返屋企」嘅快捷指令，圖標放桌面度，點一下就直接打開滴滴並將目的地設成屋企，比你自己打開 App 慢慢輸入快得多。

三、安卓自動化——Tasker 同佢嘅朋友哋

安卓呢邊，情況稍微複雜啲，但天花板亦更高。先講簡單嘅：

入門級：自動撳器

好多安卓手機自帶「自動撳器」或者可以喺應用商店下載。佢嘅原理超簡單：你手動操作一遍，佢將你撳嘅坐標同時間間隔錄低，然後可以高速回放。比如某個 App 每日要領簽到獎勵，你錄一遍撳流程，以後每日自動跑一遍就完事。缺點係：佢淨係認坐標，如果介面佈局變咗（比如彈咗個廣告），佢就點歪咗。

進階級：Tasker——安卓自動化之王

Tasker 係安卓平臺上公認最強嘅自動化工具，常年霸榜。講真佢嘅學習曲線有啲陡，介面唔算好睇，但一旦上手，你會覺得手機突然變得無所不能。Tasker 嘅核心都係「觸發任務」嘅模式：

- **配置檔案（Profile）**：定義「乜嘢時候觸發」。觸發條件可以係時間、地點、Wi-Fi 連接狀態、藍牙連接、收到特定通知、電池狀態、螢幕朝向……Tasker 支援嘅觸發條件多到離譜，有幾十種。
- **任務（Task）**：定義「觸發後做乜」。一個任務可以包含多個動作，按順序執行。動作類型包括：調整系統設定（音量、亮度、Wi-Fi、藍牙、飛行模式）、發送短訊、HTTP 請求、執行 Shell 命令、控制媒體播放、彈出對話框等等。

畀你幾個 Tasker 嘅經典實用場景：

- **到埗自動開 Wi-Fi**：配置觸發條件為「連接到特定基站」（因為你喺屋企附近會連到固定嘅基站）任務「開啟 Wi-Fi」。唔使 GPS 亦唔耗電，手機一入小區範圍就自動開 Wi-Fi。
- **會議模式**：觸發條件為「日曆事件標題包含『會議』」任務「設定靜音模式、關閉媒體音量、自動回覆微信（需要插件配合）」。再亦唔使擔心開會時手機突然響起。
- **深夜護眼**：觸發條件為「時間 22:00-06:00」任務「開啟護眼模式、降低亮度到 20%、將螢幕設為黑白（灰度模式）」。黑白螢幕對掃短影片嘅慾望有奇效——畫面一灰，你就唔想掃喇。
- **耳機插入自動播客**：觸發條件為「有線耳機插入/藍牙耳機連接」任務「打開小宇宙 App 並開始播放最新一集」。戴低耳機就能聽，唔使解鎖手機搵 App。

如果你覺得 Tasker 上手太難，可以試嚇佢嘅簡化版兄弟：**MacroDroid**。介面友善好多，模板豐富，適合唔想折騰嘅用戶。另外而家好多國產安卓系統（小米嘅 MIUI、華為嘅鴻蒙、OPPO 嘅 ColorOS）都自帶咗「場景」或「智能助手」功能，雖然冇 Tasker 咁強大，但日常自動化完全夠用。

四、自動化思維嘅日常應用

學識自動化最大嘅收穫，其實唔係慳嗰啲時間，而係**改變你思考問題嘅方式**。一旦你習慣咗「如果發生 X，就自動做 Y」嘅思維模式，你會發現生活中一大堆重複勞動都可以被幹掉。比如：

- 每次影完相，自動備份到網盤然後將本地相片壓縮成小尺寸慳位。
- 每日朝早自動由日曆讀取今日嘅安排，生成一段文字透過微信發畀家人。
- 收到快遞簽收短訊後，自動喺備忘錄入面便添加一條待辦「攞快遞」。

- 手機電量充滿時自動發出語音提醒拔低充電器（避免過充傷電池）。
- 每日收工時間到咗，如果仲喺公司附近（透過基站定位），自動提醒「該收工喇」。

呢啲聽落好高級，但其實你只要掌握咗「觸發 + 動作」呢兩個基本概念，配合快捷指令或 Tasker，都識做出嚟。關鍵係**你要先意識到「呢件事可以自動化」**——好多人唔係唔識，係根本冇往嗰方面諗。

五、踩坑指南：自動化嘅常見問題

搞自動化亦唔係一帆風順嘅，有幾個坑我幫你先踩咗：

- **位置觸發可能唔靈**：iOS 嘅「到達/離開位置」觸發用嘅係地理圍欄技術，準確度取決於 GPS 信號同系統優化。有時你都到咗屋企十分鐘先至觸發。建議用 Wi-Fi 連接或基站觸發代替位置觸發，更可靠亦更省電。
- **快捷指令嘅「自動化」需要授權**：iOS 入面便，「自動化」標籤下嘅觸發大多數需要你手動確認先至可以運行（iOS 嘅安全機制）。但 Wi-Fi 連接、藍牙連接、鬧鐘停止呢幾個觸發係可以真正全自動運行嘅，唔需要確認。
- **Tasker 需要開啟無障礙權限**：好多高級功能（比如模擬撳）需要無障礙服務權限，呢個會等手機喺設定入面便顯示一個常駐通知。呢個係安卓嘅安全設計，冇得去掉。
- **唔好將自動化搞得太複雜**：啱啱開始做自動化，由「2-3 個動作」嘅小流程開始。唔好一嚟就搞一個 20 步嘅超級流程——調試起嚟會令人崩潰。

小貼士：自動化入門嘅正確姿勢

先花 5 分鐘諗一下：你每日喺手機上重複最多嘅 3 件事係乜？比如每日朝早嘅例行操作、每次連藍牙耳機後嘅習慣、每次出門前嘅設定……由呢啲高頻小事入手，各做一個簡單嘅自動化。哪怕每個自動化淨係慳你 5 秒，日積月累都係可觀嘅。而且做出嚟之後嘅滿足感，真係會上癮。

練一練：三個入門自動化挑戰

選一個最適合你嘅完成，做完你就入門喇：

1. **挑戰一：早安自動化**：iOS：喺快捷指令 App 入面便新建快捷指令添加「獲取當前天氣」「朗讀文本」「打開 App 選網易雲」。完成後你可以對 Siri 講「早晨」嚟觸發。安卓：喺 Tasker 入面便新建任務添加「獲取天氣」「朗讀」「打開 App」設定觸發時間為每日 7:00。
2. **挑戰二：低電量自動省電**：iOS：自動化標籤新建「電池電量」低於 25% 添加「設定低電量模式開」+「設定亮度 25%」。安卓：Tasker 新建 Profile 狀態電源電量 0-25% 任務「省電模式開」+「亮度調低」。
3. **挑戰三：連接車載藍牙自動導航**：iOS：自動化新建「藍牙」選擇你嘅車載藍牙添加

「打開 App 選高德地圖」+「媒體音量調至 80%」。安卓：TaskerProfile 狀態藍牙已連接選擇車載藍牙任務「打開高德」+「媒體音量調高」。

案例 3：自動化「早安流程」

- **場景：**每日 7:00 自動播報天氣 + 關閉靜音 + 打開新聞。
- **步驟：**iOS: 快捷指令自動化時間觸發添加「獲取天氣」+「設定音量」+「打開 App」;Android: MacroDroid 觸發器 (時間 7:00) 動作 (媒體音量/打開 URL/TTS 朗讀天氣)。
- **預期結果：**免安裝即可實現，降低上手門檻。

第二十三章附：GitHub 入門指南——由代碼託管到協作平臺

上一章我哋講咗「編程思維」，而真正要將編程思維落地、同他人一齊做項目，就繞唔開一個名——*GitHub*。無論你係計算機系學生要參加比賽、想保存自己嘅代碼作品，仲係純粹想睇嚇全球開發者喺度折騰乜，*GitHub* 都係你必須認識嘅平臺。呢一節我哋由最基礎嘅概念講起，幫你建立一個清晰嘅認知。

一、GitHub 到底係乜？

簡單嚟講，GitHub 係一個基於 Git 嘅**代碼託管與協作平臺**。佢喺 2008 年上線，2018 年被微軟收購，至今仍係全球最大嘅開源代碼託管平臺，託管住數億個倉庫，覆蓋幾乎所有主流編程語言。

如果用一個類比嚟理解：**Git 係你電腦上嘅日記簿，GitHub 就係一個公開嘅部落格平臺**——你可以發布自己嘅作品，關注別人嘅項目，參與討論同協作。佢唔單止係「程序員嘅後臺倉庫」，更好似全球開發者共建嘅一座「技術城市」——項目喺呢度誕生、迭代、被討論、被重用。

對普通人嚟講，GitHub 亦並非遙不可及。好多非程序員會用佢嚟託管筆記、電子書、個人部落格，甚至用嚟備份設定檔案。只要你有一點點「想將自己嘅嘢整理好同分享出去」嘅需求，GitHub 都值得你瞭解。

二、Git 同 GitHub 嘅區別（關鍵概念！）

好多人第一次接觸時會混淆呢兩個詞，呢度幫你梳理清楚：

對比項	Git	GitHub
本質	版本控制工具（軟件）	代碼託管平臺（網站）
運行位置	本地電腦上	雲端（瀏覽器訪問）
核心職責	記錄檔案嘅每次變更（點改嘅、邊個改嘅、幾時改嘅）	將 Git 倉庫放到雲端，方便儲存、展示、協作
係咪需要網絡	唔需要，離線可用	需要聯網訪問
歸屬	開源軟件，由 Linus Torvalds（Linux 之父）創建	微軟旗下產品
同類競品	SVN、Mercurial	GitLab、Gitee（碼雲）、Bitbucket

簡單講：**Git 管「點樣記錄變更」，GitHub 管「變更放喺邊、點樣協作」**。Git 係工具，喺本地運行；GitHub 係平臺，將 Git 倉庫放到雲端，方便協作同展示。你可以淨係用 Git 唔用 GitHub（比如自己寫代碼做版本管理），但用 GitHub 一定要用 Git（因為 GitHub 就係基於 Git 嘅）。

三、GitHub 嘅核心概念與功能

基礎概念

喺動手之前，先認識幾個高頻詞彙，否則睇 GitHub 嘅介面會一頭霧水：

- **倉庫（Repository，簡稱 Repo）**：一個項目對應一個倉庫，入面便存放住呢個項目嘅所有代碼檔案、歷史版本同文檔。可以理解為一個「項目資料夾」。
- **提交（Commit）**：對代碼嘅一次修改記錄。每次 Commit 都附帶說明資訊，記錄「呢次改咗乜、點解改」。呢個係 Git/GitHub 識得「時光倒流」嘅基礎。
- **分支（Branch）**：由主線上分出嚟嘅獨立開發線。比如你想加個新功能，可以開一個分支單獨開發，開發完再合併返主線，唔影響其他人。
- **克隆（Clone）**：將雲端倉庫完整複製一份到本地電腦，之後就可以喺本地修改代碼。
- **推送（Push）/拉取（Pull）**：Push 係將本地代碼上傳到 GitHub，Pull 係由 GitHub 下載最新代碼到本地。呢個係團隊協作時同步代碼嘅兩個核心動作。
- **合併請求（Pull Request，簡稱 PR）**：當你喺一個分支上改完代碼，想合併到主分支時，發起一個 PR，等團隊成員審查代碼後再合併。呢個係 GitHub 協作嘅核心機制。
- **議題（Issue）**：用嚟記錄 bug、功能需求、討論話題。每個 Issue 好似一個討論帖，可以指派負責人、打標籤。
- **Fork**：將別人嘅倉庫複製一份到自己帳號下，你可以喺自己嘅副本上隨意修改，唔影響原倉庫。改好後可以畀原作者發 PR，請求合併你嘅改進。

核心功能

GitHub 提供嘅唔單止係代碼儲存，仲包括：

- **版本控制**：記錄每次代碼更改嘅歷史，改壞代碼隨時返到上一個正常版本。
- **團隊協作**：支援多用戶對同一項目嘅並行開發，多人同時開發唔會覆蓋、唔會衝突。
- **問題跟蹤**：用 Issue 記錄 bug 同功能請求，方便團隊跟進。
- **項目管理**：整合咗待辦事項、項目睇板（Projects）等管理工具，相當於內置咗一個輕量級 Trello。
- **自動化**：透過 **GitHub Actions** 實現代碼提交後自動編譯、測試、打包、部署，係現代「CI/CD（持續集成/持續部署）」嘅核心工具。
- **文檔管理**：README 介紹項目，Wiki 存放詳細架構同開發規範，Pages 甚至可以直接託管靜態網站。
- **代碼託管與展示**：每個倉庫都有靚嘅網頁展示，支援語法高亮，係你技術作品嘅「門面」。
- **開源社區**：你可以 Star（收藏）喜歡嘅項目、Watch（關注）項目動態、Follow（關注）開發者，參與全球開源協作。

四、註冊與上手：五分鐘創建你嘅第一個倉庫

註冊帳號

1. 訪問 github.com，撳「Sign up」。
2. 填寫郵箱、設定密碼同用戶名（用戶名會出現喺你嘅個人主頁地址入面便，建議用英文，比如 `github.com/yourname`）。
3. 完成郵箱驗證同人機驗證。
4. 選擇免費方案（Free）即可，對個人用戶完全夠用。

創建第一個倉庫

1. 登入後撳右上角「+」號，選「New repository」。
2. 填寫倉庫名（Repository name），比如 `my-first-project`。
3. 寫一段簡短描述（Description），等別人知道呢個倉庫係做乜嘅。
4. 選擇公開（Public）或私有（Private）。公開倉庫所有人可見，私有淨係自己同協作者可見。免費帳號亦支援無限私有倉庫。
5. 勾選「Add a README file」——README 係項目嘅「說明書」，每個倉庫都建議有。
6. 撳「Create repository」，你嘅第一個 GitHub 倉庫就誕生喇！

小知識：README 係乜

README.md 係一個 Markdown 格式嘅檔案，係別人打開你倉庫時第一眼見到嘅內容。一個好嘅 README 通常包含：項目簡介、安裝方法、使用示例、許可證、聯繫方式。寫好 README 係程序員嘅基本素養，亦係項目能否被他人使用嘅第一道門檻。

五、基本工作流：由本地到雲端

理解 GitHub 嘅工作流，核心係記住呢個循環：**本地修改暫存提交推送雲端更新**。下面用大白話解釋每一步：

1. **修改檔案**：喺本地電腦上用編輯器寫代碼或改文檔。
2. **git add**：將修改嘅檔案加入「暫存區」，相當於話畀 Git 聽「呢啲改動我準備記錄低」。
3. **git commit**：將暫存區嘅改動正式記錄成一個「提交」，附上一句說明，比如「修復登入 bug」。呢一步淨係喺本地完成。
4. **git push**：將本地嘅提交上傳到 GitHub 雲端，咁別人就能睇到喇。
5. **git pull**：喺開始工作前，先由雲端拉取最新代碼，確保本地同團隊其他人保持同步。

對新手嚟講，記住呢個口訣：「**先 pull 後改，改完 add，commit 帶說明，最後 push 上去**」。養成呢個習慣，團隊協作就唔會亂。

六、團隊協作流程：Pull Request 係點樣工作嘅

GitHub 最強嘅地方喺佢嘅協作機制。一個典型嘅團隊協作流程係咁樣：

1. **開分支**：由主分支（通常叫 main 或 master）開一個新分支，比如 `feature/login`，喺呢個分支上開發新功能。
2. **開發並提交**：喺新分支上寫代碼，多次 commit 記錄進度。
3. **推送到 GitHub**：將分支 push 到雲端倉庫。
4. **發起 Pull Request**：喺 GitHub 網頁上發起 PR，請求將你嘅分支合併到主分支。寫清楚呢個 PR 做咗乜改動、點解改。
5. **代碼審查（Code Review）**：團隊成員查看你嘅代碼，提出意見或建議修改。你可以根據反饋繼續 commit，PR 會自動更新。
6. **合併**：審查通過後，撳「Merge」將代碼合併到主分支。你嘅貢獻就正式進入項目喇！

呢套流程保證咗**任何代碼進入主分支前都經過審查**，大大降低咗出錯概率。開源項目亦係用呢套機制接受全世界陌生人嘅貢獻——你完全可以喺某個知名開源項目上提交 PR，被採納後你就係呢個項目嘅貢獻者喇。

七、GitHub 嘅典型應用場景

學生參加比賽與課程項目

計算機系學生參加 ACM、黑客松、畢業設計，幾乎都離唔開 GitHub。佢既可以作為團隊協作平臺，又能作為作品展示視窗——比賽評委可以直接睇你嘅倉庫，瞭解代碼質素同協作過程。好多公司招聘時亦會要求附上 GitHub 主頁，作為技術能力嘅證明。

個人知識庫與筆記託管

非程序員都能用 GitHub。好多人用佢託管學習筆記、技術部落格（配合 GitHub Pages 可以免費建站）、電子書。Markdown 格式寫作 + Git 版本管理 + 雲端備份，三合一。

參與開源項目

GitHub 係全球開源運動嘅核心陣地。你可以 Fork 喜歡嘅項目，修改後發 PR 回饋社區；亦可以純粹 Star 收藏好項目，跟蹤更新。參與開源係提升技術、積累聲譽嘅絕佳途徑。

自動化部署與 CI/CD

藉助 GitHub Actions，可以實現「代碼一 push，自動測試 + 自動部署到伺服器」嘅全自動流程。好多個人部落格、文檔站點都係咁樣搭建嘅，寫完文章 push 一下，網站就自動更新喇。

代碼備份與多設備同步

將代碼倉庫當作雲盤用，換電腦、多設備辦公時，git pull 一下就能攞到最新代碼，再亦唔使 USB 手指拷嚟拷去。

八、畀新手嘅幾點建議

- **提交說明要寫清楚**：唔好寫「update」呢種廢話說明。好嘅 commit message 寫「修復用戶登入後跳轉錯誤頁面嘅問題」，半年後回睇都知當時做咗乜。
- **勤提交，小步快跑**：每完成一個小功能就 commit 一次，而唔係寫咗一成日先至提交一次。咁樣出問題更容易定位。
- **用分支開發，唔好直接改主分支**：哪怕係個人項目，養成開分支嘅習慣，未來團隊協作時就唔會手忙腳亂。
- **善用.gitignore**：呢個檔案用嚟話畀 Git 聽邊啲檔案唔好納入版本管理（比如密碼設定、編譯產物、依賴資料夾），避免將垃圾檔案傳上去。
- **多睇優秀開源項目**：行下 GitHub Trending，睇嚇高星項目嘅代碼結構、README 寫法、協作流程，係提升最快嘅免費方式。

- **申請 GitHub 學生包**：學生認證後可免費獲得大量付費工具同服務嘅額度（Copilot、域名、雲服務等），價值上千美元。

安全提醒

- 千萬唔好將密碼、API 密鑰、數據庫連接串等敏感資訊提交到 GitHub，尤其係公開倉庫——一旦洩露，會被爬蟲秒抓，後果嚴重。
- 唔小心提交咗敏感資訊，淨係刪除檔案唔夠（歷史記錄入面便仲有），要用 `git filter-branch` 或專業工具清理歷史，並立即更換洩露嘅密鑰。
- 私有倉庫亦唔係絕對安全，敏感資訊最好用環境變數管理，唔入代碼倉庫。

國內訪問 GitHub 嘅注意事項

GitHub 喺國內訪問有時唔穩定，可能出現打唔開、克隆慢嘅情況。可以嘗試：(1) 使用國內鏡像站（如 GitClone、ghproxy）加速克隆；(2) 配置 Hosts 或使用代理；(3) 如果主要面向國內協作，亦可以考慮國產替代品 **Gitee（碼雲）**，功能類似且訪問速度快。好多團隊會同時喺 GitHub 同 Gitee 上託管，兼顧國際化同國內速度。

進階玩法：用 GitHub Pages 搭建免費個人部落格

GitHub Pages 允許你免費託管一個靜態網站，配合 Jekyll、Hexo、Hugo 等靜態站點生成器，可以輕鬆搭建個人部落格。寫完 Markdown 文章 push 到倉庫，部落格自動更新，仲自帶 HTTPS 同自訂域名。無數技術部落格嘅部落格都係咁樣搭嘅，零成本、無廣告、版本可控。

九、總結：點解每個人都應該瞭解 GitHub

GitHub 早已唔單止係「程序員嘅工具」，佢更好似一個**數字時代嘅工作方式**：版本管理等一切可追溯，協作機制等團隊高效運轉，開源精神等知識自由流動。哪怕你唔寫代碼，理解「提交、分支、合併」呢啲概念，都能幫你更好咁管理任何需要「反覆修改 + 多人協作」嘅內容——由論文到文檔，由筆記到方案。

對計算機系嘅學生嚟講，GitHub 更係必備技能：佢係你嘅作品集、協作平臺、學習資源庫，亦係未來求職時最有說服力嘅「簡歷」。早啲註冊帳號，將自己嘅課程作業、小項目傳上去，慢慢積累，幾年後你會擁有一份等面試官眼前一亮嘅技術履歷。

嚟動手：完成你嘅 GitHub 初體驗

1. 註冊一個 GitHub 帳號，完善個人資料（頭像、簡介、感興趣嘅技術領域）。
2. 創建一個名為 `hello-github` 嘅公開倉庫，勾選「Add a README file」。
3. 進入倉庫，撳 README.md 嘅編輯按鈕（鉛筆圖標），用 Markdown 寫一段自我介紹，提交更改。
4. 喺 GitHub 搜尋欄搜尋一個你感興趣嘅關鍵詞（比如 `python-tutorial`），Star 兩個你喜歡嘅項目。

5.（進階）喺電腦上安裝 Git，將倉庫 Clone 到本地，修改 README 後 push 返雲端，完成一次完整嘅「本地雲端」工作流。

第二十四章：Root 權限與 BL 鎖深度解析

呢一章係對第二十章內容嘅深入擴展，適合嗰啲想要真正理解手機底層機制、甚至嘗試刷機嘅用戶。請注意，呢啲操作有一定風險，請謹慎行事。

一、深入理解 Bootloader

Bootloader 係手機啟動時第一個運行嘅程式，佢嘅任務係初始化硬件、載入作業系統內核。可以將 Bootloader 理解為手機嘅「守門人」。

BL 鎖嘅工作原理：

- **驗證簽名**：BL 鎖會驗證啟動鏡像嘅數碼簽名，只有廠商簽名嘅系統先至可以啟動。
- **防止降級**：防止用戶刷入舊版本系統嚟利用已知漏洞。
- **保護數據**：即使刷入新系統，冇原帳號密碼亦無法訪問用戶數據。

硬件級儲存與驗證

BL 鎖嘅鎖定狀態（Locked/Unlocked）及廠商嘅公鑰，通常儲存喺受保護嘅硬件區域（如高通嘅 devinfo 分區、eFuse 或 RPMB 分區），防止被軟件篡改。每次啟動時，Bootloader 會由呢個安全區域讀取狀態，並據此決定係咪執行嚴格嘅簽名驗證。

數碼簽名驗證

廠商喺發布系統更新或鏡像時，會用自己嘅私鑰對佢進行簽名。設備啟動時，Bootloader 會使用內置嘅公鑰對系統鏡像（如 boot.img、recovery.img）進行驗證。佢嘅工作方式係：

1. 對系統鏡像重新計算哈希值。
2. 用公鑰解密鏡像自帶嘅簽名，得到一個哈希值。
3. 比較呢兩個哈希值，若一致則通過驗證。呢個確保咗系統鏡像未被第三方篡改。

安全啟動鏈

BL 鎖係整個安全啟動鏈嘅第一環。佢驗證通過後，先至會載入並執行下一階段嘅引導程式（如內核），從而層層確保系統啟動環境嘅可信。

鎖定 vs 解鎖：BL 鎖嘅雙面影響

為咗更直觀咁理解 BL 鎖嘅利弊，我哋用一個對比表嚟展示佢喺 5 個核心維度上嘅差異：

核心作用	鎖定狀態（預設）	解鎖後狀態
系統安全	防止刷入未簽名嘅非官方系統，保護系統完整性與底層安全 高安全：僅載入官方簽名鏡像，有效抵禦惡意韌體篡改	安全降低：允許刷入第三方 ROM，系統完整性校驗機制失效，惡意軟件若攞到 Root 權限可深度篡改系統
系統穩定性	確保手機運行經過廠商測試同優化嘅官方系統，保障穩定流暢 穩定：使用官方韌體，功能完整，OTA 升級可靠	風險增加：第三方 ROM 可能存在相容性問題，可能導致系統崩潰、變磚、硬件功能異常（如鏡頭、快充），OTA 更新可能失敗
保修權益	廠商透過鎖定 BL 提供穩定嘅官方系統，以提供保修服務 保修有效：享受官方免費保修服務（非人為損壞）	保修失效：絕大多數廠商將解鎖 BL 視為主動放棄保修嘅行為，即使重新上鎖，亦可能被檢測出歷史解鎖記錄而拒保
數據安全	解鎖過程通常強制清除數據，防止舊數據喺新系統環境下被惡意利用 數據安全：日常使用中，BL 鎖限制咗惡意程式對系統分區嘅修改能力	數據風險：解鎖時會強制清除所有用戶數據；若解鎖後未正確加密，數據儲存可能唔安全；遺失後更易被刷機提取數據
功能限制	確保依賴安全環境嘅功能（如支付、銀行 App）正常運行 功能完整：所有官方功能均可正常使用	功能受限：部分安全敏感應用（如 Google Pay、銀行 App）可能因檢測到 BL 解鎖狀態而拒絕運行；DRM 服務可能降級，影響高清影片播放

二、解鎖 BL 鎖嘅詳細流程（以小米為例）

1. **申請解鎖權限：**喺小米解鎖工具網站申請解鎖。

2. **綁定帳號：**喺手機設定中綁定小米帳號。

3. **等待期**：根據帳號等級，需要等待 7-30 日。
4. **解鎖操作**：使用小米解鎖工具連接電腦解鎖。

三、Root 嘅多種方案

- **Magisk**：現代 Root 方案，Systemless 設計，可隱藏 Root 狀態。
- **Kernelsu**：基於內核嘅 Root 方案，更底層，隱藏性更好。
- **SuperSU**：老牌 Root 方案，而家已經停止更新。

四、刷機入門

刷機係指畀手機安裝新嘅作業系統。常見嘅刷機類型：

- **刷官方 ROM**：安裝官方系統，用於救磚或降級。
- **刷第三方 Recovery**：如 TWRP，用於備份恢復、刷入第三方 ROM。
- **刷第三方 ROM**：如 LineageOS、Pixel Experience 等，體驗唔同嘅系統。

五、三星 Knox：BL 解鎖嘅不可逆代價

同其他安卓廠商唔同，三星手機嘅 BL 解鎖機制有一個獨特嘅「熔斷」設計：**對於三星手機嚟講，解鎖會觸發 Knox 熔斷，永久失去部分安全功能（如 Samsung Pay）；會清除所有受保護數據；影響保修**。具體嚟講：

- **熔斷器係一次性嘅**：手機內部有一顆 eFuse 熔絲，一旦檢測到 BL 被解鎖就會永久燒斷，無法恢復。
- **受影響嘅功能**：Samsung Pay（公交卡/銀行卡）、Knox 安全資料夾、企業 MDM 管控、部分健康保險類 App 都會失效。
- **強制清空數據**：熔斷觸發後手機會強制恢復出廠設定，所有本地數據被擦除（雲端數據唔受影響）。
- **保修影響**：官方明確表示 Knox 熔斷後唔再享受保修服務。

三思而後行

三星嘅 Knox 熔斷係**不可逆**嘅，一旦觸發就返唔到原廠狀態。如果你淨係想用一啲原本需要 Root 嘅功能，建議先查嚇有冇唔需要 Root 嘅替代方案。

重要警告

刷機有風險，操作需謹慎。可能導致：

- 手機變磚（無法開機）
- 失去保修
- 數據丟失
- 支付功能異常

刷機前務必做好完整備份，仔細閱讀教程，確認機型同版本匹配。

推薦資源

- **XDA Developers**：全球最大嘅安卓開發者社區，有豐富嘅刷機教程同 ROM 資源。
- **酷安**：國內知名嘅數碼社區，有好多中文教程同討論。
- **各機型論壇**：如小米社區、一加社區等，有官方同民間教程。

六、常見問題解答（Q& A）

Q1：解鎖 BL 鎖後仲能恢復嗎？

A：可以重新上鎖。通常需要刷入官方完整嘅線刷包，並使用特定嘅刷機腳本（如小米嘅 `flash_all_lock.bat`）即可重新上鎖。但注意，重新上鎖後，因解鎖而觸發嘅一啲安全熔斷（如三星嘅 Knox）可能無法逆轉。

Q2：解鎖後如何 Root？

A：解鎖 BL 鎖淨係第一步。Root 通常需要：

1. 刷入自訂 Recovery（如 TWRP）。
2. 透過 Recovery 刷入 Magisk 等 Root 管理器嚟修補 boot 鏡像，從而攞到 Root 權限。

第二十五章：二手手機處理與數字遺產

嘿，朋友哋，經過前面二十幾章嘅「修煉」，相信你已經由一個「數字新手」成長為一名相當從容嘅手機用戶喇。但科技生活總有輪迴——新歡終會變舊愛。當你準備等老夥計「退役」時，你知點樣體面、安全咁告別嗎？

一、舊手機嘅「安樂死」：安全出售或轉贈指南

核心原則：數據清理唔係「刪除」，而係「覆蓋」。好多人以為將相片拖入資源回收筒、清空相冊就萬事大吉，結果手機轉手後，新主人用數據恢復軟件一掃，舊相片、聊天記錄、身份證相片全部返曬嚟——呢種「社死現場」每年都喺度上演。

1. **自我「大掃除」**：完成最後一次完整備份，退出所有帳號（iCloud、華為/小米/三星帳號、Google 帳號），關閉「查找我嘅手機」功能——否則下一位用戶會被激活鎖攔喺門外。

2. **執行終極「格式化」**：現代手機嘅「恢復出廠設定」通常包含加密擦除——佢銷毀嘅係加密密鑰本身，密鑰一冇，整盤數據瞬間變為亂碼。

3. **最後嘅「障眼法」（僅舊設備需要）**：現代手機（iOS 8+、Android 6+）已預設全盤加密（FBE），恢復出廠設定會銷毀加密密鑰，數據理論上唔可恢復，**無需此步驟**。僅對極老舊嘅非加密設備（2015 年前），先至需要喺恢復出廠設定後用垃圾檔案填滿儲存，再執行一次恢復出廠設定。

各品牌數據擦除路徑速查表：雖然原理相通，但每個品牌將「恢復出廠設定」收埋嘅位置略有唔同，關鍵差異如下。

品牌	擦除入口	關鍵差異與注意事項
華為	設定 > 系統同更新 > 重置 > 恢復出廠設定	需先退出華為帳號並關閉「查找我嘅手機」，否則會觸發激活鎖；純淨模式下可能需要密碼驗證。
小米 / Redmi	設定 > 我嘅設備 > 恢復出廠設定	底部有「格式化虛擬 SD 卡」同「清除全部內容與設定」選項，兩者都要勾選；退出小米帳號前先關閉「查找設備」。
OPPO / 一加	設定 > 其他設定 > 還原手機 > 徹底清除全部數據	ColorOS 提供「徹底清除」模式，會多次覆蓋儲存；需輸入鎖屏密碼確認。
vivo	設定 > 系統管理 > 備份與重置 > 清除所有數據	勾選「格式化手機儲存」先至會清理 SD 卡區；需退出 vivo 帳號，關閉「查找手機」。
三星	設定 > 通用管理 > 重置 > 恢復出廠設定	需先移除 Samsung 帳號並關閉「查找我嘅手機」；部分機型有「自動覆蓋」選項，建議開啟。
iPhone	設定 > 通用 > 傳輸或還原 iPhone > 抹掉所有內容同設定	必須先退出 Apple ID，否則下一位用戶被 iCloud 激活鎖攔住；iOS 15+ 支援「退出時保留 iCloud 數據」以備後查。

最容易被忽略嘅三件事

- **未退出帳號**：手機賣畀下家後被遠程鎖定，自己都落唔到，最後淨係可以揸身份證去官方售後解鎖。
- **SD 卡未取出**：恢復出廠設定唔會格式化外置 SD 卡入面便嘅相片，記得手動取出或單獨格式化。
- **USB 手指/OTG 外設未斷開**：擦除過程中如果連住外設，可能誤清外設數據，亦可能導致擦除中斷。

二、數字遺產：你嘅社交靈魂點樣安放？

手機入面便嘅數據，早就唔係「虛擬」嘅喇。佢承載住你嘅資金、社交關係、創作成果，乃至你嘅「數字身份」。**一個人去世後，呢啲資產點樣處理？**呢個唔再係一個抽象問題，而係越來越多家庭真實面臨嘅難題——親屬唔知密碼、平臺唔畀繼承、加密貨幣永遠丟失。

法律依據：個人資訊保護法第 49 條

2021 年施行嘅《中華人民共同國個人資訊保護法》第 49 條明確規定：**自然人身故嘅，其近親屬為咗自身嘅合法、正當利益，可以對死者相關個人資訊行使本章規定嘅查閱、複製、更正、刪除等權利；死者生前另有安排嘅除外。**
呢個意味住：

- **近親屬有法律依據**向平臺申請處理死者嘅帳號同個人資訊。
- 「合法、正當利益」通常包括辦理遺產繼承、關閉帳號、刪除敏感私隱等。
- 死者生前若有遺囑或協議另行安排（如指定遺產聯絡人），從其約定。
- 平臺唔可以以「用戶私隱」為由一概拒絕配合，但具體流程各家唔同。

微信同支付寶：帳號唔可以繼承，但資金可以

呢兩個中國人最常用嘅 App，規則各有唔同：

- **微信帳號**：依據《騰訊微信軟件許可及服務協議》，帳號所有權歸騰訊，用戶僅享有使用權，**帳號本身唔可以繼承、唔可以轉讓**。但帳號內嘅**零錢、零錢通資金**屬於個人財產，近親屬可憑死亡證明、親屬關係證明、繼承公證書向財付通申請提現或轉移。
- **支付寶**：帳號同樣唔可以直接繼承，但支付寶餘額、餘額寶、理財通等**資金屬於可繼承財產**。繼承人可透過支付寶「客服熱線」或線下提交死亡證明、繼承公證書辦理；螞蟻財富類理財產品需走基金公司流程。
- **聊天記錄**：法律上屬個人私隱，平臺唔會主動提供畀家屬。如涉及遺產線索，可透過訴訟或公證申請調取。

千萬唔好做嘅事

- **唔好將死者帳號密碼直接交畀家人「接住用」**——呢個違反平臺協議，資金被凍結甚至觸發風控時舉證更難。
- **唔好嘗試用死者手機收驗證碼「提現」**——可能被認定為盜用，反而觸犯法律。
- **唔好拖延**：部分帳號長期未登入會被自動註銷，資金搵返會更麻煩。

Apple「遺產聯絡人」：提前指定，事半功倍

iOS 15.2 起蘋果推出「遺產聯絡人」功能，允許用戶提前指定信任嘅聯絡人，身故後該聯絡人可憑**死亡證明 + 遺產聯絡人訪問密鑰**申請訪問死者嘅 iCloud 數據（相片、備忘錄、通訊錄、備忘錄等）。操作步驟：

1. 喺 iPhone 上進入 [設定] > [你嘅姓名] > [登入與安全] > [遺產聯絡人]。
2. 撳「添加遺產聯絡人」，使用面容 ID 確認身份。

3. 選擇聯絡人（由通訊錄挑選），系統會生成一份**訪問密鑰**。
4. 將訪問密鑰透過短訊、郵件或列印紙本交畀對方，並提醒對方妥善保存。
5. 可添加多個聯絡人作為備份，避免單點失效。

被指定人唔使 Apple 設備，憑密鑰同死亡證明即可喺蘋果官網申請訪問。整個流程通常喺 7 日內完成。

Google「不活躍賬戶管理器」：自動處理身後事

Google 提供「不活躍賬戶管理器（Inactive Account Manager）」，能喺用戶長期未登入時自動執行預設操作——呢個係目前最完善嘅數字遺產機制之一。

1. 訪問 **myaccount.google.com/inactive**，登入 Google 帳號。
2. 設定超時期限：3 / 6 / 12 / 18 個月未登入後觸發。
3. 添加受信任聯絡人（最多 10 人），並設定每位可訪問邊啲數據（Gmail、相冊、雲端硬碟等）。
4. 可選：設定自動回覆郵件，告知來信者你已無法處理。
5. 最後選擇：到期後係**分享數據畀聯絡人**，仲係**直接刪除賬戶**。
6. 設定手機號作為兜底驗證，避免誤觸發。

國內社交帳號：紀念化與註銷流程

國內主流社交平臺對逝者帳號嘅處理分兩種思路：**「紀念化」（保留帳號等親友緬懷）**同**「註銷」（徹底刪除）**。家屬可憑死亡證明、親屬關係證明、申請人身份證向平臺客服申請。

- **微博**：支援帳號「紀念化」，由親屬提供死亡證明申請，帳號會被鎖定無法登入但內容保留，主頁會顯示「該帳號已設定為紀念帳號」。亦可申請註銷，需提供繼承公證。
- **抖音**：透過「我 - 設定 - 反饋與幫助 - 帳號與安全 - 帳號註銷」或人工客服，提交死亡證明、親屬關係證明、申請人身份證，審核通過後可註銷或轉為紀念狀態。
- **B 站**：客服渠道提交逝者證明及親屬證明，可申請帳號註銷或保留為紀念頁；UP 主嘅歷史稿件處理需結合版權聲明。
- **小紅書**：透過官方客服申訴，提交死亡證明、親屬關係證明，可申請註銷帳號並刪除內容。

關鍵提醒

- 各平臺規則會更新，辦理前先查閱官方客服最新指引。
- 所有證明檔案建議先掃描備份再提交，原件妥善保管。
- 跨平臺一次性辦理時，可統一申請一份**繼承公證書**，慳返跑幾次手續。

數字資產嘅繼承難題

除咗社交帳號，越來越多人持有**真正有價值嘅數字資產**，其繼承問題更加複雜：

- **加密貨幣（比特幣、以太坊等）**：基於「私鑰即所有權」原則，**冇私鑰就等於資產消失**。繼承人無法透過法律檔案向鏈上「申訴」。建議：將助記詞/私鑰分片存放喺多個物理安全位置（如銀行保險箱 + 屋企保險櫃），並喺遺囑中註明訪問方式。
- **交易所賬戶**：同銀行賬戶類似，可憑繼承公證書向交易所（如幣安、OKX）申請處置，但跨境交易所流程漫長。
- **域名**：屬於可繼承財產，繼承人喺域名註冊商（如阿里雲、GoDaddy）提交繼承公證後可辦理過戶。
- **知識產權（原創文章、影片、專利、軟件著作權）**：著作權中嘅財產權利可繼承（人身權唔繼承），收益權可由繼承人享有至作者身故後 50 年。
- **雲端儲存與網盤**：內容數據歸屬個人，可繼承；但部分平臺協議規定帳號本身唔可轉讓，需先下載再註銷。

加密貨幣「消失」嘅真實代價

據 Chainalysis 報告，全球已有**超過 300 萬枚比特幣**因私鑰丟失而永遠無法搵返，相當於現有總量嘅六分之一。如果你持有加密資產，請務必等至少一位你絕對信任嘅人知道**恢復路徑**——否則呢啲財富將永遠沉睡喺區塊鏈上。

一份「數字遺囑」模板

傳統嘅財產遺囑管唔到數字資產。建議你每年更新一份「數字遺囑」附件，同正式遺囑一併存放。以下模板供參考：

《個人數字資產處置附件》（模板）

第一部分：帳號清單（僅記錄帳號名與所屬平臺，**唔寫密碼**）

- 社交類：微信（帳號 xxx）、微博、抖音、B 站、小紅書等
- 支付類：支付寶、微信支付、各銀行卡 App
- 雲端類：iCloud、華為雲、小米雲、Google 帳號、微軟帳號
- 郵箱類：QQ 郵箱、網易郵箱、Gmail 等
- 其他：網盤、訂閱服務、遊戲帳號等

第二部分：密碼託管方案

- 主密碼存放方式：使用密碼管理器（如 Bitwarden、1Password），主密碼寫喺紙本中，封存於家中保險櫃。

- 恢復密鑰/助記詞：分片存放，至少兩處物理位置（如銀行保管箱 + 家中保險櫃）。
- 授權查詢人：姓名、身份證號、與本人關係、聯繫方式。

第三部分：處置意願

- 各類帳號希望「紀念化」定係「註銷」。
- iCloud / Google 數據係咪指定「遺產聯絡人」或「不活躍賬戶管理器」。
- 加密資產由邊個繼承，私鑰獲取路徑說明。
- 版權類作品收益由邊個享有。

第四部分：聯絡人與平臺

- 信任執行人姓名、聯繫方式、備用聯絡人。
- 常駐平臺客服入口與所需證明材料清單。
- 本附件更新日期：_____ 年 ___ 月 ___ 日，下次複核日期：_____ 年 ___ 月 ___ 日。

關於「密碼」嘅法律提示

將密碼直接寫入遺囑存在洩露風險，且遺囑公示後所有繼承人都能睇到。**更穩妥嘅做法係：遺囑入面便淨係寫「帳號清單 + 處置意願 + 密碼託管位置」，真正嘅密碼透過密碼管理器主密碼單獨傳畀信任執行人**。咁樣既符合法律要求，亦降低咗密碼洩露風險。

三、環保嘅最後一程

一部手機入面便含金、銀、銅、稀土等數十種金屬，隨意丟棄既污染土壤，亦浪費資源。正確嘅回收渠道有三類，各有側重：

渠道	回收價	數據安全	適用場景
官方以舊換新（華為/蘋果/小米等）	中等，常疊加補貼券	最高，官方有規範流程	正打算換同品牌新機時最劃算
第三方回收平臺（愛回收、轉轉、回收寶等）	略高，可比價	較高，有數據清除報告	想直接變現、唔綁定品牌
公益捐贈（如鄉村學校、愛心機構）	無經濟回報	需自行徹底擦除	設備仍可用、想做善事
發揮餘熱（自留改造）	慳返新設備錢	原機主本人繼續用，無風險	設備效能尚可、屋企有改造需求

- **官方回收/以舊換新**：最推薦嘅方式，流程規範，數據安全有保障。蘋果、華為、小米等品牌官網都有「以舊換新」入口，評估後可抵扣新機款。

- **正規回收平臺**：選擇信譽良好嘅大型回收平臺，落單後順豐上門取件，檢測後打款。務必選擇**提供「數據清除報告」嘅**平臺，並保留憑證。
- **公益捐贈**：可聯絡當地鄉村學校、社區公益組織，舊手機可改裝成教學平板或老人機使用。**捐贈前務必執行完整嘅擦除流程，並卸除所有 SIM/SD 卡**。
- **發揮餘熱**：改成車載導航儀、智能家居中控、監控鏡頭、BB 監視器、電子相框。一部舊手機可以再戰好幾年。

電子垃圾嘅真相

一部手機嘅電路板入面便含有**金、銀、鈀、銅**等貴金屬，每噸廢舊電路板含金量係金礦石嘅數十倍。但若隨意丟棄，其中嘅鉛、汞、鎘會滲入土壤與地下水，污染持續數十年。**正確回收，既保護環境，亦能等稀缺金屬循環再生。**

實操：為你嘅舊手機執行一場「標準葬禮」

1. 最後一次完整備份：相片上傳雲端、重要檔案複製到電腦、聊天記錄匯出存檔。
2. 退出所有帳號：iCloud / 華為 / 小米 / Google 帳號，關閉「查找我嘅手機」。
3. 取出 SIM 卡與 SD 卡，斷開所有 OTG 外設。
4. 執行加密恢復出廠設定，必要時手動勾選「格式化儲存」。
5.（僅舊設備）用垃圾檔案填滿儲存，再恢復出廠設定一次。
6. 選擇佢嘅下一站：官方以舊換新、正規回收平臺、公益捐贈，或轉贈家人。
7. 同步更新你嘅「數字遺產清單」，將已淘汰嘅設備由清單中劃走。

本月任務清單

1. 為你目前嘅主力手機指定 Apple「遺產聯絡人」或設定 Google「不活躍賬戶管理器」。
2. 用本節模板起草一份《個人數字資產處置附件》，同家人溝通存放位置。
3. 檢查屋企係咪有 2 部以上閒置舊手機，預約一次官方或正規平臺回收。

第二十六章：當下手機技術前沿

傾完舊手機嘅「身後事」，等我哋將目光拉返當下。*2024-2026* 年係手機技術嘅一個集中爆發期：*AI* 由雲端塞入咗衫袋，衛星通信由旗艦下放到中端機，摺疊屏由「嚐鮮」變成「常用」，電池技術亦迎來咗多年未見嘅材料級突破。呢一章，我哋就嚟盤點呢啲正在發生嘅技術前沿，幫你判斷「而家應唔應該換機」以及「換乜機」。

一、AI 手機：係營銷噱頭，定係真變革？

2024 年起，幾乎所有安卓廠商都喺旗艦機上打出「AI 手機」嘅旗號，蘋果亦喺 iOS 18 中上線咗 Apple Intelligence。但到底乜嘢樣嘅手機先至算「AI 手機」？業內大致形成咗三條判斷標準：

- **端側大模型**：手機本地能跑參數量喺 7B 左右嘅語言模型（如 Qwen、Llama、DeepSeek 嘅輕量版本），唔依賴雲端就能完成文本摘要、翻譯、文案生成等任務。判斷方法：飛行模式下若仲能用 AI 功能，則說明係端側模型。

- **NPU 算力（TOPS）**：NPU（神經網絡處理單元）嘅算力以 TOPS（每秒萬億次操作）為單位。2024 年旗艦晶片（如驍龍 8 Gen3、天璣 9300）嘅 NPU 算力喺 40-70 TOPS 區間，2025-2026 年嘅新一代晶片普遍突破 100 TOPS，呢個係端側跑大模型嘅硬件門檻。

- **AI Agent 場景**：唔單止識「問答」，仲能跨 App 執行任務鏈——例如「幫我將琴日開會嘅相整理成一個相冊並分享畀張三」，系統能自動調用相冊、檔案、微信等多個 App 完成。呢個係 AI 手機區別於「帶 AI 助手嘅手機」嘅核心標誌。

二、各廠商 AI 戰略對比：邊個喺度認真做，邊個喺度貼牌？

各家嘅 AI 路線差異明顯，選購時可以重點關注：

- **蘋果 Apple Intelligence（iOS 18+）**：強調整合系統級應用——郵件摘要、通知優先級、寫作潤色、相冊 AI 修圖。端側模型為主，複雜任務上雲到 Private Cloud Compute。優勢係私隱保護同系統流暢度，劣勢係中文場景上線較慢，部分功能 2025 年先至逐步支援。

- **谷歌 Gemini Nano**：Pixel 8/9 系列內置，支援「畫圈搜尋」、Magic Compose、錄音摘要等。Gemini Nano 係真正意義上嘅端側小模型，開源生態友善，係安卓 AI 手機嘅參考實現。
- **華為盤古大模型**：接入鴻蒙小藝，支援端側文案生成、AI 修圖、AI 消除（去路人）、通話摘要。鴻蒙 NEXT 5.0 後進一步系統級整合，AI 能力同系統深度融合。
- **小米澎湃大模型（MiLM）**：HyperOS 內置，支援 AI 寫真、AI 摘要、AI 畫師、超解析度。小米行「端雲協同」路線，端側跑輕量模型，重任務上雲。
- **OPPO（安第斯大模型）與 vivo（藍心大模型）**：ColorOS 15 與 OriginOS 5 深度接入，主打 AI 消除、AI 文檔摘要、AI 識圖問答、自然語言搜相（「搵琴日喺海邊影嘅相」）。

選購建議：如果你經常需要 AI 處理工作文檔、整理資訊，優先選端側算力強、AI 功能系統級整合嘅機型；如果淨係偶爾用 AI 修圖，主流中端機已經夠用。

三、衛星通信：由旗艦「噱頭」到中端標配

衛星通信係呢兩年最實喺嘅技術下沉之一。2022 年華為 Mate 60 同 iPhone 14 首發時仲淨係旗艦專屬，2025 年已經下放到 2000-3000 元檔。

- **北斗短報文下放中端機**：北斗衛星嘅短報文功能（可發送 1000 漢字以內嘅短訊 + 位置）已下放到 Redmi、realme、iQOO 等中端機型，無需月租，免費使用。適合戶外徒步、自駕遊等場景。
- **天通衛星通話**：華為 Mate 60/70 系列、榮耀 Magic6 系列等支援天通一號衛星通話功能，可撥打/接聽衛星電話、發送衛星消息。需要開通運營商衛星套餐（約每月 10-200 元），適合經常入無人區嘅用戶。
- **iPhone 衛星消息（SOS）**：iPhone 14 及以後機型支援衛星 SOS，可向緊急服務機構發送文字消息並共享位置。免費提供 2 年服務，部分地區續費可用。
- **實用建議**：城市用戶基本用唔上，但戶外愛好者、出差到偏遠地區嘅商務人士強烈建議選帶衛星功能嘅機型——關鍵時刻能救命。

四、摺疊屏生態成熟度：能當主力機嗎？

摺疊屏已由「科技嚐鮮」走向「主力機可用」。判斷摺疊屏係咪成熟，睇三點：硬件耐用性、App 適配度、交互生態。

- **硬件耐用性**：鉸鏈壽命普遍達 50 萬次以上（每日開合 100 次可用 13 年），水滴鉸鏈減少螢幕摺痕，IPX8 級防水已成標配。內屏 UTG 玻璃 + 保護膜方案等螢幕唔再「嬌貴」。
- **App 適配**：主流 App（微信、淘寶、抖音、B 站、Office、WPS）已針對大屏做適配，但仍有約 30% 嘅長尾 App 未適配，顯示為拉伸或黑邊。建議購機前先試用你常用嘅幾個 App。

- **懸停態交互**：摺疊到 90°-120° 時，部分 App 支援「上半屏顯示內容、下半屏操作」嘅懸停模式，適合視像通話、自拍、睇片。相機懸停自拍係摺疊屏獨有嘅體驗。
- **分屏與多任務**：大屏下可同時開兩個 App 分屏，甚至三個 App 小窗並存。HyperOS、ColorOS、OriginOS 嘅分屏邏輯已比較成熟，但 iOS（iPadOS 風格）嘅分屏喺 iPhone 上仍未開放。
- **價格**：橫摺（書本式）由萬元級下探到 5000-7000 元檔，豎摺（翻蓋式）已下探到 4000-5000 元檔。2025 年部分廠商推出 3000 元檔豎摺，摺疊屏真正進入大眾市場。

選購建議：如果你係重度閱讀、文檔處理、影片用戶，橫摺屏值得考慮；如果你追求便攜同社交屬性，豎摺屏更適合。建議至少去線下舖把玩 30 分鐘再決定。

五、6G 與 Wi-Fi 7：下一代連接嘅速度與邊界

雖然 6G 仲未真正落地，但相關技術已經喺路上：

- **6G 進展**：3GPP 計劃 2027 年完成 6G 標準制定，2030 年左右商用。6G 嘅理論峯值速率可達 1 Tbps（5G 嘅 50 倍），延遲低至 0.1ms，支援「通感一體化」（通信 + 感知），手機甚至能當低精度雷達用。目前各廠商仲喺原型驗證階段，離消費級產品尚遠。
- **Wi-Fi 7（802.11be）**：2024 年起成為旗艦機標配，理論峯值速率達 46 Gbps（Wi-Fi 6 嘅 4 倍），支援 320MHz 頻寬、4K-QAM、MLO（多鏈路同傳）。實測喺 Wi-Fi 7 路由器下，旗艦機傳輸速率可達 2-5 Gbps，比 Wi-Fi 6E 快 2-3 倍。如果你換咗 Wi-Fi 7 手機，記得配一臺 Wi-Fi 7 路由器先至可以發揮全部效能。
- **藍牙 5.4 與 LE Audio**：2024-2026 年新機普遍支援藍牙 5.4 及 LE Audio（低功耗音頻），可同時連兩副耳機聽同一首歌，Auracast 廣播音頻等多人喺公共場合同步聽同一音源。

六、矽碳負極電池：續航終於迎來材料級突破

手機續航多年來被詬病「一日一充」，2024 年起矽碳負極電池技術嘅普及等情況有咗實質改善。

- **技術原理**：傳統石墨負極嘅理論容量約 372 mAh/g，矽嘅理論容量高達 4200 mAh/g（約 10 倍），但矽充放電時體積膨脹達 300%，容易粉化。矽碳負極透過納米矽-碳複合結構，將矽嘅膨脹問題控制喺可接受範圍，等電池能量密度提升 15%-30%。
- **實際表現**：2024-2025 年採用矽碳負極嘅旗艦機（如小米 14 Ultra、榮耀 Magic 6 至臻版、vivo X100 Pro 等），喺 7-8mm 機身內塞進 5000-6000 mAh 電池，比上代多出約 800-1000 mAh，重度使用可撐一日半。
- **百瓦快充配合**：矽碳負極對快充相容性良好，100W/120W 快充 17 分鐘充滿已唔係新聞。2025 年部分機型支援 150W-240W，10 分鐘內充至 80%。

- **選購建議**：如果你係重度用戶，優先選採用「矽碳負極 + 百瓦快充」組合嘅 2024 年後機型，續航焦慮會大幅緩解。

七、本章速覽：一句話記住呢啲前沿

- AI 手機三標準：端側大模型、NPU 算力 TOPS、AI Agent 跨 App 場景。
- 衛星通信：北斗短報文下放中端，天通通話仍屬旗艦。
- 摺疊屏：硬件成熟、App 適配七成、懸停/分屏係獨有體驗。
- 6G 仲喺路上，Wi-Fi 7 已落地，藍牙 LE Audio 開始普及。
- 矽碳負極 + 百瓦快充：續航與充電嘅雙突破。

嚟試嚇：畀你嘅下一臺手機做一份「技術需求清單」

攞出一張紙，按「必備/加分/唔需要」三檔，列出你對 AI、衛星、摺疊屏、Wi-Fi 7、快充、電池容量嘅需求。咁樣下次去線下舖時，導購推薦嘅「賣點」你能迅速判斷係真需求定係被推銷。例如：

- 必備：5000mAh 以上電池、80W 以上快充、NFC、IP68 防水。
- 加分：端側 AI 摘要、Wi-Fi 7、北斗短報文。
- 唔需要：摺疊屏（我習慣直板）、衛星通話（唔出遠門）、240W 極限快充。

本章小結

- AI 手機三大判斷標準：端側大模型、NPU TOPS、AI Agent 場景
- 各廠商 AI 戰略：蘋果重私隱整合、谷歌端側開源、華為/小米/OPPO/vivo 各有所長
- 衛星通信：北斗短報文下放中端機，天通通話仍屬旗艦專屬
- 摺疊屏：硬件已成熟，App 適配約 70%，懸停態與分屏係獨有體驗
- 6G 標準 2027 年制定，Wi-Fi 7 已普及，藍牙 LE Audio 開始落地
- 矽碳負極電池等 5000-6000mAh 成為旗艦標配，續航焦慮大幅緩解

五、實戰篇（第 27-32 章）

第二十七章：爸媽都能輕鬆用——適老化手機完全指南

講真，寫呢一章嘅時候，我諗起我爸媽。我媽每次手機彈出一個「系統更新」，佢都唔敢點，驚整壞手機。我爸嘅微信字體比螞蟻仲細，但佢唔知點樣調大，就咁瞇住眼睇咗兩年。呢個唔係佢哋「蠢」，係手機預設將所有人都當成二十幾歲嘅後生仔。呢一章，我用最直白嘅話，將長輩哋手機使用中最頭痛嘅問題一個個解開。

零、先將心態擺正：呢個唔係「老人用唔嚟」，係「手機冇做好」

喺任何設定之前，有一個觀念要先講清楚：**而家嘅智能手機介面，由設計之初就冇點考慮老年用戶**。彈窗細到睇唔清、操作手勢需要精準觸碰、圖標生得都差唔多——呢啲唔係用戶嘅問題，係產品嘅問題。所以帶長輩用手機，第一件事唔係教，係幫佢哋**將環境調好**。環境調好咗，大半嘅問題就消失咗。

帶長輩用手機有三條鐵律：(1) **唔好催**——你用咗十年嘅智能機，佢哋可能先至用咗三個月。(2) **用類比**——「Wi-Fi 就好似屋企嘅水管，開咗水龍頭先有水」，呢種比喻比專業術語好理解一萬倍。(3) **寫低**——口頭教一遍佢哋可能唔記得，你攞張紙將關鍵步驟寫低貼喺佢手機殼背面，比咩都管用。

一、第一步：打開「簡易模式」——手機變清爽喇

而家嘅手機基本都內置咗一個「簡易模式」（有啲叫「關懷模式」「老年模式」「長輩模式」），打開之後，成部手機都會大變樣：

- **字體同圖標變得好大**——再亦唔使戴老花鏡睇手機喇。
- **桌面圖標大幅精簡**——只保留電話、短訊、相機、相冊、微信等最常用嘅幾個 App，其他全部收埋。
- **操作變簡單喇**——好多花入面胡哨嘅功能（負一屏、智能助手、浮窗等）預設關閉，介面主打一個清爽。
- **聲音變大喇**——來電鈴聲、通知音量、通話聲音都自動調到最大。

各品牌簡易模式開啟方法（直接跟住操作就得）：

品牌	模式名稱	打開方法
iPhone	輔助使用（冇單獨簡易模式，但可逐項調大）	設定 > 顯示與亮度 > 文字大小，拉到最大。再入「輔助功能 > 顯示與文字大小」開「粗體文字」「按鈕形狀」
華為/榮耀	簡易模式	設定 > 系統同更新 > 簡易模式 > 開啟
小米/紅米	極簡模式	設定 > 更多設定 > 極簡模式 > 開啟
OPPO/一加	簡易模式	設定 > 特色功能 > 簡易模式
vivo/iQOO	簡易桌面	設定 > 桌面、鎖屏與桌布 > 簡易桌面
三星	簡易模式	設定 > 顯示 > 簡易模式

如果你用嘅係其他品牌，喺手機「設定」頂部嘅搜尋框入面便直接搜「簡易」或「長輩」兩個字，一般都能搵到。搵唔到嘅話，按下面嘅「手動調大」方案一條條設定，效果都一樣。

二、如果唔開簡易模式，手動調都得——「大字 + 大圖標」五步走

有啲長輩唔想用簡易模式，覺得介面太「老氣」。咁就手動調以下五項，效果一樣好：

1. **字體調到最大**：設定 > 顯示 > 字體大小，拖到最右邊。順便將「顯示大小」（有啲叫「顯示縮放」）都調到最大，咁樣所有介面元素都跟住變大。
2. **圖標變大**：喺主螢幕雙指捏合（好似縮小相片咁樣）進入桌面設定 > 桌面佈局（或圖標大小），選格子最少嘅佈局（比如 3×5），圖標自然就大喇。
3. **打開「高對比度文字」**：等文字邊緣更銳利，喺「輔助功能」入面便可以搵到呢個開關。
4. **關閉不必要嘅通知**：設定 > 通知，將除咗電話、短訊、微信之外嘅 App 通知全部關掉。咁樣手機唔會一陣響一下，唔會等人覺得煩。
5. **將常用聯絡人放到桌面**：長按桌面空白處 > 添加小工具 > 聯絡人，將老伴、兒女、醫生、緊急聯絡人嘅快捷撥號放到桌面度。點一下就撥出去，唔使揭通訊錄。

動手：幫長輩「裝修」一下手機

1. 打開簡易模式（按上面嘅品牌表格操作）。
2. 將電量數字顯示打開（設定 > 電池 > 電量百分比），等佢哋隨時知道仲剩幾多電。
3. 調出通知欄，將藍牙、NFC、飛行模式呢啲長輩用唔着嘅快捷開關移除或收埋。
4. 喺桌面放四個快捷撥號：老伴、兒女、家庭醫生（如果有）、社區/物業。

三、防詐騙——呢章請一定帶住長輩一齊睇

老年人係電信詐騙嘅重災區。唔係佢哋警惕性低，而係騙子嘅話術經過精心設計，專門針對老年人嘅心理弱點。以下係我整理嘅最常見騙局類型同應對方法，建議你直接讀畀長輩聽，或者列印出嚟貼喺電話隔離。

六大常見騙局（見到即刻掛電話）：

- **(1)「你孫仔出事喇，趕緊匯錢」**——騙子會講「你孫仔/孫女喺出面闖禍喇，被警察拉咗，需要保證金」。記住：**任何要求轉賬先至可以「放人」嘅電話都係詐騙**。公檢法唔會電話要錢。
- **(2)「你嘅醫保卡被停用喇」**——短訊入面便講醫保卡異常，等你點連結「激活」。記住：**醫保局、社保局唔會發短訊等你點連結**。遇到呢種情況直接打 12333（社保官方熱線）核實。
- **(3)「高收益理財，穩賺唔賠」**——微信羣、短訊入面便推薦各種「養老理財」「國家扶持項目」，承諾高額回報。記住：**承諾年化收益超過 5% 仲「保本」嘅，99.9% 係騙局**。
- **(4)「我係你領導/老朋友，換號碼喇」**——冒充熟人借錢。記住：**打電話核實比咩都管用**，話「唔方便接電話」嘅八成係騙子。
- **(5)「免費領雞蛋/保健品講座」**——線下免費活動誘導老人聽「健康講座」，然後賣天價保健品。記住：**天上唔跌餡餅，免費活動背後都係生意**。
- **(6)「你嘅手機中毒喇，需要遠程修復」**——騙子等你下載一個遠程控制 App（如向日葵、TeamViewer），然後遠程操控你嘅手機轉走錢。記住：**絕對唔好等任何人遠程操控你嘅手機！**手機廠商、通信運營商、銀行都唔會畀你打電話等你裝遠程控制軟件。

三道防線，立好之後能擋掉 90% 嘅詐騙：

1. **第一道：手機自帶攔截**——安卓手機打開「騷擾攔截」（喺撥號 App 或手機管家 App 入面便），iPhone 打開「靜音未知來電」（設定 > 電話 > 靜音未知來電）。
2. **第二道：運營商攔截**——發送短訊「KTFSR」到 10086，免費開通移動嘅高頻騷擾電話攔截；電信聯通亦有類似服務。打運營商客服電話等人工幫你開通就得。
3. **第三道：國家反詐中心 App**——喺應用商店下載「國家反詐中心」App，註冊並開啟來電預警。呢個 App 同公安嘅數據庫聯動，騙子號碼一撥過嚟就能識別。

畀長輩嘅「防騙口訣」（列印出嚟貼電話隔籬）

- **陌生電話要轉賬掛掉！**
- **短訊連結等你點刪掉！**
- **話高收益保本嘅拉黑！**

- **等你裝遠程控制軟件直接掛！**
- **真警察唔會電話辦案更唔會要錢！**
- **拿唔準嘅事，先畀兒女打個電話問問。**

四、遠程協助——你喺外地都能幫爸媽修手機

有時候長輩遇到嘅問題，你喺電話入面便描述半日佢哋都搵唔到對應嘅按鈕。呢個時候「遠程協助」功能就派上用場喇——你可以直接喺自己嘅手機上見到佢哋手機嘅螢幕，幫佢哋操作。各品牌嘅方案唔一樣：

- **華為/榮耀**：自帶「暢連」App，入面便有「螢幕共享」功能。你同爸媽都裝暢連，發起視像通話後撳「共享螢幕」，你就可以喺佢哋螢幕上畫圈標註，話畀你媽媽聽「點呢度」。
- **小米/紅米**：小米視像通話 App 入面便有「遠程協助」，支援螢幕共享 + 語音通話同步。
- **OPPO/vivo**：內置「家庭空間」或「遠程守護」，喺設定入面便搜尋「遠程」就能搵到。
- **通用方案（所有品牌都可用）**：下載「向日葵遠程控制」App（免費版夠用），你哋各裝一個。註冊同一個帳號或加好友，等爸媽將手機上顯示嘅驗證碼讀畀你，你就能遠程見到同操作佢哋嘅螢幕喇。
- **iPhone 用戶**：FaceTime 通話中支援「共享螢幕」。喺 FaceTime 通話時，撳螢幕共享按鈕，對方就能見到你嘅螢幕。但 iOS 安全限制無法遠程操作，淨係可以「指揮式教學」。

五、微信嘅「適老改造」——同長輩講呢幾句就夠

微信係長輩哋用得最多嘅 App，但介面太密太雜。以下係幾個立竿見影嘅調整：

- **調大微信自己嘅字體**：微信 > 我 > 設定 > 通用 > 字體大小，拖到最大。注意，呢個淨係影響微信內部嘅字體，唔影響手機系統字體。
- **打開「關懷模式」**：微信 > 我 > 設定 > 關懷模式 > 開啟。打開後字體同按鈕都會變大，色彩對比度加強，仲能一鍵「聽文字消息」（點一下文字消息就能朗讀出嚟）。
- **幫佢哋備註好重要聯絡人**：將兒女、親戚嘅名喺通訊錄入面便改成容易認嘅——比如「大仔-小明」「二女-小紅」，等佢哋一打開微信就知邊個係邊個。
- **教佢哋用「語音輸入」而唔係打字**：聊天框隔籬嘅鍵盤上，按住空格鍵（安卓）或點麥克風圖標（iPhone），直接講嘢就能變成文字發送。識別準確率已經好高，對唔識拼音打字嘅老人嚟講係「質變級」嘅便利。
- **關閉「發現頁」入面便唔用嘅功能**：微信 > 我 > 設定 > 通用 > 發現頁管理，將「朋友圈」「影片號」「直播」「購物」「遊戲」入面便唔用嘅全部關掉，減少幹擾。

六、手機唔見咗點算？幫長輩做個「應急預案」

預則立，唔預則廢。趁手機仲未唔見，先將呢幾件事做好：

1. **確認「查找我嘅手機」已打開**：iPhone 喺設定 > 你嘅名 > 查找 > 查找我嘅 iPhone；安卓喺設定搜「查找設備」。務必確認呢個開關係亮住嘅。
2. **將重要嘅密碼同帳號記喺一張紙上**：Apple ID / 華為帳號 / 小米帳號嘅密碼，鎖喺屋企抽屜一份，畀信得過嘅兒女一份。唔見咗手機之後要登入呢啲帳號嚟遠程鎖定同定位，如果唔記得密碼就麻煩喇。
3. **教長輩一句話：「手機唔見咗，第一時間畀家人打電話。」**家人可以立刻登入 iCloud 或安卓嘅查找設備網頁，遠程標記為「丟失模式」，咁樣執到嘅人都打唔開、用唔到。

七、無線耳機同助聽器相容——被忽視嘅好功能

好多長輩有聽力問題，但唔知手機可以同助聽器協同工作。iPhone 同部分安卓手機支援 MFi（Made for iPhone）助聽器，可以直接將電話聲音傳到助聽器入面便，比揸住手機聽筒清楚太多喇。

- **連接方法**：iPhone 設定 > 輔助功能 > 助聽設備，打開藍牙，按照助聽器嘅配對說明操作。
- **如果唔用助聽器**：喺「輔助功能 > 音頻/視覺」入面便打開「電話降噪」同「單聲道音頻」，能改善通話清晰度。
- **安卓**：喺「輔助功能 > 聽力增強」入面便有類似設定，各品牌位置略有差異，直接搜「助聽」兩個字。

八、健康相關功能——手機唔淨係通訊工具

- **SOS 緊急求助**：連續按電源鍵五次（大多數手機），手機會自動撥打緊急電話並畀預設嘅緊急聯絡人發送定位。幫長輩提前設定好——iPhone：「設定 > 緊急 SOS」，安卓：「設定 > 安全 > 緊急求助」。教長輩用之前先演習一遍，免得到時緊張按唔準。
- **跌倒檢測**：Apple Watch 有呢個功能，檢測到你跌倒且一段時間唔鬱，會自動撥打急救電話同通知家人。有預算嘅話畀長輩配一個，關鍵時刻能救命。
- **用藥提醒**：iPhone 嘅「健康」App 內置用藥提醒功能，設好藥名、時間同劑量，到點就彈出提醒。安卓可以用「定時鬧鐘」或下載「食藥提醒」類 App。

本章小結

- 先開簡易模式或手動調大字體/圖標，環境調好大半問題就冇喇
- 帶長輩用手機三鐵律：唔好催、用類比、寫低

- 六大常見騙局要反覆提醒，三道防線立好能擋掉 90% 詐騙
- 遠程協助等你人喺外地都能幫爸媽修手機
- 微信開「關懷模式」+「語音輸入」係長輩最實用嘅兩個功能
- SOS 緊急求助同跌倒檢測，關鍵時刻救命嘅配置值得花五分鐘設好

第二十八章：花對錢買到心儀機——手機選購終極指南

買手機呢件事，好多人會提前睇一週評測、比一個月價格，結果最後都係稀入麸糊塗花五六千買咗臺自己其實用唔上嘅旗艦。呢章唔列參數表——嗰啲你喺官網都能見到。我想做嘅，係畀你一套「選機思維」，以後唔管市場上出咗乜新手機，你都能自己判斷佢適唔適合你。

零、三步選機法：需求預算機型

好多人喺網上問「三千蚊買乜手機好」，呢個問題其實冇得答——因為唔知你買手機主要用嚟做乜。買手機嘅正確思考順序應該係：

選機三板斧

1. **先諗清楚你最喺乎乜：**影相？打機？續航？螢幕？系統流暢？仲係淨係需要能打電話發微信就得？將需求排個序。
2. **再定預算上限：**唔係「大概幾多錢」，係「最多能接受花幾多錢」。定一個硬上限，選機嘅時候唔會越睇越貴。
3. **最後先至揀機型：**揸住你嘅需求排序同預算，去匹配機型。

一、核心參數點樣睇——唔使全識，睇識呢五項就夠

參數表動輒幾十項數據，其實普通用戶淨係需要關注呢五個維度：

維度	點樣睇	一句話總結
處理器（SoC）	決定咗手機能流暢用幾年。2026 年嘅標準：驍龍 8 Elite Gen5/天璣 9500 以上 = 頂級；驍龍 8s Gen4/天璣 8400= 中高端；驍龍 7+ Gen4/天璣 7350= 夠用三年。低於驍龍 6 系嘅建議慎重。	買新唔買舊，SoC 決定咗你手機能用幾年唔卡
螢幕	OLED 比 LCD 色彩好、更省電；更新率 120Hz 比 60Hz 掃屏流暢好多；解像度至少 1080P 以上。高頻 PWM 調光（1920Hz 以上）對眼睛更友善。	OLED+120Hz+ 高頻調光 = 好螢幕嘅及格線
鏡頭	**唔睇像素數睇傳感器尺寸**。底大一級壓死人，傳感器尺寸比像素重要得多。超廣角實用；長焦（3x 以上光學變焦）影遠處嘅嘢；微距鏡頭多數係湊數嘅。	大底主攝 + 實用長焦 > 多顆湊數鏡頭
電池同充電	4500mAh 係基礎及格線，5000mAh 以上一日一充好穩。快充睇瓦數：60W 以上算快，100W 以上飛快。無線充電方便但慢。	5000mAh+60W 快充 = 唔焦慮組合
系統更新承諾	決定咗手機能安全用幾耐。蘋果一般畀 5-6 年系統更新；三星旗艦畀 4 年系統 +5 年安全更新；谷歌 Pixel 畀 7 年；國產中端機更新承諾普遍偏短（2-3 年）。	想用耐啲，選系統更新承諾長嘅品牌

二、按預算分檔推薦（以 2026 年中市場為基準）

以下推薦基於 2026 年中國內地市場行情，價格為官方起售價。具體價格會有浮動，以你入手時嘅實際價格為準。

1000 元以下：能打電話，能掃微信，夠用就得

適合邊個：學生、後備機、對手機要求唔高、主要用嚟打電話同微信。

能買到乜：Redmi Note 系列、榮耀 X 系列、realme 數字系列嘅中低配。處理器大概驍龍

6 系或天璣 6000 系列，螢幕係 LCD 為主，影相能影清楚但對焦慢，系統流暢度一般。

選購建議：呢個價位睇兩點：電池夠唔夠大（選 5000mAh 以上嘅），記憶體夠唔夠（選 6GB+128GB 起步，唔好買 4GB 嘅會卡）。紅米喺呢個價位優勢最明顯。

1000-2000 元：性價比最高，能用三四五年唔卡嘅甜點區

適合邊個：學生、預算有限但想用得耐、對影相同遊戲冇極致追求。

代表性機型：Redmi Turbo 系列、iQOO Z 系列、榮耀 X 系列高配、一加 Ace 系列入門。

選購建議：選搭載驍龍 7+ Gen3 或天璣 7300 以上處理器嘅機型，配 8GB+256GB 記憶體。呢個配置日常使用 + 偶爾打王者榮耀完全夠用，正常能用 3-4 年唔卡。

2000-3500 元：中堅力量，效能同影相都有明顯提升

適合邊個：想要唔錯嘅影相、打機唔卡、希望用得耐嘅普通用戶。呢個係最多人買嘅價格區間。

代表性機型：小米數字系列（如小米 17 標準版降價後）、vivo S 系列高配、OPPO Reno 系列、榮耀數字系列、Redmi K 系列。

選購建議：呢個價位最大嘅特點係「水桶機」——冇明顯短板。比起 2000 元以下嘅機仔，主要提升喺：相機明顯變好（有光學防震喇）、螢幕上咗高素質 OLED、快充更快（80W 以上常見）、做工質感亦更精緻。選 12GB+256GB 或 12GB+512GB，用四五六年冇問題。

3500-5000 元：準旗艦，除咗極客玩家其他人完全夠用

適合邊個：對影相有追求嘅用戶、想體驗頂級質感嘅用戶。

代表性機型：小米數字系列標準版、vivo X 系列、OPPO Find X 系列標準版、iPhone 舊款旗艦（如 iPhone 16）。

選購建議：呢個價位已經能買到舊年嘅旗艦機型。我嘅建議係：**買舊年嘅旗艦，唔買今年嘅中端**。舊年旗艦嘅用料、做工、鏡頭硬件通常比今年嘅中端高一個檔次。前提係你接受佢唔係「最新款」。

5000 元以上：旗艦區，唔差錢或對某項體驗有極致追求

適合邊個：攝影發燒友、重度遊戲玩家、追求頂級螢幕同工藝嘅用戶、用手機影影片做內容嘅創作者。

代表性機型：iPhone 17 Pro/Pro Max、三星 S26 Ultra、華為 Pura 80 Ultra、小米 17 Ultra、vivo X300 Pro。

選購建議：呢個價位各家嘅差距越嚟越細，其實主要睇你鍾意乜系統。鍾意影相選 vivo 華為，鍾意影影片選 iPhone，鍾意螢幕同 SPen 選三星，鍾意均衡選小米。講真到咗呢個價位，**你可以任性，選你睇着順眼嘅嗰部就得**。

三、按場景選機——你嘅需求係乜？

以下係幾個最常見嘅「使用畫像」，對號入座就得：

使用場景	重點睇乜	推薦方向
影相達人	大底主攝（1 吋最佳）、光學防震、長焦（3x 以上光學變焦）、影像晶片	vivo X 系列、華為 Pura/Mate 系列、小米 Ultra 系列
遊戲玩家	頂級處理器、高觸控採樣率、散熱系統、大電池 + 快充	iQOO 數字系列、ROG Phone、紅魔、Redmi K 系列
續航優先	5500mAh 以上電池、能效好嘅處理器、LTPO 螢幕	榮耀 Magic 系列、一加、Redmi Note 大電池版
長輩使用	簡易模式、大字體、大音量、大電池、售後網點多	華為暢享、榮耀 Play、OPPO A 系列、Redmi Note
學生用機	性價比高、耐用、電池大、有家長控制功能	Redmi Note/Turbo、realme GT、iQOO Neo
蘋果生態用戶	已有 Mac/iPad/AirPods，買 iPhone 係唯一解	iPhone 17 標準版（性價比）或 Pro 版（滿血體驗）
出國旅行	eSIM 支援、全球頻段覆蓋完整、Google 服務	iPhone、三星 S 系列、Pixel、一加

四、五個容易踩嘅坑——買之前對照睇一眼

1. **坑一：四攝五攝唔等於影相好**——廠商鍾意喺參數上堆鏡頭數量，入面成日有兩顆係 200 萬像素嘅「湊數鏡頭」（微距 + 景深），實際完全冇用。睇手機影相能力，**淨係睇主攝 + 超廣角 + 長焦**呢三顆就夠。其他多出嚟嘅可以當佢唔存在。

2. **坑二：唔好被 1 億像素唬住**——像素高唔代表畫質好。傳感器尺寸、光圈、算法調教比像素重要得多。好多旗艦機用 5000 萬像素嘅大底傳感器，影出嚟嘅相比某啲 1 億像素嘅千元機好太多。

3. **坑三：唔好買運營商定製機**——「存話費送手機」睇落好劃算，但運營商定製機系統更新慢、可能有刪唔到嘅預裝 App、而且通常係運營商鎖定嘅（淨係用得呢間運營商嘅卡）。睇似慳咗錢，實則「鎖定」咗你。

4. **坑四：「曲面屏好高級」——係營銷，唔係真理**——曲面屏曾經係旗艦機嘅標緻，但而家越來越多人發現佢嘅問題：貼膜難、邊緣誤觸、爛咗維修貴。2026 年嘅趨勢係回歸「微曲」甚至「純直屏」。唔好因為「曲面嘅就係高級嘅」而選曲面屏。

5. **坑五：唔係所有「AI 手機」都真係有用**——2026 年所有廠商都喺度喊「AI 手機」，但好多淨係將已有嘅語音助手換咗個名。真正有用嘅 AI 功能係：實時翻譯、AI 消除（影相將人去掉）、AI 修圖、端側大模型推理、本地化嘅智能摘要。其他嘅花哨功能基本都係噱頭。

五、幾時買最劃算？

- **618（5 月底到 6 月中）**：一年中手機降價幅度最大嘅時段。好多手機廠商為咗衝銷量，旗艦機能降價 800-1500 元。係上半年買手機嘅最佳視窗。
- **雙 11（10 月底到 11 月中）**：下半年嘅大促，力度同 618 差唔多。呢個時間段好多品牌新一代旗艦啱啱發布，上一代旗艦降價幅度最大。
- **新機發布後一個月**：啱啱發布時原價，一個月後渠道價（第三方舖頭）一般會平 200-500。
- **春節前兩週到元宵節**：線下實體舖清庫存嘅「年貨節」，可以同店員傾價。
- **唔推薦嘅時間**：新品發布會後當日衝首發——除非你真係特別想做第一批用戶，否則首發價係最貴嘅。

六、渠道點樣選——喺邊買靠譜？

- **品牌官方商城/天貓旗艦店/京東自營**：最保險，價格冇優勢但唔使擔心翻新同假貨，售後最好。
- **拼多多「百億補貼」**：價格最低，但需要選標註咗「百億補貼」嘅舖頭。冇標註嘅唔好碰。收到貨記住錄開箱影片，有問題能退貨。
- **線下實體舖**：可以親手摸、試手感。注意：導購會推利潤高嘅機型（而唔係最適合你嘅），建議入舖之前先喺網上鎖定 2-3 款目標機型，去到就話「我淨係想睇呢幾款」。
- **二手平臺（閒魚/轉轉）**：平但有風險。如果要買二手，記住三條：要求提供購買發票或訂單截圖、要求提供 IMEI 號查驗、**淨係走平臺交易，絕唔私下轉賬**。

動手：用呢套方法選一部手機

1. 攞一張紙，寫低你嘅兩個核心需求（比如：續航第一、影相第二）。
2. 定一個預算上限（比如 3500 元）。
3. 打開京東或天貓，按價格篩選，圈出 3-5 款備選機型。
4. 去 B 站搜「XX 機型一個月使用體驗」（唔係第一日評測，係用咗至少一個月之後嘅真實感受），睇兩三條片。
5. 確認渠道靠譜後落單，收貨時錄開箱影片。

本章小結

- 選機三板斧：先諗需求再定預算最後揀機型
- 核心參數淨係睇五項：處理器、螢幕、鏡頭、電池充電、系統更新承諾
- 1500-3500 元係性價比最高、最推薦嘅甜點區間

- 買舊年嘅旗艦比買今年嘅中端更劃算
- 五大坑：多攝騙局、高像素陷阱、運營商定製、曲面屏迷信、AI 噱頭
- 618 同雙 11 係一年中最適合買手機嘅兩個時間視窗

第二十九章：數字健康——同手機建立健康關係

你有冇咁樣嘅體驗：本來淨係想掃 5 分鐘短影片，結果一抬頭已經過咗兩個鐘？或者夜晚瞓喺牀度掃手機，越掃越精神，最後凌晨兩點先瞓着？手機改變咗我哋嘅生活，但亦喺度悄悄吞噬我哋嘅時間同健康。呢一章唔係等你「戒掉手機」，而係幫你重新攞返對手機嘅主動權。

一、先睇嚇你嘅「螢幕時間」——數字等人清醒

喺改變之前，先搞清楚現狀。手機入面便有一個好多人從未打開過嘅功能，叫「螢幕使用時間」（iPhone）或「數字健康」（安卓），佢能話畀你聽每日到底喺手機上花咗幾多時間，以及呢啲時間花咗喺邊啲 App 上。

- **iPhone**：設定 > 螢幕使用時間 > 查看所有活動。你會見到每日嘅解鎖次數、攞起手機嘅次數、每個 App 嘅使用時長。
- **安卓**：設定 > 數字健康與家長控制。華為叫「健康使用手機」，小米叫「螢幕時間管理」。功能類似，都能見到每日使用數據。

第一次睇呢個數據嘅人，80% 會被嚇到。我有個朋友睇完之後發現自己每日喺抖音上花咗 4 個幾鐘，一週就係 28 小時——相當於一份兼職嘅時間。呢種「數字衝擊」係改變嘅第一步。

二、畀 App 設定「使用限額」——畀自己裝個限速器

知問題喺邊之後，就可以有針對性咁設定限制。唔需要一刀切將 App 刪咗，淨係需要畀佢設個「每日額度」：

1. **iPhone**：設定 > 螢幕使用時間 > App 限額 > 添加限額。選中你要限制嘅 App（比如抖音），設定每日 30 分鐘。到時間後 App 圖標會變灰，打開需要額外輸入密碼。
2. **安卓**：設定 > 數字健康 > 應用計時器。選中 App 後設定每日使用上限，到時間後 App 圖標會變灰。
3. **小技巧**：將限額密碼設成一個你唔常用、唔太好記嘅密碼。太容易輸嘅話，你會忍唔住每次都「續命」，限額就形同虛設喇。

三、「勿擾模式」嘅正確用法——唔係關掉通知，係關掉焦慮

好人嘅手機一日響幾十次通知，每響一次注意力就被打斷一次。你以為你淨係「睇一眼通知」，實際上每次打斷後重新集中注意力需要 15-20 分鐘。勿擾模式唔係等你「失聯」，而係幫你重新掌控注意力。

- **定時開啟：**設定勿擾模式喺夜晚 10 點到朝早 7 點自動開啟。咁樣瞓覺時手機唔會因為一條垃圾短訊將你嘈醒。
- **允許重要聯絡穿透：**喺勿擾模式設定入面便，將「允許來電」設為「收藏聯絡人」或「重複來電」（3 分鐘內第二次來電會正常響鈴）。咁樣你唔會錯過家人嘅緊急電話，但亦唔會被推銷電話打擾。
- **工作專注模式：**iPhone 嘅「專注模式」可以設自訂場景——比如「工作」模式下淨係允許工作相關 App 通知，「閱讀」模式下所有通知全靜音。

四、護眼唔靠 App 靠習慣——20-20-20 法則

長時間睇手機螢幕，眼乾澀、酸脹係必然嘅。護眼模式（藍光過濾）有一定幫助，但真正有效嘅係改變用眼習慣：

20-20-20 法則

每睇螢幕 **20 分鐘**，將視線移到 **20 英尺（約 6 米）**以外嘅地方，保持 **20 秒**。呢個簡單嘅動作能有效緩解睫狀肌疲勞。

- **開啟護眼模式：**設定 > 顯示 > 護眼模式（或夜覽模式），開啟後螢幕色溫變暖，減少藍光。建議設為日落到日出自動開啟。
- **調整螢幕亮度：**環境暗時螢幕亦要暗。好多人喺黑暗中睇手機亮度調太高，呢個係傷眼最嚴重嘅行為。開啟「自動亮度」並確保佢正常工作。
- **保持距離：**手機距離眼睛至少 30 釐米。好多人鍾意將手機擺得特別近睇，咁樣睫狀肌一直喺緊張收縮狀態，近視加深好快。

五、「低頭族」嘅頸椎自救指南

低頭睇手機時，你嘅脖子承受嘅壓力係正常站立時嘅 5-6 倍。長期低頭會導致頸椎曲度變直、骨質增生、頭暈手麻。以下係幾個簡單嘅改善方法：

- **將手機舉高：**睇手機時盡量將手機舉到與眼睛平齊嘅高度，而唔係低頭睇放喺大髀上嘅手機。雖然手臂會攰，但脖子會多謝你。
- **每 30 分鐘活動一下脖子：**慢慢向左轉頭到極限，停 3 秒，回正；再向右轉，停 3 秒。重複 5 次。然後做「米字操」——用下巴喺空中寫一個「米」字。

- **靠牆企 3 分鐘**：後腦勺、肩胛骨、臀部、腳後跟四點貼牆，每日企 3 分鐘，能有效糾正含胸低頭嘅體態。

六、睡眠與手機——臥室入面便最該改嘅習慣

手機螢幕嘅藍光會抑制褪黑素分泌，等你「越睇越精神」。好多人覺得「我掃住掃住就攰喇」，實際上係被藍光欺騙咗大腦，等你放低手機反而唔容易入睡。以下係改善建議：

- **睡前 1 小時唔睇手機**：呢個係最有效亦最難做到嘅一條。如果做不到，至少開「夜覽/護眼模式」將藍光降到最低。
- **手機放喺伸手夠唔到嘅地方**：將充電器放喺臥室另一頭，而唔係牀頭櫃。咁樣你既唔會忍唔住攞起嚟掃，鬧鐘響咗你亦要起身先至可以閂掉——一舉兩得。
- **開啟「睡眠模式」**：iPhone 嘅「睡眠」專注模式可以自動喺睡眠時段鎖定螢幕、簡化鎖屏；安卓各品牌亦有類似功能。
- **用鬧鐘代替手機**：買一個十幾蚊嘅實體鬧鐘，唔用手機當鬧鐘，你會發現少咗「睇一眼時間然後忍唔住掃一下」嘅陷阱。

七、資訊焦慮症——學識「選擇性失明」

我哋每日接收嘅資訊量，係古人一世接收嘅量。但人嘅大腦唔係為咗處理咁多資訊而設計嘅，資訊過載會導致焦慮、注意力渙散、決策疲勞。解決方法唔係「獲取更多資訊」，而係「主動屏蔽」：

- **取消關注**：打開你關注嘅所有公眾號、UP 主、博主列表，問自己一個問題：「過去一個月我從中獲得過實際價值嗎？」如果冇，取消關注。資訊渠道貴精唔貴多。
- **關閉非必要 App 推送**：除咗微信、電話、短訊、日曆之外，其他 App 嘅通知全部關掉。新聞唔需要推送——你主動去睇嘅時候自然會見到。
- **設立「資訊齋戒日」**：每週選一日（比如週日），全天唔睇手機新聞、唔掃短影片。你會發現天唔會塌落嚟，而且週一精神狀態明顯更好。

動手：一週數字健康挑戰

1. 今日：打開螢幕使用時間/數字健康，查看你上週嘅使用數據。
2. 聽日：畀耗時最長嘅娛樂 App 設定每日限額（建議 30-60 分鐘）。
3. 後日：設定勿擾模式定時開啟（22:00-07:00），允許收藏聯絡人來電。
4. 第四日：睡前 1 小時唔睇手機，手機充電器放到臥室另一頭。
5. 第五日：取消關注 5 個你唔再睇嘅公眾號或博主。
6. 第六日：嘗試一成日嘅「資訊齋戒」——唔睇新聞唔掃短影片。

7. 第七日：查看本週嘅螢幕使用時間報告，同上週對比。

本章小結

- 先睇螢幕使用時間數據，認清現狀係改變嘅第一步
- 畀娛樂 App 設限額，畀勿擾模式設定時，重新掌控注意力
- 20-20-20 法則係最簡單有效嘅護眼習慣
- 低頭睇手機時脖子承重 5-6 倍，舉高手機 + 定期活動脖子
- 睡前 1 小時唔睇手機，充電器放到伸手夠唔到嘅地方
- 資訊焦慮嘅解藥係「主動屏蔽」，唔係「獲取更多」

第三十章：投屏、演示與車載互聯

手機螢幕再大都係 *7* 吋，但你嘅電視有 *55* 吋、你嘅投影機有 *100* 吋。將手機畫面「掟」到大螢幕上，體驗係完全唔同維度嘅。呢一章講清楚投屏、演示同車載互聯呢三個高頻場景，幫你將手機嘅能力延伸到更大嘅螢幕上。

一、無線投屏三大陣營——唔好搞混

無線投屏睇落都係「手機畫面到電視上」，但底層技術有三種，互唔通用：

技術	邊個用	特點
AirPlay	iPhone/iPad/Mac	蘋果生態，體驗最流暢。需要 Apple TV 或支援 AirPlay 嘅電視
Miracast	安卓手機	Wi-Fi Direct 直連，唔需要路由器。安卓手機「無線投屏」預設用呢個
DLNA	所有平臺	淨係推送媒體檔案（影片/音樂/相片），唔鏡像螢幕。適合推影片到電視上播放

各平臺操作方法：

- **iPhone**：控制中心 > 螢幕鏡像 > 選擇你嘅電視/Apple TV。如果電視唔支援 AirPlay，可以買一個 Apple TV 盒子或支援 AirPlay 嘅電視棒。
- **安卓**：下拉通知欄 > 搵到「無線投屏」/「智能投屏」/「多屏互動」按鈕 > 選擇電視。需要手機同電視連同一個 Wi-Fi。
- **萬能方案**：如果電視太老唔支援任何無線投屏，買一個 Chromecast 或小米盒子/華為盒子插到電視 HDMI 口上，就能接收投屏喇。

二、有線投屏——更穩定、零延遲

無線投屏方便但有延遲，打機同做演示時體驗唔好。有線投屏能解決呢個問題：

- **iPhone**：需要一根 Lightning/USB-C 轉 HDMI 嘅轉換線（約 100-200 蚊）。插上後手機畫面直接輸出到電視/投影機嘅 HDMI 口。
- **安卓**：部分手機支援 USB-C 轉 HDMI 直連（需要手機支援 DP Alt Mode，大部分旗艦都支援）。買一根 USB-C to HDMI 線即可。
- **會議室場景**：如果你經常需要用手機做演示，建議喺袋入面便常備一根轉 HDMI 線。會議室嘅電視/投影機幾乎都有 HDMI 口，插上就能用，比折騰無線投屏靠譜得多。

三、手機變電腦——桌面模式

部分安卓手機支援「桌面模式」——連上顯示器後，手機會輸出一個類似電腦桌面嘅介面，可以用滑鼠鍵盤操作：

- **Samsung DeX**：三星旗艦嘅招牌功能。連接顯示器後彈出桌面介面，可以多視窗操作，體驗最接近真正嘅電腦。配合藍牙鍵鼠，出差可以唔帶筆記本。
- **華為桌面模式**：華為 Mate 系列/Pura 系列連接顯示器後亦有類似功能。支援多視窗、檔案管理、辦公 App。
- **摩托羅拉 Ready For**：Moto 旗艦亦支援，功能類似 DeX。
- **適用場景**：輕辦公（回郵件、改文檔）、演示 PPT、大屏睇片。唔適合重度 Excel 操作或專業設計。

四、車載互聯——揸車用手機嘅正確方式

揸車時操作手機係違法嘅，但導航、聽音樂、接電話又係剛需。車載互聯系統等你喺車機螢幕上使用手機嘅核心功能，手機可以放埋一邊唔碰：

- **Apple CarPlay**：iPhone 用戶嘅首選。透過 USB 或無線連接車機，車機螢幕上出現 iPhone 嘅精簡介面——導航（高德/百度）、音樂、電話、資訊、播客。Siri 語音控制全程免手。
- **Android Auto**：安卓用戶對應方案。功能與 CarPlay 類似，支援 Google Maps、Spotify 等。國內因 Google 服務限制，部分功能唔可用。
- **華為 HiCar**：華為自研方案，國內車型支援率高。除咗導航音樂，仲能用華為視像通話、泊車影像融合等特色功能。
- **百度 CarLife**：跨平臺方案，iOS 同安卓都能用。功能較少但相容性廣，好多舊車型淨係支援 CarLife。

安全提醒

無論邊種車載互聯方案，揸車時都應盡量使用語音控制。導航提前設好目的地，音樂提前選好歌單，唔好喺行駛中操作車機螢幕。

五、會議室投屏實戰——唔再手忙腳亂

開會係投屏最高頻嘅場景，亦係最容易出問題嘅場景。以下係幾條實戰經驗：

- **提前到場測試**：永遠唔好等到會議開始先至連投屏。提前 10 分鐘到場，測試好連接方式。Wi-Fi 投屏失敗嘅概率比你諗嘅高得多。
- **準備有線方案做 Plan B**：喺袋入面便放一根轉 HDMI 線。無線投屏連唔上嘅時候，有線就係救命稻草。
- **關閉通知預覽**：投屏時你嘅微信消息會顯示喺會議室大屏上。投屏前喺設定入面便關閉「鎖屏通知內容預覽」，或者開啟「演示模式」（部分手機支援，投屏時自動隱藏通知）。
- **用「演示文檔」而唔係「操作手機」**：如果你要展示 PPT，直接將 PPT 存到手機入面便用 WPS 打開全屏播放，而唔係投屏整個手機桌面。咁樣更專業、更流暢。

動手：完成一次手機到電視嘅投屏

1. 確保手機同電視連同一個 Wi-Fi。
2. 用手機打開一張相片或一個影片。
3. 嘗試無線投屏（iPhone 用螢幕鏡像，安卓用無線投屏按鈕）。
4. 如果失敗，檢查電視係咪支援對應協議，或嘗試 DLNA 推送（喺相冊/影片 App 入面便點「投屏」按鈕）。
5. 投屏成功後，試嚇喺手機上做其他操作，睇嚇大屏係咪同步顯示。

本章小結

- 無線投屏三陣營：AirPlay（蘋果）、Miracast（安卓）、DLNA（跨平臺推媒體）
- 有線投屏更穩定零延遲，會議室場景必備轉 HDMI 線
- Samsung DeX/華為桌面模式等手機變身輕辦公電腦
- 車載互聯：CarPlay/Android Auto/HiCar/CarLife 四大方案
- 會議室投屏鐵律：提前測試、準備 Plan B、關閉通知預覽

第三十一章：短影片拍攝與內容創作入門

2026 年，短影片已經唔係「後生仔嘅玩具」喇，佢變成所有人獲取資訊同表達自我嘅基礎方式。唔管你想記錄生活、分享知識，仲係做啲副業，手機就係你最好嘅創作工具——唔使買相機、唔使學 *Premiere*，一部手機就能開始。呢一章幫你由零起步。

一、拍之前先諗清楚：你要拍畀邊個睇？

好多人拍影片嘅第一個錯誤就係「攞起手機就拍」，拍完唔知畀邊個睇。喺開拍之前，先回答三個問題：

1. **我嘅觀眾係邊個？**——拍畀仔睇、拍畀同齡人睇、拍畀長輩睇，內容風格完全唔同。

2. **我能提供乜嘢價值？**——等人笑（娛樂）、等學嘢（知識）、等人饞（美食）、等人暖（情感）。至少佔一樣。

3. **我能喺邊個平臺持續更新？**——抖音/快手（泛娛樂）、小紅書（生活方式）、B 站（深度內容）、影片號（社交圈）。選一個主陣地深耕，唔好一嚟就全平臺分發。

二、手機拍攝基礎設定——開拍前必做嘅 5 件事

1. **解像度設 1080P，幀率設 30fps：**呢個係短影片平臺嘅標準。4K 檔案太大上傳慢，60fps 喺社交平臺上冇優勢反而檔案更大。

2. **抹乾淨鏡頭：**第一條亦係最重要嘅一條。拍之前用衫抹一下鏡頭，畫面立刻清晰一個檔次。

3. **鎖定對焦同曝光：**喺取景框長按畫面主體 2 秒，會出現「AE/AF 鎖定」提示。咁樣拍嘅時候唔會因為移動導致畫面忽明忽暗。

4. **橫拍定豎拍？**——發抖音/快手/影片號/小紅書**豎住拍**（9:16），發 B 站/YouTube **橫住拍**（16:9）。諗清楚平臺再決定方向。

5. **開網格線：**設定 > 相機 > 網格線。三分法構圖嘅基礎，幫你快速搵到畫面重心。

三、運鏡五招——等畫面「鬱」起嚟

新手拍影片最大嘅問題係畫面太「靜」——全程企住唔鬱拍。學識下面五個基本運鏡，畫面立刻有專業感：

- **推（Push）**：手機慢慢向被拍對象靠近。製造「靠近」嘅緊張感或親密感。
- **拉（Pull）**：手機慢慢遠離被拍對象。由細節拉到全景，交代環境。
- **搖（Pan）**：手機位置唔鬱，左右轉動。適合影風景、展示空間。
- **移（Track）**：手機平行移動，跟住被拍對象行。需要手持穩定，最好配合手機穩定器。
- **跟（Follow）**：手機跟喺被拍對象後面或前面行。Vlog 最常用嘅運鏡。

穩係第一要務：手震嘅畫面睇落好唔專業。雙手持機比單手穩；行路時膝頭微屈，用「忍者步」減少上下顛簸；如果有預算，買一個手機穩定器（大疆 OM 系列約 500-800 蚊），效果質變。

四、剪輯入門——剪映就夠

剪映（CapCut 嘅國內版）係目前最適合新手嘅手機剪輯工具，免費、功能全、模板多。以下係基礎工作流：

1. **匯入素材**：打開剪映 > 開始創作 > 選擇拍好嘅影片片段匯入。
2. **裁剪**：拖動時間軸，將每個片段開頭結尾嘅廢片剪掉。原則：每一段淨係保留有資訊量嘅部分，寧短勿長。
3. **加轉場**：兩段影片之間點「轉場」按鈕，選一個簡單嘅轉場效果。新手建議用「疊化」或「黑場」，唔好用花哨嘅 3D 轉場。
4. **加字幕**：剪映有「識別字幕」功能，一鍵自動生成字幕。準確率約 90%，手動校對一遍即可。
5. **配音樂**：剪映音樂庫入面便有大量版權免費嘅音樂。選一首節奏同內容匹配嘅 BGM，音量調到影片原聲嘅 1/3 左右。
6. **匯出**：1080P 匯出，幀率 30fps，碼率選「推薦」即可。

五、封面嘅重要性——70% 嘅人淨係睇封面決定睇唔睇

短影片嘅封面就係「門面」，決定別人要不要點入嚟。幾個做封面嘅要點：

- **封面要有一句話**：用大字寫明影片嘅核心內容，比如「3 分鐘學識手機修圖」。字體要大、顏色要亮、同背景有對比。

- **封面同內容要一致**：唔好做「標題黨」——封面講一件事，點入去係另一回事，觀眾會立刻劃走，算法會降權。
- **人物表情要生動**：如果封面有人，選一個表情生動或誇張嘅瞬間。人臉比文字更能吸引撳。
- **剪映做封面**：喺剪映入面便選一幀最好嘅畫面，加文字疊加，匯出為圖片作為封面。

六、內容創作嘅三條心法

- **前 3 秒定生死**：短影片平臺，觀眾 3 秒內決定掃走定係繼續睇。前 3 秒必須畀出「鉤子」——一個問題、一個懸念、一個意外畫面。
- **一條影片淨係講一件事**：新手最容易犯嘅錯係一條影片塞太多內容。一條影片淨係解決一個問題，講透比講全重要。
- **堅持比天賦重要**：算法推薦需要「養號」。前 20 條影片幾乎唔會有播放量，呢個係正常嘅。堅持每週更新 2-3 條，等算法認識你嘅內容方向，流量會慢慢起嚟。

動手：拍一條 30 秒嘅短影片

1. 選一個你擅長嘅小知識（比如「點樣將手機字體調大」）。
2. 用手機豎屏拍攝 3-5 段素材，每段 5-10 秒。
3. 用剪映接駁、裁剪、加字幕、配 BGM。
4. 做一張帶大字嘅封面圖。
5. 匯出後先發畀朋友睇，收集反饋再發布。

本章小結

- 拍之前諗清楚：觀眾係邊個、提供乜價值、喺邊個平臺發
- 開拍 5 件事：1080P/30fps、抹鏡頭、鎖對焦曝光、確定橫豎、開網格線
- 運鏡五招：推、拉、搖、移、跟——穩係第一要務
- 剪輯用剪映：裁剪轉場字幕 BGM 匯出，六步走
- 前 3 秒定生死，一條影片淨係講一件事，堅持比天賦重要

第三十二章：手機安全急救手冊

呢一章係一份「應急預案」。希望你永遠用唔上，但如果真係遇到手機丟失、被盜、被詐騙嘅情況，呢份手冊可以幫你將損失降到最低；地震、暴雨等災害嚟臨時，手機同樣係接收預警、定位求救、保持聯絡嘅工具。建議你而家就通讀一次，然後將關鍵步驟截圖保存，或者打印出嚟放喺屋企——因為出事嘅時候你嘅手機可能已經唔喺手上面，災害時就可能冇電或者冇訊號。

場景零：地震 / 極端天氣——手機點樣幫你保命

防嘅係人身安全，唔係帳號資金。預警設定詳見第十三章「天氣預警、自然災害預警同地震預警」；呢度只講警報響起之後應該點做。

地震警報響咗：先躲後撤，唔好只顧住拍片

1. 就近避險：趴低、掩護、揸實（枱下、承重牆角、堅固傢俬旁），護住頭頸；遠離玻璃、外牆、吊燈。
2. 唔好盲目衝樓：主震往往只有十幾秒到幾十秒，樓梯間可能更危險；電梯絕對唔可以用。
3. 震停之後再有序撤離：行樓梯、留意餘震同墮物；去開闊地，遠離電線桿、廣告牌、危樓。
4. 手機用喺刀刃上：畀緊急聯絡人發簡短訊息（「我喺 XX，安全/受傷，需要幫助」）比打長電話更慳電；可以發短訊/微信時優先文字。
5. 冇訊號時：見第七章衛星 SOS / 衛星訊息（機型支援時）；或者去高處、開闊地再試；部分場景下無網藍牙對講亦有幫助。
6. 關閉非必要功能：應急時開飛行模式再定時開、關藍牙掃描、降亮度，可以多撐幾小時；將最後一格電留畀報平安同救援訊息。

暴雨 / 颱風 / 山洪：預警階段就要行動

- 睇到紅色預警：盡量減少外出，遠離地下空間、下穿橋、河道、陡坡。
- 喺戶外被困：向堅固建築物高處行，唔好強行涉水（積水可能有井蓋缺失同漏電）；打 119/110 時講清楚地標同水位。
- 出行前：用天氣 App 同當地預警公眾號確認風險；長途自駕避開山區臨水臨崖路段。
- 家人互報：約定「若失聯，以最後微信/短訊定位為準」，避免各自瘋狂撥號耗光電量。

平時就做好嘅 4 件事

1. 開啟系統地震預警 / 正規預警 App，並允許佢突破靜音。
2. 喺通訊錄置頂 2-3 位「緊急聯絡人」，鎖屏介面可以顯示（iOS/Android 醫療急救卡或者鎖屏資訊）。
3. 屋企同袋入面備充電寶；長途出行前充滿電並帶線。
4. 記住至少一個「唔用通訊錄都打得通」嘅號碼（家人手機號），寫喺紙上放銀包。

場景一：手機唔見咗/俾人偷咗——黃金 30 分鐘

手機丟失後嘅前 30 分鐘係「黃金時間」，處理得當可以保住你嘅錢財同私隱。按以下順序操作，一步都唔好跳：

丟失後 30 分鐘標準操作流程

1. 借人哋手機撥打自己號碼（第 1 分鐘）：可能只係跌咗喺附近，好心人接咗就可以拎返嚟。
2. 遙距鎖定手機（第 2-5 分鐘）：用人哋嘅手機或者電腦登入查找服務：iPhone：iCloud.com > 查找 > 揀你嘅裝置 > 標記做遺失
3. Android 各品牌：華為 cloud.huawei.com / 小米 i.mi.com / OPPO cloud.oppo.com 等
4. 掛失 SIM 卡（第 5-10 分鐘）：打電信商客服電話（移動 10086/聯通 10010/電信 10000），報手機丟失要求暫停 SIM 卡服務。呢一步最關鍵——騙子拎到你嘅 SIM 卡就可以接收驗證碼，重設你所有密碼。
5. 凍結支付帳戶（第 10-20 分鐘）：支付寶：95188 掛失，或者喺人哋手機上登入支付寶 > 我嘅 > 設定 > 安全中心 > 緊急凍結
6. 微信：110 報警之後聯絡微信客服，或者喺微信安全中心 > 凍結帳號
7. 銀行 App：撥打銀行卡背面嘅客服電話，要求凍結網銀
8. 通知親友（第 20-25 分鐘）：喺朋友圈或者叫家人幫手發訊息，通知親友手機唔見咗，防止騙子用你嘅身份呃人。
9. 報警（第 25-30 分鐘）：撥打 110 或者去最近嘅派出所報案，攞到報案回執。後續保險理賠、銀行糾紛都需要呢個回執。

場景二：俾人呃咗——72 小時黃金處理期

發現自己俾人呃咗之後，最關鍵係冷靜。慌亂中做錯誤決定會導致二次損失。按以下步驟操作：

1. 即刻停止一切轉帳操作：唔理騙子講乜嘢，唔好再轉一分錢。「再轉一筆保證金就可以退回之前嘅錢」係經典二次詐騙話術。

2. 保留所有證據：唔好刪聊天記錄！唔好刪通話記錄！截圖保存聊天記錄、轉帳記錄、對方帳號資訊。呢啲都係報案同追回資金嘅關鍵證據。

3. 即刻撥打 96110：呢個係全國反詐專線。接線員會指導你做緊急止付——如果轉帳時間唔長（通常 24 小時內），銀行有可能凍結對方帳戶攔截資金。

4. 撥打 110 報警：帶齊手機、身份證、轉帳憑證去最近嘅派出所做筆錄。

5. 聯絡銀行/支付平臺：告知俾人呃咗嘅情況，要求凍結涉案帳戶。如果係信用卡俾人盜刷，要求銀行啟動拒付流程。

6. 修改所有密碼：喺另一部安全裝置上修改你嘅 Apple ID/品牌帳號、微信、支付寶、銀行 App 嘅密碼，開啟雙重認證。

場景三：手機中咗病毒/木馬——點算好？

如果你發現手機出現以下症狀——莫名其妙彈出廣告、自動安裝奇怪 App、話費暴增、流量異常消耗——好可能係中咗惡意軟件：

- 第一步：入安全模式——Android 手機長按電源鍵關機，再長按電源鍵開機，開機震動後長按音量減鍵唔放，直到入安全模式（左下角會顯示「安全模式」字樣）。安全模式下所有第三方 App 被停用，方便排查。

- 第二步：卸載可疑 App——喺安全模式下入設定 > 應用管理，按時間排序，搵到最近安裝嘅唔認識嘅 App 卸載佢。

- 第三步：安裝安全 App 掃描——下載「騰訊手機管家」或者「360 手機衛士」做一次全盤掃描。

- 第四步：復原出廠設定（終極方案）——如果以上方法無效，備份重要數據後復原出廠設定。留意：復原出廠會刪除所有數據，一定要先備份相片、通訊錄等重要檔案。

- iPhone 基本唔會中毒——iOS 嘅沙盒機制令惡意軟件好難生存。如果你嘅 iPhone 出現異常，大概率係某個 App 嘅 bug 或者設定問題，唔係病毒。復原出廠設定可以解決 99% 嘅問題。

場景四：帳號被盜——快速奪返控制權

1. 即刻修改密碼：如果你仲可以登入帳號，第一時間改密碼。新密碼要同舊密碼完全不同。

2. 踢走其他裝置：喺帳號安全設定入面搵到「登入裝置管理」，將唔認識嘅裝置全部踢下線。

3. 檢查登入記錄：查看最近 7 日嘅登入記錄，如果有異地登入，就表示帳號真係被盜咗。

4. 開啟雙重認證：如果你仲未開雙重認證，而家即刻開。呢個係防止帳號再次被盜最有效嘅方法。

5. 檢查綁定資訊：檢查帳號綁定嘅手機號、電郵有冇被改。如果被改咗，用帳號申訴功能搵返。

6. 檢查支付設定：如果被盜嘅係微信/支付寶，檢查免密支付、自動扣款有冇被篡改，銀行卡有冇被加到陌生帳戶。

場景五：收到勒索訊息——唔使驚，唔好畀錢

常見嘅勒索套路有兩種：(1)「我有你嘅不雅影片/相片，唔畀錢就發畀你通訊錄入面所有人」；(2)「你嘅手機俾黑客控制咗，唔畀錢就鎖死」。遇到呢種情況：

- 99.9% 係詐騙：佢哋手入面根本冇你嘅任何私隱數據。羣發同樣嘅勒索訊息畀幾萬人，總有人驚到畀錢。
- 唔好回覆，唔好畀錢：回覆咗等於話畀對方知「呢個號碼有人用」，你會被標記做「可勒索目標」，後續騷擾更多。
- 截圖保存證據：然後拉黑對方。
- 修改密碼：如果擔心，改一下你主要帳號嘅密碼，求個安心。
- 如果真係有私隱洩露風險：報警。警方有處理呢類案件嘅經驗。

場景六：AI 換臉/AI 擬聲詐騙——眼見不再為實

由 2024 年起，AI 換臉同 AI 擬聲技術已經可以令騙子喺視像通話入面「變成」你嘅家人、上司、朋友嘅面同聲音。「媽，我出事啦，快啲轉錢過嚟」——影片入面嘅人長得、聽起嚟都同你個仔一模一樣，但其實係假嘅。呢個係目前最難防、亦最危險嘅詐騙手法之一。

第一步：識別破綻——AI 偽造嘅四個常見漏洞

- 眨眼頻率異常：真人每分鐘眨眼 15-20 次，AI 換臉往往眨眼過少或者節奏僵硬。
- 嘴型同音訊唔同步：仔細睇口型同聽到嘅聲音之間，會有零點幾秒嘅錯位。
- 面部邊緣模糊：塊面嘅輪廓、耳仔附近、頭髮邊緣容易出現「毛邊」或者閃爍。
- 光影方向唔一致：塊面上面嘅高光、陰影同背景嘅光照方向對唔上，或者側面時五官「跟住轉」得唔啱。

第二步：三招驗證對方真假

- 要求做特定動作：叫對方轉頭、掩面、用手遮住個嘴再做開合動作。AI 換臉喺遮擋同大角度轉頭時最容易「穿煲」——塊面會變形或者跟唔上動作。
- 問只有對方先知嘅問題：編一個假問題，例如「上次我哋隻狗送去邊間醫院睇嘅？」——如果係騙子，佢答唔到或者支支吾吾。

- 回撥已知號碼：掛斷視像，用你通訊錄入面存嘅號碼直接回撥過去。騙子控制唔到真實嘅手機號，一撥就穿煲。

第三步：確認俾人呃咗之後嘅報警流程

1. 即刻停止轉帳：再「急」都先停一下，凡係催你「快啲轉帳、唔好掛電話」嘅，幾乎都係騙子。
2. 保留全部證據：錄屏、截圖保存視像通話畫面、聊天記錄、對方帳號、轉帳憑證——呢啲係技術鑑定嘅關鍵材料。
3. 撥打 96110 反詐專線：告知係「AI 換臉/擬聲」類詐騙，接線員會啟動緊急止付。
4. 撥打 110 報警：帶齊證據去派出所做筆錄，特別說明使用咗 AI 偽造技術，方便警方立案定性。
5. 通知親友：騙子可能用你嘅偽造影片去呃人，提前喺朋友圈或者家庭羣入面發個提醒。

場景七：SIM swap 攻擊（SIM 劫持）——你嘅號碼俾人「偷」走咗

呢個係一種好多人未聽過、但危害極大嘅攻擊。SIM swap（SIM 卡劫持）指嘅係攻擊者透過偽造你嘅身份資訊，冒充你去營業廳補辦一張新嘅 SIM 卡。一旦補辦成功，你手機嘅訊號會突然消失，而攻擊者嘅手機插住嗰張新卡，可以收到你所有嘅短訊驗證碼——支付寶、銀行、微信，全部門戶大開。

點樣發現？SIM swap 嘅三個預警信號

- 手機突然顯示「無服務」/「無 SIM 卡」，但你明明冇鬱過張卡，重啟都冇用——呢個往往係被劫持嘅瞬間。
- 收到電信商發嚟嘅「補卡成功」或者「業務辦理成功」短訊——但你本人冇辦過。
- 突然收唔到驗證碼，但其他短訊都收唔到——因為短訊被劫持咗去攻擊者嘅卡上。

防護措施——將 SIM 卡都「上鎖」

- 設定 SIM 卡 PIN 碼：路徑設定 > 電話 > SIM 卡 PIN 碼。開啟之後，每次手機重啟或者換卡，都要輸 PIN 先用得到呢張卡。攻擊者就算拎到卡都用唔到。
- 開啟電信商二次驗證：登入中國移動/聯通/電信 App，搵到「安全中心」或者「業務辦理驗證」，開啟非本人補卡短訊提醒同業務辦理二次密碼驗證。咁樣人哋去補你嘅卡時，營業員會要求額外嘅身份核驗。
- 重要帳號唔好只用短訊驗證：銀行、Apple ID 等高價值帳號，優先用認證器 App（Google Authenticator、微軟 Authenticator）或者硬件密鑰，而唔係短訊驗證碼。
- 身份證複印件加水印：辦理業務需要嘅複印件寫明「僅供 XX 業務使用」——攻擊者往往靠洩露嘅身份證資訊冒充你。

被劫持後嘅應急處理——速度就係一切

1. 即刻撥打電信商客服掛失：移動 10086 / 聯通 10010 / 電信 10000，要求即刻停機並凍結補卡業務，阻止攻擊者繼續使用你嘅號碼。
2. 用其他裝置登入關鍵帳號：用電腦或者家人手機，即刻修改銀行、支付寶、微信、Apple ID 嘅密碼，並踢走陌生裝置。
3. 凍結支付帳戶：支付寶 95188 掛失，微信安全中心凍結，銀行客服凍結網銀。
4. 帶身份證去營業廳：本人持身份證去自營廳補辦新卡，恢復號碼使用權。
5. 報警：SIM swap 已構成「侵犯公民個人資訊罪」，帶齊證據（停機時間記錄、資金損失、補卡記錄）報案。
6. 全面排查：恢復號碼後，逐一檢查近期登入記錄同資金流水，確認有冇被冒用過嘅痕跡。

預防勝於治療——日常 5 件事

1. 鎖屏密碼 + 雙重認證，兩個都開
2. 「查找我嘅手機」確認開啟
3. SIM 卡設定 PIN 碼（設定 > 電話 > SIM 卡 PIN 碼），唔見咗手機人哋唔可以用你嘅卡
4. 唔安裝來歷不明嘅 App，唔點陌生連結
5. 定期備份重要數據（相片、通訊錄、聊天記錄）

動手：做一份屬於你嘅「應急聯絡卡」

1. 搵一張紙，寫低以下號碼：電信商客服（移動 10086/聯通 10010/電信 10000）、銀行客服、96110 反詐專線、110 報警電話、家人電話。
2. 寫低你嘅 Apple ID/品牌帳號嘅註冊電郵同備用聯絡方式。
3. 將呢張紙放喺銀包入面或者貼喺雪櫃上面——唔好存喺手機入面，因為出事時手機可能唔喺身邊。
4. 而家就檢查：「查找我嘅手機」係咪已開啟，SIM 卡 PIN 碼係咪已設定。

本章小結

- 手機唔見咗黃金 30 分鐘：藉手機撥打遙距鎖定掛失 SIM 卡凍結支付通知親友報警
- 俾人呃咗之後 72 小時：停止轉帳保留證據撥 96110 報警聯絡銀行改密碼
- 手機中毒：安全模式卸載可疑 App 安全掃描復原出廠（終極方案）
- 帳號被盜：改密碼踢裝置查登入記錄開雙重認證查綁定查支付
- 勒索訊息 99.9% 係詐騙，唔回覆唔畀錢，截圖拉黑

- AI 換臉/擬聲詐騙：睇眨眼/嘴型/邊緣/光影破綻，叫對方轉頭掩面、問私密問題、回撥已知號碼
- SIM swap 劫持：手機突然「無服務」即警惕；設 SIM 卡 PIN、開電信商二次驗證、重要帳號改用認證器 App
- 日常預防五件事：鎖屏 +2FA、查找手機、SIM 卡 PIN、唔裝來歷不明 App、定期備份

六、展望篇與車聯篇（第 33-34 章）

第三十三章：未來展望——手機嘅下一程

手機唔會一夜之間消失，但佢嘅「獨霸地位」可能被新一代設備分流。呢一章我哋傾三件事：下一代設備生乜樣、手機點樣「隱身」為新身份基石、以及嗰啲我哋必須提前思考嘅倫理議題。

一、AR/VR 與空間計算：手機之外嘅「第二桌面」

下一代設備嘅競爭已經打響，最有可能分流手機時間嘅，係 AR/VR 與空間計算設備：

- **Vision Pro 後續**：蘋果 Vision Pro 嘅發布標誌住「空間計算」概念嘅正式落地，但第一代產品價格（29999 蚊起）同重量仍係門檻。預計 2026-2027 年會推出更輕、更平嘅平價版 Vision，等空間計算由「開發者玩具」走向「早期大眾」。屆時 iPhone 可能成為 Vision 眼鏡嘅算力伴侶與數據中樞。
- **AR 眼鏡**：畀你眼前開個「第二桌面」，數字資訊變得好似空氣咁無處不喺。Meta 與 Ray-Ban 合作嘅智能眼鏡已賣出數百萬副，國內 Rokid、XREAL、INMO 等廠商亦喺推出消費級 AR 眼鏡。2025-2026 年嘅 AR 眼鏡已能做實時翻譯、導航 HUD、AI 助手對話，但仍受限於光波導顯示同續航，距離「取代手機」仲有 3-5 年。
- **智能戒指與可穿戴**：Samsung Galaxy Ring、Oura Ring 等智能戒指開始流行，作為手機嘅「貼身傳感器」補充健康監測。手機嘅角色正由「萬能終端」轉向「中樞 + 多個輕量化配件」嘅組合。

二、腦機介面：思想成為交互媒介

思想本身將成為交互嘅媒介，但呢條路比想像中漫長：

- **Neuralink 最新進展**：馬斯克嘅 Neuralink 已於 2024 年完成首例人體植入，首位受試者 Noland Arbaugh 透過意念控制電腦滑鼠下國際象棋。目前植入式 BCI（腦機介面）主要面向癱瘓患者輔助場景，目標係幫助佢哋重新「操作」數字世界。
- **非侵入式 BCI**：多家公司開發非侵入式頭環（如 Kernel、OpenBCI），無需手術即可讀取腦電信號，但精度遠低於侵入式。距離「用意念打字」仲有相當距離。

- **消費級展望**：腦機介面距離消費級手機交互仲有 10-20 年，但短期可能先用於無障礙輔助、專注力監測等場景。倫理與私隱問題（「我嘅腦電波算邊個嘅私隱」）將先於技術成熟成為爭論焦點。

三、手機嘅「隱身」與新身份基石

手機嘅另一個未來進化方向或者係「隱身」——化身為無處不喺嘅「幕後推手」，成為你數字身份嘅唯一基石：

- **雲端雙因素認證**：瞬間完成「你係你」嘅認證，登入任何服務無需手動輸驗證碼。
- **數字遺產成為出廠設定**：新機激活時預設數字資產繼承方案，等「身後事」成為出廠預設。
- **由「設備中心」到「身份中心」**：你嘅雲端個人數據保險櫃先至係中心，硬件淨係訪問入口。換機、換設備，身份與數據始終跟住你。

四、倫理思考：我哋，仲係手機嘅主人嗎？

一個終極拷問始終存在：我哋係被技術賦能嘅「神」，仲係被算法餵養嘅「缸中之腦」？

- **AI 倫理**：當端側 AI 比你自己更懂你嘅偏好，你嘅「選擇」係自主意志定係被引導嘅結果？AI Agent 代你落單、回覆消息時，責任與邊界喺邊度？
- **數據主權**：你嘅健康數據、腦電數據、生物特徵數據，係屬於你、屬於設備廠商仲係屬於雲服務商？《個人資訊保護法》同歐盟 GDPR 淨係起點，「數據可攜帶權」同「被遺忘權」嘅真正落地仲有好長嘅路。
- **數字鴻溝**：AI 手機、摺疊屏、衛星通信都喺拉大「用得起」同「用唔起」之間嘅差距。數字基礎設施嘅公平接入，將成為新嘅社會議題——就好似 20 年前「識唔識上網」咁樣關鍵。
- 當算法比你更懂你想睇乜，你嘅「喜歡」有幾多係出於自發？
- 當你每隔幾分鐘就唔自覺咁點亮螢幕，呢個係你嘅習慣定係被設計出嚟嘅「上癮行為」？

五、保持連接，亦保持獨立——你嘅數字生活哲學

無論技術點樣進化，下面呢幾條數字生活哲學，10 年後依然適用：

1. 做資源嘅分配者，而唔係被推送嘅接收者。
2. 等自動化成為你忠實嘅「副駕駛」。
3. 建立你穩固嘅「數字後方」。
4. 定期執行「數字斷捨離」與「社交齋戒」。

5. 永遠將安全同私隱作為一切便利嘅前提。

嚟試嚇：制定你嘅《個人數字設備使用服務條款》

為你同你嘅手機（以及未來嘅 AR 眼鏡、AI 助手）寫一份簡單嘅「服務條款」，嚟定義你哋嘅關係。例如：

- 第 1 條：我嘅注意力時間屬於我。你喺我工作/陪伴家人時應主動進入「勿擾」狀態。
- 第 2 條：我嘅個人數據係我嘅財產。你嘅首要職責係安全加密與便捷同步。
- 第 3 條：我哋嘅共同目標係提升生活效率與體驗。若你透過誘導性設計等我沉迷，我有權將你卸載。

本章小結

- Vision Pro 與 AR 眼鏡係手機之外嘅新計算入口，但短期難以取代手機
- 腦機介面距離消費級仲有 10-20 年，倫理問題先於技術成熟
- 手機正朝「隱身」方向進化，由設備中心走向身份中心
- AI 倫理、數據主權、數字鴻溝係未來 10 年嘅核心議題
- 保持連接亦保持獨立，係數字生活不變嘅哲學

第三十四章：車機互聯深度指南——手機與汽車嘅融合

汽車正由「四個轆加梳化」進化為「四個轆加智能手機」。無論你揸嘅係燃油車定新能源，手機同車機嘅關係都決定咗你嘅駕駛體驗。呢一章帶你釐清車機互聯嘅三大路線、各派系車機系統嘅特點，以及嗰啲等揸車更順手嘅小技巧。

一、車機互聯嘅三大路線

當前手機與汽車「聯網」主要有三種思路，理解咗佢哋就理解咗整個生態：

- **手機投屏到車機**：車機淨係當一塊「顯示器」同「音箱」，運算同地圖數據都來自手機。代表方案：Apple CarPlay、Google Android Auto、華為 HiCar、百度 CarLife。優點係手機生態直接搬上車，唔使重新裝 App；缺點係車機本身嘅硬件能力被閒置。

- **手機與車機深度融合**：車機係獨立系統，但同手機共享帳號、應用、數據，甚至能「接力」任務。代表方案：鴻蒙座艙（問界/智界）、比亞迪 DiLink、蔚來 NIO OS、理想車機系統。優點係體驗一體化；缺點係通常綁定同一品牌生態。

- **CarPlay 2.0 深度集成版**：2025 年蘋果發布嘅新一代 CarPlay，唔再淨係「投屏」，而係接管車機嘅儀錶板、空調控制、座椅調節等核心功能。法拉利、阿斯頓 • 馬田、保時捷等品牌率先搭載，標誌車機系統進入「手機廠商主導」嘅新階段。

二、CarPlay / Android Auto / HiCar 對比

三大主流投屏方案各有側重，選車或選手機時可以對照下表：

方案	支援手機系統	支援車型	功能豐富度	國內可用性
Apple CarPlay	僅 iPhone	極廣，絕大多數車型支援	高，地圖/音樂/電話/消息均可	好，原生可用
Android Auto	Android（國內手機需裝服務框架）	較廣，合資/進口車居多	高，與 CarPlay 相當	一般，需科學上網激活
華為 HiCar	華為手機為主，部分支援其他安卓	國產車與部分合資車型	較高，可與鴻蒙座艙聯動	好好，本土生態
百度 CarLife	iOS 與 Android 均可	較廣，舊車型後裝常見	中等，更新較慢	好好，本土生態

三、鴻蒙座艙：手機與車機嘅無縫流轉

如果講 CarPlay 係「機投到車機」，鴻蒙座艙則係「手機同車機本來就係一家」。華為手機與問界/智界車機嘅協同，代表咗當前國內車機互聯嘅最高水準：

- **超級桌面**：車機直接調用手機上嘅 App，唔使喺車機重新安裝。手機入面便喺聽嘅音樂、喺導航嘅路線，上車自動續接到車機螢幕，落車再返到手機，全程無縫。
- **應用接力**：手機上設好嘅導航，埋車門時車機自動接管；到達停車場，落車後手機繼續步行導航到最終目的地。「最後一公里」唔再斷檔。
- **手機車機共用帳號**：華為帳號一登，車機自動同步你嘅音樂會員、地圖收藏、日程、聯絡人。車機同手機係同一身份嘅唔同終端，而唔係兩套獨立系統。

四、造車新勢力嘅車機系統

國產新能源車嘅車機已經由「導航 + 音樂」進化為移動智能終端，各家嘅玩法各有特色：

- **蔚來 NIO OS**：NOMI 語音助手係行業標杆，能識別多指令、上文下連續對話，仲帶情感表情。NIO Phone 則將手機做成「車匙 + 第二屏」，深度綁定蔚來生態。
- **理想車機**：多屏交互係亮點，副駕屏可獨立睇劇、仔屏可睇卡通。「任務大師」等你自訂場景，比如「開門自動暫停音樂、埋門繼續播放」。
- **小米澎湃車機**：同小米手機、手錶、家電深度聯動。手機上設嘅鬧鐘會喺車機提醒；車上導航返屋企，屋企冷氣提前開啟。HyperOS 打通咗「人車家全生態」。
- **比亞迪 DiLink**：標誌性嘅旋轉屏，豎屏睇短影片、橫屏睇地圖，自適應 App 方向。自帶應用市場，生態豐富度喺傳統車企中領先。

五、手機當車機——用得嗎？

如果你嘅車機太老或太難用，能否直接用手機「頂上」？答案係：可以，但要睇場景。

- **摩托羅拉 Ready For / 桌面模式：**部分摩托羅拉、三星、華為手機支援「桌面模式」，連上顯示器就係一臺簡易電腦。理論上接車機屏都能用，但實際相容性參差不齊，且缺少汽車專屬交互（如方向盤按鍵）。
- **手機支架 vs 原車車機：**如果原車車機卡頓、地圖唔更新，一個穩固嘅手機支架 + 手機導航係務實之選。但要注意：(1) 手機螢幕細，揸車時睇唔清；(2) 夏天暴曬會等手機過熱降頻甚至自動關機；(3) 長時間導航耗電快，需配車載充電器。
- **建議：**短途或臨時用車，手機支架夠用；日常通勤長途，建議升級支援 CarPlay/HiCar 嘅車機或加裝後裝車機，體驗同安全都更好。

安全提醒

無論用邊種方案，揸車時操作手機都違反交規。手機支架淨係可以用嚟「睇導航」，唔可以用嚟「操作手機」。設定目的地、切換歌曲等操作，請務必停車後再做，或使用語音控制。

六、實用技巧

- **無線 CarPlay 連接唔穩定：**(1) 關閉車機藍牙再重新配對；(2) 刪除車機端已配對設備記錄後重連；(3) 檢查手機係咪同時連咗多個藍牙設備造成衝突；(4) 升級車機韌體同手機系統到最新版；(5) 真係唔得，改用有線 CarPlay，更穩定。
- **車機流量用完：**(1) 用手機開熱點，車機連手機熱點上網；(2) 喺車機設定中將「高耗流量」應用（如影片、線上音樂）切到 Wi-Fi；(3) 定期清理車機離線地圖快取。
- **駕駛模式自動觸發：**(1)iOS「專注模式」設定「連接車載藍牙時自動開啟」，自動屏蔽非緊急通知；(2) 安卓「駕駛模式」或第三方 App（如 Tasker）可實現連藍牙自動靜音、自動回覆消息；(3) 華為手機可設定「智感支付」，上車自動切到車機播放音樂。

選車時嘅小建議

如果喺乎車機體驗，選車時務必去 4S 店實測：(1) 帶自己嘅手機試 CarPlay/HiCar 連接係咪穩定；(2) 試語音助手嘅反應速度同指令識別率；(3) 睇應用市場有冇你常用嘅 App；(4) 關注後續 OTA 升級頻率——好車機係用軟件迭代出嚟嘅。

動手：優化你嘅車機互聯體驗

1. 確認你嘅手機同車機支援嘅互聯方案（CarPlay / HiCar / Android Auto / CarLife）。
2. 清理一次車機嘅藍牙配對列表，淨係保留常用手機，減少連接衝突。
3. 設定一個「駕駛專注模式」，連接車機藍牙時自動開啟。
4. 下載車機品牌 App（如鴻蒙智行、理想汽車、蔚來 App），睇嚇能否遠程控制冷氣、查

看車況。

本章小結

- 車機互聯三大路線：投屏（CarPlay 等）、深度融合（鴻蒙座艙等）、CarPlay 2.0 深度集成
- CarPlay 生態最廣；HiCar 國內體驗好；Android Auto 國內需折騰；CarLife 適合舊車後裝
- 鴻蒙座艙嘅超級桌面、應用接力、共用帳號代表咗深度融合方向
- 蔚來 NOMI、理想多屏、小米人車家、比亞迪旋轉屏各有特色
- 手機當車機僅適合臨時場景，長期建議升級或後裝車機
- 實用技巧：CarPlay 重連、手機熱點補流量、藍牙觸發駕駛模式

兒童與手機專題（第 35 章）

第三十五章：兒童與手機——數字時代嘅教育指南

本章唔講「手機係洪水猛獸」，亦唔鼓吹「越早接觸越聰明」。我哋按年齡分段畀設備、設規則、教素養、護健康，一步步嚟。

一、幾歲畀仔手機？

冇標準答案，但有年齡基線。過早畀手機，仔大腦前額葉尚未發育成熟，自控力跟不上算法嘅「誘惑設計」；過晚畀，又可能等佢喺同伴中「社交脫節」。下面係分齡建議：

- **3-6 歲**：盡量唔碰手機。如需使用，單次唔超過 15 分鐘，內容以家長精選嘅教育動畫為主，且必須**陪同觀睇**。呢一階段螢幕對視力與注意力發育影響最大。
- **7-9 歲**：可喺家長設備上「借用」，用於網課、聽故事、查資料。開始建立「使用前申請、用完歸還」嘅規則，唔配置專屬手機。
- **10-12 歲**：可考慮畀一臺「功能機」或兒童手錶，解決上下學聯絡。如確需智能機，應開啟嚴格嘅家長控制。
- **13 歲以上**：進入初中，社交需求增強，可以畀智能機，但需配合「使用公約」同定期復盤。

> **「夠用就好」原則**
>
> 仔嘅第一部手機唔需要旗艦。處理器嘅「過剩效能」往往被用嚟跑遊戲。選一臺電池夠大、效能中等、螢幕護眼嘅千元機或官方翻新機，反而更有利管控。預算參考：1500-2500 元區間即可滿足絕大多數學生需求。

智能手錶作為過渡方案

如果淨係想解決「放學後能不能聯絡上」嘅問題，兒童手錶比手機更合適。佢淨係可以打電話、發語音、定位，冇瀏覽器同短影片，天然限制咗「沉迷入口」：

- **小天才電話手錶**：生態最完整，同齡人社交「碰一碰加好友」係其核心賣點，但專屬 App 生態較封閉。

- **華為兒童手錶**：同華為手機聯動好，定位精準，支援視像通話，性價比高。
- **小米兒童手錶**：價格親民，基礎功能齊全，適合預算有限嘅家庭。

買手錶時注意：選擇支援 **4G 全網通**嘅型號（2G/3G 正逐步退網），並確認運營商兒童套餐資費。

二、家長控制系統全攻略

設備畀到仔之前，第一件事唔係開機，而係設定家長控制。兩大平臺都有原生方案：

iOS：螢幕使用時間

- **App 限額**：設定 > 螢幕使用時間 > App 限額，可為每個 App 類別設定每日可用時長（如遊戲 30 分鐘）。
- **停用時間**：設定每日「熄燈時段」（如 22:00-7:00），期間所有 App 變灰禁用，淨係電話可用。
- **內容限制**：可按年齡段過濾影視、網頁、App 安裝，並禁止 App 內購買。
- **家庭共享**：用家長嘅 iPhone 遠程管理仔嘅設備，批准下載請求、查看使用報告。

安卓：Google Family Link / 品牌家長控制

- **Google Family Link**：谷歌官方方案，可遠程管理設備使用時長、鎖定設備、查看活動報告。國內安卓需視機型可用性。
- **小米兒童空間**：設定 > 兒童空間，可設定使用時長、白名單 App，退出需家長密碼。
- **華為學生模式**：設定 > 健康使用手機 > 學生模式，限制使用時段與 App，支援遠程管理。
- **OPPO/vivo 兒童模式**：喺「設定」或專屬「兒童空間」App 中開啟，原理類似。

功能	iOS 螢幕使用時間	安卓 Family Link / 品牌方案
App 限額	支援，按類別	支援，按 App
停用時間	支援	支援（品牌方案略有差異）
內容過濾	系統級，較徹底	依賴品牌，參差不齊
遠程管理	家庭共享	Family Link / 品牌雲
安裝 App 審批	需家長批准	Family Link 支援

家長控制唔係「一勞永逸」

仔總能搵到繞過嘅方法（網上搜「點樣破解螢幕使用時間」嘅教程一大把）。技術手段淨係底線，真正嘅防線係親子溝通同信任。建議每月同仔一齊復盤使用報告，而唔係單方面監

控。

三、青少年模式

各大內容平臺都內置咗「青少年模式」，開啟後會限制使用時長、過濾唔良內容、關閉充值打賞。呢個係國家法規要求嘅強制功能，但**需要家長主動開啟並設定密碼**：

- **抖音/快手青少年模式**：每日限 40 分鐘（22:00-6:00 禁用），內容淨係展示精選科普、教育類影片，無法直播、無法打賞、無法充值。
- **微信青少年模式**：限制微信支付（需家長授權額度），限制公眾號、影片號內容，搖一搖、附近嘅人等功能預設關閉。
- **遊戲防沉迷系統**：國家層面嘅實名認證 + 時間限制。未成年人僅可喺週五、週六、週日及法定節假日嘅 20:00-21:00 登入遊戲，每日 1 小時。

實名認證唔好用家長身份證「借號」

好多仔用家長身份證註冊遊戲帳號繞過防沉迷，呢個既違反用戶協議，亦等仔失去保護。家長應保管好自己嘅支付密碼與身份資訊，唔好為咗慳事「借號」畀仔。

四、網絡素養教育

技術能攔截 90% 嘅風險，剩低 10% 靠仔嘅判斷力。比起「唔畀上網」，更重要嘅係教佢哋「點樣上網」：

- **私隱保護**：唔向陌生人透露真實姓名、學校、家庭住址；唔發送個人相片、身份證、家庭合影；定位功能預設關閉，僅喺使用時開啟。
- **網絡欺凌識別與應對**：被辱罵、孤立、惡意 P 圖都屬於欺凌。話畀仔聽遇到時唔好對罵、唔好刪除證據、第一時間話畀家長聽。微信可長按消息投訴，平臺會處理施暴帳號。
- **內容批判**：教仔分辨「事實」同「觀點」，識別標題黨、震驚體；理解算法淨係推你想睇嘅，唔代表世界嘅全貌；遇到「天上跌餡餅」嘅廣告一律唔信。
- **網絡安全**：唔撳陌生連結、唔掃來歷不明嘅二維碼、唔下載非官方商店嘅 App；密碼唔分享畀任何人（包括「好朋友」）；遇到索要驗證碼嘅一律視為騙子。

動手：同仔一齊簽一份《手機使用公約》

將規則寫低、雙方簽字貼喺顯眼處，比口頭約定有效得多。建議包含：

- 使用時段：如「作業完成後 19:30-20:30 可用」。
- 禁用區域：餐桌上、臥室睡前、課堂上手機一律收起。
- 內容邊界：唔安裝未告知家長嘅 App，唔同陌生人私聊。

- 違約後果：如「超時一次，次日使用時長減半」。
- 家長承諾：唔喺仔面前「掃手機」超過約定時長，以身作則。

五、健康使用習慣

仔嘅眼睛、頸椎、睡眠都喺發育期，手機對身體嘅傷害比成年人更大：

- **螢幕時間建議**：3-6 歲每日唔超過 1 小時；6-12 歲學習類唔限制，娛樂類唔超過 1 小時；12 歲以上同仔協商，但娛樂類唔超過 2 小時。
- **護眼習慣**：遵守「20-20-20 法則」——每睇螢幕 20 分鐘，抬頭睇 20 英尺（約 6 米）外嘅遠處 20 秒；睡前 1 小時唔睇手機，藍光會抑制褪黑素影響睡眠。
- **姿勢管理**：手機螢幕與眼睛平齊，唔好低頭含胸；連續使用唔超過 30 分鐘就要起身活動。
- **親子共同規則**：餐桌、臥室、書房設「無手機區」；全家一齊睇戲、戶外活動時手機統一收起，等「放低手機」成為家庭文化而唔係對仔嘅懲罰。

家長係最好嘅「防火牆」

研究表明，父母自身手機使用時長與仔呈強正相關。要求仔唔睇手機，自己卻掃短影片到深夜，規則就會失效。改變仔，先由放低自己嘅手機開始。

六、數字遺產與兒童

呢一節容易被忽略，但好重要。仔嘅數字資產（帳號、相片、充值餘額、遊戲裝備）同樣面臨繼承與私隱問題：

- **兒童數字資產嘅繼承**：仔不幸離世時，其社交帳號、遊戲帳號、雲盤相片嘅繼承流程與成年人相同，需提供死亡證明、親屬關係證明。建議家長平時就為仔嘅重要帳號做「遺產聯絡人」或「緊急聯絡人」設定（如蘋果遺產聯絡人、微信帳號繼承）。
- **未成年人嘅數字私隱**：《個人資訊保護法》規定，處理不滿 14 周歲未成年人個人資訊需取得監護人同意，且應制定專門嘅私隱規則。家長應定期審查仔常用 App 嘅私隱政策，關閉「個性化推薦」同「數據共享」開關。
- **帳號註冊建議**：為仔註冊嘅帳號應使用家長可控嘅郵箱/手機號，便於喺未成年階段統一管理；仔成年後可再做帳號移交。

警惕「兒童數字遺產」嘅另一面

仔由出世起就喺度被「記錄」——家長曬嘅成長相、幼兒園發嘅影片、智能手錶嘅健康數據。呢啲數據一旦上傳就難以徹底刪除。發布前諗一諗：十年後仔會唔會因為呢張相感到尷尬？尊重仔嘅數字人格，由剋制「曬娃」開始。

本章小結

- 畀手機嘅年齡冇標準答案，但 3-6 歲盡量唔碰、13 歲前用兒童手錶或功能機過渡
- 「夠用就好」：第一部手機選千元機而唔係旗艦，效能過剩等於誘惑過剩
- iOS 螢幕使用時間與安卓 Family Link/品牌家長控制係兩套技術防線，配合使用更穩
- 青少年模式與遊戲防沉迷係法規要求，但需家長主動開啟並守護密碼
- 網絡素養比技術攔截更重要：教仔識別欺凌、保護私隱、批判內容
- 健康習慣要由家長做起——你放低手機，仔先至會放低
- 兒童數字遺產與私隱係新興議題，平時就應做「遺產聯絡人」設定並剋制曬娃

第三十六章：手機同智能家居——令屋企變聰明

手機仲可以做一件越嚟越重要嘅事——當成個屋企嘅「遙控器」。由一盞燈、一個插座到一把門鎖，智能家居正在由「極客玩具」變成「家常配置」。呢一章幫你理清協議、生態、中樞同入門套件，再用一份安全清單收尾，令你唔會俾「智能」反噬。

一、智能家居協議：揀邊個好重要

買智能裝置之前先睇佢支援乜嘢協議，呢個決定咗佢可唔可以俾你嘅手機同生態「差遣」。目前主流協議有四種，各自嘅脾氣如下：

- Matter 協議：2022 年由 CSA（連接標準聯盟）發佈，背後係蘋果、Google、Amazon、三星等巨頭聯手。佢嘅目標只有一個——令所有品牌嘅智能裝置講同一種語言。2023-2026 年正快速成為統一標準，支援佢嘅裝置可以同時接入 Apple HomeKit、Google Home、Amazon Alexa 同米家，唔再被單一生態綁架。
- Thread 協議：Matter 嘅底層網絡「管道」。低功耗、自組網（某個節點斷電，訊號會自動繞路），反應快、唔卡頓。買有 Thread 邊界路由器嘅裝置（例如 HomePod mini、Apple TV 4K、小米中樞閘道器），體驗明顯更穩。
- Wi-Fi：帶寬大、可以直接連路由器，但功耗高、裝置多咗容易擠佔頻道，適合攝像機、掃地機呢類需要傳大數據嘅裝置。
- 藍牙（BLE）：配網簡單、慳電，但距離短、唔可以自組網，適合門鎖、溫濕度計呢類小數據裝置。
- Zigbee：老牌低功耗自組網協議，要專用閘道器，裝置便宜穩定，但跨品牌互通差，正被 Matter+Thread 逐步替代。

一句話原則

2026 年買智能裝置，優先睇有冇「Matter over Thread」標識——呢個係目前最經得起時間考驗嘅選擇。舊裝置若果只支援 Wi-Fi 或者藍牙都唔使急住淘汰，先用住，新購時再轉去 Matter。

二、三大生態對比

協議係「語言」，生態係「圈子」。即使都支援 Matter，你日常仍然要揀一個主生態嚟搭自動化同語音控制。三大主流生態對比如下：

生態	優勢	短板	適用人羣
Apple HomeKit	私隱最強（本地處理為主）、同 iPhone/Siri/快捷指令聯動順滑	裝置偏少偏貴、要 HomePod 或者 Apple TV 做中樞	全家蘋果裝置、重視私隱同體驗嘅用戶
米家（小米）	裝置最豐富、性價比最高、小愛同學好用	生態相對封閉、數據多上雲、廣告偏多	預算有限、想一站式配齊全屋嘅入門用戶
華為智聯（鴻蒙）	同鴻蒙手機/平板/手錶無縫流轉、裝置數量快速增長	海外裝置支援少、生態仍喺追趕	華為手機用戶、睇重「超級終端」多裝置協同

揀生態嘅決策建議

睇手機揀生態：iPhone 用戶優先 HomeKit，華為用戶優先智聯，其他 Android 用戶米家最慳心。靠 Matter 協議，唔同生態嘅裝置都可以俾同一部手機控制，唔使一次過「企隊」。

三、手機當家居中樞

喺多數入門場景入面，手機就係你嘅「家居中樞」——透過 App 控制燈光、冷氣、窗簾、安防，仲可以搭建自動化場景，令屋企「自己鬱起嚟」：

- 裝置控制：打開 HomeKit/米家/智慧生活 App，可以遙距或者本地開關燈、調色溫、設定冷氣溫度、拉合窗簾、查看攝像機畫面。
- 自動化場景：將「條件動作」組合起嚟，常用嘅三大場景：返屋企模式：定位到屋企 + 天黑開玄關燈 + 開冷氣 + 拉窗簾 + 門門鎖。
- 離家模式：最後一個出門（定位離家或者門鎖上鎖）關燈 + 關冷氣 + 關插座 + 啟動攝像機 + 佈防。
- 睡眠模式：每晚 23:00 關全屋主燈 + 拉窗簾 + 冷氣切 26 度 + 開夜燈。

NFC 碰一碰：喺門貼一張 NFC 標籤，手機一碰就觸發對應場景（碰一下開門燈、再碰一下切睡眠）。iPhone 用「快捷指令」自動化，Android 用「銀包」或者廠商 App 都可以寫入。

嚟試下：搭一個「返屋企模式」

喺手機智能家居 App 嘅「自動化」入面新增一條：觸發條件揀「到達某個地點」（屋企），動作依次加「開玄關燈」「冷氣設 26 度」「閂窗簾」。儲存之後出街行一個圈，返到屋企睇嚇佢會唔會自己著燈——第一次成功嗰陣嘅滿足感，仲過癮過落單買新設備。

四、入門套件推薦

唔使一次配齊，由呢四件套開始，就可以覆蓋八成日常需求：

- 智能音箱：小愛同學（米家生態，性價比高）/ HomePod mini（蘋果生態，兼做 Thread 中樞）/ Alexa（海外生態廣）。留意：音箱決定咗你嘅語音生態，先定生態再買音箱。
- 智能燈泡：Yeelight（米家/Matter，色溫亮度可調，便宜）/ Philips Hue（色彩好、穩定，但要 Hue 閘道器，偏貴）。留意：買之前確認燈座口徑（E27/B22）同係咪需要調色。
- 智能插座：小米智能插座（便宜、電量統計）/ TP-Link Kasa（海外 Matter 支援好）。留意：用喺大功率電器（熱水器、電暖器）時務必睇額定功率，唔好過載。
- 智能門鎖：鹿客/小米（性價比、米家聯動）/ 凱迪仕（傳統鎖具大廠、機械結構紮實）。留意：確認係咪支援 Matter 或者你所在生態，留意電池續航同應急鎖匙孔位置。

選購順序建議

先買智能音箱（定生態）再買智能燈泡同插座（低門檻、即刻有反饋）最後買門鎖同攝像機（涉及安全，謹慎揀）。一次只加一類裝置，熟悉之後再擴，避免「買返嚟唔識用」。

五、安全注意事項

智能裝置越多嘅屋企，攻擊面亦越大。下面四條係底線，請逐條對照：

1. 改預設密碼：攝像機、路由器、門鎖出廠預設密碼喺網上全部係公開嘅，買到手第一件事就係改成「長而且唔重複」嘅強密碼。
2. 及時更新固件：廠商推送嘅固件往往修復咗安全漏洞，喺 App 入面開啟「自動更新」，或者至少每月手動檢查一次。
3. 網絡隔離：將 IoT 裝置單獨放喺一個 Wi-Fi（或者訪客網絡）上，同手機、電腦嘅主網絡分開。萬一某個裝置被攻破，亦難以橫向威脅你嘅重要數據。
4. 私隱保護：攝像機開啟「私隱區域」遮蔽（例如睡房、洗手間唔影），關閉雲端 24 小時錄像；唔喺屋企嘅攝像機畫面唔好長期開畀第三方 App。

呢幾個坑唔好踩

- 唔好用門鎖嘅「遙距開鎖」功能畀陌生人——優先用臨時密碼，用完即失效。
- 唔好將攝像機直接對住張牀、對住出窗——洩露嘅係你自己嘅私隱。
- 唔好貪平買「白牌」無品牌裝置——多數唔更新固件，等於喺屋企裝咗個後門。

本章小結

- Matter+Thread 係 2023-2026 年嘅統一方向，買新裝置優先睇呢個標識
- 三大生態各有所長：蘋果重私隱體驗、米家重性價比、華為重多裝置協同
- 手機可以當中樞，搭好「返屋企/離家/睡眠」三大場景，屋企就可以自己鬱起嚟
- 入門四件套：智能音箱、燈泡、插座、門鎖，按呢個順序逐步添置
- 改預設密碼、更新固件、網絡隔離、私隱遮蔽——四條安全底線唔可以慳

七、AI 編程實戰篇（第 37-47 章）

第三十七章：範式革命——由「寫代碼」到「指揮 AI 寫代碼」

恭喜你，打開呢本書嘅你，正企喺一場職業範式革命嘅正中央。呢場革命嘅起點，係 *2025* 年一個程序員喺社交媒體上發嘅一句話。聽起嚟有啲戲劇性？但事實就係咁。

一、Vibe Coding 嘅誕生：一句話引發嘅運動

2025 年 2 月，Andrej Karpathy——呢個名請先記住，後面仲會見到佢好幾次——喺社交媒體上發咗一條後來被反覆引用嘅推文：「有一種新編程方式叫 Vibe Coding，你完全憑感覺（vibes）嚟，忽略代碼細節，等 AI 包攬一切。你睇到乜就講乜，複製貼上，基本上用得就可以。」

呢條睇起嚟隨口一講嘅發言，其實精準命名咗一場已經悄然發生嘅範式轉移：人類唔再逐行編寫代碼，而係用自然語言描述意圖，由 AI 完成實現。Karpathy 嘅措辭帶住一絲戲謔，但背後係個深刻嘅洞察：當 AI 嘅代碼生成能力跨過某個閾值之後，編程嘅瓶頸就唔再係「呢行代碼我寫唔寫得出嚟」，而係「呢件事我到底想唔想得清楚」。

> **小貼士：Vibe 係乜嘢？**
>
> Vibe 直譯係「氛圍、感覺」。Vibe Coding 就係「跟住感覺指揮 AI 編程」——你描述想要乜嘢，AI 負責實現。記住呢個詞，佢係呢場革命嘅「出生證明」。

由嗰一刻開始，「編碼」呢個詞嘅含義開始分裂：傳統嘅編碼係將設計翻譯成語法；新嘅編碼係將意圖翻譯成指令。前者考驗手指，後者考驗腦袋。

Vibe Coding 呢個詞好快由戲謔變成嚴肅嘅工程概念。到 2026 年中期，佢已經由一個社交媒體上嘅梗，演化為一套完整嘅方法論體系——涵蓋需求表達、上下文管理、迭代驗證到質量保障嘅全流程。人們亦意識到：「憑感覺」只係入門階段嘅狀態，真正高效嘅 AI 編程需要一套系統化嘅思維方式。而呢套思維方式，就係本書要畀你嘅嘢。

二、2026 年現狀：Agentic Engineering 時代

到咗 2026 年 8 月，AI 編程領域已經經歷咗幾次重要嘅範式迭代。Vibe Coding 係最初嘅火花，但好快大家發現：光「憑感覺」同 AI 傾偈，產出嘅代碼質量極唔穩定——有時驚艷，有時翻車。於係範式開始進化：由簡單嘅對話式編程，走向結構化嘅 Agent 工程。

又係 Karpathy，2026 年初提出咗 Agentic Engineering 嘅概念。同 Vibe Coding 嘅「講咗就生成」唔同，Agentic Engineering 強調：畀 AI 一個目標，等佢自主規劃步驟、調用工具、驗證結果、迭代修正。AI 唔再只係一個代碼生成器，而係一個能夠感知環境、做出決策、執行動作嘅智能體（Agent）。

呢個轉變帶來咗三個關鍵變化：

1. 互動模式變咗：由「一問一答」變成「設定目標後放權」。
2. 工作產物變咗：由「一段代碼」變成「一個完整嘅項目」。
3. 人嘅角色變咗：由「逐行審核代碼」變成「定義邊界、驗收結果」。

數字唔會講大話：Cursor 喺 2026 年 4 月發佈嘅 Cursor 3 版本入面，Agent 模式嘅使用率首次反超 Tab 補全模式，標誌住開發者已經大規模轉向呢種工作方式；而 Claude Code 呢個終端原生嘅 AI 編程工具，到 2026 年中期已達到年化收入 25 億美元嘅規模。下面嘅趨勢圖可以畀你直觀感受呢個交叉時刻：

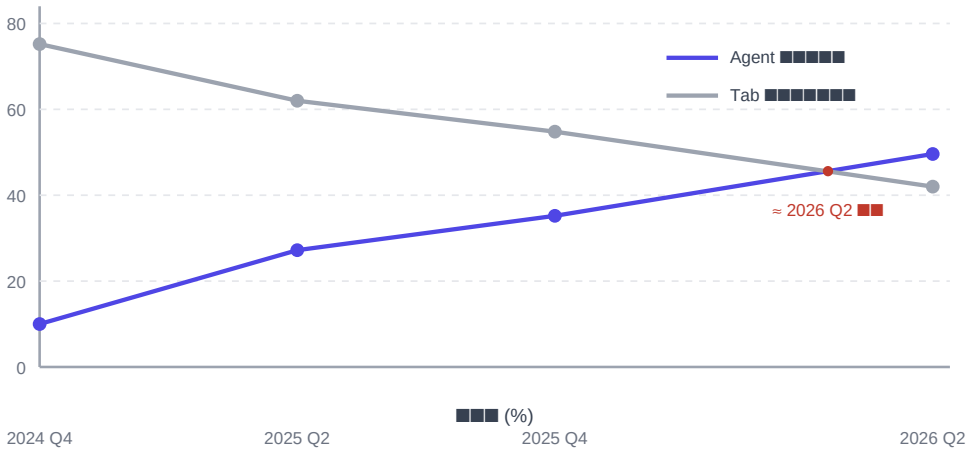


圖 1：AI 編程互動模式使用率變化趨勢（基於行業調研數據估算，2026 Q2 Agent 模式反超 Tab 補全）

但講句實喺話：工具同模型嘅變化只係表象。無論你用 Cursor 定係 Claude Code，無論底層模型係 GPT-5 定係 Claude 4.5，成功嘅 AI 編程實踐者都喺度遵循同一套底層方法論。呢個就係本指南要教你嘅嘢。

三、核心認知轉變：由 Coder 到 Conductor

傳統編程教育教你點樣成為一個 Coder——理解語法、記憶 API、調試邏輯錯誤。AI 編程時代，呢個角色正在被重新定義。你唔再需要親手將每一行代碼寫出嚟，但你需要成為一個「指揮官」（Conductor）：定義戰場、制定規則、分配任務、驗收成果。

呢個轉變嘅核心唔喺於「你識唔識寫代碼」，而喺於「你識唔識想清楚要做乜嘢」。一個優秀嘅指揮官唔一定係最好嘅士兵，但佢必須最清楚戰役嘅目標、地形同兵力部署。同理，一個優

秀嘅 AI 編程實踐者唔一定精通每一種編程語言嘅語法細節，但佢必須最清楚項目嘅目標、架構約束同質量標準。

呢張表值得你多睇兩次——佢概括咗呢次角色轉變嘅全部內涵：

維度	傳統 Coder	AI 指揮官（Conductor）
核心技能	語法記憶、API 調用、調試	需求定義、上下文管理、驗收判斷
工作方式	逐行編寫代碼	描述意圖，審查產出
質量把控	自己寫嘅，自己心入面有數	透過測試同審查驗證 AI 產出
知識要求	深度掌握特定語言/框架	廣度理解技術全貌，深度理解業務邏輯
瓶頸所在	編碼速度同語法記憶	思考清晰度同表達能力
錯誤來源	語法錯誤、邏輯遺漏	需求含糊、上下文唔夠

傳統 Coder 同 AI 指揮官嘅能力維度對比

呢個絕對唔係話編程知識唔重要喇。啱啱相反：當你睇得明 AI 寫嘅代碼時，你先能夠有效咁審查同修正佢；當你理解數據結構同算法時，你先能夠判斷 AI 嘅方案係咪合理。知識儲備決定咗你嘅「指揮上限」——但知識嘅角色由「執行工具」變成「判斷依據」。

四、點解方法論比工具更重要

工具每三個月迭代一次，方法論每三年先進化一次。學識一件工具嘅操作，三個月之後可能已經過時；掌握一套思維方式，三年之後依然啱用。

好多人喺入門 AI 編程時，第一件事就係追問「邊個工具最好」「邊個模型最強」。呢啲問題唔係唔重要，但佢哋問錯咗優先級。2026 年 8 月嘅工具格局同 2025 年 8 月已經完全不同——Trae、Qoder 等工具興衰起伏，Cursor 由插件變成獨立 IDE 又進化到 Cursor 3，Claude Code 由實驗性項目變成終端編程嘅主流選擇。但喺呢劇烈變化背後，成功實踐者遵循嘅思維原則幾乎冇變。

呢個就係點解本書唔會將篇幅花喺教你某個工具嘅具體操作上——嗰啲內容寫完就會過時。相反，本書嘅全部重心放喺一套經過驗證嘅思維方法論上：點樣思考需求、點樣同 AI 溝通、點樣驗證產出、點樣持續迭代。掌握咗呢套方法論，你換任何工具都可以快速上手；只學咗工具操作，換一個工具就要由零開始。

畀零基礎讀者嘅定心丸

如果你係零基礎，唔好畀「方法論」三個字嚇親。方法論唔係抽象嘅哲學，佢就係一套「遇到問題應該點諗」嘅操作手冊。就好似學煮餸：你可以跟住菜譜一步步嚟（工具操作），但真正嘅大廚理解嘅係「點解要咁樣調味」「火候點樣控制」（方法論）。本書會由最基礎嘅概念講起，每個原則都配具體嘅對比案例，確保你唔單止理解「係乜嘢」，更能夠掌握「點樣做」。

嚟動手：畀而家嘅你影張「角色相」

攞出一張紙，寫低三個問題嘅答案：(1) 你而家係點樣用 AI 幫你寫代碼嘅（如果用過嘅話）？(2) 你更似上表中嘅「Coder」定係「指揮官」？(3) 你希望三個月後嘅自己會點樣描述自己嘅工作方式？寫完之後夾喺書入面，讀完第四十七章再返嚟睇——你會發現變化比想像中快。

本章小結

- 2025 年 Karpathy 用「Vibe Coding」命名咗 AI 編程革命：由逐行寫代碼，變成用自然語言描述意圖
- 2026 年進化到 Agentic Engineering：畀目標、放權、AI 自主規劃執行，人負責驗收
- Cursor 3 嘅 Agent 模式使用率首次反超 Tab 補全；Claude Code 年化收入達 25 億美元
- 你嘅新角色係 Conductor（指揮官）：定義戰場、制定規則、分配任務、驗收成果
- 工具每三個月迭代一次，方法論每三年先進化一次——學方法論，唔學操作

第三十八章：AI 編程五大思維原則

如果呢本書只可以讓你記住一章，咁就必須係呢一章。五大思維原則係全書嘅心臟，後面所有章節——上下文、需求、對話、質量——都係呢五條原則喺唔同場景下嘅展開。放心，每一條都唔難明，但每一條都值得你用幾個項目去「長入肌肉入面」。

故事嘅起點好有意思：Andrej Karpathy 喺開源佢嘅項目時，附咗一份僅 70 行嘅 CLAUDE.md 檔案，總結咗同 AI 協作嘅四條核心原則。呢份檔案喺 GitHub 上獲得咗超過 70,000 個 Star，成為全球 AI 編程實踐者最廣泛引用嘅準則。呢四條原則唔係空洞嘅口號，而係由大量實踐中提煉出嚟嘅、可操作嘅思維習慣。本書喺佢嘅基礎上補充咗第五條——迭代驗證——構成完整嘅五大原則體系。

70 行檔案點解咁值錢？

70,000 個 Star 係乜嘢概念？GitHub 上絕大多數項目一世都到唔到呢個數字。一個 70 行嘅文本檔案能夠獲得佢，恰恰證明咗一件事：AI 時代最稀缺嘅唔係代碼，而係「想清楚」嘅能力。呢 70 行，就係「想清楚」嘅濃縮版。

一、原則一：Think Before Coding（先思考，再編碼）

呢個係五條原則中最反直覺嘅一條。既然 AI 能夠幫我哋寫代碼，點解仲要「先思考」？答案好簡單：AI 嘅產出質量同你嘅輸入質量正相關。你自己都冇想清楚要做乜嘢，AI 畀出嘅答案只會讓你陷入無休止嘅「改咗又改」循環——俗稱「AI 陪你一齊蕩失路」。

「先思考」意味住喺畀 AI 落指令之前，你需要回答三個問題，佢哋構成一個缺一不可嘅「思考三角」：

1. 我要解決乜嘢問題？（目標）

2. 我期望嘅結果係點樣嘅？（驗收標準）

3. 有邊啲約束條件需要遵守？（邊界）

思考維度	模糊輸入（差）	清晰輸入（好）
問題定義	幫我做個網站	幫我做一個個人工具站，展示 3 個 AI 小工具嘅入口卡片
期望結果	好睇一啲	響應式佈局，暗色主題，卡片有懸停動畫，移動端可用
約束條件	（未說明）	唔使用框架，純 HTML+CSS+JS，單檔案部署，唔需要後端

同一需求嘅模糊輸入同清晰輸入對比

留意：先思考 ≠ 先寫完美文檔

「先思考」唔等於「寫一份完美嘅 PRD 再開始」。佢係一個快速但有條理嘅思考過程——花 30 秒想清楚上面三個問題，遠比花 30 分鐘喺對話中反覆修改嚟得高效。小任務三問可以好短；大項目可以將佢哋擴展成完整嘅需求文檔（詳見第四十章）。

二、原則二：Simplicity First（簡單至上）

AI 編程最大嘅陷阱唔係「做唔出嚟」，而係「做得太多」。當你對 AI 講「做一個用戶管理系統」，佢可能會生成認證、權限、角色、審計日誌、數據導出等一整套功能——但你而家可能只需要一個簡單嘅登入頁面。過度生成，係 AI 代碼膨脹嘅主要原因。

「簡單至上」要求你喺每次落指令時都遵循「最小可行」原則：用最少嘅代碼實現核心功能，用最簡單嘅方案解決問題，用最少嘅功能驗證想法。Karpathy 對此有個精闢嘅比喻：「如果你要計算一個折扣價格，你唔需要一個電商引擎。」

喺實踐中，「簡單至上」意味住幾個具體嘅操作習慣：

- 每次只畀 AI 實現一個功能點，而唔係一口氣描述成個系統；
- 優先揀標準庫同簡單方案，而唔係引入複雜嘅依賴；
- 可以用 50 行代碼解決嘅問題，唔好畀 AI 寫 500 行；
- 當 AI 嘅產出超出預期範圍時，勇敢咁話佢知「斬走多餘嘅部分」。

場景	過度方案（避免）	簡單方案（推薦）
需要儲存數據	搭建 MySQL 數據庫 + ORM	先用 JSON 檔案或者 SQLite
需要用戶登入	OAuth2.0 + JWT + 角色權限	先做簡單嘅密碼驗證
需要前端框架	React + Redux + Router	先試純 HTML + 原生 JS
需要部署上線	Docker + CI/CD + K8s	先喺本地跑通再講

常見場景嘅過度方案同簡單方案對比

三、原則三：Surgical Changes（精準修改）

當你喺迭代一個已有項目時，每次修改都應該好似外科手術一樣精準——只鬱該鬱嘅地方，唔好碰唔該碰嘅。呢一點喺 AI 編程中尤其重要，因為 AI 有「順手改一下」嘅傾向：你畀佢修一個 bug，佢可能順便重構咗半個檔案。

「精準修改」嘅核心要求係：每次畀 AI 嘅指令，都明確指定修改嘅範圍同邊界。話佢知「只修改 calculate_discount 函數」「唔好鬱其他部分」「保持現有嘅代碼風格」。當 AI 嘅修改超出咗你要求嘅範圍時，即刻要求佢回退多餘嘅部分。

Karpathy 用一個 diff（差異對比）嘅例子說明咗呢一點。假設你畀 AI 畀購物車嘅折扣計算函數添加「會員價」邏輯：好嘅產出係只修改呢一個函數、增加幾行代碼；差嘅產出係將成個檔案重構咗一次——雖然新代碼可能更「優雅」，但你唔能夠確定佢有冇引入新 bug。喺 AI 編程中，「睇起嚟更好」永遠唔能夠成為偏離指令嘅理由。

差嘅指令（範圍模糊）

```
幫我優化呢個項目嘅 折扣計算邏輯
```

好嘅指令（精準指定範圍）

```
只修改 cart.py 入面嘅 calculate\_discount 函數： 1. 喺現有折扣基礎上增加會員價邏輯 2.
    會員用戶額外享受 10\% 折扣 3. 唔好修改函數簽名同其他函數 4. 保持現有嘅變數命名風
    格
```

四、原則四：Goal-Driven Execution（目標驅動執行）

呢條原則回答咗一個關鍵問題：你應該畀 AI 落「點樣做」嘅指令，定係「做乜嘢」嘅指令？Karpathy 嘅答案係後者。與其話畀 AI 知「先創建一個類，然後添加方法，然後寫測試」，不如話佢知「我需要一個能夠解析 CSV 檔案並統計每列唯一值數量嘅工具」。等 AI 自己決定實現路徑，你負責驗收結果。

「目標驅動」嘅好處係雙重嘅。對 AI 來說，佢獲得咗更大嘅自主空間，可以揀最優嘅實現路徑，而唔係被你嘅中間步驟束縛；對你來說，你將注意力由「點樣做」轉移到咗「做出嚟啱唔啱」——呢個正正係指揮官該做嘅嘢。

留意：目標驅動 ≠ 放任不管

佢需要你喺 AI 執行後進行嚴格嘅驗收：結果係咪滿足需求？邊界情況係咪處理咗？有冇明顯嘅過度設計？如果結果唔符合預期，提供具體嘅反饋——唔係「重寫」，而係「呢度嘅問題喺於空列表輸入會報 IndexError，請修改為添加空值檢查」。

指令風格	例子	結果特點
命令式（點樣做）	「先創建一個 Python 類，叫 DataParser，加一個 parse 方法，接收 filename 參數，用 csv 模組讀取…」	AI 變成打字員，你嘅中間步驟可能唔係最優路徑
目標式（做乜嘢）	「我需要一個工具：讀取 CSV 檔案，統計每列嘅唯一值數量，輸出為 JSON 格式。請畀出完整實現。」	AI 自主揀最優路徑，你負責驗收結果

命令式指令同目標式指令嘅對比

五、原則五：Iterate and Verify（迭代驗證）

呢個係本書喺 Karpathy 四原則基礎上補充嘅第五條。佢嘅核心信念係：AI 嘅第一次產出幾乎永遠唔係最終版本。有效嘅 AI 編程係一個「生成-驗證-修正」嘅迭代循環，而唔係一次性嘅「指令-產出」。

「迭代驗證」唔係被動嘅「唔啱就再試」，而係一個有結構嘅驗證流程。每次 AI 畀出產出後，按三個層次進行驗證：

- 第一層・功能驗證：佢跑唔跑得鬱？輸入輸出啱唔啱？
- 第二層・邊界驗證：空輸入、超長輸入、異常輸入會點？
- 第三層・架構驗證：代碼結構係咪合理？有冇硬編碼？係咪方便後續擴展？

定義目標
AI 生成代碼
功能驗證
通過
邊界驗證
通過
架構審查
通過，進入下一步
不通過
提供具體反饋
重新生成

圖 2：AI 編程嘅迭代驗證循環

呢個循環嘅關鍵喺於「提供具體反饋」呢一步。當驗證唔通過時，唔好講「唔啱，重做」，而要講「函數喺輸入空列表時報 IndexError，請喺第 15 行添加空值檢查」。具體嘅反饋令 AI 能夠精準修正；模糊嘅反饋只會令佢重新猜測你嘅意圖。

六、Harness Engineering：Agent = Model + Harness

2026 年中期，一個叫 Harness Engineering 嘅概念開始喺 AI 編程社區流行。佢嘅核心公式好簡單：

Agent 嘅能力 = 模型能力 × Harness 質量

模型能力係 AI 大腦嘅智力上限；Harness 係圍繞模型搭建嘅「工作環境」——包括上下文管理、工具整合、流程編排、記憶系統等。

呢個概念解釋咗一個常見嘅困惑：點解同樣用 GPT-4 或者 Claude，有啲人能夠做出驚艷嘅項目，有啲人只能夠得到平庸嘅代碼？區別唔喺模型，而喺 Harness。一個好嘅 Harness 包括：清晰嘅項目規範檔案、合理嘅工具整合、有條理嘅對話上下文、完善嘅驗證流程。模型決定咗智力上限，Harness 決定咗你能夠幾大程度發揮呢個上限。

天才實習生比喻

你可以將模型想像成一個天才實習生——智商好高，但如果不畀佢項目背景文檔、唔話佢知用邊啲工具、唔畀佢設定期限同驗收標準，佢亦做唔出好嘢。Harness Engineering 就係「點樣畀天才實習生搭好工作環境」嘅工程。

本書嘅後續章節，本質上都喺度教你點樣構建一個好嘅 Harness：第三十九章教你管理上下文（項目記憶系統），第四十章教你定義需求（PRD），第四十三章教你對話技術（Prompt 分層），第四十五章教你質量保障（驗證流程）。呢啲組合起嚟，就係你嘅 Harness。

嚟動手：用「思考三角」拆解你嘅下一個想法

揀一個你一直想做但未動手嘅小嘢（例如一個倒數日網頁、一個飲水提醒腳本）。花 3 分鐘回答：(1) 佢解決乜嘢問題？(2) 做出嚟係點樣嘅？(3) 有乜嘢約束（時間、技術、環境）？然後將答案寫成一段話。恭喜你，你已經完成咗人生第一次「Think Before Coding」——呢段話就係一條合格嘅高質量指令。

本章小結

- 五大原則源自 Karpathy 70 行嘅 CLAUDE.md（70K+ Star），本書補充第五條「迭代驗證」
- Think Before Coding：開口前回答三問——做乜嘢？結果係點嘅？有乜嘢約束？
- Simplicity First：AI 最大嘅坑係「做得太多」——算折扣唔需要電商引擎
- Surgical Changes：好似外科手術一樣精準修改，拒絕 AI 嘅「順手改一下」
- Goal-Driven Execution：講「做乜嘢」，唔講「點樣做」，等 AI 自主揀路徑
- Iterate and Verify：三層驗證（功能/邊界/架構）+ 具體反饋，循環迭代

- Harness Engineering：能力 = 模型 × Harness，將 AI 當「天才實習生」嚟管理環境

第三十九章：上下文工程——令 AI 理解你嘅意圖

你有冇過呢種體驗：換個新對話視窗，*AI* 就好似失憶咗一樣，將你項目嘅基本情況全部唔記得？呢個唔係 *AI* 特意激你，而係佢天生「善忘」。呢一章教你畀 *AI* 裝上一套「長期記憶系統」，順便認識 *2026* 年最火嘅開發範式之一——規範驅動開發。

一、項目記憶檔案：AI 嘅「長期記憶」

AI 模型有一個根本性限制：佢嘅記憶係「短時」嘅。每次對話開始時，AI 對你項目嘅瞭解為零。你需要喺對話中不斷話佢知「我哋用嘅係 React」「後端係 Node.js」「數據庫係 PostgreSQL」「代碼風格用雙引號」。呢個唔單止低效，而且容易遺漏——就好似每日都要重新畀新嚟嘅同事做入職培訓。

解決方案係「項目記憶檔案」——一個放喺項目根目錄嘅純文本檔案，AI 工具會喺每次對話開始時自動讀取佢。唔同嘅工具有唔同嘅命名約定：

工具	檔案名
Cursor	`.cursorrules`
Claude Code	`CLAUDE.md`
GitHub Copilot	`copilot-instructions.md`
通用方案	`AGENTS.md`

各工具嘅項目記憶檔案命名約定

雖然命名唔同，但原理一樣：將你希望 AI「始終記住」嘅資訊寫喺呢個檔案入面。一個好嘅項目記憶檔案應該包含：項目概述（呢個係乜嘢項目）、技術棧（用咗乜嘢技術）、代碼規範（命名、縮排、註釋風格）、目錄結構（代碼點樣組織嘅）、開發約定（分支策略、提交規範、測試要求）。Karpathy 嘅 CLAUDE.md 只有 70 行，但覆蓋咗所有關鍵資訊——簡潔但完備。

```
# 項目：個人記賬應用
## 概述
呢個係一個全棧個人記賬應用，幫用戶記錄日常收支。
## 技術棧
- 前端：React + TypeScript + Tailwind CSS
```

```
- 後端：Node.js + Express + TypeScript
- 數據庫：SQLite（開發）/ PostgreSQL（生產）
- 測試：Vitest
## 代碼規範
- 用雙引號
- 縮進2個空格
- 函數名用camelCase
- 類型定義用PascalCase
- 每個函數必須有JSDoc註釋
## 目錄結構
/src
  /frontend  - 前端代碼
  /backend   - 後端代碼
  /shared    - 前後端共享嘅類型
  /tests     - 測試檔案
## 開發約定
- 提交資訊用中文
- 新功能必須先寫測試
- API返回統一用 { success, data, error } 格式
```

留意：唔好寫成「第二份需求文檔」

留意呢個檔案嘅長度——唔好寫成幾十頁嘅需求文檔。項目記憶檔案嘅目的係「令 AI 快速建立背景認知」，唔係「詳細描述每個功能」。保持簡潔、結構化、只寫「AI 需要知道嘅」。

二、規範驅動開發（SDD）

Spec-Driven Development（SDD，規範驅動開發）係 2026 年逐漸成熟嘅一種開發範式，核心理念一句話：先寫規範，再寫代碼。同傳統 TDD（測試驅動開發）唔同嘅係，SDD 嘅「規範」唔單止係測試用例，仲包括需求描述、接口定義、數據模型等。AI 根據呢啲規範嚟生成代碼，人類嘅職責係編寫同維護規範。

SDD 嘅工作流程五步走：

1. 用自然語言描述需求規範（寫喺一個.md 檔案中）；
2. 定義接口同數據結構（寫喺規範檔案中）；
3. 等 AI 根據規範生成代碼實現；
4. 運行測試驗證產出係咪符合規範；
5. 如果唔符合，修正規範或者等 AI 修正代碼。

喺呢個流程中，規範檔案係「唯一真理源」——代碼可以重新生成，但規範必須先正確。

編寫規範檔案

審查規範係否完備？

完備

AI 根據規範生成代碼

運行測試驗證產出

不完備 / 測試不過

代碼問題修正代碼；規範問題修正規範

回到驗證

全部通過交付

圖 3：規範驅動開發（SDD）嘅工作流程

SDD 嘅優勢喺於：規範檔案比代碼更穩定。當底層模型或者工具變化時，你嘅規範檔案依然有效，只需要畀新嘅 AI 工具重新生成代碼就可以。呢個亦係點解本書強調「方法論比工具重要」——規範檔案就係你方法論嘅具體載體。

三、上下文視窗管理

即使有咗項目記憶檔案，你仍然會喺對話中遇到「上下文唔夠用」嘅問題。AI 模型嘅上下文視窗（能夠同時處理嘅資訊量）係有限嘅。當對話太長、引用嘅檔案太多時，AI 會「唔記得」早期嘅資訊，導致產出質量下降——就好似你同一個朋友傾偈傾偈傾偈傾咗六個鐘，佢一早唔記得你開頭講嘅第一件事。

管理上下文視窗嘅核心策略係「分層引用」：

上下文層次	內容	管理方式
第一層：項目記憶	技術棧、規範、目錄結構	寫喺 CLAUDE.md 等檔案中，自動加載，唔使重複
第二層：檔案引用	當前任務涉及嘅代碼檔案	用 @ 符號精確引用，唔好貼全文到對話
第三層：對話上下文	當前任務嘅對話歷史	完成一個任務後開新對話，避免上下文膨脹
第四層：即時指令	當前呢一步嘅具體要求	清晰、簡潔、有針對性，避免冗長描述

上下文視窗嘅分層管理策略

四、上下文工程嘅實踐要點

上下文工程唔係一次性嘅設定，而係一個持續優化嘅過程。以下係幾個喺實踐中被反覆驗證嘅要點：

1. 記憶檔案隨項目演進同步更新。引入新技術或者改變架構時，第一時間更新記憶檔案，而唔係喺對話中臨時說明。

2. 善用 @ 引用而唔係全文貼上。大多數 AI 編程工具支援 @file 或者 @folder 語法，佢會喺後臺將檔案內容注入 AI 嘅上下文，比手動貼上更高效而且唔會污染對話。

3. 對話要「短小精悍」。一個對話只處理一個任務或者一個功能點。完成後開新對話處理下一個。長對話唔單止消耗上下文視窗，仲會令 AI「分心」。

4. 畀 AI「睇」結果而唔係「描述」結果。遇到 bug 時，將錯誤日誌直接 @ 引用畀 AI，而唔係用自己嘅話描述「好似有個乜嘢錯誤」。AI 睇到原始資訊比睇到你嘅轉述更有效。

5. 定期清理同歸檔。已完成嘅對話同臨時檔案會佔用工作空間。定期清理無關內容，保持項目目錄整潔，亦有助於 AI 更高效咁理解項目結構。

嚟動手：畀你嘅項目寫一份 70 行記憶檔案

揀一個你正在做（或者打算做）嘅項目，寫一份 CLAUDE.md：項目概述（2-3 句話）、技術棧（列出來）、代碼規範（3-5 條）、目錄結構（畫個樹）、開發約定（2-3 條）。控制喺 70 行以內——寫完你會發現自己對項目嘅理解都清晰咗好多。

本章小結

- AI 嘅記憶係「短時」嘅：每次新對話，佢對項目嘅瞭解為零
- 項目記憶檔案（CLAUDE.md 等）係 AI 嘅「長期記憶」，每次對話自動加載
- 好嘅記憶檔案 = 概述 + 技術棧 + 規範 + 目錄 + 約定，70 行夠曬
- SDD 規範驅動開發：先寫規範再寫代碼，規範係「唯一真理源」，比代碼更穩定
- 上下文分四層管理：項目記憶 @ 檔案引用對話歷史即時指令
- 五個實踐要點：同步更新、@ 引用、短對話、畀原始資訊、定期清理

第四十章：需求工程——將想法變成 AI 可執行嘅指令

如果話上下文工程解決嘅係「令 *AI* 瞭解背景」，咁需求工程解決嘅係「令 *AI* 知道今次具體要做乜嘢」。呢個係 *AI* 編程中最能夠體現人同人差距嘅環節——同樣嘅工具、同樣嘅模型，有人能夠做出令人驚艷嘅項目，有人只能夠做出平庸嘅玩具。區別往往唔喺技術能力，而喺需求表達能力。

一、由模糊想法到結構化需求

每個項目都始於一個模糊嘅想法——「我想做一個記帳 App」「我想做個個人網站」「我想做個待辦清單工具」。呢啲想法本身冇問題，但如果你直接將佢丟畀 AI，產出嘅質量完全取決於運氣。需求工程嘅目標，就係將呢個模糊嘅想法，變成一個 AI 能夠精確執行嘅結構化需求。

好消息係：結構化需求唔需要你寫一份幾十頁嘅 PRD。佢只需要你回答五個問題，我哋稱之為「需求五要素」：

要素	問題	示例：記帳 App
Who	畀邊個用？	個人使用，記錄日常收支
What	做乜嘢？	一個能夠記錄每筆收支、按月匯總、可視化嘅應用
Why	點解做？	想了解自己嘅消費結構，搵出不必要嘅支出
How	大致點樣做？	前端用 React，數據存瀏覽器本地，先唔做後端同登入
Constraint	有乜嘢限制？	單人開發，2 週內完成，唔需要上線部署

需求五要素及示例

呢五個要素唔需要寫成正式文檔，甚至喺腦海中快速過一次就足夠。關鍵係養成「先想清楚再開口」嘅習慣。你會發現，當你能夠清晰咁回答呢五個問題時，你嘅需求描述自然就變咗結構化。

二、需求拆解嘅分層藝術

對於較大嘅項目，一個需求描述係唔夠嘅。你需要將大需求拆解成細嘅、可獨立驗證嘅子需求。呢個過程叫做「需求拆解」，核心原則係：每個子需求都應該能夠獨立完成、獨立驗證、獨立交付。

以「做一個個人記帳 App」為例，一個合理嘅拆解方式係：

1. 實現基礎嘅收支記錄功能（能夠添加、查看、刪除記錄）；
2. 添加分類功能（畀每筆記錄打上餐飲、交通等標籤）；
3. 實現月度匯總功能（按月統計收支情況）；
4. 添加數據可視化（用圖表展示消費結構）。

每一步都係一個可以獨立運行同驗證嘅功能增量。

需求拆解嘅黃金法則：每一步做完之後，個項目都應該係一件" 用得嘅嘢"——就算功能好簡陋都好。千祈唔好令個項目停留喺" 乜嘢都做咗一半但乜嘢都用唔到" 嘅狀態。

呢種拆解方式有兩個好處。首先，佢令你始終能夠睇到進展——每完成一步，你嘅 App 就多咗一個功能，呢種正反饋對於長期項目至關重要（人都係需要成就感㗎嘛）。其次，佢令 AI 嘅每次任務都足夠細，降低咗「AI 理解偏差」嘅風險——任務越細，AI 嘅理解越準確，產出質量越高。

三、PRD 寫作模板同實踐

對於較大嘅項目（例如一個有前後端、多個功能模組嘅全棧應用），僅憑需求五要素可能唔夠。呢時你需要寫一份更正式嘅項目需求文檔（PRD）。一個容易誤解嘅點係：PRD 唔係寫畀 AI 睇嘅——佢係寫畀你自己嘅，幫你理清思路，然後你先能夠畀 AI 提供更精確嘅指令。

一份好嘅 PRD 唔需要好長，但應該包含六個部分：項目概述（一段話講清楚做乜嘢）、功能列表（每個功能點係乜嘢）、技術方案（用乜嘢技術棧、大致架構）、數據模型（核心數據結構）、介面描述（主要頁面係點樣嘅）、開發計劃（分幾步完成）。

```
# 個人記賬應用 PRD
## 項目概述
一個輕量級個人記賬應用，支援收支記錄、分類、月度匯總同數據可視化。
## 功能列表
1. 記錄收支：金額、類型(收入/支出)、分類、備註、日期
2. 分類管理：預設餐飲、交通、購物、娛樂、其他
3. 月度匯總：按月統計總收入、總支出、結餘
4. 消費可視化：餅圖展示各分類佔比
5. 數據匯出：匯出為CSV
## 技術方案
- 前端：React + TypeScript + Tailwind CSS
- 數據儲存：localStorage（無需後端）
```

```
- 圖表：使用Chart.js
- 構建：Vite
## 數據模型
Record {
  id: string
  amount: number
  type: 'income' | 'expense'
  category: string
  note: string
  date: string  // YYYY-MM-DD
}
## 開發計劃
Step 1: 基礎記錄功能（CRUD）
Step 2: 分類管理
Step 3: 月度匯總
Step 4: 數據可視化
Step 5: 數據匯出
```

留意呢份 PRD 嘅寫作風格：用簡潔嘅列表同結構化嘅格式，而唔係長篇大論。每個功能點都有明確嘅定義，數據模型用簡單嘅類型描述。呢種風格嘅 PRD 唔單止你自己睇得清楚，當你將佢提供畀 AI 時，AI 亦能夠精確理解每個需求點。

四、實戰：由想法到方案嘅完整推演

紙上談兵冇意思，我哋用一個完整嘅例子行一次需求工程嘅全過程。假設你嘅想法係：「我想做一個能夠追蹤我每日學咗乜嘢嘅學習追蹤器」。

第一步，過一次需求五要素。

- Who：個人使用。
- What：一個能夠記錄每日學習內容、追蹤學習時長、回顧學習進度嘅工具。
- Why：想養成持續學習嘅習慣，睇到自己嘅累積。
- How：先做一個網頁版，數據存喺瀏覽器本地。
- Constraint：零基礎開發，1 週內完成核心功能。

第二步，拆解需求。

1. Step 1：實現「記錄學習條目」功能（日期、內容、時長）；
2. Step 2：添加「學習時長統計」（今日、本週累計）；
3. Step 3：實現「學習日曆」視圖（好似 GitHub 貢獻圖咁樣展示）；
4. Step 4：添加「學習回顧」功能（隨機展示過去某日嘅學習記錄，幫助溫習）。

第三步，寫一份簡短 PRD。唔使好長——上面嘅拆解本身就構成咗 PRD 嘅主體。將佢整理成一個.md 檔案，放喺項目根目錄，作為你同 AI 嘅共同參考。

第四步，開始第一步嘅開發。畀 AI 嘅指令可以係：

```
我需要一個學習追蹤器嘅第一個功能——記錄學習條目。
用戶可以添加一條學習記錄，包含日期、學習內容、學習時長（分鐘）。
數據存在localStorage。
前端用React + TypeScript。
請畀出完整實現。
```

留意呢個指令嘅特點：佢只描述當前呢一步嘅目標（記錄功能），唔摻雜其他功能；佢明確咗技術約束（React、TypeScript、localStorage）；佢畀咗足夠嘅資訊令 AI 知道「做乜嘢」同「點樣做」，但具體實現由 AI 決定。呢個就係一個經過需求工程加工後嘅、高質量嘅 AI 編程指令。

嚟動手：畀你嘅「心頭好」寫一份五要素卡

揀一個你醞釀已久嘅小項目，花 5 分鐘填一張卡：Who / What / Why / How / Constraint，各一句話。然後將佢拆成 3-5 個「每一步都用得」嘅步驟。你會發現：將想法寫低之後，佢就由「白日夢」變成「待辦清單」。

本章小結

- 需求五要素：Who / What / Why / How / Constraint——想清楚再開口
- 需求拆解黃金法則：每一步完成後項目都係「用得嘅嘢」
- PRD 係寫畀自己嘅：概述、功能列表、技術方案、數據模型、介面、計劃
- 好指令 = 只描述當前一步 + 明確約束 + 講「做乜嘢」唔講「點樣做」
- 完整推演模板：五要素拆解 PRD 畀指令，四步走

第四十一章：2026 年 AI 編程工具全景

先打個預防針：本章係 *2026* 年 *8* 月工具生態嘅一張「快照」。半年之後佢就會過時——呢個恰恰係本章想教你嘅第一課。本章嘅目的唔係教你「用邊個工具」，而係幫你建立「點樣你建立「點樣揀工具」嘅判斷框架。當你理解咗唔同工具嘅設計哲學，你就能夠喺任何時候做出適合自己嘅選擇。

一、三大流派同選擇框架

2026 年嘅 AI 編程工具大致分為三大流派，每種流派背後係一種唔同嘅設計哲學。先睇呢張全景圖：

流派	代表工具	設計哲學	適合場景
IDE 內置 AI	Cursor 3, Trae, Windsurf, CodeBuddy	將 AI 能力深度整合到編輯器中	日常開發、代碼審查、漸進式增強
終端 Agent	Claude Code, Codex CLI, Aider	以終端為介面，AI 直接操作檔案系統同命令	項目級操作、批量修改、CI/CD 整合
雲端 Builder	v0, Bolt, Replit Agent	喺瀏覽器中由零創建完整項目	快速原型、零配置上手、即時預覽

AI 編程工具嘅三大流派

揀工具嘅關鍵唔係「邊個最好」，而係「邊個最適合你當前嘅場景」。三個實用嘅判斷維度：

- 你嘅編程基礎點樣？零基礎揀雲端 Builder，有基礎揀 IDE 內置 AI，進階用戶揀終端 Agent。
- 你嘅項目階段喺邊度？由零開始揀雲端 Builder，迭代開發揀 IDE 內置 AI，維護重構揀終端 Agent。
- 你嘅部署需求係乜嘢？快速驗證揀雲端 Builder，認真項目揀 IDE 內置 AI，自動化流程揀終端 Agent。

二、IDE 內置 AI 流派

IDE 內置 AI 係最主流嘅流派：將 AI 能力嵌入到你嘅代碼編輯器入面，喺保留傳統編程體驗嘅同時疊加 AI 增強。2026 年呢個流派嘅代表係 Cursor 3、Trae、Windsurf 同 CodeBuddy。

Cursor 3

Cursor 係目前全球使用最廣泛嘅 AI 編程 IDE。2026 年 4 月發佈嘅 Cursor 3 標誌住 AI 編程工具嘅一個重要轉折點：Agent 模式嘅使用率首次超過 Tab 補全模式——開發者由「AI 幫我補全代碼」轉向咗「AI 幫我完成任務」。Cursor 3 嘅核心特性包括：多檔案編輯（AI 可以同時修改多個檔案嚟完成任務）、上下文感知（自動理解項目結構同依賴關係）、同終端嘅深度整合（AI 可以喺編輯器內直接運行命令同查看輸出）。

Trae

Trae 係字節跳動推出嘅 AI 編程 IDE，喺國內市場有較大嘅用戶基礎。佢嘅 SOLO 模式（亦稱為 Builder 模式）係典型嘅「一句話創建項目」能力——你描述需求，AI 自主完成由項目初始化到代碼生成嘅全流程。Trae 嘅優勢喺於中文支援同國內網絡環境適配，適合國內用戶快速上手。

其他工具

Windsurf（原 Codeium）以流暢嘅代碼補全體驗著稱，適合追求「打字如飛」體驗嘅開發者。騰訊雲 CodeBuddy 作為 VS Code 插件，擅長同雲生態整合，適合計劃部署到騰訊雲或者開發微信小程式嘅項目。阿里通義靈碼同百度 Comate 亦提供咗類似嘅 IDE 整合體驗，各自喺中文場景優化上有特色。

三、終端 Agent 流派

終端 Agent 係 2026 年增長最快嘅流派。核心理念：AI 唔應該只係一個編輯器插件，佢應該能夠好似開發者一樣操作成個開發環境——讀寫檔案、運行命令、查看輸出、安裝依賴。Claude Code 係呢個流派嘅代表。

Claude Code

Claude Code 係 Anthropic 推出嘅終端 AI 編程工具，到 2026 年中期已達到年化收入 25 億美元嘅規模。佢嘅工作方式同傳統 IDE 完全唔同：你喺終端中用自然語言描述任務，Claude Code 會自主讀取相關檔案、修改代碼、運行測試、查看結果、迭代修正——成個過程唔需要你打開編輯器。

Claude Code 嘅優勢喺於「全項目感知」：佢唔受編輯器視窗限制，可以自由咁瀏覽成個項目目錄，理解檔案之間嘅依賴關係。佢亦支援 CLAUDE.md 項目記憶檔案，令 AI 喺每次會

話開始時就瞭解項目背景。對於大型項目嘅重構、批量修改、測試編寫等任務，終端 Agent 比 IDE 插件更高效。

Codex CLI 同 Aider

OpenAI 嘅 Codex CLI 提供咗類似嘅終端編程體驗，並支援 /import 功能——可以導入現有嘅代碼庫畀 AI 理解。Aider 係一個開源嘅終端 AI 編程工具，支援多種模型（GPT、Claude、DeepSeek 等），適合想要靈活切換模型嘅用戶。呢啲工具雖然各有特色，但核心理念一致：令 AI 喺終端中擁有完整嘅開發環境操作能力。

四、雲端 Builder 流派

雲端 Builder 係最適合零基礎用戶嘅流派。你唔需要安裝任何軟件，只需要打開瀏覽器，描述你嘅需求，AI 就會喺雲端創建完整嘅項目，並提供即時預覽。

- v0（Vercel 出品）：擅長生成前端 UI 組件同頁面，適合快速製作網頁原型。
- Bolt：支援由一句話創建全棧應用，包含前端、後端同數據庫。
- Replit Agent：喺 Replit 嘅雲端 IDE 中工作，唔單止能夠創建項目，仲能夠直接喺雲端運行同部署。

呢啲工具嘅共同優勢係「零配置上手」——打開瀏覽器就可以開始。侷限喺於：當項目變複雜之後，雲端編輯環境嘅靈活性不如本地 IDE。因此佢哋最適合「由 0 到 1」嘅原型階段：用雲端 Builder 快速驗證想法，驗證通過之後再遷移到本地 IDE 進行深度開發。

五、工具會變，方法論唔變

如果聽日所有 AI 編程工具都消失咗，你仍然應該保持嘅三個習慣：諗清楚先行動、每次只做一小步、做完就驗證。工具係放大器，方法論係地基。放大器可以換，地基唔可以鬱。

回顧 2024 到 2026 年嘅工具變遷：2024 年 GitHub Copilot 係王者，2025 年 Cursor 取代佢，2026 年 Claude Code 異軍突起。每一個新工具出現時，都有人宣稱「今次唔同」，但最終發現：用好新工具嘅人，恰恰係嗰啲方法論紮實嘅人。佢哋換工具只需要幾日適應期，而只學咗工具操作嘅人換一次就要重新學起。

因此，本書建議嘅學習路徑係：先用一個工具（任何一個都得）入門，喺實踐中體會五大思維原則；當你嘅方法論足夠紮實之後，再嘗試唔同流派嘅工具，感受佢哋嘅設計哲學差異；最終，你會形成自己嘅工具組合——例如用 Cursor 做日常開發，用 Claude Code 做重構同測試，用 v0 做快速原型。工具組合因人而異，但方法論係共通嘅。

嚟動手：畀自己配一套「兵器組合」

用三個判斷維度（編程基礎、項目階段、部署需求）畀自己定位：如果你係零基礎試下 Bolt 或者 v0 做一個頁面；有基礎用 Cursor 或者 Trae 做一個迭代任務；想挑戰喺終端入面跑一次 Claude Code。然後回答：邊種流派最符合你當前嘅階段？

本章小結

- 三大流派：IDE 內置 AI（日常開發）、終端 Agent（項目級操作）、雲端 Builder（零基礎原型）
- 揀工具睇三問：編程基礎？項目階段？部署需求？
- Cursor 3 嘅 Agent 模式首次反超 Tab 補全；Claude Code 年化收入 25 億美元
- 雲端 Builder 適合「由 0 到 1」，驗證之後遷返本地 IDE 深度開發
- 工具每三個月迭代一次：學方法論嘅人換工具只需要幾日適應期

第四十二章：大模型全景同 API 實戰

前面幾章講嘅都係「點樣同 *AI* 協作」，本章解決一個更基礎嘅問題：「用邊個 *AI*？點樣用？」*2026* 年 *8* 月嘅大模型格局已經極其豐富——國際有 *GPT*、*Claude*、*Gemini* 三強爭霸，國內有 *DeepSeek*、通義千問、*GLM*、*Kimi*、豆包百花齊放。每個模型都有自己嘅性格同特長：揀啱咗事半功倍，揀錯咗事倍功半。本章唔單止帶你認識呢啲模型，仲會教你點樣透過 *API* 將佢哋接入自己嘅應用，以及點樣根據自己嘅需求揀合適嘅 *Token Plan*（用量套餐）。

一、國際主流大模型：三強格局

2026 年 8 月嘅國際大模型市場，形成咗 OpenAI（GPT 系列）、Anthropic（Claude 系列）、Google（Gemini 系列）三強鼎立嘅格局。每家都有自己鮮明嘅特色——我哋一個一個認識。

OpenAI GPT-5.6 系列：全能均衡嘅六邊形戰士

OpenAI 係大模型浪潮嘅開啟者，GPT 系列至今仍然係全球使用最廣泛嘅模型。2026 年 7 月發佈嘅 GPT-5.6 系列採用三檔定位策略：旗艦款 GPT-5.6 Sol 面向高端推理需求，均衡款 GPT-5.6 Terra 面向日常開發，輕量款 GPT-5.6 Luna 面向高並發低成本場景。三款模型共享 105 萬 Token 嘅上下文視窗，都支援圖像輸入、函數調用同結構化輸出。

型號	定位	輸入 (/百萬 Token)	輸出 (/百萬 Token)	上下文視窗
GPT-5.6 Sol	旗艦推理	US$5.00（約 36 蚊）	US$30.00（約 217 蚊）	105 萬 Token
GPT-5.6 Terra	均衡日常	US$2.50（約 18 蚊）	US$15.00（約 109 蚊）	105 萬 Token
GPT-5.6 Luna	輕量低成本	US$1.00（約 7 蚊）	US$6.00（約 44 蚊）	105 萬 Token

OpenAI GPT-5.6 系列定價（2026 年 8 月）

GPT 系列嘅核心優勢喺於「均衡」——推理、編程、寫作、多模態，冇明顯短板。加上 OpenAI 龐大嘅插件生態同開發者社區，適配文檔同工具鏈最為完善。對於「唔知揀乜嘢」嘅場景，GPT 係「盲揀都唔會錯」嘅預設答案。但佢嘅短板係：國內訪問需要特殊網絡，中文表達偶爾有「翻譯腔」，API 價格處於中高水平。

好消息

OpenAI 喺 2026 年 8 月宣佈免費用戶預設模型升級為 GPT-5.6 Luna，並開放無限量文本聊天。呢個意味住你可以零成本體驗 GPT 嘅基礎能力，只有喺需要更強推理或者 API 調用時先需要付費。

Anthropic Claude 系列：編程同嚴謹嘅天花板

如果話 GPT 係全能選手，咁 Claude 就係編程領域嘅「天花板」。Anthropic 嘅 Claude 系列喺 SWE-bench（軟件工程基準測試）上長期斷層領先，代碼生成質量同工程級項目落地能力業界第一。更難得嘅係，Claude 嘅幻覺率（生成虛假資訊嘅概率）全網最低，僅約 1%——呢個令佢特別適合學術、法務等高精度場景。

Claude 嘅另一個殺手鐧係超長上下文：支援 100 萬 Token 嘅上下文視窗，能夠一次性「讀完」幾百頁嘅合約或者成個代碼庫唔漏細節。呢個喺代碼重構、長文檔分析等場景中幾乎無可替代。Claude Code 作為終端 AI 編程工具，到 2026 年中期年化收入已達 25 億美元——呢個係 Claude 編程能力最受認可嘅證據。

型號	定位	輸入 (/百萬 Token)	輸出 (/百萬 Token)	上下文視窗
Claude Fable 5	旗艦款	US$10.00（約 72 蚊）	US$50.00（約 362 蚊）	100 萬 Token
Claude Opus 4.8	專業款	US$5.00（約 36 蚊）	US$25.00（約 181 蚊）	100 萬 Token
Claude Sonnet 5	性價比款	US$2.00（約 14 蚊）	US$10.00（約 72 蚊）	100 萬 Token

Anthropic Claude 系列定價（2026 年 8 月，Sonnet 5 為優惠價）

Claude 嘅短板係價格——Fable 5 嘅輸出價格係所有模型中最高嘅，每百萬 Token 要 50 美元。多模態能力（圖像、視像理解）亦相對較弱。對於預算有限嘅個人開發者，Sonnet 5 喺優惠期內嘅性價比非常突出。

Google Gemini 系列：多模態嘅王者

Google 嘅 Gemini 系列喺 2026 年已迭代至 3.1 Pro 版本。Gemini 嘅最大殺手鐧係原生多模態——圖文解析精準、OCR 表格識別無誤、視像幀提取高效，仲支援音視像深度解析。佢同 Google 生態（搜尋、電郵、雲服務）深度綁定，喺即時資訊檢索同跨模態理解上表現突出。Gemini 仲支援 200 萬 Token 嘅超長上下文輸入，係目前上下文視窗最大嘅模型。

Gemini 嘅定位係「科學推理同多模態場景嘅首選」。如果你嘅項目涉及大量圖像分析、視像理解或者科學計算，Gemini 幾乎冇對手。但佢喺純文本代碼生成上略遜於 Claude，中文表達亦不如國產模型自然。

對比項目	GPT-5.6	Claude Fable 5	Gemini 3.1 Pro
綜合得分	94.8	95.0	92.1
編程能力 (SWE-bench)	74.9%	64.3% (Pro)	中等
上下文視窗	105 萬 Token	100 萬 Token	200 萬 Token
多模態能力	均衡	較弱	最強
幻覺率	較低	最低 (約 1%)	較低
國內可用性	需特殊網絡	需特殊網絡	需特殊網絡
API 生態	最完善	成長中	Google 生態綁定

國際三大模型橫向對比（2026 年 8 月）

留意：國內使用嘅現實門檻

對於國內開發者來說，GPT、Claude、Gemini 嘅核心障礙係網絡訪問。雖然可以透過 Azure 代理、API 聚合平臺等方式間接使用，但穩定性同合規性需要自行評估。如果你嘅項目對數據安全有嚴格要求，建議優先考慮國產模型。

二、國產主流大模型：百花齊放

2026 年嘅國產大模型已經由「追趕者」變成「並跑者」，喺某些領域甚至實現咗「領跑」。同 2023 年「套殼」爭議唔同，2026 年嘅國產旗艦模型喺邏輯推理、代碼生成、中文理解等方面已經同國際一線模型旗鼓相當，差距縮減到 5% 以內。更重要嘅係，國產模型喺中文場景、價格成本同合規部署上有不可替代嘅優勢。

DeepSeek V4：性價比之王

DeepSeek（深度求索）係國產大模型中現象級嘅存在。2025 年初，DeepSeek-V3 以「比 GPT 平十分之一但能力相當」嘅定位震撼市場，被業內稱為「通用 AI 大模型斬殺線」——比佢平嘅冇佢好用，比佢貴嘅冇佢量大管飽。到 2026 年 8 月，DeepSeek V4-Pro 正式版發佈：1.6 萬億參數 MoE 架構，激活約 49B，支援 100 萬無損上下文，代碼性價比被廣泛認為係全球天花板。

DeepSeek V4 喺 Agent 能力上實現咗大幅躍升——DeepSWE 基準由 12.8 跳至 62.7（+390%），NL2Repo 由 38.5 升至 61.5。API 新增咗 Responses API 同 Anthropic API 雙協議支援，已兼容 Codex 同 Claude Code 等主流編程工具。呢個意味住你可以用 DeepSeek 嘅 API 替代 Claude 嘅 API，喺 Claude Code 中使用——成本降低數倍，體驗幾乎一致。

型號	定位	輸入 (/百萬 Token)	輸出 (/百萬 Token)	上下文視窗
DeepSeek V4 Pro	旗艦推理	3 蚊（約 US$0.43）	6 蚊（約 US$0.87）	100 萬 Token
DeepSeek V4 Flash	輕量快速	1 蚊（約 US$0.14）	2 蚊（約 US$0.28）	100 萬 Token

DeepSeek V4 系列定價（2026 年 8 月，促銷價）

價格對比：同樣 100 萬 Token 嘅輸入 + 100 萬 Token 嘅輸出，DeepSeek V4 Flash 花費約 0.42 美元，GPT-5.6 Sol 花費約 35 美元，Claude Fable 5 花費約 60 美元。差距高達 80-140 倍。

避坑提醒：DeepSeek 要加價喇

DeepSeek 官方已於 2026 年 8 月 6 日公告「計劃近期大幅上調 API 定價」，促銷價可能唔會持續太久。部分媒體傳聞漲幅可能達 30 倍，截至 8 月 14 日官方尚未公佈最終調價方案。此外，DeepSeek V4 正式版引入咗峯谷定價機制——高峯時段調用價格直接翻倍。建議關注官方公告獲取最新定價。

通義千問 Qwen3.7 Max：阿里雲嘅企業級選擇

阿里雲嘅通義千問（Qwen）系列依託阿里雲生態，喺企業級部署上有天然優勢。2026 年 7 月嘅 Qwen3.7 Max 係當前旗艦：235B 參數稠密模型，256K 原生上下文，可擴展至 1M。Qwen 嘅核心優勢喺於：開源生態成熟（由 7B 到 235B 全系列開源）、中文場景理解精準、同阿里雲基礎設施深度整合。

型號	輸出價格 (/百萬 Token)	適用場景
qwen-turbo	0.6 蚊	極簡問答、文本分類、高並發
qwen-long	2.0 蚊	10M 超長上下文處理
qwen3.7-flash	7.2 蚊	通用對話、內容生成
qwen3.7-plus	12.0 蚊	多模態（圖片 + 視像）、中文創作
qwen3.7-max	36.0 蚊	旗艦中文能力、複雜任務

通義千問 Qwen 系列階梯定價（2026 年 8 月）

Qwen 嘅定價採用階梯策略——由極低成本嘅 qwen-turbo（輸出 0.6 蚊/百萬 Token）到旗艦 qwen3.7-max（輸出 36 蚊/百萬 Token），覆蓋由輕量問答到複雜推理嘅全場景需求。對於已經喺度用阿里雲嘅企業，Qwen 係最順手嘅整合選擇。

智譜 GLM-5.1：國產 SWE-bench 第一

智譜 AI 嘅 GLM 系列喺 2026 年表現亮眼。GLM-5.1 喺 SWE-bench 上攞到國產第一嘅成績：744B MoE 稠密混合架構，支援 100 萬上下文。佢嘅特點係中文語義理解精準，支援國內合規私有化部署——呢個對數據安全要求高嘅金融、政務場景非常重要。GLM-5.1 嘅定價為輸入 6 蚊、輸出 24 蚊每百萬 Token，處於國產模型中游水平。

Kimi K3：超長文本之王

月之暗面嘅 Kimi 系列以超長文本處理能力著稱。2026 年 7 月發佈嘅 Kimi K3 擁有 2.8 萬億參數規模，喺多項複雜任務評測中進入全球第一梯隊。Kimi 嘅核心場景係長代碼庫分析同

長文檔檢索增強——佢能夠處理成本書嘅文本或者成個項目嘅代碼，喺「大海撈針」式檢索任務中表現卓越。Kimi 嘅定價相對較高（輸出約 12-20 蚊/百萬 Token），但喺長文本場景嘅性價比無人能及。

豆包 Doubao：字節跳動嘅全能選手

字節跳動嘅豆包系列喺 2026 年表現穩定。Doubao Seed 2.1 Pro 係火山引擎旗艦模型，256K 上下文，喺實測中綜合發揮穩定，中文場景體驗突出。豆包嘅優勢喺於：註冊即送大量免費 Token，火山引擎生態整合完善，定價處於國內最低梯隊之一。對於想要快速體驗 AI 編程嘅初學者，豆包係零門檻嘅入門選擇。

對比項目	DeepSeek V4	Qwen3.7 Max	GLM-5.1	Kimi K3	豆包 2.1 Pro
綜合得分	87.5	89.7	90.5	88+	85+
編程能力	極強	強	國產 SWE 第一	強	中上
中文理解	優秀	優秀	精準	優秀	優秀
上下文視窗	100 萬	256K-1M	100 萬	超長	256K
輸出價格 (蚊/M)	6	36	24	12-20	30
開源	MIT 協議	全系列開源	部分開源	閉源	閉源
私有化部署	支援	支援	支援	唔支援	唔支援

國產五大模型橫向對比（2026 年 8 月）

一個重要嘅趨勢係：國產模型同 GPT/Claude 嘅「代差感」已經消失。喺邏輯推理同數學競賽上，國產旗艦同 GPT-5 嘅分差基本上喺個位數。喺系統化架構思維同複雜工程推理上，Claude 仍然領先——佢能夠畀出更有「靈氣」嘅架構方案，而國產模型有時顯得「教條」。但喺中文場景、成本控制同合規部署上，國產模型有壓倒性優勢。

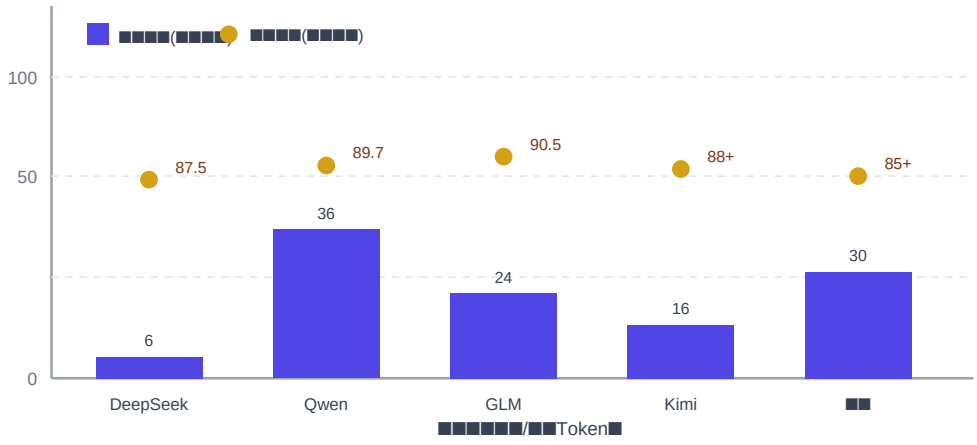


圖 4：五款模型嘅價格同性能對比（價格越低越好，得分越高越好）

三、模型選擇決策框架

瞭解咗各模型嘅特點之後，點樣為自己嘅項目揀模型？呢度送你一套「四問決策法」——回答四個問題，就能夠鎖定最適合你嘅模型。

你個項目係喺國內定係國際部署？

國內選國產模型

|

國際可選海外模型

核心任務係編程定係其他？

編程 Claude 或 DeepSeek

|

其他繼續判斷

需要超長上下文 (>50 萬 Token)？

需要 Gemini 或 Kimi

|

不需要繼續判斷

使唔使考慮預算？

敏感 DeepSeek 或 Qwen-turbo

|

不敏感 GPT-5.6 或 Claude

圖 5：大模型選擇四問決策法

小貼士：組合策略

呢個決策法唔係教條——好多場景需要組合使用多個模型。一個常見嘅組合策略：用 DeepSeek 或者 Qwen 做日常開發（成本低、中文好），用 Claude 做關鍵架構設計（推理強、幻覺低），用 Gemini 做多模態分析（圖像視像處理）。按需切換模型，而唔係「一個模型用到死」。

四、API 調用入門：由零開始接入大模型

好多 AI 編程教程只教你點樣用對話介面，但真正嘅「AI 編程」——令你嘅應用調用大模型——需要透過 API。API（Application Programming Interface，應用程序編程接口）係程序之間嘅通訊協議。透過 API，你嘅代碼可以好似調用一個函數一樣，調用大模型嘅能力。

Token 係乜嘢

喺瞭解 API 之前，必須先理解「Token」呢個概念。Token 係大模型計費嘅基本單位，可以理解為「AI 嘅閱讀同寫作單位」。一個 Token 大約對應：英文中 0.75 個單詞（4 個字符），中

文中 0.5-1 個漢字。一段 1000 字嘅中文文本，大約消耗 1500-2000 個 Token。

Token 唔係字數，亦唔係字符數。大模型使用專門嘅分詞器（Tokenizer）將文本切分成 Token。同一個詞喺唔同模型中嘅 Token 數量可能唔同——例如「人工智能」喺 GPT 中可能係 3 個 Token，喺 DeepSeek 中可能係 2 個。大多數模型嘅 API 會返回實際消耗嘅 Token 數量，你可以喺 API 響應中查看。

Token 成本快速估算

100 萬字中文約等於 150 萬 Token。用 DeepSeek V4 Flash 處理 100 萬字輸入 + 100 萬字輸出，成本約 0.42 美元（約 3 蚊人民幣）；用 Claude Fable 5 處理同樣嘅量，成本約 60 美元（約 434 蚊人民幣）。呢個就係點解模型選擇直接影響你嘅項目成本。

取得 API Key

使用大模型 API 嘅第一步係取得 API Key——一串用嚟做身份驗證嘅密鑰。每個模型供應商都有自己嘅 API 平臺，你需要註冊帳號並創建 API Key。以下係主流模型嘅 API 平臺入口同獲取流程：

模型	API 平臺	獲取流程	免費額度
DeepSeek	platform.deepseek.com	註冊實名認證創建 API Key	新用戶贈送額度
通義千問	bailian.aliyun.com（百煉平臺）	阿里雲帳號登入開通服務創建 Key	新用戶免費試用
智譜 GLM	open.bigmodel.cn	註冊實名認證創建 API Key	新用戶贈送額度
Kimi	platform.moonshot.cn	註冊實名認證創建 API Key	有限免費額度
豆包	volcengine.com（火山引擎）	註冊開通方舟創建 API Key	註冊送大量 Token
OpenAI GPT	platform.openai.com	註冊綁定信用卡創建 API Key	冇免費額度（要付費）
Anthropic Claude	console.anthropic.com	註冊綁卡創建 API Key	新用戶少量額度

主流模型 API 平臺同 Key 獲取方式（2026 年 8 月）

安全提示：API Key = 銀包密碼

API Key 等同於你嘅銀包密碼——任何拎到 Key 嘅人都可以用你嘅額度調用模型。因此：永遠唔好將 API Key 寫喺代碼中（用環境變數管理）；永遠唔好將含 Key 嘅代碼提交到公開嘅 Git 倉庫；永遠唔好喺社交媒體分享含 Key 嘅截圖。如果 Key 洩露咗，即刻喺平臺上刪除並重新創建。

第一次 API 調用

取得 API Key 之後，你可以用任何編程語言調用大模型 API。以下用 Python 演示一個最簡單嘅調用——向 DeepSeek 發送一條訊息並攞回覆。

```
# 安裝SDK：pip install openai
# DeepSeek兼容OpenAI SDK格式，可以直接用
from openai import OpenAI
import os

# 1. 建立客戶端（API Key從環境變數讀取）
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

# 2. 發送請求
response = client.chat.completions.create(
    model="deepseek-chat",  # 指定模型
    messages=[
        {"role": "system", "content": "你係一個Python編程助手。"},
        {"role": "user", "content": "寫一個快速排序函數。"}
    ]
)

# 3. 攞回覆
print(response.choices[0].message.content)

# 4. 睇Token消耗
print(f"輸入Token: {response.usage.prompt_tokens}")
print(f"輸出Token: {response.usage.completion_tokens}")
print(f"總Token: {response.usage.total_tokens}")
```

呢段代碼嘅核心邏輯係：創建用戶端（帶住 API Key 同模型地址）構造訊息列表（system 設定角色，user 係用戶輸入）發送請求攞回覆。呢個模式幾乎適用於所有兼容 OpenAI 格式嘅模型——只需要更換 api_key 同 base_url 就能夠切換模型。呢個就係點解好多國產模型揀兼容 OpenAI SDK 格式：開發者只需改兩行代碼就能夠切換模型，遷移成本極低。

以下係切換到其他模型嘅代碼差異——留意只需要改兩行：

```
# DeepSeek
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)
# model="deepseek-chat"

# 通義千問（百鍊平臺）
```

```
client = OpenAI(
    api_key=os.environ.get("DASHSCOPE_API_KEY"),
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1"
)
# model="qwen-plus"

# 智譜GLM
client = OpenAI(
    api_key=os.environ.get("GLM_API_KEY"),
    base_url="https://open.bigmodel.cn/api/paas/v4"
)
# model="glm-4"

# OpenAI GPT
client = OpenAI(
    api_key=os.environ.get("OPENAI_API_KEY"),
    # base_url使用預設值即可
)
# model="gpt-5.6-terra"
```

流式響應處理

喺實際應用中，你通常唔希望等 AI 寫完所有內容先至一次性顯示——咁樣會令用戶等太耐。流式響應（Streaming）可以令你喺 AI 生成嘅同時逐字顯示，大幅提升用戶體驗。就好似打字機一樣，字一個一個彈出嚟，用戶嘅感覺係完全唔同嘅。

```
# 串流回應：逐字輸出
stream = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "user", "content": "寫一個Python嘅冒泡排序教程"}
    ],
    stream=True  # 開啓流式響應
)

# 逐塊接收並打印
for chunk in stream:
    if chunk.choices[0].delta.content is not None:
        print(chunk.choices[0].delta.content, end="", flush=True)
# 輸出會好似打字機嗰逐字顯示
```

五、Token Plan 購買指南

大多數模型供應商有兩種計費方式：按量計費（Pay-as-you-go）同套餐訂閱（Token Plan / Subscription）。理解呢兩種方式嘅區別，能夠幫你大幅降低使用成本。

按量計費 vs 套餐訂閱

維度	按量計費 (Pay-as-you-go)	套餐訂閱 (Subscription)
計費方式	用幾多 Token 付幾多錢	預付固定費用，獲得額度
適合場景	使用量唔穩定、偶爾調用	使用量穩定、高頻調用
價格優勢	標準價	平過按量計 20%-50%
靈活性	隨時可用，唔使承諾	需要預付，額度過期可能作廢
超額處理	自動按量計費	超出額度後轉為按量計費
代表平臺	各個模型嘅官方 API 平臺	OpenAI Plus/Pro, Claude Pro, Cursor Pro

兩種計費方式嘅對比

對於初學者，建議由按量計費開始——唔需要預付費用，用幾多花幾多。當你嘅月使用量穩定喺一個水平之後（例如每月超過 50 蚊），再考慮切換到套餐訂閱以獲得更低單價。

主流訂閱套餐一覽

套餐	月費	包含內容	適合人羣
ChatGPT Free	免費	GPT-5.6 Luna 無限文本對話	零基礎體驗
ChatGPT Plus	US$20/月	GPT-5.6 Sol、圖像生成、數據分析、插件訪問	中度用戶
ChatGPT Pro	US$200/月	無限制使用所有模型、優先訪問新功能	重度專業用戶
Claude Free	免費	有限次數 Claude 對話	零基礎體驗
Claude Pro	US$20/月	無限制 Claude 使用、優先訪問新模型	編程開發者
Cursor Free	免費	有限次數 AI 補全同對話	初學者體驗
Cursor Pro	US$20/月	無限 AI 補全、500 次/月進階請求	日常開發者
Cursor Business	US$40/月/人	團隊協作、管理後臺、私隱模式	團隊使用

主流 AI 工具訂閱套餐（2026 年 8 月）

國內模型嘅訂閱方案略有唔同。DeepSeek 主要按量計費，冇月度訂閱套餐——你充值後按實際使用量扣費。通義千問喺百煉平臺提供預付費資源包（購買一定量嘅 Token 額度，享受折扣價）。豆包喺火山引擎上亦有類似嘅預付費方案。對於國內開發者，最經濟嘅組合通常係：DeepSeek 按量計費（日常開發）+ 豆包免費額度（體驗測試）。

API 聚合平臺

如果你需要同時使用多個模型，API 聚合平臺係一個慳心嘅選擇。聚合平臺提供一個統一嘅 API Key，你就能夠調用全球幾十種模型，唔使喺每個平臺分別註冊。

平臺	特點	價格優勢	適合場景
OpenRouter (openrouter.ai)	海外聚合平臺，一個 Key 調用全球模型	價格透明，支援即時比價	需要靈活切換多個海外模型
SiliconFlow (siliconflow.cn)	國內聚合平臺，提供多種國產模型 API	平過官方 20%-30%	國內開發者，多模型比價
Azure OpenAI（微軟代理）	微軟 Azure 代理嘅 OpenAI API	Batch API 可以打五折	企業級 OpenAI 調用，大批量非同步任務
百度千帆（qianfan.baidubce.com）	百度 AI 雲平臺，提供文心等模型	新用戶送額度	百度生態用戶

主流 API 聚合平臺（2026 年 8 月）

避坑提醒：唔好買「共享 Key」

唔好喺淘寶、閒魚等平臺買啲所謂嘅「共享 API Key」——呢啲 Key 通常來路不明，隨時可能被封號，仲有資料外洩嘅風險。記住一定要經官方平臺或者正規聚合平臺攞 API Key。

成本優化策略

揀啱模型只係第一步，點樣「用得慳錢」同樣重要。以下係幾個經過實踐驗證嘅成本優化策略：

1. 善用快取命中。好多模型都有「快取」功能——當你嘅請求包含之前處理過嘅內容（例如系統提示詞），呢部分只會用超低價錢計費。DeepSeek 嘅快取命中價錢只係普通價錢嘅 1/60。將固定嘅系統提示放喺訊息列表開頭，就可以令快取命中最大化。

2. 分級路由——用平價模型處理簡單任務，貴價模型處理複雜任務。例如用 DeepSeek V4 Flash 做文本分類（輸出 2 元/百萬 Token），用 Claude 做架構設計（輸出 181 元/百萬 Token）。一個項目入面 80% 嘅呼叫都可以用平價模型搞掂。

3. 限制輸出長度。API 係按輸出 Token 計費，輸出越長越貴。喺唔需要長篇大論嘅場景，設定 max_tokens 參數去限制輸出長度，免得 AI「廢話太多」。

4. 用 Batch API 處理非實時任務。如果你有大量唔需要即時回應嘅任務（例如批量數據標註、檔案摘要），用 Batch API 可以平一半。OpenAI 同 Azure 都有呢個功能。

5. 監察 Token 用量。大部分 API 平臺都有用量統計面板。定期睇嚇用量分佈，搵出消耗 Token 最多嘅呼叫，再針對性咁優化。小小一個 Prompt 調整（例如將「請詳細解釋」改成「請用 3 句話解釋」）隨時可以慳返 50% 嘅 Token。

優化策略	慳到幾多	實施難度	說明
快取命中	最高 98%	低	將固定內容放喺訊息開頭
分級路由	50%-80%	中	簡單任務用平價模型
限制輸出長度	30%-60%	低	設定 max_tokens 參數
Batch API	50%	低	非實時任務用批處理
Prompt 優化	20%-50%	中	精簡 Prompt，減少冗餘

成本優化策略及效果（2026 年 8 月）

六、實戰：喺項目入面接入大模型 API

等我哋用一個完整嘅實戰嚟完結呢一章——整一個「AI 摘要產生器」：用戶輸入一段長文本，呼叫大模型 API 產生摘要。呢個實戰會展示 API 呼叫嘅完整流程：環境配置、API 呼叫、錯誤處理、成本控制。

```
# ai_summarizer.py - AI摘要生成器
import os
from openai import OpenAI

# 1. 配置（API Key從環境變數讀取，絕不硬編碼）
client = OpenAI(
    api_key=os.environ.get("DEEPSEEK_API_KEY"),
    base_url="https://api.deepseek.com"
)

def summarize(text: str, max_length: int = 200) -> dict:
    """生成文本摘要

    Args:
        text: 要摘要嘅原文
        max_length: 摘要最大長度（Token數）

    Returns:
        dict: 包含摘要同Token消耗資訊
    """
    try:
        response = client.chat.completions.create(
            model="deepseek-chat",
            messages=[
                {
                    "role": "system",
                    "content": "你係一個摘要生成器。"
```

```
                                "請用簡潔嘅中文總結用戶提供嘅文本。"
                                "摘要不超過3句話。"
                },
                {
                    "role": "user",
                    "content": f"請摘要以下文本:\n\n{text}"
                }
            ],
            max_tokens=max_length,  # 限制輸出長度
            temperature=0.3  # 低溫度=更穩定
        )

        return {
            "success": True,
            "summary": response.choices[0].message.content,
            "input_tokens": response.usage.prompt_tokens,
            "output_tokens": response.usage.completion_tokens,
            "total_tokens": response.usage.total_tokens
        }

    except Exception as e:
        return {
            "success": False,
            "error": str(e)
        }

# 2. 使用示例
if __name__ == "__main__":
    long_text = """
    （粘貼一段長文本...）
    """

    result = summarize(long_text)

    if result["success"]:
        print(f"摘要: {result['summary']}")
        print(f"消耗Token: {result['total_tokens']}")
        # 估算成本：DeepSeek V4 Flash
        cost = result['total_tokens'] * 0.000004  # 約0.4分/千Token
        print(f"預估成本: {cost:.4f}元")
    else:
        print(f"出錯咗: {result['error']}")
```

呢段代碼體現咗 API 呼叫嘅幾個最佳實踐：API Key 係透過環境變數讀取，絕對唔會寫死喺代碼入面；函數有清晰嘅文檔同類型提示；設定咗 max_tokens 去限制輸出長度；temperature 設做低值（0.3）等輸出更穩定；有完整嘅錯誤處理；仲會傳回 Token 消耗資訊用嚟追蹤成本。

當你用 AI 嚟寫呢段代碼嗰陣，可以話畀佢知呢啲要求——佢完全有能力幫你實現。

本章嘅最終啟示：模型係" 彈藥"，方法論係" 戰術"。揀啱彈藥可以令戰鬥順啲，但冇好戰術，幾勁嘅彈藥都打唔贏仗。瞭解到各模型嘅特點同 API 用法之後，你就有一個充足嘅" 彈藥庫"——但係點樣打呢場仗，仲係要靠前幾章學到嘅思維方法論。

嚟動手：搞掂你嘅第一次 API 呼叫

註冊一個國產模型平臺（推薦 DeepSeek 或者智譜，兩家都送新用戶額度），整一個 API Key，將佢寫入系統環境變數（唔好寫入代碼入面！），然後行嚇上面「第一次 API 呼叫」嘅代碼。睇到 AI 嘅回覆打印出嚟嗰一刻，你就跨過咗「用 AI」同「整 AI 應用」之間嘅分界線。

本章小結

- 國際三強：GPT-5.6（均衡全能）、Claude（編程天花板 + 幻覺率只係 1%）、Gemini（多模態之王 +200 萬上下文）
- 國產五強：DeepSeek V4（性價比之王，留意加價公告）、Qwen3.7（企業級）、GLM-5.1（SWE 國產第一）、Kimi K3（超長文本）、豆包（免費入門）
- 四問決策法：部署喺邊？編程定係其他？使唔使超長上下文？預算敏唔敏感？
- Token 係計費單位：中文 1 個字大約 1.5-2 Token；100 萬字大約 150 萬 Token
- API Key = 銀包密碼：用環境變數管理，絕對唔好寫死、唔好擺上 Git
- 國產模型普遍都兼容 OpenAI SDK：轉模型只需改 api_key 同 base_url 兩行
- 慳錢五招：快取命中、分級路由、限制輸出、Batch API、監察用量

第四十三章：對話工程——同 AI 高效協作嘅對話技術

對話係 AI 編程嘅核心互動方式。但係留意：喺呢度「對話」唔等於「傾偈」——高效嘅 AI 編程對話係一種有結構嘅溝通技術，我哋叫佢做「對話工程」。佢研究嘅係：點樣用最少嘅對話輪次，攞到最高質素嘅代碼產出。講白啲，呢一章就係教你點樣同 AI「講清楚啲嘢」。

一、Prompt 分層策略

同 AI 嘅對話可以分做三個層次，每層負責唔同嘅功能。明白呢三個層次，你就可以好似砌積木咁組合出高效嘅 Prompt。

層次	功能	示例	頻率
系統層	設定 AI 嘅角色同全局約束	「你係一個 React 專家，代碼必須用 TypeScript」	項目開始時設定一次
任務層	描述而家要做嘅具體任務	「整一個表單組件，包含用戶名同密碼欄位」	每個任務開始時畀出嚟
回饋層	對 AI 嘅產出作出評價同修正	「密碼欄位缺少顯示/隱藏切換，請加上」	根據產出質素畀出嚟

對話 Prompt 嘅三個層次

系統層嘅 Prompt 通常寫喺項目記憶檔案入面（例如 CLAUDE.md），唔需要每次對話都重複。任務層係你對話時最常用嘅——佢應該包含「做乜嘢」「用乜嘢技術」「有乜嘢約束」三個要素。回饋層就最考功夫——好嘅回饋可以令 AI 精準修正，衰嘅回饋只會令佢重新估過。

小貼士：回饋三板斧

一個實用嘅回饋模板係「指出問題 + 講明期望 + 畀上下文」。例如：「第 25 行嘅錯誤處理只係用咗 console.log（指出問題），生產環境應該用統一嘅錯誤處理器，並且傳回用戶友好提示（講明期望），項目入面有一個 handleError 函數可以用（畀上下文）」。呢種回饋比

「改嚇錯誤處理啦」有效十倍。

二、除錯對話：點樣描述錯誤等 AI 快速定位

除錯係 AI 編程入面最常見嘅場景之一。當你撞到 bug 嗰陣，點樣向 AI 描述問題，直接決定咗佢能否快速幫你定位同修復。以下係除錯對話嘅四個要點：

1. 畀 AI「睇」原始資訊，而唔係你自己嘅轉述。將錯誤日誌、異常堆疊、相關代碼直接 @ 引用畀 AI，而唔係用「好似有個乜嘢報錯」嚟描述。AI 睇到原始資訊會比你嘅理解更準確。

2. 描述「期望行為」同「實際行為」嘅分別。「我期望撳咗按鈕之後會跳去首頁（期望），但實際撳完個頁面都冇反應（實際）」——呢種描述比「個按鈕壞咗」有用得多。

3. 提供重現步驟。「先輸入用戶名，然後撳登入按鈕，就會出現呢個錯誤」——重現步驟等 AI 可以模擬你嘅操作路徑，更容易定位問題。

4. 講清楚你已經試過乜嘢。「我已經檢查咗網絡請求，回傳 200，數據正常」——咁樣可以幫 AI 排除已經驗證過嘅方向，唔使嗰時間重複排查。

差嘅除錯描述

```
撳個按鈕冇反應，幫我睇嚇
```

好嘅除錯描述

```
撳登入按鈕之後個頁面冇反應。 期望行為：撳完之後跳去 /home 頁面 實際行為：撳完完全冇反應，控制檯冇報錯 重現步驟： 1. 喺登入頁輸入用戶名admin同密碼123456 2. 撳「登入」按鈕 3. 個頁面冇反應 已排查： - 網絡請求正常，回傳200 - onClick事件已綁定 - 檢查過LoginButton組件嘅代碼 相關檔案：@src/components/LoginButton.tsx @src/pages/Login.tsx
```

三、迭代策略：小步快跑 vs 大步重構

AI 編程入面嘅迭代有兩種基本策略，適用於唔同場景。明白佢哋嘅分別，可以幫你喺唔同情況下做出正確選擇。

「小步快跑」策略適用於功能開發同 bug 修復。佢嘅核心係：每次叫 AI 只做一個小修改，驗證通過先再做下一個。優勢係每一步都可以驗證、可以回退，風險可控；劣勢係速度慢啲，需要頻密嘅對話互動。適合：新功能開發、唔肯定邊個方案最好時嘅探索、對 AI 產出唔係好放心時嘅謹慎迭代。

「大步重構」策略適用於清楚知道目標嘅大規模修改。佢嘅核心係：一次過描述完整嘅目標，畀 AI 自主規劃同執行成個修改。優勢係速度快，AI 睇到全局圖景，可以做出更好嘅架構決策；劣勢係風險高，如果 AI 理解有偏差，修正成本好大。適合：明確嘅架構升級、大規模代碼遷移、有充分測試保障嘅重構。

維度	小步快跑	大步重構
每次修改量	一個功能點或者一個 bug	一個模組或者一個架構層
驗證頻率	每次修改完即刻驗證	全部修改完先統一驗證
對話輪次	多（5-10 輪/功能）	少（1-3 輪/模組）
風險	低，每步可以回退	高，糾偏成本大
適用場景	新功能、探索性開發	目標明確嘅重構、遷移
前提條件	冇特別要求	需要有測試覆蓋

兩種迭代策略嘅對比

實踐入面，兩種策略成日會夾埋用。例如重構一個模組嗰陣，先用「大步重構」叫 AI 畀個整體方案出嚟，審查完方案，再用「小步快跑」逐步執行。咁樣既保證咗全局視角，又控制咗執行風險。

四、多輪對話嘅上下文管理

隨住對話輪次增加，你需要主動管理對話上下文，避免「AI 忘記咗早期資訊」嘅問題。以下係幾個實用嘅上下文管理技巧。

第一，「錨定」關鍵資訊。對話開始嗰陣，將關鍵約束寫清楚，並且喺之後嘅對話中適時重申。例如「記住，我哋嘅數據存喺 localStorage，唔可以用數據庫」——喺相關嘅修改請求入面重申呢個約束，防止 AI 唔記得。

第二，善用「總結-續接」技巧。當對話變得好長嗰陣，可以叫 AI「總結嚇而家嘅進展同待辦事項」，然後開一個新對話，將呢個總結做起點。咁樣既保留咗關鍵上下文，又避免咗長對話嘅退化。

第三，「一個對話一個任務」。呢個係最簡單亦都最有效嘅上下文管理策略。每個對話只處理一個任務或者一個功能點，做完就開新對話處理下一個。咁樣自然限制咗對話長度，保持上下文清晰。

第四，善用工具嘅上下文管理功能。大部分 AI 編程工具都有「清除上下文」或者「新會話」掣——當你覺得 AI 嘅回答質素下降嗰陣，唔使猶豫，開一個新會話啦。唔見咗啲對話歷史，好過繼續喺退化咗嘅上下文入面掙紮好多。

嚟動手：做一次你 AI 嘅「病人」

特登喺瀏覽器入面整一個小頁面，製造一個 bug（例如個掣冇反應）。然後跟本章「好嘅除錯描述」嘅格式，向 AI 完整描述一次：原始資訊、期望 vs 實際、重現步驟、已排查項。感受嚇 AI 定位問題嘅速度分別——你就會明白點解「描述問題」本身就係一門手藝。

本章小結

- Prompt 三層：系統層（寫喺記憶檔案）、任務層（做乜嘢 + 技術 + 約束）、回饋層（指出問題 + 期望 + 上下文）
- 除錯四要點：畀原始資訊、期望 vs 實際、重現步驟、已排查項

- 小步快跑：低風險高輪次，適合新功能；大步重構：高風險低輪次，需要測試保障
- 上下文管理四招：錨定關鍵資訊、總結-續接、一個對話一個任務、果斷開新會話

第四十四章：從 0 到 1 實戰——完整項目全流程

理論學完喇，而家係動手嘅時候。本章唔係簡單嘅「跟住打」教程，而係一套「點諗、點問、點驗」嘅實戰方法論。我哋會用三個唔同難度嘅項目，展示由需求到成品嘅完整思考過程。無論你用邊個工具，呢套流程都啱用。

一、項目一：極簡個人工具站（入門級）

需求思考

目標：整一個個人工具站網頁，展示 3-4 個實用小工具嘅入口。呢個係一個純前端項目，唔需要後端同數據庫。重點唔係工具本身有幾複雜，而係體驗「由需求到成品」嘅完整流程。

跟住需求五要素：Who——個人使用。What——一個展示工具入口嘅網頁。Why——將常用工具集中喺一個頁面方便存取。How——單檔案 HTML+CSS+JS，唔用框架。Constraint——30 分鐘內完成，唔使部署。

下達指令

根據 Think Before Coding 原則，你嘅指令應該包含足夠嘅資訊畀 AI 明白目標，但唔使指定每個實現細節：

```
請整一個個人工具站網頁，要求：
1. 單檔案HTML（內聯CSS同JS），唔用框架
2. 暗色主題，現代簡約風格
3. 展示4個工具入口卡片：
   - 文字摘要工具
   - 字數統計工具
   - 顏色選擇器
   - 二維碼生成器
4. 每個卡片有圖標、標題、簡短描述
5. 卡片有懸停動畫效果
6. 響應式佈局，手機上用得
7. 頂部有一個標題欄
請直接畀出完整嘅HTML檔案。
```

驗收同迭代

AI 畀咗代碼之後，跟迭代驗證嘅三層檢查嚟驗收。第一層功能驗證：檔案能否喺瀏覽器入面打開？4 張卡係咪全部都顯示咗？懸停動畫正唔正常？第二層邊界驗證：喺手機螢幕尺寸下佈局正唔正常？窗口好窄嗰陣啲卡會唔會換行？第三層架構驗證：代碼可唔可以讀得明？CSS 係咪內聯喺一個 style 標籤入面？有冇明顯嘅冗餘？

如果某個工具卡嘅功能都想實現埋（而唔係齋入口），可以逐步添加——每次添加一個工具嘅功能，驗證完先添加下一個。呢個就係「小步快跑」策略嘅實際應用。

二、項目二：全端記賬系統（進階級）

需求思考同 PRD

目標：一個有前端、後端同數據庫嘅全端記賬應用。呢個項目比第一個複雜好多，需要用到第四十章嘅需求工程方法。

先寫 PRD。跟第四十章第三節嘅模板，寫出項目概述、功能列表、技術方案、數據模型同開發計劃。揀技術方案要考慮 Simplicity First 原則——前端用 React 但唔引入 Redux 等複雜狀態管理，後端用 Node.js 但唔引入微服務，數據庫用 SQLite 而唔係 PostgreSQL（開發階段）。

開發計劃拆解做 5 步：Step 1 後端 API 骨架（Express 伺服器 + SQLite + CRUD 接口）；Step 2 前端基礎頁面（記錄列表 + 添加表單）；Step 3 前後端對接（API 呼叫 + 數據展示）；Step 4 分類同篩選功能；Step 5 月度匯總同簡單圖表。每步做完都係「用得嘅嘢」。

設定項目記憶檔案

喺開始寫代碼之前，先整項目記憶檔案。呢個係上下文工程嘅核心實踐——畀 AI 每次對話開始嗰陣都瞭解項目背景。

```
# 項目：全棧記賬應用
## 概述
個人記賬應用，支援收支記錄、分類、月度匯總。
## 技術棧
- 前端：React + TypeScript + Vite
- 後端：Node.js + Express + TypeScript
- 數據庫：SQLite（通過 better-sqlite3）
- 前後端共享類型定義
## API約定
- 基礎路徑：/api
- 返回格式：{ success: boolean, data: any, error: string }
- RESTful風格
## 數據模型
Record { id, amount, type, category, note, date }
## 代碼規範
- TypeScript嚴格模式
- 縮進2空格
```

```
- 函數有JSDoc註釋
- 中文註釋同提交資訊
```

逐步實現

跟住開發計劃，每一步用一個獨立嘅對話嚟完成。以下係第一步嘅指令例子：

```
我需要整一個後端API骨架。
要求：
1. Express伺服器，端口3001
2. SQLite數據庫，表名records
3. records表字段：
   id (TEXT PRIMARY KEY)
   amount (REAL)
   type (TEXT: 'income'|'expense')
   category (TEXT)
   note (TEXT)
   date (TEXT: YYYY-MM-DD)
4. CRUD接口：
   GET /api/records - 攞曬所有記錄
   POST /api/records - 新增記錄
   PUT /api/records/:id - 更新記錄
   DELETE /api/records/:id - 刪除記錄
5. 錯誤處理中間件
請畀出完整嘅後端代碼結構。
```

留意呢個指令嘅特點：佢只描述而家呢一步（後端骨架），冇溝埋前端需求；佢明確列出咗欄位同接口（畀足約束 AI），但具體實現由 AI 決定；佢跟足 Goal-Driven Execution 原則——講「做乜嘢」而唔係「點樣做」。

AI 畀咗代碼之後，即刻驗證：行嚇伺服器，用 curl 或者 Postman 測試每個接口係咪正常運作。確認冇錯之後，開新對話處理 Step 2（前端基礎頁面）。每一步都跟「指令-驗證-新對話」嘅循環。

三、項目三：AI 增強應用（高級）

項目構思

目標：一個接入大模型 API 嘅應用——例如一個「AI 學習助手」，用戶輸入學習內容，AI 產生摘要、關聯知識點、推薦學習資源。呢個項目嘅特別之處喺於：你嘅應用本身就喺呼叫 AI，你需要處理 API 呼叫、prompt 工程、錯誤處理等新問題。

呢個項目教你最重要嘅一課係：當你「用 AI 嚟寫呼叫 AI 嘅代碼」嗰陣，你需要份外留意 prompt 工程。你嘅應用呼叫大模型 API 時用嘅 prompt，都要跟本指南討論嘅原則——清晰、簡潔、目標驅動。

關鍵技術點

呢個項目涉及幾個你喺頭兩個項目入面未遇過嘅技術點：

- API 金鑰管理：金鑰絕對唔可以寫死喺代碼入面，要用環境變數管理；
- 串流回應處理：大模型嘅回覆通常係串流式嘅，需要逐個字咁顯示；
- Token 用量控制：大模型按 token 計費，需要喺應用層面控制用量；
- Prompt 模板管理：唔同功能用唔同嘅 prompt 模板，需要結構化管理。

呢啲技術點都可以叫 AI 幫你實現，但你需要明白每個技術點嘅「點解」——點解金鑰唔可以寫死？點解需要串流回應？點解 token 用量需要控制？明白咗「點解」，你先至可以判斷 AI 嘅方案合唔合理。

四、通用實戰流程清單

無論項目大細，以下係一個通用嘅 AI 編程實戰流程清單。每次開新項目嗰陣，行一遍呢個清單，確保冇漏咗關鍵步驟。

階段	做乜嘢	對應原則
1. 需求思考	行一遍需求五要素，諗清楚要做乜嘢	Think Before Coding
2. 需求拆解	將大需求拆做可以獨立驗證嘅小步驟	Simplicity First
3. 寫 PRD	整理成結構化文檔，用嚟做參考	Goal-Driven
4. 設定記憶檔案	整 CLAUDE.md 或者同類檔案	Harness Engineering
5. 逐步實現	每步一個對話，指令-驗證-下一步	Surgical Changes
6. 測試驗證	每步做完即刻測試功能正確性	Iterate & Verify
7. 代碼審查	檢查架構合理性、安全性、可維護性	Iterate & Verify
8. 迭代優化	根據測試同審查結果迭代改進	Iterate & Verify

嚟動手：今個星期完成「項目一」

將第四十四章第一節嘅「個人工具站」當係你第一份 AI 編程功課：花 5 分鐘行五要素，將指令發畀 AI（邊個工具都得），再跟三層驗證嚟驗收。30 分鐘內完成佢——你嘅第一個 AI 編程作品就誕生喇。做完記得存好佢，呢個係你由「讀者」變成「實踐者」嘅紀念品。

本章小結

- 項目一（入門）：純前端工具站——體驗完整流程，30 分鐘搞得掂
- 項目二（進階）：全端記賬系統——PRD + 記憶檔案 + 5 步逐步實現，每步都「用得」
- 項目三（高級）：AI 學習助手——用 AI 寫呼叫 AI 嘅代碼，留意 prompt 工程
- 通用流程八步：需求思考拆解 PRD 記憶檔案逐步實現測試審查迭代
- 每一步都跟「指令-驗證-新對話」循環

第四十五章：質素保障——令 AI 寫嘅代碼可靠

AI 寫嘅代碼有一個特點：佢通常「睇落好啱」——變數命名規範、註解齊全、邏輯清晰。但「睇落啱」唔等於「真係啱」。*AI* 代碼入面隱藏嘅問題往往比手寫代碼更難發現，因為佢嘅表面質素太高，容易令人放鬆戒備。本章教你點樣為 *AI* 代碼建立有效嘅質素保障體系——用測試、審查、版本控制三道防線，將「睇落啱」變成「真係啱」。

一、測試驅動：叫 AI 寫測試

測試係 AI 編程質素保障嘅第一道防線。好消息係：叫 AI 寫測試比叫佢寫功能代碼更容易、更可靠——因為測試嘅預期行為係明確嘅，冇咁容易產生「睇落啱但實際唔啱」嘅代碼。

測試驅動嘅 AI 編程流程係：先叫 AI 寫功能代碼，驗證基本功能通過之後，即刻叫佢寫測試用例。測試用例應該覆蓋正常路徑、邊界情況同異常情況。然後再叫 AI 根據測試回饋修正功能代碼。呢個流程嘅好處係：測試一寫好，就係持續嘅驗證工具——之後任何修改都可以透過行測試嚟確認冇引入回歸問題。

```
請爲 calculate_discount 函數編寫單元測試。
測試用例應覆蓋：
1. 正常路徑：
   - 100元商品打8折 -> 80
   - 200元商品打5折 -> 100
2. 邊界情況：
   - 0元商品打8折 -> 0
   - 100元商品打0折 -> 100（不打折）
   - 100元商品打10折 -> 100（全價）
3. 異常情況：
   - 負數價格 -> 應拋出錯誤
   - 折扣率 > 1 -> 應拋出錯誤
   - 非數字輸入 -> 應拋出錯誤
使用Vitest框架，畀出完整測試檔案。
```

留意呢個指令嘅特點：佢明確列出咗三類測試用例（正常、邊界、異常），唔畀空間 AI 自由發揮去「選擇性測試」。AI 有時會齋寫正常路徑嘅測試嚟「快快手手交差」，明確列出所有類別就可以避免呢個問題。

二、代碼審查技巧

就算有測試覆蓋，你仍然需要對 AI 產出嘅代碼做人工審查。代碼審查唔係逐行去讀——咁樣太唪時間，亦都太依賴你嘅編程水平。有效嘅 AI 代碼審查應該留意以下幾個關鍵維度：

審查維度	留意乜嘢	常見問題
安全性	輸入驗證、金鑰處理、SQL 注入防護	AI 成日唔記得驗證輸入，金鑰寫死喺代碼入面
錯誤處理	異常有冇被捕獲，錯誤資訊合唔合理	AI 鍾意用空 catch 塊或者齋 console.log
效能	有冇明顯嘅效能問題，例如 N+1 查詢、無限循環	AI 有時會寫出 $O(n^2)$ 嘅演算法
可維護性	代碼結構清唔清晰，命名有冇意義	AI 間中會產生過度抽象嘅代碼
依賴管理	引入嘅依賴係咪必要，版本係咪明確	AI 傾向引入唔必要嘅函式庫

AI 代碼審查嘅關鍵維度

審查時唔使自己逐項檢查——可以叫 AI 幫你做第一輪審查。指令可以係：「請審查呢段代碼嘅安全性、錯誤處理同效能，指出潛喺問題。」AI 嘅自我審查能力不斷提升，雖然唔可以完全取代人工審查，但可以幫你過濾走大部分明顯問題。

三、版本控制：Git 喺 AI 編程入面嘅角色

Git 喺 AI 編程入面嘅角色比喺傳統編程入面更加重要。因為 AI 每次修改都可能引入不可預知嘅變化，你需要一個可靠嘅「後悔藥」——版本控制就係嗰個後悔藥。

AI 編程入面 Git 嘅最佳實踐係「頻密提交、小步快進」。每完成一個可以驗證嘅功能點，就提交一次。咁樣當 AI 某次修改出咗問題，你可以精準回退去上一個可用狀態，唔使將所有工作推倒重嚟。

另一個實用技巧係「分支隔離」。對於唔肯定結果嘅 AI 修改（例如大步重構），先開一個新分支嚟做。如果結果滿意，合併返入主分支；如果唔滿意，直接刪咗個分支，唔會影響主分支。咁樣比喺主分支度反覆修改安全得多。

1. 每次 AI 修改之前，確保工作區係乾淨嘅（冇未提交嘅更改）。咁樣如果 AI 嘅修改唔滿意，可以用 git checkout . 快速回退。

2. 每次 AI 修改完、驗證通過之後，即刻提交。提交資訊描述今次修改做咗乜嘢，例如「feat: 添加用戶註冊功能」。

3. 對於大步重構，先開分支：git checkout -b refactor/auth-system。喺分支上面完成修改，測試通過之後先合併。

4. 養成睇 diff 嘅習慣。每次 AI 修改完，用 git diff 睇嚇具體改咗乜嘢。咁樣唔單止幫你發現問題，仲幫你學習代碼。

四、常見陷阱同避坑指南

AI 編程入面有幾個重複出現嘅陷阱，幾乎所有初學者都會踩。早啲知道佢哋，可以幫你少行好多冤枉路。

陷阱一：信任陷阱

「AI 寫嘅代碼睇落好專業，一定冇問題啩？」——呢個係最危險嘅陷阱。AI 產生嘅代碼喺表面上非常企理——命名規範、註解齊全、結構清晰——呢種「表面專業感」會令人放鬆戒備。但表面企理唔代表邏輯正確。解藥：永遠透過測試嚟驗證，而唔係靠「睇落啱唔啱」嚟判斷。

陷阱二：膨脹陷阱

「既然 AI 幫到我寫代碼，不如將功能做得全面啲？」——咁樣會令代碼膨脹。AI 好擅長「多畀你」：你叫佢整一個登入頁面，佢會畀你一套完整嘅用戶管理系統。多餘嘅功能唔單止增加複雜度，仲增加咗出 bug 嘅機會率。解藥：嚴格跟 Simplicity First 原則，明確話畀 AI 知「淨係做呢個功能，唔好額外添加」。

陷阱三：理解陷阱

「AI 應該明我嘅意思啩？」——唔一定。AI 嘅理解能力雖然好強，但對住含糊嘅需求仍然會「估」。當佢估錯咗，產出嘅代碼就係錯嘅。更麻煩嘅係，佢估錯嘅代碼通常「睇落好合理」，令你誤會佢明咗。解藥：喺指令入面畀足具體細節，減少 AI 需要「估」嘅空間。

陷阱四：上下文陷阱

「我喺之前嘅對話講過㗎喇，AI 應該仲記得啩？」——好大機會唔記得。隨住對話變長或者開新會話，AI 會唔見咗早期上下文。如果某個關鍵約束冇寫入項目記憶檔案，佢好大機會喺之後嘅修改入面被忽略。解藥：關鍵約束寫入項目記憶檔案，唔好依賴對話記憶。

四大陷阱速記

信任陷阱——唔好信「睇落啱」；膨脹陷阱——AI 愛「多畀你」；理解陷阱——含糊需求佢會「估」；上下文陷阱——對話記憶會「唔記得」。對應嘅解藥分別係：測試驗證、斬走多餘、畀足細節、寫入記憶檔案。

嚟動手：為你嘅項目建三道防線

如果你已經開始做項目一（工具站）：(1) 叫 AI 為核心函數寫 3 類測試用例；(2) 跟五個維度審查一遍代碼（安全性/錯誤處理/效能/可維護性/依賴）；(3) 用 git 初始化項目，做第一次提交。三道防線齊咗，你嘅項目先至真正屬於你。

本章小結

- AI 代碼「睇落啱」≠「真係啱」——表面質素太高反而容易放鬆戒備
- 叫 AI 寫測試：明確列出正常/邊界/異常三類用例，防止「選擇性測試」
- 審查五維度：安全性、錯誤處理、效能、可維護性、依賴管理
- Git 係「後悔藥」：頻密提交、小步快進、分支隔離、成日睇 diff
- 四大陷阱：信任（測試驗證）、膨脹（斬走多餘）、理解（畀足細節）、上下文（寫入記憶檔案）

第四十六章：進階之路——由用戶到架構師

當你掌握咗第三十七至四十四章嘅方法論，可以熟練咁用 *AI* 完成項目開發之後，下一步係乜嘢？本章探討由「*AI* 工具用戶」到「*AI* 系統架構師」嘅進階之路。呢個階段嘅關注點唔再係「點樣叫 *AI* 寫好代碼」，而係「點樣令 *AI* 更好咁融入成個開發流程」。

一、MCP 協議同工具擴展

MCP（Model Context Protocol）係 2025 年出現、2026 年逐漸普及嘅一個開放協議，佢定義咗 AI 模型同外部工具之間嘅通訊標準。簡單嚟講，MCP 令 AI 可以「使用工具」——唔止產生代碼，仲可以執行命令、查詢數據庫、呼叫 API、操作檔案系統。

喺 MCP 之前，每個 AI 工具都有自己嘅工具集成方式，互不兼容。MCP 嘅出現意味住：一個符合 MCP 標準嘅工具可以被任何支援 MCP 嘅 AI 工具使用。咁樣大大降低咗工具集成嘅成本，亦都令 AI 嘅能力邊界由「產生文本」擴展到「執行動作」。

對進階用戶嚟講，MCP 嘅意義喺於：你可以為 AI 構建定製化嘅工具能力。例如，如果你嘅團隊有一套內部嘅部署系統，你可以寫一個 MCP 工具畀 AI 直接呼叫部署系統，唔需要人手切換去終端操作。呢種「AI 呼叫工具」嘅能力，係 Agentic Engineering 嘅基礎設施。

二、自訂工作流同 Skills

當你重複執行某啲固定嘅 AI 編程流程嗰陣，可以將佢哋固化做「自訂工作流」或者「Skills」。例如，你嘅團隊可能有一套標準嘅代碼審查流程——先檢查安全性，再檢查效能，再檢查可維護性。你可以將呢個流程寫成一個 Skill，畀 AI 每次代碼審查時自動跟呢個流程執行。

自訂工作流嘅核心價值係「一致性」——佢確保每次執行都跟同樣嘅標準，唔會因為對話長短或者上下文唔同而產生偏差。呢樣嘢對團隊協作尤其重要：每個成員用同樣嘅工作流，產出嘅代碼風格同質素標準就會趨於一致。

構建自訂工作流嘅方法因工具而異。Cursor 支援 Rules 同 Custom Instructions，Claude Code 支援自訂命令，Replit 支援 Runme。但核心思路一樣：將你嘅最佳實踐「編碼」成 AI 可以執行嘅指令模板。

三、多 Agent 協作模式

到 2026 年，多 Agent 協作開始成為一個熱門話題。佢嘅核心理念係：唔係齋用一個 AI 嚟完成任務，而係用多個分工唔同嘅 AI 嚟協作。例如，一個 AI 負責寫代碼，一個 AI 負責寫測試，一個 AI 負責代碼審查——佢哋各司其職，互相檢查。

多 Agent 協作嘅優勢喺於「分工」同「交叉驗證」：每個 Agent 專注自己嘅領域，產出質素更高；Agent 之間嘅交叉檢查可以發現自己可能忽略嘅問題。但劣勢都好明顯：協調成本高，通訊開銷大，除錯更複雜。

小貼士：一個人都可以玩「多 Agent」

對個人開發者嚟講，你唔使搭建複雜嘅多 Agent 系統——你可以「扮演」唔同嘅角色嚟模擬多 Agent 協作。例如：先用一個對話叫 AI 寫功能代碼；然後開一個新對話（提供 CLAUDE.md 上下文），叫 AI「以代碼審查者嘅身份審查呢段代碼」；最後再開一個對話叫 AI「根據審查意見修正代碼」。呢種「一人多角」嘅方式，可以獲得多 Agent 協作嘅大部分好處，而又唔使複雜嘅基礎設施。

四、持續學習路徑

AI 編程係一個快速變化嘅領域。今日嘅最佳實踐，聽日可能就要更新。以下係幾條持續學習嘅建議：

第一，留意源頭而唔係二手解讀。Andrej Karpathy 嘅原始推文同 CLAUDE.md、Anthropic 同 OpenAI 嘅官方文檔、各大工具嘅更新日誌——呢啲係第一手資訊。社羣文章同教程係對呢啲資訊嘅解讀同總結，有價值但可能失真。養成讀原始資料嘅習慣。

第二，實踐驅動學習。唔好「學完先再做」，而要「邊做邊學」。每次遇到新嘅問題類型（例如第一次處理檔案上載、第一次接入第三方 API），就以此為機會學習相關知識。實踐入面嘅學習效率遠高於脫離場景嘅理論學習。

第三，保持對工具嘅敏感度但唔好過度追新。每隔一兩個月，花半日時間瞭解最新嘅工具同趨勢——但唔好每次有新工具就轉。判斷標準係：新工具可唔可以顯著改善你嘅某個瓶頸？如果唔可以，而家嘅組合就夠喇。

第四，構建個人知識庫。將你每次解決複雜問題嘅對話、每次踩坑嘅教訓、每次有效嘅方法記錄低。唔使公開發布——呢個係你自己嘅「方法論筆記」。隨住時間累積，佢會成為你最寶貴嘅資產。

嚟動手：開一個「踩坑筆記本」

今日就開一個文檔（筆記軟件或者.md 檔案都得），寫低你到而家為止最想記住嘅三條 AI 編程心得——就算係「記得先諗清楚再問」都好。之後每次踩坑、每次學到新招，都記一條入去。半年後返轉頭睇，你會多謝今日嘅自己。

本章小結

- MCP 係 AI 同工具之間嘅通訊標準：令 AI 可以執行命令、查數據庫、呼叫 API
- 自訂工作流/Skills 嘅核心價值係「一致性」：將最佳實踐編碼成指令模板
- 多 Agent = 分工 + 交叉驗證；個人開發者可以用「一人多角」嚟模擬
- 持續學習四招：讀源頭、邊做邊學、唔好過度追新、建個人知識庫

第四十七章：速查手冊同資源

本章係一份可以獨立使用嘅速查手冊。當你記唔清某個原則或者需要快速決策嗰陣，直接揭到嚟呢度。建議你將佢當係「口袋卡」：五大原則速查、工具決策樹、常見問題、學習資源，一頁都唔漏。

一、思維模型速查卡

原則	一句講曬	操作要點
Think Before Coding	先諗清楚先開口	答三條問題：做乜嘢？結果係點嘅？有乜嘢約束？
Simplicity First	用最少代碼解決問題	每次只做一個功能，斬走唔必要嘅複雜度
Surgical Changes	精準修改唔越界	明確指定修改範圍，拒絕 AI 嘅「順手改嚇」
Goal-Driven Execution	講「做乜嘢」唔好講「點樣做」	畀目標唔畀步驟，畀 AI 自主揀路徑
Iterate & Verify	產生-驗證-修正循環	三層驗證：功能、邊界、架構

五大思維原則速查卡

二、工具選擇決策樹

當你需要揀 AI 編程工具嗰陣，跟以下決策路徑嚟揀。留意：呢個唔係「邊個最好」嘅排名，而係「邊個最啱你」嘅判斷框架。

你想做乜嘢？

零基礎，想快速體驗揀雲端 Builder (v0/Bolt/Replit)

日常開發，需要代碼補全揀 IDE 內置 AI (Cursor/Trae)

大型項目，需要批量操作揀終端 Agent (Claude Code)

快速原型驗證想法揀雲端 Builder 或 Trae SOLO

圖 6：AI 編程工具選擇決策樹

三、常見問題 FAQ

Q1：我完全唔識編程，可以用 AI 編程嗎？

得。2026 年嘅 AI 編程工具已經夠曬強大，你完全可以用自然語言描述需求，叫 AI 產生代碼。但有兩點建議：第一，由雲端 Builder 工具（例如 v0、Bolt）開始，佢哋唔使配置開發環境；第二，邊用邊學基礎概念——唔使精通語法，但你需要可以「睇得明」AI 寫嘅代碼嘅大意，咁樣先可以有效驗收。

Q2：我應該學邊個編程語言？

睇你想做乜嘢。做網站前端——JavaScript/TypeScript；做數據處理或者 AI——Python；做企業級後端——Java 或者 Go。但更重要嘅係：喺 AI 編程時代，「語言」嘅重要性喺度下降。AI 可以幫你處理語法細節，你需要明白嘅係「程序嘅結構」——乜嘢係函數、乜嘢係循環、乜嘢係數據結構。呢啲概念係跨語言嘅，一旦明白咗，轉乜嘢語言都快。

Q3：AI 寫嘅代碼有冇版權問題？

截至 2026 年 8 月，呢個仍然係一個有爭議嘅法律問題。一般嚟講，AI 產生嘅代碼喺大部分司法管轄區唔會被視為有明確嘅版權擁有者。但如果你用喺商業項目，建議：(1) 瞭解你用緊嘅 AI 工具嘅服務條款入面關於知識產權嘅條款；(2) 對關鍵代碼做人工審查同修改，令佢有你嘅創造性投入；(3) 避免叫 AI 直接抄可能有版權嘅開源代碼。

Q4：AI 會唔會取代程序員？

短期內唔會。AI 取代嘅唔係「程序員」呢個職業，而係「逐行編碼」呢樣工作。就好似計數機冇取代數學家，而係令數學家由繁瑣嘅手算入面解放出嚟咁。AI 編程工具令程序員由「寫代碼」轉向「諗問題」——呢樣嘢實際上提高咗程序員嘅價值上限。將來嘅競爭力唔喺於「可以寫幾多行代碼」，而喺於「可以解決幾複雜嘅問題」。

Q5：點樣判斷 AI 嘅產出啱唔啱？

三層驗證法。第一層：功能驗證——佢行唔行到？輸入輸出啱唔啱？呢個係最基本嘅。第二層：邊界驗證——空輸入、超長輸入、異常輸入會點？呢層可以發現隱藏嘅問題。第三層：架構驗證——代碼結構合唔合理？有冇寫死嘅嘢？方唔方便擴展？呢層需要你有一定嘅技術判斷力。如果你判斷唔到第三層，至少做到頭兩層——行到、邊界正常，就已經比大部分初學者強。

四、推薦學習資源

以下係截至 2026 年 8 月嘅推薦學習資源。留意，互聯網資源連結可能會變，如果某個連結失效，搜尋佢個標題通常可以搵到最新版本。

類型	資源	推薦理由
原始文獻	Andrej Karpathy 嘅 CLAUDE.md	五大原則嘅源頭，70 行但字字珠璣
原始文獻	Anthropic 官方文檔 (Claude Code)	終端 Agent 嘅最佳實踐指南
教程	Vibe Coding 社羣整理嘅 Skills 合集	社羣實踐者總結嘅實戰技巧集合
教程	Cursor 官方文檔同 YouTube 頻道	IDE 內置 AI 嘅權威使用指南
社羣	GitHub 上面嘅 awesome-claude-code	Claude Code 嘅最佳實踐同插件集
社羣	Reddit 嘅 r/cursor 社羣	全球 Cursor 用戶嘅經驗分享同討論
基礎	MDN Web Docs (developer.mozilla.org)	前端開發最權威嘅參考文檔
基礎	Python 官方教程 (docs.python.org)	Python 學習嘅官方起點

推薦學習資源清單

五、結語：工具喺手，方法論喺心

返返去本書嘅核心信念：工具每三個月迭代一次，方法論每三年先進化一次。喺 2026 年 8 月嘅今日，AI 編程工具嘅能力已經強大到令人驚嘆——你可以用一句話整一個網站，用一段描述產生一個全端應用，用一條指令完成一個複雜重構。但工具越強大，方法論嘅重要性反而越突出——因為工具嘅強大會放大你嘅每一個決策。

如果你嘅需求清晰，AI 可以幫你高效實現；如果你嘅需求含糊，AI 都會高效咁做出錯嘅嘢。如果你跟簡單至上，AI 可以幫你快速驗證諗法；如果你貪多求全，AI 都會幫你快速堆積技術債。如果你精準修改，AI 可以幫你好似外科手術咁優化代碼；如果你指令含糊，AI 都會幫你大刀闊斧咁引入 bug。

呢個就係點解本書花咗大部分篇幅講方法論而唔係工具操作。唔係因為工具唔重要，而係因為方法論係工具嘅「乘數」——方法論越好，工具嘅價值越大；方法論係零，工具幾強都只係零乘任何數。

你唔再係一個人喺度寫程式。你係一個指揮官，AI 係你嘅軍團。你思考嘅質素決定咗軍團嘅戰鬥力。由今日開始，用方法論武裝自己嘅思維，用工具放大自己嘅能力——呢個就係 AI 編程嘅全部祕密。

参考文献

1. Andrej Karpathy. (2025). Vibe Coding. Social media post introducing the concept of vibe-driven programming with AI.

2. Andrej Karpathy. (2025). CLAUDE.md - Project guidelines for AI-assisted development. GitHub repository, 70+K stars.

3. Anthropic. (2026). Claude Code: Terminal-native AI programming. Official documentation and best practices.

4. Cursor Team. (2026). Cursor 3: The next generation of AI-first IDE. Release notes and documentation.

5. OpenAI. (2026). Codex CLI: Bringing AI to the terminal. Technical documentation.

6. Model Context Protocol (MCP) Specification. (2025-2026). Open protocol for AI-tool integration.

7. Vibe Coding Community. (2026). Community-curated skills and best practices for AI-assisted programming.

8. Spec-Driven Development (SDD) methodology. (2026). Emerging practices in specification-first AI programming.

八、安卓開發實戰篇（第 48-58 章）

第四十八章：認識安卓——由操作系統到開發生態

恭喜你做出呢個決定——由「玩手機嘅人」變成「做手機應用嘅人」。喺動手之前，我哋先攤開張地圖：安卓由邊度嚟？點解全世界嘅開發者都幫佢寫應用？一個 *App* 由你手指敲鍵盤到用戶手機上面，中間發生咗乜嘢事？

一、安卓嘅身世：由「小機械人」到「全球最大系統」

安卓（Android）嘅故事始於 2003 年——一個叫 Andy Rubin 嘅工程師創辦咗 Android 公司，目標好樸素：做一套「更識用相機」嘅智能操作系統。後來谷歌喺 2005 年收購咗佢，2007 年聯同幾十間廠商成立開放手機聯盟（OHA），2008 年 9 月，第一款安卓手機 HTC Dream（G1）正式發售。

後面嘅故事大家都知喇：到 2026 年，安卓係全球市場份額最大嘅流動操作系統，活躍設備數以十億計。你嘅手機、電視盒、車載大螢幕、智能手錶，好可能都喺度行緊安卓。呢個意味住乜嘢？意味住你寫嘅每一行代碼，都有機會畀世界上最多人用到。

> **小貼士：安卓嘅開源基因**
>
> 安卓嘅核心代碼（AOSP，Android Open Source Project）係開源嘅——任何廠商都可以攞去修改定製。所以小米有 MIUI（而家叫澎湃 OS）、華為有鴻蒙兼容層、三星有 One UI。呢樣嘢對開發者係好事：你學嘅係「安卓開發」，唔係「某個品牌嘅開發」，一套功夫，通喫全網。

版本號嘅進化都好有趣：由 1.5 嘅紙杯蛋糕（Cupcake）開始，每個大版本都以甜品命名——甜甜圈、閃電泡芙、凍乳酪、薑餅……一直到 2019 年嘅安卓 10 先改成純數字。到 2026 年 8 月，最新正式版係 Android 17，而開發時你真正需要關心嘅「最低支援版本」，主流 App 一般定喺 Android 8（API 26）到 Android 10（API 29）之間——支援到幾年前嘅設備，你嘅用戶羣先夠大。

版本	代號/年份	對開發者重要嘅事
Android 8 (API 26)	Oreo • 2017	通知渠道、後臺限制——好多舊機型嘅最低線
Android 10 (API 29)	2019	分區儲存登場，檔案存取規矩大改
Android 12 (API 31)	2021	Material You 動態取色，介面觀感大升級
Android 13 (API 33)	2022	執行時通知權限，彈窗問你「準唔準發通知」
Android 14-15 (API 34-35)	2023-2024	預測性返回、16KB 頁、私隱進一步收緊
Android 16-17 (API 36-37)	2025-2026	更快嘅小版本節奏、AI 能力整合（例如 Gemini 擴展）

安卓版本簡史（截至 2026 年 8 月，開發重點關注項）

二、用乜嘢語言寫：Java 定 Kotlin？

安卓開發歷史上出現過兩種主流語言。Java 係「開國元老」——安卓誕生之初就用佢；Kotlin 係 2017 年穀歌欽點嘅「一等公民」，2019 年起官方明確「Kotlin 優先」（Kotlin-first），到 2026 年，新項目幾乎全部用 Kotlin，舊項目都大部分陸續遷移緊。

講句實喺話：如果你係完全零基礎，直接學 Kotlin，唔好返轉頭掂 Java。原因有三：一係 Kotlin 代碼量比 Java 少 30%-40%（同樣嘅功能，Kotlin 更短更清晰）；二係佢天生防到「空指標」呢種新手噩夢；三係所有現代教程、官方文檔、AI 編程工具（仲記唔記得我哋本 AI 編程書呀？）都以 Kotlin 為準。Java 只係喺維護舊項目、睇舊代碼嗰陣先會遇到。

Java 風格（長氣版）

```
public class MainActivity extends Activity \{
@Override
protected void onCreate(Bundle b) \{
super.onCreate(b);
setContentView(R.layout.activity);
\}
\}
```

Kotlin 風格（清爽版）

```
class MainActivity : ComponentActivity() \{
override fun onCreate(b: Bundle?) \{
super.onCreate(b)
setContent \{ App() \}
\}
\}
```

同樣一件事，Kotlin 少咗一堆花括號同樣板。第 50 章我哋會正式學佢，而家你只需要記住：安卓開發嘅「普通話」叫 Kotlin。

三、一個 App 係點樣誕生嘅？

好多人以為「開發 App」就係打開電腦狂寫代碼。實際上，一個 App 由諗法到用戶手機上面，大致要經過咁一條流水線：

想法

設計（界面 + 功能）

寫代碼

編譯打包（APK/AAB）

測試（模擬器 + 真機）

發佈（商店/安裝包）

圖 1：一個安卓 App 由諗法到上架嘅全流程

流程入面有兩個概念你而家就要認識：

- APK：安卓應用嘅安裝包格式（.apk），雙擊就可以裝到手機度，就好似 Windows 嘅.exe 咁。平時自己測試、傳畀朋友玩下，就用佢。

- AAB：Android App Bundle（.aab），上架 Google Play 用嘅「高級打包格式」。商店會根據用戶嘅手機型號動態裁剪，令每個人下載嘅包更細。自己裝唔到，只可以交畀商店。

另外你仲會不斷聽到一個詞：SDK（Software Development Kit，軟件開發工具包）。佢就係安卓官方提供嘅一整套「開發裝備」——包括各版本嘅 API 庫、除錯工具、模擬器映像等。你寫代碼時呼叫嘅 Activity、Button 呢啲類，都係嚟自 SDK。

四、安卓技術棧全景：你嘅「武器清單」

正式學習之前，先認識嚇 2026 年主流安卓項目嘅標準配置——呢張清單你而家睇唔明唔緊要，全書學完返轉頭睇，就會豁然開朗：

層次	主流選擇（2026）	做乜嘢用嘅
語言	Kotlin 2.x	寫邏輯嘅語言
介面	Jetpack Compose + Material 3	用代碼畫介面（本書主線）
架構	MVVM + ViewModel	將介面同邏輯分開，代碼唔會亂
數據儲存	Room（數據庫）+ DataStore（配置）	將數據存入手機
網絡	Retrofit + OkHttp + Kotlin 協程	同伺服器對話
構建工具	Gradle + Kotlin DSL	將代碼變成安裝包
開發環境	Android Studio	谷歌官方嘅「全家桶」IDE

2026 年安卓主流技術棧速覽

唔好畀呢串英文字嚇親。佢哋就好似廚房入面嘅鑊鏟罌罌嗷——你而家只需要知道「煮飯要入廚房」，至於每樣工具點樣用，後面章節會逐個拆開畀你睇。而且有個好消息：Android Studio 開新項目嗰陣會自動幫你將成套嘢配置好，你淨係負責炒菜就得。

五、安卓 vs iOS：開發者第一堂「揀邊邊」課

你手上可能同時有安卓同蘋果嘅設備。作為開發者，兩邊嘅體驗差異值得提前瞭解——唔係「邊個更好」嘅站隊，而係「邊一邊更加適合你起步」嘅現實賬：

維度	安卓開發	iOS 開發
開發電腦	Windows / Mac / Linux 都得	一定要 Mac（Xcode 獨佔）
開發工具	Android Studio（免費）	Xcode（免費）+ 上架年費 99 美元
上架審核	寬鬆，幾個鐘至幾日	嚴格，1 至 7 日係常態
裝機試玩	APK 隨便發、隨便裝	分發受限，測試要簽名 / TestFlight
收入環境	以廣告 / 國內渠道為主	App Store 內購付費習慣更好
學習成本	概念比較多（四大組件等等）	生態封閉但體系統一、文檔精良

安卓同 iOS 開發路徑對比（2026 年 8 月）

講句實喺話：如果你部電腦唔係 Mac，呢條題根本唔使揀——安卓係唯一選項。如果你有 Mac，本書第八、九兩篇可以對照住讀，編程思路係相通嘅，學完一邊再學另一邊會快好多。

嚟動手：睇嚇你手機入面嘅「安卓」

打開手機嘅 [設定] [關於手機]（或者「關於本機」），搵到「安卓版本」或者「Android 版本」嗰一欄——連續篤佢幾下，仲可以觸發系統嘅彩蛋小遊戲。記低你嘅版本號，對照上面嘅版本表睇嚇佢屬於邊一代。你部手機，就係你嘅第一部「開發測試機」。

本章小結

- 安卓 2008 年誕生，2026 年係全球市場份額最大嘅流動系統，活躍設備有幾十億部
- 最新正式版係 Android 17；主流 App 最低支援 Android 8 至 10，用戶盤子先至夠大
- 新項目一律用 Kotlin：程式碼少 30% 以上、防空指標、官方欽定
- APK 自己裝，AAB 上商店；SDK 係官方開發裝備包
- 2026 標準技術棧：Kotlin + Compose + MVVM + Room + Retrofit + Gradle + Android Studio

第四十九章：搭建開發環境——Android Studio 同第一個項目

工欲善其事，必先利其器。呢一章我哋將個「廚房」搭好：裝好 *Android Studio*、行通模擬器、開人生第一個安卓項目，再將工程結構拆開睇個明明白白。裝好環境之後，你會第一次見到自己寫嘅程式碼變成一個篤得到嘅掣——呢一刻，好多人會記一世。

一、安裝 Android Studio

Android Studio 係谷歌官方嘅集成開發環境（IDE），免費，功能全家桶：寫程式碼、睇介面預覽、除錯、打包全部都喺入面。安裝四步：

1. 打開官網 developer.android.com/studio，下載最新穩定版（2026 年 8 月係 Narwhal 之後嘅版本，Windows/Mac/Linux 都有）；
2. 行安裝包，一路撳「下一步」——安裝精靈會自動帶埋 Android SDK 同模擬器組件；
3. 第一次啟動嗰陣跟提示下載 SDK 組件（用預設勾選就好，可能要幾分鐘同幾個 GB 嘅空間）；
4. 搞掂之後你會見到歡迎頁——恭喜，個「廚房」搭好喇。

注意：硬件要求心入面有數

Android Studio 係個「食記憶體」嘅傢伙：建議 16GB 記憶體起步（8GB 都行到，但係一開模擬器就會比較卡）、磁碟預留 20GB 以上、CPU 要支援虛擬化（喺 BIOS 入面開 VT-x/AMD-V）。如果部電腦真係舊，可以用「真機除錯」代替模擬器（本章第四節會講），體驗反而更加暢順。

二、開你嘅第一個項目

喺歡迎頁撳 New Project，揀 Empty Activity 模板（最乾淨嘅新手模板），跟住填三樣嘢：

配置項	填乜嘢	說明
Name	My First App	App 名字，會顯示喺手機桌面上面
Package name	com.example.myfirstapp	全球唯一標識，慣常做法係「域名反寫」，先用預設
Minimum SDK	API 26（Android 8.0）	最低支援版本，覆蓋 95% 以上設備

開項目要填嘅三樣嘢

撳完 Finish 之後，Android Studio 會用 Gradle 自動下載依賴、構建項目（第一次要等幾分鐘，之後每次會快好多）。構建完成之後，你就有一個行到嘅 App——雖然佢仲淨係識顯示一行「Hello Android!」。

小貼士：包名嘅門道

包名（package name）就好似 App 嘅「身份證號碼」，全球唔可以重複，上咗架之後就唔可以改。個人練習用 com.example.xxx 就得；正式產品要換成你自己嘅域名反寫（例如 com.你個名.應用名）。唔好攞 com.example 嚟上架——商店會拒收。

三、行起上嚟：模擬器同真機

程式碼寫好咗，就要搵部「手機」嚟行。你有兩個選擇：

模擬器（Emulator）

模擬器係喺電腦入面虛擬出一部安卓手機。喺 Android Studio 入面打開 Device Manager → Create Device，揀一部設備（推薦 Pixel 系列）揀一個系統映像（帶 Google APIs 嗰種）下載啟動。跟住撳工具列嘅綠色三角 Run，幾秒之後，「手機」螢幕上面就會出現你個 App。

真機除錯（更加推薦）

真機永遠係最真實嘅測試環境——滑動手感、真網絡、真感應器，模擬器都比唔上。安卓真機除錯好簡單：

1. 手機上面打開 [設定] → [關於手機]，連續篤「版本號」7 次，開啟「開發者選項」；
2. 喺 [開發者選項] 入面打開 USB 除錯；
3. 用傳輸線駁電腦，手機彈窗揀「允許 USB 除錯」；
4. 返去 Android Studio，工具列嘅設備下拉清單入面就會出現你部手機，撳 Run。

避坑提醒：USB 除錯唔好長開

USB 除錯等於畀部電腦開咗個「後門」。喺公眾場合充電、駁去唔信任嘅電腦嗰陣，一定要閂咗 USB 除錯，防止其他人透過佢讀取你嘅資料。平時開發用完就閂，養成習慣。

四、工程結構拆解：每個檔案係做乜嘢嘅

第一次打開項目，左邊嘅檔案樹可能會令你頭暈。唔使慌，日常開發你 90% 嘅時間淨係會同下面呢幾個打交道：



圖 2：新手必識嘅工程結構（口訣：淨係掂 app/ 目錄）

- MainActivity.kt：App 嘅「大門」。用戶篤開個圖示，第一個被喚醒嘅就係佢。而家嘅模板入面佢負責顯示「Hello Android!」。
- AndroidManifest.xml：App 嘅「戶口簿」——聲明呢個 App 叫乜嘢名、圖示係邊個、有邊啲頁面、要用邊啲權限（相機、定位等等）。
- res/ 目錄：資源倉庫，存圖片、圖示、字串、顏色。將文字放喺資源檔而唔係寫死喺程式碼入面，係安卓開發嘅好習慣（換語言、換主題都方便）。
- build.gradle.kts：構建配方——聲明最低支援版本、依賴邊啲函式庫（例如 Compose）。

五、小試牛刀：將「Hello」換成你個名

打開 MainActivity.kt，你會見到 Compose 模板生成嘅程式碼。搵到 Text(“Hello Android!”) 呢一行，將引號入面嘅字換成你個名，例如 Text(“你好，張三！呢個係我嘅第一個 App”)，跟住撳 Ctrl+R（Mac 係 Cmd+R）重新運行。幾秒之後，模擬器上面就會顯示你個名。

```
setContent {
    MyFirstAppTheme {
        Surface(modifier = Modifier.fillMaxSize()) {
            // ↓ 把呢度改成你自己嘅文字
            Text("你好，張三！呢個係我第一個App")
        }
    }
}
```

恭喜，你已經完成咗一次完整嘅「改程式碼編譯安裝運行」循環。呢個循環，就係將嚟每一日嘅開發日常。趁熱打鐵，試嚇再改啲其他嘢：將字號調大、轉個顏色——下一章我哋就會學到呢啲「魔法」係點樣嚟嘅。

六、常見報錯急救：新手頭兩個禮拜嘅「急症室」

搭環境同第一次構建嘅階段，你好大機會遇到下面三種報錯。唔使慌，全部都有解藥：

報錯現象	常見原因	解藥
Gradle sync failed / 依賴下載慢	網絡訪問倉庫唔暢順	畀 Gradle 配置國內鏡像倉庫（阿里雲 / 騰訊雲 maven 鏡像），上網搜「gradle 國內鏡像」跟住改
SDK 組件缺失	安裝嗰陣跳過咗 SDK 下載	Settings SDK Manager 補裝缺失嘅平臺同工具
模擬器啟動失敗 / 黑屏	虛擬化未開啟、映像損壞	喺 BIOS 開 VT-x/AMD-V；刪咗模擬器再建過；或者直接用真機除錯

新手環境報錯急救表

小貼士：搜報錯嘅方法論

報錯資訊第一行係最有價值嘅——將佢原封不動複製入搜尋引擎，加埋「android studio」呢個關鍵詞。唔好搜「我裝唔到環境點算好」呢啲籠統問題，要搜「Gradle sync failed: Could not resolve com.android.tools.build」呢啲具體錯誤。精確嘅問題，先至有精確嘅答案。

嚟動手：完成你嘅「開發儀式感」清單

(1) 裝好 Android Studio 並且行通模擬器；(2) 用真機行一次自己個 App 並且截圖留念；(3) 將 Hello 嗰句字改成你個名；(4) 將項目壓縮存個檔。做完呢四步，你就可以喺朋友圈宣佈：我都係安卓開發者喇（有圖為證）。

本章小結

- Android Studio = 官方免費 IDE；建議 16GB 記憶體 + 20GB 磁碟 + 開虛擬化
- 新建項目三要素：名字、包名（全球唯一，com.example 唔好攞去上架）、最低 SDK（API 26 起）
- 兩種運行方式：模擬器（電腦入面虛擬手機）同真機（開 USB 除錯，用完記得閂）
- 工程口訣：淨係掂 app/ 目錄——MainActivity.kt 係大門，Manifest 係戶口簿，res/ 係資源倉庫
- 開發日常 = 改程式碼 Run 睇效果，循環往復

第五十章：Kotlin 快速上手——安卓開發嘅第一語言

呢一章我哋正式學語言。唔使緊張——*Kotlin* 係出名「對新手友好」，而且你唔需要死背文法書：先掌握 *20%* 嘅文法，就可以睇得明同寫得出 *80%* 嘅日常程式碼。剩低嘅就邊做項目邊查。記住我哋嘅學習原則：睇得明 > 記得住，用得到 > 精通。

一、變數同類型：畀資料改個名

程式就係「資料 + 邏輯」。變數，就係畀資料改個名儲起。Kotlin 有兩種聲明方式：

```
val name = "張三"          // val：只讀變數（推薦預設用呢個）
var age = 18               // var：可變變數（之後仲可以改）

age = 19                   // <i class="fas fa-check-circle mr-1"></i> var 可以改
// name = "李四"           // <i class="fas fa-times-circle mr-1"></i> val 不能改，編譯會報錯

val score: Int = 100       // 都可以明確寫明類型
val pi = 3.14              // 唔寫都會自動推斷
```

常用類型就得幾種，一張表記住：

類型	存乜嘢	示例
Int	整數	42
Double	小數	3.14
String	文字	「你好」
Boolean	真 / 假	true / false
List< T>	一組資料	listOf(1, 2, 3)

Kotlin 常用基礎類型

小貼士：val 優先原則

Kotlin 社區嘅鐵律：預設用 val，真係有需要先用 var。用得 val 嘅地方用咗 var，IDE 會用黃字警告你。呢個習慣可以幫你避開一大類「變數被意外改咗」嘅 bug——程式碼變化越少，越容易推理。

二、函數：將邏輯打包成「掣」

函數係一段有名嘅邏輯，好似遙控器上面嘅掣——撳一下（調用），就執行一串動作。Kotlin 入面用 fun 聲明：

```
fun greet(name: String): String {
    return "你好，$name！"      // $ 符號把變數插進字串
}

println(greet("張三"))         // 輸出：你好，張三！

// 單行函數可以簡寫成表達式
fun add(a: Int, b: Int) = a + b

println(add(3, 5))             // 輸出：8
```

幾個要點：參數寫喺括號入面（名喺前、類型喺後，用冒號隔開）；返回類型寫喺括號後面；字串模板 $ 變數名係 Kotlin 嘅招牌便利——唔使再好似 Java 咁將「字串 + 變數」拼嚟拼去。

三、條件同循環：畀程式識「思考」

程式嘅智能嚟自兩樣嘢：識判斷（if/when）同識重複（for/while）。

```
// 判斷：if / else
val age = 17
if (age >= 18) {
    println("成年人")
} else {
    println("未成年")
}

// 多分支：when（比 switch 好用一百倍）
val score = 85
val level = when {
    score >= 90 -> "優秀"
    score >= 60 -> "及格"
    else -> "再接再厲"
}

// 重複：for 循環
```

```
val fruits = listOf("蘋果", "香蕉", "橙子")
for (fruit in fruits) {
    println("我愛喫$fruit")
}
```

when 係 Kotlin 入面最常用嘅多分支寫法，安卓開發入面你會日日都用佢（例如根據狀態顯示唔同介面）。

四、空安全：Kotlin 嘅「保命符」

Java 年代最出名嘅 bug 叫 NullPointerException（空指標異常）——程式訪問咗一個「乜嘢都冇」嘅嘢，即場崩潰。統計指出佢係安卓崩潰排行榜嘅常年第一名。Kotlin 喺語言層面解決咗佢：預設唔允許空值。

```
var name: String = "張三"    // 普通類型：絕對唔會係 null
// name = null                  // <i class="fas fa-times-circle mr-1"></i> 編譯錯誤，直接將你擋住

var nickname: String? = null // 加個問號：允許爲 null
nickname = "三兒"

// 用可空類型嗰陣，必須安全咁處理：
println(nickname?.length)    // 安全調用：null 就返回 null，唔會崩
val len = nickname?.length ?: 0   // Elvis 操作符：null 時畀預設值
println(len)
```

注意：? 同?: 唔好撈亂

String? 係「允許為空」，a ?: b 係「a 為空嗰陣就用 b」（個樣好似貓王髮型，所以叫 Elvis 操作符）。啱啱開始寫程式碼嗰陣，見到編譯器報「null 相關錯誤」，十居其九係你唔記得咗處理可空類型——呢個正正係語言喺度保護你。

五、類同對象：整「零件」嘅模具

類係對象嘅「模具」，對象係類嘅「成品」。安卓入面嘅所有嘢——頁面、按鈕、資料——都係對象。Kotlin 入面有個神器叫 data class（資料類），專門用嚟「裝資料」，一行頂 Java 幾十行：

```
// 普通類：模具
class Dog(val name: String) {
    fun bark() = println("$name 汪汪！")
}

val dog = Dog("旺財")       // 整物件
dog.bark()                  // 旺財 汪汪！
```

```
// data class：裝數據嘅盒（自動帶 equals/copy 等）
data class User(val id: Int, val name: String, val email: String)

val me = User(1, "張三", "zhangsan@example.com")
val twin = me.copy(name = "張三豐")   // 複製並改一個字段
```

第日你做記賬本，一筆賬就係一個 data class：data class Record(val amount: Double, val category: String, val date: String)。睇，你已經寫得出真實項目嘅程式碼結構喇。

六、lambda 同高階函數：Kotlin 嘅「語法糖」

安卓程式碼入面你會日日都見到 { ... } 呢種「冇名嘅函數」，佢叫 lambda（匿名函數）。Button 嘅 onClick = { count++ } 就係一個 lambda——將「撳完之後做乜嘢」打包成一嚿，遞畀個按鈕。

```
// 普通寫法：定義一個有名函數再傳入
fun double(x: Int) = x * 2
val result = listOf(1, 2, 3).map(::double)

// lambda 寫法：就地定義，輕快直接
val result2 = listOf(1, 2, 3).map { it * 2 }   // it = 當前元素

// 篩選 + 變換連招（真實項目入面極常見）
val bills = listOf(12.5, 88.0, 5.0, 200.0)
val bigBills = bills.filter { it > 50 }.sorted()
// [88.0, 200.0]——"超過 50 嘅賬單，排好曬序"
```

幾個高頻「集合連招」記住佢：filter（篩選）、map（變換）、sorted（排序）、sum（求同）。你將來 90% 嘅列表處理，都係呢四招嘅組合。Kotlin 嘅哲學就係：一句話講得清楚嘅邏輯，就絕對唔寫五句。

七、寫畀零基礎嘅文法速查卡

唔好諗住一日背曬所有文法。將呢張卡儲低，寫程式碼嗰陣對照住查，兩個禮拜之後自然就熟：

要做乜嘢	寫法
定義唯讀變數	`val x = 1`
定義可變變數	`var x = 1`
定義函數	fun 名字 (參數): 返回類型 { ... }
判斷	if (條件) {...} else {...}
多分支	when { 條件 -> 結果}
遍歷	for (x in 列表) { ... }
字串拼接	「你好，$name」
處理可空	`x?.length ?: 0`
定義資料類	`data class X(val a: Int, val b: String)`

Kotlin 文法速查卡（新手版）

嚟動手：唔使裝任何軟件，先玩 10 分鐘 Kotlin

瀏覽器打開 play.kotlinlang.org（Kotlin 官方線上演練場），將本章每一段程式碼都貼入去行一次，改嚇數字同文字，睇嚇輸出點樣變。呢個「先玩後學」嘅 10 分鐘，會令你第 51 章上手嗰陣輕鬆一倍。

本章小結

- val 優先，var 慎用；類型可以自動推斷，冒號後面顯式聲明都得
- 函數 = 打包咗嘅邏輯；字串模板 $ 係招牌便利
- if/when 判斷，for 循環；when 係安卓開發入面嘅「多面手」
- 空安全係 Kotlin 嘅保命符：普通類型拒絕 null，可空類型必須用? / ?: 處理
- data class 係裝資料嘅盒，一個欄位一行，真實項目日日都用
- 20% 文法做 80% 嘅嘢：先用得，再精通

第五十一章：介面開發——Jetpack Compose 入門

介面係用戶見到嘅所有嘢。喺 *Compose* 出現之前，安卓介面用 *XML* 寫——好似填空題咁麻煩；*2021* 年穀歌推出咗 *Jetpack Compose*：直接用 *Kotlin* 程式碼「畫」介面，所見即所得，改一行程式碼預覽即刻刷新。到 *2026* 年，*Compose* 已經係新項目嘅絕對主流。呢一章，我哋學識「畫畫」。

一、Compose 嘅思想：介面就係函數

Compose 嘅核心思想一句講曬：介面 = 函數。每樣可以顯示喺螢幕上面嘅嘢（文字、按鈕、圖片）都係一個「可組合函數」（Composable），用 @Composable 標記。將佢哋好似積木噉嵌套埋一齊，就係介面。

```
@Composable
fun Greeting(name: String) {
    Text(text = "你好，$name！")      // 一嚿"文字積木"
}

// 巢狀組合：大積木套小積木
@Composable
fun HomeScreen() {
    Column {                            // 直向排列
        Greeting(name = "張三")
        Greeting(name = "李四")
        Button(onClick = { /* 撳咗做乜 */ }) {
            Text("撳我")
        }
    }
}
```

留意 @Composable 呢頂「帽」——佢話畀編譯器知「呢個係一嚿介面積木」。積木可以無限嵌套，呢個就係 Compose 嘅全部秘密。

二、基礎組件：文字、按鈕、輸入框、圖片

Compose 嘅組件就係「積木」，最常見嘅四種：

組件	做乜嘢	必備參數
Text	顯示文字	text（要顯示嘅字）
Button	按鈕	onClick（點擊回調）
TextField	輸入框	value、onValueChange（輸入變化回調）
Image	顯示圖片	painter（圖片資源）

Compose 四大基礎組件

```
@Composable
fun BasicWidgets() {
    Column {
        Text(text = "呢度係一段文字")
        Button(onClick = { println("被撳咗") }) {
            Text("按鈕")
        }
        var input by remember { mutableStateOf("") }
        TextField(
            value = input,
            onValueChange = { input = it },
            label = { Text("輸入啲乜嘢") }
        )
    }
}
```

輸入框嗰兩行入面出現咗 remember { mutableStateOf(“”) }——呢個就係 Compose 嘅「狀態」魔法，本章第五節會詳細講。而家先混個面熟。

三、佈局三件套：Column、Row、Box

組件有咗，點樣擺放？記住三個「盒」：

- Column：打直排（好似排隊打飯嗰）；
- Row：打橫排（好似並排坐嗰）；
- Box：疊住放（好似啤牌疊埋一齊，適合做角標、遮罩）。

```
Column {
    Text("第一行")
    Row {
```

```
        Text("左邊")
        Text("右邊")
    }
    Box {
        Text("底層")
        Text("疊喺上面嘅角標")
    }
}
```

想佈局更加好睇？畀組件加 Modifier（修飾符）——佢就好似畀積木「上妝」嘅：

```
Text(
    text = "靚嘅文字",
    modifier = Modifier
        .padding(16.dp)           // 四圍留白
        .fillMaxWidth()           // 撐滿闊度
        .background(Color.Blue)  // 背景色
        .clickable { /* 攰 */ }// 攰得
)
// 鏈式調用：一個接一個，次序有意義（先 padding 再背景，背景唔包邊距）
```

注意：Modifier 嘅次序有講究

padding(16.dp).background(Blue) 同 background(Blue).padding(16.dp) 效果唔同——前者係「先留白再塗色」（背景唔包邊距），後者係「先塗色再留白」（背景包埋邊距）。新手最容易喺呢度畫錯「奇怪嘅白邊」。唔確定嗰陣，調轉次序試吓，預覽即刻話你知答案。

四、狀態：畀介面「活」起上嚟

靜態介面冇乜意思。想撳掣之後數字 +1、輸入框可以打字，就需要狀態（State）。Compose 嘅規則：介面永遠忠實反映狀態，狀態一變，介面自動重畫。

```
@Composable
fun Counter() {
    // remember + mutableStateOf：記住呢個會變嘅值
    var count by remember { mutableStateOf(0) }

    Column {
        Text(text = "你攰咗 $count 次")
        Button(onClick = { count++ }) {   // 改狀態
            Text("攰我 +1")                 // 界面自動刷新
        }
    }
}
```

三行程式碼，一個識郁嘅計數器。呢個就係 Compose 比舊式寫法正嘅地方：唔使手動「刷

新介面」，改資料就得。記住兩個關鍵函數：remember（令個值喺介面重組嗰陣唔會被清空）同 mutableStateOf（話畀 Compose 知「呢個值變咗要重畫」）。

用戶點擊

狀態改變（count++）

Compose 感知到變化

自動重畫界面（重組）

圖 3：Compose 嘅「狀態介面」單向資料流

五、主題同深色模式：一次配置，全局生效

模板項目入面嘅 MyFirstAppTheme { ... } 就係主題。佢基於 Material 3 設計規範，幫你將顏色、字體、形狀統一管理。支援深色模式只需要三步：

1. 喺主題入面定義好淺色 / 深色兩套配色（模板已經自動生成咗 ColorScheme）；
2. 系統轉去深色模式嗰陣，isSystemInDarkTheme() 會自動判斷；
3. 用主題提供嘅顏色（例如 MaterialTheme.colorScheme.primary）而唔係寫死顏色值。

小貼士：Material 3 動態取色

Android 12 開始嘅「Material You」可以令 App 由你嘅桌布入面提取顏色。Compose 模板預設支援（dynamicColor）。你個 App 喺唔同用戶嘅手機上面會「自動換裝」，呢個就係現代安卓嘅浪漫。

六、預覽、動畫同實時編輯：Compose 嘅「外掛」

Compose 三個開發體驗上嘅「外掛」，一用咗就返唔到轉頭：

- @Preview 預覽：畀 Composable 加 @Preview 註解，Android Studio 右邊即刻 render 出個介面——改程式碼所見即所得，唔使每次都裝落手機度；
- 實時編輯（Live Edit）：2026 年嘅 Android Studio 支援喺模擬器 / 真機上面「熱更新」部分改動，改文字顏色呢類小調整，設備上面即刻變；
- 動畫 API：animateFloatAsState、AnimatedVisibility 幾行程式碼就可以令元素平滑過渡——「高級感」往往就差咗呢幾十行。

```
@Composable
fun AnimatedBox() {
    var big by remember { mutableStateOf(false) }
    // 值變嗰陣自動平滑過渡到目標值
```

```
    val size by animateDpAsState(if (big) 120.dp else 60.dp)

    Box(Modifier
        .size(size)
        .background(MaterialTheme.colorScheme.primary)
        .clickable { big = !big }
    )
}
```

小貼士：先功能後動畫

新手成日犯嘅毛病係「介面都未整啱，就開始糾結動畫」。先令功能行通，最後先統一加動效——呢個亦都係「簡單至上」原則：動畫係錦上添花，功能係雪中送炭。

嚟動手：整一個「心情按鈕」

將模板項目改成一個心情切換器：一個按鈕，點擊之後喺開心難過嘅之間循環切換，文字同背景色跟住變。提示：用 when 根據狀態揀文案同顏色。做完傳畀朋友玩，收穫第一波「嘩你識整 App 喇」。

本章小結

- Compose 思想：介面 = @Composable 函數，積木式嵌套
- 四大組件：Text / Button / TextField / Image
- 佈局三件套：Column 打直排、Row 打橫排、Box 疊放
- Modifier 鏈式「上妝」：padding、fillMaxWidth、background、clickable——次序有意義
- 狀態係靈魂：remember + mutableStateOf，改資料介面自動重畫
- 主題用 Material 3 + 動態取色，深色模式一次配置全局生效

第五十二章：四大組件同 Activity 生命週期

如果話 *Compose* 係 *App* 嘅「面」，咁呢一章講嘅就係 *App* 嘅「骨架」。安卓系統有四大基本組件，所有 *App* 都係由佢哋砌出嚟嘅；而當中最重要嘅 *Activity*，有一張每個安卓開發者都必須刻入腦嘅「生命週期圖」。本章學完，你就可以理解「點解切咗去後台返嚟，我個 *App* 嘅狀態就冇咗」呢類千古之謎。

一、四大組件：安卓嘅「四樑八柱」

安卓官方定義咗四種基本組件，分別負責一類工作：

組件	職責	打個比方
Activity	一個睇得到嘅介面	舖頭嘅門面——用戶見到、又互動到
Service	喺後台做嘢嘅「隱形人」	倉庫工人——播音樂、下載檔案，唔需要介面
BroadcastReceiver	接收系統廣播	傳達室——「冇電喇」「斷網喇」嘅喇叭
ContentProvider	跨 App 共享資料	公共資料櫃——例如將聯絡人共享畀其他 App

安卓四大組件速覽

新手階段你 95% 嘅時間都係同 Activity 打交道；Service 同 BroadcastReceiver 做音樂播放、訊息推送嗰陣會用到；ContentProvider 用得最少（系統已經將常用資料封裝好）。另外留意：四大組件全部都要喺 AndroidManifest.xml 入面「登記戶口」，唔登記嘅話系統就搵唔到佢哋。

二、Activity 生命週期：安卓開發嘅第一張「藏寶圖」

手機上面嘅 App 隨時可能被打斷：嚟電話喇、用戶切咗去覆微信、系統記憶體唔夠將你殺咗……Activity 嘅一生，就係喺呢張圖度循環旅行：

onCreate（出生：初始化）

onStart（可見了）

onResume（前臺交互中）

onPause（部分被遮住）

onStop（完全看不見）

onDestroy（銷燬）

onRestart / onStart / onResume：回到前臺時原路返回

圖 4：Activity 生命週期（前進方向：上排出世下排離場，返嚟嗰陣原路折返）

回調	幾時觸發	典型用途
onCreate	Activity 開嗰陣（一次）	載入介面、初始化資料
onStart	介面變為可見	開始啲輕量邏輯
onResume	攞到焦點、可以互動	開始動畫、恢復播放
onPause	被部分遮住 / 切走嗰一刻	暫停動畫、儲存草稿
onStop	完全睇唔到	釋放重磅資源、暫停下載
onDestroy	被銷毀之前	最後嘅清理

六個生命週期回調嘅典型用途

避坑提醒：轉螢幕會「殺死」Activity

新手最經典嘅炒車現場：App 轉一轉螢幕，輸入框入面嘅字全部冇晒。原因：轉螢幕預設會銷毀並重建 Activity（行一次 onDestroy onCreate）。解法：用 rememberSaveable 儲存狀態（Compose 入面代替 remember），佢會喺重建嗰陣自動將個值搵返。第 6、7 章會反覆用到。

三、Intent：App 內外通行嘅「車飛」

Intent（意圖）係組件之間通訊嘅「車飛」。你想由一個頁面跳去另一個頁面，或者打開瀏覽器、叫起相機，都要「買飛上車」：

```
// 方式一：顯式 Intent——明確指定去邊個頁面
val intent = Intent(this, DetailActivity::class.java)
intent.putExtra("name", "張三")      // 車上帶埋"行李"（數據）
startActivity(intent)

// 方式二：隱式 Intent——淨係講要做乜，系統搵邊個做
val web = Intent(Intent.ACTION_VIEW, Uri.parse("https://developer.android.com"))
startActivity(web)   // 系統會開瀏覽器 / 或者畀你揀
```

兩種方式各有用途：顯式用於自己 App 入面嘅頁面跳轉；隱式用於調用系統或者其他 App 嘅能力（開網頁、打電話、分享）。Compose 項目入面，喺 Composable 入面用 LocalContext.current 攞到 Context 就可以 startActivity。

四、多頁面同導航

真實嘅 App 都有好多個頁面。Compose 官方推薦嘅導航方案係 Navigation Compose（依賴庫 androidx.navigation）：

```
val navController = rememberNavController()

NavHost(navController, startDestination = "home") {
    composable("home") { HomeScreen(navController) }
    composable("detail/{name}") { entry ->
        val name = entry.arguments?.getString("name")
        DetailScreen(name ?: "未知")
    }
}

// 喺頁面入面跳轉：
navController.navigate("detail/張三")      // 帶參數
navController.popBackStack()               // 返回上一頁
```

小貼士：頁面多咗點算？

項目大咗之後，將所有頁面路由集中寫喺一個檔案入面（例如 AppNavGraph.kt），統一管理，唔好畀「跳去邊」嘅邏輯散落喺各個頁面。呢一步係由「行到」走向「可維護」嘅第一步。

五、順便認識：Fragment 係乜嘢（同埋點解本書冇教）

查舊教程嗰陣你一定會撞到 Fragment——佢係 Activity 入面嘅「可重用介面片段」，喺舊式多頁面架構入面係絕對主角。但係 Compose 年代，頁面直接用 Composable 函數 + Navigation 路由就夠，官方亦都已經淡化咗 Fragment 嘅地位。本書因此冇將佢放入主線。

畀你嘅建議：新項目唔好主動用 Fragment；但係睇舊程式碼、面試被問到嗰陣，知道佢係「Activity 入面嵌住嘅介面片段」就得。學習資源永遠優先揀 2023 年之後嘅——安卓技術棧嘅「老黃曆」更新得比想像中快。

嚟動手：畀計數器裝上「記憶」

將第 51 章嘅計數器改一改：(1) 將 remember 換成 rememberSaveable，轉螢幕驗證個數字仲喺唔喺度；(2) 加一個「關於」按鈕，用 Navigation 跳去第二個頁面顯示版本號；(3) 喺兩個頁面嘅 onCreate/onPause 入面 print 日誌（Log.d），用 Logcat 觀察生命週期調用次序。做完呢三步，生命週期就唔再係紙上談兵。

本章小結

- 四大組件：Activity（門面）、Service（後台）、BroadcastReceiver（喇叭）、ContentProvider（資料櫃）

- 生命週期六回調：onCreate onStart onResume onPause onStop onDestroy
- 轉螢幕會銷毀重建 Activity：用 rememberSaveable 保住狀態
- Intent 係「車飛」：顯式跳自己頁面，隱式調系統能力
- 多頁面用 Navigation Compose：NavHost + composable 路由

第五十三章：資料儲存同網絡請求

一個淨係得個「睇」字嘅 *App* 係死嘅；識儲資料、識連網絡嘅 *App* 先至係生嘅。呢一章認識安卓資料層嘅「三件套」：*DataStore*（存小配置）、*Room*（存結構化資料）、*Retrofit*（同伺服器傾偈），仲有佢哋嘅「總調度」——協程。學完呢章，你就可以寫到識記住用戶習慣、識實時更新內容嘅真 *App* 喇。

一、儲存方案點樣揀：先睇呢張表

方案	適合存乜嘢	特點	本書邊度用
DataStore	零散配置（主題、開關）	鍵值對，簡單安全	第 54 章記賬本設定
Room	結構化資料（賬目、用戶）	真正嘅 SQLite 資料庫封裝，可以查詢	第 54 章賬目記錄
Retrofit	（唔儲，上網攞）伺服器資料	HTTP 請求庫嘅事實標準	第 55 章天氣資料

資料三件套嘅分工

記一個口訣：「小配置入 DataStore，成批資料入 Room，外面嘅資料行 Retrofit」。舊教程入面常見嘅 SharedPreferences 已經被 DataStore 取代，SQLite 手寫 SQL 亦都已被 Room 取代——直接學新嘅，唔好行舊路。

二、DataStore：存啲小配置

想記住「用戶上次揀嘅幣種」「有冇開深色模式」呢類零散值，用 DataStore（Preferences 版）：

```
val Context.dataStore by preferencesDataStore(name = "settings")

// 寫：存一個值
suspend fun saveTheme(isDark: Boolean) {
    context.dataStore.edit { prefs ->
        prefs[THEME_KEY] = isDark
    }
```

```
}

// 讀：攞出嚟（Flow 會自動通知變化）
val themeFlow: Flow<Boolean> = context.dataStore.data
    .map { it[THEME_KEY] ?: false }
```

留意返回嘅係 Flow——一個會「流動」嘅資料流。個值一改，所有觀察佢嘅地方自動收到新值。呢個概念（響應式）係現代安卓嘅基石，第 54 章會實戰。

三、Room：畀資料安個「真正嘅屋企」

記賬本有成百上千筆賬，仲要按分類、按月份查詢統計——呢樣嘢需要資料庫。Room 係 SQLite 嘅官方封裝，將「建表、查資料」變到好似寫 Kotlin 嗰自然。三樣嘢：

```
// 1. Entity：表結構（一張表 = 一個 data class）
@Entity(tableName = "records")
data class Record(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val amount: Double,
    val category: String,
    val note: String,
    val date: String
)

// 2. Dao：操作接口（增刪改查）
@Dao
interface RecordDao {
    @Query("SELECT * FROM records ORDER BY date DESC")
    fun getAll(): Flow<List<Record>>

    @Insert
    suspend fun insert(record: Record)

    @Delete
    suspend fun delete(record: Record)

    @Query("SELECT SUM(amount) FROM records WHERE category = :c")
    suspend fun sumByCategory(c: String): Double
}

// 3. Database：總裝
@Database(entities = [Record::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun recordDao(): RecordDao
}
```

呢套「三件套」模板可以原封不動搬入你任何項目——換吓欄位名就得。查詢返回 Flow，介面跟住資料自動刷新，妙不可言。

四、Retrofit：同伺服器講嘢

天氣、新聞、匯率……呢啲資料唔喺手機入面，喺其他人嘅伺服器度。Retrofit 令「向伺服器攞資料」變到好似調函數嗷簡單。三步行：

```
// 1. 定義接口：將 HTTP 請求寫成函數
interface WeatherApi {
    @GET("v1/weather")
    suspend fun getWeather(@Query("city") city: String): WeatherResponse
}

// 2. 建立實例
val api = Retrofit.Builder()
    .baseUrl("https://api.example.com/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()
    .create(WeatherApi::class.java)

// 3. 像調函數一樣調用（喺協程入面）
val result = api.getWeather("北京")
```

避坑提醒：兩條網絡鐵律

(1) 網絡請求絕對唔可以喺主線程做——會卡死介面甚至崩潰（NetworkOnMainThreadException）。所有網絡調用必須攞喺協程（viewModelScope.launch）入面。(2) 明文 HTTP 預設被禁止：安卓 9 開始要求 HTTPS。測試嗰陣如果要 http:// 嘅本地地址，需要喺 Manifest 入面顯式聲明 usesCleartextTraffic——但係正式產品必須全部行 HTTPS。

五、JSON 解析：畀網絡資料「翻譯」

伺服器返回嘅資料大多係 JSON——一串好似「嵌套字典」嘅文字。Retrofit 需要一個「翻譯官」將佢變成 Kotlin 對象。2026 年嘅兩大主流方案：

方案	特點	推薦場景
Gson / Moshi	老牌反射方案，配置零成本	快速起步、欄位唔多嘅小項目
kotlinx.serialization	官方編譯期方案，更快更安全	新項目首選（2026 年主流）

JSON 解析兩大方案

```
// 畀數據類加 @Serializable 註解
@Serializable
data class WeatherResponse(
```

```
    val weather: List<Weather>,
    val main: Main,
    val name: String
)

// Retrofit 掛上對應 Converter（以 Retrofit 2.11+ 官方支援爲例）：
// .addConverterFactory(json.asConverterFactory("application/json".toMediaType()))
```

無論邊種方案，鐵律都係：先睇真實返回嘅 JSON，再定義欄位。好多「解析失敗」嘅 bug，都係因為欄位名寫錯、嵌套層級估錯——將原始 JSON print 咗出嚟對照，十秒定位。

六、協程：安卓並發嘅「普通話」

八、協程：安卓並發嘅「普通話」

上面反覆出現嘅 suspend 同 launch 嚟自 Kotlin 協程（Coroutine）——安卓官方嘅並發方案。你可以粗略咁理解為：令「要等一陣」嘅操作（網絡、資料庫）唔會阻塞介面。

```
// 喺 ViewModel 入面啓動協程
viewModelScope.launch {
    val records = dao.getAll().first()  // suspend：會自動掛起等
    // 更新界面狀態……
}

// 關鍵詞速記：
// suspend   —— "我要等待，別攔着別人"
// launch    —— "開工！"
// viewModelScope —— 跟着頁面走嘅"工作間"，頁面銷燬自動收工
```

零基礎先記住用法就好，背後嘅「掛起同恢復」原理可以第日再深究。日常開發你寫嘅 90% 都係呢個模板。

嚟動手：整一個「每日一句」小應用

搵一個免費嘅名言 API（例如 hitokoto 呢類），用 Retrofit 拉取「每日一句」，顯示喺螢幕中央，加個「換一句」按鈕（喺協程入面重新請求）。再將鍾意嘅句子用 Room 儲入「收藏夾」。呢個練習將 DataStore/Room/Retrofit/協程全部用咗一次——做完佢，第 7、8 章就係小菜一碟。

本章小結

- 口訣：小配置入 DataStore，成批資料入 Room，外面嘅資料行 Retrofit
- DataStore 返回 Flow：個值一變，觀察者自動收到
- Room 三件套：@Entity（表）+ @Dao（操作）+ @Database（總裝），模板可以直接重用

- Retrofit 三步：定義接口建立實例協程入面調用
- 兩條鐵律：網絡唔入主線程；正式產品必須 HTTPS
- 協程 = 唔阻塞介面嘅等待；viewModelScope 係頁面專屬工作間

第五十四章：項目一——我嘅記賬本（本地版）

歡迎嚟到你第一個「正式項目」。呢個記賬本雖然唔上網，但五臟俱全：記到一筆賬、睇到流水、按月統計到、又刪改到。做完佢，你就完整行咗一次「資料邏輯介面」嘅全流程。唔好心急抄程式碼——先諗清楚先至郁手，呢個係我哋一貫嘅原則。

一、需求思考：先問五條問題

郁手之前，用「需求五要素」將個諗法捋清楚：

要素	答案
Who（畀邊個用）	自己用，屋企人朋友都得
What（做乜嘢）	記收支、睇流水、按月匯總、按分類統計
Why（點解）	練手 + 搞清楚啲錢使咗去邊
How（點樣做）	Kotlin + Compose + Room + MVVM，純本地冇後端
Constraint（約束）	一個禮拜之內完成；最低支援 Android 8；唔上架都先按上架標準寫

記賬本需求五要素

跟住拆解成「每一步都用得」嘅小步驟：(1) 建 Room 資料層（儲到一筆賬）；(2) 列表頁（睇到流水）；(3) 錄入頁（加到賬）；(4) 刪除同編輯；(5) 月度匯總卡。每完成一步，App 都係一樣「用得嘅嘢」。

二、資料層：畀「賬」有個屋企

先定義一筆賬係咩樣——欄位要夠用但唔好貪多：

```
@Entity(tableName = "records")
data class Record(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val type: String,        // "income" 收入 / "expense" 支出
```

```
    val amount: Double,       // 金額（元）
    val category: String,     // 分類：餐飲、交通、人工……
    val note: String = "",    // 備註（可選）
    val date: String          // "2026-08-15"
)
```

Dao 入面加兩條「統計用」嘅查詢——呢個係記賬本嘅靈魂：

```
@Dao
interface RecordDao {
    @Query("SELECT * FROM records ORDER BY date DESC, id DESC")
    fun getAll(): Flow<List<Record>>

    @Insert
    suspend fun insert(record: Record)

    @Delete
    suspend fun delete(record: Record)

    @Query("SELECT COALESCE(SUM(amount),0) FROM records WHERE type='income' AND
    date LIKE :month || '%'")
    suspend fun monthIncome(month: String): Double   // month 形如 "2026-08"

    @Query("SELECT COALESCE(SUM(amount),0) FROM records WHERE type='expense' AND
    date LIKE :month || '%'")
    suspend fun monthExpense(month: String): Double
}
```

小貼士：金額點解用 Double？

嚴謹嘅金融 App 會用 BigDecimal 或者「以『分』為單位嘅 Int」嚟避免浮點誤差（0.1+0.2≠0.3 呢個老問題）。練手項目用 Double 就夠；如果第日做真金融產品，記得轉做用「分」做單位——呢個係無數前輩踩出嚟嘅坑。

三、MVVM：介面同邏輯「分家」

直接畀介面去操作資料庫？行到，但介面程式碼會又長又臭。MVVM 架構令大家各司其職：View（介面）淨係負責畫，ViewModel（邏輯中樞）管資料，Model（Room）管儲存。

View（Compose 界面）

ViewModel（狀態與邏輯）

Model（Room 數據層）

圖 5：MVVM 三層結構——資料單向流動，各層互唔過界

```
class RecordViewModel(private val dao: RecordDao) : ViewModel() {
```

```
    // 將 Flow 轉成界面觀察到嘅狀態
    val records = dao.getAll()
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(), emptyList())

    // 用戶操作：調用 suspend 函數
    fun addRecord(type: String, amount: Double, category: String, note: String,
    date: String) {
        viewModelScope.launch {
            dao.insert(Record(type = type, amount = amount,
                category = category, note = note, date = date))
        }
    }

    fun deleteRecord(record: Record) {
        viewModelScope.launch { dao.delete(record) }
    }
}
```

介面（Compose）入面咁樣用：val vm: RecordViewModel = viewModel()，跟住 val list by vm.records.collectAsState()——資料一變，列表自動刷新，你乜都唔使做。呢個就係 MVVM + Flow + Compose 三劍合璧嘅威力。

四、介面：列表頁 + 錄入頁

列表頁用 LazyColumn（懶載入列表）——資料幾多都唔卡：

```
@Composable
fun RecordListScreen(vm: RecordViewModel) {
    val records by vm.records.collectAsState()

    Column {
        // 頂部彙總卡片（收入-支出=結餘）
        SummaryCard(records)

        // 流水列表
        LazyColumn {
            items(records, key = { it.id }) { record ->
                RecordRow(record) { vm.deleteRecord(record) }
            }
        }

        // 懸浮"記一筆"按鈕　彈錄入對話框
        FloatingActionButton(onClick = { showAddDialog = true }) {
            Text("+")
        }
```

```
    }
}

@Composable
fun RecordRow(record: Record, onDelete: () -> Unit) {
    Row(Modifier.fillMaxWidth().padding(12.dp)) {
        Text(record.category, fontWeight = FontWeight.Bold)
        Spacer(Modifier.weight(1f))
        Text(if (record.type == "income") "+${record.amount}" else "-${record.
    amount}",
             color = if (record.type == "income") Color(0xFF27AE60) else Color(0
    xFFC0392B))
    }
}
```

錄入用 AlertDialog（對話框）最慳事：幾個輸入框 + 收入 / 支出切換 + 分類下拉，撳確定就調 vm.addRecord(...)。

注意：輸入校驗唔好偷懶

金額輸入框一定要校驗：空值、負數、「abc」呢類非法輸入全部要欄住（用 toDoubleOrNull() 判斷）。App 冧咗最常見嘅原因之一就係「將用戶嘅輸入當咗係乖仔」。記住：用戶輸入嘅所有嘢，都要當敵人咁審查。

五、編輯同刪除：畀資料「入得到又出得到」

淨係識加唔識刪嘅記賬本，用三日就報廢。補返兩個操作：

```
// Dao 入面補兩條：
@Query("UPDATE records SET amount=:amount, category=:category, note=:note WHERE id
    =:id")
suspend fun update(id: Long, amount: Double, category: String, note: String)

@Query("DELETE FROM records WHERE id=:id")
suspend fun deleteById(id: Long)

// 界面上：長撳列表項彈"編輯 / 刪除"菜單（Compose 用 DropdownMenu 或 AlertDialog）
// 刪除之前最好彈確認框——"刪錯嘢"係記賬類 App 用戶最憎嘅事
```

互動細節兩條經驗：刪除必須二次確認（防止手快快），編輯盡量預填返原本個值（改一個字比起重新輸入慳事）。唔好睇小呢兩個細節——產品感就係呢啲細節堆出嚟嘅。

六、驗收：三層檢查 + 兩個進階挑戰

八、驗收：三層檢查 + 兩個進階挑戰

項目做完，按「功能邊界架構」三層驗收：記唔記到、刪唔刪到、重開 App 資料仲喺唔喺度（功能）；金額輸 0 或者負數會唔會冧、轉屏會唔會冇咗資料（邊界）；介面同資料有冇撈埋一齊、欄位有冇寫死（架構）。

基礎版驗收過關之後，畀自己加兩味「進階菜」：

1. 月度篩選：頂部加月份切換（例如 2026-08 v），列表淨係顯示當月流水；
2. 分類圓餅圖：用 Compose 嘅 Canvas 或者引入輕量圖表庫，將「錢使咗去邊」畫成圓餅圖——做完呢樣嘢，記賬本就有啲「產品樣」喇。

嚟動手：今個禮拜搞掂佢

按「資料層 ViewModel 列表頁錄入頁匯總卡」嘅次序郁手，每日一步。卡咗就將報錯原封不動丟畀搜尋引擎或者 AI（仲記唔記得第七篇《AI 編程實戰篇》？描述問題嗰陣講清楚「期望行為、實際行為、重現步驟」）。完成之後用它記一個禮拜賬——自己整嘅 App 用起上嚟，成就感會推住你繼續向前行。

本章小結

- 需求五要素拆成「每步用得」嘅小步驟，先至郁手
- 資料層：Record 實體 + Dao（包埋月份統計 SQL）+ Database
- MVVM 分工：View 淨係負責畫，ViewModel 管邏輯，Model 管儲存
- Flow + stateIn + collectAsState：資料一變介面自動刷新
- LazyColumn 懶載入列表；AlertDialog 做錄入
- 三層驗收（功能 / 邊界 / 架構）+ 進階挑戰（月度篩選、分類圓餅圖）

第五十五章：項目二——天氣查詢 App（網絡版）

記賬本係「練內功」，天氣 *App* 係「見世面」——佢要同互聯網打交道。呢個項目你會學到：申請免費 *API*、解析 *JSON*、處理「載入中 / 成功 / 失敗」三種介面狀態，同埋點樣將 *API* 密匙收埋好。做完佢，你就具備咗整任何「上網 *App*」嘅能力——新聞、匯率、翻譯，套路一模一樣。

一、申請免費天氣 API

個人練手，推薦 OpenWeatherMap 或者國內嘅和風天氣（QWeather）——都有免費額度，註冊即刻用得。以 OpenWeatherMap 為例：

1. 註冊賬號喺 API keys 頁面生成一個 Key（apikey）；
2. 免費版可以調用 Current Weather（當前天氣）接口：api.openweathermap.org/data/2.5/weather?q=beijing&appid= 你嘅 Key&units=metric；
3. 喺瀏覽器度先試一次，睇吓返回嘅 JSON 係咩樣——呢個係接任何 API 之前必做嘅「預習」。

避坑提醒：密匙絕對唔可以寫死喺程式碼入面

API Key 等於你嘅「銀包密碼」，寫入程式碼一提交上 GitHub，幾個鐘之內就會被爬蟲掃走盜刷。正確做法：擺入 local.properties（呢個檔案已經被 gitignore 忽略），再透過 Gradle 嘅 buildConfig 讀入程式碼。搜「Android 隱藏 API Key 嘅最佳實踐」跟住改就得。

二、網絡層：Retrofit + JSON 解析

先照 API 返回嘅 JSON 定義「資料模型」，對唔上嘅欄位一律忽略：

```
// JSON: {"weather":[{"description":"晴"}],"main":{"temp":26.5,"humidity":40},"name
    ":"北京"}
data class WeatherResponse(
    val weather: List<Weather>,
```

```
    val main: Main,
    val name: String
)
data class Weather(val description: String)
data class Main(val temp: Double, val humidity: Int)

interface WeatherApi {
    @GET("data/2.5/weather")
    suspend fun getWeather(
        @Query("q") city: String,
        @Query("appid") key: String,
        @Query("units") metric: String = "metric",
        @Query("lang") lang: String = "zh_cn"
    ): WeatherResponse
}
```

Gson 會自動將 JSON 塞入呢啲 data class——欄位名一致，魔法自動發生。

三、三種介面狀態：載入中 / 成功 / 失敗

上網 App 嘅靈魂在於「狀態管理」。網絡慢、斷網、城市名打錯……每種情況介面都要畀用戶一個交代。用一個 sealed class（密封類）優雅咁表示：

```
sealed class UiState {
    object Loading : UiState()                    // 兜緊圈
    data class Success(val data: WeatherResponse) : UiState()  // 成功
    data class Error(val message: String) : UiState()          // 失敗
}

// ViewModel 入面：
fun search(city: String) {
    viewModelScope.launch {
        _state.value = UiState.Loading
        _state.value = try {
            UiState.Success(api.getWeather(city))
        } catch (e: Exception) {
            UiState.Error("搵唔到呢個城市，轉個名再試吓？")
        }
    }
}

// 界面入面：
when (val s = state) {
    is UiState.Loading -> CircularProgressIndicator()   // 兜圈
    is UiState.Success -> WeatherContent(s.data)        // 展示
    is UiState.Error -> Text(s.message)                 // 報錯
```

```
}
```

呢個 Loading/Success/Error 三段式係上網介面嘅「萬金油模板」，先背低佢，第日所有上網 App 照住個模畫個樣。

四、實戰介面程式碼：將天氣畫出嚟

狀態有咗，介面就係咁樣（關鍵片段）：

```
@Composable
fun WeatherContent(data: WeatherResponse) {
    Column(
        Modifier.fillMaxSize().padding(24.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(data.name, fontSize = 28.sp, fontWeight = FontWeight.Bold)
        Text(
            "${data.main.temp}°C",
            fontSize = 72.sp,
            fontWeight = FontWeight.Light
        )
        Text(
            data.weather.firstOrNull()?.description ?: "未知",
            fontSize = 20.sp,
            color = Color.Gray
        )
        Spacer(Modifier.height(16.dp))
        Row {
            Text(" 溼度 ${data.main.humidity}%")
            Spacer(Modifier.width(24.dp))
            Text(" 體感 ${data.main.temp}°C")
        }
    }
}
```

幾個小細節值得學：firstOrNull() 安全攞第一個元素（列表可能係空嘅），?: “未知”畀個兜底文案——空安全原則喺介面入面同樣適用。溫度用 72.sp 嘅大字號撐場面，「輕」字重（Light）令個數字望落優雅，呢個係天氣類 App 嘅經典配方。

五、介面同互動細節

七、介面同互動細節

- 搜尋框：頂部 TextField，輸入城市名撳搜尋（鍵盤上面設「搜尋」動作，一撳即搜）；

- 天氣卡：大字號溫度（26.5°）、天氣描述（晴）、濕度、城市名；
- 收藏城市：將成日睇嘅城市儲入 DataStore，下次打開自動幫你查——順手溫習返第 6 章；
- 下拉刷新：喺 Compose 用 PullToRefreshBox（Material 3 提供），手感即刻「專業」起上嚟。

小貼士：圖片令界面活起嚟

幫每種天氣配一個 emoji 或者圖標（），用 when(description) 嚟對應——三行代碼，界面嘅溫度感即刻翻倍。亦都可以載入第三方嘅天氣圖標庫，但練手階段 emoji 已經夠晒用。

嚟動手：天氣 App 驗收清單

(1) 正常搜得出天氣；(2) 斷開網絡，App 優雅報錯唔會崩潰（開飛行模式嚟測！）；(3) 輸入一個唔存在嘅城市，提示都要友好；(4) 收藏兩個城市，重啟 App 之後仲喺度；(5) 轉屏唔會冇咗狀態。五條全部過關，呢個項目先算畢業。

本章小結

- 先用瀏覽器試 API、睇明 JSON，先至寫代碼——「預習」永遠唔蝕底
- API Key 收埋入 local.properties，千祈唔好入代碼庫
- Retrofit 接口 + data class：欄位對得返，JSON 自動解析
- Loading/Success/Error 三段式狀態係聯網界面嘅萬金油
- 斷網、空輸入、城市唔存在——錯誤處理係聯網 App 嘅及格線

第五十六章：打磨同發布——測試、除錯同上架 Google Play

App 做咗出嚟，跟住點呢？呢一章回答「由跑得郁」到「可以交付」嘅三件事：點樣高效咁除錯搵 *bug*、點樣寫測試防走樣、點樣將 *App* 攞上 *Google Play* 畀全世界下載。上架唔只係儀式感——佢係將一件作品變成產品嘅分水嶺。

一、除錯三板斧：Logcat、斷點、UI 檢查器

工具	做乜嘢	點用
Logcat	睇系統/自己打嘅日誌	代碼入面 Log.d(”TAG”,” 值 =$x”)，喺面板搜 TAG 過濾
斷點除錯	令程式停喺某一行，睇變數值	喺行號旁邊撳一下出紅點，用 Debug 模式行，逐步咁行
Layout Inspector	睇運行中嘅界面結構	選單 Tools Layout Inspector，實時睇組件樹

除錯三板斧

小貼士：報錯點樣睇？

崩潰之後 Logcat 會有紅色堆疊（Stacktrace），由下往上搵第一行以自己包名開頭嘅（例如 com.example.yourapp），嗰行基本上就係案發現場。先睇「Exception 類型」（空指標？類型轉換？），再睇「at ... 你嘅檔案: 行號」。呢一招可以解決 80% 嘅崩潰。

二、測試：畀代碼買保險

手動測試靠唔住——改咗呢個功能，另一個靜靜雞壞咗。測試就係「後悔藥」兼「防彈衣」。安卓測試分三層，新手先讀透第一層：

層級	測乜嘢	擺喺邊	速度
單元測試	純邏輯（計算、校驗）	app/src/test/	秒級
儀器測試	真實設備上嘅界面行為	app/src/androidTest/	分鐘級
手動測試	真人感受、體驗	——	永遠都需要

測試三層

```
// 把"金額字串轉數字"呢種邏輯抽成純函數
fun parseAmount(input: String): Double? {
    val v = input.trim().toDoubleOrNull()
    return if (v != null && v > 0) v else null
}

// 測試佢（JUnit4）
@Test
fun emptyInputReturnsNull() {
    assertEquals(null, parseAmount(""))
}
@Test
fun negativeReturnsNull() {
    assertEquals(null, parseAmount("-5"))
}
@Test
fun normalValueParses() {
    assertEquals(12.5, parseAmount("12.5"))
}
```

記賬本入面嘅金額校驗、統計計算，都應該有單元測試罩住。行測試：右鍵 test 目錄 Run。全部綠色過關，就可以放心改代碼。

三、簽名打包：畀 App 蓋上「印章」

安卓要求所有安裝包必須簽名——就好似蓋公章咁，證明「呢個包係我出嘅，冇畀人改過」。除錯嗰陣 Android Studio 會自動用 debug 簽名；發布就要用自己嘅 release 簽名。流程係：

1. 選單 Build Generate Signed Bundle / APK；
2. 建立 keystore 檔案（密鑰庫）——設定密碼同埋妥善保管；
3. 揀 release + AAB 格式，構建完成。

避坑提醒：keystore 唔見咗 = App 唔見咗

上架之後更新 App 必須用同一個簽名，否則商店會拒絕更新。keystore 檔案同密碼一旦唔見咗，你就只可以另開一個新應用重新上架——舊用戶全部接唔返。一定要將 keystore 同密碼備份去安全嘅地方（雲盤加密 + 記事本），呢個係安卓開發者最昂貴嘅教訓之一。

四、上架 Google Play 全流程

註冊開發者賬號

準備商店素材

上傳 AAB

填數據安全表

審覈（數小時 ~ 數天）

上架

圖 6：Google Play 上架六步走

- 註冊：一次性 25 美元註冊費，需要一個用得嘅 Google 賬號；
- 素材：應用圖標（512×512）、截圖（最少 2 張）、功能描述、私隱政策連結（有收集數據就必填）；
- 數據安全表：如實申報收集咗邊啲數據——唔聯網嘅記賬本可以「零收集」；
- 審核：新賬號首次上架好多時會經人工審核，幾個鐘到幾日唔等，被打回頭就照意見改。

國內用戶注意：Google Play 需要特殊網絡環境，而且中國大陸地區分發通常行國內商店（華為、小米、應用寶等）。上架國內商店嘅流程類似，但各商店有自己嘅審核口味——同一個 AAB/APK 可以多間店一齊分發。作為練手，先行通一間就已經贏咗。

五、商店素材點樣做：第一印象就係下載量

商店頁係用戶「行店」嗰陣見到嘅櫥窗。三樣素材決定第一印象：

- 圖標：512×512，唔好攞截圖求其算數。冇設計能力就用 AI 工具生成一個簡潔幾何圖形，喺手機桌面縮細之後都仲可以一眼認得出先算合格；
- 截圖：最少 2 張，最好 4-6 張，展示最核心嘅功能場景；每張配一句講人話嘅文案（「一秒記一筆」好過「高效嘅記賬解決方案」十倍）；
- 描述：第一句講清楚「呢個係乜嘢 + 畀邊個用」，唔好亂堆「最專業、最勁」——商店搜索權重主要睇關鍵詞同下載轉化，真誠好過誇張。

小貼士：A/B 睇吓同行

上架之前行一圈商店入面同類 App 嘅前十名，截圖嚟對比：佢哋啲圖標係咩風格？截圖講咩賣點？標題點寫？模仿唔係抄襲——學套路、換自己做嘅內容，呢個係所有產品冷啟動最快嘅一課。

五、性能優化同崩潰上報小清單

App 上架之前，呢兩件事值得各花半個鐘：

性能自查四條

1. 列表用 LazyColumn：長列表千祈唔好用 Column 硬排——懶加載係安卓性能嘅第一課；
2. 圖片唔好硬塞原圖：用 Coil 圖片庫自動縮放同緩存，一張 5MB 嘅原圖足以卡爆低端機；
3. 避免喺主線程做重活：數據庫、網絡、大檔案都擺喺協程度做；
4. 用 Profiler 度一度：Android Studio 自帶嘅性能分析器睇到內存、CPU、耗電——先量度，再優化，唔好齋靠感覺。

崩潰上報

你測試嗰陣冇崩，唔代表用戶部手機唔崩。上架之前接入 Firebase Crashlytics（谷歌官方，免費）：幾行配置，用戶崩潰嗰陣自動上報堆疊，你喺後台就會見到「邊個型號、邊個版本、崩喺邊一行」。冇崩潰監控就上架，等於瞇埋眼揸車。

嚟動手：將記賬本「交付」一次

(1) 幫 parseAmount 呢啲邏輯補 3 個單元測試；(2) 用 Logcat 親手捉一個自己整出嚟嘅崩潰然後修好佢；(3) 生成一個 release 簽名嘅 APK，發畀朋友安裝（呢個先係「作品」嘅儀式感）；(4) 有餘力就註冊開發者賬號，將記賬本真正提交上架。前三條今晚就做得完。

本章小結

- 除錯三板斧：Logcat 睇日誌、斷點睇變數、Layout Inspector 睇界面
- 讀崩潰：由堆疊下面往上搵自己包名嘅第一行
- 測試三層：單元（秒級，先學呢個）/儀器/手動
- 發布必須簽名：keystore 同密碼備份好，唔見咗個 App 就等於「換咗人」喇
- 上架六步：註冊素材上傳 AAB 數據安全表審核上架

第五十七章：進階之路——權限、通知同現代架構

做到 *App* 出嚟只係開始。呢一章同你傾吓由「識做」到「做好」嘅幾道坎：點樣優雅咁申請權限、點樣發通知唔惹人嬲、點樣將架構再升高一層，仲有——喺 *2026* 年，點樣同 *AI* 拍檔寫安卓代碼。每一條都係你喺真實項目入面即刻會遇到嘅。

一、運行時權限：先敲門，再入門

相機、定位、通訊錄、通知……呢啲「敏感地帶」，安卓要求 App 喺運行時向用戶申請（Android 6 起）。規矩係：先聲明（Manifest），再申請（代碼），用戶拒絕咗就要優雅降級。喺 Compose 用官方庫 Accompanist 嘅 rememberPermissionState 最省心：

```
// Manifest 入面先聲明：
// <uses-permission android:name="android.permission.CAMERA" />

@Composable
fun CameraButton() {
    val permission = rememberPermissionState(
        permission = android.Manifest.permission.CAMERA
    )
    Button(onClick = {
        when {
            permission.status.isGranted -> openCamera()
            permission.status.shouldShowRationale -> showExplainDialog()
            else -> permission.launchPermissionRequest()
        }
    }) { Text("影相") }
}
```

注意：畀人永久拒絕咗點算？

用戶撳咗「不再詢問」之後，launchPermissionRequest 就唔會再彈窗。嗰陣應該引導用戶去系統設定度手動開啟（用 Intent 跳去應用設定頁），並且喺界面講明「呢個權限用嚟做乜嘢」。態度好啲，用戶開權限嘅意欲會高好多——申請權限嘅文案，值得認真寫。

二、通知：喺啱嘅時間講有用嘅嘢

通知係 App 接觸用戶嘅通道，亦都係最容易被用戶「拉黑」嘅功能。發通知三條紀律：齋發有用嘅、畀用戶關得到、唔好半夜嘈人。基礎用法：

```
// Android 13+ 必須先申請 POST_NOTIFICATIONS 權限（見上一節）

// 1. 建立通知渠道（Android 8+ 必須）
val channel = NotificationChannel(
    "reminder", "提醒", NotificationManager.IMPORTANCE_DEFAULT
)
val manager = getSystemService(NotificationManager::class.java)
manager.createNotificationChannel(channel)

// 2. 發通知
val notification = NotificationCompat.Builder(this, "reminder")
    .setSmallIcon(R.drawable.ic_notification)
    .setContentTitle("記賬提醒")
    .setContentText("今日仲未記賬喎")
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    .build()
if (ActivityCompat.checkSelfPermission(this,
        Manifest.permission.POST_NOTIFICATIONS) == PackageManager.
    PERMISSION_GRANTED
) {
    manager.notify(1, notification)
}
```

定時任務（例如「每日 20:00 提醒記賬」）用 WorkManager——佢係官方推薦嘅「後台調度管家」，可以保證任務喺合適時機執行，唔使自己操心電池優化。鬧鐘類需求用佢實冇錯。

三、架構進階：依賴注入同模組化

項目變大之後，兩個「甜蜜嘅煩惱」會自己搵上門：

- 物件創建亂：每個頁面都喺度自己 new 數據庫、new API——用 Hilt（依賴注入框架）統一管理，「邊個要乜嘢，開聲就有人遞」；
- 代碼堆成山：單模組項目越改越慢——拆做多模組（app / core / feature-xxx），各功能獨立開發、獨立測試。

小貼士：唔好過度設計

Hilt 同模組化都係好工具，但單人項目唔好咁快上。記賬本單模組 + 手動創建依賴完全夠用。等代碼量超過幾萬行、編譯開始變慢、多人協作嗰陣，先至按需引入。工具係服務目標嘅，唔係攞嚟炫耀嘅——呢個都係我哋呢本 AI 編程書入面嘅「簡單至上」原則。

四、2026 年嘅新玩法：AI 輔助安卓開發

2026 年，AI 已經深度嵌入咗安卓開發流程。如果你想提速，呢套組合拳好成熟：

環節	AI 點樣幫	你嘅職責
寫代碼	Android Studio 內置 Gemini/第三方 AI 插件嚟補全同生成	描述清楚需求，審查產出
查資料	將官方文檔同報錯信息掟畀 AI 解讀	關鍵結論返去官方文檔核實
寫測試	叫 AI 按「正常/邊界/異常」三類生成用例	確認用例符合業務語義
改 bug	貼堆疊 + 相關代碼，AI 定位修復	睇明改動，行測試驗證

AI 輔助安卓開發嘅四個場景

具體方法論——點樣畀 AI 下指令、點樣驗收、點樣避坑——完整嘅 11 章內容喺本書第七篇《AI 編程實戰篇》（第 37-47 章）入面，強烈建議配住睇：安卓語法交畀呢一篇，AI 協作嘅心法交畀第七篇。

五、Compose 性能優化同除錯工具

Compose 寫起上嚟好爽，但「重組過多」都會卡。三個實用技巧：

- 穩定參數：畀數據類加 @Immutable 或者用 derivedStateOf，避免無關狀態變化觸發大面積重組；
- 記得用 key：items(list, key = { it.id }) 令列表增刪嗰陣齋郁應該郁嗰啲項——唔加 key，刪一項可能成個表重畫；
- Layout Inspector 嘅 Compose 模式：實時睇每個 Composable 嘅重組次數，邊個頻密重組一目了然，照住熱點嚟優化。

另外將 Compose 編譯器報告（Metrics）打開，可以生成一份「邊啲組件唔穩定」嘅報告——進階路上，數據永遠比感覺可靠。

嚟動手：畀記賬本加「三件套」

(1) 畀記賬本加「每日 20:00 記賬提醒」（WorkManager + 通知渠道 + Android 13 通知權限）；(2) 將數據庫同 API 嘅創建改成 Hilt 注入；(3) 試吓用 AI 工具令「月度篩選」功能提速實現，然後按三層驗收法檢查產出。三件套做完，你個項目就有「正規軍」嘅樣喇。

本章小結

- 敏感權限：Manifest 聲明 + 運行時申請 + 被拒之後優雅引導
- 通知三紀律：齋發有用嘅、可以關、唔嘈人；渠道 + 權限缺一不可
- 定時任務用 WorkManager，唔好自己重新發明車轆
- Hilt 管物件、模組化管代碼——單人小項目唔好咁快上
- AI 輔助四場景：寫碼、查資料、寫測試、改 bug；方法論見第七篇（第 37-47 章）

第五十八章：速查手冊同資源

呢一章係一份可以獨立使用嘅速查手冊。當你記唔清某個寫法或者需要快速決策嗰陣，直接揭到嚟呢度。建議當佢係「口袋卡片」：常用代碼速查、常見問題、學習資源，一頁都唔漏。

一、常用代碼速查卡

要做乜嘢	關鍵代碼
記住一個可變狀態	var x by remember { mutableStateOf(初值) }
轉屏都唔會冇咗狀態	rememberSaveable { mutableStateOf(初值) }
直排/橫排佈局	`Column { }/Row { }`
懶加載列表	LazyColumn { items(列表) { ... } }
頁面跳轉	navController.navigate(" 路由")
喺協程入面做要等嘅嘢	`viewModelScope.launch { ...}`
觀察數據流	`val list by flow.collectAsState()`
儲存小設定	context.dataStore.edit { it[KEY] = 值}
存結構化數據	Room 三件套：@Entity / @Dao / @Database
聯網攞數據	Retrofit 接口 + suspend 函數
發通知	NotificationChannel + NotificationCompat.Builder
打日誌	`Log.d("TAG", "message")`

安卓開發速查卡（2026 年 8 月版）

二、常見問題 FAQ

Q1：我完全唔識編程，學安卓開發要幾耐？

跟本書嘅路線行：第 1-2 周裝環境、學 Kotlin 基礎；第 3-4 周學 Compose 做到界面出嚟；第 5-6 周完成記賬本項目；第 7-8 周做完天氣項目。兩個月，由零到識得獨立做小 App。前提係每日投入 1-2 個鐘，而且要動手敲代碼，唔係齋睇書。

Q2：Java 仲使唔使學？

唔使。新項目、新教程、新崗位基本上都以 Kotlin 為準。只有「維護十年老項目」嗰陣先會遇到大量 Java。學完 Kotlin 之後如果工作需要，再補 Java 會好快——兩者底層係同一個虛擬機生態。

Q3：模擬器定係真機？

開發除錯用模擬器方便（有快照、多機型），但驗收一定要過真機——觸感、性能、真網絡、感應器，模擬器代替唔到。條件許可嘅話，買一部中端真機做測試機，係最劃算嘅投資。

Q4：安卓開發容唔容易搵到工？

2026 年嘅行情：齋「識寫界面」嘅初級崗位競爭激烈，但「識 Kotlin + 懂架構 + 有上線作品」嘅開發者依然搶手。學習期嘅你，最好嘅履歷就係商店入面嗰個自己做嘅 App。

Q5：遇到報錯卡住咗點算？

黃金三步：(1) 將報錯第一行原原本本攞去搜 StackOverflow/搜尋引擎；(2) 查官方文檔對應頁面；(3) 仲係唔得，就將「期望行為 + 實際現象 + 報錯全文 + 相關代碼」打包發畀 AI 或者社區發問。絕大多數問題，第一步就搞掂。

三、推薦學習資源

類型	資源	推薦理由
官方	developer.android.com	安卓官方文檔，Compose/Kotlin 教程都喺入面，第一權威
官方	Kotlin 官網 (kotlinlang.org)	語言官方文檔 + 網上演練場
官方	Google Codelabs	手把手嘅分步實驗課，新手友好度滿分
社區	Stack Overflow (android 標籤)	報錯求救第一站
社區	掘金 / CSDN 安卓專區	中文實戰文章多，注意分辨新定舊（先睇日期）
影片	Google Developers 官方頻道	每年嘅開發者大會新技術講解
配套	第七篇《AI 編程實戰篇》（第 37-47 章）	用 AI 輔助安卓開發嘅完整心法

安卓開發學習資源清單（2026 年 8 月）

四、Gradle 常用命令速查

平時撚掣就夠，但命令行有時更高效（尤其係配合 AI 工具或者 CI 流水線嗰陣）：

命令	作用
`./gradlew assembleDebug`	打一個除錯版 APK（Windows 記得將./ 攞走）
`./gradlew assembleRelease`	打發布版（要配好簽名）
`./gradlew test`	行晒全部單元測試
`./gradlew lint`	靜態檢查：搵出潛在 bug 同唔規範嘅寫法
`./gradlew clean`	清空構建產物（「重啟大法」嘅編譯版）
`./gradlew :app: dependencies`	睇依賴樹（排查版本衝突神器）

Gradle 常用命令速查

小貼士：構建出問題先 clean

「尋日仲行得今日又唔得」嘅靈異事件，十件有六七件係構建緩存錯亂。./gradlew clean 之後重新構建，可以醫好一大半玄學問題。仲唔得就重啟 Android Studio——呢招請配合 File Invalidate Caches 一齊服用。

五、結語：由第一個 App 到第 N 個 App

七、結語：由第一個 App 到第 N 個 App

返轉頭睇，你由「唔知 Activity 係乜嘢」，到裝好環境、識咗 Kotlin、用 Compose 畫出界面、畀 App 接上數據庫同網絡，再到測試、打包、上架——呢條路上你已經行完咗由零到一最難嗰一段。

技術嘅世界變得好快：框架會升級，工具會換代，AI 會接手越嚟越多「敲鍵盤」嘅嘢。但有一樣嘢唔會過時：你將一個朦朧嘅想法，拆解成清晰嘅步驟，再一步一步實現佢嘅能力。工具係放大器，思路係地基。放大器可以換，地基唔可以鬆動。

你唔再只係" 識玩手機嘅人"。你可以將腦入面嘅諗法，變成喺三億部安卓裝置上面真實運行嘅 App。由今日開始，用鍵盤去創造——呢個就係安卓開發者嘅浪漫。

參考文獻

1. Android Developers. (2026). Android Developer Guides and Jetpack Compose Documentation. developer.android.com.
2. Kotlin Foundation. (2026). Kotlin Language Documentation. kotlinlang.org.
3. Android Open Source Project. (2026). Android Platform Version History and API Levels.
4. Google Play. (2026). Play Console Help: Publishing and App Signing.
5. AndroidX Release Notes. (2026). Room / DataStore / Navigation / WorkManager releases.

6. Square. (2026). Retrofit Documentation. github.com/square/retrofit.

九、iOS 開發實戰篇（第 59-69 章）

第五十九章：認識 iOS——蘋果生態同開發生態

恭喜你做出呢個決定——由「用 *iPhone* 嘅人」變成「畀 *iPhone* 寫應用嘅人」。動手之前，我哋先攤開張地圖：*iOS* 由邊度嚟？蘋果嘅「圍牆花園」對開發者嚟講即係乜嘢？一個 *App* 由你隻手敲鍵盤去到用戶部手機，中間發生咗乜嘢事？

一、iOS 嘅身世：一部手機改變一個行業

2007 年 1 月，喬布斯喺發布會上面攞咗第一代 iPhone 出嚟。嗰陣佢連第三方應用都唔支援——直到 2008 年，蘋果推出 App Store 同 iPhone SDK，准許全世界開發者畀 iPhone 寫軟件。嗰一刻，「應用經濟」就誕生咗：到今日，App Store 上面有幾百萬個應用，為開發者累計分帳超過幾千億美元。

系統本身都喺度進化：最初叫 iPhone OS，2010 年改名 iOS；版本號一路行到 iOS 17（2023），跟住喺 2025 年，蘋果將命名改做跟年份——iOS 26 於 2025 年發布，2026 年 8 月嘅當前版本係 iOS 27。命名會變，但有一件事冇變過：iOS 嘅用戶付費習慣同軟件質素口碑，係移動生態入面公認最好嘅——同一個 App，iOS 端嘅收入往往明顯高過安卓端。

節點	年份	對開發者重要嘅事
App Store 上線	2008	第三方開發生態開啟
Swift 發布	2014	現代 iOS 開發語言登場
SwiftUI 發布	2019	聲明式界面框架，本書主線
iOS 26（改名元年）	2025	命名跟年份；Apple Intelligence 全面整合
iOS 27	2026	當前最新版（本書基準）

iOS 開發大事記（截至 2026 年 8 月）

二、圍牆花園：蘋果生態嘅「規矩」同「紅利」

蘋果生態成日畀人形容做「圍牆花園」（Walled Garden）——規則多、管得嚴：App 必須審核先可以上架、開發必須用 Mac 同 Xcode、分發主要行 App Store。聽落好似好屈？但對開發者嚟講，呢套規矩換到嚟真金白銀嘅紅利：

- 設備碎片化極細：iPhone 型號就得嗰幾款，屏幕尺寸、系統版本高度統一——你適配 5 部機型，就已經覆蓋咗 95% 嘅用戶（安卓可能要適配幾十款）；
- 系統版本更新率極高：新系統發布幾個禮拜之內就有七八成用戶升級，新 API 可以大膽用，唔使遷就「五年前嘅老系統」；
- 用戶付費習慣好：App Store 用戶更願意為應用同內購畀錢；
- 審核雖然嚴格，但商店乾淨：山寨、流氓軟件少，好產品更容易畀人睇到。

注意：先講清楚成本

iOS 開發有硬門檻：必須有一部 Mac（Xcode 係 macOS 獨佔），上架需要 99 美元/年嘅開發者賬號（個人開發者免費賬號可以本地真機除錯，但唔可以上架）。諗清楚呢筆投入先啟程——不過好消息係，學習階段用免費賬號完全夠用，第 60 章會講。

三、用乜嘢語言寫：Objective-C 定係 Swift？

iOS 開發歷史上都有兩位「主角」。Objective-C 係開國元老——由 2008 年用到 2014 年；Swift 係蘋果 2014 年發布嘅新語言，之後一路進化到 Swift 6（2024 年發布，引入嚴格並發檢查），到 2026 年，新項目幾乎全部用 Swift。

畀零基礎嘅你一句實在話：直接學 Swift，Objective-C 連睇都唔使睇——除非你入職之後要維護十年前嘅遺產代碼。Swift 嘅好處：語法清爽（冇 C 語言年代嘅分號同頭檔案包袱）、可選值（Optional）機制由語言層面消滅「空指標崩潰」、同 SwiftUI 無縫配合、仲有 iPad 上面嘅 Swift Playgrounds 可以邊玩邊學。

Objective-C 風格（上古遺跡）

```
NSString *name = @"张三"; [name stringByAppendingString:@"你好"]; // 方括號 + 指標 + 分號三件套
```

Swift 風格（現代清爽）

```
var name = "张三" name += "你好" // 似自然語言咁直白
```

四、一個 App 係點樣誕生嘅？

同所有平臺一樣，iOS App 嘅誕生都有一條流水線。但蘋果版多咗兩道「安檢」：

想法

設計（界面 + 功能）

寫代碼（Swift + SwiftUI）

編譯簽名

測試（模擬器 + 真機）

蘋果審覈

上架 App Store

圖 1：一個 iOS App 由諗法到上架嘅全流程（注意多出嘅「審核」環節）

流程入面嘅關鍵概念：

- 簽名（Signing）：每個 iOS App 都必須「簽名」先可以安裝——證明「呢個包出自你手，冇畀人改過」。學習階段 Xcode 會用你嘅 Apple ID 自動簽名；上架嗰陣用開發者證書。第 67 章會詳細講。
- 模擬器（Simulator）：喺 Mac 上面虛擬出嚟嘅 iPhone，係開發除錯嘅主戰場。佢行嘅係 Mac 嘅 CPU，所以特別暢順，體驗好過安卓模擬器一大截。
- TestFlight：蘋果官方嘅「內測分發」工具——上架之前將測試版發畀最多 10000 名測試用戶，佢哋用 TestFlight 應用安裝試用。呢個係 iOS 生態獨有嘅優雅。

五、iOS 技術棧全景：你嘅「武器清單」

層次	主流選擇（2026）	用嚟做乜嘢
語言	Swift 6	寫邏輯嘅語言
界面	SwiftUI ＋ 蘋果設計規範（HIG）	用代碼畫界面（本書主線）
架構	MVVM + @Observable	將界面同邏輯分開
數據存儲	SwiftData（數據庫）＋ UserDefaults（配置）	將數據存入手機
網絡	URLSession ＋ async/await + Codable	同服務器對話
構建工具	Xcode Build System / xcodebuild	將代碼變成安裝包
開發環境	Xcode（macOS 獨佔）	蘋果官方 IDE

2026 年 iOS 主流技術棧速覽

注意到未：呢套清單比安卓嘅短一截——因為蘋果將好多嘢都「包辦」咗。封閉生態嘅代價係自由度，回報係省心：唔使糾結揀邊個網絡庫、邊個數據庫，官方畀嘅往往就係最優解。

嚟動手：睇吓你部手機嘅「蘋果戶口」

打開 iPhone 嘅 [設定] [一般] [關於本機]，記低三樣嘢：iOS 版本、機型名稱、可用容量。再去 App Store 嘅「今天」頁求其行吓，留意嗰啲精品 App 嘅截圖同文案——由今日起，你行商店嘅視角要由「用戶」轉做「開發者」喇。

本章小結

- 2008 年 App Store 上線，「應用經濟」誕生；2026 年當前版本係 iOS 27
- 圍牆花園條數：規矩多，但設備少、版本新、用戶肯畀錢、商店乾淨
- 硬門檻：必須有 Mac；上架要 99 美元/年開發者賬號；學習階段免費賬號夠用
- 語言直接學 Swift；Objective-C 只屬於遺產代碼
- 流水線比安卓多一環：蘋果審核；TestFlight 係內測分發神器
- 2026 標準技術棧：Swift 6 + SwiftUI + MVVM + SwiftData + URLSession + Xcode

第六十章：搭建開發環境——Xcode 同第一個項目

工欲善其事，必先利其器。呢一章我哋將個「廚房」搭好：裝好 *Xcode*、行通模擬器、建立人生第一個 *iOS* 項目，再將工程結構拆開睇個明白。裝好環境之後，你會第一次見到自己寫嘅代碼變成一個攞得到嘅掣——呢個瞬間，好多人會記一世。

一、準備一部 Mac

先講門檻：iOS 開發必須喺 macOS 上面進行。如果你冇 Mac，選擇有三：買一部（就算係二手 Mac mini 或者 MacBook Air 都夠用）、短期租雲 Mac、或者先讀第八篇《安卓開發實戰篇》（Windows 就得）。

配置項	最低要求	舒適線（推薦）
晶片	Apple Silicon（M1 起）	M2 或更新
記憶體	8GB	16GB
磁碟	預留 40GB+（Xcode 本體好肥）	256GB 起步
系統	最新 macOS	保持自動更新

Mac 開發機配置參考（2026 年 8 月）

小貼士：冇 Mac 都可以先學語言

iPad 上面免費嘅 Swift Playgrounds 可以學 Swift 語言基礎（唔使 Mac）；甚至 iPhone 上面都裝到。先將第 61 章嘅語言知識喺 Playgrounds 度玩熟，等攞到 Mac 之後就即刻起飛。磨刀唔誤砍柴工，講嘅就係呢樣。

二、安裝 Xcode

Xcode 係蘋果官方嘅集成開發環境，免費。三步裝好：

1. 喺 Mac 打開 App Store，搜「Xcode」，撳安裝——留意佢有成幾個 GB，搵個網速好嘅時段；

2. 裝完第一次啟動，同意協議、裝完組件（自動完成）；
3. 選單 Xcode Settings Locations，確認 Command Line Tools 已經揀咗——命令行工具係好多腳本同第三方工具嘅前提。

> **注意：版本同系統嘅「配對遊戲」**
>
> 新 Xcode 好多時都要求最新 macOS，舊 Mac 就會卡喺舊版本。上架 App Store 嗰陣，蘋果會要求用較新嘅 Xcode 構建（通常會畀一段寬限期）。所以系統更新唔好拖太耐——iOS 開發者係「追新版」最勤力嘅一群人。

三、建立你嘅第一個項目

打開 Xcode Create New Project 揀 iOS App 模板，跟住填：

配置項	填乜嘢	說明
Product Name	MyFirstApp	應用名
Team	你嘅 Apple ID（免費）	簽名用，個人賬號就得
Organization Identifier	com.example	同 Product Name 拼埋組成包標識
Interface	SwiftUI	本書主線框架
Language	Swift	唔使解釋

建立項目要填嘅五樣嘢

撳 Create 之後，Xcode 會生成一個可以直接行嘅項目。左上角揀一部模擬器（例如 iPhone 17），撳 ▶ 運行——幾秒之後，模擬器屏幕上面會出現一個寫住 “Hello, world!” 嘅窗口。你嘅第一個 iOS App 已經誕生咗喇。

四、工程結構拆解：每個檔案係做乜嘢嘅

左邊係工程導航器，新手只需要識呢幾個：

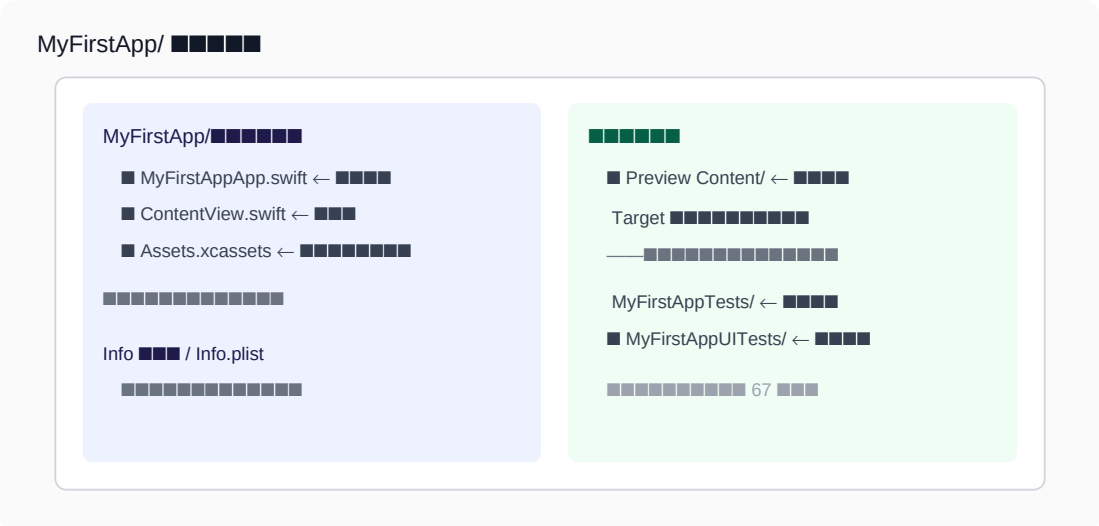


圖 2：新手必識嘅 Xcode 工程結構

- MyFirstAppApp.swift：程式入口，喺 SwiftUI 入面佢只有幾行——「啟動，跟住顯示主界面」；
- ContentView.swift：主界面視圖，你而家見到嘅一切都在呢度；
- Assets.xcassets：資源倉庫——圖標、圖片、顏色都擺喺呢度；
- Target 設定：撳左邊藍色工程圖標入去，版本號、最低 iOS 版本、簽名都在入面。App 顯示名、圖標都係喺呢度配置。

五、真機除錯：將 App 裝入你嘅 iPhone

五、Xcode 效率技巧：新手嘅五個「快捷鍵外掛」

用熟呢幾個，你會顯得「好專業」：

操作	快捷鍵	作用
運行	CmdR	編譯並且行起上嚟
打開/切換畫布預覽	OptionCmd	所見即所得嘅開關
格式化縮排	CtrlI	自動整理選中代碼嘅縮排
快速打開檔案	CmdO	輸入檔案名即刻開到（最常用！）
註釋/取消註釋	Cmd/	除錯嗰陣順手開關代碼

Xcode 高頻快捷鍵（新手五件套）

另外兩個隱藏福利：多開模擬器（同時行 iPhone 同 iPad 對比佈局）同代碼片段庫（將常用嘅幾行代碼拖入右邊 Snippets 面板，下次一鍵插入）——效率工具先用起上嚟，手感自然就上嚟喇。

六、真機除錯：將 App 裝入你嘅 iPhone

模擬器雖然好，但真機先係終點。用免費 Apple ID 都可以真機除錯：

1. 數據線駁上 iPhone（或者同 Wi-Fi 無線除錯）；
2. Xcode 頂部設備列表揀你部手機，第一次會提示登入 Apple ID 並且自動配置簽名；
3. iPhone 上面 [設定] [一般] [VPN 與設備管理] 信任你嘅開發者證書；
4. 撳 >，App 就裝入真機。

注意：免費賬號嘅「七日限制」

免費 Apple ID 簽名嘅 App 7 日之後會失效（重新運行一次就可以續期）。呢個係蘋果嘅防濫用機制，學習階段完全夠用；付費開發者賬號（99 美元/年）就冇呢個限制，第 67 章會詳細講。

七、小試牛刀 + 常見報錯急救

九、小試牛刀 + 常見報錯急救

打開 ContentView.swift，將 Text(“Hello, world!”) 入面嘅字換成你個名，撳 CmdR 重新運行——幾秒之後模擬器上面就會顯示你個名。恭喜，你完成咗第一次完整嘅「改代碼編譯安裝運行」循環。

報錯現象	常見原因	解藥
Signing 報錯	Team 冇揀、證書過期	喺 Target Signing 度剔返你嘅賬號；免費賬號重新行一次
模擬器黑屏/開唔到	映像損壞	選單 Device Erase All Content；或者刪咗佢重新加返模擬器
Xcode 下載慢/裝唔到	App Store 網絡波動	轉個時段；或者去蘋果開發者網站下載.xip 離線包

新手環境報錯急救表

嚟動手：完成你嘅「開發儀式感」清單

(1) 裝好 Xcode 並且行通模擬器；(2) 用真機行一次自己嘅 App 並且截圖留念；(3) 將 Hello 文字改成你個名；(4) 去 Target 設定度將 App 顯示名改成「我嘅第一個 App」。做完呢四步，你已經係一個有作品嘅人喇。

本章小結

- 硬門檻：Mac + Xcode；冇 Mac 就先用 iPad 上面嘅 Swift Playgrounds 學語言
- Xcode 由 App Store 裝；新版本要求新系統，更新唔好拖
- 新建立項目五要素：名、Team、組織標識、SwiftUI、Swift
- 工程口訣：代碼都在 MyFirstApp/ 目錄；Target 設定管版本同簽名
- 免費 Apple ID 真機除錯有 7 日簽名限制；付費賬號冇呢個限制
- 開發日常 = 改代碼 CmdR 睇效果，循環不息

第六十一章：Swift 快速上手——蘋果開發嘅第一語言

呢一章我哋正式學語言。唔使緊張——*Swift* 係出名「對新手友好」，而且你唔需要背語法書：先掌握 *20%* 嘅語法，就已經睇得明同寫得出 *80%* 嘅日常代碼。淨低嘅一邊做項目一邊查。記住我哋嘅學習原則：睇得明 > 記得住，識用 > 精通。

一、變數同類型：畀數據改個名

程式就係「數據 + 邏輯」。變數，就係畀數據改個名儲起佢。Swift 用 let 同 var 嚟聲明：

```
let name = "張三"          // let：常量（推薦預設用呢個）
var age = 18               // var：變數（之後仲可以改）

age = 19                   // <i class="fas fa-check-circle mr-1"></i> var 可以改
// name = "李四"           // <i class="fas fa-times-circle mr-1"></i> let 唔可以改，
    編譯報錯

let score: Int = 100       // 都可以明確寫明類型
let pi = 3.14              // 唔寫都會自動推斷

print("你好，\(name)")     // 字串插值：反斜槓 + 括號
```

常用類型一張表記住：

類型	存乜嘢	示例
Int	整數	42
Double	小數	3.14
String	文字	” 你好”
Bool	真/假	true / false
[Int]（陣列）	一組數據	[1, 2, 3]

Swift 常用基礎類型

小貼士：let 優先原則

Swift 社區嘅鐵律：預設用 let，真係有需要先用 var。可以用 let 嘅地方用 var，Xcode 會出黃字警告你。呢個習慣幫你避開一大類「變數畀人意外改咗」嘅 bug——代碼變化越少，就越容易推理。

二、可選值（Optional）：Swift 嘅“防崩護身符”

呢個係 Swift 最重要嘅概念，冇之一。一句講晒：當一個值可能“唔存在”嗰陣，Swift 會迫你處理佢——喺語言層面消滅咗“空指針崩潰”呢個行業噩夢。

```
var name: String = "張三"    // 普通類型：絕對唔會係 nil
// name = nil                // <i class="fas fa-times-circle mr-1"></i> 編譯錯誤，直接擋住

var nickname: String? = nil  // 加個問號：允許爲 nil（可選值）
nickname = "三兒"

// 用可選值，必須"解包"：
if let n = nickname {        // if let：安全解包
    print("你好，\(n)")
} else {
    print("仲未有暱稱")
}

let len = nickname?.count ?? 0   // ?? 畀預設值（空合運算符）
print(len)
```

注意：！係”高危操作”

代碼入面寫 nickname!（強制解包）等於對編譯器嗌“我保證佢唔係 nil”——一估錯，App 就會即場崩潰。新手期原則：盡量用 if let / ??，唔好用!。Xcode 補全代碼嗰陣有時會順手加個!，記得改返佢。

三、函數同閉包：將邏輯打包成“按鈕”

函數係一段有名嘅邏輯，好似搖控器上面嘅按鈕。閉包（Closure）就係“冇名嘅函數”，可以隨手遞畀人：

```
// 函數：名字喺前，參數同返回類型喺後
func greet(name: String) -> String {
    return "你好，\(name)！"
}
print(greet(name: "張三"))      // 調用時要帶參數名

// 閉包：就地定義，輕快直接
```

```
let numbers = [1, 2, 3, 4]
let doubled = numbers.map { $0 * 2 }   // $0 = 當前元素
// [2, 4, 6, 8]

let big = numbers.filter { $0 > 2 }    // 篩選
// [3, 4]
```

SwiftUI 入面閉包無處不在：按鈕嘅 Button(“點我”) { ... } 後面嗰對大括號就係一個閉包。map、filter、sorted 呢幾招“集合連招”請記實——將來 90% 嘅列表處理都係佢哋嘅組合。

四、結構體同協議：Swift 嘅“樂高”體系

Swift 官方嘅偏好好明確：優先用結構體（struct），要到需要繼承嗰陣先用類（class）。結構體係“值類型”——複製嗰陣各自各嘅，天然少一類 bug：

```
// 結構體：裝數據嘅盒
struct Record {
    var id = UUID()
    var amount: Double
    var category: String
    var date: Date

    // 結構體自帶"逐一成員構造器"：直接就建到
}
let r = Record(amount: 25.5, category: "餐飲", date: Date())

// 協議：約定"必須具備邊啲能力"
protocol Identifiable {
    var id: UUID { get }
}
struct Record: Identifiable { ... }   // 遵守協議 = 承諾實現其要求
```

SwiftUI 入面幾乎所有嘢都係 struct（View 就係協議）。呢句而家唔明唔緊要，第 62 章會日日見面——喺 Swift 嘅世界入面，先擁抱 struct。

五、寫畀零基礎嘅語法速查卡

想做乜嘢	寫法
定義常量 / 變數	`let x = 1` / `var x = 1`
字串插值	”你好，\(name)”
定義函數	func 名字 (參數: 類型) -> 返回類型 { ... }
判斷	if 條件 { ... } else { ... }
遍歷	for x in 陣列 { ... }
安全解包	if let x = 可選值 { ... }
預設值	x ?? 預設值
定義結構體	`struct X { var a: Int; var b: String }`
閉包	`{ $0 * 2 }`

Swift 語法速查卡（新手版）

嚟動手：喺 Playgrounds 玩 15 分鐘 Swift

有 Mac 就開 Xcode Playground；冇 Mac 就用 iPad 上嘅 Swift Playgrounds 或者網頁版演練場。將本章每一段代碼都行一次，改改數字同文字，故意整幾個錯誤睇吓 Xcode 點提示。呢個“先玩後學”嘅 15 分鐘，會令你第 62 章上手嗰陣輕鬆一倍。

本章小結

- let 優先，var 慎用；類型自動推斷，冒號後面顯式聲明都得
- 可選值係 Swift 嘅靈魂：可能係空就必須處理，if let / ?? 係護身符，! 係高危操作
- 函數調用要帶參數名；閉包 = 冇名嘅函數，SwiftUI 入面無處不在
- 優先 struct，遵守協議（Protocol）；SwiftUI 入面幾乎所有嘢都係 struct
- 20% 語法做 80% 嘅嘢：先用得，再精通

第六十二章：界面開發——SwiftUI 入門

界面係用戶睇到嘅一切。*SwiftUI* 係蘋果 *2019* 年推出嘅聲明式界面框架——你只需要“聲明”界面生乜嘢樣，系統自動負責渲染、適配深色模式、適配各種螢幕。到 *2026* 年，*SwiftUI* 已經係新項目嘅絕對主流。呢一章，我哋學識“畫畫”。

一、SwiftUI 嘅思想：聲明式，唔係命令式

舊式界面開發係“命令式”：先話畀系統聽“建立按鈕放喺 (20,30) 設定顏色”，一步一步指揮。SwiftUI 係“聲明式”：你直接描述“界面上面有一個紅色按鈕寫住’點我’”，點樣畫、幾時重畫，系統自己搞掂。

```
import SwiftUI

struct ContentView: View {
    var body: some View {      // body：描述"界面係乜嘢樣"
        VStack {               // 直向排列
            Text("你好，世界！")
                .font(.largeTitle)      // 修飾符：大標題
            Button("攆我") {
                print("被攆咗")
            }
            .buttonStyle(.borderedProminent)  // 藍色實心按鈕
        }
        .padding()
    }
}
```

關鍵點：視圖 = 遵守 View 協議嘅結構體，body 屬性描述界面，. 修飾符 () 鏈式“上妝”。呢個就係 SwiftUI 嘅全部基本盤。

二、基礎視圖：文字、按鈕、輸入框、圖片

視圖	做乜嘢	示例
Text	顯示文字	Text(” 你好”)
Button	按鈕	Button(” 點我”) { 動作}
TextField	輸入框	TextField(” 提示”, text: $ 文本)
Image	顯示圖片	`Image(systemName: "star.fill")`

SwiftUI 四大基礎視圖

注意 Image(systemName:)——蘋果內置咗 SF Symbols 圖標庫（幾千個免費圖標），“star.fill”呢類名就可以直接顯示一個實心星星。呢個係 iOS 開發嘅隱藏福利：做界面幾乎唔使搵圖標素材。

三、佈局三件套：VStack、HStack、ZStack

- VStack：打直咁排（好似排隊打飯）；
- HStack：打橫咁排（好似並排坐）；
- ZStack：疊住放（好似撲克牌疊埋一齊，適合做角標、遮罩）。

```
VStack(alignment: .leading, spacing: 12) {
    HStack {
        Text("賬單")
        Spacer()                    // 彈性空間：將右邊推到盡頭
        Text("-25.5 元").foregroundColor(.red)
    }
    Divider()                       // 分隔線
    Text("備註：午餐")
}
.padding(16)                        // 四圍留白
.background(Color(.systemGray6)) // 淺灰底
.cornerRadius(12)                   // 圓角

// 修飾符次序有意義：先 padding 再 background，背景唔包邊距
```

注意：修飾符次序有講究

padding(16).background(灰) 同 background(灰).padding(16) 效果唔同——前者“先留白再塗色”（背景唔包含邊距），後者“先塗色再留白”（背景包含邊距）。新手最常見嘅“奇怪白邊”都係由呢度嚟。唔肯定嗰陣調轉次序，預覽即刻話你知答案。

四、狀態同綁定：令界面“活”起嚟

靜態界面冇乜意思。SwiftUI 嘅規則：界面永遠忠實反映狀態，狀態一變，界面自動重畫。

```
struct CounterView: View {
    @State private var count = 0   // @State：呢個值變咗要重畫

    var body: some View {
        VStack {
            Text("你撳咗 \(count) 次")
            Button("撳我 +1") {
                count += 1          // 改狀態，界面自動刷新
            }
        }
    }
}
```

輸入框要用“雙向綁定”——$ 符號將“值同修改權”一齊傳入去：

```
struct InputView: View {
    @State private var text = ""

    var body: some View {
        TextField("輸入啲乜嘢", text: $text)  // $ 雙向綁定
            .textFieldStyle(.roundedBorder)
        Text("你輸入咗：\(text)")   // 輸入嗰陣即時更新
    }
}
```

用戶操作

狀態改變（count += 1）

SwiftUI 感知到變化

自動重畫界面（body 重新計算）

圖 3：SwiftUI 嘅“狀態界面”單向數據流

五、預覽、主題同動畫：SwiftUI 嘅“外掛”

- #Preview 巨集：喺檔案底部加 #Preview { ContentView() }，右邊畫布即時渲染——改代碼所見即所得；
- 深色模式免費：用系統顏色（Color(.systemGray6)、.primary）而唔係寫死顏色，深色模式自動適配，一行都唔使多寫；
- 動畫一行搞掂：withAnimation { count += 1 } 或者畀視圖加.animation(.default, value: count)——過渡絲滑。

```
struct AnimatedHeartView: View {
    @State private var liked = false

    var body: some View {
        Button {
            withAnimation(.spring(duration: 0.4)) {
                liked.toggle()
            }
        } label: {
            Image(systemName: liked ? "heart.fill" : "heart")
                .font(.system(size: 48))
                .foregroundColor(liked ? .red : .gray)
                .scaleEffect(liked ? 1.3 : 1.0)
        }
    }
}
```

小貼士：先功能後動畫

新手常犯嘅毛病係“界面仲未做啱，就開始糾結動畫”。先令功能行得通，最後先統一加動效——動畫係錦上添花，功能係雪中送炭。呢個都係我哋一貫嘅“簡單至上”原則。

嚟動手：做一個”心情按鈕”

六、常用修飾符速查：一頁在手，界面唔使愁

想做乜嘢	修飾符
留白	`.padding(16)`
圓角	`.cornerRadius(12)`
背景色	`.background(Color(.systemGray6))`
字體同字號	`.font(.title.bold())`/`.font(.system(size: 20))`
前景色（文字 / 圖標）	`.foregroundColor(.red)`
撐滿闊度	`.frame(maxWidth: .infinity)`
固定尺寸	`.frame(width: 100, height: 44)`
對齊方式	`VStack(alignment: .leading)`
透明度	`.opacity(0.6)`
陰影	`.shadow(radius: 5)`
點擊手勢	`.onTapGesture { ...}`
鍵盤類型	`.keyboardType(.decimalPad)`

SwiftUI 常用修飾符速查

記唔住唔緊要——呢一頁就係用嚟查嘅。用得最多嘅其實就係頭四個：padding、圓角、背景、字體。

將模板項目改成一個心情切換器：一個按鈕，撳咗之後喺開心難過嘅之間循環切換，文字同背景色跟住變。提示：用陣列存三組（文字 + 顏色），@State 存下標，撳嗰陣 (index + 1) % 3。做完傳畀朋友玩，收穫第一波“哇你識做 App 喇”。

本章小結

- SwiftUI 係聲明式：描述”生乜嘢樣”，系統負責畫同重畫
- 視圖 = 遵守 View 協議嘅結構體；body 描述界面；修飾符鏈式”上妝”
- 四大基礎視圖：Text / Button / TextField / Image；SF Symbols 圖標庫免費任用
- 佈局三件套：VStack 直排、HStack 橫排、ZStack 疊放；Spacer 係彈性空間
- @State 令界面活起嚟；$ 雙向綁定餵畀輸入框
- #Preview 所見即所得；系統顏色自動適配深色模式；withAnimation 一行動效

第六十三章：視圖、狀態同導航

單頁面嘅玩具你已經識做喇。呢一章解決兩個“長大”問題：點樣喺多個頁面之間跳嚟跳去（導航），以及數據喺頁面之間點樣流動（狀態管理）。學完本章，你就可以搭出“列表詳情編輯”呢種經典嘅多頁面 *App* 骨架。

一、狀態管理嘅“數據流全景”

SwiftUI 嘅狀態工具唔止 @State 一個。先睇呢張全景圖，搞清楚“幾時用邊個”：

工具	管乜嘢	邊個用	典型場景
@State	頁面私有嘅臨時狀態	當前視圖	計數器、輸入框文字
@Binding	借人哋嘅狀態	子視圖改父視圖	自訂組件
@StateObject / @Observable	跨頁面共享嘅邏輯	視圖模型（ViewModel）	數據加載、業務邏輯
@EnvironmentObject / .environment	全局共享嘅狀態	成個 App	用戶登入狀態、主題設定

SwiftUI 狀態工具全景（2026 年，@Observable 已經係主流）

```
// iOS 17+ 嘅現代寫法：@Observable 宏（新建項目建議用）
import Observation

@Observable
class CounterModel {                    // 視圖模型
    var count = 0
    func increment() { count += 1 }
}

struct CounterView: View {
    @State private var model = CounterModel()   // 持有模型

    var body: some View {
        VStack {
            Text("你攰咗 \(model.count) 次")
            Button("攰我") { model.increment() }
```

```
        }
    }
}
```

畀新手嘅粗暴建議：頁面私有嘅用 @State，跨頁面 / 有邏輯嘅用 @Observable 類。舊教程入面嘅 @StateObject/@ObservableObject 係上一代寫法，睇得明就得，新代碼照上面咁做。

二、NavigationStack：頁面跳轉嘅“棧”

iOS 16 開始，多頁面導航用 NavigationStack——好似一疊碟咁，推一個入一個，返回嗰陣逐個彈出嚟：

```
struct HomeView: View {
    var body: some View {
        NavigationStack {
            List {
                NavigationLink("張三嘅賬本") {
                    DetailView(name: "張三嘅賬本")   // 目標頁面
                }
                NavigationLink("李四嘅賬本") {
                    DetailView(name: "李四嘅賬本")
                }
            }
            .navigationTitle("賬本列表")   // 頂部大標題
        }
    }
}

struct DetailView: View {
    let name: String
    var body: some View {
        Text("\(name) 嘅詳情頁")
            .navigationTitle(name)
            .navigationBarTitleDisplayMode(.inline)
    }
}
```

NavigationLink 就係“撳得嘅跳轉連結”；navigationTitle 自動生成導航欄標題同返回按鈕。要編程式跳轉（譬如撳完“保存”自動返回）嗰陣，用 @Environment(\.dismiss) 關閉當前頁面——呢兩招一齊用，90% 嘅導航需求就齊喇。

三、彈窗同底部標籤：sheet 同 TabView

除咗“推頁面”，仲有兩種經典交互：

```
// 1. 底部彈出半屏/全屏（錄入表單最愛）
```

```
struct HomeView: View {
    @State private var showAdd = false
    var body: some View {
        Button("記一筆") { showAdd = true }
            .sheet(isPresented: $showAdd) {
                AddRecordView()      // 彈出嘅錄入頁
            }
    }
}

// 2. 底部標籤欄（App 嘅"骨架"）
struct MainTabView: View {
    var body: some View {
        TabView {
            RecordListView()
                .tabItem { Label("賬本", systemImage: "book") }
            StatsView()
                .tabItem { Label("統計", systemImage: "chart.pie") }
            SettingsView()
                .tabItem { Label("設定", systemImage: "gear") }
        }
    }
}
```

小貼士：幾時用邊種？

經驗法則：主流程用 NavigationStack 推頁面（列表詳情），臨時操作用 sheet 彈窗（新增、編輯、確認），並列嘅幾大塊用 TabView（底部標籤）。三者組合，就係絕大多數 App 嘅骨架。

四、List：iOS 嘅“國民列表”

iOS 上面最常用、最有“蘋果味”嘅組件就係 List——天生帶分割線、滑動刪除、撳咗會高亮：

```
struct RecordListView: View {
    @State private var records = ["午餐 25.5", "地鐵 6.0", "人工 +8000"]

    var body: some View {
        List {
            ForEach(records, id: \.self) { r in
                Text(r)
            }
            .onDelete { indexSet in          // 左掃刪除
                records.remove(atOffsets: indexSet)
            }
```

```
        }
    }
}
```

列表 + NavigationLink + sheet + TabView——將呢四樣組合埋一齊，你已經可以搭出“賬本列表撳開詳情撳 + 彈錄入”嘅完整 App 骨架喇。下一章嘅記賬本項目，就係咁搭嘅。

嚟動手：搭一個”三頁骨架”

五、程式生命週期：前台同後台

同安卓嘅 Activity 生命週期差唔多，iOS App 都有“出生、前台、後台、死亡”嘅狀態變化。SwiftUI 入面用 scenePhase 環境值嚟觀察：

```
@main
struct MyApp: App {
    @Environment(\.scenePhase) private var phase

    var body: some Scene {
        WindowGroup {
            ContentView()
        }
        .onChange(of: phase) { _, newPhase in
            switch newPhase {
            case .active:    print("回到前臺，恢復刷新")
            case .inactive:  print("過渡中（來電/切後臺瞬間）")
            case .background: print("進後臺，保存草稿")
            @unknown default: break
            }
        }
    }
}
```

典型用途：返到前台嗰陣刷新數據，入咗後台嗰陣保存草稿。iOS 對後台運行管得好嚴——普通 App 入咗後台好快就會被掛起（凍結），唔好指望喺後台偷偷哋做重活；真係要定時任務，用第 68 章嘅本地通知，而唔係“後台常駐”。
做一個假數據版本嘅記賬本骨架：TabView 三個標籤（賬本 / 統計 / 設定）賬本頁用 List 顯示假數據、撳行跳詳情（NavigationStack）右上角“+”按鈕彈錄入 sheet。唔存數據、唔上網，純練骨架——呢個骨架做好咗，第 65 章再入去填真數據就水到渠成。

本章小結

- 狀態四工具：@State（頁面私有）、@Binding（借用）、@Observable（邏輯）、.environment（全局）
- 新項目用 @Observable 巨集寫視圖模型；舊教程嘅 @ObservableObject 睇得明就得
- NavigationStack 推頁面；@Environment(\.dismiss) 編程式返回

- 主流程推頁面、臨時操作彈 sheet、並列大塊用 TabView
- List 係”國民列表”：分割線、滑動刪除開箱即用

第六十四章：數據存儲同網絡請求

一個淨係得“睇”嘅 *App* 係死嘅；識存數據、識連網絡嘅 *App* 先至係生嘅。呢一章認識 *iOS* 數據層嘅“三件套”：*UserDefaults*（存小配置）、*SwiftData*（存結構化數據）、*URLSession*（同伺服器對話），仲有解析 *JSON* 嘅 *Codable* 同現代化嘅 *async/await*。學完呢章，你就可以寫到識記住用戶習慣、識實時更新內容嘅真 *App* 喇。

一、存儲方案點樣揀：先睇呢張表

方案	適合存乜嘢	特點	本書邊度用
UserDefaults	零散配置（開關、偏好）	鍵值對，簡單直接	第 65 章設定、第 66 章收藏城市
SwiftData	結構化數據（賬目、記錄）	官方數據庫封裝，@Model 一行聲明	第 65 章賬目記錄
URLSession	（唔存，上網攞）伺服器數據	系統自帶網絡框架，冇第三方依賴	第 66 章天氣數據

數據三件套嘅分工

記一個口訣：“小配置入 UserDefaults，成批數據入 SwiftData，出面嘅數據行 URLSession”。舊教程入面嘅 Core Data 依然用得（SwiftData 就係佢嘅現代馬甲），手寫 SQLite 就基本唔使掂——直接學新嘅，咪行舊路。

二、UserDefaults：存啲小配置

```
// 寫：存一個值
UserDefaults.standard.set(true, forKey: "isDarkMode")
UserDefaults.standard.set("北京", forKey: "favoriteCity")

// 讀：取出來（冇就返回預設值）
let isDark = UserDefaults.standard.bool(forKey: "isDarkMode")
let city = UserDefaults.standard.string(forKey: "favoriteCity") ?? "北京"

// 刪：移除一個鍵
UserDefaults.standard.removeObject(forKey: "isDarkMode")
```

注意佢只係適合零散嘅小配置——幾十上百條記錄請交畀 SwiftData，唔好將 UserDefaults 當數據庫咁用（會拖慢啟動、亦都唔方便查詢）。

三、SwiftData：畀數據安個“真正嘅屋企”

SwiftData 係 iOS 17 開始嘅官方數據庫方案，將“建表、增刪改查”簡化到極致——一個 @Model 註解，數據類就變成數據庫表：

```
import SwiftData

// 1. @Model：呢個類自動變成"表"
@Model
class Record {
    var amount: Double
    var category: String
    var note: String
    var date: Date

    init(amount: Double, category: String, note: String, date: Date) {
        self.amount = amount
        self.category = category
        self.note = note
        self.date = date
    }
}

// 2. App 入口掛上容器
@main
struct MyApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
        .modelContainer(for: Record.self)   // 數據庫就緒
    }
}

// 3. 視圖入面用 @Query 讀、@Environment 寫
struct RecordListView: View {
    @Query(sort: \Record.date, order: .reverse) var records: [Record]
    @Environment(\.modelContext) private var context

    var body: some View {
        List(records) { r in
            Text("\(r.category) \(r.amount) 元")
        }
```

```
    }

    func add(amount: Double, category: String) {
        let r = Record(amount: amount, category: category, note: "", date: Date())
        context.insert(r)     // 存
        try? context.save()   // 落盤
    }
}
```

呢套“@Model + modelContainer + @Query”模板可以原封不動搬入你任何項目——換下字段名就得。數據一變，@Query 自動刷新界面，妙不可言。

四、Codable：畀 JSON“翻譯”

伺服器嘅數據大多係 JSON 文本。Swift 嘅 Codable 協議令“JSON 結構體”嘅轉換自動發生——字段名對得上，魔法就生效：

```
// JSON: {"name":"北京","main":{"temp":26.5},"weather":[{"description":"晴"}]}
struct WeatherResponse: Codable {
    let name: String
    let main: Main
    let weather: [Weather]
}
struct Main: Codable { let temp: Double }
struct Weather: Codable { let description: String }

// JSON 字串  結構體
let decoder = JSONDecoder()
let data = jsonString.data(using: .utf8)!
let result = try decoder.decode(WeatherResponse.self, from: data)
print(result.weather[0].description)   // "晴"
```

鐵律：先睇真實返回嘅 JSON，再定義字段。好多“解析失敗”嘅 bug 都係字段名寫錯、類型估錯（伺服器發“26.5”字串，你聲明成 Double）——將原始 JSON 打印出嚟對照，十秒定位。

五、URLSession + async/await：同伺服器講嘢

iOS 上網用系統自帶嘅 URLSession，配合 Swift 嘅 async/await（異步函數），“等網絡”就好似“等快遞”咁自然：

```
struct WeatherService {
    // 異步函數：async 標記"我要等一陣"，throws 標記"可能失敗"
    func fetchWeather(city: String) async throws -> WeatherResponse {
        let url = URL(string:
            "https://api.example.com/weather?city=\(city)")!
```

```
        let (data, _) = try await URLSession.shared.data(from: url)
        return try JSONDecoder().decode(WeatherResponse.self, from: data)
    }
}

// 調用處（喺 .task 入面）：
.task {
    do {
        let weather = try await service.fetchWeather(city: "北京")
        // 更新界面……
    } catch {
        print("出咗錯：\(error)")
    }
}
```

避坑提醒：兩條網絡鐵律

(1) 上網必須 async/await，絕不阻塞主線程——卡住界面係崩潰級體驗。Swift 嘅結構化並發令呢樣嘢變得好自然，但唔好唔記得 try await 同 do/catch。(2) ATS 預設禁止明文 HTTP：iOS 要求 HTTPS。測試本地 http:// 服務嗰陣需要喺 Info.plist 入面臨時開例外（NSAppTransportSecurity），但正式產品必須全部行 HTTPS——審核都會盯住呢樣。

嚟動手：做一個”每日一句”小應用

六、網絡錯誤都要“分級”：唔好將所有報錯都掟畀用戶

斷網、超時、404、伺服器 500……唔同嘅錯，用戶應該睇到嘅訊息都唔同：

```
func friendlyMessage(_ error: Error) -> String {
    if let urlError = error as? URLError {
        switch urlError.code {
        case .notConnectedToInternet: return "網絡斷咗，檢查吓連接？"
        case .timedOut:               return "請求超時，遲啲再試？"
        case .cannotFindHost:         return "伺服器搵不到咗"
        default:                      return "網絡出咗問題：\(urlError.code)"
        }
    }
    if let d = error as? DecodingError {
        return "數據格式唔啱（可能係接口變咗）"
    }
    return "出咗錯：\(error.localizedDescription)"
}
```

畀用戶睇“人話”，畀日誌睇“原話”（print 完整 error）。唔好將 error.localizedDescription 嘅英文天書直接掟畀用戶——產品感就藏喺呢啲細節入面。

搵一個免費嘅名言 API（譬如 hitokoto 呢類），用 URLSession + async/await 拉取“每日一句”，顯示喺螢幕中央，加個“換一句”按鈕。再將鍾意嘅句子用 SwiftData 存入“收

藏夾”。呢個練習將 UserDefaults/SwiftData/Codable/URLSession 全部用咗一次——做完佢，第 7、8 章就係小菜一碟。

本章小結

- 口訣：小配置入 UserDefaults，成批數據入 SwiftData，出面嘅數據行 URLSession
- SwiftData 三板斧：@Model 聲明表、modelContainer 掛容器、@Query 讀 + context.insert 寫
- Codable 自動” 翻譯” JSON：字段對得上，魔法就生效
- URLSession + async/await：上網好似等快遞；do/catch 處理失敗
- 兩條鐵律：唔阻塞主線程；正式產品必須 HTTPS（ATS）

第六十五章：項目一——我嘅記賬本（本地版）

歡迎嚟到你第一個“正經項目”。呢個記賬本雖然唔上網，但五臟俱全：記到一筆賬、睇到流水、統計到每月、又刪改到。做完佢，你就完整行咗一次“數據邏輯界面”嘅全流程。咪心急抄代碼——先諗清楚先動手，呢個係我哋一貫嘅原則。

一、需求思考：先問五個問題

要素	答案
Who（畀邊個用）	自己用，屋企人朋友都得
What（做乜嘢）	記收支、睇流水、按月匯總、按分類統計
Why（點解）	練手 + 搞清楚啲錢花咗去邊
How（點樣做）	Swift + SwiftUI + SwiftData + MVVM，純本地冇後端
Constraint（約束）	一星期內完成；最低支援 iOS 17（SwiftData 門檻）

記賬本需求五要素

然後拆解成“每一步都用得”嘅小步驟：(1) 建 SwiftData 數據層（存到一筆賬）；(2) 列表頁（睇到流水）；(3) 錄入彈窗（加到賬）；(4) 刪除同編輯；(5) 月度匯總卡片。每完成一步，App 都係一件“用得嘅嘢”。

二、數據層：畀“賬”有個屋企

先定義一筆賬生乜嘢樣——字段要夠用但咪貪多：

```
import SwiftData

@Model
class Record {
    var type: String      // "income" 收入 / "expense" 支出
    var amount: Double    // 金額（元）
```

```
    var category: String  // 分類：餐飲、交通、人工……
    var note: String      // 備註（可選）
    var date: Date        // 記賬日期

    init(type: String, amount: Double, category: String, note: String = "", date:
    Date = Date()) {
        self.type = type
        self.amount = amount
        self.category = category
        self.note = note
        self.date = date
    }
}
```

小貼士：金額點解用 Double？

嚴謹嘅金融 App 會用 Decimal（精確十進制）嚟避免浮點誤差（0.1+0.2≠0.3 呢個老問題）。練手項目用 Double 就夠；如果第日做真金融產品，記得換成 Decimal 或者“以分為單位嘅 Int”——呢個係無數前輩踩出嚟嘅坑。

三、MVVM：界面同邏輯“分家”

直接畀界面操作數據庫？行得，但界面代碼會又臭又長。MVVM 架構令大家各司其職：View（界面）淨係管畫，ViewModel（邏輯中樞）管數據，Model（SwiftData）管存儲。

View（SwiftUI 界面）

ViewModel（@Observable 邏輯中樞）

Model（SwiftData 數據層）

圖 4：MVVM 三層結構——數據單向流動，各層互不越界

```
import Observation

@Observable
class RecordViewModel {
    var records: [Record] = []
    private let context: ModelContext

    init(context: ModelContext) { self.context = context }

    func load() {
        // SwiftData 嘅 #Predicate 篩選 + 排序
        let descriptor = FetchDescriptor<Record>(
            sortBy: [SortDescriptor(\.date, order: .reverse)]
        )
        records = (try? context.fetch(descriptor)) ?? []
```

```
    }

    func add(type: String, amount: Double, category: String, note: String) {
        let r = Record(type: type, amount: amount, category: category, note: note)
        context.insert(r)
        try? context.save()
        load()                          // 重新載入，界面刷新
    }

    func delete(_ record: Record) {
        context.delete(record)
        try? context.save()
        load()
    }
}
```

界面入面 @State private var vm = RecordViewModel(...)，然後直接用 vm.records 渲染列表——數據一變，界面自動刷新。

四、界面：列表頁 + 錄入彈窗

```
struct RecordListView: View {
    @State private var vm: RecordViewModel
    @State private var showAdd = false

    var body: some View {
        NavigationStack {
            List {
                Section("本月結餘") {           // 彙總卡片
                    SummaryRow(records: vm.records)
                }
                Section("流水") {
                    ForEach(vm.records) { r in
                        RecordRow(record: r)
                    }
                    .onDelete { idx in
                        idx.forEach { vm.delete(vm.records[$0]) }
                    }
                }
            }
            .navigationTitle("我嘅記賬本")
            .toolbar {
                Button { showAdd = true } label: {
                    Image(systemName: "plus")
                }
            }
```

```
            .sheet(isPresented: $showAdd) {
                AddRecordView(vm: vm)          // 錄入彈窗
            }
        }
    }
}
```

錄入彈窗要點：金額輸入框鍵盤用.keyboardType(.decimalPad)（數字鍵盤），收入 / 支出用分段控件（Picker），分類用菜單揀，撳“保存”就調用 vm.add(...) 同 dismiss() 關窗。

注意：輸入校驗咪偷懶

金額輸入框一定要校驗：空值、負數、“abc”呢類非法輸入都要攔住（用 Double(text) 判斷，失敗返回 nil）。App 崩潰最常見嘅原因之一就係“將用戶嘅輸入當成乖仔”。記住：用戶輸入嘅所有嘢，都要當敵人咁審查。

五、編輯、刪除同匯總

- 刪除：List 嘅.onDelete 左滑刪除 + 確認彈窗（刪錯嘢係記賬類 App 用戶最憎嘅事）；
- 編輯：撳行入編輯彈窗，預填原本嘅值（改一個字好過重新輸入）；
- 匯總：用 filter + reduce 計” 本月收入 / 支出 / 結餘”，頂部卡片展示：

```
func monthSummary(_ records: [Record]) -> (income: Double, expense: Double) {
    let cal = Calendar.current
    let monthRecords = records.filter {
        cal.isDate($0.date, equalTo: Date(), toGranularity: .month)
    }
    let income = monthRecords
        .filter { $0.type == "income" }
        .reduce(0) { $0 + $1.amount }
    let expense = monthRecords
        .filter { $0.type == "expense" }
        .reduce(0) { $0 + $1.amount }
    return (income, expense)
}
```

六、驗收：三層檢查 + 兩個進階挑戰

項目做完，按“功能邊界架構”三層驗收：記唔記到、刪唔刪到、重啟 App 數據仲喺唔喺度（功能）；金額輸 0 或者負數會唔會崩（邊界）；界面同數據有冇溝埋一齊、字段有冇硬編碼（架構）。

基礎版驗收通過之後，畀自己加兩味“進階菜”：

1. 分類統計：加一個” 統計” 標籤頁，用 Swift Charts 將分類支出畫成餅圖 / 柱狀圖——蘋果官方圖表庫，幾行代碼出圖；

2. iCloud 同步：SwiftData 開 iCloud 同步只需要配幾項設定——賬本喺 iPhone/iPad 之間自動同步，即刻” 高級” 咗。

嚟動手：今個禮拜完成佢

按“數據層 ViewModel 列表頁錄入彈窗匯總卡片”嘅次序動手，每日一步。卡住咗就將報錯原原本本掟畀搜尋引擎或者 AI（仲記唔記得第七篇《AI 編程實戰篇》呀？描述問題嗰陣講清楚“期望行為、實際行為、復現步驟”）。完成之後用它記一星期賬——自己整嘅 App 攞嚟用，成就感會推住你繼續向前行。

本章小結

- 需求五要素拆成” 每步都用得” 嘅小步驟，再動手
- 數據層：@Model 模型 + modelContainer 容器 + FetchDescriptor 查詢
- MVVM 分工：View 淨係管畫，@Observable 嘅 ViewModel 管邏輯，SwiftData 管存儲
- 界面：List + Section + toolbar + sheet 錄入彈窗
- 三層驗收（功能 / 邊界 / 架構）+ 進階挑戰（Swift Charts、iCloud 同步）

第六十六章：項目二——天氣查詢 App（網絡版）

記賬本係“練內功”，天氣 *App* 係“見世面”——佢要同互聯網打交道。呢個項目你會學到：申請免費 *API*、解析 *JSON*、處理“加載中 / 成功 / 失敗”三種界面狀態，仲有點樣將 *API* 密鑰收埋好。做完佢，你就具備咗做任何“上網 *App*”嘅能力——新聞、匯率、翻譯，套路一模一樣。

一、申請免費天氣 API

個人練手，推薦 OpenWeatherMap 或者國內嘅和風天氣（QWeather）——都有免費額度，註冊就即刻用得。以 OpenWeatherMap 為例：

1. 註冊賬號喺 API keys 頁面生成一個 Key（apikey）；
2. 免費版可以調用 Current Weather 接口：api.openweathermap.org/data/2.5/weather?q=beijing&appid= 你的 Key&units=metric；
3. 喺瀏覽器先試一次，睇吓返回嘅 JSON 生乜嘢樣——呢個係接任何 API 之前必做嘅”預習”。

避坑提醒：密鑰絕對唔可以寫死喺代碼入面

API Key 等於你嘅“銀包密碼”，寫入代碼一提交上 GitHub，幾個鐘之內就會畀爬蟲掃走盜刷。iOS 嘅正確做法：放入 Config.xcconfig（配置檔案，已經畀 gitignore 忽略），通過 Info.plist 讀入代碼。搜尋“iOS xcconfig 管理密鑰”照住改就得。

二、網絡層：URLSession + Codable

先照 API 返回嘅 JSON 定義模型，再寫服務：

```
// JSON: {"weather":[{"description":"晴"}],"main":{"temp":26.5,"humidity":40},"name
    ":"北京"}
struct WeatherResponse: Codable {
    let weather: [Weather]
```

```
    let main: Main
    let name: String
}
struct Weather: Codable { let description: String }
struct Main: Codable { let temp: Double; let humidity: Int }

struct WeatherService {
    let key: String   // 從 xcconfig 讀入

    func fetchWeather(city: String) async throws -> WeatherResponse {
        var comps = URLComponents(string: "https://api.openweathermap.org/data/2.5/
    weather")!
        comps.queryItems = [
            URLQueryItem(name: "q", value: city),
            URLQueryItem(name: "appid", value: key),
            URLQueryItem(name: "units", value: "metric"),
            URLQueryItem(name: "lang", value: "zh_cn")
        ]
        let (data, _) = try await URLSession.shared.data(from: comps.url!)
        return try JSONDecoder().decode(WeatherResponse.self, from: data)
    }
}
```

三、三種界面狀態：加載中 / 成功 / 失敗

上網 App 嘅靈魂在於“狀態管理”。網絡慢、斷網、城市名打錯……每種情況界面都要向用戶交代。用枚舉 + switch 優雅咁表示：

```
enum LoadState {
    case loading                  // 兜緊圈
    case success(WeatherResponse) // 成功
    case error(String)            // 失敗
}

@Observable
class WeatherViewModel {
    var state: LoadState = .loading
    private let service = WeatherService(key: Config.apiKey)

    func search(city: String) async {
        state = .loading
        do {
            let data = try await service.fetchWeather(city: city)
            state = .success(data)
        } catch {
            state = .error("搵唔到呢個城市，轉個名再試吓？")
```

```
        }
    }
}

// 界面入面：
switch vm.state {
case .loading: ProgressView("載入中…")
case .success(let d): WeatherContent(data: d)
case .error(let msg): Text(msg).foregroundColor(.secondary)
}
```

呢個 loading/success/error 三段式係上網界面嘅“萬金油模板”，先背咗佢，以後所有上網 App 都照辦煮碗。

四、實戰界面代碼：將天氣畫出嚟

```
struct WeatherContent: View {
    let data: WeatherResponse

    var body: some View {
        VStack(spacing: 12) {
            Text(data.name)
                .font(.title.bold())
            Text("\(Int(data.main.temp))°C")
                .font(.system(size: 76, weight: .thin))
            Text(data.weather.first?.description ?? "未知")
                .font(.title3)
                .foregroundColor(.secondary)
            HStack(spacing: 24) {
                Label("溼度 \(data.main.humidity)%", systemImage: "humidity")
                Label("體感 \(Int(data.main.temp))°C", systemImage: "thermometer")
            }
        }
        .padding()
    }
}
```

幾個小細節值得學：first? 安全攞第一個元素（列表可能係空），?? “未知”做兜底文案；76 號幼字重嘅溫度數字係天氣類 App 嘅經典配方；Label 係“圖標 + 文字”嘅官方組件，好過人手砌 HStack 咁規範。

五、交互細節同收藏城市

- 搜尋框：頂部.searchable 修飾符，系統自帶搜尋交互，提交嗰陣調用 vm.search；

- 收藏城市：將成日睇嘅城市存入 UserDefaults，下次打開自動查；
- 下拉刷新：.refreshable { await vm.search(...) } 一行搞掂；
- 天氣圖標：用 Image(systemName:) 按描述映射（sun.max / cloud.rain / snowflake），界面即刻活起嚟。

小貼士：中文城市嘅”梗”

OpenWeatherMap 對中文城市名支援一般，搜“北京”可能冇結果而“beijing”就正常。產品化做法係內置一份“城市名英文 / 坐標”映射表，或者接國內天氣服務。練手階段先支援拼音 / 英文城市名就得，咪同佢硬撼。

嚟動手：天氣 App 驗收清單

六、網絡請求點樣調試：打印、抓包同模擬慢網

上網 App 出問題，十居其九都係“請求發得唔啱”或者“響應解析得唔啱”。三個調試手段：

1. 打印原始數據：將 String(data: data, encoding: .utf8) 打印出嚟睇原始 JSON——字段名、類型一目了然，好過估快十倍；
2. 抓包工具：Mac 上免費嘅 Proxyman / 開源嘅 Charles 可以截獲 App 嘅全部請求同響應，好似 X 光咁睇清網絡層；
3. 模擬弱網：模擬器菜單 Features Network Link Conditioner（需要由開發者網站下載工具），可以模擬 3G、高延遲——測出”轉圈轉半日”嘅體驗問題。

小貼士：請求過唔到 ATS？

如果請求一發出就報“App Transport Security”錯誤，好大可能係用咗 http:// 明文地址。正式接口必須 HTTPS；本地調試臨時開例外（Info.plist 加 NSAllowsLocalNetworking），上架之前記得清乾淨。

(1) 正常搜尋出天氣；(2) 斷開網絡，App 優雅報錯唔崩潰（喺模擬器入面閂咗 Mac 嘅網絡嚟測！）；(3) 輸入一個唔存在嘅城市，提示要友好；(4) 收藏兩個城市，重啟 App 仲喺度；(5) 深色模式下界面唔刺眼。五條全部過關，呢個項目先算畢業。

本章小結

- 先喺瀏覽器試 API、睇得明 JSON，再寫代碼——”預習”永遠唔蝕
- API Key 收入 xcconfig，絕不進代碼庫
- Codable 模型 + URLSession async/await：字段對上，JSON 自動解析
- loading/success/error 三段式狀態係上網界面嘅萬金油
- 斷網、空輸入、城市唔存在——錯誤處理係上網 App 嘅及格線

第六十七章：打磨同發佈——測試、調試同上架 App Store

App 整好咗，跟住點呢？呢一章回答“由行得到”到“交到貨”嘅三件事：點樣高效咁調試搵 *bug*、點樣寫測試防回歸、點樣將 *App* 送上 *App Store* 畀全世界下載。上架唔止係儀式感——佢係將一件作品變成產品嘅分水嶺。

一、調試三板斧：print、斷點、預覽

工具	做乜嘢	點樣用
print + 控制檯	睇運行日誌	代碼入面 print(” 值 =\(x)”)，喺 Xcode 底部控制台睇輸出
斷點調試	令程式停喺某一行，睇變量值	喺行號旁邊撳一下出藍標，運行到嗰度就停，逐步行
SwiftUI 預覽 + 調試器	即時睇界面同視圖層級	#Preview 畫布 + 運行時嘅 View Hierarchy 檢查器

調試三板斧

小貼士：崩潰咗點樣讀？

Swift 崩潰嗰陣控制台會打印“Fatal error”加原因，Xcode 會停喺出錯嗰一行並高亮。真機上面嘅歷史崩潰可以喺 Xcode Window Organizer Crashes 入面睇（需要付費賬號）。關鍵詞優先搜“錯誤原因第一句 + swiftui”——精確嘅問題，先至有精確嘅答案。

二、測試：畀代碼上保險

手測靠唔住——改咗呢個功能，另一個靜靜哋壞咗。測試就係“後悔藥”兼“防彈衣”。iOS 用 XCTest 框架，分兩種：

層級	測乜嘢	Target	速度
單元測試	純邏輯（計算、校驗）	MyFirstAppTests	秒級
UI 測試	真實界面行為	MyFirstAppUITests	分鐘級
手動測試	真人感受、體驗	——	永遠需要

iOS 測試層級（模板項目已預置兩個 Test Target）

```
import XCTest

// 把"金額字串轉數字"呢種邏輯抽成純函數
func parseAmount(_ input: String) -> Double? {
    guard let v = Double(input.trimmingCharacters(in: .whitespaces)),
          v > 0 else { return nil }
    return v
}

final class AmountTests: XCTestCase {
    func testEmptyInputReturnsNil() {
        XCTAssertNil(parseAmount(""))
    }
    func testNegativeReturnsNil() {
        XCTAssertNil(parseAmount("-5"))
    }
    func testNormalValueParses() {
        XCTAssertEqual(parseAmount("12.5"), 12.5)
    }
}
```

行測試：CmdU。全綠之後再改代碼，心裏有底。記賬本入面嘅金額校驗、匯總計算，都應該有單元測試罩住。

三、簽名同打包：畀 App 蓋上“印章”

每個 iOS App 都必須簽名先安裝到——證明“呢個包出自你手，冇畀人改過”。層級關係：

- 學習階段：免費 Apple ID + Xcode 自動簽名（Personal Team），真機行得到，7 日續期一次；
- 分發階段：付費開發者賬號（99 美元 / 年）簽發證書同描述檔，先可以上架同長期安裝。

打包：Xcode 菜單 Product Archive（歸檔），構建完成之後喺 Organizer 入面 Distribute App 揀 App Store Connect 上傳。歸檔必須用 Generic iOS Device 做目標（唔係模擬器），呢一步係新手成日唔記得嘅。

避坑提醒：證書丟咗 = 更新斷纜

簽名證書同私鑰一旦丟失，你就冇得再畀舊 App 發更新——用戶只可以裝“全新”應用。妥當備份證書（鑰匙串導出.p12）並記好密碼。好在蘋果提供咗“自動簽名”同雲端證書管理（Xcode 入面剔選 Automatically manage signing），可以慳返大部分手工操作嘅坑。

四、上架 App Store 全流程

註冊開發者賬號

準備商店素材

上傳構建包

填隱私與審覈資訊

蘋果審覈（1-7 天）

上架

圖 5：App Store 上架六步走

- 註冊：developer.apple.com，個人賬號 99 美元 / 年；
- 素材：圖標（1024×1024，咪帶圓角——蘋果自己切）、截圖（iPhone 各尺寸）、描述、私隱政策連結；
- 私隱問卷：如實申報收集咗哪些數據——唔上網嘅記賬本可以”唔收集任何數據”；
- 審核：首次提交通常要等 1-7 日，被拒就按”審核備註”逐條改——蘋果嘅拒絕理由通常寫得好具體，照做就得；
- TestFlight：上架之前先將測試版傳畀朋友內測（最多 10000 人），收集一輪反饋再正式提交，專業範十足。

五、商店素材點樣做：第一印象就係下載量

商店頁係用戶“行街”嗰陣睇到嘅櫥窗。三個素材決定第一印象：

- 圖標：1024×1024，咪用截圖求其頂住。冇設計能力就用 AI 工具生成一個簡潔圖形，喺桌面縮細之後仲一眼認得出先算合格；
- 截圖：至少 2 套（iPhone 主流尺寸），展示最核心嘅功能場景；每張配一句貼地文案（”一秒記一筆”好過”高效的記賬解決方案”十倍）；
- 描述：第一句講清楚”呢個係乜嘢 + 畀邊個用”，咪堆砌”最專業、最強”——真誠好過誇張。

嚟動手：將記賬本”交付”一次

六、崩潰上報同性能優化小清單

App 上架之前，呢兩件事值得各花半個鐘：

崩潰監控

你測試嗰陣冇崩，唔代表用戶部手機唔崩。蘋果喺 Xcode Organizer 入面提供免費嘅崩潰報告（付費賬號可以睇真機崩潰日誌）；更省心嘅方案係接入第三方 SDK（好似 Firebase Crashlytics 咁）。冇崩潰監控就上架，等於合埋眼揸車。

性能自查四條

1. 列表用 LazyVStack / List：長列表咪用普通 VStack 硬排——懶加載係性能第一課；
2. 圖片咪硬塞原圖：用 AsyncImage（遠程圖）同系統快取，5MB 原圖足以卡爆舊機型；
3. 咪喺主線程做重活：網絡、大文件都行 async/await；
4. 用 Instruments 度一度：Xcode 自帶嘅性能分析器（Time Profiler、記憶體泄漏檢查）——先測量，再優化，咪靠感覺。

(1) 畀 parseAmount 等邏輯補 3 個單元測試（CmdU 行到全綠）；(2) 用斷點親手捉一個自己整出嚟嘅崩潰並整好佢；(3) Product Archive 打一個包，行 TestFlight 邀請朋友內測；(4) 有餘力就註冊開發者賬號，將記賬本真正提交上架。頭三步今晚就做得完。

本章小結

- 調試三板斧：print 日誌、斷點、SwiftUI 預覽同視圖檢查器
- XCTest：單元測試（CmdU）+ UI 測試 + 手動測試，模板已預置
- 簽名：免費賬號自動簽名（7 日續期）；付費賬號先可以分發
- 打包：Generic iOS Device + Archive + Distribute；證書務必備份
- 上架六步：註冊素材上傳私隱問卷審核上架；TestFlight 先內測

第六十八章：進階之路——權限、通知同現代架構

做到 *App* 出嚟只係開始。呢一章傾吓由“識做”到“做好”嘅幾道坎：點樣優雅咁申請權限、點樣發通知唔惹人嬲、點樣將架構再抬高一層，仲有——喺 *2026* 年，點樣同 *AI* 拍檔寫 *iOS* 代碼。每一條都係你喺真實項目入面好快會遇到嘅。

一、權限：先敲門，再入門

相機、定位、通訊錄、通知……呢啲“敏感地帶”，iOS 要求 App 喺用到嗰陣向用戶彈窗申請。規矩係“聲明 + 申請 + 優雅降級”三步：

```
// 1. Info.plist 入面聲明用途文案（不聲明直接崩潰！）
// Privacy - Location When In Use Usage Description: "用嚟查你所在城市嘅天氣"

// 2. 用嘅時候申請
import CoreLocation

@Observable
class LocationModel {
    private let manager = CLLocationManager()

    func request() {
        switch manager.authorizationStatus {
        case .notDetermined: manager.requestWhenInUseAuthorization()
        case .denied, .restricted: /* 引導去設定入面開 */
        default: /* 已授權，開搞 */
        }
    }
}

// 3. 被拒之後優雅降級：畀提示 + 跳去設定
// UIApplication.shared.open(URL(string: UIApplication.openSettingsURLString)!)
```

注意：iOS 權限嘅”一次性”

iOS 嘅權限彈窗只彈一次——用戶拒絕咗之後就唔會再彈。所以：申請嘅時機要揀喺“用戶就嚟要用嗰一刻”（而唔係一啟動就彈一堆），文案要講明用途。被拒之後嘅界面引導（“去設定入面開啟定位”）係標配，咪令用戶困喺死胡同入面。

二、通知：喺啱嘅時間講有用嘅嘢

通知係 App 接觸用戶嘅通道，亦都係最容易被用戶“封殺”嘅功能。iOS 用 UserNotifications 框架，發通知三條紀律：淨係發有用嘅、畀用戶閂得、咪半夜嘈住人。基礎用法：

```
import UserNotifications

// 1. 申請通知權限（彈一次窗）
UNUserNotificationCenter.current()
    .requestAuthorization(options: [.alert, .sound, .badge]) { granted, _ in
        print(granted ? "已授權" : "被拒絕")
    }

// 2. 安排一條"每日 20:00 記賬提醒"
func scheduleDailyReminder() {
    let content = UNMutableNotificationContent()
    content.title = "記賬提醒"
    content.body = "今日仲未記賬喎"
    content.sound = .default

    var comps = DateComponents()
    comps.hour = 20
    let trigger = UNCalendarNotificationTrigger(dateMatching: comps, repeats: true)

    let request = UNNotificationRequest(
        identifier: "daily-reminder",
        content: content,
        trigger: trigger
    )
    UNUserNotificationCenter.current().add(request)
}
```

注意：iOS 嘅通知權限同樣“只彈一次”，而且用戶隨時可以喺系統設定入面閂咗佢——所以通知內容嘅價值感好重要，濫發一次可能就永遠失去用戶。

三、架構進階：MVVM 深入同可測試性

第 65 章用過 MVVM，呢度補三個進階要點：

- 協議抽象：令 ViewModel 依賴協議（protocol）而唔係具體類——測試嗰陣可以注入”假

數據源”，呢個係可測試性嘅基石；

- 單一職責：一個 ViewModel 淨係管一件事（記賬嘅唔會理天氣）；功能多咗就拆；
- 咪過度設計：單人小項目，”@Observable 類 + 協議抽象” 就夠。VIPER、Clean Architecture 呢類重架構，等團隊同複雜度找上門先再講——工具係服務目標嘅，唔係用嚟炫耀嘅。

小貼士：SwiftUI 嘅” 生態紅利”

iOS 生態仲有兩個“只此一家”嘅進階玩法：小組件（WidgetKit）令用戶唔使打開 App 都睇到資訊（記賬本嘅“本月支出”放桌面），快捷指令（App Intents）令用戶用 Siri 或者自動化調用你嘅功能。兩者都有官方框架，入門成本唔高，但對用戶體驗嘅加成好大——進階路上值得優先解鎖。

四、2026 年嘅新玩法：AI 輔助 iOS 開發

2026 年，AI 已經深度嵌入蘋果開發流程。如果你想提速，呢套組合拳好成熟：

環節	AI 點樣幫	你嘅職責
寫代碼	Xcode 內置代碼補全 + 第三方 AI 插件生成	描述清楚需求，審查產出
查資料	將官方文檔同報錯資訊掟畀 AI 解讀	關鍵結論返去官方文檔核實
寫測試	令 AI 按” 正常 / 邊界 / 異常” 三類生成 XCTest 用例	確認用例符合業務語義
改 bug	貼崩潰日誌 + 相關代碼，AI 定位修復	睇得明改動，行測試驗證

AI 輔助 iOS 開發嘅四個場景

具體方法論——點樣畀 AI 下指令、點樣驗收、點樣避坑——完整嘅 11 章內容喺本書第七篇《AI 編程實戰篇》（第 37-47 章）入面，強烈建議配合閱讀：Swift 語法交畀本篇，AI 協作嘅心法交畀第七篇。

嚟動手：畀記賬本加” 三件套”

五、Swift 並發要點：@MainActor 同 Task

Swift 6 嘅並發體系有兩條新手必識嘅規則：

- 界面只可以喺主線程更新：SwiftUI 視圖預設係 @MainActor，網絡回調返到主線程先改狀態（await MainActor.run { ... } 或者將 ViewModel 標成 @MainActor）；
- 異步任務用 Task 管理：.task { } 修飾符跟住視圖出現自動開始、消失自動取消——呢個係 SwiftUI 入面啟動異步工作嘅標準姿勢，咪喺 body 入面亂咁寫 Task 裸奔。

```
struct WeatherScreen: View {
```

```
    @State private var vm = WeatherViewModel()

    var body: some View {
        Group {
            switch vm.state {
            case .loading: ProgressView()
            case .success(let d): WeatherContent(data: d)
            case .error(let m): Text(m)
            }
        }
        .task {                          // 出現就開始，消失就取消
            await vm.search(city: "北京")
        }
    }
}
```

呢兩條規則背後係 Swift 6 嘅“數據競爭安全”設計——編譯器會喺你寫錯嗰陣直接報錯，而唔係等到 App 崩咗先發現。編譯器嘅報錯唔係同你搵碴，係替你擋子彈。

(1) 畀記賬本加“每日 20:00 記賬提醒”（通知權限 + 本地通知）；(2) 將 ViewModel 嘅數據源改成協議抽象，並寫一個注入假數據嘅單元測試；(3) 加一個“本月支出”桌面小組件。三件套做完，你嘅項目就有“正規軍”嘅樣喇。

本章小結

- 權限三步：Info.plist 聲明（漏咗必崩）、用嗰陣申請、被拒引導去設定
- iOS 權限彈窗只彈一次：時機揀啱” 就嚟要用嗰一刻”，文案寫清楚用途
- 通知三紀律：淨係發有用嘅、閂得、唔嘈住人；UserNotifications 框架
- MVVM 進階：協議抽象（可測試）、單一職責、唔過度設計
- 生態紅利：WidgetKit 小組件 + App Intents 快捷指令
- AI 輔助四場景：寫碼、查資料、寫測試、改 bug；方法論見第七篇（第 37-47 章）

第六十九章：速查手冊同資源

本章係一份可以獨立使用嘅速查手冊。當你記唔清某個寫法或者需要快速決策嗰陣，直接揭到呢度。建議將佢當成“口袋卡”：常用代碼速查、常見問題、學習資源，一頁都唔漏。

一、常用代碼速查卡

想做乜嘢	關鍵代碼
頁面私有狀態	@State private var x = 初值
跨頁面邏輯	`@Observable class VM { var x = 0 }`
直排 / 橫排佈局	`VStack { }/HStack { }`
彈性空間	`Spacer()`
列表	List { ForEach(陣列) { ... } }
頁面跳轉	`NavigationStack + NavigationLink`
底部彈窗	.sheet(isPresented: $show) { 內容}
閂當前頁	`@Environment(\.dismiss) var dismiss`
上網攞數據	`try await URLSession.shared.data(from: url)`
JSON 解析	`struct X: Codable+JSONDecoder().decode`
存小配置	UserDefaults.standard.set(值, forKey: ”鍵”)
儲存結構化數據	`@Model+modelContainer+@Query`
深色模式適配	用系統顏色（.primary / Color(.systemGray6)），唔好寫死
行測試	CmdU

iOS 開發速查卡（2026 年 8 月版）

二、常見問題 FAQ

Q1：我冇 Mac，可唔可以學 iOS 開發？

可以學語言，但做唔到完整開發。iPad/iPhone 上的 Swift Playgrounds 可以學完第 61 章嘅全部內容；不過 Xcode 係 macOS 獨佔，最終一定要有一部 Mac。預算有限可以先租雲端

Mac 或者買二手 Mac mini 過渡，學完語言再決定投入幾多。

Q2：學 iOS 開發要使幾多錢？

學習階段：0 蚊（Xcode 免費 + 免費 Apple ID 真機調試）。發佈階段：開發者帳號 99 美元一年（大概 700 蚊）。加埋一部 Mac 嘅硬件投入，呢個就係全部成本。比起回報——App Store 上就算一個小工具都有人肯畀錢——呢條數好劃算。

Q3：SwiftUI 同 UIKit 應該先學邊個？

SwiftUI。2026 年嘅新項目、新教程、新職位都以 SwiftUI 為主，本書全程都用佢。UIKit 係上一代框架，仍然喺大量舊項目入面用緊——等你 SwiftUI 熟練之後，按需要補返 UIKit（官方有互操作教程），上手會好快。次序唔好掉轉。

Q4：學識 iOS 開發容唔容易搵工？

2026 年嘅行情：淨係「識畫介面」嘅初級職位競爭好激烈，但「識 Swift + SwiftUI + 有上架作品」嘅開發者依然搶手。iOS 職位嘅人工中位數喺各個技術棧入面長期排前列。學習期嘅你，最好嘅履歷就係 App Store 入面你自己整嗰個 App。

Q5：App 俾審核拒絕咗點算？

唔使慌，好正常——第一次提交被拒嘅機率唔低。蘋果嘅拒絕理由通常寫得好具體（邊個頁面、邊條準則），逐條修改之後喺 App Store Connect 入面回覆說明，再重新提交就得。三大常見拒絕原因：權限用途講唔清、介面有 bug、元數據（截圖 / 描述）同實物唔符。提早自查呢三項，可以慳返一輪等待。

三、Xcode 高頻快捷鍵速查

操作	快捷鍵
運行 / 測試	CmdR / CmdU
打開畫布預覽	OptionCmd
快速打開檔案	CmdO
全域搜尋	CmdF
格式化縮排	CtrlI
註解 / 取消註解	Cmd/
清理構建快取	CmdK
跳到定義	Cmd 點按
重新命名（全域安全改名）	右鍵 Refactor Rename

Xcode 高頻快捷鍵速查

小貼士：編譯玄學先清快取

「琴日仲行到今日就行唔到」呢啲靈異事件，十居其六七都係構建快取錯亂。CmdK 清理完重新構建，可以醫好一大半玄學問題。仲唔得就重啟 Xcode（連 Mac 一齊），再唔得就「剷走 DerivedData 資料夾」——呢個係 iOS 開發者口耳相傳嘅終極三連。

四、推薦學習資源

六、推薦學習資源

類型	資源	推薦理由
官方	developer.apple.com	蘋果官方文件，SwiftUI/Swift 教程全部都喺入面，第一權威
官方	Apple 嘅 SwiftUI Tutorials	官方手把手課程，對新手友好度滿分
官方	Swift 官網 (swift.org)	語言官方文件同埋線上演練場
社區	Hacking with Swift	Paul Hudson 嘅免費教程庫，質素公認第一梯隊
社區	Stack Overflow (swiftui 標籤)	報錯求救第一站
影片	Apple Developer 官方頻道（WWDC 影片）	每年新技術嘅權威講解
設計	Apple 人機介面指南（HIG）	做出「蘋果味」介面嘅官方聖經
配套	第七篇《AI 編程實戰篇》（第 37-47 章）	用 AI 輔助 iOS 開發嘅完整心法

iOS 開發學習資源清單（2026 年 8 月）

五、結語：由第一個 App 到第 N 個 App

回頭看，你由「唔知 SwiftUI 係乜嘢」，到裝好 Xcode、學識 Swift、用 SwiftUI 畫出介面、幫個 App 接上資料庫同網絡，再到測試、打包、上架——呢條路上你已經行完咗由零到一最難嗰一段。

技術嘅世界變得好快：框架會升級，工具會換代，AI 會接手越嚟越多「敲鍵盤」嘅嘢。但有一樣嘢唔會過時：你將一個模糊嘅諗法，拆解成清晰嘅步驟，再一步一步實現佢嘅能力。工具係放大器，思路係地基。放大器可以換，地基唔可以郁。

你唔再只係" 識玩手機嘅人"。你可以將腦入面嘅諗法，變成喺幾億部 iPhone 上面真實運行嘅 App。由今日開始，用鍵盤去創造——呢個就係 iOS 開發者嘅浪漫。

參考文獻

1. Apple Inc. (2026). SwiftUI Framework Documentation. developer.apple.com.

2. Apple Inc. (2026). Swift Programming Language Guide. swift.org.

3. Apple Inc. (2026). SwiftData Framework Documentation. developer.apple.com.

4. Apple Inc. (2026). App Store Review Guidelines. developer.apple.com/app-store/review.

5. Apple Inc. (2026). Human Interface Guidelines (HIG). developer.apple.com/design.

6. Apple Inc. (2026). Xcode 27 Release Notes and App Distribution Guide.

十、附錄

附錄 A：新手機開箱「保姆級」流程

開箱驗機「三部曲」：

- **睇外觀**：檢查機身、螢幕有無劃痕、磕碰。
- **對資訊**：核對盒子上嘅 IMEI/序列號同發票上嘅係咪一致。
- **試功能**：撥個電話試下聽筒話筒，拍張相檢查攝像頭。

激活與初始設定「重點提醒」：

- **網絡**：先連上穩定嘅 Wi-Fi。
- **帳號**：登入你嘅蘋果 ID、華為帳號、小米帳號等。
- **基礎設定**：將「查找我嘅手機」打開，設定一個靠譜嘅鎖屏密碼。
- **eSIM 設定**：如果你嘅手機支持 eSIM（例如 iPhone、部分安卓旗艦），可以喺設定入面添加 eSIM 套餐，無需插入實體 SIM 卡。出國嗰陣 eSIM 尤其方便，可以直接購買當地運營商嘅 eSIM 套餐。
- **數據遷移**：由舊手機遷移數據嗰陣，確保兩台手機電量充足。iPhone 用「快速開始」，安卓用官方換機 App。

裝 App「安全第一」：

- **首選官方商店**：蘋果用 App Store，安卓用手機自帶嘅應用商店。
- **權限唔好亂畀**：安裝嗰陣彈出嘅權限請求，諗清楚「呢個 App 真係需要嗎?」

附錄 B：動手練習小參考

- **基礎篇練習：**清理手機存儲，目標就係搵到並刪除嗰啲「唔記得係咩」嘅舊影片、重複截圖同好耐唔用嘅 App。
- **軟件篇練習：**設定微信嘅存儲清理，路徑通常係【我】-【設定】-【通用】-【存儲空間】。
- **進階篇練習：**查看電池健康度，iPhone 喺【設定】-【電池】-【電池健康】入面。
- **通用技巧：**遇到問題，先重啟！呢個係解決一半以上手機小毛病嘅萬能鑰匙。

附錄 C：極簡術語「黑話」對照表

你可能聽到嘅詞	其實就係指……	一句話解釋
SoC / 晶片	手機嘅大腦同心臟	決定手機快唔快、遊戲卡唔卡嘅核心部件。
RAM (記憶體)	手機嘅「短期工作桌」	桌越大，可以同時開嘅 App 就越多，切換越流暢。
ROM (存儲)	手機嘅「大倉庫」	存相、影片、App 嘅地方。128G、256G 講嘅就係佢。
OLED / LCD	螢幕嘅兩種材質	OLED 色彩更艷、更省電；LCD 更護眼、成本低。
NFC	近距離無線通訊	等你手機可以變身公交卡、門禁卡、銀行卡嘅「貼貼」功能。
OTA 更新	無線系統升級	喺【設定】入面撳嘅嗰個「系統更新」，就係佢。
Root	安卓超級管理員權限	獲得後可以修改系統檔案，但都有風險。
BL 鎖	Bootloader 鎖	廠商加嘅安全鎖，解鎖後先可以刷機。
NPU	神經網絡處理器	專門處理 AI 任務嘅晶片，比 CPU 快幾十倍、功耗更低。
端側 AI	手機本地運行嘅 AI	AI 喺手機上直接運行，唔需要聯網，速度快、私隱好。
Qi2	新一代無線充電標準	帶磁吸定位嘅無線充電，類似 MagSafe，充電更方便。
eSIM	嵌入式 SIM 卡	冇實體卡嘅 SIM，直接寫入手機，換運營商唔使換卡。
LTPO	自適應刷新率技術	螢幕刷新率根據內容自動調節（1-120Hz），流暢又省電。

附錄 D：靠譜資源去邊度搵？

官方支持係首選：

- **蘋果用戶**：apple.com/cn 同 iCloud.com。
- **安卓用戶**：手機品牌嘅官網都有「服務與支持」板塊。
- **通用工具**：蘋果用 iTunes，華為用華為手機助手，小米用小米換機。

實用網站同博主：

- **查技巧**：喺 B 站搜尋「【你嘅手機型號】技巧」。
- **睇測評**：多睇幾個唔同博主嘅測評影片。
- **學影相**：關注手機攝影教程博主。

安全自查一句話清單：

1. 鎖屏密碼設咗未？
2. 「查找我嘅手機」開咗未？
3. 重要帳號嘅雙重認證開咗未？
4. 定期畀相、微信聊天記錄做備份咗未？
5. 記住防騙口訣：唔輕信、唔洩露、唔轉賬、唔貪心、唔急躁。

附錄 E：多謝與說明

多謝你堅持讀完呢本「手機課」！寫作過程入面，我哋參考咗大量廠商嘅官方說明書、用戶手冊同埋公開嘅技術科普文章，以確保資訊嘅準確性。

技術嘅發展日新月異，書入面嘅內容係基於撰稿時嘅主流技術同產品。當你讀到佢嘅時候，或者又有新嘅功能同產品出現。希望呢本書傳授嘅思路同方法，可以幫你從容面對將來更多嘅數碼新玩意。

6 版更新要點

- 擴充「數碼資產價值與管理理念」，新增「圖片與檔案系統管理」章節（時間 + 事件命名、分類整理、72 小時自動清理、EXIF 脫敏）。
- 完善「純淨 / 防護模式」同「電子寄生蟲」防護體系。
- 新增 AI 時代常見詐騙套路（換臉擬聲 / 冒充權威 / 虛假投資）與破局防護思路。
- 重寫「開盒」與社工攻擊防護。
- 擴充無障礙章節（旁白 / TalkBack 完整操作、放大與色彩濾鏡、AI 圖像描述、實時字幕、RTT/TTY 通話雙通道）。
- 用「角色-背景-任務-限制」思路重寫提示詞框架，附 3 個完整場景案例。
- 新增三篇技術進階附錄：第一章附：安卓手機記憶體拓展解析（本質定義、zRAM/swap 原理、速度瓶頸、適用場景邊界、各廠商命名與開啟方式）。
- 第二十章附：Termux——Android 設備上的便攜式 Linux 終端環境（核心功能、插件生態、安裝配置、流動開發 / 伺服器運維 / 安全測試等典型場景）。
- 第二十三章附：GitHub 入門指南——由代碼託管到協作平臺（Git vs GitHub 區分、核心概念、註冊上手、Pull Request 協作流、安全提醒、國內訪問方案）。

7 版更新要點

一、新增 4 章內容

- 第 29 章：數碼健康——屏幕使用時間分析、App 限額、勿擾模式、20-20-20 護眼法則、頸椎自救、睡眠管理、資訊焦慮「選擇性失明」、一星期數碼健康挑戰。
- 第 30 章：投屏與車載——AirPlay / Miracast / DLNA 三陣營對照、有線投屏方案、Samsung DeX 桌面模式、CarPlay / Android Auto / HiCar / CarLife 四大車載方案、會議室投屏實戰。
- 第 31 章：短視頻創作——選題三問、拍攝 5 步設置、運鏡五招（推拉搖移跟）、剪映六步工作流、封面製作要點、「頭 3 秒定生死」內容心法。
- 第 32 章：安全急救手冊——丟失黃金 30 分鐘 SOP、被騙 72 小時處理流程、中毒安全模式排查、賬號被盜奪回六步、勒索資訊應對、日常預防五件事。

二、新增兩章內容

- 第 27 章：適老化與親情——簡易模式品牌對照表、防詐騙六大騙局與三道防線、遙距協助（暢連 / 向日葵）、微信關懷模式與語音輸入、SOS / 跌倒檢測 / 用藥提醒。
- 第 28 章：選購指南——三步選機法、「只睇五項」參數表、五檔預算推薦、七大場景選機表、五大購機陷阱、購買時機與渠道分析。

三、新增朗讀與互動功能

- 浮動朗讀控制欄：支援播放 / 暫停、上一段 / 下一段、0.5x-2.5x 倍速調節、停止。
- 右邊快捷耳機按鈕：一鍵啟動朗讀，自動由當前閱讀位置開始。
- 鍵盤快捷鍵：R 切換朗讀，ESC 停止。
- 朗讀體驗優化：自動連續翻章，列印時朗讀控件自動隱藏。

四、體系更新

目錄、學習路徑（四步五步）、章節地圖、版本號（1.61.7）、附錄編號（五六）、元數據全部同步刷新。

五、內容時效性更新

- 處理器標準：驍龍 8 Elite Gen5 / 天璣 9500 劃入頂級檔（2026 年基準）。
- 旗艦機型參考：iPhone 17 Pro、三星 S26 Ultra、小米 17 Ultra、華為 Pura 80 Ultra、vivo X300 Pro。
- AI 章節：更新到 DeepSeek R1、ChatGPT-5、端側 NPU 80 TOPS 算力標準。
- 摺疊屏：價格區間下探到 4000-5000 元檔。
- 選購指南：全部預算分檔同場景推薦刷新至 2026 年市場。
- 網絡與生態：Wi-Fi 7 / 5G-A / 鴻蒙生態 / LE Audio 全部更新到最新規範。

六、新增功能模組：學習儀表盤（位於 Hero 區域下方）

- 閱讀進度環形圖：實時顯示已讀章節百分比，配 SVG 圓環動畫。
- 手機健康自檢：8 項安全檢查清單（密碼、雙重認證、查找手機、雲備份、微信備份、支付安全、反詐 App、免密清理）。
- 狀態持久化：勾選狀態寫入 localStorage，健康分動態計算並以漸變進度條呈現。
- 分級提示語：根據得分顯示「滿分 / 唔錯 / 合格 / 安全隱患」等唔同反饋。

8 版更新要點

新增第七篇「AI 編程實戰篇」（第 37-47 章，共 11 章）：範式革命與 Agentic Engineering、AI 編程五大思維原則、上下文工程與規範驅動開發（SDD）、需求工程、2026 年 AI 編程工具全景、大模型全景與 API 實戰、對話工程、由 0 到 1 完整項目全流程、質量保障、進階之路（MCP / 多 Agent 協作）與速查手冊。同步更新目錄、學習路徑（五步六步）、章節地圖與速查索引。內容源自《AI 編程思維：從零基礎到全棧實戰的方法論指南》2026 年 8 月版。

新增第八篇「安卓開發實戰篇」（第 48-58 章，共 11 章）與第九篇「iOS 開發實戰篇」（第 59-69 章，共 11 章）：兩大平臺均覆蓋環境搭建（Android Studio / Xcode）、語言速成（Kotlin / Swift）、介面框架（Jetpack Compose / SwiftUI）、數據與網絡、兩個完整實戰項目（記賬本 + 天氣查詢）、測試與上架（Google Play / App Store）及速查手冊。同步更新目錄、學習路徑（六步八步）、章節地圖與速查索引。內容源自《從零開始的安卓開發》與《從零開始 iOS 開發》2026 年 8 月版。

內容增補：實用軟件與騷擾攔截

- 第十三章新增第六節「實用系統工具」：整理清濁等清理工具、系統自帶存儲清理、國家反詐中心、品牌手機管家、運營商防騷擾與 AdGuard，並給出「系統優先 / 官方補充 / 第三方克制」嘅選型原則。
- 手機自帶騷擾攔截品牌對照表：覆蓋 iOS、小米/Redmi、華為/榮耀、OPPO/realme/一加、vivo/iQOO、三星嘅入口路徑與推薦開啟項。
- 第八章騷擾攔截能力擴充：補充來電識別、黑白名單、短訊關鍵詞過濾、iOS 靜音未知來電、勿擾聯動等實操說明，並與第十三章互鏈。
- 三層防護建議：系統攔截 + 運營商防騷擾 + 國家反詐中心，強調白名單防誤殺。

內容增補：天氣預警 / 自然災害預警 / 地震預警

- 第十三章新增第七節：天氣預警（官方氣象渠道 + 系統天氣 App + 顏色等級）、自然災害預警（氣象 / 應急多渠道）、地震預警原理與誤區（預警 $\neq$ 預報）。

- 地震預警品牌開啟表：小米、華為/榮耀、OPPO 系、vivo、iPhone、三星等常見入口與權限注意事項。
- 第三十二章新增「場景零」：地震警報後嘅避險順序、暴雨 / 颱風應對、與衛星 SOS / 無網通訊嘅銜接；急救手冊與第十三章預警設置互鏈。
- 天氣小節聯動：旅行出行入面嘅天氣 App 條目補充預警通知與「打開通知先收到」提醒。

內容增補：跨設備協同與效率工具

- 第十三章第八節：手機遙控電腦（向日葵、ToDesk）及反詐紅線；AI 辦公（Trae / WorkBuddy / Dumate）與手機遙控電腦工作流；米哈遊人工桌面動態壁紙。
- 局域網快傳與剪貼板：LocalSend、飛鴿傳書、隔空投送；KDE Connect、通用剪貼板、品牌手機電腦互傳。
- 動態口令 2FA：Aegis、Microsoft Authenticator，備用碼與換機注意。
- 滴答清單與紙質掃描：日程 / 任務用法展開；掃描類工具與拍照技巧補全。

祝你玩轉手機，享受科技帶嚟嘅便利與樂趣，亦都成為一個清醒、自信嘅數碼公民。

——你嘅數碼生活嚮導

附錄 F：折疊螢幕選購與使用專題

折疊螢幕已經由「未來科技」變成「而家買得到」嘅成熟品類。但佢同直板機喺形態、使用習慣、保養方式上差異好大，值唔值得買、點樣揀、點樣用，呢一篇幫你理清楚。

一、折疊螢幕而家發展到乜嘢程度？

先講結論：**折疊螢幕已經唔係「嘗鮮玩具」，但都唔係「人人都要買」嘅主流形態**。佢介乎小眾極客產品同大眾消費品之間，適合特定人羣。

目前主流分兩類：

- **橫向折疊（書本式）**：合上係普通手機大小，展開係接近方形嘅小平板。代表：華為 Mate X 系列、三星 Galaxy Z Fold 系列、小米 MIX Fold 系列、vivo X Fold 系列。主打「一部手機 = 手機 + 平板」。
- **縱向折疊（翻蓋式）**：合上似粉餅盒，展開係正常尺寸手機。代表：華為 Pocket 系列、三星 Galaxy Z Flip 系列、OPPO Find N Flip、小米 MIX Flip。主打「小巧便攜 + 外屏速覽」。

2024-2026 年嘅關鍵進步：鉸鏈壽命普遍標稱 50 萬次折疊（每日開合 100 次可以用 13 年）、UTG 超薄玻璃取代早期塑料膜、摺痕肉眼可見但手感已經好輕微、價格由萬元檔下探到 4000-5000 元檔。

一句話判斷

如果你而家用直板機完全夠用、冇「螢幕唔夠大」嘅痛點，就唔好買折疊螢幕。佢係「錦上添花」嘅設備，唔係「雪中送炭」嘅設備。

二、乜嘢人適合買折疊螢幕？

根據真實用戶反饋，呢幾類人買折疊螢幕「真香」概率高：

1. **重度閱讀/文檔處理者**：經常喺手機上睇 PDF、批註文檔、讀長文。展開後接近 7.5 英寸嘅內屏，睇 A4 PDF 唔使縮放，體驗質變。

2. **經常出差/開會嘅商務用戶**：唔想帶平板又嫌手機屏細。橫向折疊螢幕展開睇 PPT、簽合同、雙屏對照郵件，比直板機效率高一截。
3. **鍾意「懸停態」創作嘅人**：折到 90 度放喺桌上，上半屏取景下半屏撳快門（自拍/延時攝影）、上半屏影片下半屏記筆記、上半屏直播下半屏睇評論——呢個係直板機做到唔到嘅形態。
4. **追求便攜又想要大屏嘅女性用戶**：縱向折疊螢幕合上可以塞入細包，展開係正常手機，外屏仲可以當化妝鏡、睇通知。

呢幾類人唔好買

- 手機淨係用嚟刷短影片、傾微信、打電話嘅——折疊螢幕對你毫無增益，反而更重更厚。
- 手細或者手腕有腱鞘炎嘅——橫向折疊螢幕展開後普遍 230g+，單手握持攰。
- 預算緊張、一部手機要用 4 年以上嘅——折疊螢幕二手保值率低於直板機，維修成本高。

三、選購要點：睇呢 6 個指標

唔好俾「折疊」兩個字晃花咗眼，選購嗰陣重點睇：

- **鉸鏈類型**：水滴鉸鏈（摺痕淺、壽命長，目前主流）優於 U 型鉸鏈（摺痕深、早期方案）。問銷售或查評測確認係邊種。
- **內屏材質**：必須係 UTG 超薄玻璃，唔好早期嗰種塑料膜。UTG 手感接近玻璃，塑料膜指甲劃過會有凹陷感而且易留痕。
- **摺痕表現**：去線下店親手展開睇。2026 年嘅合格水平係：亮屏嗰陣正面幾乎睇唔出，斜光下可以睇出一條淺溝，手指劃過有輕微凹陷但唔割手。
- **外屏尺寸**：橫向折疊螢幕嘅外屏越大越好（最好 6.4 英寸以上），合上可以當正常手機用；縱向折疊螢幕嘅外屏越大越實用（4 英寸以上可以回覆消息、睇導航）。
- **重量與厚度**：橫向折疊展開後 230-260g 係常態，低於 230g 算優秀；合上厚度 12-14mm，低於 12mm 算薄。縱向折疊合上後可以輕鬆放進恤衫口袋先算合格。
- **售後與保修**：重點睇內屏碎裂嘅維修價格。有啲品牌首年免費換一次內屏，有啲自費要 2000-4000 元。呢個係折疊螢幕最大嘅隱性成本。

線下體驗清單

1. 親手開合 10 次，感受鉸鏈阻尼係咪順滑、有冇異響。
2. 展開後斜對住燈光睇摺痕，再正面亮屏睇。
3. 單手握持展開狀態 30 秒，感受重量係咪可以接受。

4. 試用懸停態：折 90 度放桌上，試下影相同睇影片。

5. 問清楚內屏單獨維修價格同保修政策。

四、折疊螢幕獨有嘅使用技巧

買返嚟之後，呢幾個玩法係直板機體驗唔到嘅：

分屏與多任務

橫向折疊螢幕展開後，大部分系統支持上下/左右分屏。典型場景：上半屏睇網課影片，下半屏記筆記；左邊微信右邊文檔；上面地圖導航下面音樂。比直板機分屏實用得多，因為螢幕夠大，每半邊都唔擠。

懸停態（90 度折疊）

呢個係折疊螢幕嘅「殺手鐧」用法：

- **自拍/合影**：放桌上折 90 度，上半屏取景，下半屏撳快門，唔使搵人幫手影。
- **延時攝影/夜景**：同樣放桌上穩定拍攝，省三腳架。
- **視像通話**：解放雙手，對方睇你角度都更自然。
- **睇影片**：放桌上當細電視，飛機/高鐵細桌板上特別實用。
- **外屏預覽 + 內屏拍攝**：用後置攝像頭影自己嗰陣，外屏可以實時顯示畫面，構圖更準。

外屏速覽

縱向折疊螢幕合上後，外屏可以：睇時間/通知/天氣、回微信（語音轉文字或快捷回覆）、刷公交卡、睇導航、當化妝鏡（調用前置攝像頭）。唔使每次都展開，省電又省事。

電腦模式

部分橫向折疊螢幕（華為、三星）展開後可以切換「電腦模式」，外接顯示器鍵鼠後似台簡易 PC，臨時處理文檔、回郵件夠用。唔係主力生產力工具，但應急好香。

五、保養要點：等折疊螢幕活得耐啲

折疊螢幕最嬌貴嘅就係內屏同鉸鏈，呢幾條習慣可以顯著延長壽命：

- **唔好喺低温下強行開合**：0℃ 以下 UTG 玻璃變脆，開合前先揣喺衫度暖幾分鐘。呢個係官方說明書都會寫但冇人睇嘅重點。

- **避免指甲/硬物接觸內屏**：內屏雖然有保護層但比外屏軟，用指腹操作，唔好用指甲篤。
- **合上嗰陣確認屏內無異物**：一粒沙夾喺中間合上，可能直接壓壞內屏。合上前掃一眼。
- **唔好貼普通鋼化膜**：內屏淨係可以用官方專用膜或 UTG 保護膜，普通鋼化膜會影響折疊同觸控。建議去官方售後貼膜。
- **鉸鏈唔好入塵液體**：雖然多數標稱 IPX8 防水，但防塵等級普遍較低。沙灘、工地等環境要小心。
- **開合力度均勻**：唔好單手暴力甩開，雙手由中間勻速展開最穩。

內屏損壞嘅高發場景

- 合上嗰陣夾住耳機線/充電線——直接壓裂內屏。
- 放包入面同鑰匙混裝——外屏冇事，內屏俾鑰匙硌出裂痕。
- 冬天由凍嘅地方入暖房立刻開合——温差導致玻璃應力變化。
- 用咗一年之後覺得「摺痕變深咗」去撕原廠膜——撕膜會連內屏一齊損壞。

六、折疊螢幕嘅「坑」同心理預期

買之前心入面要有呢幾本帳：

- **摺痕會一直喺度**：2026 年嘅技術做到唔到「完全無痕」，淨係做到「亮屏正面幾乎睇唔出」。如果你係完美主義者，每日盯住摺痕難受，唔好買。
- **更重更厚**：橫向折疊螢幕展開後普遍 230g+，單手握持比直板機攰；合上後厚度 12-14mm，褲袋入面更鼓。
- **續航普遍一般**：因為要塞鉸鏈機構，電池空間俾壓縮，同價位續航通常唔及直板機。
- **保值率低**：一年後二手價通常係新機嘅 50-60%，比直板機跌得快。
- **維修貴**：內屏總成維修價 2000-4000 元，鉸鏈維修 500-1500 元。買之前查清楚保修政策。
- **App 適配參差**：主流 App 已適配大屏，但小眾 App 可能淨係「拉伸放大」，體驗一般。

折疊螢幕唔係「更好嘅手機」，而係「另一種形態嘅手機」。買佢係因為你需要佢獨特嘅形態，唔係因為佢「更先進」。

——關於折疊螢幕嘅選購哲學

七、主流品牌點樣揀（2026 年）

唔寫具體型號（會過時），淨係寫品牌調性，幫你快速定位：

- **華為 Mate X 系列**：橫向折疊標杆，鴻蒙生態協同強，外屏比例接近正常手機，合上可以當主力機用。價格高但保值率相對最好。
- **三星 Galaxy Z Fold/Flip**：折疊螢幕鼻祖，鉸鏈調校成熟，Z Flip 係縱向折疊嘅顏值擔當。國內售後網點較少係短板。
- **小米 MIX Fold/Flip**：性價比路線，配置堆料足，價格相對親民。適合想嘗鮮又唔想花太多嘅用戶。
- **vivo X Fold 系列**：橫向折疊入面外屏比例做得最接近正常手機嘅，合上當直板機用體驗好。
- **OPPO Find N 系列**：橫向折疊入面尺寸偏緊湊嘅，單手可以握持，適合覺得大折疊太重嘅用戶。
- **華為 Pocket 系列**：縱向折疊入面外屏最大最實用嘅，化妝鏡、防曬檢測等女性向功能做得細緻。

選購建議：預算充足追求生態揀華為 Mate X；想要最成熟鉸鏈揀三星 Z Fold；性價比首選小米/vivo；縱向折疊睇華為 Pocket 同三星 Z Flip。

最後提醒

折疊螢幕技術每年都喺度迭代，本附錄寫於 2026 年。買之前務必查最新評測同用戶反饋，重點關注：鉸鏈壽命實測、摺痕表現、內屏維修政策呢三項。線下親手摸永遠比睇評測重要。

結語：保持好奇，保持清醒

恭喜你讀到咗呢度。

呢本書由手機硬件拆到軟件生態，由日常設定挖到開發者模式，由安全防護傾到數字遺產。如果佢幫你解決咗一個實際問題——哪怕淨係學識關閉一個唔應該開嘅權限、養成咗一次備份嘅習慣——咁呢啲字就冇白寫。

但比任何具體技巧都重要嘅，係兩件事：

- **保持好奇**：手機每年都喺度變，AI 喺度變，折疊螢幕喺度變，衛星通信喺度變。今日學嘅操作步驟聽日可能就過時，但「想知道點解」嘅好奇心永遠唔會過時。多問一句「呢個權限係做乜嘢嘅」「呢個數據存咗喺邊度」「呢個功能會唔會洩露私隱」，比記住一百個操作步驟都有用。
- **保持清醒**：手機係工具，唔係主人。佢設計出嚟就係為咗吸引你嘅注意力、佔據你嘅時間、引導你嘅消費。用佢係你嘅選擇，俾佢用係你嘅失控。定期抬頭睇下螢幕時間，定期諗下「我係喺度用手機做嘢，定係喺度俾手機消磨」，呢個唔係矯情，呢個係數字時代嘅生存本能。

技術喺度變，但人嘅需求冇變——我哋需要安全感、需要連接、需要俾人尊重。手機淨係一個載體。真正嘅「手機課」唔喺書入面，喺你每日解鎖螢幕嗰一刻。

最後一個練習

合上呢本書，拿起你嘅手機，做一件事：**用 30 秒諗下，你嘅手機入面有邊三樣嘢係「唔見咗會真正心痛」嘅？**然後，確保佢哋俾人備份咗。呢本書就值啦。

附錄 G：全書速查索引

按拼音排序，方便你快速定位到對應章節。

A-B

- BL 鎖第二十四章
- ADB 第二十章
- App 安裝與管理第四章
- AI 助手第二十二章
- AI 手機第二十六章
- AI 換臉詐騙第三十二章
- AI 編程第三十七～四十七章
- AI 編程五大原則第三十八章
- API Key 第四十二章
- Agent 智能體第三十七章
- CLAUDE.md 第三十九章
- 安卓開發第四十八～五十八章
- Android Studio 第四十九章
- Activity 生命週期第五十二章
- App Store 上架第六十七章

C-D

- 車機互聯第三十四章
- 電池與續航第十五章

- 密碼管理第十四章
- Passkey 第十四章
- 開發者模式第二十章
- 兒童與手機第三十五章
- Claude Code / Cursor 第四十一章
- DeepSeek 第四十二章
- Compose 第五十一章
- DataStore 第五十三章

F-H

- Foldable 摺疊屏附錄 F
- 故障排查第十九章
- Google Play 上架第五十六章
- GaN 充電器第十五章
- iCloud 第十六章
- 智能家居第三十六章
- 計算攝影第十一章附

K-R

- 拍照技巧第十一章
- 權限管理第四章
- SafetyNet 認證第二十章
- SIM swap 攻擊第三十二章
- Root 權限第二十章 / 第二十四章
- Termux 第二十章附
- MCP 協議第四十六章
- PRD 需求文件第四十章
- Prompt 工程第四十三章
- Kotlin 第五十章

- MVVM 架構第五十七章・第六十八章
- Room 數據庫第五十三章
- Retrofit 第五十三章

S-Z

- 數碼遺產第二十五章
- 數據備份第十八章
- 騷擾攔截第八章・第十三章
- 天氣預警第十三章
- 自然災害預警第十三章・第三十二章
- 地震預警第十三章・第三十二章
- 實用軟件與清理工具第十三章
- 清濁第十三章
- UWB 超寬帶第十七章
- 網絡安全第二十一章
- 選購指南第二十八章
- 雲服務第十六章
- SDD 規範驅動開發第三十九章
- Token 第四十二章
- Vibe Coding 第三十七章
- Swift 第六十一章
- SwiftUI 第六十二章
- SwiftData 第六十四章
- URLSession 第六十四章
- Xcode 第六十章
- iOS 開發第五十九～六十九章

其他

- 檔案管理第二章
- 適老化第二十七章
- 工具類 App 推薦第十三章
- 防騷擾 / 國家反詐中心第十三章・第二十七章
- 遙控電腦（向日葵/ToDesk）第十三章
- AI 辦公（Trae 等）第十三章・第四十一章
- LocalSend / 局域網快傳第十三章
- KDE Connect / 剪貼板同步第十三章・第十七章
- 2FA 動態口令（Aegis 等）第十三章・第十四章
- 滴答清單第十三章
- 紙質文件掃描第十三章
- 人工桌面（動態壁紙）第十三章
- 地震 / 災害手機應急第三十二章
- 投屏與車載第三十章
- 短視頻創作第三十一章
- 安全急救第三十二章
- Git/GitHub 第二十三章附
- Git 版本控制第四十五章
- 上下文工程第三十九章
- 需求工程第四十章
- 記賬本 App 第五十四章・第六十五章
- 天氣 App 第五十五章・第六十六章
- WidgetKit 小工具第六十八章

附錄 H：版本路線圖

呢本書仲喺度持續更新。以下係未來版本嘅規劃方向，亦都歡迎你喺反饋入面話畀我哋知你最想睇嘅內容。

v1.8 已落地：新增第七、八、九篇「AI 編程實戰篇」「安卓開發實戰篇」「iOS 開發實戰篇」（第 37-69 章，共 33 章）

由 AI 編程嘅範式革命、五大思維原則、上下文與需求工程、2026 工具與模型全景、API 實戰、對話工程、三個完整項目、質量保障與進階之路——帶你由「識用 AI」升級到「指揮 AI 寫代碼」；再到安卓與 iOS 兩大平臺嘅完整開發鏈路：環境搭建（Android Studio / Xcode）、語言速成（Kotlin / Swift）、介面框架（Jetpack Compose / SwiftUI）、數據與網絡、實戰項目（記賬本 + 天氣查詢）、測試與上架（Google Play / App Store）及速查手冊——帶你由數碼用戶一路成長為 App 開發者。

下一版（v1.9）計劃方向

- 手機與可穿戴設備：Apple Watch、華為手錶、智能戒指（Oura/Galaxy Ring）嘅深度協同指南
- 旅行與手機：eSIM 實操、國際流量方案、離線地圖、海外支付、翻譯 App 對比
- 手機與健康管理：運動健康、睡眠監測、心率血氧、女性健康、心理健康 App
- AI 短視頻創作：剪映 AI、可靈 AI、數字人、AI 配音嘅實操教程
- 豎屏短劇入門：劇本結構、拍攝技巧、平臺政策

關於時效性嘅說明

手機技術迭代好快，本書內容基於 2026 年中嘅技術同市場狀況編寫。AI 能力、晶片型號、系統版本、價格區間都可能喺你閱讀嘅時候已經有變化。涉及具體操作步驟嘅，請以你設備嘅實際系統版本為準；涉及購買決策嘅，請查閱最新評測同用戶反饋。